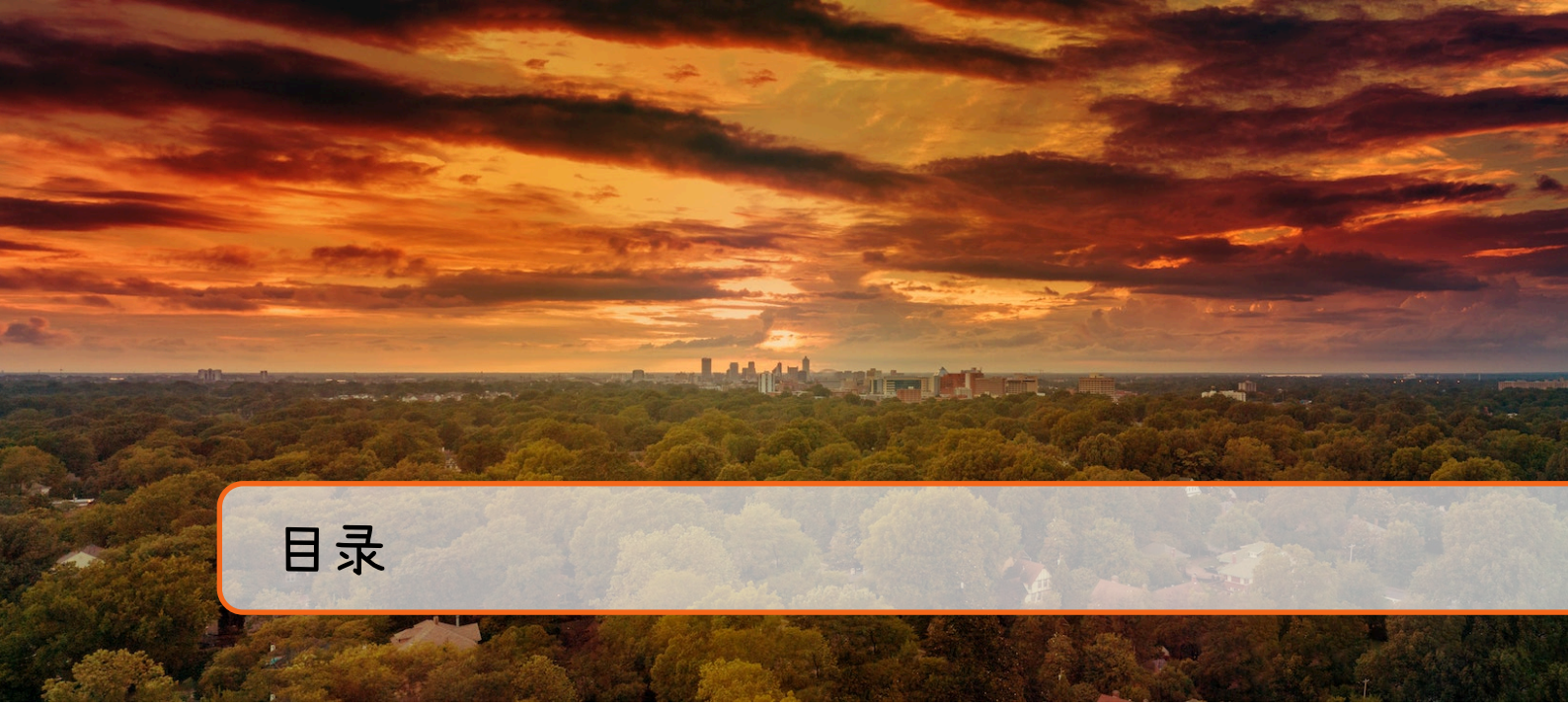
大语言模型、强化学习和多模态教程

理论与实践

左元



目录

I 大语言模型

1 LLM 简介 17
1.1 大语言模型要解决的问题是什么？ 17
1.2 如何训练一个能预测下一个 token 的模型？ 17
1.3 训练数据（输入-目标对）长什么样子？ 18
1.4 模型结构的设计 18
1.5 损失函数的设计 20

2 GPT-2 21
2.1 GPT-2 模型结构的定义以及数据集的准备 21
2.1.1 数据集的准备 21
2.1.2 模型结构 23
2.2 GPT-2 模型的训练 36
2.3 加载 OpenAI 开源的 GPT-2 预训练权重 41

II 强化学习

3 强化学习简介 47
3.1 基本概念 47
3.2 价值函数（Value Function） 50
3.3 倒立摆环境编程实践 51
3.4 马尔可夫决策过程 55
3.4.1 基本概念 55
3.4.2 状态转移 55
3.4.3 奖励函数 56
3.4.4 策略 56
3.4.5 MDP 的目标 57

3.4.6 收益 . . . 57
3.4.7 状态价值函数 . . . 57
3.5 贝尔曼方程 . . . 57
3.6 下一步 . . . 60

4 策略梯度法 . . . 61
4.1 原始策略梯度法 . . . 61
4.1.1 理论介绍 . . . 61
4.1.2 代码实现 . . . 66
4.1.3 深入讨论 . . . 72
4.2 REINFORCE . . . 73
4.2.1 REINFORCE 算法原理 . . . 73
4.2.2 REINFORCE 的代码实现 . . . 74
4.2.3 REINFORCE 证明 . . . 75
4.3 带基线（baseline）的策略梯度法 . . . 77
4.4 Actor-Critic（演员-评论家架构） . . . 78
4.4.1 Actor-Critic 的数学推导 . . . 79
4.4.2 Actic-Critic 的代码实现 . . . 81
4.5 多步 TD 误差 . . . 84
4.6 广义优势估计 . . . 85
4.7 相关数学证明 . . . 86
4.7.1 策略梯度法的证明 . . . 86
4.7.2 基线（Baseline）的推导 . . . 89
4.8 总结 . . . 90

5 近端策略优化（PPO） . . . 91
5.1 策略梯度法存在的问题 . . . 91
5.2 近端策略优化 . . . 93

6 PPO 实战 . . . 99
6.1 网络结构的定义 . . . 99
6.2 PPO 算法的实现 . . . 99
6.3 广义优势估计的计算 . . . 102
6.4 训练循环 . . . 102

7 PPO 背后的数学 . . . 105
7.1 数学背景知识 . . . 105
7.1.1 KL 散度 . . . 105
7.1.2 信息论 . . . 107
7.1.3 重要性采样（Importance Sampling） . . . 108
7.2 替代目标 . . . 112
7.3 补充知识 . . . 116
7.3.1 状态访问分布 . . . 116
7.3.2 全变差距离（Total Variation Distance） . . . 118

8 组相对策略优化（GRPO） 119
8.1 GRPO 原理 119
8.2 使用 GRPO 玩倒立摆游戏 120

III 基于人类反馈的强化学习

9 大语言模型训练概述 127
9.1 LLM 训练流程 127
9.1.1 预训练 127
9.1.2 监督微调 128
9.2 RLHF 的关键技术 129
9.2.1 近端策略优化（PPO） 129
9.2.2 直接偏好优化（DPO） 130
9.2.3 组相对策略优化（GRPO） 130

10 使用 DPO 微调大语言模型 133
10.1 DPO 理论介绍以及 DPO 存在的问题 133
10.1.1 偏好数据集 133
10.1.2 DPO 目标函数 134
10.2 DPO 实战 135
10.2.1 第一步：对预训练模型进行基于指令的监督微调（SFT） 135
10.2.2 第二步：使用 DPO 算法对 SFT 后的模型进行微调 148

11 使用 PPO 微调大语言模型--复刻 InstructGPT 159
11.1 InstructGPT 训练流程 159
11.2 即时奖励R_t的计算 160
11.3 PPO 微调 LLM 实战 163
11.3.1 监督微调（SFT） 163
11.3.2 训练奖励模型（Reward Model） 165
11.3.3 对 gpt2-sft 进行 PPO 微调 169

12 使用 GRPO 微调大语言模型——复刻 DeepSeek-R1 181
12.1 GRPO 简介 181
12.2 使用 DAPO（GRPO 的变种）微调 Qwen2.5-3B-Instruct 182
12.2.1 实现思路及任务要求 182
12.2.2 奖励函数的设计 184
12.2.3 代码实现 184
12.3 GRPO 应用场景 209
12.3.1 医疗思维链 209
12.3.2 Text-To-SQL 215

IV 多模态

13 Vision Transformer . . . 219
13.1 导入库和模块 . . . 220
13.2 补丁嵌入（Patch Embedding） . . . 220
13.3 类别对应的 token 和位置编码 . . . 222
13.4 注意力头 . . . 224
13.5 多头注意力 . . . 226
13.6 Transformer 编码器 . . . 227
13.7 Vision Transformer . . . 228
13.8 训练参数 . . . 230
13.9 加载 MNIST 数据集 . . . 230
13.10 训练循环 . . . 231
13.11 评估模型 . . . 231

14 CLIP . . . 233
14.1 导入库和模块 . . . 233
14.2 图像和文本编码器 . . . 234
14.2.1 位置嵌入 . . . 234
14.2.2 注意力头 . . . 234
14.2.3 多头注意力 . . . 235
14.2.4 Transformer 编码器 . . . 235
14.2.5 文本的分词器 . . . 236
14.2.6 文本编码器 . . . 237
14.2.7 图像编码器 . . . 239
14.3 CLIP 模型 . . . 240
14.3.1 数据 . . . 243
14.3.2 训练参数 . . . 245
14.3.3 加载数据集 . . . 246
14.3.4 训练模型 . . . 246
14.3.5 评估模型 . . . 247
14.3.6 零样本分类 . . . 248

15 ClipCap（图生文） . . . 251
15.1 ClipCap 的工作原理 . . . 251
15.2 关于训练 . . . 252
15.3 推理 . . . 252
15.4 ClipCap 的代码实现 . . . 253
15.4.1 项目配置 . . . 253
15.4.2 处理数据 . . . 253
15.4.3 模型结构 . . . 255
15.4.4 准备训练数据集 . . . 257
15.4.5 训练 . . . 258
15.4.6 推理 . . . 260
15.5 应用：图生文 . . . 263
15.6 拓展：Qwen3-VL . . . 264

16 扩散模型 265
16.1 通俗讲解 265
16.1.1 扩散 265
16.1.2 扩散模型的架构 266
16.1.3 模型设计 268
16.2 扩散模型理论简介 270
16.2.1 概述 270
16.2.2 正向扩散过程 271
16.2.3 反向扩散过程 274
16.2.4 U-Net 模型 274
16.3 代码实现 279
16.3.1 扩散过程相关代码 279
16.3.2 U-Net 神经网络 281
16.4 完整代码 289

17 扩散模型背后的数学理论 297
17.1 正向扩散过程 298
17.2 逆向扩散过程 300

18 条件扩散模型 311
18.1 向扩散模型添加条件 311
18.2 条件扩散模型的实现 313
18.3 得分函数 321
18.3.1 什么是得分函数 321
18.3.2 式 (18.6) 的证明 323
18.4 分类器指引 324
18.4.1 什么是分类器 324
18.4.2 分类器指引的推导 325
18.5 无分类器指引 327
18.5.1 无分类器指引的理论知识 327
18.5.2 无分类器指引的实现 328
18.6 Stable Diffusion 334

19 DALL-E2 （文生图） 337
19.1 CLIP 338
19.2 先验模型（Prior Model） 338
19.2.1 先验模型的训练 339
19.2.2 先验模型的推理 340
19.2.3 关键代码解释 341
19.3 条件扩散模型 352
19.3.1 概述 353
19.3.2 残差块 354
19.3.3 注意力块 357
19.3.4 下采样 360
19.3.5 上采样 360
19.3.6 模型的训练 361

19.3.7 损失的计算 370
19.4 训练配置 371
19.5 训练 373
19.6 训练结果 376

20 总结 379

Appendices 381

A 反向传播算法 381
A.1 损失函数 381
A.2 前向传播 381
A.3 反向传播 382
A.4 代码实现 382

B uv 教程 383

C KV Cache 385

List of Figures

1.1 预测下一个 token . . . 17
1.2 数据组织方法 . . . 18
1.3 GPT-2 架构 . . . 19
1.4 LLM 本质是个分类模型 . . . 20
2.1 分词 . . . 21
2.2 数据集 . . . 23
2.3 词嵌入 . . . 24
2.4 一条数据和一批数据的词嵌入示例 . . . 24
2.5 进入 `self.trf_blocks` 的数据 . . . 27
2.6 TransformerBlock 的架构图 . . . 29
2.7 `self.trf_blocks` 输出的数据 . . . 30
2.8 层归一化 . . . 30
2.9 公式注解 . . . 31
3.1 强化学习交互循环 . . . 47
3.2 倒立摆环境 . . . 48
3.3 策略是一个函数 . . . 49
3.4 策略函数 . . . 49
3.5 无数条轨迹 . . . 50
3.6 倒立摆环境运行 2 个时间步 . . . 51
3.7 推导第二项 . . . 59
4.1 输出的动作是概率 . . . 61
4.2 ResNet18 . . . 63
4.3 策略梯度法的目标函数 . . . 64
4.4 策略梯度法的梯度上升算法 . . . 64
4.5 无数条轨迹 . . . 65
4.6 只采样n条轨迹 . . . 65
4.7 只采样1条轨迹 . . . 66
4.8 使用一条轨迹计算策略梯度 . . . 66
4.9 策略梯度算法获得的奖励 . . . 71
4.10 梯度爆炸式更新 . . . 73
4.11 REINFORCE 策略梯度算法获得的奖励 . . . 75
4.12 无论怎么推，倒立摆都会在 3 步之后倒下 . . . 78
4.13 演员评论家架构 . . . 79

4.14 蒙特卡洛方法和 TD 方法对比 . . . 80
4.15 actor-critic 每一轮的回报走势图 . . . 84
5.1 梯度上升算法中步幅太大，策略直接崩溃了 . . . 92
5.2 左图是步幅太大，中间的图是理想的步幅，右边的图是方向不对 . . . 92
5.3 梯度爆炸更新 . . . 93
5.4 左图为线搜索，右图为置信域 . . . 94
5.5 置信域方法 . . . 94
5.6 PPO 目标函数中的比值 . . . 96
5.7 带裁剪的 PPO 算法伪代码 . . . 98
6.1 PPO 算法获得的奖励 . . . 103
7.1 概率分布π和b . . . 109
7.2 将每个数据作为二维平面上的点时的方差的示意图（圆心是平均值） . . . 110
7.3 基于概率分布b采样3的例子 . . . 111
7.4 使用下界函数优化目标函数 . . . 115
8.1 组内优势的计算，要注意的是动作的优势等于所在轨迹的组内优势 . . . 121
9.1 大语言模型训练流程 . . . 127
9.2 RLHF 常用算法 . . . 129
9.3 GRPO 目标函数 . . . 130
9.4 GRPO 替代损失函数 . . . 131
10.1 SFT 的损失函数 . . . 136
10.2 基于指令的监督微调只针对答案部分计算损失 . . . 137
10.3 将只针对答案计算损失推广到批次 . . . 137
10.4 我们是从 `model_inputs` 中寻找回答部分，然后计算预测的损失 . . . 147
11.1 InstructGPT 训练流程 . . . 160
11.2 输出一个 token（采取动作），针对输出的这个 token，怎么给即时奖励R_t？ . . . 161
11.3 奖励模型是针对一条完整的补全给出得分的 . . . 161
11.4 奖励模型训练流程示意图 . . . 165
11.5 提示词为“这本书”，补全为“真好看”的R_t的计算 . . . 172
11.6 去掉最后一个的价值 . . . 174
13.1 Vit 架构图 . . . 219
13.2 补丁的例子 . . . 220
13.3 将图像转换为补丁：第一步 . . . 221
13.4 将图像转换为补丁：第二步 . . . 222
13.5 将图像转换为补丁：第三步 . . . 222
13.6 位置编码的作用 . . . 223
13.7 左图：使用 `Conv2d` 运算分成 16 个 8x8 块的 32x32 MNIST 图像。右图：添加位置编码和类别 token 后的 16 个图像补丁，使用随机数据初始化。 . . . 224
13.8 注意力机制 . . . 225
14.1 clip 原理，来自原始论文 . . . 233
14.2 注意力分数的掩码 . . . 235
14.3 分词过程 . . . 237
14.4 余弦相似度的计算 . . . 242
14.5 复制掩码 . . . 244
14.6 因果注意力分数的掩码 . . . 245
15.1 图片标题模型的简单示例 . . . 251
15.2 将图片的 CLIP 嵌入映射到 GPT2 的嵌入空间 . . . 252
15.3 ClipCap 的推理过程 . . . 253
15.4 clipcap 的模型设计要点 . . . 255
15.5 clipcap 的训练目标 . . . 258
15.6 clipcap 只根据图片的嵌入预测文本的 token，也就是看图说话，不再需要文本了！ . . . 260

15.7 A800 训练 10 个小时的效果 . . . 263
15.8 Qwen2.5-VL 框架展示了视觉编码器与语言模型解码器的集成，用以处理包括图像和视频在内的多模态输入。视觉编码器被设计为处理原生分辨率的输入并支持动态帧率采样。不同尺寸的图像和具有不同时帧率的视频帧被动态映射为长度各异的 token 序列。值得注意的是，MRoPE 在时间维度上将时间 ID 与绝对时间对齐，使模型能够更好地理解时间动态，例如事件的节奏和精确的时刻定位。处理后的视觉数据随后被输入到 Qwen2.5 LM 解码器。我们对 ViT 架构进行了重构，加入了诸如带 SwiGLU 激活的前馈网络（FFN）、用于归一化的 RMSNorm 以及基于窗口的注意力机制等先进组件，以提升性能和效率。 . . . 264
16.1 水滴的扩散过程 . . . 265
16.2 高清图像的扩散过程 . . . 266
16.3 从高斯分布中采样一个随机值 . . . 267
16.4 预测图像与实际图像之间的差异使用损失函数计算 . . . 267
16.5 将噪声作为预测目标 . . . 268
16.6 U-net 架构（以最低分辨率为 32×32 像素为例）。每个蓝色方框对应一个多通道特征图。方框顶部标注了通道数量。方框左下角标注了 x-y 尺寸。白色方框表示复制的特征图。箭头表示不同的操作。 . . . 269
16.7 共享模型 . . . 270
16.8 正向扩散和反向扩散 . . . 270
16.9 正向扩散公式图解 . . . 271
16.10 以噪声为预测目标 . . . 274
16.11 训练 unet . . . 275
16.12 U-net 训练算法 . . . 275
16.13 扩散模型训练步骤 . . . 276
16.14 采样算法（去噪算法，反向扩散算法） . . . 277
16.15 扩散模型的采样过程 . . . 278
16.16 以噪声为预测目标 . . . 280
16.17 采样算法（去噪算法，反向扩散算法） . . . 281
16.18 U-Net 执行语义分割 . . . 281
16.19 本章要实现的 U-Net . . . 282
16.20 ConvBlock 类执行的处理 . . . 283
16.21 我们实现的 U-Net . . . 286
16.22 加入正弦位置编码的 U-Net . . . 287
16.23 新的 ConvBlock 类 . . . 288
17.1 郎之万动力学 . . . 297
17.2 正向扩散和反向扩散 . . . 298
17.3 正向扩散公式图解 . . . 298
17.4 去噪的解析解无法计算 . . . 301
17.5 通过贝叶斯定理可以对反向分布$q(x_{t-1}|x_t)$进行推断。右图上的红色曲线显示了用 3 个高斯分布混合表示的边缘分布$q(x_{t-1})$，而左图显示了以x_{t-1}为中心的高斯前向噪声过程$q(x_t|x_{t-1})$作为x_t的分布。通过将它们相乘并进行归一化，我们得到了蓝色曲线所示的对于特定选择的x_t的分布$q(x_{t-1}|x_t)$。因为左图的分布相对较宽，对应着较大的方差β_t，所以分布$q(x_{t-1}|x_t)$具有复杂的多峰结构。 . . . 301
17.6 左图中的高斯分布$q(x_t|x_{t-1})$的方差β_t要小的多。我们可以看到右图中相应的分布$q(x_t|x_{t-1})$（蓝色）接近高斯分布，具有与$q(x_t|x_{t-1})$类似的方差 . . . 302
17.7 高斯混合模型（GMM） . . . 303
18.1 本章将要实现的条件扩散模型 . . . 311
18.2 预测时刻$t-1$的图像（正态分布的均值向量）的模型（上）和预测添加到x_t中的噪声的模型（下） . . . 312
18.3 条件扩散模型的神经网络 . . . 313

18.4 在扩散模型中添加嵌入层 . . . 313
18.5 得分函数的示意图，图中显示了两种信息的融合：由高斯混合分布构成的二维分布，表示为热图；以及由 式 (18.5) 定义的相应得分函数，其作为向量绘制在x值的规则网络上 . . . 322
18.6 扩散模型中使用的推断噪声ε的神经网络 . . . 322
18.7 用于推断得分的神经网络 . . . 323
18.8 使用预训练神经网络进行分类的示例（参数为ϕ） . . . 325
18.9 通过反向传播可以求出$\nabla_{x_t} \log p(y|x_t)$ . . . 326
18.10 使用了推断得分的神经网络的分类器指引 . . . 326
18.11 上面式子的可视化 . . . 327
18.12 Stable Diffusion 生成的高清图像 . . . 335
19.1 DALL-E2 模型整体架构 . . . 338
19.2 先验模型架构图 . . . 339
19.3 先验模型的训练原理 . . . 339
19.4 先验模型的推理流程 . . . 340
19.5 线性调度方差和余弦调度方差的对比 . . . 341
19.6 解码器架构 . . . 353
19.7 残差块的架构 . . . 354
19.8 注意力块 . . . 357
19.9 解码器训练流程 . . . 361
19.10 训练结果 . . . 377

List of Tables

1.1 输入和预测目标 18
2.1 上下文长度为 256 的位置嵌入查找表 26
4.1 只有策略函数是我们可以控制的，环境和奖励我们无法控制 61
5.1 对 PPO 目标函数根据比值的不同进行分情况讨论 97
7.1 硬币的真实概率分布 106
7.2 第一个人对硬币的概率分布做出的估计 106
7.3 第二个人对硬币的概率分布做出的估计 106
7.4 第三个人对硬币的概率分布做出的估计 107
9.1 预训练和监督微调 128

I

大语言模型

1 LLM 简介 17
1.1 大语言模型要解决的问题是什么？ 17
1.2 如何训练一个能预测下一个 token 的模型？ . 17
1.3 训练数据（输入-目标对）长什么样子？ . 18
1.4 模型结构的设计 18
1.5 损失函数的设计 20

2 GPT-2 21
2.1 GPT-2 模型结构的定义以及数据集的准备 . 21
2.2 GPT-2 模型的训练 36
2.3 加载 OpenAI 开源的 GPT-2 预训练权重 . 41

1. LLM 简介

1.1 大语言模型要解决的问题是什么？

大语言模型要解决的问题是预测下一个 token（next token prediction，NTP）。

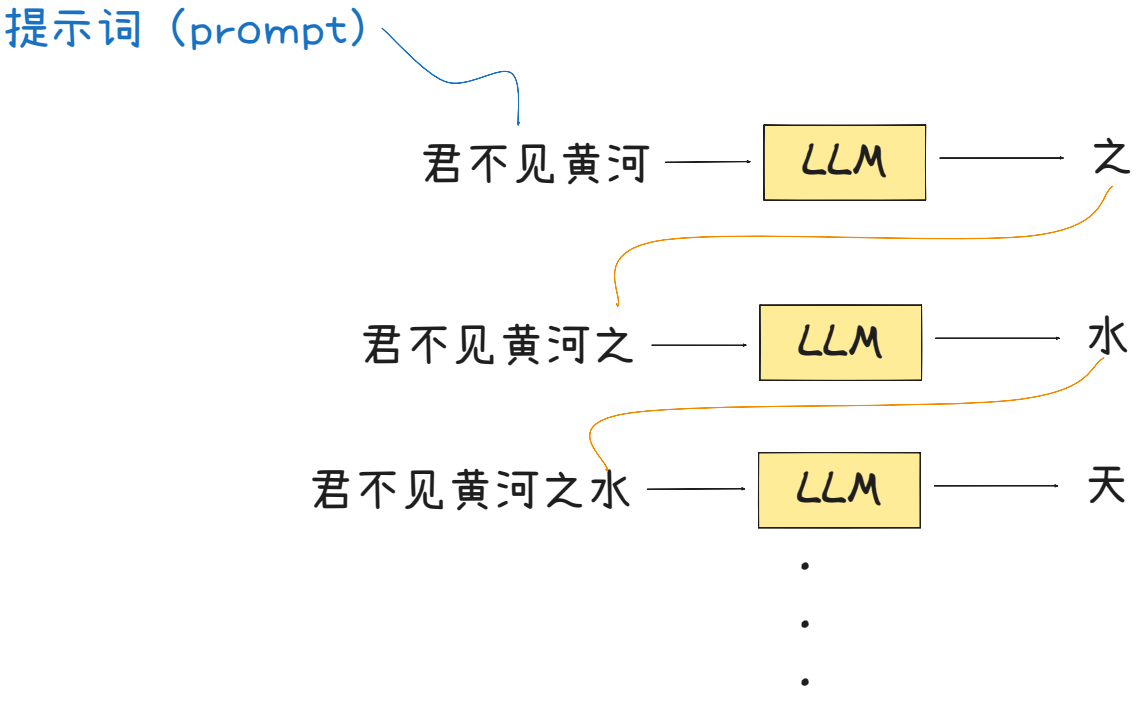


图 1.1 预测下一个 token

1.2 如何训练一个能预测下一个 token 的模型？

- 训练数据（输入-目标对）长什么样子？
- 模型结构如何设计？神经网络本质上是一个函数，这个函数长什么样子？参数有多少？
- 神经网络接收输入之后，输出和目标之间的损失（差异）怎么衡量？也就是说损失函数怎么设计？
- 如何让损失函数最小化？梯度下降法以及变种：SGD，Adam，AdamW，Muon 等优化器

- 梯度下降法需要求损失函数对于参数的导数（梯度），那么梯度如何得来？反向传播算法求参数的梯度：`loss.backward()`。

1.3 训练数据（输入-目标对）长什么样子？

文本数据："君不见黄河之水天上来"

输入	预测目标
君不见黄河	之
君不见黄河之	水
君不见黄河之水	天
君不见黄河之水天	上
君不见黄河之水天上	来

表 1.1 输入和预测目标

可以看到训练数据可以通过程序处理原始文本数据，自动生成，所以叫做"自监督学习"。
也就是说，预测目标不是人类标注而来，而是程序通过切分文本自动标注的。
为了充分利用 GPU 的并行计算能力，训练数据一般如下组织。

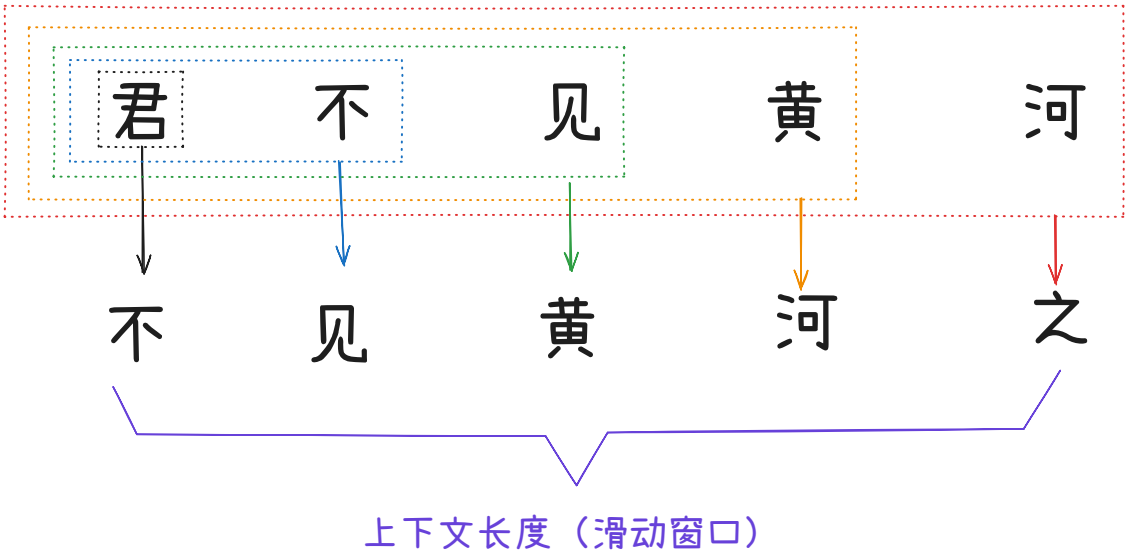


图 1.2 数据组织方法

1.4 模型结构的设计

`Decoder-Only Transformer`。仅解码器的 `Transformer` 架构。

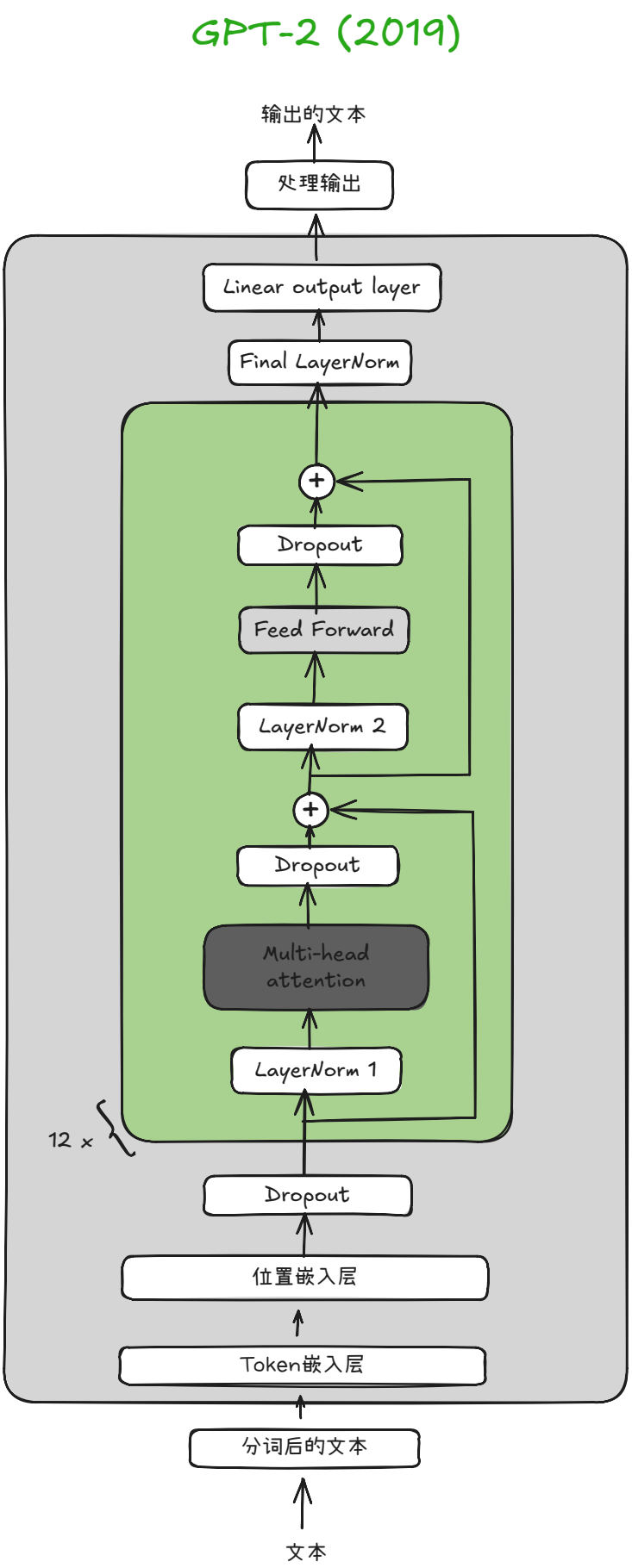

GPT-2 (2019)
输出的文本
处理输出
Linear output layer
Final LayerNorm
Dropout
Feed Forward
LayerNorm 2
Dropout
Multi-head attention
LayerNorm 1
12 x
Dropout
位置嵌入层
Token嵌入层
分词后的文本
文本

图 1.3 GPT-2 架构

1.5 损失函数的设计

LLM 本质上是一个分类模型。

从极大似然估计的角度看到 LLM。

假设训练数据为“abcd”，那么我们希望下面的概率越大越好。

$$P_\theta(\mathrm{d}|\mathrm{abc}) \cdot P_\theta(\mathrm{c}|\mathrm{ab}) \cdot P_\theta(\mathrm{b}|\mathrm{a}) \tag{1.1}$$

其中θ是神经网络的参数。最大化上面的式子等价于最大化下面的式子

$$\begin{aligned} &\log\{P_\theta(\mathrm{d}|\mathrm{abc}) \cdot P_\theta(\mathrm{c}|\mathrm{ab}) \cdot P_\theta(\mathrm{b}|\mathrm{a})\} \\ =\ &\log P_\theta(\mathrm{d}|\mathrm{abc}) + \log P_\theta(\mathrm{c}|\mathrm{ab}) + \log P_\theta(\mathrm{b}|\mathrm{a}) \end{aligned} \tag{1.2}$$

而最大化上面的式子，等价于最小化下面的式子

$$-\log P_\theta(\mathrm{d}|\mathrm{abc}) - \log P_\theta(\mathrm{c}|\mathrm{ab}) - \log P_\theta(\mathrm{b}|\mathrm{a}) \tag{1.3}$$

而上面的这个式子就是交叉熵损失函数！

也就是给定输入“ab”，我们希望预测的下一个 token 属于分类“c”的概率越大越好。

给定输入“abc”，我们希望预测的下一个 token 属于分类“d”的概率越大越好。

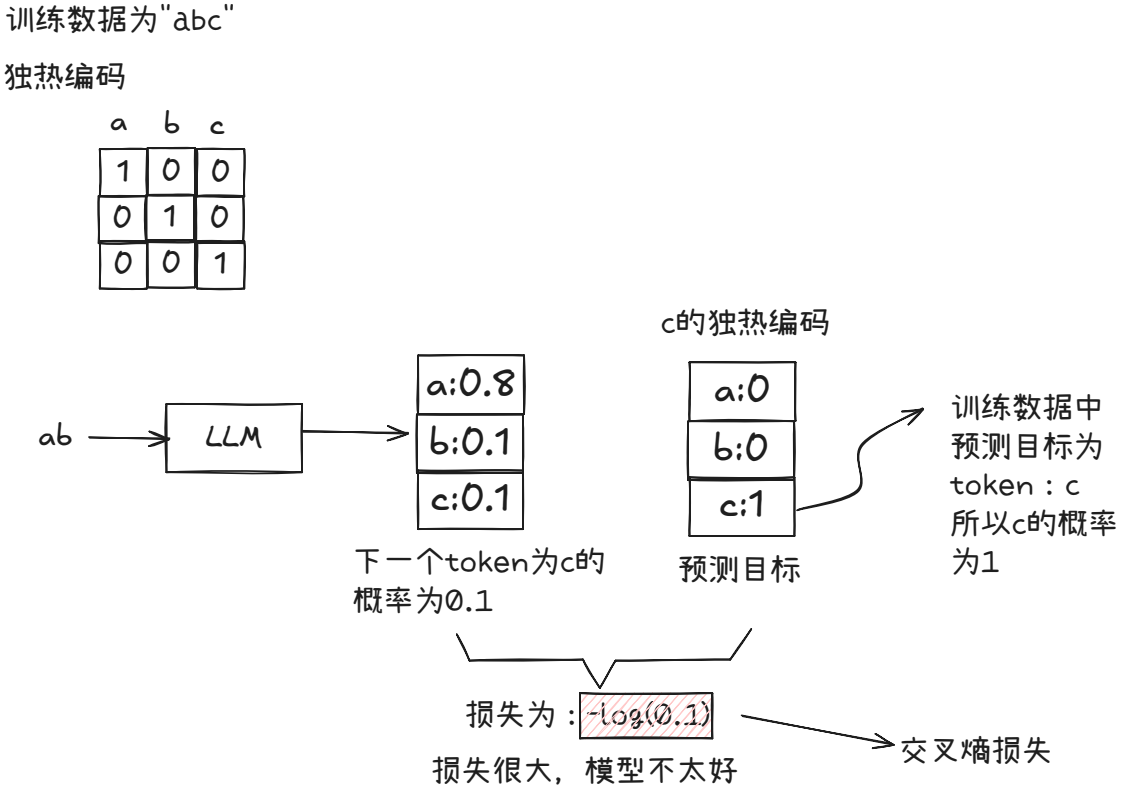


图 1.4 LLM 本质是个分类模型

2. GPT-2

2.1 GPT-2 模型结构的定义以及数据集的准备

创建一个文件 gpt_model.py。
我们首先导入需要的依赖。

```python
import tiktoken # 分词器库
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
```

2.1.1 数据集的准备

图 2.1 分词

接下来我们构建数据集。

```python
class GPTDatasetV1(Dataset):
    def __init__(self, txt, tokenizer, max_length, stride):
        self.input_ids = []
        self.target_ids = []

        # 对整个文本进行分词
        token_ids = tokenizer.encode(txt, allowed_special={"◁endoftext▷"})

        # 使用滑动窗口将文本切割成**重叠的**长度为`max_length`的序列
        for i in range(0, len(token_ids) - max_length, stride):
            input_chunk = token_ids[i : i + max_length]
            target_chunk = token_ids[i + 1 : i + max_length + 1]
            self.input_ids.append(torch.tensor(input_chunk))
            self.target_ids.append(torch.tensor(target_chunk))

    def __len__(self):
        return len(self.input_ids)

    def __getitem__(self, idx):
        return self.input_ids[idx], self.target_ids[idx]
```

然后我们封装一个工具函数，用来创建训练数据集。

```python
def create_dataloader_v1(
  txt,
  batch_size=4,
  max_length=256,
  stride=128,
  shuffle=True,
  drop_last=True,
  num_workers=0,
):
    # 初始化分词器
    tokenizer = tiktoken.get_encoding("gpt2")
    # 创建数据集
    dataset = GPTDatasetV1(txt, tokenizer, max_length, stride)
    # 创建dataloader
    dataloader = DataLoader(
      dataset,
      batch_size=batch_size,
      shuffle=shuffle,
      drop_last=drop_last,
      num_workers=num_workers,
    )

    return dataloader
```

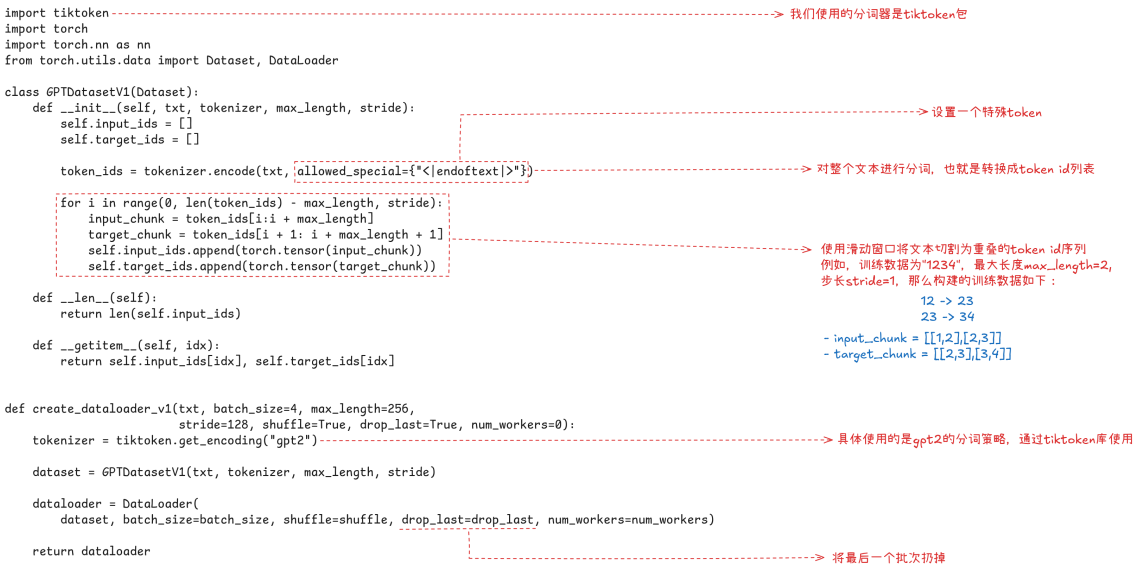


图 2.2 数据集

2.1.2 模型结构

接下来我们自顶向下的来定义模型结构。

```python
class GPTModel(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.tok_emb = nn.Embedding(cfg["vocab_size"], cfg["emb_dim"])
        self.pos_emb = nn.Embedding(cfg["context_length"], cfg["emb_dim"])
        self.drop_emb = nn.Dropout(cfg["drop_rate"])

        self.trf_blocks = nn.Sequential(
          *[TransformerBlock(cfg) for _ in range(cfg["n_layers"])])

        self.final_norm = LayerNorm(cfg["emb_dim"])
        self.out_head = nn.Linear(cfg["emb_dim"], cfg["vocab_size"], bias=False)

    def forward(self, in_idx):
        batch_size, seq_len = in_idx.shape
        tok_embeds = self.tok_emb(in_idx)
        pos_embeds = self.pos_emb(torch.arange(seq_len, device=in_idx.device))
        x = tok_embeds + pos_embeds # 形状：[batch_size, num_tokens, emb_size]
        x = self.drop_emb(x)
        x = self.trf_blocks(x)
        x = self.final_norm(x)
        logits = self.out_head(x)
        return logits
```

神经网络本身是一个函数，而神经网络这个函数的定义关键要看.forward方法，也就是.forward方法定义了神经网络的计算图长什么样子。

2.1.2.1 词嵌入

首先 GPT-2 模型的输入 in_idx 是一批 token id 列表，形状为(batch_size, seq_len)。

所以 batch_size, seq_len = in_idx.shape 这一行可以提取批次的大小 batch_size 和每条数据序列的长度 seq_len。

而 tok_embeds = self.tok_emb(in_idx)将输入的 token id 列表转换成了形状为(batch_size, seq_len, emb_dim)的张量。

nn.Embedding 的作用是将 token id 转换成一个向量，如 GPT-2 会将一个 token id 转换成一个 768 个浮点数元素的向量。

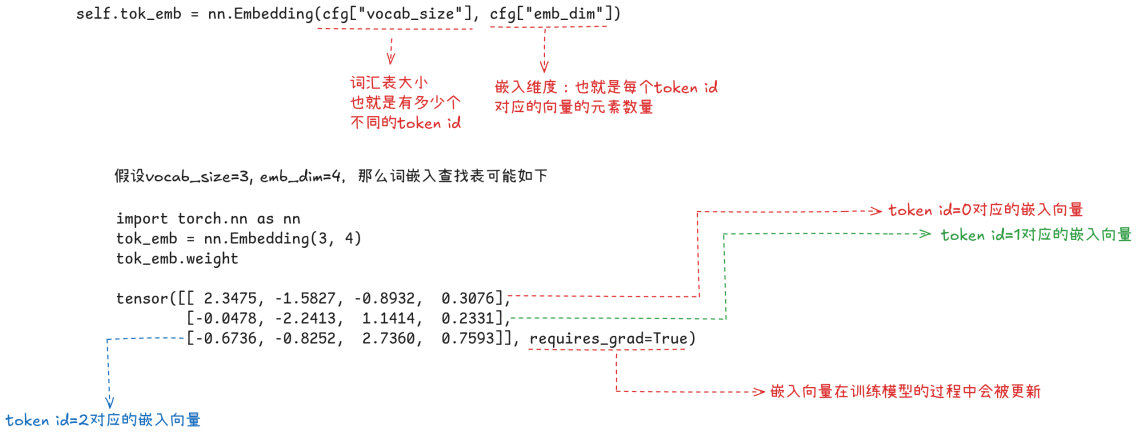


图 2.3 词嵌入

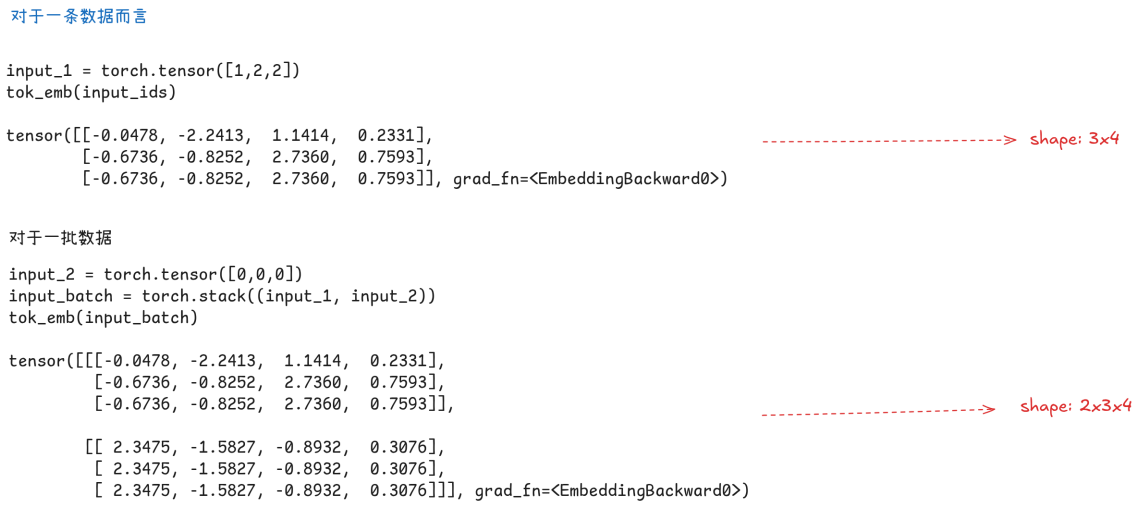


图 2.4 一条数据和一批数据的词嵌入示例

在 PyTorch 中，嵌入层实现的功能与执行矩阵乘法的线性层相同；我们使用嵌入层主要是出于计算效率的考虑。

```python
# 假设我们有以下3个训练样本，
# 它们可能代表LLM上下文中的token ID
idx = torch.tensor([2, 3, 1])

# 嵌入矩阵的行数可以通过
# 获取最大的token ID + 1来确定。
```

```
# 如果最大的token ID是3，那么我们需要4行，用于可能的
# token ID: 0, 1, 2, 3
num_idx = max(idx)+1

# 期望的嵌入维度是一个超参数
out_dim = 5
```

实现一个简单的嵌入层

```python
torch.manual_seed(123)
embedding = torch.nn.Embedding(num_idx, out_dim)
print(embedding.weight) # 查看嵌入权重
# 获取训练示例ID为1的向量表示
print(embedding(torch.tensor([1])))
# 获取训练示例ID为2的向量表示
print(embedding(torch.tensor([2])))
# 查找一批ID的向量表示
idx = torch.tensor([2, 3, 1])
print(embedding(idx))
```

现在我们用独热编码和 `nn.Linear` 实现和上面的嵌入层一样的功能。

首先将 `token ID` 转换为独热编码

```python
onehot = torch.nn.functional.one_hot(idx)
print(onehot)
```

接下来，我们初始化一个线性变换层，它会执行一个矩阵乘法：XW^T

```python
torch.manual_seed(123)
linear = torch.nn.Linear(num_idx, out_dim, bias=False)
print(linear.weight)
```

请注意，`PyTorch` 中的线性层同样是用小的随机权重初始化的；为了与上面的 `Embedding` 层直接比较，我们必须使用相同的小随机权重，这就是我们在此处重新赋值它们的原因：

```python
linear.weight = torch.nn.Parameter(embedding.weight.T)
```

现在我们可以将线性层应用于输入数据的独热编码表示

```python
print(linear(onehot.float()))
print(embedding(idx))
```

正如我们所见，这与使用嵌入层时得到的结果完全相同。

底层执行的是对第一个训练示例的 `token ID` 进行的如下计算：

由于每行独热编码中除一个索引外全为 `0`（这是设计的必然结果），该矩阵乘法实质上等同于对独热元素的查表操作。

这种在独热编码上使用矩阵乘法的做法等价于嵌入层查找操作，但当处理大型嵌入矩阵时会效率低下，因为存在大量与零相乘的无效计算。

2.1.2.2 位置编码

GPT-2 使用的是可学习位置编码。

```python
self.pos_emb = nn.Embedding(cfg["context_length"], cfg["emb_dim"])
```

为 `token id` 的位置构建一个嵌入查找表，例如如果我们的 `context_length` 为 256 的话。那么查找表如下：

token id 的索引	位置嵌入向量
0	768 个元素的向量
1	768 个元素的向量
2	768 个元素的向量
…	…
255	768 个元素的向量

表 2.1 上下文长度为 256 的位置嵌入查找表

所以

```python
pos_embeds = self.pos_emb(torch.arange(seq_len, device=in_idx.device))
```

为`[0, 1, ..., seq_len - 1]`中的每个位置分配了一个 768 元素的向量，作为对应的位置编码。也就是将序列的每个位置编号组成的序列变换成形状`(batch_size, seq_len, emb_dim)`。

然后我们就会将 token 的词嵌入向量和 token 所在位置的位置嵌入向量直接相加。

```python
x = tok_embeds + pos_embeds
```

初看起来，在 token 向量上面添加位置信息会破坏输入向量，使得网络的任务变得更加难以完成。不过，从两个随机选择的不相关向量在高维空间中几乎是正交的这一现象中，我们可以了解到这一方法为什么能够很好的发挥作用，这表明网络能够相对独立的处理 token 的身份信息和位置信息。另外，请注意，由于 Transformer 中的层与层之间都有残差连接，因此位置信息从一层转到下一层的过程中不会丢失。

这里我们发现了，每个 token id 对应的 768 元素向量，在训练结束时，会固定下来。例如：假设“hello”这个 token 的 id 是 1，那么 1 对应的 768 个元素向量就固定了。

而序列中的为个位置（索引）对应的 768 个元素向量，在训练结束时也会固定下来。

设想有两条文本：

- “我是左元”
- “我是莎士比亚”

两句话中的 token“我”的 token 嵌入和位置嵌入一模一样

- 相同 token id 对应的 token 嵌入向量一定相同
- 相同位置（两个“我”都为索引 0）对应的位置嵌入向量也一定相同

所以 x = tok_embeds + pos_embeds 是一样的！但很明显，两个“我”指代的人名是不一样的，所以这两个语义在 GPT-2 中如何进行区分呢？那就是注意力机制，或者说 Transformer 模块了！

2.1.2.3 仅解码器的 Transformer

在模型的初始化时，我们堆叠了很多层的 TransformerBlock。

```python
self.trf_blocks = nn.Sequential(*[TransformerBlock(cfg) for _ in range(cfg["n_layers"])])
```

然后在前向传播的过程中，

```python
x = self.trf_blocks(x)
```

堆叠很多层 TransformerBlock，然后顺序执行。这里也可以看出 TransformerBlock 的输入和输出的形状必须一致！否则无法堆叠！

送进 transformer 模块的是 token 嵌入+位置嵌入，形状为：`(batch_size, seq_len, emb_dim)`。

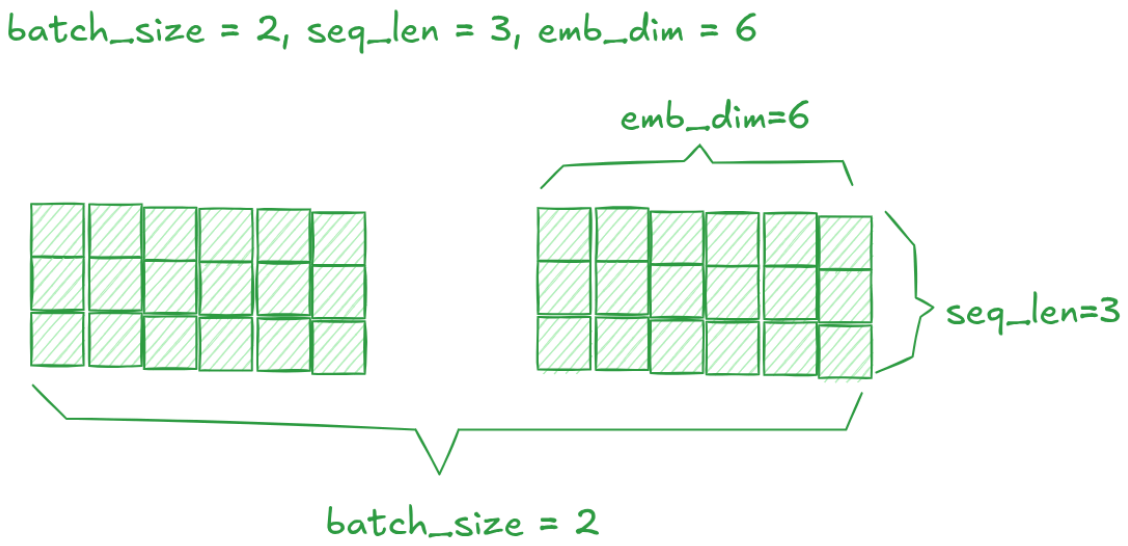


图 2.5 进入 self.trf_blocks 的数据

接下来我们编写 TransformerBlock 的代码

```python
class TransformerBlock(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.att = MultiHeadAttention(
            d_in=cfg["emb_dim"],
            d_out=cfg["emb_dim"],
            context_length=cfg["context_length"],
            num_heads=cfg["n_heads"],
            dropout=cfg["drop_rate"],
            qkv_bias=cfg["qkv_bias"],
        )
        self.ff = FeedForward(cfg)
        self.norm1 = LayerNorm(cfg["emb_dim"])
        self.norm2 = LayerNorm(cfg["emb_dim"])
        self.drop_shortcut = nn.Dropout(cfg["drop_rate"])

    def forward(self, x):
        # 连接到注意力模块输出的残差连接
        shortcut = x
        x = self.norm1(x)
        x = self.att(x) # 形状：[batch_size, num_tokens, emb_size]
        x = self.drop_shortcut(x)
        x = x + shortcut # 将原始的输入加过来

        # 连接到feed-forward block输出的残差连接
        shortcut = x
        x = self.norm2(x)
        x = self.ff(x)
        x = self.drop_shortcut(x)
```

```
30          x = x + shortcut # 将原始的输入加过来
31
32          return x
```

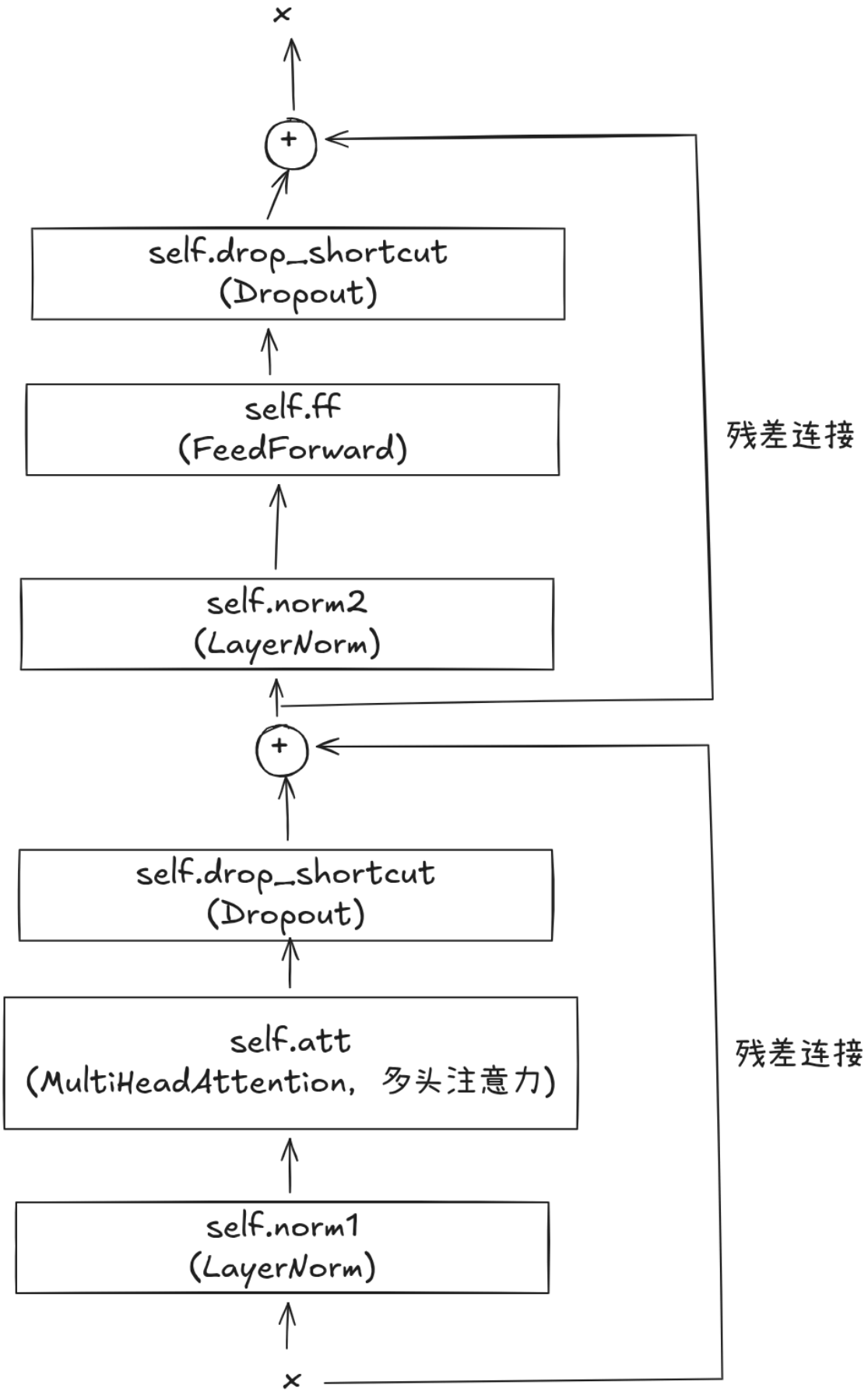


图 2.6 TransformerBlock 的架构图

`self.trf_blocks(x)`的输入的形状也是下面的形状

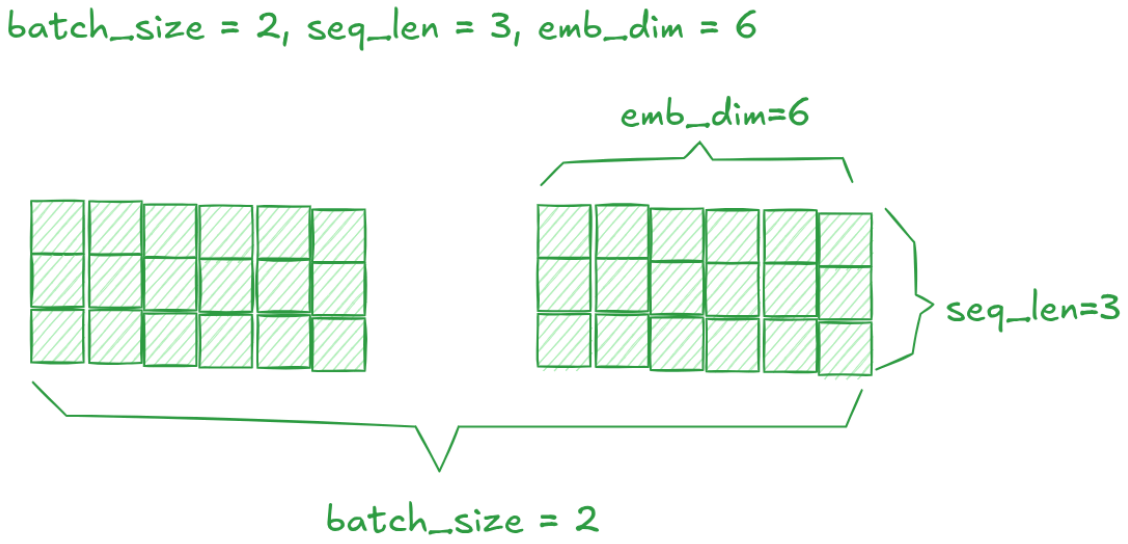
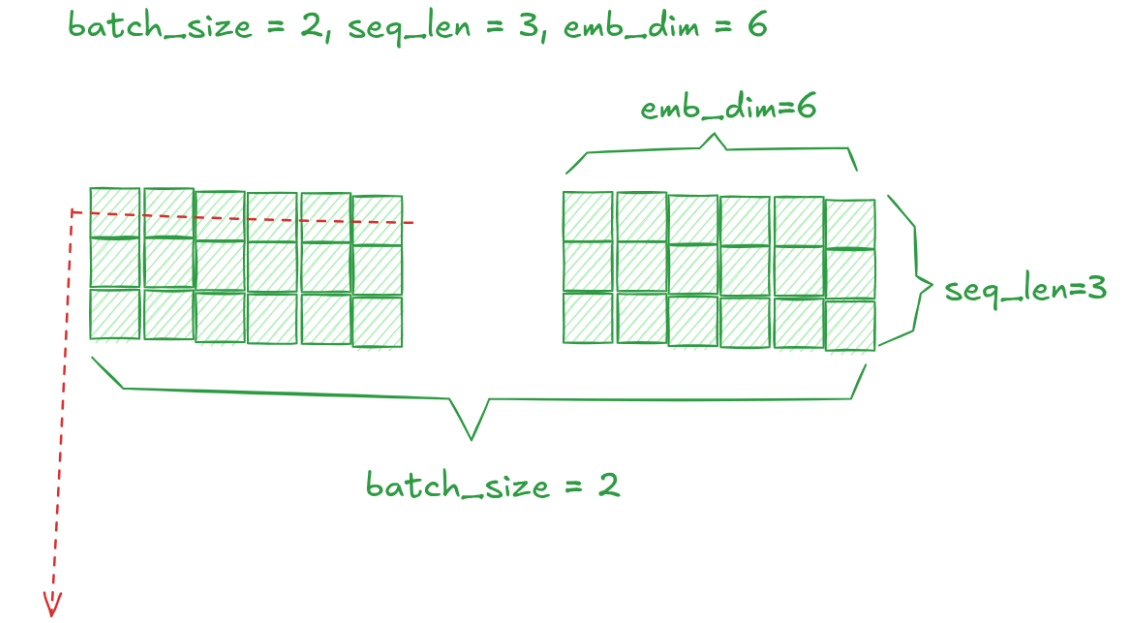


图 2.7 `self.trf_blocks` 输出的数据

2.1.2.4 层归一化（LayerNorm）

$$y = \frac{x - \mathrm{E}[x]}{\sqrt{\mathrm{Var}[x] + \varepsilon}} * \gamma + \beta \tag{2.1}$$

层归一化模块(LayerNorm)：在emb_dim这个维度进行归一化



针对这一行也就是嵌入向量进行归一化，这样不受批次中数据多少的影响
有助于训练稳定性

图 2.8 层归一化

```python
class LayerNorm(nn.Module):
    def __init__(self, emb_dim):
        super().__init__()
        self.eps = 1e-5 # ε
        self.scale = nn.Parameter(torch.ones(emb_dim)) # γ
        self.shift = nn.Parameter(torch.zeros(emb_dim)) # β

    def forward(self, x):
        mean = x.mean(dim=-1, keepdim=True) # E[x]
        var = x.var(dim=-1, keepdim=True, unbiased=False) # Var[x]
        # (x-E[x]) / sqrt(Var[x]+ε)
        norm_x = (x - mean) / torch.sqrt(var + self.eps)
        # y = (x-E[x]) / sqrt(Var[x]+ε) * γ + β
        return self.scale * norm_x + self.shift
```

层归一化模块(LayerNorm)：在 emb_dim 这个维度进行归一化。

这里要注意的是：LayerNorm 的输入和输出形状相同！也就是说，输入 x 的形状(batch_size, seq_len, emb_dim)，输出的形状也是(batch_size, seq_len, emb_dim)。只是在 emb_dim 这一个维度进行了归一化。

图 2.9 公式注解

LayerNorm 中有两个参数γ和β会在训练时更新。

2.1.2.5 多头注意力

```python
class MultiHeadAttention(nn.Module):
    def __init__(self, d_in, d_out, context_length, dropout, num_heads,
    qkv_bias=False):
        super().__init__()
        assert d_out % num_heads == 0, "d_out 必须能被 n_heads 整除"

        self.d_out = d_out
        self.num_heads = num_heads
        self.head_dim = d_out // num_heads #

        self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_key = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.out_proj = nn.Linear(d_out, d_out)
        self.dropout = nn.Dropout(dropout)
```

```
        self.register_buffer("mask", torch.triu(torch.ones(context_length, context_length), diagonal=1))

    def forward(self, x):
        b, num_tokens, d_in = x.shape
        # K = X · W_K^T
        keys = self.W_key(x) # 形状：(b, num_tokens, d_out)
        # Q = X · W_Q^T
        queries = self.W_query(x)
        # V = X · W_V^T
        values = self.W_value(x)

        # 通过添加一个`num_heads`维度，我们隐式的将矩阵进行了split
        # 将最后一个维度展开：(b, num_tokens, d_out) → (b, num_tokens, num_heads, head_dim)
        keys = keys.view(b, num_tokens, self.num_heads, self.head_dim)
        values = values.view(b, num_tokens, self.num_heads, self.head_dim)
        queries = queries.view(b, num_tokens, self.num_heads, self.head_dim)

        # 转置：(b, num_tokens, num_heads, head_dim) → (b, num_heads, num_tokens, head_dim)
        keys = keys.transpose(1, 2)
        queries = queries.transpose(1, 2)
        values = values.transpose(1, 2)

        # 计算缩放点积注意力（scaled dot-product attention）也就是自注意力，使用因果注意力掩码（causal mask）
        # attention_scores = QK^T
        attn_scores = queries @ keys.transpose(2, 3) # 每个头计算点积

        # 原始的掩码方阵需要切割token序列的长度，然后转换成布尔类型
        mask_bool = self.mask.bool()[:num_tokens, :num_tokens]

        # 使用掩码填充注意力分数
        attn_scores.masked_fill_(mask_bool, -torch.inf)

        attn_weights = torch.softmax(attn_scores / keys.shape[-1]**0.5, dim=-1)
        attn_weights = self.dropout(attn_weights)

        # 形状：(b, num_tokens, num_heads, head_dim)
        context_vec = (attn_weights @ values).transpose(1, 2)

        # 将多个头组合起来，这里 self.d_out = self.num_heads * self.head_dim
        context_vec = context_vec.reshape(b, num_tokens, self.d_out)
        context_vec = self.out_proj(context_vec) # 可选的线性变换（投影）

        return context_vec
```

2.1.2.6 前馈层

TransformerBlock 里的 Feed Forward Network，简称 FFN/MLP，可以理解为：在注意力机制完成“不同 token 之间的信息交换”之后，对每个 token 各自做一次非线性特征加工和升维变换。Self-Attention 负责“看别人、融合上下文”，FFN 负责“想一想、重新组织这个 token 的内部表示”。两者配合起来，Transformer 才既能建模上下文关系，又有足够强的表达能力。

也就是说：

- Attention 子层：让 token 之间互相通信
- FFN 子层：对每个 token 的表示单独进行非线性变换

Transformer 中常见的 FFN 是一个两层 MLP：

$$\mathrm{FFN}(x) = W_2\sigma(W_1x + b_1) + b_2 \tag{2.2}$$

其中：

- x是某个 token 的隐藏状态（hidden state）。
- W_1把维度从d_{model}扩展到d_{ff}
- σ是激活函数，例如 ReLU、GELU、SwiGLU 等。
- W_2再把维度从d_{ff}压回d_{model}。

中间维度通常是模型维度的 4 倍左右。

Self-Attention 本质上主要是在不同 token 的表示之间做加权组合。

如果没有 FFN，模型会更像是在反复做“上下文混合”，但缺少足够强的非线性加工能力。

FFN 中的激活函数，例如 GELU，会引入非线性，这使模型能够表达更复杂的函数关系，而不是简单地线性混合信息。

可以类比一下：

- Attention：把相关信息从上下文中找出来
- FFN：对找到的信息进行加工、判断、抽象

就像开会时，Attention 是“听取大家意见”，FFN 是“自己消化一下”。不然只听不想，会议纪要会很热闹，但决策不一定靠谱。

FFN 对序列中的每个位置使用同一套参数，但每个 token 独立处理。

它不会直接让 token 之间交流。

token 之间的交流主要发生在 Self-Attention 中。

FFN 通常会先升维，再降维。这个结构很重要。升维到更大的空间后，模型可以形成更多中间特征；再压回原维度，相当于筛选和组合这些特征。

可以理解为：

- 把 token 表示投影到更丰富的特征空间
- 通过激活函数筛选重要模式
- 再压缩回原来的表示维度

这有点像把一张小照片放大后仔细修图，再缩回合适尺寸。不是为了变大，而是为了在更大的空间里做更细的处理。

在大语言模型里，FFN/MLP 层往往被认为承担了大量“知识存储”和“模式记忆”的功能。

有一些研究观察到：

- Attention 更偏向于路由信息：决定从哪里取信息
- FFN 更偏向于内容变换：决定把信息变成什么
- 某些 MLP 神经元会对特定概念、语言模式、事实关联有响应

当然，这不是说“知识只存在 FFN 中”，但 FFN 确实是参数量很大的部分，往往承载了大量可学习模式。

Attention 把信息聚合过来之后，需要有一个模块对聚合后的信息做“解释”。

例如句子：

```
1 The bank is near the river.
```

Attention 可以让 bank 关注 river，但 FFN 可以进一步把 bank 的表示推向“河岸”这个语义区域。

另一个句子：

```
1 I deposited money in the bank.
```

Attention 可以让 bank 关注 money，FFN 再把它加工成“银行机构”的语义表示。

当然这是简化描述，但直觉上很有用。

FFN 的参数量很大。

一个标准 FFN 参数量大约是：

$$d_{\text{model}} \times d_{\text{ff}} + f_{\text{ff}} \times d_{\text{model}} \tag{2.3}$$

如果

$$d_{\text{ff}} = 4d_{\text{model}} \tag{2.4}$$

那么 FFN 参数量约为：

$$8d_{\text{model}}^2 \tag{2.5}$$

而 Attention 中 Q、K、V、O 四个矩阵大约是：

$$4d_{\text{model}}^2 \tag{2.6}$$

所以 FFN 往往比 Attention 参数还多。

这也是为什么它对模型能力非常关键。

Transformer block 中的 Feed Forward Network 的作用是：

对每个 token 的上下文表示进行独立的非线性变换，提升模型表达能力、重组特征，并承担大量模式/知识存储功能。

Python

```python
class GELU(nn.Module):
    def __init__(self):
        super().__init__()

    def forward(self, x):
        return 0.5 * x * (1 + torch.tanh(
            torch.sqrt(torch.tensor(2.0 / torch.pi)) *
            (x + 0.044715 * torch.pow(x, 3))
        ))


class FeedForward(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(cfg["emb_dim"], 4 * cfg["emb_dim"]),
            GELU(),
            nn.Linear(4 * cfg["emb_dim"], cfg["emb_dim"]),
        )

    def forward(self, x):
        return self.layers(x)
```

2.1.2.7 简单的推理代码

```python
def generate_text_simple(model, idx, max_new_tokens, context_size):
    # idx的形状为(B, T)
    for _ in range(max_new_tokens):
        # 将当前上下文裁剪到我们支持的上下文长度
        # 例如LLM如果只支持5个token，而上下文长度为10，
        # 那么只保留最后5个token作为上下文
        idx_cond = idx[:, -context_size:]

        # 预测
        with torch.no_grad():
            logits = model(idx_cond)

        # 只取最后一个
        # (batch, n_token, vocab_size) → (batch, vocab_size)
        logits = logits[:, -1, :]

        # 获取logits分数最高的值的索引
        idx_next = torch.argmax(logits, dim=-1, keepdim=True) # (batch, 1)

        # 将下一个token在词汇表中的索引追加到序列后面
        idx = torch.cat((idx, idx_next), dim=1) # (batch, n_tokens + 1)

    return idx
```

main 函数如下：

```python
if __name__ == "__main__":

    GPT_CONFIG_124M = {
        "vocab_size": 50257,     # 词汇表大小
        "context_length": 1024,  # 上下文长度
        "emb_dim": 768,          # 词嵌入维度
        "n_heads": 12,           # 注意力头的数量
        "n_layers": 12,          # transformer block的层数
        "drop_rate": 0.1,        # Dropout rate
        "qkv_bias": False        # Query-Key-Value bias
    }

    torch.manual_seed(123)
    model = GPTModel(GPT_CONFIG_124M)
    model.eval()  # 禁用dropout

    start_context = "Hello, I am"

    tokenizer = tiktoken.get_encoding("gpt2")
    encoded = tokenizer.encode(start_context)
    encoded_tensor = torch.tensor(encoded).unsqueeze(0)

    print(f"\n{50*'='}\n{22*' '}IN\n{50*'='}")
```

```
    print("\nInput text:", start_context)
    print("Encoded input text:", encoded)
    print("encoded_tensor.shape:", encoded_tensor.shape)

    out = generate_text_simple(
        model=model,
        idx=encoded_tensor,
        max_new_tokens=10,
        context_size=GPT_CONFIG_124M["context_length"]
    )
    decoded_text = tokenizer.decode(out.squeeze(0).tolist())

    print(f"\n\n{50*'='}\n{22*' '}OUT\n{50*'='}")
    print("\nOutput:", out)
    print("Output length:", len(out[0]))
    print("Output text:", decoded_text)
```

2.2 GPT-2 模型的训练

创建文件 gpt_train.py。
首先导入我们要用的库。

```python
import matplotlib.pyplot as plt
import os
import torch
import tiktoken
# 从本地库导入
from gpt_model import GPTModel, create_dataloader_v1, generate_text_simple
```

接下来我们编写 token_ids 和 text 之间的转换代码。

```python
def text_to_token_ids(text, tokenizer):
    encoded = tokenizer.encode(text)
    encoded_tensor = torch.tensor(encoded).unsqueeze(0)
    return encoded_tensor
```

接下来我们编写计算损失的代码。

```python
def calc_loss_batch(input_batch, target_batch, model, device):
    input_batch, target_batch = input_batch.to(device), target_batch.to(device)
    logits = model(input_batch)
    loss = torch.nn.functional.cross_entropy(logits.flatten(0, 1),
    target_batch.flatten())
    return loss
```

然后编写整个数据集的损失的计算

```python
 def calc_loss_loader(data_loader, model, device, num_batches=None):
     total_loss = 0.
     if len(data_loader) == 0:
         return float("nan")
     elif num_batches is None:
         num_batches = len(data_loader)
     else:
```

```
            num_batches = min(num_batches, len(data_loader))
        for i, (input_batch, target_batch) in enumerate(data_loader):
            if i < num_batches:
                loss = calc_loss_batch(input_batch, target_batch, model, device)
                total_loss += loss.item()
            else:
                break
        return total_loss / num_batches
```

然后编写评估模型的函数

```python
def evaluate_model(model, train_loader, val_loader, device, eval_iter):
    model.eval()
    with torch.no_grad():
        train_loss = calc_loss_loader(train_loader, model, device,
        num_batches=eval_iter)
        val_loss = calc_loss_loader(val_loader, model, device,
        num_batches=eval_iter)
    model.train()
    return train_loss, val_loss
```

然后编写一下简单的生成补全的代码。

```python
def generate_and_print_sample(model, tokenizer, device, start_context):
    model.eval()
    context_size = model.pos_emb.weight.shape[0]
    encoded = text_to_token_ids(start_context, tokenizer).to(device)
    with torch.no_grad():
        token_ids = generate_text_simple(
            model=model,
            idx=encoded,
            max_new_tokens=50,
            context_size=context_size,
        )
        decoded_text = token_ids_to_text(token_ids, tokenizer)
        print(decoded_text.replace("\n", " ")) # 压缩一下打印的格式
    model.train()
```

然后我们编写训练模型的代码

```python
def train_model_simple(
    model,
    train_loader,
    val_loader,
    optimizer,
    device,
    num_epochs,
    eval_freq,
    eval_iter,
    start_context,
    tokenizer,
):
    # 初始化一些列表，用来跟踪损失和其它一些指标
```

```
    train_losses, val_losses, track_tokens_seen = [], [], []
    tokens_seen = 0
    global_step = -1

    # 主训练循环
    for epoch in range(num_epochs):
        model.train() # 将模型设置为训练模式

        for input_batch, target_batch in train_loader:
            optimizer.zero_grad() # 将上一批数据的损失的梯度清空
            loss = calc_loss_batch(input_batch, target_batch, model, device)
            loss.backward() # 计算损失函数中参数的梯度
            optimizer.step() # 使用梯度更新模型的参数
            tokens_seen += input_batch.numel()
            global_step += 1

            # 可选的评估步骤
            if global_step % eval_freq == 0:
                train_loss, val_loss = evaluate_model(
                    model,
                    train_loader,
                    val_loader,
                    device,
                    eval_iter,
                )
                train_losses.append(train_loss)
                val_losses.append(val_loss)
                track_tokens_seen.append(tokens_seen)
                print(f"Ep {epoch+1} (Step {global_step:06d}): "
                      f"Train loss {train_loss:.3f}, Val loss {val_loss:.3f}")

        # 每一轮训练完毕之后，测试一下补全的文本
        generate_and_print_sample(
            model, tokenizer, device, start_context
        )

    return train_losses, val_losses, track_tokens_seen
```

编写一下绘制损失图像的代码

```python
def plot_losses(epochs_seen, tokens_seen, train_losses, val_losses):
    fig, ax1 = plt.subplots()

    # 绘制每一个epoch的训练损失和验证损失
    ax1.plot(epochs_seen, train_losses, label="Training loss")
    ax1.plot(epochs_seen, val_losses, linestyle="-.", label="Validation loss")
    ax1.set_xlabel("Epochs")
    ax1.set_ylabel("Loss")
    ax1.legend(loc="upper right")

    ax2 = ax1.twiny()
```

```
    ax2.plot(tokens_seen, train_losses, alpha=0)
    ax2.set_xlabel("Tokens seen")

    fig.tight_layout()
    # plt.show()
```

接下来编写 main 函数。

```python
def main(gpt_config, settings):

    torch.manual_seed(123)
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    ##############################
    # 准备训练数据
    ##############################

    file_path = "data.txt"
    with open(file_path, "r", encoding="utf-8") as file:
        text_data = file.read()

    ##############################
    # 初始化模型
    ##############################

    model = GPTModel(gpt_config)
    model.to(device)
    optimizer = torch.optim.AdamW(
        model.parameters(), lr=settings["learning_rate"],
        weight_decay=settings["weight_decay"]
    )

    ##############################
    # 准备训练数据集和验证数据集
    ##############################

    # Train/validation ratio
    train_ratio = 0.90
    split_idx = int(train_ratio * len(text_data))

    train_loader = create_dataloader_v1(
        text_data[:split_idx],
        batch_size=settings["batch_size"],
        max_length=gpt_config["context_length"],
        stride=gpt_config["context_length"],
        drop_last=True,
        shuffle=True,
        num_workers=0
    )

    val_loader = create_dataloader_v1(
```

```
        text_data[split_idx:],
        batch_size=settings["batch_size"],
        max_length=gpt_config["context_length"],
        stride=gpt_config["context_length"],
        drop_last=False,
        shuffle=False,
        num_workers=0
    )

    ##############################
    # 训练模型
    ##############################

    tokenizer = tiktoken.get_encoding("gpt2")

    train_losses, val_losses, tokens_seen = train_model_simple(
        model, train_loader, val_loader, optimizer, device,
        num_epochs=settings["num_epochs"], eval_freq=5, eval_iter=1,
        start_context="Every effort moves you", tokenizer=tokenizer
    )

    return train_losses, val_losses, tokens_seen, model
```

然后编写文件的入口点函数

```python
if __name__ == "__main__":

    GPT_CONFIG_124M = {
        "vocab_size": 50257,    # 词汇表大小
        "context_length": 256,  # 上下文长度（开源的gpt-2模型的上下文长度是1024）
        "emb_dim": 768,         # 词嵌入维度
        "n_heads": 12,          # 注意力头的数量
        "n_layers": 12,         # transformer block的数量
        "drop_rate": 0.1,       # Dropout rate
        "qkv_bias": False       # Query-key-value bias
    }

    OTHER_SETTINGS = {
        "learning_rate": 5e-4,
        "num_epochs": 10,
        "batch_size": 2,
        "weight_decay": 0.1
    }

    ############################
    # 训练模型
    ############################

    train_losses, val_losses, tokens_seen, model = main(GPT_CONFIG_124M,
    OTHER_SETTINGS)

```

```
    ############################
    # 训练完成之后
    ############################

    # 绘制损失图像
    epochs_tensor = torch.linspace(0, OTHER_SETTINGS["num_epochs"], len(train_losses))
    plot_losses(epochs_tensor, tokens_seen, train_losses, val_losses)
    plt.savefig("loss.pdf")

    # 保存和加载模型的代码
    torch.save(model.state_dict(), "model.pth")
    model = GPTModel(GPT_CONFIG_124M)
    model.load_state_dict(torch.load("model.pth", weights_only=True))
```

2.3 加载 OpenAI 开源的 GPT-2 预训练权重

Python

```
BASE_CONFIG = {
    "vocab_size": 50257,    # Vocabulary size
    "context_length": 1024, # Context length
    "drop_rate": 0.0,       # Dropout rate
    "qkv_bias": True        # Query-key-value bias
}

model_configs = {
    "gpt2-small (124M)": {"emb_dim": 768, "n_layers": 12, "n_heads": 12},
    "gpt2-medium (355M)": {"emb_dim": 1024, "n_layers": 24, "n_heads": 16},
    "gpt2-large (774M)": {"emb_dim": 1280, "n_layers": 36, "n_heads": 20},
    "gpt2-xl (1558M)": {"emb_dim": 1600, "n_layers": 48, "n_heads": 25},
}


CHOOSE_MODEL = "gpt2-small (124M)"
BASE_CONFIG.update(model_configs[CHOOSE_MODEL])
```

接下来下载预训练权重。

Python

```
file_name = "gpt2-small-124M.pth"
# file_name = "gpt2-medium-355M.pth"
# file_name = "gpt2-large-774M.pth"
# file_name = "gpt2-xl-1558M.pth"

import os
import requests

url = f"https://huggingface.co/rasbt/gpt2-from-scratch-pytorch/resolve/main/{file_name}"

if not os.path.exists(file_name):
    response = requests.get(url, timeout=60)
    response.raise_for_status()
```

```
    with open(file_name, "wb") as f:
        f.write(response.content)
    print(f"Downloaded to {file_name}")
```

加载权重

```python
import torch
from gpt_model import GPTModel

gpt = GPTModel(BASE_CONFIG)
gpt.load_state_dict(torch.load(file_name, weights_only=True))
gpt.eval()

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
gpt.to(device)
```

生成文本

```python
import tiktoken

def text_to_token_ids(text, tokenizer):
    encoded = tokenizer.encode(text, allowed_special={"◁endoftext▷"})
    encoded_tensor = torch.tensor(encoded).unsqueeze(0)  # 添加批次维度
    return encoded_tensor


def token_ids_to_text(token_ids, tokenizer):
    flat = token_ids.squeeze(0)  # 去掉批次维度
    return tokenizer.decode(flat.tolist())

def generate(model, idx, max_new_tokens, context_size, temperature=0.0, top_k=None, eos_id=None):

    # 计算下一个token的分数，只关注最后一个token
    for _ in range(max_new_tokens):
        idx_cond = idx[:, -context_size:]
        with torch.no_grad():
            logits = model(idx_cond)
        logits = logits[:, -1, :]

        # top_k采样策略
        if top_k is not None:
            # 选分数最大的k个token作为候选token
            top_logits, _ = torch.topk(logits, top_k)
            min_val = top_logits[:, -1]
            logits = torch.where(logits < min_val, torch.tensor(float("-inf")).to(logits.device), logits)

        # 温度缩放
        if temperature > 0.0:
            logits = logits / temperature

```

```
            # 数值稳定性技巧：在softmax之前，减去一行中的最大值。
            # 因为softmax是按行进行的
            logits = logits - logits.max(dim=-1, keepdim=True).values

            # 使用softmax获取概率值
            probs = torch.softmax(logits, dim=-1)  # (batch_size, context_len)

            # 从分布中进行采样（多项分布）
            idx_next = torch.multinomial(probs, num_samples=1)  # (batch_size, 1)

        # 贪婪策略：直接选择分数最高的token
        else:
            idx_next = torch.argmax(logits, dim=-1, keepdim=True)  # (batch_size,
            1)

        # 如果设置了eos_id，然后生成了eos_token，那么停止生成
        if idx_next == eos_id:
            break

        # 将采样得到的下一个token的id添加到序列的末尾
        idx = torch.cat((idx, idx_next), dim=1)  # (batch_size, num_tokens+1)

    return idx

torch.manual_seed(123)

tokenizer = tiktoken.get_encoding("gpt2")

token_ids = generate(
    model=gpt.to(device),
    idx=text_to_token_ids("Every effort moves", tokenizer).to(device),
    max_new_tokens=30,
    context_size=BASE_CONFIG["context_length"],
    top_k=1,
    temperature=1.0
)

print("Output text:\n", token_ids_to_text(token_ids, tokenizer))
```

II

强化学习

3 强化学习简介 47
3.1 基本概念 47
3.2 价值函数（Value Function） 50
3.3 倒立摆环境编程实践 51
3.4 马尔可夫决策过程 55
3.5 贝尔曼方程 57
3.6 下一步 60

4 策略梯度法 61
4.1 原始策略梯度法 61
4.2 REINFORCE 73
4.3 带基线（baseline）的策略梯度法 . . . 77
4.4 Actor-Critic（演员-评论家架构） . . 78
4.5 多步 TD 误差 84
4.6 广义优势估计 85
4.7 相关数学证明 86
4.8 总结 90

5 近端策略优化（PPO） 91
5.1 策略梯度法存在的问题 91
5.2 近端策略优化 93

6 PPO 实战 99
6.1 网络结构的定义 99
6.2 PPO 算法的实现 99
6.3 广义优势估计的计算 102
6.4 训练循环 102

7 PPO 背后的数学 105
7.1 数学背景知识 105
7.2 替代目标 112
7.3 补充知识 116

8 组相对策略优化（GRPO） 119
8.1 GRPO 原理 119
8.2 使用 GRPO 玩倒立摆游戏 120

3. 强化学习简介

3.1 基本概念

强化学习（reinforcement learning，RL）讨论的问题是智能体（agent）怎么在复杂、不确定的环境（environment）中最大化它能获得的奖励。

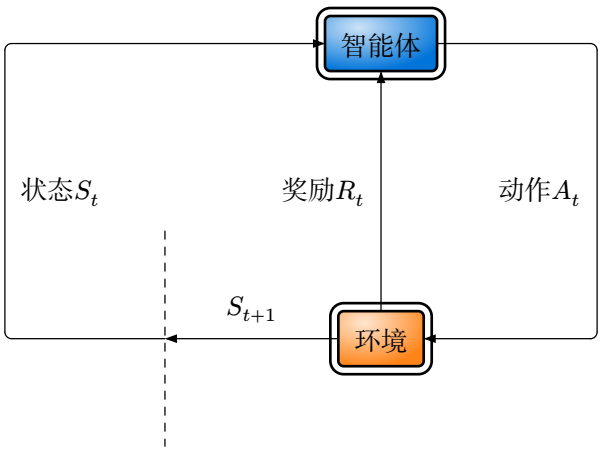


图 3.1 强化学习交互循环

> 名词
>
> - Agent：智能体，代理，智能代理
> - Environment：环境
> - State：状态
> - Reward：奖励
> - Action：动作，行动

强化学习由两部分组成：**智能体**和**环境**。在强化学习过程中，智能体与环境一直在交互。智能体在环境中获取某个状态后，它会利用该状态输出一个动作（action）。然后这个动作会在环境中被执行，环境会根据智能体采取的动作，输出下一个状态以及当前这个动作带来的奖励。智能体的目的就是尽可能多地从环境中获取奖励。

下图就是一个倒立摆环境。

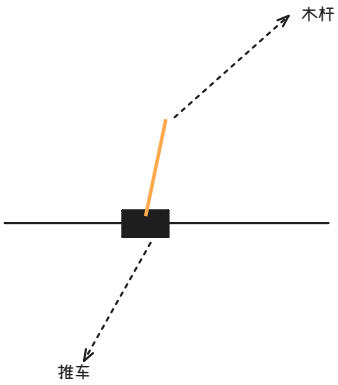


图 3.2 倒立摆环境

在这个环境中，智能体是推车。

- 动作空间：推车有两个动作，**向左推**和**向右推**。
- 状态：
  - 推车的位置
  - 推车的速度
  - 木杆的角度
  - 木杆的角速度
- 奖励：推车采取向左推或者向右推的动作之后，只要木杆不倒下，奖励就是 1。

游戏的结束条件：

- 推车将杆子推出了屏幕（失败）
- 杆子倒下（失败）
- 采取了 200 次推车的动作，杆子也没倒下（成功）

那么推车应该采取什么样的动作，杆子才能不倒下呢？或者说，推车应该采取什么样的**策略**（policy），杆子才能不倒下呢？因为只有杆子不倒下，我们才能不停的获得奖励。

如果推车每采取一个动作，杆子还能保持平衡，那么我们获得的**即时奖励**是 1。

如果到游戏结束，杆子也没有倒下，那么我们将获得最大的奖励 200。

而推车每次要采取什么动作，是取决于环境的状态的。也就是推车要根据环境的状态来决定下一步采取什么动作。例如：此时杆子向左偏，那么推车可能需要采取向左推的动作，才能使杆子不倒下。

换句话说，推车要采取的策略是一个**函数**。

要采取的动作的概率分布=策略函数(环境的状态)

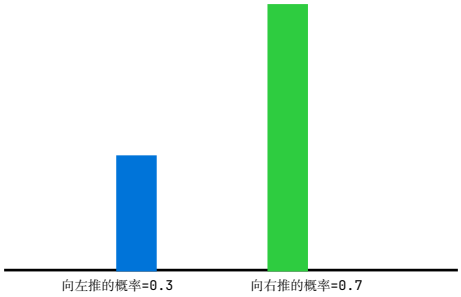


图 3.3 策略是一个函数

而这里**要采取的动作**一般来说是一个概率分布。例如：向左推的概率=0.3，向右推的概率=0.7。

然后从这个分布中进行采样，采样一个动作出来。当然也可以直接选概率最大的动作。

而我们使用什么样的策略呢？当然是能够让我们在游戏结束时，能够获得最大奖励的策略啦。

当然，这里选择策略有很多的讲究。如果我们每次采取的都是概率最大的动作，那么每次可能都会获得最大的即时奖励。如果这样做的话，可能到了一定的时候，不管向左推还是向右推获得的即时奖励可能都是 0，也就是不管怎么推，杆子都会倒下（杆子偏离的角度太大，怎么推都没用）。也就是说，我们太看重眼前利益，忽略了长远的利益。这样就是只**利用**而不**探索**。所以有时也需要采取一下概率低的动作，探索一下环境，万一未来获得的奖励更多呢。

所以偶尔也需要采取一下概率小的动作。

> 提示
>
> 所谓**从概率分布中采样**的意思是，例如概率分布是向左推的概率是 0.7，向右推的概率是 0.3。那么从这个概率分布中采样一个动作出来，有 70% 的概率采取的动作是向左推。但也有 30% 的概率向右推。

举个例子：如果我们每次都去最熟悉的餐馆吃饭，可能体验都还可以。而如果去不熟悉的餐厅吃饭，可能体验不好，也可能体验超过了之前的餐馆。在舒适区只利用不探索，就会固步自封。冒险可能会受到伤害，但长期来看，可能会得到提升。

策略是智能体的动作模型，它决定了智能体的动作。它其实是一个函数，用于把输入的状态变成动作。

策略的数学符号是 π ，所以策略也就是 π 函数。即 $\pi(a|s)=p(a_t=a|s_t=s)$ 。输入一个状态 s ，输出一个概率分布。用条件概率来看，就是在条件：状态为 s 的情况下，策略采取动作 a 的概率是多少。这个概率是智能体所有动作的概率，然后对这个概率分布进行采样，可得到智能体将采取的动作。比如可能是有 0.7 的概率往左，0.3 的概率往右，那么通过采样就可以得到智能体将采取的动作。

$$\pi(a|s)=p(a_t=a|s_t=s) \tag{3.1}$$

在状态为s时，采取动作a的概率

t时刻环境的状态为s

图 3.4 策略函数

所以有如下：

$$\begin{aligned}\pi(a=a_{\text{向左推}}|s)&=0.7\\ \pi(a=a_{\text{向右推}}|s)&=0.3\end{aligned}\tag{3.2}$$

如果这个 π 函数是一个神经网络，那么这就是**深度学习 + 强化学习 = 深度强化学习**。当今 AI 界最热门的话题。

所以玩倒立摆游戏的一个**回合**（episode）就是环境的状态为 S_0，推车采取动作 A_0，然后获得奖励 R_0，然后环境的状态转移到了 S_1，推车接着采取动作 A_1，然后获得奖励 R_1，…。下标是时刻，或者时间步。把它们写到一起，就是一个**轨迹**（trajectory）。轨迹用数学符号 τ 表示，读作**掏**。

$$\tau=(S_0,A_0,R_0,S_1,A_1,R_1,S_2,A_2,R_2,\cdots)\tag{3.3}$$

由于根据环境的状态，采取的动作是从一个概率分布中采样得到的，所以轨迹会有很多很多条。

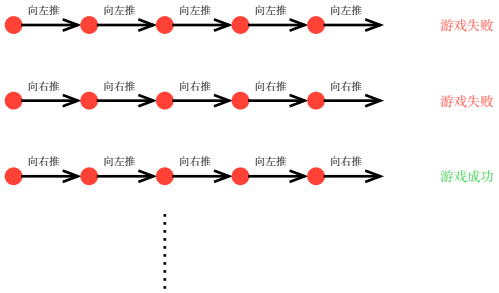


图 3.5 无数条轨迹

3.2 价值函数（Value Function）

当位于时刻 t 时，环境此时处于状态 S_t，然后我们根据策略函数开始采取动作，那么未来我们一共能获得多少奖励呢？环境处于状态 S_t，我们采取的动作是 A_t，获取的奖励是 R_t，然后环境的状态从 S_t 转移到了 S_{t+1}，然后采取动作 A_{t+1}，然后获得即时奖励 R_{t+1}，然后环境的状态从 S_{t+1} 转移到了 S_{t+2}，然后环境会给我们即时奖励 R_{t+2}，……。

但是未来的奖励不如现在的奖励有吸引力，所以需要**打折**。那么，从 t 时刻起，未来一共获得的奖励叫做**回报**（或者收益，Return）。

$$G_t=R_t+\gamma R_{t+1}+\gamma^2R_{t+2}+\cdots\tag{3.4}$$

由上面的式子可以推导出如下递推公式：

$$G_t=R_t+\gamma G_{t+1}\tag{3.5}$$

γ 叫做折扣因子。随着时间的推移，奖励会被 γ 指数级削弱。这个 γ 被称为折扣因子（discount rate），其被设定为 0.0 和 1.0 之间的实数。如果折扣因子是 0.9，那么有以下式子成立。

$$G_t=R_t+0.9R_{t+1}+0.81R_{t+2}+\cdots\tag{3.6}$$

引入折扣因子主要是为了防止连续性任务的收益变得无穷大。在连续性任务中，如果没有折扣因子（或 $\gamma=1$），那么收益就会发散到无穷大。因此，设置折扣因子可以防止收益的发散。

折扣因子也使近期的奖励显得更加重要。这解释了人类乃至生物的许多行动原理。例如，你会选择今天拿到 10000 元还是一年后拿到 20000 元？如果折扣因子使未来的回报呈指数级下降，那么眼前的回报就会更有吸引力。

如果我们用倒立摆作为例子，然后我们运行两个时间步。得到下图。

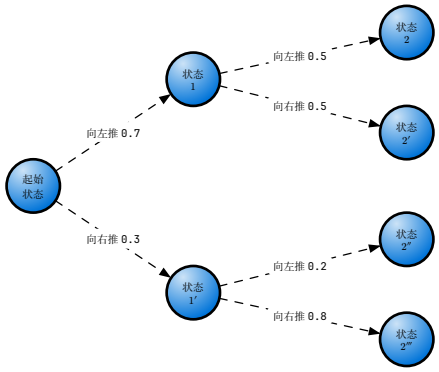


图 3.6 倒立摆环境运行 2 个时间步

可以看到，一共有 4 条轨迹。每条轨迹都有一个总的回报。而每条轨迹也都有一个产生的概率。那么我们如何评估在环境处于状态 S_t 时，一直采取策略 π 未来会获得多少回报呢？也就是未来的预期回报（回报的期望值）是多少呢？那就是**状态价值函数**（State Value Function）。

$$\begin{aligned} V_\pi(s) &= \mathbb{E}_\pi[G_t|S_t=s] \\ &= \mathbb{E}_\pi\left[\sum_{k=0}^{\infty}\gamma^k R_{t+k} \mid S_t=s\right], \text{对所有的 } s\in S \end{aligned} \tag{3.7}$$

> 提示
>
> 状态价值函数，衡量的是在环境处于状态 s 时，一直按照策略 π 来采取动作，最终的预期回报。

状态价值函数的另一种重要的表示形式如下：

$$V_\pi(S_t=s) = \mathbb{E}_\pi[R_t + \gamma V_\pi(S_{t+1}=s'|S_t=s)] \tag{3.8}$$

> 贝尔曼期望方程
>
> $$V_\pi(S_t) = \mathbb{E}_\pi[R_t + \gamma V_\pi(S_{t+1}|S_t)] \tag{3.9}$$
>
> 直观解释如下：
>
> - 当前环境对你的价值 = 当前环境现在给你的奖励 + 折扣因子 × 未来环境对你的价值
> - 你对公司的评估 = 现在公司给你的薪资 + 折扣因子 × 你对公司未来的评估

3.3 倒立摆环境编程实践

安装依赖 Shell

```
$ pip install gym==0.25.2
$ pip install numpy==1.26
$ pip install pygame
```

后续基于倒立摆环境的所有依赖如下：

倒立摆环境需要的所有依赖 Python

```python
import gym
import random
```

```
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from matplotlib import rc
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.distributions import Categorical
```

然后我们来创建一个倒立摆环境。

创建倒立摆环境 Python

```
# version确认
print(gym.__version__)  # 0.25.2版本
# 创建倒立摆环境
env = gym.make("CartPole-v0")
```

这样就生成了倒立摆的环境。

将推车向右或向左移动，以保持杆子的平衡。倒立摆的结束条件是杆子的平衡被打破（杆子超过一定的角度），或者推车的移动位置超出了某个范围。

继续执行下面的代码

打印初始状态和动作空间维度 Python

```
state = env.reset() # 重置环境的状态为S_0
print(state) # 初始状态S_0
action_space = env.action_space # 推车有几个动作？（向左推，向右推）
print(action_space) # 动作空间的维度=2
```

上面的代码通过 `state = env.reset()`获得了初始状态。观察它的输出，你会发现它是拥有 4 个元素的数组。作为参考，下面依次列出这 4 个元素。

- 推车的位置
- 推车的速度
- 木杆的角度
- 木杆的角速度

另外，我们可以通过 `env.action_space` 获得行动的维度（可采取的行动数）。它的输出是一个名为 `Discrete(2)` 的类实例。这意味着有两个候选行动。具体来说，`0` 对应的是向左移动推车的行动，`1` 对应的是向右移动推车的行动。下面实际地采取行动，向前推进一个时间步。

执行1步动作 Python

```
action = 0 # 或者 1
next_state, reward, done, info = env.step(action)
print(next_state)
```

代码通过 `env.step(action)` 采取行动。作为结果 ，我们得到了以下 4 个信息。

执行 1 步动作后得到的信息：

- 下一个状态 `next_state`：S_1
- 奖励 `reward`：R_0
- 是否结束的标志位 `done`
- 附加信息 `info`

reward 是标量值（float）。这次的任务在保持平衡的时候总是会得到奖励 1。info 包含有助于调试的信息（如环境模型）。但在实现和评估强化学习的算法时，基本不会用到 info。

我们先来实现一个随机智能体。也就是这个智能体的策略非常简单，无论环境处于什么状态，我们都是从两个动作中随机采样一个动作。代码如下。

```python
state = env.reset() # 重置环境的状态
done = False # 游戏是否结束，初始值为False，也就是未结束
episode_rewards = []  # 每回合奖励[$R_0, R_1, \cdots, R_T$]
total_reward = 0 # 带折扣因子的回报（总奖励）
frames = []          # 保存每一帧
gamma = 0.95 # 折扣因子$\gamma$

# done=True时结束
while not done:
    # 渲染画面，并保存帧
    frames.append(env.render(mode="rgb_array"))

    # 随机选择一个动作$A_t$，0：向左推，1：向右推
    action = random.choice([0, 1])

    # 下一个状态$S_{t+1}$，即时奖励$R_t$，是否结束，_
    next_state, reward, done, _ = env.step(action)
    # [$R_0, R_1, \cdots, R_T$]
    episode_rewards.append(reward)

# 逆序计算$G_t = R_t + \gamma G_{t+1}$
for r in episode_rewards[::-1]:
    # $G_t = R_t + \gamma G_{t+1}$
    total_reward = r + gamma * total_reward

print("total_reward:", total_reward)
env.close()
```

动态规划思想

回报（总奖励的逆序计算）

- 从时间 0 开始的回报为$G_0 = R_0 + \gamma R_1 + \gamma^2 R_2 + \gamma^3 R_3 = R_0 + \gamma G_1$
- 从时间 1 开始的回报为$G_1 = R_1 + \gamma R_2 + \gamma^2 R_3 = R_1 + \gamma G_2$
- 从时间 2 开始的回报为$G_2 = R_2 + \gamma R_3 = R_2 + \gamma G_3$
- 从时间 3 开始的回报为$G_3 = R_3$

所以有：$G_t = R_t + \gamma G_{t+1}$
所以我们要进行逆序计算。

霍纳法则（秦九韶算法）

$$\begin{aligned} & a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n \\ = & a_0 + x(a_1 + x(a_2 + x(a_3 + \cdots + x(a_{n-1} + x a_n)\cdots))) \end{aligned} \tag{3.10}$$

动态规划

- 前向过程：采样一条轨迹，得到每个时刻的即时奖励。
- 反向过程：逆序计算回报。

然后我们将创建一个函数 show_animation 来将倒立摆环境的每一帧保存成动画，后续就可以播放了。代码如下：

Python

```python
def show_animation(imgs):
    rc("animation", html="jshtml")
    fig, ax = plt.subplots(1, 1, figsize=(5, 3))
    frames = []

    text = ax.text(10, 20, "", fontsize=12, color="black")

    for i, img in enumerate(imgs):
        frame = [ax.imshow(img, animated=True)]
        frame.append(ax.text(10, 20, f"Step: {i+1}", animated=True))  # Step数表示
        frames.append(frame)

    ax.axis("off")

    ani = animation.ArtistAnimation(fig, frames, interval=100, blit=True)

    # 保存动画
    ani.save("cartpole.mp4", writer="ffmpeg")
    ani.save("cartpole.gif", writer="pillow")

    plt.close(fig)
    return ani
```

然后我们播放动画。

```python
show_animation(frames)
```

由于我们使用的是**随机策略**，所以倒立摆游戏很快就结束了。

3.4 马尔可夫决策过程

3.4.1 基本概念

马尔可夫决策过程（MDP，Markov Decision Process）。决策过程是智能体通过与环境互动决定其行动的过程。

智能体所处的环境根据其行动而发生改变。在强化学习中，这种情况被称为环境的“状态”（state）。在MDP中，状态的变化取决于智能体的行动，智能体在环境的状态转移后执行新的行动。

在MDP中，我们需要“时间”的概念。在某一时刻，智能体会采取行动并因此转移到一个新的状态。此时的时间单位叫作“时间步”。由于时间步是智能体做出决定的间隔时间，因此它的实际单位取决于问题。

智能体要考虑的是将来获得的奖励总和，而不是眼前的奖励。换句话说，智能体的目标是实现奖励总和最大化。

在MDP中，智能体与环境之间会进行互动。要点在于当智能体采取行动时，状态会发生转移，随之获得的奖励也会相应改变。

参见图 3.1 。

假设在时刻t的状态是S_t。基于这个状态S_t，智能体执行行动A_t，获得奖励R_t并转移到下一个状态S_{t+1}。智能体与环境之间的这种实际的互动产生了以下转移。

$$S_0, A_0, R_0, S_1, A_1, R_1, S_2, A_2, R_2, \cdots \tag{3.11}$$

这个时间序列数据从第一个状态S_0开始。在状态S_0，智能体执行动作A_0并获得奖励R_0。时刻变为1，状态变为S_1。接下来，基于状态S_1，智能体执行动作A_1并获得奖励R_1，然后进入下一个状态S_2……这个流程不断持续下去。

MDP通过数学式来表示智能体、环境以及二者之间的互动。要做到这一点，需要用数学式来表达以下3个要素。

- 状态转移：状态如何转移。
- 奖励：如何给予奖励。
- 策略：智能体如何决定行动。

3.4.2 状态转移

假设智能体现在处于状态s并执行了动作a，那么转移到下一个状态s'的概率可以用如下方式表示。

$$p(s'|s,a) \tag{3.12}$$

> 状态和观察
>
> - 状态（state）：如果能观察到整个环境完整的信息，就叫做状态
> - 观察（observation）：只能观察到环境的部分状态（信息）

竖杠|的右侧是表示“条件”的概率变量。对于当前问题，条件对应于在状态s选择了行动a。在给定这两个条件的情况下，转移到 s' 的概率可以表示为 $p(s'|s,a)$ 。像 $p(s'|s,a)$ 这样的概率叫作状态转移概率（state transition probability）。

给定当前状态s和采取的动作a的情况下，**不一定**会确定性的跳转到某个状态s'，所以是状态转移概率。

对于倒立摆环境而言，给定当前状态，采取向左推的动作，一定会确定性的跳转到某个状态。

但对于下棋环境来说，棋盘是状态，当 Agent 下了一个子儿后，棋盘的状态取决于 Agent 的对手将棋下在哪里，所以状态转移是不确定的。

$p(s'|s,a)$决定了下一个状态s'只取决于当前状态s和行动a。

$$p(s_t|s_{t-1},a_{t-1}) = p(s_t|s_{t-1},a_{t-1},s_{t-2},a_{t-2},\cdots,s_0,a_0) \tag{3.13}$$

换句话说，状态转移不需要过去的信息——此前处于什么状态以及执行了哪些行动。这个特性被称为**马尔可夫性质**（Markov property）。

> 提示
>
> MDP 通过假设马尔可夫性质的存在来模拟状态转移和奖励。引入马尔可夫性质主要是为了使问题更容易解决。如果不假定马尔可夫性质，那么就必须考虑之前的所有状态和行动，而且组合的数量会呈指数级增长。

3.4.3 奖励函数

当智能体处于状态 s ，执行行动 a，下一个状态是 s' 时，得到的奖励由函数 $r(s,a,s')$ 定义。$r(s,a,s')$ 叫做**奖励函数**。

> 倒立摆环境中的奖励函数
>
> 倒立摆环境中的奖励是确定性的，只要木杆不倒下，就给奖励 1。

奖励函数的建模是一个非常困难的问题，例如：我们的机器人 Agent，它的任务是将插头正确的插入插座，如果只有正确的插入插头才会给奖励 1，其它情况都给奖励 0，那么机器人可能一辈子也拿不到奖励了。这叫做**奖励稀疏性问题**。

3.4.4 策略

策略表示智能体如何决定其行动。策略的关键在于它使得智能体仅根据当前状态来决定其行动。之所以说只基于当前状态就足够了，是因为环境的转移是符合马尔可夫性质的。

环境的状态转移只以当前状态s和动作a为条件来决定下一个状态s'，而不需要先前的信息。同样，奖励也是基于当前状态s、动作a和转移后的状态s'来决定的。这意味着关于环境的所有必要信息都在当前状态中。因此，智能体只需基于当前状态即可决定其行动。

> 提示
>
> MDP 的马尔可夫性质可以被看作是对环境而不是对智能体的约束。这意味着为了满足马尔可夫性质，环境需要保持某个“状态”。从智能体的角度来看，在当前状态下有足够的信息来做出最佳选择，所以它可以在此基础上采取行动。

智能体的行动是由随机性策略决定的，数学式如下所示。

$$\pi(a|s) \tag{3.14}$$

$\pi(a|s)$ 表示在状态 s 下采取行动 a 的概率。

我们已经成功地用数学式来表示状态转移、奖励函数和策略。接下来让我们使用这三者来定义 MDP 的目标。

3.4.5 MDP 的目标

到目前为止，我们已经用数学式描述了环境和智能体的行动。简要回顾一下，智能体根据策略$\pi(a|s)$采取行动。首先，它根据它的行动和状态转移概率$p(s'|s,a)$转移到下一个状态。然后，它根据奖励函数$r(s,a,s')$获得奖励。在这个框架内，MDP 的目标是找到**最优策略**（optimal policy）。最优策略是使收益最大化的策略。

3.4.6 收益

为了定义收益，我们思考这样一个场景：设时刻为t，状态为S_t（其中t是任意值）。然后，智能体根据策略π执行动作A_t，获得奖励R_t，之后转移到新状态S_{t+1}，这个过程不断重复进行。在这种情况下，收益G_t的定义如下所示。

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots \tag{3.15}$$

γ为折扣因子。

3.4.7 状态价值函数

我们已经重新定义了“收益”。智能体的目标是使这种收益最大化。这里有一点需要注意，那就是智能体和环境的行动可能是“随机性”的。智能体可能随机地决定行动，状态也可能随机转移。在这种情况下，获得的收益将呈现随机的特点。即使从相同的状态开始，不同回合的收益也随机变化。例如，某个回合的收益为10.4，另一个回合的收益为8.7。

为了处理这种随机行动，需要使用期望值或“收益的期望值”作为衡量标准。收益的期望值的数学式如下所示。

$$v_\pi(s) = \mathbb{E}[G_t|S_t = s, \pi] \tag{3.16}$$

我们指定的条件是状态 S_t 为 s 智能体的策略为 π （其中时刻 t 是任意值）。在这些条件下，智能体获得的收益的期望值为上面的公式。在这里，我们用特殊符号 $v_\pi(s)$ 来表示收益的期望值。就是**状态价值函数**（state-value function）。

在公式的右侧，智能体的策略π被作为条件给出。这是因为如果策略π发生变化，那么智能体获得的奖励也会发生变化，而这些奖励的总和，即收益也会发生变化。为了明确这一点，状态价值函数通常会写作$v_\pi(s)$，将π写在v的右下角。此外，上面的公式也可以写成下面的形式。

$$v_\pi(s) = \mathbb{E}_\pi[G_t|S_t = s] \tag{3.17}$$

在上面的式子中，记载 π 的位置是 $\mathbb{E}_\pi$ 。与式 **(3.16)** 一样，这么写的含义是策略 π 是作为条件给出的。从现在起，本教程将采取式式 **(3.17)** 的风格来书写式子。

在强化学习中，我们的目标是获得最优策略。

最优策略的状态价值函数叫作**最优状态价值函数**（optimal state-value function）。可以使用 v_* 来表示最优状态价值函数。

3.5 贝尔曼方程

贝尔曼方程是在 MDP 中成立的最重要的方程，为许多强化学习算法提供了重要基础。

这里使用骰子作为例子。我们使用的骰子是理想的六面体，每一面的数字出现的概率都是$\frac{1}{6}$。在数学式中，我们用随机变量x表示掷骰子出现的数字，x是1和6之间的整数。那么每个数字出现的概率就是$\frac{1}{6}$。 我们用$p(x) = \frac{1}{6}$表示骰子的数字出现的概率。现在来计算掷骰子的期望值。计算式如下所示。

$$\begin{aligned}\mathbb{E}[x] &= 1 \times \frac{1}{6} + 2 \times \frac{1}{6} + 3 \times \frac{1}{6} + 4 \times \frac{1}{6} + 5 \times \frac{1}{6} + 6 \times \frac{1}{6} \\ &= 3.5\end{aligned} \tag{3.18}$$

如上所述，期望值是在所有情况下“出现的数字”和“概率”相乘并加在一起得到的和。顺带一提，如果使用∑符号，那么期望值的数学式如下所示。

定义 **3.1 — 期望的定义以及相关性质.**

$$\begin{aligned}\mathbb{E}[x] &= \sum_x xp(x) \\ \mathbb{E}[f(x)] &= \sum_x f(x)p(x) \\ \mathbb{E}[X] &= \mathbb{E}[\mathbb{E}[X|Y]] \quad \text{（全期望公式）} \\ \mathbb{E}[X|Y] &= \mathbb{E}[\mathbb{E}[X|Z]|Y] \text{（条件期望的迭代公式）}\end{aligned} \tag{3.19}$$

另外，x和y同时发生的概率（这叫作“联合概率”）如所示。

$$p(x,y) = p(x)p(y|x) = p(y)p(x|y) \tag{3.20}$$

假设奖励用 $r(x,y)$ 来表示，那么奖励的期望值如下

$$\begin{aligned}\mathbb{E}[r(x,y)] &= \sum_x \sum_y p(x,y)r(x,y) \\ &= \sum_x \sum_y p(x)p(y|x)r(x,y)\end{aligned} \tag{3.21}$$

回顾收益（回报）的定义。

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots \tag{3.22}$$

那么有以下公式

$$G_{t+1} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \tag{3.23}$$

所以有以下递推公式：

$$\begin{aligned}G_t &= R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots \\ &= R_t + \gamma(R_{t+1} + \gamma R_{t+2} + \cdots) \\ &= R_t + \gamma G_{t+1}\end{aligned} \tag{3.24}$$

这种递推关系被用于强化学习的许多理论和算法中。

下面将递推公式代入状态价值函数的定义式中。状态价值函数是收益的期望值，其数学式的定义如下所示。

$$v_\pi(s) = \mathbb{E}_\pi[G_t|S_t = s] \tag{3.25}$$

如上面的公式所示，状态 s 的价值函数被表示为 $v_\pi(s)$ 。将递推公式带入上面的式子的 G_t 中，得到下面的式子：

$$\begin{aligned}v_\pi(s) &= \mathbb{E}_\pi[G_t|S_t = s] \\ &= \mathbb{E}_\pi[R_t + \gamma G_{t+1}|S_t = s] \\ &= \mathbb{E}_\pi[R_t|S_t = s] + \gamma\mathbb{E}_\pi[G_{t+1}|S_t = s]\end{aligned} \tag{3.26}$$

🔥 期望的线性性质

$\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y]$

先来推导上面公式中的第一项 $\mathbb{E}_\pi[R_t|S_t = s]$ 。

$\mathbb{E}_\pi[R_t|S_t = s]$ 的含义是在 t 时刻，环境的状态是 s ，那么在 t 时刻，获得的即时奖励的期望是多少呢？那么需要考虑所有的情况，然后加起来就可以了。

$$\mathbb{E}_\pi[R_t|S_t=s]=\sum_a\sum_{s'}\pi(a|s)p(s'|s,a)r(s,a,s') \tag{3.27}$$

如上面的数学式所示，将智能体行动的概率$\pi(a|s)$、要转移的状态的概率$p(s'|s,a)$和奖励函数$r(s,a,s')$相乘。对所有候选项都进行上述计算，得到它们的总和。

> 提示
>
> 式 (3.27) 中只有s是确定的，假设可选择的动作只有两个$\{a_1,a_2\}$，环境状态只有两种$\{s_1,s_2\}$。那么状态转移和概率如下：
>
> $$\begin{aligned}
> &s\to a_1\to s_1:\pi(a=a_1|s)=0.2,p(s'=s_1|s,a=a_1)=0.3,r(s,a=a_1,s'=s_1)=10\\
> &s\to a_1\to s_2:\pi(a=a_1|s)=0.2,p(s'=s_2|s,a=a_1)=0.7,r(s,a=a_1,s'=s_2)=20\\
> &s\to a_2\to s_1:\pi(a=a_2|s)=0.8,p(s'=s_1|s,a=a_2)=0.5,r(s,a=a_2,s'=s_1)=30\\
> &s\to a_2\to s_2:\pi(a=a_2|s)=0.8,p(s'=s_2|s,a=a_2)=0.5,r(s,a=a_2,s'=s_2)=40
> \end{aligned}$$
>
> 那么即时奖励的期望是：
>
> $$\begin{aligned}
> \mathbb{E}&=0.2\times0.3\times10+0.2\times0.7\times20+0.8\times0.5\times30+0.8\times0.5\times40\\
> &=0.6+2.8+12.0+16.0\\
> &=31.4
> \end{aligned}$$

第一项推导完毕，接下来推导第二项。

$$v_\pi(s)=\underbrace{\mathbb{E}_\pi[R_t|S_t=s]}_{\sum_{a,s'}\pi(a|s)p(s'|s,a)r(s,a,s')}+\underbrace{\gamma\mathbb{E}_\pi[G_{t+1}|S_t=s]}_{\text{接下来推导这一项}} \tag{3.28}$$

图 3.7 推导第二项

剩下的项是$\gamma\mathbb{E}_\pi[G_{t+1}|S_t=s]$。由于$\gamma$是常数，因此我们要看的是$\mathbb{E}_\pi[G_{t+1}|S_t=s]$。这个式子虽然与状态价值函数的定义式相似，但在$G_{t+1}$的部分有所不同。状态价值函数的式子如下所示，式子中是G_t，而不是G_{t+1}。

$$v_\pi(s)=\mathbb{E}_\pi[G_t|S_t=s] \tag{3.29}$$

因此，我们首先要将$t+1$代入上面的式子的t中。式子变化如下。

$$v_\pi(s')=\mathbb{E}_\pi[G_{t+1}|S_{t+1}=s'] \tag{3.30}$$

这就是状态$S_{t+1}=s'$时的价值函数。接下来要关注的是$\mathbb{E}_\pi[G_{t+1}|S_t=s]$。这是在当前时刻为$t$时，下一个时刻$(t+1)$的收益期望值。解决的关键在于将条件$S_t=s$变为$S_{t+1}=s$的形式。换句话说，就是要进入下一个时刻。

通过观察可以得到

$$\begin{aligned}
\mathbb{E}_\pi[G_{t+1}|S_t=s]&=\sum_{a,s'}\pi(a|s)p(s'|s,a)\mathbb{E}_\pi[G_{t+1}|S_{t+1}=s']\\
&=\sum_{a,s'}\pi(a|s)p(s'|s,a)v_\pi(s')
\end{aligned} \tag{3.31}$$

完成第二项的展开以后，汇总一下得到

定理 3.1 — 贝尔曼方程.

$$\begin{aligned} v_\pi(s) &= \mathbb{E}_\pi[R_t|S_t=s] + \gamma\mathbb{E}_\pi[G_{t+1}|S_t=s] \\ &= \sum_{a,s'}\pi(a|s)p(s'|s,a)r(s,a,s') + \gamma\sum_{a,s'}\pi(a|s)p(s'|s,a)v_\pi(s') \\ &= \sum_{a,s'}\pi(a|s)p(s'|s,a)\{r(s,a,s') + \gamma v_\pi(s')\} \end{aligned} \tag{3.32}$$

上面的式子就是大名鼎鼎的**贝尔曼方程**。贝尔曼方程是表示状态s的价值函数和下一个可能的状态s'的价值函数之间关系的式子。这个贝尔曼方程对所有状态s和所有策略 π 都成立。

状态价值函数的贝尔曼方程的另一种重要的表示形式：**贝尔曼期望方程**。

定理 3.2 — 贝尔曼期望方程.

$$V_\pi(S_t) = \mathbb{E}_\pi[R_t + \gamma V_\pi(S_{t+1})|S_t] \tag{3.33}$$

贝尔曼期望方程的推导过程如下：

$$\begin{aligned} V_\pi(S_t) &= \mathbb{E}_\pi[G_t|S_t] \\ &= \mathbb{E}_\pi\left[\sum_{k=0}^{\infty}\gamma^k R_{t+k} \mid S_t\right] \\ &= \mathbb{E}_\pi\left[R_t + \gamma\sum_{k=0}^{\infty}\gamma^k R_{t+k+1} \mid S_t\right] \\ &= \mathbb{E}_\pi[R_t + \gamma G_{t+1}|S_t] \\ &= \mathbb{E}_\pi[R_t|S_t] + \gamma\mathbb{E}_\pi[G_{t+1}|S_t] \\ &= \mathbb{E}_\pi[R_t|S_t] + \gamma\mathbb{E}_\pi[\mathbb{E}_\pi[G_{t+1}|S_{t+1}]|S_t] \\ &= \mathbb{E}_\pi[R_t|S_t] + \gamma\mathbb{E}_\pi[V_\pi(S_{t+1})|S_t] \\ &= \mathbb{E}_\pi[R_t + \gamma V_\pi(S_{t+1})|S_t] \end{aligned} \tag{3.34}$$

3.6 下一步

智能体的目标是使收益最大化。这里有一点需要注意，那就是智能体和环境的行动可能是“随机性”的。智能体可能随机地决定行动，状态也可能随机转移。在这种情况下，获得的收益将呈现随机的特点。即使从相同的状态开始，不同回合的收益也随机变化。例如，某个回合的收益为10.4，另一个回合的收益为8.7。

在倒立摆环境中，我们如何选择智能体的“策略”来让木杆多坚持一段时间呢？

接下来我们来学习**策略梯度法**（`policy gradient method`）。

4. 策略梯度法

4.1 原始策略梯度法

4.1.1 理论介绍

在倒立摆游戏的环境中，我们唯一能控制的，就是推车所采用的**策略**。而环境和奖励函数我们是无法控制的。

我们可以控制的	我们无法控制的	我们无法控制的
策略函数	环境	奖励函数
推车的策略	倒立摆环境	保持平衡，奖励为 1

表 4.1　只有策略函数是我们可以控制的，环境和奖励我们无法控制

现在通行的做法就是策略函数是一个神经网络。输入是环境的状态，输出是动作的概率分布。

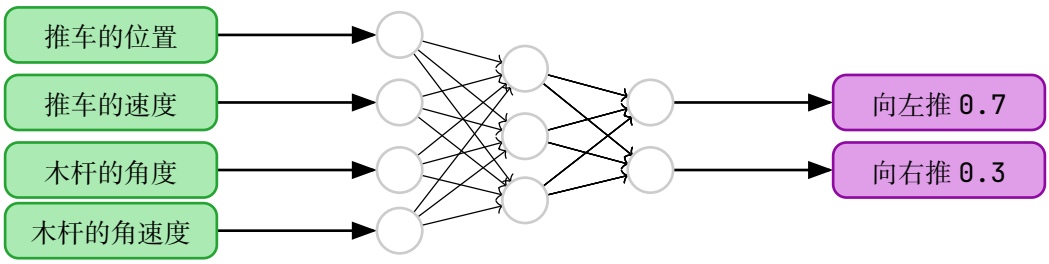


图 4.1　输出的动作是概率

上图就是一个例子。玩倒立摆游戏。策略函数是一个神经网络；输入是倒立摆环境的状态，一个浮点数数组；输出是我们可以执行的动作，有几个动作，输出层就有几个神经元。假设我们现在可以执行的动作有 2 个，输出层就有 2 个神经元，每个神经元对应一个可以采取的动作。输入一个东西后，策略神经网络会给每一个可以采取的动作一个分数。我们可以把这个分数当作概率，智能体根据概率的分布来决定它要采取的动作，比如 0.7 的概率向左推、0.3 的概率向右推。概率分布不同，推车采取的动作就会不一样。

如下面的式子所示，首先，环境是一个函数，我们可以把倒立摆环境看成一个函数，虽然它不一定是神经网络，可能是基于规则的（rule-based）模型，但我们可以把它看作一个函数。

$$\textbf{倒立摆环境}(S_t, A_t) \to (S_{t+1}, R_t) \tag{4.1}$$

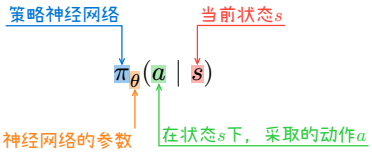
倒立摆环境就是一个基于规则的函数，只要推车不倒下，就返回奖励 1。

讲了这么多策略有关的东西，大家应该明白策略是什么东西了。

而通过神经网络等方法将策略模型化，并使用梯度来优化策略的方法叫作**策略梯度法**（policy gradient method）。

研究者们提出了各种基于策略梯度法的算法。本章首先介绍最简单的策略梯度法。然后，在改进这个简单的策略梯度法的过程中，我们推导出了被称为 REINFORCE 的算法。接下来，在进一步改进 REINFORCE 的过程中， 我们又推导出了**带基线**的 REINFORCE 方法和 Actor-Critic（演员-评论家） 方法。

随机性策略用数学式可以表示为$\pi(a|s)$。$\pi(a|s)$是在状态s下采取动作a的概率。这里采用神经网络对策略进行建模。此时用符号θ来汇总表示神经网络的所有权重参数（θ是将所有参数的元素排成一列的向量）。另外，可以将基于神经网络的策略表示为$\pi_\theta(a|s)$，如下所示。



(4.2)

还是倒立摆环境，每当倒立摆环境处于某个状态 s 时，我们就会使用神经网络 π_θ 来决定要采取什么动作。

那么，问题是：策略神经网络怎么训练？

> 神经网络的训练
>
> 想要训练一个神经网络，需要有**输入–输出**对。例如，MNIST 手写数字数据集，要训练一个可以识别手写数字的卷积神经网络，需要构建一个网络结构，然后提供输入（图片）以及输出（分类标签）。
>
> 当然，还得有**损失函数**，例如交叉熵损失函数或者均方误差损失函数，等等。

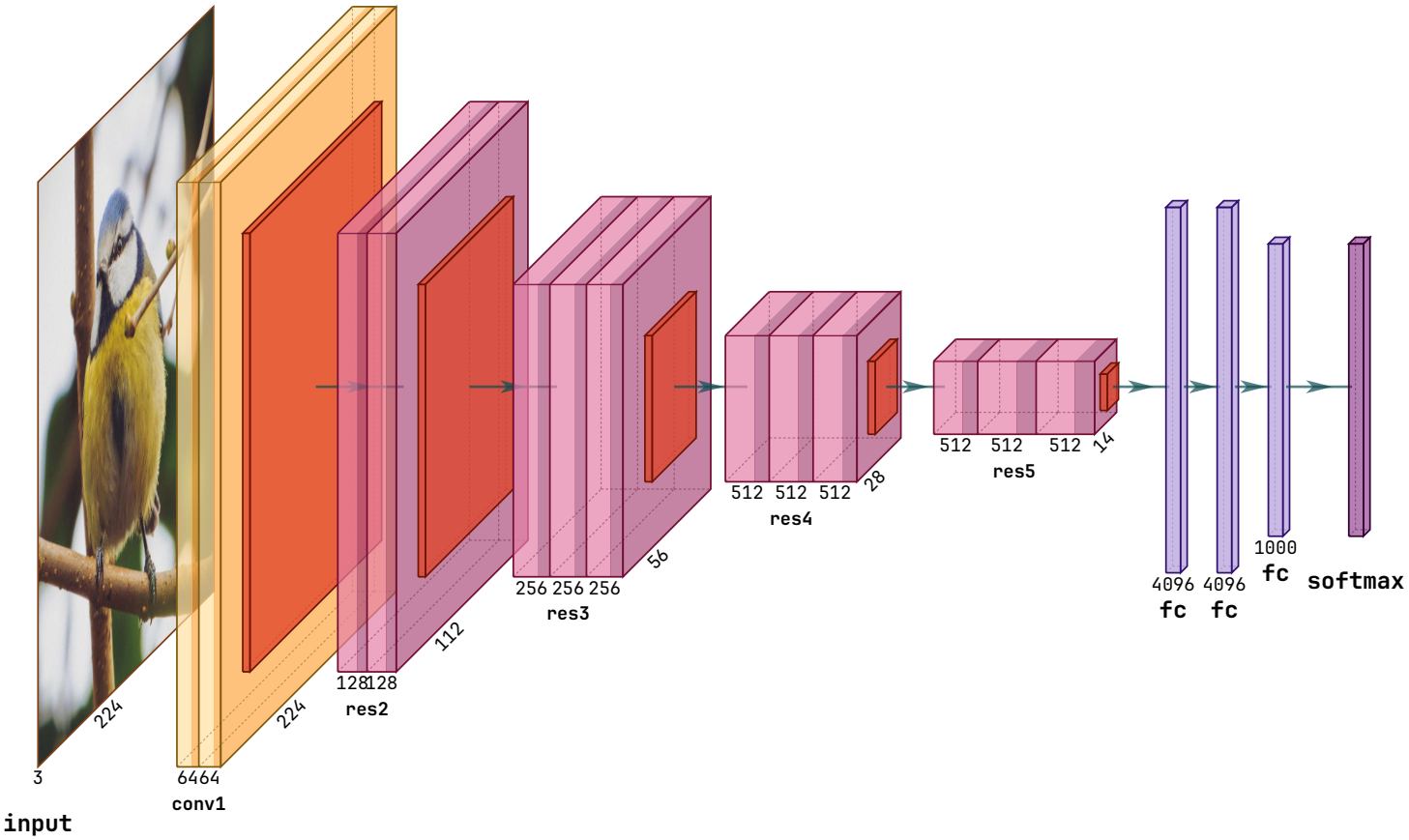


图 4.2 ResNet18

但是对于倒立摆环境，想要训练推车的策略神经网络，输入是什么，输出是什么？以及损失函数又是什么？

首先回顾一下第一章的知识。

首先明确问题的设定。这里考虑的是回合制任务，并基于策略π_θ选择动作的情况。在这种情况下，假定得到了以下由“状态、动作、奖励”构成的时间序列数据。

$$\tau = (S_0, A_0, R_0, S_1, A_1, R_1, \cdots, S_{T+1}) \tag{4.3}$$

这个τ也叫作轨迹（trajectory）。

而一条轨迹发生的概率是：

$$\begin{aligned} \Pr(\tau) &= p(S_0)\pi_\theta(A_0|S_0)p(S_1|S_0,A_0)\pi_\theta(A_1|S_1)p(S_2|S_1,A_1)\cdots\pi_\theta(A_T|S_T)p(S_{T+1}|S_T,A_T) \\ &= p(S_0)\prod_{t=1}^{T}\pi_\theta(A_t|S_t)p(S_{t+1}|S_t,A_t) \end{aligned} \tag{4.4}$$

此时可以使用折扣因子γ对回报（Return，收益）作如下定义。

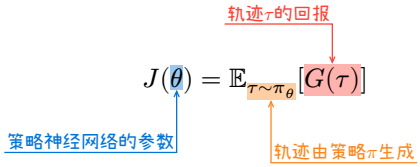
$$G(\tau) = R_0 + \gamma R_1 + \gamma^2 R_2 + \cdots + \gamma^T R_T \tag{4.5}$$

为了表明回报可以由τ计算出来，上面的式子将其表示为了$G(\tau)$。此时，目标函数$J(\theta)$可以表示为下面的式子。

定义 4.1 — 强化学习的目标函数.

$$J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[G(\tau)] \tag{4.6}$$

回报$G(\tau)$是随机变动的，所以它的期望值是目标函数。上式中期望值$\mathbb{E}$的下标为$\tau \sim \pi_\theta$ ，这个下标表示τ是基于π_θ 生成的。



$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau)] \tag{4.7}$$

图 4.3 策略梯度法的目标函数

我们的目标是让$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau)]$最大。但是我们发现$G(\tau)$这个轨迹的回报无法求导，换句话说就是对目标函数**无法进行优化**。

目标函数可以优化的意思其实就是可以求导。

在神经网络的训练中，我们的目的是让**损失函数**最小。而在策略梯度法中，我们的目的是让**目标函数**最大。所以都可以使用梯度法。让损失函数最小，使用梯度下降法。让目标函数最大，使用梯度上升法。

所以，我们还是得求解$J(\theta)$的梯度。需要求导的参数是θ。

确定了目标函数后，下一步是计算它的梯度。这里将参数θ的梯度表示为∇_θ。我们的目标是求$\nabla_\theta J(\theta)$。∇是梯度符号。

定理 4.1 — **策略梯度定理.**

$$\begin{aligned} \nabla_\theta J(\theta) &= \nabla_\theta \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau)] \\ &= \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{T} G(\tau) \nabla_\theta \log \pi_\theta(A_t|S_t)\right] \end{aligned} \tag{4.8}$$

上面的式子中值得注意的是，∇_θ在$\mathbb{E}$中（梯度计算的部分是$\nabla_\theta \log \pi_\theta(A_t|S_t)$）。后面会对此做详细介绍。求出$\nabla_\theta J(\theta)$之后，接下来更新神经网络的参数。最优化方法多种多样，下面的式子表示的是一种简单的方法。

$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta) \tag{4.9}$$

图 4.4 策略梯度法的梯度上升算法

上面的式子朝着梯度的方向更新参数θ。更新的值与α相关。这里的α表示学习率。这是属于梯度上升法的算法。

只要沿着梯度上升的方向更新参数θ，那么目标函数$J(\theta)$就会越来越大，也就是所有轨迹的期望就会越来越大，那么就是我们的策略越来越好了。

> ⚡ 危险
>
> 策略梯度法求得的梯度是一个期望值，而期望值无法计算。

这里有一个问题，那就是**真正的期望**是无法准确求出来的。因为期望是所有的轨迹得到的奖励计算出来的。而轨迹有无数条。如果我们能够走满倒立摆的 200 步，那么不同轨迹的数量可能是2^{200}，如下图所示，这是一个天文数字。所以我们希望能够求出**近似**期望值的数值。比如利用**大数定理**，多采样几条轨迹，那么就会比较接近期望值。

$$
\begin{aligned}
&S_0^{\tau_0} \to A_0^{\tau_0} \to S_1^{\tau_0} \to A_1^{\tau_0} \to \cdots \\
&S_0^{\tau_1} \to A_0^{\tau_1} \to S_1^{\tau_1} \to A_1^{\tau_1} \to \cdots \\
&S_0^{\tau_2} \to A_0^{\tau_2} \to S_1^{\tau_2} \to A_1^{\tau_2} \to \cdots \\
&\cdots\cdots
\end{aligned} \tag{4.10}
$$

图 4.5 无数条轨迹

大数定律

样本数量越多，则其算术平均值就有越高的概率接近期望。

例如，抛掷一颗均匀的 6 面的骰子，1，2，3，4，5，6 应等概率出现，所以每次扔出骰子后，出现点数的期望是

$$
\frac{1+2+3+4+5+6}{6} = 3.5 \tag{4.11}
$$

根据大数定理，如果多次抛掷骰子，随着抛掷次数的增加，平均值(样本平均值)应该接近 3.5。

如果随机变量$X_1, X_2, ...$是独立同分布，且期望$\mathbb{E}(X_1) = \mathbb{E}(X_2) = \cdots = \mu$。那么

$$
\overline{X}_n = \frac{1}{n}(X_1 + X_2 + \cdots + X_n) \tag{4.12}
$$

当$n \to \infty$时，收敛于真值

$$
\overline{X}_n \to \mu \tag{4.13}
$$

$$
\begin{aligned}
&S_0^{\tau_0} \to A_0^{\tau_0} \to S_1^{\tau_0} \to A_1^{\tau_0} \to \cdots \\
&S_0^{\tau_1} \to A_0^{\tau_1} \to S_1^{\tau_1} \to A_1^{\tau_1} \to \cdots \\
&S_0^{\tau_2} \to A_0^{\tau_2} \to S_1^{\tau_2} \to A_1^{\tau_2} \to \cdots \\
&S_0^{\tau_3} \to A_0^{\tau_3} \to S_1^{\tau_3} \to A_1^{\tau_3} \to \cdots \\
&S_0^{\tau_4} \to A_0^{\tau_4} \to S_1^{\tau_4} \to A_1^{\tau_4} \to \cdots \\
&\cdots\cdots
\end{aligned} \tag{4.14}
$$

图 4.6 只采样n条轨迹

如式 (4.8) 所示，$\nabla_\theta J(\theta)$表示期望值。接下来我们来计算期望值。这里，我们令策略π_θ的智能体实际采取动作，得到n个轨迹τ，如上图所示。此时，通过对每个τ计算式子的期望值内部的式子($\sum_{t=0}^{T} G(\tau)\nabla_\theta \log \pi_\theta(A_t|S_t)$)，并求出其平均值，从而近似得到$\nabla_\theta J(\theta)$。数学式如下所示。

$$
\begin{aligned}
&\textbf{采样}: \tau^{(i)} \sim \pi_\theta \quad (i = 1, 2, \cdots, n) \\
&x^{(i)} = \sum_{t=0}^{T} G\left(\tau^{(i)}\right)\nabla_\theta \log \pi_\theta\left(A_t^{(i)}|S_t^{(i)}\right) \\
&\nabla_\theta J(\theta) \approx \frac{x^{(1)} + x^{(2)} + \cdots + x^{(n)}}{n}
\end{aligned} \tag{4.15}
$$

上面式子中的$\tau^{(i)}$，表示在第i回合得到的轨迹，$A_t^{(i)}$表示在第i回合的时刻t的动作，$S_t^{(i)}$表示在第i回合的时刻t的状态。

另外，再思考一下蒙特卡洛方法的样本数为1，即上式中$n = 1$的情况，如下图所示。

$$
\begin{aligned}
&S_0^{\tau_0} \rightarrow A_0^{\tau_0} \rightarrow S_1^{\tau_0} \rightarrow A_1^{\tau_0} \rightarrow \cdots \\
&S_0^{\tau_1} \rightarrow A_0^{\tau_1} \rightarrow S_1^{\tau_1} \rightarrow A_1^{\tau_1} \rightarrow \cdots \\
&S_0^{\tau_2} \rightarrow A_0^{\tau_2} \rightarrow S_1^{\tau_2} \rightarrow A_1^{\tau_2} \rightarrow \cdots \\
&S_0^{\tau_3} \rightarrow A_0^{\tau_3} \rightarrow S_1^{\tau_3} \rightarrow A_1^{\tau_3} \rightarrow \cdots \\
&S_0^{\tau_4} \rightarrow A_0^{\tau_4} \rightarrow S_1^{\tau_4} \rightarrow A_1^{\tau_4} \rightarrow \cdots \\
&\cdots\cdots
\end{aligned}
\tag{4.16}
$$

图 4.7 只采样1条轨迹

在这种情况下，数学式可以简化为如下形式。

$$
\begin{gathered}
\textbf{采样}：\tau \sim \pi_\theta \\
\nabla_\theta J(\theta) \approx \sum_{t=0}^{T} G(\tau) \nabla_\theta \log \pi_\theta(A_t|S_t)
\end{gathered}
\tag{4.17}
$$

为了简单起见，本章将使用以上面的式子为对象的策略梯度法。上面的式子的计算就是对所有时刻（$t = 0 \sim T$）求$\nabla_\theta \log \pi_\theta(A_t|S_t)$，然后将各梯度乘以作为权重的回报$G(\tau)$，最后求它们的和。这个计算过程如下图所示。

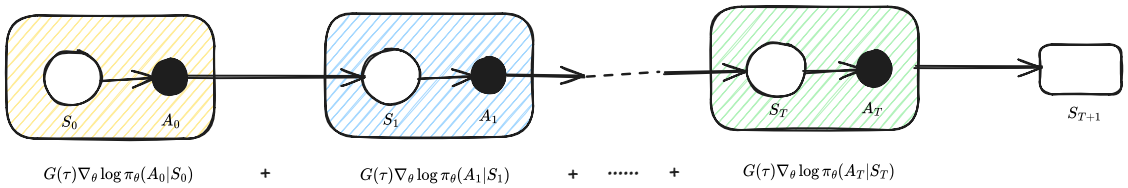


图 4.8 使用一条轨迹计算策略梯度

4.1.2 代码实现

接下来，我们用代码来实现一下策略梯度法。

我们讲一些实现细节。我们可以把强化学习想成一个分类问题，这个分类问题就是输入倒立摆的状态，输出某个类。在解决分类问题时，我们要收集一些训练数据，数据中要有输入与输出的对。在实现的时候，我们把倒立摆的状态当作分类器的输入，就像在解决图像分类的问题，只是现在的类不是图像里面的东西，而是看到倒立摆的状态我们要采取什么样的动作，每一个动作就是一个类。比如第一个类是向左，第二个类是向右。

在解决分类问题时，我们要有输入和正确的输出，要有训练数据。但在强化学习中，我们通过采样来获得训练数据。假设在采样的过程中，在某个状态下，我们采样到要采取动作 A，那么就把动作 A 当作标准答案（`ground truth`）。比如，我们在某个状态下，采样到要向左。因为是采样，所以向左这个动作不一定概率最高。假设我们采样到向左，在训练的时候，让智能体调整网络的参数，如果看到某个状态，我们就向左。在一般的分类问题里面，我们在实现分类的时候，目标函数都会写成最小化交叉熵（`cross entropy`），最小化交叉熵就是最大化对数似然（`log likelihood`）。

从极大似然估计的角度看策略梯度

$$
\begin{aligned}
J(\theta) &= \sum_{t=0}^{T} G(\tau) \log \pi_\theta(A_t|S_t) \\
&= G(\tau) \log \left(\prod_{t=0}^{T} \pi_\theta(A_t|S_t) \right)
\end{aligned}
\tag{4.18}
$$

最大化目标就会提升某些状态对应的动作的概率。

策略神经网络π_θ Python

```python
class PolicyNet(nn.Module):
    def __init__(self, action_size):
        super().__init__()
        self.l1 = nn.Linear(4, 128) # 推车环境中状态是一个4维数组
        self.l2 = nn.Linear(128, action_size) # 输出是一个2维数组

    def forward(self, x):
        x = F.relu(self.l1(x))
        x = F.softmax(self.l2(x), dim=1)
        return x # 输出：[$\pi_\theta(a=$向左推$|s), \pi_\theta(a=$向右推$|s)$]
```

上面的代码实现的神经网络模型由两层全连接层构成。最终输出的元素数是动作的数量（`action_size`）。由于这个最终输出是`softmax`函数的输出，因此可以得到每个动作的概率。

这个网络结构就是我们要训练的策略神经网络π_θ。其中θ就是神经网络的参数。

我们还记得，策略神经网络的输入是环境的状态，输出是要采取的动作的概率分布。所谓动作的概率分布就是：向左推的概率是多少，向右推的概率是多少。

而我们的网络结构输入的张量的维度是4。为什么呢？因为倒立摆环境的状态有4个维度。

输出是`action_size`个维度。也就是动作的数量个维度。

softmax

如果将具有n个元素的向量输入到`softmax`函数中，那么输出的同样是具有n个元素的向量。此时，第i个输出y_i的式子如下所示。

$$
y_i = \frac{\mathrm{e}^{x_i}}{\sum_{k=1}^{n} \mathrm{e}^{x_k}} \tag{4.19}
$$

这里的e是自然常数（值为2.71828...的无限小数）。`softmax`函数的输出值全部为0以上1以下的实数，它们的合计值为1（$\sum_{i=1}^{n} y_i = 1$）。因此，`softmax`函数的输出可以作为概率使用。

下面是`Agent`类的代码。首先显示初始化和`get_action`方法。

```python
class Agent:
    def __init__(self):
        self.gamma = 0.98 # 折扣因子γ
        self.lr = 0.0002 # 学习率α
        self.action_size = 2 # 两个动作

        self.pi = PolicyNet(self.action_size) # 策略神经网络π_θ
        self.optimizer = optim.Adam(self.pi.parameters(), lr=self.lr)

    def get_action(self, state):
        probs = self.pi(torch.tensor(state).unsqueeze(0)).squeeze(0)
        # 根据动作的概率分布创建一个二项分布
        m = Categorical(probs)
        # 使用二项分布采样动作π_θ(a_t|s_t)
        action = m.sample().item()

        # (采样的动作, [向左推的概率, 向右推的概率])
        return action, probs
```

代码 4.1 智能体代码

get_action 方法决定了在 state 状态下采取的动作。为此，可以通过 self.pi(state)进行神经网络的前向传播，得到概率分布 probs。然后，基于该概率分布，进行一次动作的采样。该方法还返回了所有动作的概率（上面代码中的 probs）。

下面来试用一下 get_action 方法。代码如下所示。

执行1步动作然后反向传播，观察结果

```python
env = gym.make("CartPole-v0")
state = env.reset() # S_0
agent = Agent()

action, probs = agent.get_action(state)
print("动作：", action) # A_0
print("动作的概率：", probs[action].item()) # π_θ(A_0|S_0)

G = 100.0 # G(τ)
J = -G * probs[action].log() # −G(τ) log π_θ(A_0|S_0)
print("J: ", J)

J.backward() # 求导（梯度）−∇_θ G(τ) log π_θ(A_0|S_0)}
agent.optimizer.step() # 梯度下降：θ = θ + α∇_θ G(τ) log π_θ(A_0|S_0)
# 在相同的状态state下，采取动作的概率变大了
# （测试一下）将G变为负值，也就是负的奖励，会发现采取动作的概率下降了
_, probs = agent.get_action(state)
print("动作：", action)
print("动作的概率：", probs[action].item())
```

上面的代码取出了初始状态下的动作及其概率。另外，它还显示了使用虚拟的权重来计算由下面的式子表示的梯度的代码（这是从式子取出的$t = 0$的相关项的式子）。

$$G(\tau)\nabla_\theta \log \pi_\theta(A_0|S_0) \tag{4.20}$$

作为参考，下面对照列出了上面的代码中出现的变量与相应的数学式。

- `probs[action]`：$\pi_\theta(A_0|S_0)$
- `G`：$G(\tau)$
- `J`：$G(\tau)\log \pi_\theta(A_0|S_0)$

求出 `J` 之后，通过 `J.backward()`求$G(\tau)\nabla_\theta \log \pi_\theta(A_0|S_0)$。下面是 `Agent` 类剩下的代码。

我们先来编写采集一条轨迹的详细信息的代码。

```python
class Agent:
    ...

    def collect_trajectory(self, env):
        state = env.reset()
        states, actions, rewards = [], [], []
        done = False

        while not done:
            action, _ = self.get_action(state)
            next_state, reward, done, _ = env.step(action)
            states.append(state) # S_t
            actions.append(action) # A_t
            rewards.append(reward) # R_t
            state = next_state # S_t → S_{t+1}
        # [S_0, S_1, ···, S_T]
        # [A_0, A_1, ···, A_T]
        # [R_0, R_1, ···, R_T]
        return states, actions, rewards
```

代码 4.2 采集一条轨迹的详细数据，接代码 4.1

然后编写更新策略的代码：

```python
class Agent:
    ...

    def update(self, trajectory):
        states, actions, rewards = trajectory
        # 逆序计算G(τ)
        G = 0
        for r in rewards[::-1]:
            G = r + self.gamma * G
        loss = 0
        for s, a in zip(states, actions):
            probs = self.pi(torch.tensor(s).unsqueeze(0)).squeeze(0)
            log_prob = torch.log(probs)[a] # log π_θ(A_t|S_t)
            loss += -log_prob * G # −∑_{t=0}^{T} G(τ) log π_θ(A_t|S_t)

        self.optimizer.zero_grad()
        loss.backward() # ∇_θ{−∑_{t=0}^{T} G(τ) log π_θ(A_t|S_t)}
        self.optimizer.step() # θ = θ − α∇_θ{−∑_{t=0}^{T} G(τ) log π_θ(A_t|S_t)}
```

代码 4.3 更新策略的代码，接代码 4.2

在训练神经网络时，通常要设置损失函数。对于这个例子，我们可以将目标函数$J(\theta)$乘以-1所得到的$-J(\theta)$作为损失函数，此时可以通过梯度下降法的最优化方法（SGD、Adam 等）更新参数。

代码 4.3 中第 10 ~ 14 行可以改写成并行版本，代码如下：

并行版本

```python
states = torch.tensor(states)
actions = torch.tensor(actions).view(-1, 1)
log_probs = torch.log(self.pi(states).gather(1, actions))
loss = -torch.sum(log_probs) * G
```

最后在倒立摆环境中训练智能体。代码如下所示。

```python
env = gym.make("CartPole-v0")
agent = Agent()
return_list = []
episode_list = []

for episode in range(3000):
    trajectory = agent.collect_trajectory(env)
    reward_list = trajectory[2]
    return_list.append(sum(reward_list))
    episode_list.append(episode)

    agent.update(trajectory)

    if episode % 100 == 0:
        print("回合:{}, 总奖励:{:.1f}".format(episode, sum(reward_list)))
```

首先，在 while 语句中，增加智能体获得的奖励（reward）和行动的概率（prob）。然后在离开 while 语句后（回合结束时），通过 agent.update()更新策略。

运行此代码，随着回合的推进，获得的奖励也会增加。下图是结果的示意图。
绘制示意图的代码如下：

```python
def plot_loss(episode_list, return_list, filename):
    f = plt.figure()
    plt.plot(episode_list, return_list)
    plt.xlabel("Episodes")
    plt.ylabel("Returns")
    plt.title("CartPole-v0")
    plt.show()
    f.savefig(filename, bbox_inches="tight")

plot_loss(episode_list, return_list, "pg-loss.pdf")
```

通过观察绘制出来的图像，我们发现每回合的奖励随着训练震荡的很厉害，但是随着训练的进行，每回合获得的总奖励确实越来越多了。

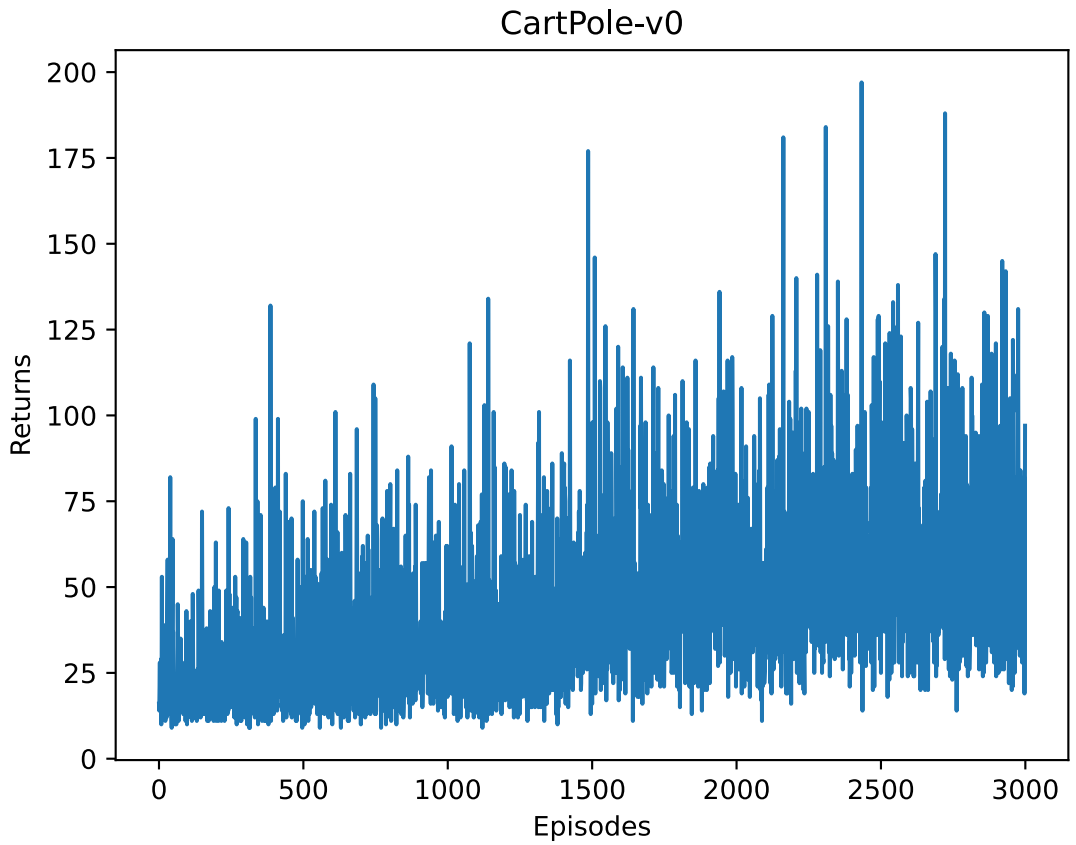


图 4.9 策略梯度算法获得的奖励

我们可以测试一下训练的策略神经网络。看看倒立摆能不能坚持很长时间。

```python
def test_agent(agent, env):
    state = env.reset()
    done = False
    frames = []

    while not done:
```

```
            frames.append(env.render(mode="rgb_array"))
            action, _ = agent.get_action(state)
            next_state, _, done, _ = env.step(action)
            state = next_state

        env.close()
        show_animation(frames)

test_agent(agent, env)
```

我们大概可以知道，随着回合的推进，奖励的总和会逐渐增加。但即使经历了 3000 个回合，依然没有达到这次任务的上限值 200，所以似乎还有改进的余地。下面让我们来改进一下这里推导的最简单的策略梯度法。这个改进算法就是著名的 REINFORCE 算法。

4.1.3 深入讨论

采样一条轨迹的梯度是$\nabla_\theta J(\theta) \approx \sum_{t=0}^{T} G(\tau)\nabla_\theta \log \pi_\theta(A_t|S_t)$。

如果将梯度符号去掉，可以得到如下：

$$\begin{aligned} J(\theta) &\approx \sum_{t=0}^{T} G(\tau) \log \pi_\theta(A_t|S_t) \\ &= G(\tau) \sum_{t=0}^{T} \log \pi_\theta(A_t|S_t) \\ &= G(\tau)(\log \pi_\theta(A_0|S_0) + \log \pi_\theta(A_1|S_1) + \cdots) \end{aligned} \tag{4.21}$$

我们的目标是让$J(\theta)$越大越好，那么在经过梯度上升算法之后，$J(\theta)$会变大，那么肯定有的$\log \pi_\theta(A_t|S_t)$变大了，也可能有的$\log \pi_\theta(A_{t'}|S_{t'})$变小了，但总和肯定变大了。也就是说，策略梯度法的本质是，我们使用**当前策略**π_θ采样一条轨迹τ，如果采样的轨迹的回报$G(\tau)$很高，那么说明我们在某个状态S_t采取的动作A_t比较好，导致了总的奖励也就是回报比较高。那么通过梯度上升法，更新θ之后，新的策略$\pi_{\theta_{\text{new}}}$会提升在$S_t$时执行$A_t$的概率。例如$A_t$是“向左推车”，那么在$S_t$状态下策略采取“向左推车”的概率就会提升，而同时策略采取“向右推车”的概率就会下降。

举一个最简单的例子，我们采样的轨迹只执行了一步动作，就结束了。那么有如下公式

$$J(\theta) = G(\tau) \log \pi_\theta(A_0|S_0) \tag{4.22}$$

- 如果$G > 0$，那么对$J(\theta)$优化之后，策略π_θ在状态S_0时采取动作A_0的概率一定会变大。
- 如果$G < 0$，那么对$J(\theta)$优化之后，策略π_θ在状态S_0时采取动作A_0的概率一定会变小。

如果G特别大，例如$G = 10000$，或者G特别小，例如$G = -10000$，那么梯度上升算法的步长$\alpha G \nabla_\theta \log \pi_\theta(A_0|S_0)$的绝对值一定会很大，进而导致策略的参数$\theta$更新幅度巨大，进而导致策略$\pi_\theta$爆炸式的更新。

换句话说，我们只是提高了在状态$S_0 = s$是采取动作A_0的概率，但是在玩倒立摆游戏时，并不能保证状态s经常出现。碰到其它的状态，更新后的策略表现可能非常的差。好比一个人，突然间中了 1000 万，那么如此大的奖励会直接彻底改变这个人的策略，他的策略彻底变成了靠运气生活。或者一个人犯了一点点小错误，却得到了巨大的惩罚，如此大的惩罚也会让一个人信心崩溃。这就是策略崩溃的例子。

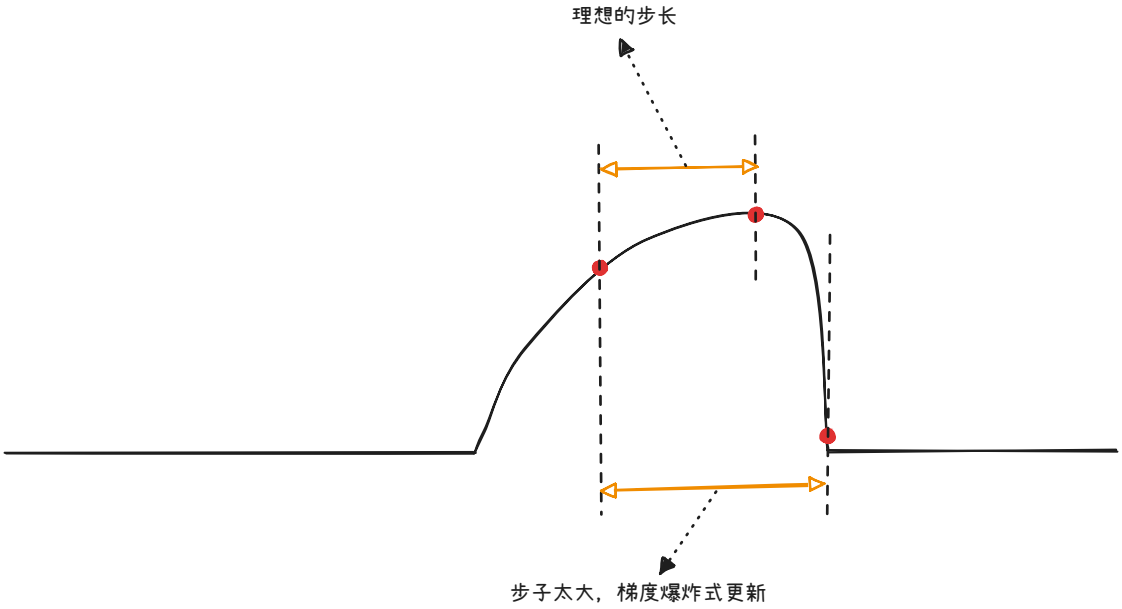


图 4.10 梯度爆炸式更新

4.2 REINFORCE

4.2.1 REINFORCE 算法原理

REINFORCE 是对上一节的策略梯度法的改进算法。本节首先会基于数学式推导 REINFORCE 算法，然后会通过修改之前的部分代码的做法来实现 REINFORCE。

> 提示
>
> REINFORCE 这个名字是“REward Increment = Nonnegative Factor × Offset Reinforcement × Characteristic Eligibility”（奖励增量=非负因子×偏移强化×特征资格）的首字母缩写。

先来复习一下第一节。最简单的梯度策略法是基于下面的公式实现的。

$$\begin{aligned}\nabla_\theta J(\theta) &= \nabla_\theta \mathbb{E}_{\tau\sim\pi_\theta}[G(\tau)] \\ &= \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} G(\tau)\nabla_\theta \log \pi_\theta(A_t|S_t)\right]\end{aligned} \tag{4.23}$$

上面的式子中的$G(\tau)$是目前为止获得的所有奖励的总和（准确地说是“带折扣因子”的奖励的总和）。这里要思考的问题是，无论在哪个时刻t，式子中都是$G(\tau)\nabla_\theta \log \pi_\theta(A_t|S_t)$，我们始终会使用固定不变的权重$G(\tau)$来增加（或减少）采取行动$A_t$的概率。

智能体行动的好坏是根据行动之后获得的奖励总和来评估的（回顾一下价值函数的定义）。反过来说，采取某个行动之前获得的奖励与该行动的好坏无关。如果要评估在某个时刻t采取的行动A_t，那么在此之前做了什么以及获得了多少奖励都无所谓。我们是根据采取行动A_t之后的结果（在时刻t以后获得的奖励的总和）来判断行动A_t的好坏的。

上面的式子中行动A_t的权重是$G(\tau)$。这个权重$G(\tau)$包括在时刻t之前的奖励。也就是说，原本不相关的奖励作为噪声数据包含在内了。为了改进这一点（去除噪声数据），可以对权重$G(\tau)$作如下修改。

定理 4.2 — REINFORCE 版策略梯度定理.

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} G_t \nabla_\theta \log \pi_\theta(A_t|S_t)\right] \tag{4.24}$$
$$G_t = R_t + \gamma R_{t+1} + \cdots + \gamma^{T-1} R_T$$

如上式所示，权重变成了G_t。权重G_t是在时刻$t \sim T$获得的奖励的总和。因此，选择行动A_t的概率将由不包含时刻t之前的奖励的权重G_t增强。这就是改进第一节的策略梯度法的思路。基于上式的算法叫作 REINFORCE。

提示

基于上式的 REINFORCE 算法优于最简单的策略梯度法（基于第一节的公式的算法）。通过无限增加的样本数，两个公式都会收敛到正确的$\nabla_\theta J(\theta)$（可以说是无偏差的）。但第一个公式的方差更大，因为公式中的权重包含了无关的数据（噪声）。

4.2.2 REINFORCE 的代码实现

由于 REINFORCE 的方差小，因此即使数据样本少，也能高精度地近似数据。下面我们来实现 REINFORCE 以验证其精度。REINFORCE 的代码与上一节中的代码基本相同，不同之处只有 Agent 类的 update 方法。下面仅列出了不同部分的代码。

```python
class Agent:
    ...

    def update(self, trajectory):
        states, actions, rewards = trajectory
        G, loss = 0, 0
        for r, s, a in zip(rewards[::-1], states[::-1], actions[::-1]):
            G = r + self.gamma * G # G_t = R_t + γG_{t+1}
            probs = self.pi(torch.tensor(s).unsqueeze(0)).squeeze(0)
            log_prob = torch.log(probs)[a] # log π_θ(A_t|S_t)
            loss += -log_prob * G # -Σ_{t=0}^{T} G_t log π_θ(A_t|S_t)

        self.optimizer.zero_grad()
        loss.backward()
        self.optimizer.step()
```

我们使用可视化代码将回合和奖励绘制出来。

```python
plot_loss(episode_list, return_list, "reinforce-pg-loss.pdf")
test_agent(agent, env)
```

从上图中可以看出，随着回合的推进，奖励的总和会逐渐增加。与上一次的结果相比，不但训练稳定了，训练速度也提高了。

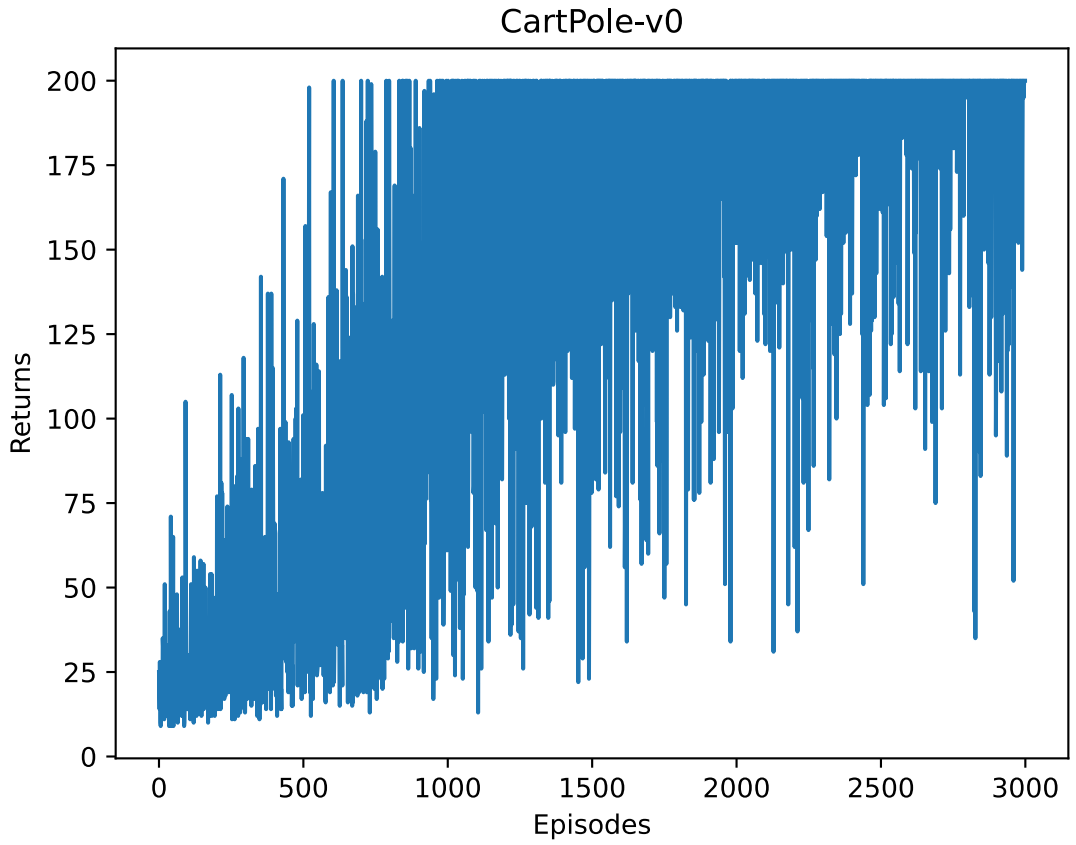


图 4.11 REINFORCE 策略梯度算法获得的奖励

4.2.3 REINFORCE 证明

可以证明：把整条轨迹回报$G(\tau)$换成从时刻t开始的回报$G(t)$，本质上是利用了**因果性**：时刻t的动作不会影响时刻t之前已经发生的奖励。因此，过去奖励那部分在期望下对梯度贡献为0。

将$G(\tau)$拆成过去和未来两部分

对某个时刻t，可以把整条轨迹回报拆成：

$$G(\tau)=\sum_{k=0}^{t-1}\gamma^k R_k+\sum_{k=t}^{T-1}\gamma^k R_k \tag{4.25}$$

记过去回报为

$$G_{<t}=\sum_{k=0}^{t-1}\gamma^k R_k \tag{4.26}$$

未来回报为

$$G_{\geq t}=\sum_{k=t}^{T-1}\gamma^k R_k \tag{4.27}$$

于是有

$$G(\tau)=G_{<t}+G_{\geq t} \tag{4.28}$$

代入梯度中的第t项：

$$\mathbb{E}[\nabla_\theta \log \pi_\theta(A_t|S_t) G(\tau)] \tag{4.29}$$

得到

$$\mathbb{E}[\nabla_\theta \log \pi_\theta(A_t|S_t) G_{<t}] + \mathbb{E}[\nabla_\theta \log \pi_\theta(A_t|S_t) G_{\geq t}] \tag{4.30}$$

关键是证明第一项为 `0`。

证明过去奖励项的期望为 `0`。

注意$G_{<t}$只依赖于时刻t之前发生的事情，也就是历史信息：

$$H_t = (S_0, A_0, R_0, ..., S_t) \tag{4.31}$$

因此$G_{<t}$在给定H_t后是确定的。

考虑条件期望：

$$\mathbb{E}[G_{<t} \nabla_\theta \pi_\theta(A_t|S_t)|H_t] \tag{4.32}$$

因为$G_{<t}$对于H_t已知，可以将$G_{<t}$提到外面。得到

$$G_{<t} \mathbb{E}[\nabla_\theta \pi_\theta(A_t|S_t)|H_t] \tag{4.33}$$

而给定H_t之后，只有A_t由策略$\pi_{\theta(\cdot \mid S_t)}$采样，因此

$$\mathbb{E}[\nabla_\theta \log \pi_\theta(A_t|S_t) \mid H_t] = \sum_a \pi_\theta(a|S_t) \nabla_\theta \log \pi_\theta(a \mid S_t) \tag{4.34}$$

利用 `log` 梯度技巧

$$\pi_\theta(a|S_t) \nabla_\theta \log \pi_\theta(a|S_t) = \nabla_\theta \pi_\theta(a|S_t) \tag{4.35}$$

得到

$$\sum_a \pi_\theta(a|S_t) \nabla_\theta \log \pi_\theta(a|S_t) = \sum_a \nabla_\theta \pi_\theta(a|S_t) \tag{4.36}$$

而

$$\sum_a \pi_\theta(a|S_t) = 1 \tag{4.37}$$

所以

$$\sum_a \nabla_\theta \pi_\theta(a|S_t) = \nabla_\theta \sum_a \pi_\theta(a|S_t) = \nabla_\theta 1 = 0 \tag{4.38}$$

因此

$$\mathbb{E}[\nabla_\theta \log \pi_\theta(A_t|S_t)|H_t] = 0 \tag{4.39}$$

从而

$$\mathbb{E}[G_{<t} \nabla_\theta \log \pi_\theta(A_t|S_t)] = 0 \tag{4.40}$$

这说明：时刻t之前的奖励可以从第t项的回报中删掉，不改变期望梯度。

因此可以替换为“`reward-to-go`”

所以：

$$\mathbb{E}[\nabla_\theta \log \pi_\theta(A_t|S_t) G(\tau)] = \mathbb{E}[\nabla_\theta \log \pi_\theta(A_t|S_t) G_{\geq t}] \tag{4.41}$$

于是整体梯度可以写成：

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T-1}\nabla_\theta \log \pi_\theta(A_t|S_t)\sum_{k=t}^{T-1}\gamma^k R_k\right] \tag{4.42}$$

如果定义从时刻t开始的“reward-to-go”为

$$G_t = \sum_{k=t}^{T-1}\gamma^{k-t}R_k \tag{4.43}$$

那么上式也常写为

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T-1}\gamma^t\nabla_\theta \log \pi_\theta(A_t|S_t)G_t\right] \tag{4.44}$$

很多教材会把折扣因子的位置简化或吸收到G_t的定义中，因此也常见写法：

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T-1}\nabla_\theta \log \pi_\theta(A_t|S_t)G_t\right] \tag{4.45}$$

直观解释

原始形式中，第t个策略梯度项是：

$$\nabla_\theta \log \pi_\theta(A_t|S_t)G(\tau) \tag{4.46}$$

但A_t只会影响：

- 当前奖励；
- 未来状态；
- 未来奖励。

它不可能影响已经发生的过去奖励。因此把过去奖励乘到

$$\nabla_\theta \log \pi_\theta(A_t|S_t) \tag{4.47}$$

上，只会增加噪声，不会改变期望值。

所以用G_t代替$G(\tau)$不会改变梯度的无偏性，但可以降低方差。

结论

REINFORCE 中将整条轨迹回报$G(\tau)$替换为从当前时刻开始的回报$G(t)$是成立的，因为对于任意时刻t，过去奖励部分满足

$$\mathbb{E}\left[\nabla_\theta \log \pi_\theta(A_t|S_t)\sum_{k=0}^{t-1}\gamma^k R_k\right] = 0 \tag{4.48}$$

因此删掉过去奖励不会引入偏差，只是减少了方差。这就是所谓的“reward-to-go trick”或“因果性改进”。

4.3 带基线（baseline）的策略梯度法

前面介绍的 REINFORCE 玩倒立摆有一个问题，那就是不管我们怎么推车，获得的奖励都是正奖励。也就是说不管怎么玩，我们都会提升某个状态采取某个动作的概率，即使奖励很少也是如此。其实我们在玩倒立摆时，获得的奖励很少时，应该降低某个状态下采取某个动作的概率。也就是说，我们应该在获得的奖励很多时，给予策略正的奖励，而在获得奖励很少时，给予策略负的奖励。这样会更加的合理。这就是**基线**的技术。

倒立摆环境中只要倒立摆没有倒，给出的奖励总是 1。也就是不管策略多么的差劲，采样出的轨迹的回报一定是正的。也就是说，即使轨迹的回报很低，也会提升在S_t采取A_t的概率，只是可能提升的不大。

如果加入基线，那么如果轨迹的回报很低，权重就会成为负值，从而在策略更新之后，直接降低在S_t采取A_t的概率。所以训练速度会加快。

下面介绍基线（baseline）技术，该技术可以改进 REINFORCE。让我们先通过一个简单的例子来了解一下基线的思路，然后再将基线应用于 REINFORCE。

下式是 REINFORCE 的数学式。

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} G_t \nabla_\theta \log \pi_\theta(A_t|S_t)\right] \tag{4.49}$$

将基线应用于这个 REINFORCE 的数学式如下式所示。

定理 4.3 — 带基线的 REINFORCE 策略梯度定理.

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} (G_t - b(S_t)) \nabla_\theta \log \pi_\theta(A_t|S_t)\right] \tag{4.50}$$

上式用$G_t - b(S_t)$代替了G_t。这里$b(S_t)$可以是任何函数。也就是说，只要输入是S_t，$b(S_t)$是什么函数都行。这个$b(S_t)$就是基线（baseline）。

上式中的$b(S_t)$可以是任何函数。例如，在状态S_t下，可以考虑使用之前获得的奖励的平均值作为$b(S_t)$。实践中经常使用的是价值函数，数学式为$b(S_t) = V_{\pi_\theta}(S_t)$。如果能够使用基线减小方差，那么就可以进行样本效率更高的训练。另外，将价值函数作为基线使用时，我们是不知道真正的价值函数$v_{\pi_\theta}(S_t)$的。这种情况下还需要训练价值函数神经网络。

最后，我们再通过直观介绍补充说明一下为什么使用基线更好。这里以倒立摆为例，思考下图的状态。

图 4.12 无论怎么推，倒立摆都会在 3 步之后倒下

上图表示游戏结束之前的杆子失去平衡的状态。在这种状态下，无论采取什么样的行动，在几个时间步之后游戏都将结束。

设图的状态为s，在此状态下采取的行动为a。假定从状态s开始几个时间步（比如 3 个时间步）后游戏一定会结束。在这种情况下，状态s的回报G为3（这里设折扣因子γ为1。如果使用的是没有基线的 REINFORCE，那么状态s下的行动a就会因为权重3而被增强（状态s下选择行动a的概率会变高）。但无论采取什么样的行动，3 个时间步之后游戏一定会结束，所以可以说这种提高行动a被选择的概率的工作是无意义的。

此时就要用到基线了。这里使用价值函数作为基线，假设我们已经知道图中的例子中的$V_{\pi_\theta}(S_t) = 3$。此时的权重为$G_t - V_{\pi_\theta}$所以是 0。由于权重是 0，因此无论选择什么行动，采取那个行动的概率都不会变大，也不会变小。像这样使用基线，有望减少无谓的训练。

4.4 Actor-Critic（演员-评论家架构）

如果在上节介绍的带基线的 REINFORCE 中使用价值函数作为基线，那么就可以将其视为基于价值且基于策略的方法。本节将进一步改进带基线的 REINFORCE，推导一个叫作 Actor-Critic 的算法。Actor-Critic 也是基于价值且基于策略的方法。

图 4.13 演员评论家架构

4.4.1 Actor-Critic 的数学推导

首先从复习带基线的 REINFORCE 开始。带基线的 REINFORCE 的目标函数的梯度的数学式如下所示。

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T}(G_t - b(S_t))\nabla_\theta \log \pi_\theta(A_t|S_t)\right] \tag{4.51}$$

上式中的G_t表示回报，$b(S_t)$表示基线。可以使用任何函数作为基线。这里我们使用基于神经网络建模的价值函数作为基线。因此，我们要用到以下这些新的记号。

- ω：表示价值函数的神经网络的所有权重参数。
- $V_\omega(S_t)$：将价值函数模型化的神经网络。

此时目标函数的梯度的数学式如下式所示

定理 4.4 — Actor–Critic 架构的 REINFORCE 策略梯度定理.

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T}(G_t - V_\omega(S_t))\nabla_\theta \log \pi_\theta(A_t|S_t)\right] \tag{4.52}$$

上式中存在一个问题，即只要没抵达目标，就无法确定回报G_t的值。也就是说，在抵达目标之前，无法更新策略和价值函数。如果这是基于蒙特卡洛方法的算法，那么二者无法更新就是它的缺点。

> 蒙特卡洛采样
>
> 我们之前使用的是**蒙特卡洛采样**，也就是采样出一条完整的轨迹。然后更新策略神经网络π_θ。

消除这个缺点的方法是**时序-差分方法**（TD 方法，Time Difference Method）。使用 TD 方法训练价值函数时，使用 1 个时间步（或n个时间步）后的结果就能进行更新，如图所示。

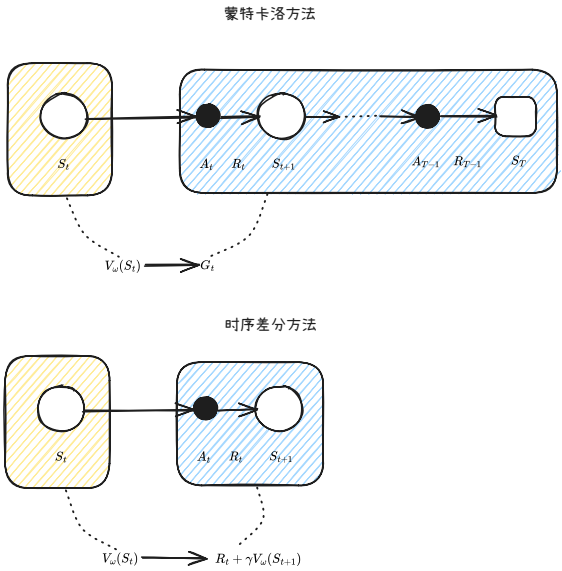


图 4.14 蒙特卡洛方法和 TD 方法对比

如图所示，在训练价值函数$V_\omega(S_t)$时，蒙特卡洛方法使用的是回报G_t，而 TD 方法使用的是$R_t+\gamma V_\omega(S_{t+1})$ 。

🔥 时序差分误差

使用神经网络对价值函数建模时，我们以接近$R_t+\gamma V_\omega(S_{t+1})$为目标训练$V_\omega(S_t)$的值。具体来说就是将$V_\omega(S_t)$和$R_t+\gamma V_\omega(S_{t+1})$的均方差作为损失函数，通过梯度下降法更新神经网络的权重。

贝尔曼期望方程：$V_\omega(S_t=s)=\mathbb{E}_\pi[G_t|S_t=s]=\mathbb{E}_\pi[R_t+\gamma V_\omega(S_{t+1}=s')\mid S_t=s]$

如果我们的价值函数是真实的价值函数的话，那么用$R_t+\gamma V_\omega(S_{t+1})$替换掉$G_t$是**无偏的**。两者区别在于：

- G_t是真实的未来回报。
- $R_t+\gamma V_\omega(S_{t+1})$是用价值函数来估计未来的回报。

如果V是真实的价值函数，那么在期望意义下，$V_\omega(S_t=s)=\mathbb{E}_\pi[G_t|S_t=s]=\mathbb{E}_\pi[R_t+\gamma V_\omega(S_{t+1}=s')\mid S_t=s]$。

而我们的V_ω是一个神经网络，显然不是真实的价值函数，所以不能保证是无偏估计。所以希望随着训练，能够逼近真实的价值函数。但也有可能偏差越来越大！

可以看到$V_\omega(S_t)\to R_t+\gamma V_\omega(S_{t+1})$是由贝尔曼期望方程而来的。

下面将基于蒙特卡洛方法的公式切换为 TD 方法，其中，代替G_t的是$R_t+\gamma V_\omega(S_{t+1})$。此时得到的式子如下所示。

定理 4.5 — 1 步 TD 误差的 Actor–Critic 策略梯度定理.

$$\nabla_\theta J(\theta)=\mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T}(R_t+\gamma V_\omega(S_{t+1})-V_\omega(S_t))\nabla_\theta\log\pi_\theta(A_t|S_t)\right] \tag{4.53}$$

1 步 TD 误差经常表示为$\delta=R_t+\gamma V_\omega(S_{t+1})-V_\omega(S_t)$。

基于上面的公式的算法就是Actor-Critic。策略π_θ和价值函数V_ω是神经网络，我们要同时训练这两个神经网络。具体来说，对于策略π_θ要基于上面的公式进行训练；而对于价值函数V_ω，则通过TD方法，以接近$R_t+\gamma V_\omega(S_{t+1})$为目标训练$V_\omega(S_t)$这个神经网络。

> 提示
>
> Actor-Critic中的Actor是**演员**（采取动作的人）的意思，也就是采取动作的人，相当于策略π_θ。而Critic是**评论家**的意思，相当于价值函数V_ω。因此，Actor-Critic的意思是“使用V_ω，来评论基于策略π_θ采取的动作的好坏”。

> 为什么可以用$R_t+\gamma V_\omega(S_{t+1})$替换掉$G_t$？
>
> 首先我们知道$G_t=R_t+\gamma G_{t+1}$以及$V_\omega(S_{t+1})=\mathbb{E}[G_{t+1}]$。这里我们假设训练到理想情况，$V_\omega(S_{t+1})$接近于真实的$V(S_{t+1})$。所以我们做了替换。

4.4.2 Actic-Critic的代码实现

下面实现Actor-Critic。策略和价值函数这两个神经网络的代码如下所示。

策略网络和价值网络 Python

```python
class PolicyNet(nn.Module):
    def __init__(self, action_size):
        super().__init__()
        self.l1 = nn.Linear(4, 128)
        self.l2 = nn.Linear(128, action_size)

    def forward(self, x):
        x = F.relu(self.l1(x))
        x = F.softmax(self.l2(x), dim=1)
        return x

class ValueNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.l1 = nn.Linear(4, 128)
        self.l2 = nn.Linear(128, 1)

    def forward(self, x):
        x = F.relu(self.l1(x))
        x = self.l2(x)
        return x
```

上面的代码将策略的网络实现为了PolicyNet类，将价值函数的网络实现为了ValueNet类。策略的最终输出是Softmax函数的输出，所以输出的是概率。接下来是Agent类的代码。

演员评论家架构中的智能体 Python

```python
class Agent:
    def __init__(self):
        self.gamma = 0.98 # 折扣因子γ
        self.lr_pi = 0.0002 # 策略网络的学习率α_actor
        self.lr_v = 0.0005 # 价值网络的学习率α_critic
```

```
6           self.action_size = 2
7           # 初始化策略网络（演员网络）π_θ
8           self.pi = PolicyNet(self.action_size)
9           # 初始化价值函数网络（评论家网络）V_ω
10          self.v = ValueNet()
11
12          self.optimizer_pi = optim.Adam(self.pi.parameters(), lr=self.lr_pi)
13          self.optimizer_v = optim.Adam(self.v.parameters(), lr=self.lr_v)
14
15      def get_action(self, state):
16          probs = self.pi(torch.tensor(state).unsqueeze(0)).squeeze(0)
17          m = Categorical(probs)
18          action = m.sample().item()
19          return action, probs
20
21      def update(
22          self,
23          state, # S_t
24          action_prob, # π_θ(A_t|S_t)
25          reward, # R_t
26          next_state, # S_{t+1}
27          done
28      ):
29          state = torch.tensor(state).unsqueeze(0) # S_t
30          next_state = torch.tensor(next_state).unsqueeze(0) # S_{t+1}
31
32          # ① self.v的损失：均方差
33          # TD目标 = R_t + γV_ω(S_{t+1})
34          target = reward + self.gamma * self.v(next_state) * (1 - done)
35          # 从计算图中剥离，不参与反向传播
36          target.detach()
37          v = self.v(state) # V_ω(S_t)
38          loss_fn = nn.MSELoss() # (R_t + γV_ω(S_{t+1}) - V_ω(S_t))^2 → 0
39          loss_v = loss_fn(v, target)
40
41          # ② self.pi的损失
42          delta = target - v # δ = R_t + γV_ω(S_{t+1}) - V_ω(S_t)
43          # -(R_t + γV_ω(S_{t+1}) - V_ω(S_t)) log π_θ(A_t|S_t)
44          loss_pi = -torch.log(action_prob) * delta.detach().item()
45
46          self.optimizer_v.zero_grad()
47          self.optimizer_pi.zero_grad()
48          loss_v.backward() # ∇_ω(R_t + γV_{ω(S_{t+1})} - V_ω(S_t))^2
49          loss_pi.backward() # -(R_t + γV_{ω(S_{t+1})} - V_ω(S_t))∇_θ log π_θ(A_t|S_t)
50          self.optimizer_v.step()
51          self.optimizer_pi.step()
```

`get_action` 方法可以基于策略取出动作。需要注意的是，由于输入到神经网络中的数据将作为小批量进行处理，因此在处理其中一个数据（状态）时需要小批量的轴。另外，`get_action` 方法返回了两个值，即选择的动作及其概率。选择动作的概率将在稍后的损失函数计算中使用。

update方法可以训练价值函数和策略。在代码①处为价值函数(self.v)计算损失。为此要计算TD目标(target)，求出当前状态下其与价值函数(v)的均方差。然后，在代码②处为策略(self.pi)计算损失。需要将其乘以−1的值作为损失。剩下的就是一直以来的神经网络的训练代码。

训练循环 Python

```python
env = gym.make("CartPole-v0")
agent = Agent()
return_list = []
episode_list = []

for episode in range(2000):
    state = env.reset()
    done = False
    total_reward = 0

    while not done:
        action, probs = agent.get_action(state)
        next_state, reward, done, info = env.step(action)
        # 执行完一个动作，立即更新$\pi_\theta$和$V_\omega$这两个网络
        agent.update(state, probs[action], reward, next_state, done)

        state = next_state
        total_reward += reward

    return_list.append(total_reward)
    episode_list.append(episode)
    if episode % 100 == 0:
        print("回合:{}, 总奖励:{:.1f}".format(episode, total_reward))

plot_loss(episode_list, return_list, "actor-critic-pg-loss.pdf")
test_agent(agent, env)
```

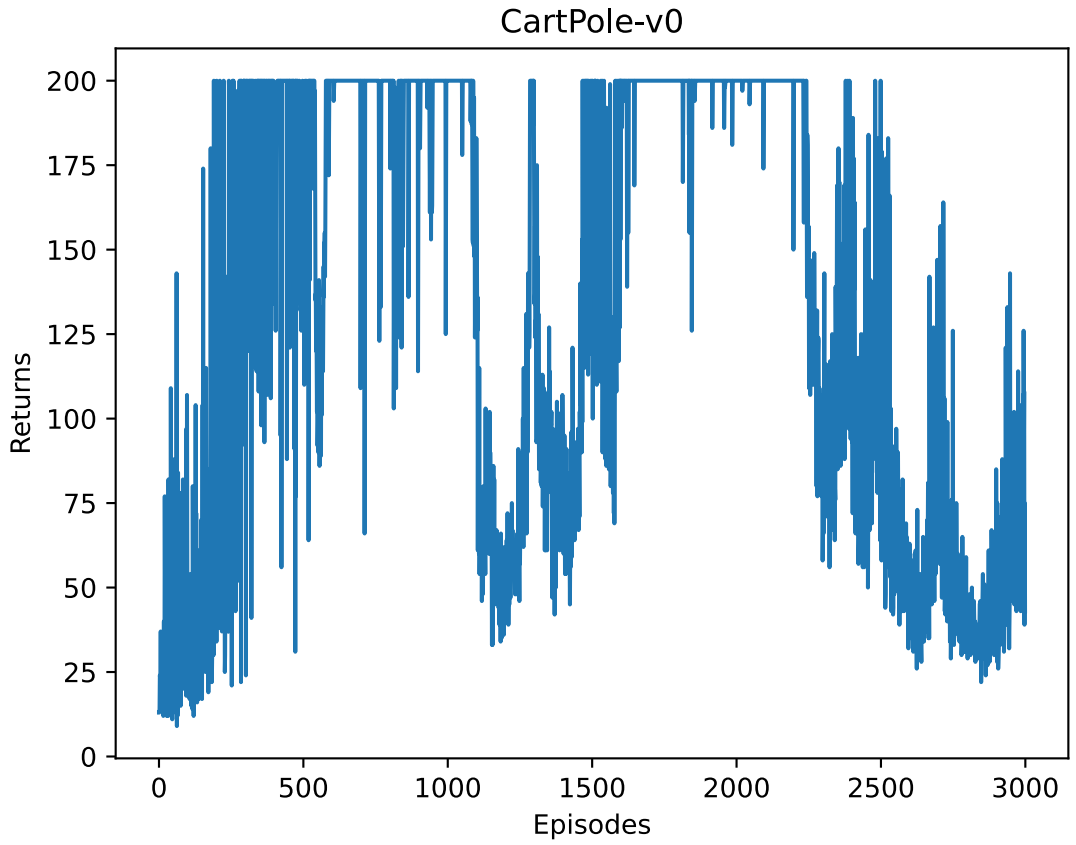


图 4.15 actor-critic 每一轮的回报走势图

4.5 多步 TD 误差

每个时刻的 1 步 TD 误差计算如下：

$$\begin{aligned}\delta_t &= R_t + \gamma V(S_{t+1}) - V(S_t)\\ \delta_{t+1} &= R_{t+1} + \gamma V(S_{t+2}) - V(S_{t+1})\\ \delta_{t+2} &= R_{t+2} + \gamma V(S_{t+3}) - V(S_{t+2})\\ \delta_{t+3} &= R_{t+3} + \gamma V(S_{t+4}) - V(S_{t+3})\\ &\cdots\end{aligned} \tag{4.54}$$

1 步 TD 误差为：

$$\begin{aligned}A_t^{(1)} &= R_t + \gamma V(S_{t+1}) - V(S_t)\\ &= -V(S_t) + R_t + \gamma V(S_{t+1})\\ &= \delta_t\end{aligned} \tag{4.55}$$

> 贝尔曼期望方程
>
> $$V(S_{t+1}) = \mathbb{E}[R_{t+1} + \gamma V(S_{t+2})] \tag{4.56}$$

将式子中的$V(S_{t+1})$由$R_{t+1} + \gamma V(S_{t+2})$替换，得到如下式子，就是 2 步 TD 误差。

$$
\begin{aligned}
A_t^{(2)} &= -V(S_t) + R_t + \gamma(R_{t+1} + \gamma V(S_{t+2})) \\
&= -V(S_t) + R_t + \gamma R_{t+1} + \gamma^2 V(S_{t+2}) \\
&= \underbrace{R_t + \gamma V(S_{t+1}) - V(S_t)}_{\delta_t} + \gamma \left(\underbrace{R_{t+1} + \gamma V(S_{t+2}) - V(S_{t+1})}_{\delta_{t+1}} \right) \\
&= \delta_t + \gamma \delta_{t+1}
\end{aligned} \tag{4.57}
$$

4.6 广义优势估计

> 提示
>
> 广义优势估计：Generalized Advantage Estimation，GAE

上面的 1 步 TD 误差表示为：$\delta = R_t + \gamma V_\omega(S_{t+1}) - V_\omega(S_t)$。在很多强化学习文献中，这个叫做优势 A（Advantage）。

也就是 1 步 TD 目标$R_t + \gamma V_\omega(S_{t+1})$相对于基线$V_\omega(S_t)$的优势是多少。

我们在上一节的 Actor-Critic 中使用 1 步 TD 误差来作为优势。而现在常用的一种方法是针对**多步 TD 误差**计算指数加权平均的方式来估算优势，也就是**广义优势估计**。

首先，t时刻的 1 步 TD 误差是：

$$
\delta_t = R_t + \gamma V(s_{t+1}) - V(s_t) \tag{4.58}
$$

其中V是价值函数神经网络V_ω。那么我们计算一下多步 TD 误差：

$$
\begin{aligned}
A_t^{(1)} &= \delta_t && = -V(s_t) + R_t + \gamma V(s_{t+1}) \\
A_t^{(2)} &= \delta_t + \gamma \delta_{t+1} && = -V(s_t) + R_t + \gamma R_{t+1} + \gamma^2 V(s_{t+2}) \\
A_t^{(3)} &= \delta_t + \gamma \delta_{t+1} + \gamma^2 \delta_{t+2} && = -V(s_t) + R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 V(s_{t+3}) \\
&\vdots && \vdots \\
A_t^{(k)} &= \sum_{l=0}^{k-1} \gamma^l \delta_{t+l} && = -V(s_t) + R_t + \gamma R_{t+1} + \cdots + \gamma^{k-1} R_{t+k-1} + \gamma^k V(s_{t+k})
\end{aligned} \tag{4.59}
$$

然后，GAE 将这些不同步数的优势估计进行指数加权平均：

$$
\begin{aligned}
A_t^{\mathrm{GAE}} &= (1-\lambda)\left(A_t^{(1)} + \lambda A_t^{(2)} + \lambda^2 A_t^{(3)} + \cdots\right) \\
&= (1-\lambda)(\delta_t + \lambda(\delta_t + \gamma\delta_{t+1}) + \lambda^2(\delta_t + \gamma\delta_{t+1} + \gamma^2\delta_{t+2}) + \cdots) \\
&= (1-\lambda)(\delta_t(1 + \lambda + \lambda^2 + \cdots) + \gamma\delta_{t+1}(\lambda + \lambda^2 + \lambda^3 + \cdots) + \gamma^2\delta_{t+2}(\lambda^2 + \lambda^3 + \lambda^4 + \cdots) + \cdots) \\
&= (1-\lambda)\left(\delta_t \frac{1}{1-\lambda} + \gamma\delta_{t+1}\frac{\lambda}{1-\lambda} + \gamma^2\delta_{t+2}\frac{\lambda^2}{1-\lambda} + \cdots\right) \\
&= \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}
\end{aligned} \tag{4.60}
$$

其中，$\lambda \in [0,1]$是在 GAE 中额外引入的一个超参数。当$\lambda = 0$时，$A_t^{\mathrm{GAE}} = \delta_t = R_t + \gamma V(s_{t+1}) - V(s_t)$，也就是仅仅只看一步差分得到的优势；当$\lambda = 1$时，$A_t^{\mathrm{GAE}} = \sum_{l=0}^{\infty} \gamma^l \delta_{t+l} = \sum_{l=0}^{\infty} \gamma^l R_{t+l} - V(s_t)$，则是看每一步差分得到的优势的完全平均值。

有上面的式子，我们还可以推导出一个递推公式

$$
A_t^{\mathrm{GAE}} = \delta_t + \gamma\lambda A_{t+1}^{\mathrm{GAE}} \tag{4.61}
$$

下面是计算 GAE 的过程，给定 γ 和 λ 以及每个时间步的 δ_t 之后，我们可以根据公式直接进行优势估计。

先前向（forward）计算每一个时刻的 1 步 TD 误差，然后再逆向（backward）计算每一个时刻的 GAE。

首先，我们有 n 个 1 步 TD 误差。

$$\mathrm{TD}_\delta = [\delta_t, \delta_{t+1}, \cdots, \delta_{t+n}] \tag{4.62}$$

先把数组逆序

$$\mathrm{TD}_\delta = [\delta_{t+n}, \delta_{t+n-1}, \cdots, \delta_t] \tag{4.63}$$

然后遍历逆序数组，有如下结果：

$$\begin{aligned} A_{t+n}^{\mathrm{GAE}} &= \delta_{t+n} \\ A_{t+n-1}^{\mathrm{GAE}} &= \delta_{t+n-1} + \gamma\lambda A_{t+n}^{\mathrm{GAE}} \\ A_{t+n-2}^{\mathrm{GAE}} &= \delta_{t+n-2} + \gamma\lambda A_{t+n-1}^{\mathrm{GAE}} \\ &\vdots \\ A_t^{\mathrm{GAE}} &= \delta_t + \gamma\lambda A_{t+1}^{\mathrm{GAE}} \end{aligned} \tag{4.64}$$

这样每一个时间步的广义优势估计就计算出来了。

GAE 目标

$$\text{GAE Target} = A_t^{\mathrm{GAE}} + V(S_t)$$

定理 4.6 — GAE 的 Actor–Critic 策略梯度定理.

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} A_t^{\mathrm{GAE}} \nabla_\theta \log \pi_\theta(a_t|s_t)\right] \tag{4.65}$$

提示

为了使符号更清晰，将动作A_t改为a_t，将状态S_t改为s_t。A_t^{GAE}中的A表示“Advantage”（优势）。

4.7 相关数学证明

4.7.1 策略梯度法的证明

当$J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[G(\tau)]$时，其梯度如下面的式子所示。

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} G(\tau) \nabla_\theta \log \pi_\theta(A_t|S_t)\right] \tag{4.66}$$

下面对上面的式子进行证明。

$$
\begin{aligned}
\nabla_\theta J(\theta) &= \nabla_\theta \mathbb{E}_{\tau\sim\pi_\theta}[G(\tau)] \\
&= \nabla_\theta \sum_\tau \Pr(\tau|\theta) G(\tau) \text{（展开期望值）} \\
&= \sum_\tau \nabla_\theta (\Pr(\tau|\theta) G(\tau)) \text{（将}\nabla_\theta\text{移动到}\textstyle\sum\text{中）} \\
&= \sum_\tau \{G(\tau)\nabla_\theta \Pr(\tau|\theta) + \Pr(\tau|\theta)\nabla_\theta G(\tau)\} \text{（积的微分）} \\
&= \sum_\tau G(\tau)\nabla_\theta \Pr(\tau|\theta) \text{（}\nabla_\theta G(\tau)\text{永远为 0）} \\
&= \sum_\tau G(\tau)\Pr(\tau|\theta)\frac{\nabla_\theta \Pr(\tau|\theta)}{\Pr(\tau|\theta)} \text{（乘以}\frac{\Pr(\tau|\theta)}{\Pr(\tau|\theta)}\text{）} \\
&= \sum_\tau G(\tau)\Pr(\tau|\theta)\nabla_\theta \log \Pr(\tau|\theta) \text{（log 梯度技巧）} \\
&= \mathbb{E}_{\tau\sim\pi_\theta}[G(\tau)\nabla_\theta \log \Pr(\tau|\theta)]
\end{aligned}
\tag{4.67}
$$

这里对“log 梯度的技巧”进行说明。这个技巧利用了以下等式。

$$
\nabla_\theta \log \Pr(\tau|\theta) = \frac{\nabla_\theta \Pr(\tau|\theta)}{\Pr(\tau|\theta)} \tag{4.68}
$$

> **log 梯度技巧**
>
> $$
> (\log f(x))' = \frac{f'(x)}{f(x)} \tag{4.69}
> $$

根据上面的式子，我们就知道

$$
\nabla_\theta \Pr(\tau|\theta) = \Pr(\tau|\theta)\nabla_\theta \log \Pr(\tau|\theta) \tag{4.70}
$$

这就是著名的 **log 梯度的技巧**。是机器学习领域常用的数学式的变形形式。

接下来，我们将利用以下等式进一步展开式 (4.67)。

$$
\begin{aligned}
\Pr(\tau|\theta) &= p(S_0)\pi_\theta(A_0|S_0)p(S_1|S_0,A_0)\cdots\pi_\theta(A_T|S_T)p(S_{T+1}|S_T,A_T) \\
&= p(S_0)\prod_{t=0}^{T}\pi_\theta(A_t|S_t)p(S_{t+1}|S_t,A_t)
\end{aligned}
\tag{4.71}
$$

🔥 轨迹的概率

轨迹是：$\tau=(S_0,A_0,S_1,A_1,S_2,A_2,S_3)$，那么轨迹的概率如下计算：

$$\begin{aligned}
\Pr(\tau) &= p(S_0,A_0,S_1,A_1,S_2,A_2,S_3)\\
&= p(S_3|S_0,A_0,S_1,A_1,S_2,A_2)p(S_0,A_0,S_1,A_1,S_2,A_2)\\
&= p(S_3|S_2,A_2)p(S_0,A_0,S_1,A_1,S_2,A_2)\\
&= p(S_3|S_2,A_2)p(A_2|S_0,A_0,S_1,A_1,S_2)p(S_0,A_0,S_1,A_1,S_2)\\
&= p(S_3|S_2,A_2)p(A_2|S_2)p(S_0,A_0,S_1,A_1,S_2)\\
&= p(S_3|S_2,A_2)\pi_\theta(A_2|S_2)p(S_0,A_0,S_1,A_1,S_2)\\
&= p(S_3|S_2,A_2)\pi_\theta(A_2|S_2)p(S_2|S_0,A_0,S_1,A_1)p(S_0,A_0,S_1,A_1)\\
&= p(S_3|S_2,A_2)\pi_\theta(A_2|S_2)p(S_2|S_1,A_1)p(S_0,A_0,S_1,A_1)\\
&= p(S_3|S_2,A_2)\pi_\theta(A_2|S_2)p(S_2|S_1,A_1)p(A_1|S_0,A_0,S_1)p(S_0,A_0,S_1)\\
&= p(S_3|S_2,A_2)\pi_\theta(A_2|S_2)p(S_2|S_1,A_1)p(A_1|S_1)p(S_0,A_0,S_1)\\
&= p(S_3|S_2,A_2)\pi_\theta(A_2|S_2)p(S_2|S_1,A_1)\pi_\theta(A_1|S_1)p(S_0,A_0,S_1)\\
&= p(S_3|S_2,A_2)\pi_\theta(A_2|S_2)p(S_2|S_1,A_1)\pi_\theta(A_1|S_1)p(S_1|S_0,A_0)p(S_0,A_0)\\
&= p(S_3|S_2,A_2)\pi_\theta(A_2|S_2)p(S_2|S_1,A_1)\pi_\theta(A_1|S_1)p(S_1|S_0,A_0)p(A_0|S_0)p(S_0)\\
&= p(S_3|S_2,A_2)\pi_\theta(A_2|S_2)p(S_2|S_1,A_1)\pi_\theta(A_1|S_1)p(S_1|S_0,A_0)\pi_\theta(A_0|S_0)p(S_0)
\end{aligned} \tag{4.72}$$

这里，$p(S_0)$表示初始状态S_0的概率。上面的式子表明，得到轨迹τ的概率可以用初始状态的概率、策略以及下一个状态的转移概率的乘积来表示。另外，我们可以用下面的式子来表示$\log\Pr(\tau|\theta)$。

$$\log\Pr(\tau|\theta)=\log p(S_0)+\sum_{t=0}^{T}\log p(S_{t+1}|S_t,A_t)+\sum_{t=0}^{T}\log\pi_\theta(A_t|S_t) \tag{4.73}$$

由于$\log xy=\log x+\log y$，所以可以像上面的式子那样表示为和的形式。基于上面的式子，可以将$\nabla_\theta\log\Pr(\tau|\theta)$展开为如下形式。

$$\begin{aligned}
\nabla_\theta\log\Pr(\tau|\theta) &= \nabla_\theta\left\{\log p(S_0)+\sum_{t=0}^{T}\log p(S_{t+1}|S_t,A_t)+\sum_{t=0}^{T}\log\pi_\theta(A_t|S_t)\right\}\\
&= \nabla_\theta\sum_{t=0}^{T}\log\pi_\theta(A_t|S_t)
\end{aligned} \tag{4.74}$$

∇_θ是对θ的梯度。与θ无关的元素的梯度$\nabla_\theta\log p(S_0)$和$\nabla_\theta\sum_{t=0}^{T}\log p(S_{t+1}|S_t,A_t)\}$为`0`。因此，从上面的式子可以得到下列式子。

$$\begin{aligned}
\nabla_\theta J(\theta) &= \mathbb{E}_{\tau\sim\pi_\theta}[G(\tau)\nabla_\theta\log\Pr(\tau|\theta)]\\
&= \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T}G(\tau)\nabla_\theta\log\pi_\theta(A_t|S_t)\right]
\end{aligned} \tag{4.75}$$

这样我们就完成了$\nabla_\theta J(\theta)$的推导。

从极大似然估计的角度看策略梯度

$$\begin{aligned} J(\theta) &= \sum_{t=0}^{T} G(\tau) \log \pi_\theta(A_t|S_t) \\ &= G(\tau) \log \left(\prod_{t=0}^{T} \pi_\theta(A_t|S_t) \right) \end{aligned} \tag{4.76}$$

最大化目标就会提升某些状态对应的动作的概率。

4.7.2 基线（Baseline）的推导

$$\begin{aligned} \nabla_\theta J(\theta) &= \mathbb{E}_{\tau\sim\pi_\theta} \left[\sum_{t=0}^{T} G_t \nabla_\theta \log \pi_\theta(A_t|S_t) \right] \\ &= \mathbb{E}_{\tau\sim\pi_\theta} \left[\sum_{t=0}^{T} (G_t - b(S_t)) \nabla_\theta \log \pi_\theta(A_t|S_t) \right] \\ &= \mathbb{E}_{\tau\sim\pi_\theta} \left[\sum_{t=0}^{T} G_t \nabla_\theta \log \pi_\theta(A_t|S_t) \right] - \underbrace{\mathbb{E}_{\tau\sim\pi_\theta} \left[\sum_{t=0}^{T} b(S_t) \nabla_\theta \log \pi_\theta(A_t|S_t) \right]}_{\text{证明这一项等于 0 即可}} \\ &= \mathbb{E}_{\tau\sim\pi_\theta} \left[\sum_{t=0}^{T} G_t \nabla_\theta \log \pi_\theta(A_t|S_t) \right] \end{aligned} \tag{4.77}$$

如上面的式子所示，我们可以使用$G_t - b(S_t)$代替G_t。$b(S_t)$是**任何函数**，我们称之为“基线”。下面进行上式的推导。

首先，证明以下式子成立。

$$\mathbb{E}_{x\sim P_\theta}[\nabla_\theta \log P_\theta(x)] = 0 \tag{4.78}$$

这里假设随机变量x是基于概率分布$P_\theta(x)$生成的。$P_\theta(x)$会根据参数θ改变概率分布的形状。此时有以下式子成立。

$$\sum_x P_\theta(x) = 1 \tag{4.79}$$

由于$P_\theta(x)$是概率分布，因此所有x的值的和为 1。然后，求这个式子的梯度。

$$\nabla_\theta \sum_x P_\theta(x) = \nabla_\theta 1 = 0 \tag{4.80}$$

接下来，使用 log 梯度的技巧将式子展开，过程如下所示。

$$\begin{aligned} 0 &= \nabla_\theta \sum_x P_\theta(x) \\ &= \sum_x \nabla_\theta P_\theta(x) \\ &= \sum_x P_\theta(x) \nabla_\theta \log P_\theta(x) \quad \text{（log 梯度技巧）} \\ &= \mathbb{E}_{x\sim P_\theta}[\nabla_\theta \log P_\theta(x)] \end{aligned} \tag{4.81}$$

式 (4.78) 证明完毕。接下来将证明的式子用于我们的问题。具体来说，用A_t代替式 (4.78) 中的x，然后使用$\pi_\theta(\cdot|S_t)$代替$P_\theta(\cdot)$。这样就可以得到以下式子。

$$\mathbb{E}_{A_t\sim\pi_\theta}[\nabla_\theta \log \pi_\theta(A_t|S_t)] = 0 \tag{4.82}$$

上面的式子是对A_t的期望值。因此，我们可以像下面的式子那样，将任何函数$b(S_t)$放入期望值中。$E[x]=0 \to E[cx]=c\cdot 0=0$。

$$\mathbb{E}_{A_t\sim\pi_\theta}[b(S_t)\nabla_\theta \log \pi_\theta(A_t|S_t)]=0 \tag{4.83}$$

$b(S_t)$是以S_t为参数的函数，即使A_t发生变化，它的值也不会改变。由于式（4.83）是对A_t的期望值，因此即使在期望值中加入函数$b(S_t)$，等式也成立。

> ⚡ 危险
>
> 动作A_t的变化会导致收益G_t的变化，因此以下式子不成立。
>
> $$\mathbb{E}_{A_t\sim\pi_\theta}[G_t\nabla_\theta \log \pi_\theta(A_t|S_t)]=0 \tag{4.84}$$

式（4.83）在整个$t=0\sim T$的范围都成立，所以可以得到以下式子。

$$\mathbb{E}_{A_t\sim\pi_\theta}\left[\sum_{t=0}^{T} b(S_t)\nabla_\theta \log \pi_\theta(A_t|S_t)\right]=0 \tag{4.85}$$

所以基线证明完毕。

4.8 总结

在本章中，我们学习了基于策略的方法——策略梯度法。具体来说，我们学习了几种策略梯度法的算法。它们的统一的数学式如下所示。

$$\nabla_\theta J(\theta)=\mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T}\Phi_t\nabla_\theta \log \pi_\theta(A_t|S_t)\right] \tag{4.86}$$

$$\begin{aligned}
&1.\quad \Phi_t=G(\tau) && \text{（最简单的策略梯度法）}\\
&2.\quad \Phi_t=G_t && \text{（REINFORCE）}\\
&3.\quad \Phi_t=G_t-b(S_t) && \text{（带基线的 REINFORCE）}\\
&4.\quad \Phi_t=R_t+\gamma V(S_{t+1})-V(S_t) && \text{（Actor-Critic）}\\
&5.\quad \Phi_t=A_t^{\mathrm{GAE}} && \text{（广义优势估计）}
\end{aligned} \tag{4.87}$$

上面几种方法的权重Φ_t各不相同。最简单的策略梯度法在所有时刻的权重都是$G(\tau)$。对它进行改进，将时刻t的收益G_t作为权重进行评估的是 REINFORCE。另外，通过加入“基线”这一方法，减小了方差的方法是带基线的 REINFORCE。Actor-Critic 是一种除了策略之外，对价值函数也用神经网络建模的方法。期待(1) → (2) → (3) → (4) → (5)这样的编号一直延续下去，从而有更高级的方法出现，产生更好的结果。

> 🔥 优势函数
>
> Φ_t也叫做优势函数（Advantage Function），优势函数经常在文献中表示为$A(s,a)$，也就是在状态s采取动作a时，策略的优势是多少。
>
> 强化学习中的很多花活儿都是针对如何改进Φ_t而来的。比如 GRPO 算法使用的**组相对优势**。

5. 近端策略优化（PPO）

> 提示
>
> PPO: Proximal Policy Optimization，近端策略优化

5.1 策略梯度法存在的问题

回顾一下策略梯度法的公式。

$$\begin{aligned}\nabla_\theta J(\theta) &= \nabla_\theta \mathbb{E}_{\tau\sim\pi_\theta}[G(\tau)] \\ &= \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} G(\tau)\nabla_\theta \log \pi_\theta(A_t|S_t)\right]\end{aligned} \tag{5.1}$$

还有梯度上升算法

$$\theta \leftarrow \theta + \alpha\nabla_\theta J(\theta) \tag{5.2}$$

步子迈得太大会导致灾难。但如果步子太小，模型学习速度就会太慢。把奖励函数想象成高耸的山峰。如果新策略走得太远，它采取的行动可能会差之毫厘，最终从悬崖上掉下来。当我们重新开始探索时，我们从一个表现不佳的状态开始，采用局部糟糕的策略。性能会崩溃，而且需要很长时间才能恢复。

图 5.1 梯度上升算法中步幅太大，策略直接崩溃了

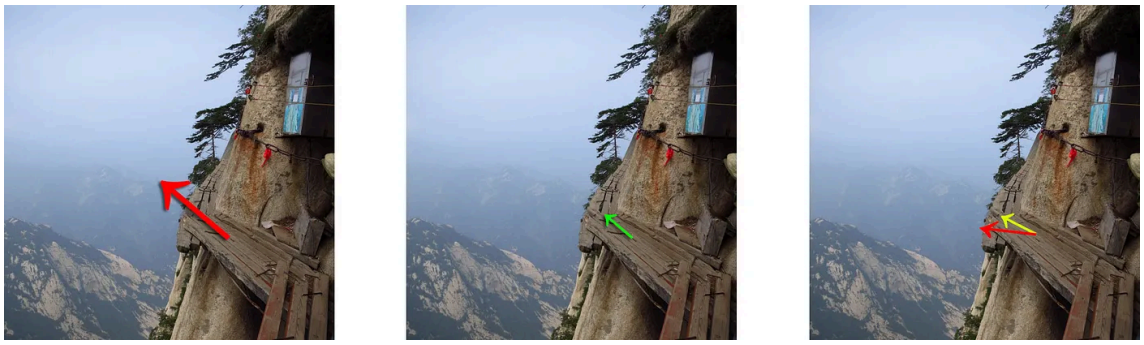


图 5.2 左图是步幅太大，中间的图是理想的步幅，右边的图是方向不对

🔥 为什么在深度学习中这个问题没有那么严重呢？

深度学习中，使用的训练数据集是固定不变的。换句话说，不管神经网络的参数怎么变，训练数据集是不会变的。也就是说，神经网络的参数和训练数据集没关系。

而强化学习中，训练数据是每一轮根据策略来采样的新的轨迹。而采样的轨迹的质量好坏无法控制。也就是说训练数据集和策略神经网络的参数是有关系的。这就麻烦了。

在强化学习中很难找到合适的学习率。假设学习率是专门针对上图黄点调整的。该区域相对平坦，因此为了获得良好的学习速度，学习率应该高于平均值。但是，一步走错，我们就会从悬崖上掉到红点。红点处的梯度很高，当前的学习率会触发爆炸式策略更新。由于学习率对地形不敏感，策略梯度算法的收敛问题非常严重。

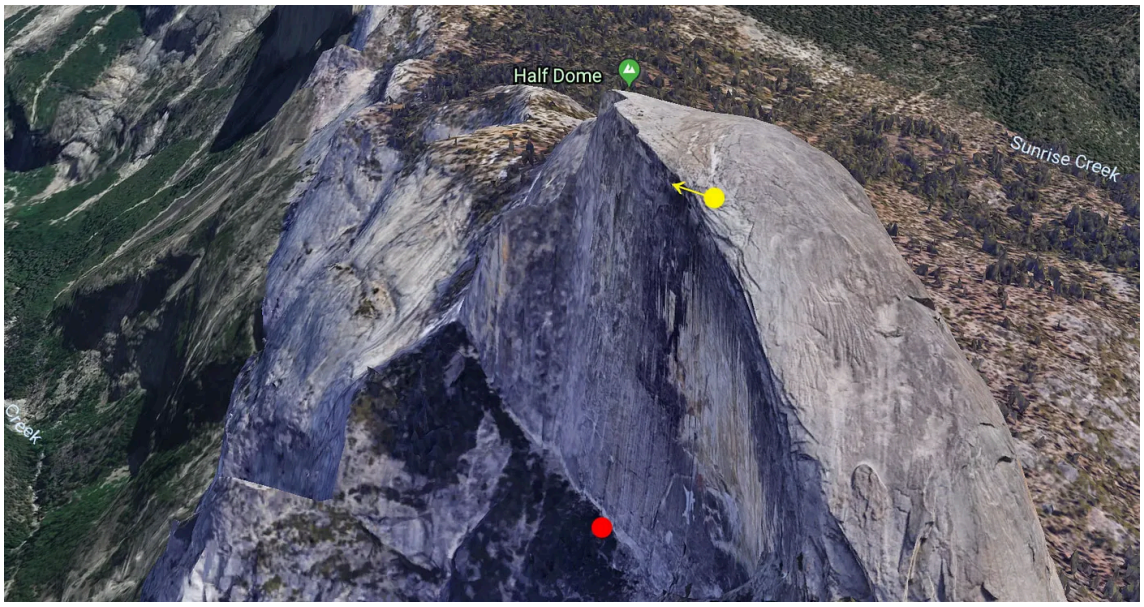


图 5.3 梯度爆炸更新

⚡ 策略梯度法存在的问题是训练出来的新策略可能比旧策略还要差

能否找到一个算法，使得训练出来的新策略一定比旧策略好呢？也就是能否保证下面的式子成立呢？

$$J(\theta_{\text{new}}) - J(\theta_{\text{old}}) \geq 0 \tag{5.3}$$

5.2 近端策略优化

在最优化理论中，优化方法主要有两种：**线搜索**和**置信域**。梯度下降算法是一种线搜索。我们首先确定下降方向，然后朝该方向迈出一步。

🔥 提示

`Trust Region Method:` 置信域方法

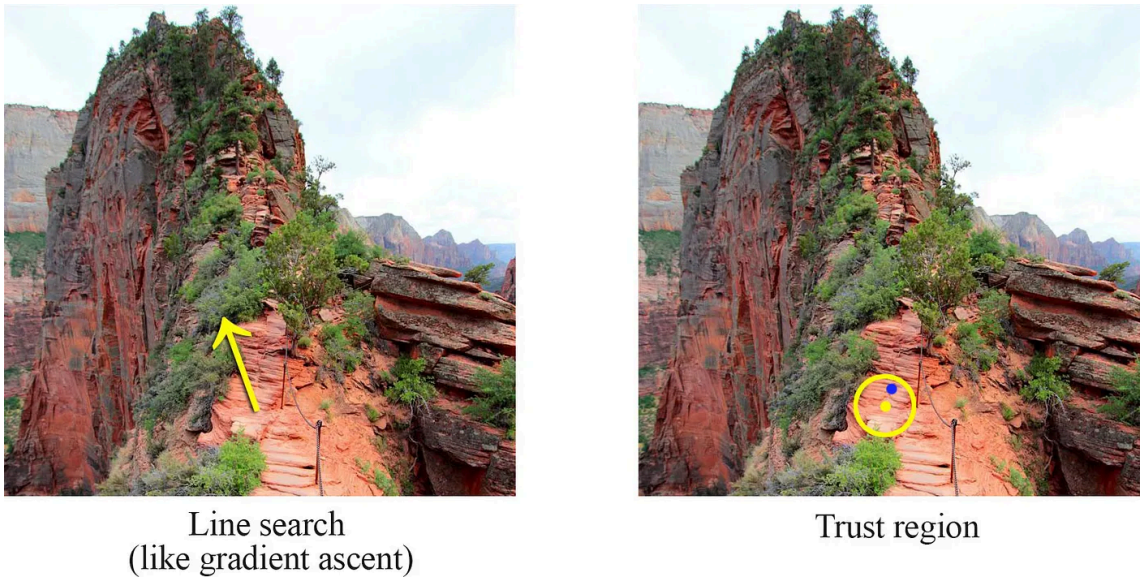


图 5.4 左图为线搜索，右图为置信域

而在置信域中，我们确定想要探索的最大步长，然后在该置信域内找到最优点。

图 5.5 置信域方法

也就是，我们在梯度上升的时候，先周围看一圈，然后找一个安全的步长，然后进行梯度上升。

为了实现这个目的，我们的梯度公式$\nabla_\theta J(\theta)$就要改变了。

PPO 算法是 Actor-Critic 架构的，也就是演员-评论家架构。

所以我们先来回顾一下策略梯度法的演员-评论家架构的梯度公式。

策略梯度法的目标函数如下

$$J(\theta)=\mathbb{E}_{\tau\sim\pi_\theta}[G(\tau)] \tag{5.4}$$

策略梯度法的目标函数的梯度如下

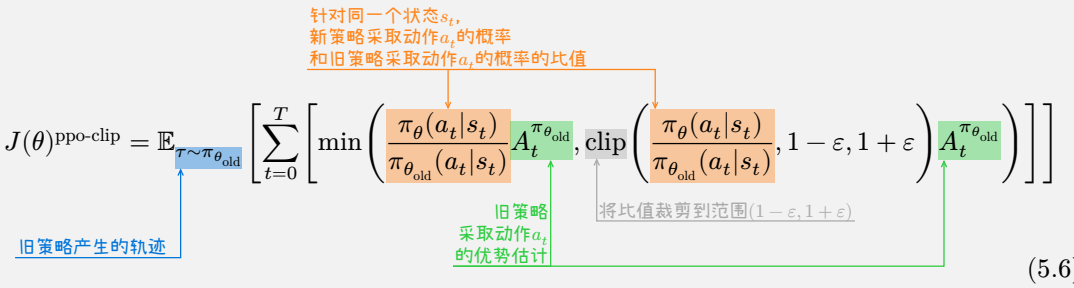
$$\nabla_\theta J(\theta)=\mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T}A_t^{\pi_\theta}\nabla_\theta\log\pi_\theta(a_t|s_t)\right] \tag{5.5}$$

其中$A_t^{\pi_\theta}$为估计的优势。这里的A表示优势，也就是`Advantage`。表示当前策略采取动作a_t之后，相对于价值函数评估的价值的优势是多少。

> ⚡ 危险
>
> 为了不和前面的公式混淆，我们将动作的数学符号由A_t改为a_t，将状态的数学符号由S_t改为s_t。这样A就代表优势了。

而 PPO 的目标函数变成了下式，也就是**替代目标函数**（`surrogate objection function`）。

定理 5.1 — 带裁剪的 PPO 目标函数.



(5.6)

只要计算出这个目标函数，然后进行反向传播（`.backward()`也就是求$\nabla_\theta J(\theta)^{\text{ppo-clip}}$）就可以更新策略神经网络了。

当我们运行旧策略$\pi_{\theta_{\text{old}}}$玩一局倒立摆游戏时，会产生一条轨迹$\tau=(s_0,a_0,R_0,s_1,a_1,R_1,\cdots)$。将这条轨迹保存下来，我们可以使用新策略$\pi_\theta$来计算在每个状态$s_t$，新策略采取动作$a_t$的概率$\pi_\theta(a_t|s_t)$是多少。

例如：当我们使用旧策略玩了一把倒立摆游戏之后，保存的轨迹中

$$\pi_{\theta_{\text{old}}}(a_1|s_1)=[\textbf{向左推的概率}=0.3,\textbf{向右推的概率}=0.7] \tag{5.7}$$

那么新策略也可以针对s_1计算一下采取动作的概率分布。

危险

在训练最开始时，新策略和旧策略相同，所以比值为$\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}=1$，也就是这个比值不会被裁剪。

所以在训练最开始时，也就是 PPO 的目标函数第一次进行反向传播时，有如下成立：

$$J(\theta)^{\text{ppo-clip}}=\mathbb{E}_{\tau\sim\pi_{\theta_{\text{old}}}}\left[\sum_{t=0}^{T}\frac{\pi_\theta(a_t|s_t)|_{\theta=\theta_{\text{old}}}}{\pi_{\theta_{\text{old}}}(a_t|s_t)}A_t^{\pi_{\theta_{\text{old}}}}\right] \tag{5.8}$$

对两边进行求导，注意这里求导时$\theta=\theta_{\text{old}}$。所以有如下：

$$\begin{aligned}\nabla_\theta J(\theta)^{\text{ppo-clip}}&=\nabla_\theta\mathbb{E}_{\tau\sim\pi_{\theta_{\text{old}}}}\left[\sum_{t=0}^{T}\frac{\pi_\theta(a_t|s_t)|_{\theta=\theta_{\text{old}}}}{\pi_{\theta_{\text{old}}}(a_t|s_t)}A_t^{\pi_{\theta_{\text{old}}}}\right]\\&=\mathbb{E}_{\tau\sim\pi_{\theta_{\text{old}}}}\left[\sum_{t=0}^{T}\frac{\nabla_\theta\pi_\theta(a_t|s_t)|_{\theta=\theta_{\text{old}}}}{\pi_{\theta_{\text{old}}}(a_t|s_t)}A_t^{\pi_{\theta_{\text{old}}}}\right]\ \text{(log 梯度技巧)}\\&=\mathbb{E}_{\tau\sim\pi_{\theta_{\text{old}}}}\left[\sum_{t=0}^{T}A_t^{\pi_{\theta_{\text{old}}}}\nabla_\theta\log\pi_\theta(a_t|s_t)\right]\end{aligned} \tag{5.9}$$

我们会发现，PPO 的第一次反向传播，等价于原始的策略梯度法。

如果新策略与旧策略的概率比超出$(1-\varepsilon)$和$(1+\varepsilon)$的范围，则优势将被剪裁。在 PPO 论文中的实验中，ε设置为0.2。

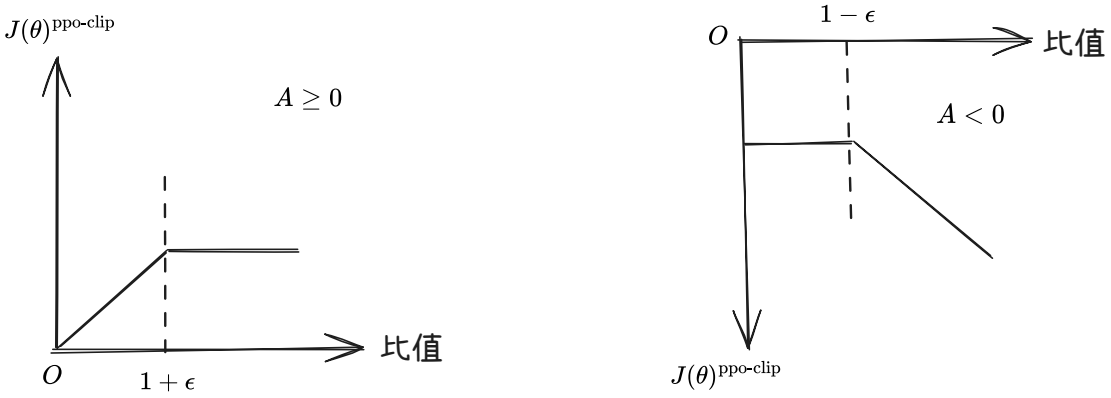


图 5.6 PPO 目标函数中的比值

实际上，如果策略发生重大变化超出了我们的舒适区，那么这将阻碍策略的实现。

我们假设比率

$$p_t(\theta)=\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \tag{5.10}$$

	$p_t(\theta) > 0$	A_t	min 函数的结果	目标函数是否被裁剪?	目标函数的符号	梯度
1	$p_t(\theta) \in [1-\varepsilon, 1+\varepsilon]$	+	$p_t(\theta)A_t$	否	+	✓
2	$p_t(\theta) \in [1-\varepsilon, 1+\varepsilon]$	-	$p_t(\theta)A_t$	否	-	✓
3	$p_t(\theta) < 1-\varepsilon$	+	$p_t(\theta)A_t$	否	+	✓
4	$p_t(\theta) < 1-\varepsilon$	-	$(1-\varepsilon)A_t$	是	-	0
5	$p_t(\theta) > 1+\varepsilon$	+	$(1+\varepsilon)A_t$	是	+	0
6	$p_t(\theta) > 1+\varepsilon$	-	$p_t(\theta)A_t$	否	-	✓

表 5.1 对 PPO 目标函数根据比值的不同进行分情况讨论

我们有 6 种不同的情况，见上面的表。首先记住，我们取裁剪目标和非裁剪目标中的最小值。

- **情况 1 和 2：比率介于裁剪范围之内**

在情况 1 和 2 中，由于比例介于范围$[1-\varepsilon, 1+\varepsilon]$之间，因此目标函数不会被裁剪。

在情况 1 中，我们具有正优势：该动作优于该状态下所有动作的平均值。因此，我们应该鼓励当前策略提高在该状态下采取该动作的概率。

由于该比率是在区间之间的，因此我们可以增加我们的策略在该状态下采取该行动的概率。

在情况 2 中，我们有一个负优势：该动作比该状态下所有动作的平均值更差。因此，我们应该阻止当前策略在该状态下采取该动作。

由于该比率是在区间之间的，因此我们可以降低我们的策略在该状态下采取该行动的概率。

- **情况 3 和 4：比率小于$1-\varepsilon$**

如果概率比低于$1-\varepsilon$，则当前策略在该状态下采取该行动的概率比旧策略低得多。

如果像情况 3 中一样，优势估计为正（$A > 0$），那么我们希望增加在该状态下采取该行动的概率。

但是，如果像情况 4 那样，优势估计为负，我们不想进一步降低在该状态下采取该行动的概率。因此，**梯度** $= 0$（因为我们在一条平线上），所以我们不会更新权重。

- **情况 5 和 6：比率大于$1+\varepsilon$**

如果概率比高于$1+\varepsilon$，则当前策略中该状态下采取该行动的概率远高于前一策略。

如果像情况 5 那样，优势为正，我们不应该太贪心。因为在当前状态下，我们采取该行动的概率已经比之前的策略更高了。因此，梯度 = 0（因为我们在一条平线上），所以我们不会更新权重。

如果像情况 6 中那样，优势是负的，我们希望降低在该状态下采取该行动的概率。

总结一下，我们只用未裁剪的目标函数部分来更新策略。当最小值是裁剪后的目标函数部分时，我们不会更新策略权重，因为梯度将等于 0。

因此，我们仅在以下情况下更新我们的策略：

- 比率在 $(1-\varepsilon, 1+\varepsilon)$ 之内
- 比率不在 $(1-\varepsilon, 1+\varepsilon)$ 范围内，但优势使我们更接近这个范围。
  - 比率小于 $1-\varepsilon$ ，但优势 > 0 。
  - 比率大于 $1+\varepsilon$ ，但优势 < 0 。

你可能会想，为什么当最小值是截断比率时，梯度为 0。当比率被截断时，在这种情况下的导数将不是 $p_t(\theta)A_t$ 的导数。而是 $(1-\varepsilon)A_t$ 或者 $(1+\varepsilon)A_t$ 的导数，而两者的导数都是 0。

> 提示
>
> A_t不是θ的函数。所以对θ求导为 0。

总而言之，得益于这个裁剪的替代目标函数（替换了原来的简单优美的目标函数（$J(\theta)=\mathbb{E}_{\tau\sim\pi_\theta}[G(\tau)]$），我们限制了当前策略与旧策略之间的差异范围。因为我们消除了比率超出区间的诱因，因为裁剪会对梯度产生影响。如果比率为$>1+\varepsilon$或$<1-\varepsilon$，则梯度将等于0。

PPO 增加了一个软约束（裁剪机制），可以通过一阶优化器（求一阶导数）进行优化。我们偶尔可能会做出一些错误的决策，但它在优化速度上取得了良好的平衡。实验结果证明，这种平衡能够以最简单的方式实现最佳性能。

带裁剪的 PPO 算法

输入：初始化策略参数θ_0
初始化裁剪阈值参数$\varepsilon=0.2$
for $k=0,1,2,\cdots$ do
    使用策略π_{θ_k}采集一条轨迹数据$\mathcal{D}_k$
    计算优势$A_t^{\pi_{\theta_k}}$ ▷ 使用广义优势估计（GAE）
    更新策略 ▷ 执行 N 步的梯度下降
    $\theta_1'=\theta_k$ ▷ 将θ_k作为梯度更新的起点
    for $i=1,2,\cdots,N$ do ▷ 更新 N 次梯度
        $p=\frac{\pi_{\theta_i'}(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)}$ ▷ 计算比值
        $\text{clipped}=\text{clip}(p,1-\varepsilon,1+\varepsilon)$ ▷ 裁剪比值
        $J(\theta_i')=\sum_{t=0}^{T}\left[\min\left(p\cdot A_t^{\pi_{\theta_k}},\text{clipped}\cdot A_t^{\pi_{\theta_k}}\right)\right]$ ▷ 累加所有时间步的最小值
        $\theta_{i+1}'=\theta_i'+\alpha\nabla_{\theta_i'}J(\theta_i')$ ▷ 更新梯度
    $\theta_{k+1}=\theta_N'$ ▷ 将θ_N'作为下一次更新的起点

图 5.7 带裁剪的 PPO 算法伪代码

6. PPO 实战

6.1 网络结构的定义

- 演员网络：策略网络
- 评论家网络：价值函数网络

策略网络和价值函数网络的定义 Python

```python
class PolicyNet(nn.Module):
    def __init__(self, action_size):
        super().__init__()
        self.l1 = nn.Linear(4, 128)
        self.l2 = nn.Linear(128, action_size)

    def forward(self, x):
        x = F.relu(self.l1(x))
        x = F.softmax(self.l2(x), dim=1)
        return x

class ValueNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.l1 = nn.Linear(4, 128)
        self.l2 = nn.Linear(128, 1)

    def forward(self, x):
        x = F.relu(self.l1(x))
        x = self.l2(x)
        return x
```

6.2 PPO 算法的实现

我们来实现 Agent，代码如下：

```python
class Agent:
```

```
    def __init__(self):
        self.gamma = 0.98
        self.lr_pi = 0.001
        self.lr_v = 0.02
        self.action_size = 2

        self.pi = PolicyNet(self.action_size)
        self.v = ValueNet()

        self.optimizer_pi = optim.Adam(self.pi.parameters(), lr=self.lr_pi)
        self.optimizer_v = optim.Adam(self.v.parameters(), lr=self.lr_v)

    def get_action(self, state):
        probs = self.pi(torch.tensor(state).unsqueeze(0)).squeeze(0)
        m = Categorical(probs)
        action = m.sample().item()
        return action, probs

    def collect_trajectory(self, env):
        """采集一条轨迹"""
        state = env.reset()
        states, next_states, actions, action_probs, rewards, dones = [], [], [],
        [], [], []
        done = False

        while not done:
            action, probs = self.get_action(state)
            next_state, reward, done, _ = env.step(action)
            states.append(state) # s_t
            actions.append(action) # a_t
            action_probs.append(probs[action]) # π(a_t|s_t)
            next_states.append(next_state) # s_{t+1}
            rewards.append(reward) # R_t
            dones.append(done) # done_t
            state = next_state # 状态转移

        return states, next_states, actions, action_probs, rewards, dones
```

接下来我们要实现智能体的更新（update）代码，在 PPO 算法中，我们需要用到旧策略采样的轨迹。所以需要保存旧策略采样的轨迹的完整信息。

我们采样的轨迹是：

$$\tau = [s_0, a_0, R_0, s_1, a_1, R_1, \cdots, s_T, a_T, R_T] \tag{6.1}$$

那么我们使用元组来保存一条轨迹的信息。结构如下

$$\text{一条轨迹的详细信息} = \begin{cases} \text{states} : [s_0, s_1, ..., s_{T-1}] \\ \text{next_states} : [s_1, s_2, ..., s_T] \\ \text{actions} : [a_0, a_1, ..., a_{T-1}] \\ \text{action_probs} : \left[\pi_{\theta_{\text{old}}}(a_0|s_0), \pi_{\theta_{\text{old}}}(a_1|s_1), ..., \pi_{\theta_{\text{old}}}(a_{T-1}|s_{T-1})\right] \\ \text{rewards} : [R_0, R_1, ..., R_{T-1}] \\ \text{dones} : [\text{False}, \text{False}, ..., \text{True}] \end{cases} \tag{6.2}$$

有了旧策略采样的轨迹的所有详细信息，我们就可以来实现 PPO 算法了。代码如下。

```python
class Agent:
    ...

    def update(self, trajectory):
        states, next_states, actions, action_probs, rewards, dones = trajectory
        # [s_0, s_1, ..., s_{T-1}]
        states = torch.tensor(states)
        # [a_0, a_1, ..., a_{T-1}]
        actions = torch.tensor(actions).view(-1, 1)
        # [R_0, R_1, ..., R_{T-1}]
        rewards = torch.tensor(rewards).view(-1, 1)
        # [s_1, s_2, ..., s_T]
        next_states = torch.tensor(next_states)
        # [False_1, False_2, ..., True_T]
        dones = torch.tensor(dones, dtype=torch.float).view(-1, 1)
        # [V(s_0), V(s_1), ..., V(s_{T-1})]
        V = self.v(states).detach()
        # TD-target_t = R_t + γV(s_{t+1})
        # [TD-target_0, TD-target_1, ..., TD-target_{T-1}]
        td_target = rewards + self.gamma * self.v(next_states) * (1 - dones)
        # 一步TD误差：δ_t = R_t + γV(s_{t+1}) - V(s_t)
        # [δ_0, δ_1, ..., δ_{T-1}]
        td_delta = td_target - V
        # 计算每个时刻t的广义优势估计（GAE）
        # [A_0^GAE, A_1^GAE, ..., A_{T-1}^GAE]
        gae = compute_gae(self.gamma, td_delta.cpu())
        # 冻结一份旧策略采取动作的对数概率log π_{θ_old}(a_t|s_t)
        # [log π_{θ_old}(a_0|s_0), log π_{θ_old}(a_1|s_1), ..., log π_{θ_old}(a_{T-1}|s_{T-1})]
        old_probs = torch.tensor(action_probs).view(-1, 1)
        old_log_probs = torch.log(old_probs).detach()
        # 每条轨迹使用10次，也就是更新10次策略网络
        for _ in range(10):
            # 新策略采取动作的对数概率：log π_θ(a_t|s_t)
            # [log π_θ(a_0|s_0), log π_θ(a_1|s_1), ..., log π_θ(a_{T-1}|s_{T-1})]
            log_probs = torch.log(self.pi(states).gather(1, actions))
            # 计算比率：p_t(θ) = π_θ(a_t|s_t) / π_{θ_old}(a_t|s_t)
            ratio = torch.exp(log_probs - old_log_probs)
            # p_t(θ) A_t^{π_{θ_old}}
            # [p_0 A_0^{π_{θ_old}}, p_1 A_1^{π_{θ_old}}, ..., p_{T-1} A_{T-1}^{π_{θ_old}}]
            surr1 = ratio * gae
            # clip(p_t(θ), 1-ε, 1+ε) A_t^{π_{θ_old}} |_{ε=0.2}
            # [clip(p_0) A_0^GAE, clip(p_1) A_1^GAE, ..., clip(p_{T-1}) A_{T-1}^GAE]
            surr2 = torch.clamp(ratio, 1 - 0.2, 1 + 0.2) * gae
            # π_loss = -(1/T) Σ_{t=0}^{T-1} min(surr1_t, surr2_t)
            pi_loss = torch.mean(-torch.min(surr1, surr2))
            # V_loss = (1/T) Σ_{t=0}^{T-1} (V(s_t) - (A_t^GAE + V(s_t)))
            v_loss = torch.mean(F.mse_loss(self.v(states), gae + V))
            self.optimizer_pi.zero_grad()
```

```
49              self.optimizer_v.zero_grad()
50              pi_loss.backward()
51              v_loss.backward()
52              self.optimizer_pi.step()
53              self.optimizer_v.step()
```

比值的计算方法

$$\begin{aligned} &\because A = e^{\log A} \\ &\therefore \frac{A}{B} = e^{\log A - \log B} = \frac{e^{\log A}}{e^{\log B}} \end{aligned} \tag{6.3}$$

6.3 广义优势估计的计算

Python

```
1  def compute_gae(gamma, td_delta):
2      # δ_t
3      td_delta = td_delta.detach().numpy()
4      gae_list = []
5      last_gae = 0.0
6      lmbda = 0.95
7      # A_t = δ_t + γλA_{t+1}
8      for delta in td_delta[::-1]:
9          last_gae = gamma * lmbda * last_gae + delta
10         gae_list.append(last_gae)
11     gae_list.reverse()
12     return torch.tensor(gae_list)
```

6.4 训练循环

Python

```
1  def train(env, agent):
2      return_list = []
3      episode_list = []
4      for episode in range(500):
5          trajectory = agent.collect_trajectory(env)
6          agent.update(trajectory)
7          # 统计信息
8          episode_reward = sum(trajectory[4])
9          return_list.append(episode_reward)
10         episode_list.append(episode)
11
12         if (episode + 1) % 10 == 0:
13             print(f"回合: {episode}, 回报: {episode_reward}")
14
15     return return_list, episode_list
16
17 def main():
18     env = gym.make("CartPole-v0")
19     env.seed(0)
20     torch.manual_seed(0)
```

```
21      agent = Agent()
22      return_list, episode_list = train(env, agent)
23      plot_loss(episode_list, return_list, "ppo-loss.pdf")
24      test_agent(agent, env)
25
26  if __name__ == "__main__":
27      main()
```

获得的奖励可视化如下：

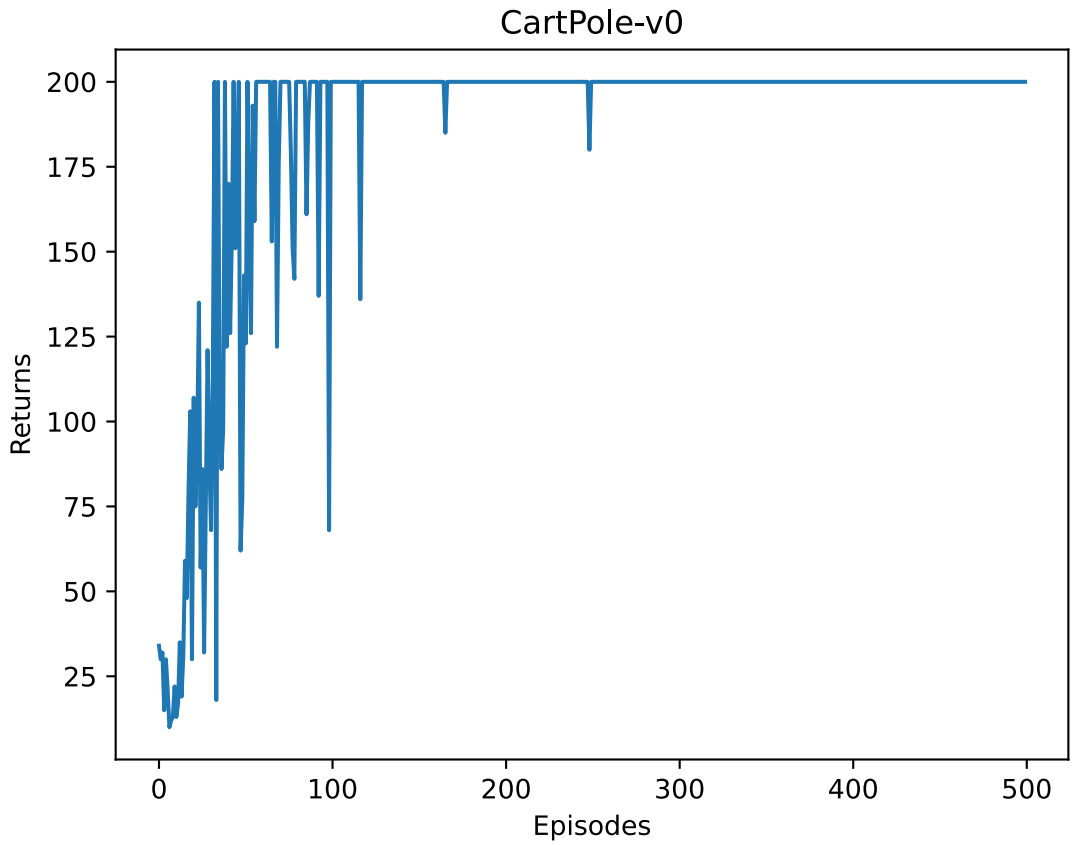


图 6.1 PPO 算法获得的奖励

7. PPO 背后的数学

在使用策略梯度算法训练智能体时，一个挑战是它们容易出现性能崩溃：智能体会突然表现很差。这种情况很难恢复，因为智能体会开始生成质量较差的轨迹，而这些轨迹又被用于进一步训练策略。我们还看到，on-policy（同策略）算法的样本效率较低，因为它们无法重用数据。

Schulman 等人提出的近端策略优化（Proximal Policy Optimization, PPO）是一类优化算法，用来解决这两个问题。PPO 的核心思想是在目标函数中引入一个代理（surrogate）目标，它通过保证策略的单调改进来避免性能崩溃。该目标还有一个好处，就是可以在训练过程中重用离策略（off-policy）数据。

PPO 可以通过将原始的目标函数$J(\theta)$替换为修改后的 PPO 目标，来扩展 REINFORCE 或者 Actor-Critic。此修改带来了更稳定、且样本效率更高的训练过程。

在本章中，我们首先讨论性能崩溃问题。随后通过单调改进理论来应对该问题。我们将该理论应用于策略梯度目标，将其改造为一个代理目标。

在介绍完理论基础后，我们讨论 PPO 算法。

7.1 数学背景知识

7.1.1 KL 散度

KL 散度（Kullback-Leibler Divergence, KLD）非常重要。

假设有一个连续型随机变量x，其概率密度表示为$p(x)$。此时，函数$f(x)$的期望值可以用下面的数学式表示。

$$\mathbb{E}_{p(x)}[f(x)] = \int f(x)p(x)\mathrm{d}x \tag{7.1}$$

关于概率分布$q(x)$的期望值可以用下面的数学式表示。

$$\mathbb{E}_{q(x)}[f(x)] = \int f(x)q(x)\mathrm{d}x \tag{7.2}$$

KL 散度的定义

衡量两个概率分布之间差异的一种方法是 KL 散度。当给定两个概率分布$p(x)$和$q(x)$时，KL 散度可以用下面的数学式表示。

定义 **7.1** — **连续型随机变量的 KL 散度.**

$$D_{\mathrm{KL}}(p \parallel q) = \int p(x) \log \frac{p(x)}{q(x)} \mathrm{d}x \tag{7.3}$$

上面的式子是当x为连续型随机变量时的 KL 散度。当x为离散型随机变量时，数学式如下所示。

定义 **7.2** — **离散型随机变量的 KL 散度.**

$$D_{\mathrm{KL}}(p \parallel q) = \sum_{x} p(x) \log \frac{p(x)}{q(x)} \tag{7.4}$$

KL 散度具有以下特性：

- 两个概率分布的差异越大，KL 散度的值就越大
- KL 散度的值大于或等于 0，且仅当两个概率分布相同时，其值才为 0
- KL 散度是非对称的衡量指标，因此$D_{\mathrm{KL}}(p \parallel q)$和$D_{\mathrm{KL}}(q \parallel p)$的值不同

这些特性使得 KL 散度可以用来衡量两个概率分布的差异程度。下面我们通过具体的例子来了解这些特性。这里以抛硬币为例进行说明。假设一枚硬币正面朝上和反面朝上的概率是确定的，如下表所示。

正面朝上的概率	70%
反面朝上的概率	30%

表 7.1 硬币的真实概率分布

上表是硬币的“真实概率分布”，在这里我们用符号p来表示。假设有人对这枚硬币的概率分布做了如下估计。

正面朝上的概率	50%
反面朝上的概率	50%

表 7.2 第一个人对硬币的概率分布做出的估计

我们用q来表示这个估计的概率分布。此时，“真实概率分布p”和“估计概率分布q”之间的 KL 散度可以做如下计算。

$$\begin{aligned} D_{\mathrm{KL}}(p \parallel q) &= 0.7 \log \frac{0.7}{0.5} + 0.3 \log \frac{0.3}{0.5} \\ &= 0.082\cdots \end{aligned} \tag{7.5}$$

KL 散度约为 0.082。假设另一个人对这枚硬币的概率分布做了如下估计。

正面朝上的概率	20%
反面朝上的概率	80%

表 7.3 第二个人对硬币的概率分布做出的估计

这与真实的概率分布值大相径庭。此时的 KL 散度的值如下所示。

$$
\begin{aligned}
D_{\mathrm{KL}}(p \parallel q) &= 0.7 \log \frac{0.7}{0.2} + 0.3 \log \frac{0.3}{0.8} \\
&= 0.58\cdots
\end{aligned}
\tag{7.6}
$$

这个值比第一个 KL 散度的值要大。最后，假设又有一个人对这枚硬币的概率分布做了如下估计。

正面朝上的概率	70%
反面朝上的概率	30%

表 7.4 第三个人对硬币的概率分布做出的估计

这个分布与真实的概率分布相同。此时的 KL 散度的值如下所示。

$$
\begin{aligned}
D_{\mathrm{KL}} &= 0.7 \log \frac{0.7}{0.7} + 0.3 \frac{0.3}{0.3} \\
&= 0.7 \log 1 + 0.3 \log 1 \quad (\log 1 = 0) \\
&= 0
\end{aligned}
\tag{7.7}
$$

由于p和q是相同的概率分布，因此 KL 散度为 0。

从以上结果可以看出，KL 散度可以用来衡量两个概率分布的差异程度。当两个概率分布相同时，KL 散度取最小值 0。两个概率分布的差异越大，KL 散度的值就越大。

7.1.2 信息论

信息量（Self-Information）：

- 一个事件发生的信息量定义为：$I(x) = -\log P(x)$
- 概率越小的事件，包含的信息量越大
- 例如："太阳从东边升起"（高概率）vs "中彩票"（低概率）

熵（Entropy）：

- 衡量随机变量的不确定性：$H(P) = -\sum_i P(x_i) \log P(x_i)$
- 熵越大，不确定性越大

交叉熵（Cross-Entropy）：

- 衡量两个概率分布之间的差异：$H(P, Q) = -\sum_i P(x_i) \log Q(x_i)$
- 其中 P 是真实分布，Q 是预测分布

极大似然估计和交叉熵

假设我们有训练数据 $\{(x_1, y_1), (x_2, y_2), ..., (x_n, y_n)\}$，其中 y_i 是真实标签。

似然函数：

$$
L(\theta) = \prod_{i=1}^{n} P(y_i \mid x_i; \theta) \tag{7.8}
$$

对数似然：

$$
\log L(\theta) = \sum_{i=1}^{n} \log P(y_i \mid x_i; \theta) \tag{7.9}
$$

最大化对数似然 = 最小化负对数似然：

$$
\text{Loss} = -\frac{1}{n} \sum_{i=1}^{n} \log P(y_i \mid x_i; \theta) \tag{7.10}
$$

这就是**交叉熵损失!**

KL 散度和交叉熵

KL 散度衡量两个分布的差异：

$$\begin{aligned} D_{\mathrm{KL}}(P \parallel Q) &= \sum_i P(x_i) \log \frac{P(x_i)}{Q(x_i)} \\ &= \sum_i P(x_i) \log P(x_i) - \sum_i P(x_i) \log Q(x_i) \end{aligned} \tag{7.11}$$

$$D_{\mathrm{KL}}(P \parallel Q) = -H(P) + H(P, Q) \tag{7.12}$$

$$\frac{\partial D_{\mathrm{KL}}(P \parallel Q)}{\partial \theta} = \frac{\partial H(P, Q)}{\partial \theta} \tag{7.13}$$

其中：

- $H(P) = -\sum_i P(x_i) \log P(x_i)$ 是真实分布的熵（常数）
- $H(P, Q) = -\sum_i P(x_i) \log Q(x_i)$ 是交叉熵

最小化 KL 散度 = 最小化交叉熵（因为真实分布的熵是常数）

7.1.3 重要性采样（Importance Sampling）

7.1.3.1 什么是重要性采样?

重要性采样利用从其他概率分布中采样的数据来计算某个概率分布的期望值。为了说明重要性采样，下面来看一个计算期望值$\mathbb{E}_\pi[x]$的简单例子。设x是随机变量，x的概率用$\pi(x)$表示。期望值的式子如下所示。

$$\mathbb{E}_\pi[x] = \sum x\pi(x) \tag{7.14}$$

我们先复习一下学过的内容。为了用蒙特卡洛方法近似这个期望值，我们从概率分布π对x进行采样，并取其平均值。数学式如下所示：

$$\begin{aligned} &\textbf{采样}: x^{(i)} \sim \pi \quad (i = 1, 2, \cdots, n) \\ &\mathbb{E}_\pi[x] \simeq \frac{x^{(1)} + x^{(2)} + \cdots + x^{(n)}}{n} \end{aligned} \tag{7.15}$$

式子中的符号$x^{(i)} \sim \pi$表示第i个数据$x(i)$是从概率分布π中采样的。

接下来回到正题。我们现在探讨的是从别的概率分布中采样x的场景。假设x是从概率分布b（而不是π）中采样的。在这种情况下，期望值$\mathbb{E}_\pi[x]$如何被近似呢？解决这个问题的关键在于下面这个式子的变形。

$$\begin{aligned} \mathbb{E}_\pi[x] &= \sum x\pi(x) \\ &= \sum x\frac{b(x)}{b(x)}\pi(x) \\ &= \sum x\frac{\pi(x)}{b(x)}b(x) \end{aligned} \tag{7.16}$$

这里的要点是插入$\frac{b(x)}{b(x)}$。$\frac{b(x)}{b(x)}$总是为1，所以等式成立。然后，我们将式子变为$\sum \cdots b(x)$的形式，如上面的式子所示，这样它就可以被视为概率分布$b(x)$的期望值了。实际上对上面的式子变形，可以得到以下式子。

$$\begin{aligned}\mathbb{E}_{\pi}[x] &= \sum x\frac{\pi(x)}{b(x)}b(x) \\ &= \mathbb{E}_{b}\left[x\frac{\pi(x)}{b(x)}\right]\end{aligned} \tag{7.17}$$

这里需要注意的是式子中的$\mathbb{E}_b$，它表示的是概率分布b的期望值。另外每个x都要乘以$\frac{\pi(x)}{b(x)}$这一点也很重要。如果$\rho(x)=\frac{\pi(x)}{b(x)}$，那么我们就可以把式子看作每个$x$都乘以权重$\rho(x)$。根据以上探讨，基于式的蒙特卡洛方法如下所示。

$$\begin{gathered}\text{采样}: x^{(i)} \sim b \quad (i=0,1,\cdots,n) \\ \mathbb{E}_{\pi}[x] \simeq \frac{\rho(x^{(1)})x^{(1)}+\rho(x^{(2)})x^{(2)}+\cdots+\rho(x^{(n)})x^{(n)}}{n}\end{gathered} \tag{7.18}$$

这样我们就使用从不同于π的概率分布b中采样的数据计算出了$\mathbb{E}_{\pi}[x]$。下面来实现重要性采样的代码 。这里我们对下图所示的概率分布进行重要性采样。

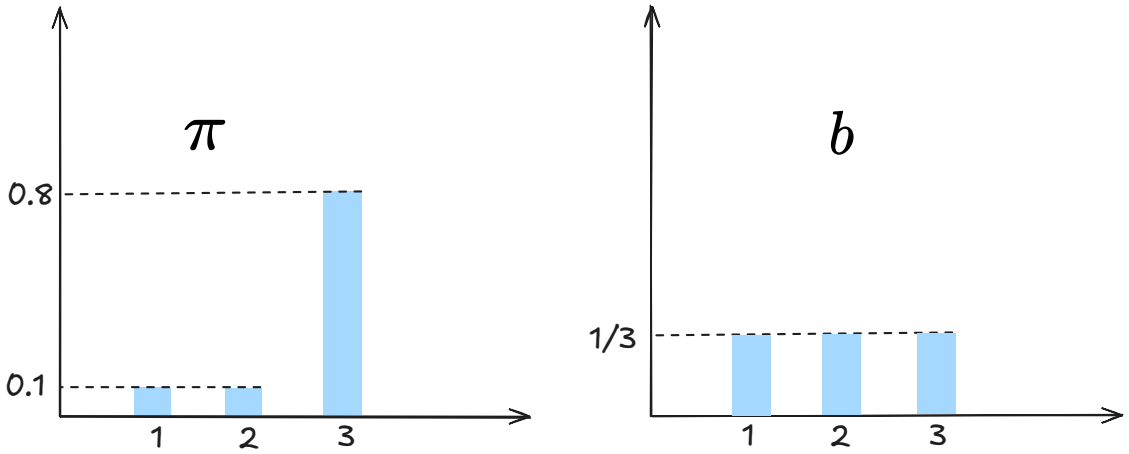


图 7.1 概率分布π和b

这里的目标是求出期望值$\mathbb{E}_{\pi}[x]$。我们先尝试用普通的蒙特卡洛方法求概率分布π的期望值。代码如下所示。

用普通的蒙特卡洛方法求概率分布π的期望值 Python

```python
import numpy as np

x = np.array([1, 2, 3])
pi = np.array([0.1, 0.1, 0.8])

# 期望值E_pi[x]
e = np.sum(x * pi)
print("E_pi[x]", e) # 输出: E_pi[x] 2.7

# 蒙特卡洛方法
n = 100
samples = []
for _ in range(n):
    s = np.random.choice(x, p=pi) # 使用pi进行采样
    samples.append(s)
```

```
16
17 mean = np.mean(samples)
18 var = np.var(samples)
19 print("MC: {:.2f} (var: {:.2f})".format(mean, var)) # 输出：MC: 2.78 (var: 0.27)
```

首先，套用定义式求期望值。求出的结果是2.7（这是真实的值）。然后，使用蒙特卡洛方法来求这个值。这里只基于概率分布 pi 采样 100 个数据，然后求平均值。为此，我们用 NumPy 的 np.mean 方法求出了平均值。结果是2.78，接近于真实值。作为参考，我们还使用 NumPy 的 np.var 方法求出了方差。方差的值是0.27。在与下一次重要性采样的结果进行比较时，这个值将被用作参考。方差用于表示数据的离散程度。期望值和方差之间的关系可以用以下式子表示。

$$\mathrm{Var}[X] = \mathbb{E}[(X - \mathbb{E}[X])^2] \tag{7.19}$$

方差是数据X和X的平均值$\mathbb{E}[X]$之间的差的平方的期望值。直观地说，它代表了数据的离散程度，如下图所示。

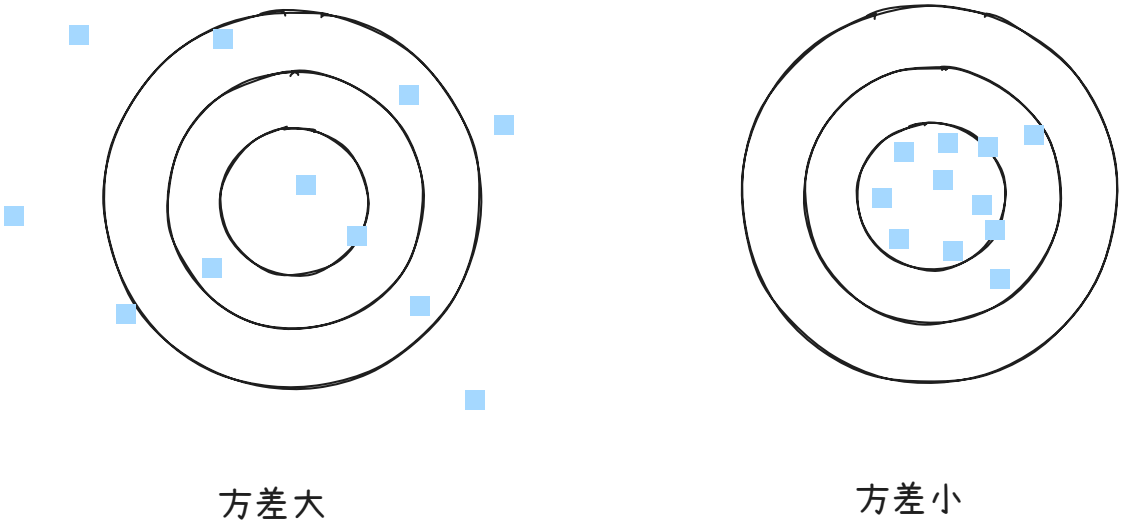


图 7.2 将每个数据作为二维平面上的点时的方差的示意图（圆心是平均值）

下面尝试用重要性采样的方法来求期望值。代码如下所示。

用重要性采样的方法来求期望值 Python

```
1  b = np.array([1/3, 1/3, 1/3])
2  n = 100
3  samples = []
4
5  for _ in range(n):
6      idx = np.arange(len(b)) # [0, 1, 2]
7      i = np.random.choice(idx, p=b) # 使用 b 进行采样
8      s = x[i]
9      rho = pi[i] / b[i] # ρ
10     samples.append(rho * s)
11
12 mean = np.mean(samples)
13 var = np.var(samples)
14 print("IS: {:.2f} (var: {:.2f})".format(mean, var)) # 输出：IS: 2.95 (var: 10.63)
```

这里使用概率分布 b 进行采样，不过进行采样的目标是 b 的索引（[0, 1, 2]）。这是因为在计算权重 rho 时，要用到的是采样得到的索引。

接下来看看上面的结果。平均值是2.95，虽然与真实值2.7有一些差距，但也还算接近。此外，方差是10.63，这说明数据的离散程度大于蒙特卡洛方法的结果（使用蒙特卡洛方法时的方差是0.27）。

7.1.3.2 如何减小方差

方差越小，用较少的样本得到的近似值就越准确。反之，方差越大，越需要更多的样本才能得到准确的近似值。下面介绍如何用重要性采样来减小方差。首先，我们结合下图来了解一下为什么重要性采样方法的方差会比较大。

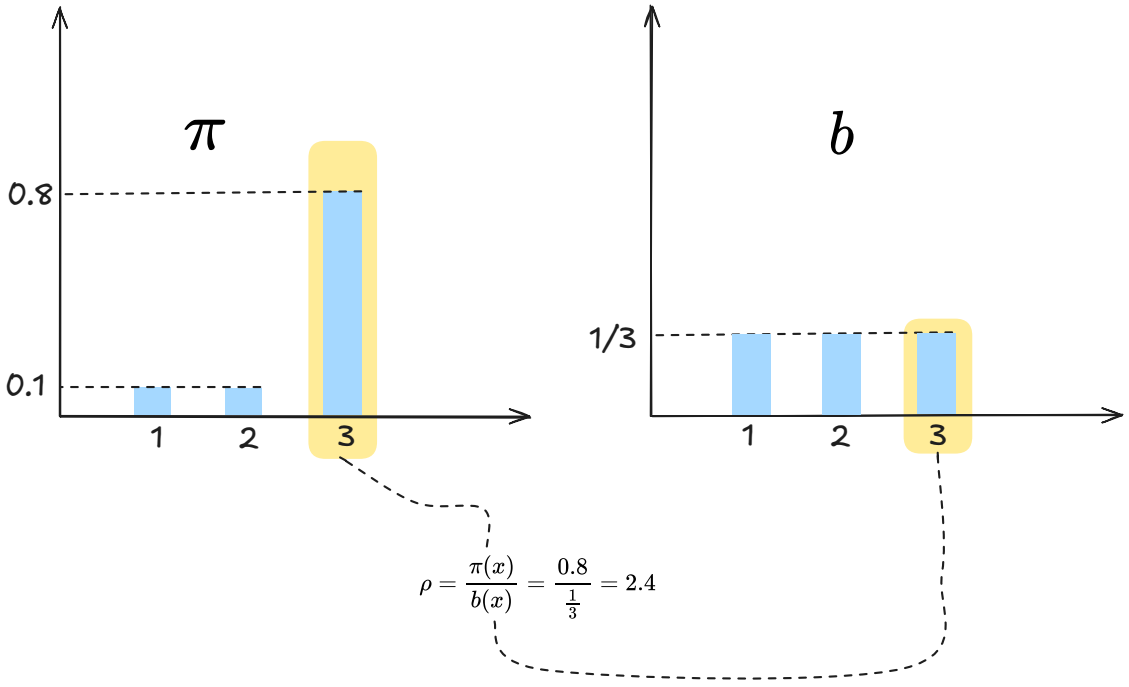


图 7.3 基于概率分布b采样3的例子

上图展示了选择3作为样本数据的例子。此时的权重ρ为2.4。因此，3这个值要乘以2.4。这意味着尽管我们得到的值是3，但实际得到的值是$3 \times 2.4 = 7.2$。听起来可能有些奇怪，但这是有道理的。原因如下。

- 由于3是概率分布π的代表值，因此本来它应该被更频繁地采样。
- 但在概率分布b中，3这个值并不经常出现。
- 为了填补这个落差，当3被采样时，它会被调整为乘以权重，从而使该值变得更大。

考虑到概率分布π和b之间的差异，我们将采样值乘以其权重来调整采样值，这么做是合理的。可是，明明采样的值是3，它却被当作了7.2，如果这是第一个采样数据，那么此时的估计值就是7.2。与真正的值2.7相比，7.2的偏离较大。因此，通过权重ρ对实际得到的值的填补越大，方差与真正的值的差异就越大。

那么，怎样才能减小方差呢？一种方法是使两个概率分布（b和π）更加接近。这样可以使权重ρ的值更接近于1。下面来做个实验。与之前的代码相比，这次只有概率分布中的b的值被改变了。

减小方差的方法 Python

```
1  b = np.array([0.2, 0.2, 0.6])
2  n = 100
3  samples = []
```

```
4
5   for _ in range(n):
6       idx = np.arange(len(b)) # [0, 1, 2]
7       i = np.random.choice(idx, p=b) # 使用 b 进行采样
8       s = x[i]
9       rho = pi[i] / b[i] # ρ
10      samples.append(rho * s)
11
12  mean = np.mean(samples)
13  var = np.var(samples)
14  print("IS: {:.2f} (var: {:.2f})".format(mean, var)) # 输出：IS: 2.72 (var: 2.48)
```

上面的代码假设b的概率分布为`[0.2, 0.2, 0.6]`，其形状更接近于`pi`的概率分布。结果是，平均值为2.72，更接近于正确答案。另外，方差是2.48，比以前小。

因此，在进行重要性采样时，可以通过使两个概率分布更加接近来减小方差。不过，强化学习的重点是让一个策略（概率分布）进行“探索”，另一个策略进行“利用”。在满足这个条件的基础上，就可以通过使两个概率分布尽可能地接近来减小方差了。

以上就是对重要性采样的介绍。

7.2 替代目标

直观上，原始策略梯度法的问题出在步长上，因此可以通过加入一个约束，将步长限制在安全范围以防止性能崩塌。

我们的目的是：

$$J(\theta) - J(\theta_{\text{old}}) \geq 0 \tag{7.20}$$

所以首先要将差值计算出来。差值计算出的结果如下：

$$J(\theta) - J(\theta_{\text{old}}) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T}\gamma^t A_t^{\pi_{\theta_{\text{old}}}}\right] \tag{7.21}$$

上式的证明如下。我们从反方向证明。

$$\begin{aligned}
&\mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T}\gamma^t A_t^{\pi_{\theta_{\text{old}}}}\right] && (1)\\
&= \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T}\gamma^t (R_t + \gamma V(S_{t+1}) - V(S_t))\right] && (2)\\
&= \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T}\gamma^t R_t + \sum_{t=0}^{T}\gamma^{t+1} V(S_{t+1}) - \sum_{t=0}^{T}\gamma^t V(S_t)\right] && (3)\\
&= \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T}\gamma^t R_t\right] + \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T}\gamma^{t+1} V(S_{t+1}) - \sum_{t=0}^{T}\gamma^t V(S_t)\right] && (4)\\
&= J(\theta) + \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=1}^{T}\gamma^t V(S_t) - \sum_{t=0}^{T}\gamma^t V(S_t)\right] && (5)\\
&= J(\theta) - \mathbb{E}_{\tau\sim\pi_\theta}[V(S_0)] && (6)\\
&= J(\theta) - \mathbb{E}_{\tau\sim\pi_\theta}[J(\theta_{\text{old}})] && (7)\\
&= J(\theta) - J(\theta_{\text{old}}) && (8)
\end{aligned} \tag{7.22}$$

第(6)步到第(7)步依赖了状态价值函数的定义：$V(S_0) = \mathbb{E}_{\tau\sim\pi_{\theta_{\text{old}}}}[G_0] = \mathbb{E}_{\tau\sim\pi_{\theta_{\text{old}}}}[G(\tau)] = J(\theta_{\text{old}})$。

第(7)步到第(8)步依赖了期望的性质，也就是由于策略π_θ和$\pi_{\theta_{\text{old}}}$相互独立，所以$\mathbb{E}_{\tau\sim\pi_\theta}[J(\theta_{\text{old}})] = J(\theta_{\text{old}})$。

$J(\theta) - J(\theta_{\text{old}})$可作为度量策略改进的指标。若该差值为正，则新策略 π_θ 比旧策略 $\pi_{\theta_{\text{old}}}$ 更优。在一次策略迭代过程中，理想情况下应选择使这一差值最大化的新策略π_θ。因此，最大化目标 $J(\theta)$ 等价于最大化该差值，两者均可通过梯度上升实现。

$$\max_{\pi_\theta} J(\pi_\theta) \Longleftrightarrow \max_{\pi_\theta}\Big(J(\pi_\theta) - J\big(\pi_{\theta_{\text{old}}}\big)\Big) \tag{7.23}$$

以这种方式刻画目标也意味着，每次策略迭代都应保证非负（单调）的改进——即$J(\pi_\theta) - J\big(\pi_{\theta_{\text{old}}}\big) \geq 0$——因为在最坏情况下我们可以简单地令$\pi_\theta = \pi_{\theta_{\text{old}}}$，也就是不改进策略。在此条件下，整个训练过程中将不会发生性能崩塌，这正是我们所期望的性质。

然而，这一差值作为目标函数有一个限制使其无法直接使用。注意，在表达式$\mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T}\gamma^t A_t^{\pi_{\theta_{\text{old}}}}\right]$中，期望要求使用新策略$\pi_\theta$采样轨迹以进行更新，但在完成更新之前，新策略$\pi_\theta$并不可用。为了解决这一悖论，我们需要设法将其改写为使用已可用的旧策略$\pi_{\theta_{\text{old}}}$。

🔥 重要性采样复习

$$\mathbb{E}_{x\sim P}[f(x)] = \mathbb{E}_{x\sim Q}\left[\frac{P(x)}{Q(x)}f(x)\right] \approx \frac{1}{|D|}\sum_{x\in D}\frac{P(x)}{Q(x)}f(x), \quad D\sim Q \tag{7.24}$$

比值$\frac{P(x)}{Q(x)}$是x的**重要性采样权重**。 重要性采样估计器的方差是什么呢?

$$\begin{aligned}
\operatorname{var}(\hat{\mu}_Q) &= \frac{1}{N}\operatorname{var}\left(\frac{P(x)}{Q(x)}f(x)\right) \\
&= \frac{1}{N}\left(\mathbb{E}_{x\sim Q}\left[\left(\frac{P(x)}{Q(x)}f(x)\right)^2\right] - \mathbb{E}_{x\sim Q}\left[\frac{P(x)}{Q(x)}f(x)\right]^2\right) \\
&= \frac{1}{N}\left(\mathbb{E}_{x\sim P}\left[\frac{Q(x)}{P(x)}\left(\frac{P(x)}{Q(x)}f(x)\right)^2\right] - \mathbb{E}_{x\sim Q}\left[\frac{P(x)}{Q(x)}f(x)\right]^2\right) \\
&= \frac{1}{N}\left(\mathbb{E}_{x\sim P}\left[\frac{P(x)}{Q(x)}f(x)^2\right] - \mathbb{E}_{x\sim P}[f(x)]^2\right)
\end{aligned} \tag{7.25}$$

红色部分很有问题！——如果$\frac{P(x)}{Q(x)}$在错误的地方很大，那么估计的方差直接炸了。

🔥 方差的计算

$$\operatorname{var}(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2 \tag{7.26}$$

这里就要用到重要性采样了。我们使用的重要性采样权重为$\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$。也就是我们使用新旧策略采取动作的概率的比值来改进旧策略$\pi_{\theta_{\text{old}}}$产生的轨迹的回报值。在新策略$\pi_\theta$下更可能发生的动作所对应的回报会被上调权重，而在$\pi_\theta$下相对不太可能的动作所对应的回报会被下调权重。该近似在下面的式子中给出。

$$\begin{aligned}
J(\theta) - J(\theta_{\text{old}}) &= \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T}\gamma^t A_t^{\pi_{\theta_{\text{old}}}}\right] \\
&\approx \mathbb{E}_{\tau\sim\pi_{\theta_{\text{old}}}}\left[\sum_{t=0}^{T} A_t^{\pi_{\theta_{\text{old}}}}\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}\right] \quad \text{（为简单起见，将}\gamma\text{设置为 1）} \\
&= J^{\text{CPI}}_{\pi_{\theta_{\text{old}}}}(\theta)
\end{aligned} \tag{7.27}$$

上面的式子也就是$J^{\mathrm{CPI}}_{\pi_{\theta_{\text{old}}}}(\theta)$叫做**替代目标**（surrogate objective）。因为新目标中包含了新旧策略的比值，所以叫做“替代目标”。上标 CPI 的意思是“保守策略迭代”（conservative policy iteration）。

现在我们有了一个新的目标函数。要将其用于策略梯度算法，需要检查在该目标下进行优化是否仍然是在执行策略梯度上升。幸运的是，我们可以证明替代目标的梯度等于策略梯度，如下式所述。

$$\nabla_\theta J^{\mathrm{CPI}}_{\pi_{\theta_{\text{old}}}}(\theta)\ |_{\theta=\theta_{\text{old}}} = \nabla_\theta J(\theta)|_{\theta=\theta_{\text{old}}} \tag{7.28}$$

上面的式子证明如下。也就是我们要证明代理目标的梯度等于策略梯度。

$$\begin{aligned}
\nabla_\theta J^{\mathrm{CPI}}_{\pi_{\theta_{\text{old}}}}(\theta)|_{\theta=\theta_{\text{old}}} &= \nabla_\theta \mathbb{E}_{\tau\sim\pi_{\theta_{\text{old}}}}\left[\sum_{t=0}^{T} A_t^{\pi_{\theta_{\text{old}}}} \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}\right]|_{\theta=\theta_{\text{old}}} \\
&= \mathbb{E}_{\tau\sim\pi_{\theta_{\text{old}}}}\left[\sum_{t=0}^{T} A_t^{\pi_{\theta_{\text{old}}}} \frac{\nabla_\theta \pi_\theta(a_t|s_t)|_{\theta=\theta_{\text{old}}}}{\pi_{\theta_{\text{old}}}(a_t|s_t)}\right] \qquad \log \text{梯度技巧} \\
&= \mathbb{E}_{\tau\sim\pi_{\theta_{\text{old}}}}\left[\sum_{t=0}^{T} A_t^{\pi_{\theta_{\text{old}}}} \nabla_\theta \log \pi_\theta(a_t|s_t)|_{\theta=\theta_{\text{old}}}\right] \\
&= \nabla_\theta J(\theta)|_{\theta=\theta_{\text{old}}}
\end{aligned} \tag{7.29}$$

上式表明，代理目标的梯度等于策略梯度。这保证了在代理目标下进行的优化仍然是在执行策略梯度上升。这也很有用，因为现在可以直接度量策略改进，最大化该度量就意味着最大化策略改进。此外，我们还知道，在上式中，$J^{\mathrm{CPI}}_{\pi_{\theta_{\text{old}}}}(\theta)$是对$J(\theta)-J(\theta_{\text{old}})$的线性近似，因为她们的一阶导数（梯度）相等。

由于我们推导出了$J^{\mathrm{CPI}}_{\pi_{\theta_{\text{old}}}}(\theta)\approx J(\theta)-J(\theta_{\text{old}})$，注意这里是约等于。也就是

$$(J(\theta)-J(\theta_{\text{old}}))-J^{\mathrm{CPI}}_{\pi_{\theta_{\text{old}}}}(\theta)\approx 0 \tag{7.30}$$

这里的误差是因为引入重要性采样带来的，也就是说如果在某些时刻如果$\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$很大，那么重要性采样的方差估计会爆炸！所以策略$\pi_\theta(a_t|s_t)$和$\pi_{\theta_{\text{old}}}(a_t|s_t)$越接近，那么上面的式子越接近于 0。

而衡量两个分布的差异的数学手段是 KL 散度。所以上面的约等于 0，可以建模为

$$|(J(\theta)-J(\theta_{\text{old}}))-J^{\mathrm{CPI}}_{\pi_{\theta_{\text{old}}}}(\theta)|\le C\sqrt{\frac{1}{T}\sum_{t=0}^{T}\Big(\mathrm{KL}\big(\pi_\theta(a_t|s_t)\,\|\,\pi_{\theta_{\text{old}}}(a_t|s_t)\big)\Big)} \tag{7.31}$$

将绝对值展开可以得到如下式子

$$J(\theta)-J(\theta_{\text{old}})\ge \underbrace{J^{\mathrm{CPI}}_{\pi_{\theta_{\text{old}}}}(\theta)-C\sqrt{\frac{1}{T}\sum_{t=0}^{T}\Big(\mathrm{KL}\big(\pi_\theta(a_t|s_t)\,\|\,\pi_{\theta_{\text{old}}}(a_t|s_t)\big)\Big)}}_{\text{下界函数}} \tag{7.32}$$

其中C是超参数。如果我们想让上式中的左手边大于 0，那么只需要保证右手边大于 0 就可以了。右手边的式子叫做**下界函数**。

当我们优化下界函数时，我们能保证优化完毕之后，右手边的式子大于等于 0 吗？

现在，让我们看看策略迭代某一步中的最坏情况。考虑对新策略π_θ的所有选择，其中也包含旧策略$\pi_{\theta_{\text{old}}}$（参数更新幅度为 0）。如果没有候选策略表现更好，就直接设定$\pi_\theta=\pi_{\theta_{\text{old}}}$，并在该次迭代中不进行更新。在这种情况下，式 (7.27) 告诉我们$J^{\mathrm{CPI}}_{\pi_{\theta_{\text{old}}}}(\theta)=\mathbb{E}_{\tau\sim\pi_{\theta_{\text{old}}}}\left[\sum_{t=0}^{T} A_t^{\pi_{\theta_{\text{old}}}}\frac{\pi_{\theta_{\text{old}}}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}\right]=0$。因为一个策略相对于自身没有期望优势。KL 散度满足$\mathrm{KL}(\pi\,\|\,\pi)=0$。

为了接受策略的变更，上面的式子表明，估计的策略改进$J^{\mathrm{CPI}}_{\pi_{\theta_{\text{old}}}}(\theta)$必须大于$C\sqrt{\frac{1}{T}\sum_{t=0}^{T}\Big(\mathrm{KL}\big(\pi_\theta(a_t|s_t)\,\|\,\pi_{\theta_{\text{old}}}(a_t|s_t)\big)\Big)}$。

如果将误差上界作为惩罚项加入到优化问题中，我们就能保证策略的单调改进。此时优化问题变成了如下形式：

$$\begin{aligned}\underset{\pi_\theta}{\operatorname{argmax}}\left(J^{\mathrm{CPI}}_{\pi_{\theta_{\mathrm{old}}}}(\theta)-C\sqrt{\frac{1}{T}\sum_{t=0}^{T}\Big(\mathrm{KL}\big(\pi_\theta(a_t|s_t)\parallel\pi_{\theta_{\mathrm{old}}}(a_t|s_t)\big)\Big)}\right)\\ \Longrightarrow J(\theta)-J(\theta_{\mathrm{old}})\geq 0\end{aligned}\tag{7.33}$$

这一结果满足了我们的最终要求。它使我们能够避免在使用原始目标 $J(\theta_{\mathrm{old}})$ 时可能出现的性能崩溃。需要注意的一个关键区别是，单调改进并不保证收敛到最优策略 π^*。例如，策略优化仍可能停留在一个局部极大值处，在该处每次策略迭代都不产生改进——即$J(\theta)-J(\theta_{\mathrm{old}})=0$。保证收敛仍是一个困难的没有解决的问题。

最后一步是考虑如何在实践中实现上式提出的优化问题。一个思路是直接约束KL散度的期望，如下式所示。

$$\frac{1}{T}\sum_{t=0}^{T}\Big(\mathrm{KL}\big(\pi_\theta(a_t|s_t)\parallel\pi_{\theta_{\mathrm{old}}}(a_t|s_t)\big)\Big)\leq\delta\tag{7.34}$$

δ限制了KL散度的大小，因此实际上约束了新策略 π_θ 相对旧策略 $\pi_{\theta_{\mathrm{old}}}$ 能偏离的程度。只有策略空间中位于 $\pi_{\theta_{\mathrm{old}}}$ 附近的小邻域内的候选策略才会被考虑。这个邻域称为置信域（`trust region`），而上式称为置信域约束。需要注意的是，δ是一个需要调参的超参数。

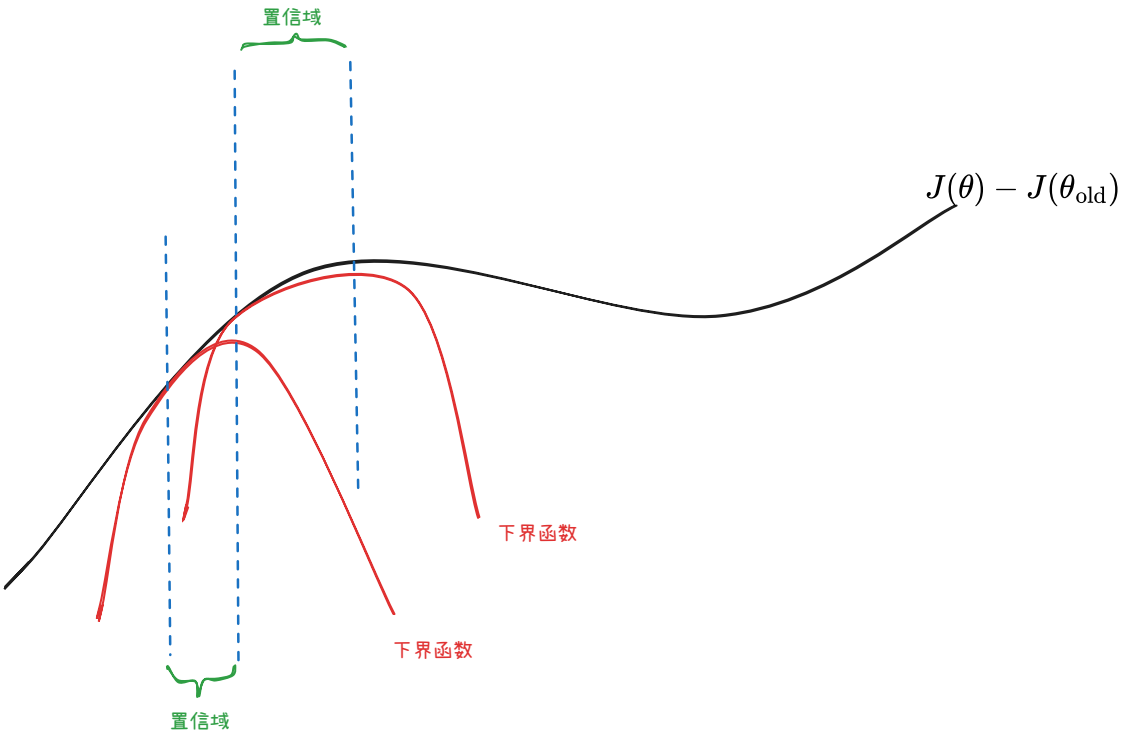
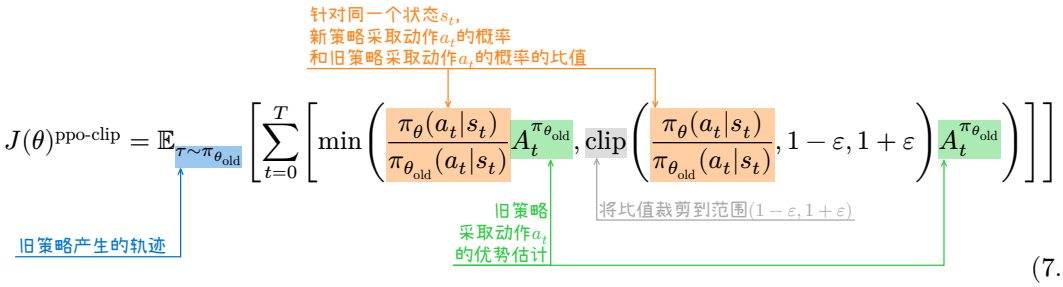


图 7.4 使用下界函数优化目标函数

将约束和替代目标合并以后，置信域策略优化问题如下所示。

$$\begin{aligned}&\max_\theta J^{\mathrm{CPI}}_{\pi_{\theta_{\mathrm{old}}}}(\theta)=\max_\theta\mathbb{E}_{\tau\sim\pi_{\theta_{\mathrm{old}}}}\left[\sum_{t=0}^{T}A_t^{\pi_{\theta_{\mathrm{old}}}}\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\mathrm{old}}}(a_t|s_t)}\right]\\ &\text{需满足约束条件}\quad\frac{1}{T}\sum_{t=0}^{T}\Big(\mathrm{KL}\big(\pi_\theta(a_t|s_t)\parallel\pi_{\theta_{\mathrm{old}}}(a_t|s_t)\big)\Big)\leq\delta\end{aligned}\tag{7.35}$$

上式中的约束条件非常的难以求解(需要求解二阶导数黑塞矩阵),所以我们不如将约束条件直接做进目标函数，从而有了 PPO 算法!



(7.36)

> ⚡ 危险
>
> 由于我们将硬约束条件变成软约束条件做进了目标函数，所以约束就没那么强了。这也是 PPO 偶尔会训练退化的原因了。

7.3 补充知识

7.3.1 状态访问分布

状态访问分布描述的是：在某个策略下，智能体在环境中运行时，访问各个状态的概率有多大。
也就是说，一个策略不只是决定动作，还会决定智能体未来更可能到哪些状态。
给定一个策略：

$$\pi(a|s) \tag{7.37}$$

初始状态分布为：

$$\rho_0(s) \tag{7.38}$$

环境的状态转移概率为：

$$P(s'|s,a) \tag{7.39}$$

那么在策略π下，第t步处于状态s的概率记作：

$$d_\pi^t(s) = \Pr(S_t = s|\pi) \tag{7.40}$$

这就是时刻t的状态访问分布。
例如：

$$d_\pi^0(s) = \rho_0(s) \tag{7.41}$$

因为第 0 步状态来自初始状态分布。
第 1 步：

$$d_\pi^1(s') = \sum_s d_\pi^0(s)\sum_a \pi(a|s)P(s'|s,a) \tag{7.42}$$

第 2 步：

$$d_\pi^2(s'') = \sum_{s'} d_\pi^1(s')\sum_a \pi(a|s)P(s'|s,a) \tag{7.43}$$

一般递推为：

$$d_{\pi}^{t+1}(s') = \sum_{s} d_{\pi}^{t}(s) \sum_{a} \pi(a|s) P(s'|s,a) \tag{7.44}$$

假设一个简单的走廊：A — B — C。初始状态在 B，那么如果策略总是向左走，那么产生的轨迹如下：

$$B \to A \to A \to A \to \cdots \tag{7.45}$$

所以状态访问分布是：

$$\begin{aligned} d_{\pi}^{0}(B) &= 1 \\ d_{\pi}^{1}(A) &= 1 \\ d_{\pi}^{2}(A) &= 1 \end{aligned} \tag{7.46}$$

如果策略总是向右走，那么产生的轨迹如下：

$$B \to C \to C \to C \cdots \tag{7.47}$$

所以状态访问分布是：

$$\begin{aligned} d_{\pi}^{0}(B) &= 1 \\ d_{\pi}^{1}(C) &= 1 \\ d_{\pi}^{2}(C) &= 1 \end{aligned} \tag{7.48}$$

所以即使两个策略只是在状态 B 的动作不同，之后访问的状态分布也完全不同。

所以策略会影响状态访问分布。

因为状态的转移满足

$$S_t \xrightarrow{A_t} S_{t+1} \tag{7.49}$$

而动作来自策略：

$$A_t \sim \pi(\cdot \mid S_t) \tag{7.50}$$

所以策略改变动作概率，动作又改变下一状态概率。

完整的链条如下：

策略$\pi(a|s)$ $\longrightarrow$动作分布变化$\longrightarrow$环境状态转移$P(s'|s,a)$ $\longrightarrow$未来状态分布变化$\longrightarrow$未来采样到的数据也变化。

这就是强化学习和普通监督学习的一个核心区别：策略不仅影响输出动作，还影响之后的数据分布。

如果任务长度为T，那么每个时间步都有一个状态分布：

$$d_{\pi(s)}^{0}, d_{\pi(s)}^{1}, ..., d_{\pi(s)}^{T} \tag{7.51}$$

目标函数通常写作

$$\begin{aligned} J(\theta) &= \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau)] \\ &= \sum_{t=0}^{T} \mathbb{E}_{s_t \sim d_{\pi}^{t}, a_t \sim \pi}[r(s_t, a_t)] \\ &= \sum_{t=0}^{T} \sum_{s} d_{\pi}^{t}(s) \sum_{a} \pi(a|s) r(s,a) \end{aligned} \tag{7.52}$$

这表示：总收益等于每个时刻访问某状态的概率 × 在该状态选动作的概率 × 奖励。

7.3.2 全变差距离（Total Variation Distance）

全变差距离表示：两个分布在所有事件上的概率差异，最大能有多大。
假设有两个离散概率分布P和Q，定义在同一个样本空间$\mathcal{X}$上。

$$D_{\mathrm{TV}}(P,Q)=\frac{1}{2}\sum_{x\in\mathcal{X}}|P(x)-Q(x)| \tag{7.53}$$

例如：

$$\begin{aligned}P&=(0.3,0.7)\\Q&=(0.4,0.6)\end{aligned} \tag{7.54}$$

那么全变差距离为：

$$D_{\mathrm{TV}}(P,Q)=\frac{1}{2}(|0.7-0.4|+|0.3-0.6|)=0.3 \tag{7.55}$$

TV 距离和 KL 散度之间有一个重要的关系，叫做 Pinsker 不等式。

$$D_{\mathrm{TV}}(P,Q)\le\sqrt{\frac{1}{2}D_{\mathrm{KL}}(P\parallel Q)} \tag{7.56}$$

这说明：如果 KL 很小，那么 TV 一定很小。

8. 组相对策略优化（GRPO）

8.1 GRPO 原理

> **GRPO**
>
> Group Relative Policy Optimization：组相对策略优化

定理 8.1 — GRPO 的目标函数.

$$J(\theta)^{\text{GRPO}} = \frac{1}{G}\sum_{i=1}^{G}\frac{1}{|\tau_i|}\sum_{t=1}^{|\tau_i|}\min\left[pA_{\tau_i,t}, \text{clip}(p, 1-\varepsilon, 1+\varepsilon)A_{\tau_i,t}\right] - \beta D_{\text{KL}}[\pi_\theta \parallel \pi_{\text{ref}}] \tag{8.1}$$

$$\text{其中比值} \quad p = \frac{\pi_\theta\left(a_{\tau_i,t}|s_{\tau_i,t}\right)}{\pi_{\theta_{\text{old}}}\left(a_{\tau_i,t}|s_{\tau_i,t}\right)}$$

先来说明一下 GRPO 目标函数中每个数学符号的含义：

- π_θ表示正在更新的策略。
- $\pi_{\theta_{\text{old}}}$表示上一轮训练好的旧策略。
- π_{ref}表示冻结的参考模型。
- G表示使用旧策略$\pi_{\theta_{\text{old}}}$采样的一组轨迹的数量，也就是如果我们使用旧策略采样了 10 条轨迹，那么$G = 10$。
- τ_i表示第i条轨迹。
- $|\tau_i|$表示第i条轨迹的动作数量。
- $\pi_\theta\left(a_{\tau_i,t}|s_{\tau_i,t}\right)$表示第$i$条轨迹的第$t$个时刻的状态为$s_{\tau_i,t}$，以及在这个状态下正在更新的策略$\pi_\theta$采取动作$a_{\tau_i,t}$的概率。
- $\pi_{\theta_{\text{old}}}\left(a_{\tau_i,t}|s_{\tau_i,t}\right)$表示第$i$条轨迹的第$t$个时刻的状态为$s_{\tau_i,t}$，以及在这个状态下正在更新的策略$\pi_{\theta_{\text{old}}}$采取动作$a_{\tau_i,t}$的概率。
- $A_{\tau_i,t}$表示第i条轨迹的第t个时刻的动作的优势。
- β是超参数。$D_{\text{KL}}[\pi_\theta|\pi_{\text{ref}}]$表示$\pi_\theta$和$\pi_{\text{ref}}$而偏离程度，也就是 KL 散度。

8.2 使用 GRPO 玩倒立摆游戏

策略网络

```python
class PolicyNet(nn.Module):
    def __init__(self, action_size):
        super().__init__()
        self.l1 = nn.Linear(4, 128)
        self.l2 = nn.Linear(128, action_size)

    def forward(self, x):
        x = F.relu(self.l1(x))
        x = F.softmax(self.l2(x), dim=1)
        return x
```

智能体代码如下：

```python
class Agent:
    def __init__(self):
        self.lr = 0.0002
        self.action_size = 2
        self.pi = PolicyNet(self.action_size)
        self.optimizer = optim.Adam(self.pi.parameters(), lr=self.lr)

    def get_action(self, state):
        probs = self.pi(torch.tensor(state).unsqueeze(0)).squeeze(0)
        m = Categorical(probs)
        action = m.sample().item()

        return action, probs
```

将采样一条轨迹封装为一个函数

```python
class Agent:
    ...

    def collect_trajectory(self, env):
        """采样一条轨迹"""
        state = env.reset()
        states, log_probs, actions = [], [], []
        episode_reward = 0
        done = False

        while not done:
            action, probs = self.get_action(state)
            next_state, reward, done, _ = env.step(action)

            states.append(state)
            actions.append(action)
            log_prob = torch.log(probs)[action]
            log_probs.append(log_prob.item())

            state = next_state
```

```
            episode_reward += reward

        # 归一化奖励
        normalized_reward = episode_reward / 200.0

        return states, log_probs, actions, normalized_reward
```

这里的归一化奖励需要说一下，我们已经知道木杆坚持 200 步不倒下，游戏就成功结束了。那么如果木杆坚持了 3 步就倒下，这条轨迹的奖励应该如何计算呢？这里我们选择$\frac{3}{200}=0.015$。这就是我们给这条轨迹的奖励。

GRPO 的优势计算是和 PPO 的优势计算有区别的地方。PPO 使用了价值函数网络评估每个动作的价值，并且使用了广义优势估计（GAE）。而 GRPO 创造性的提出了组相对优势。

也就是轨迹τ_i相对于**组内**其它轨迹的优势是多少？也就是如下

轨迹τ_i的归一化奖励是：

$$R_{\tau_i}^{\text{normalized}}=\frac{G(\tau_i)}{200} \tag{8.2}$$

而一组轨迹的平均奖励是

$$\text{reward}_{\text{mean}}=\frac{1}{G}\sum_{i=0}^{G}R_{\tau_i}^{\text{normalized}} \tag{8.3}$$

那么轨迹τ_i相对于组内其它轨迹的优势为：

$$A_{\tau_i}=\frac{R_{\tau_i}^{\text{normalized}}-\text{reward}_{\text{mean}}}{\text{reward}_{\text{std}}} \tag{8.4}$$

轨迹中每一个动作的优势就等于这个动作所在轨迹的优势，也就是：

$$A_{\tau_i,t}=A_{\tau_i} \tag{8.5}$$

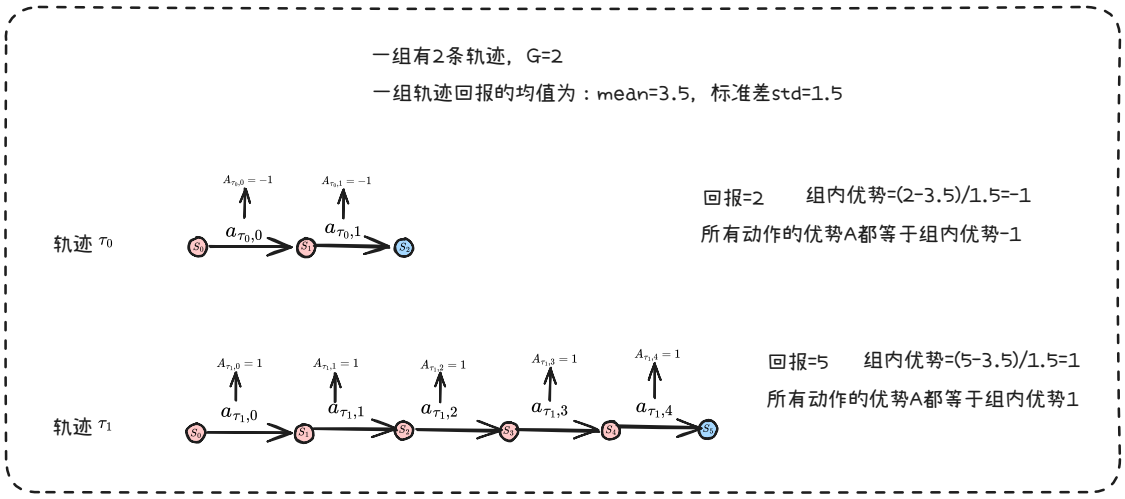


图 8.1 组内优势的计算，要注意的是动作的优势等于所在轨迹的组内优势

```
class Agent:                                                    Python
    ...

    def calc_advantages_with_grpo(self, trajectories):
```

```
        """使用一组轨迹计算某条轨迹的组内优势"""
        # [轨迹0的归一化奖励，轨迹1的归一化奖励，...]
        rewards = [r for o, l, a, r in trajectories]
        mean_reward = sum(rewards) / len(rewards)
        std_reward = np.std(rewards) + 1e-8
        # [轨迹0的组相对优势，轨迹1的组相对优势，...]
        advantages = [(r - mean_reward) / std_reward for r in rewards]

        return advantages
```

使用 GRPO 算法更新策略的代码如下

```python
class Agent:
    ...

    def update(self, trajectories):
        advantages = self.calc_advantages_with_grpo(trajectories)

        for step in range(20):
            loss = 0.0
            for traj, advantage in zip(trajectories, advantages):
                """遍历组里面的每一条轨迹和对应的组内优势"""
                states, log_probs, actions, _ = traj
                states = torch.tensor(states)
                log_probs = torch.tensor(log_probs).view(-1, 1)
                actions = torch.tensor(actions).view(-1, 1)
                new_log_probs = torch.log(self.pi(states).gather(1, actions))
                ratio = torch.exp(new_log_probs - log_probs)
                clipped_ratio = torch.clamp(ratio, 0.8, 1.2)
                traj_loss = torch.mean(
                    -torch.min(ratio * advantage, clipped_ratio * advantage))

                loss += traj_loss
            loss = loss / len(trajectories)

            self.optimizer.zero_grad()
            loss.backward()
            self.optimizer.step()

        return None
```

训练循环如下：

```python
def train(agent, env):
    G = 5  # 一组轨迹有5条
    trial_num = 0
    while True:
        for episode in range(20):
            trajectories, episode_rewards = [], []
            for _ in range(G):
                states, log_probs, actions, normalized_reward =
                agent.collect_trajectory(
```

```
                    env)
                trajectories.append(
                    (states, log_probs, actions, normalized_reward))
                episode_rewards.append(normalized_reward * 200)
            agent.update(trajectories)

        # 一组轨迹的平均奖励
        avg_reward = sum(episode_rewards) / len(episode_rewards)
        trial_num += 1

        if avg_reward > 195:
            print("训练结束，训练回合数：", trial_num)
            return
        else:
            print(f"训练回合数：{trial_num}，平均奖励：{avg_reward}")
```

III 基于人类反馈的强化学习

9 大语言模型训练概述 127
9.1 LLM 训练流程 127
9.2 RLHF 的关键技术 129

10 使用 DPO 微调大语言模型 133
10.1 DPO 理论介绍以及 DPO 存在的问题 . . . 133
10.2 DPO 实战 135

11 使用 PPO 微调大语言模型--复刻 InstructGPT 159
11.1 InstructGPT 训练流程 159
11.2 即时奖励R_t的计算 160
11.3 PPO 微调 LLM 实战 163

12 使用 GRPO 微调大语言模型——复刻 DeepSeek-R1 181
12.1 GRPO 简介 181
12.2 使用 DAPO（GRPO 的变种）微调 Qwen2.5-3B-Instruct 182
12.3 GRPO 应用场景 209

9. 大语言模型训练概述

9.1 LLM 训练流程

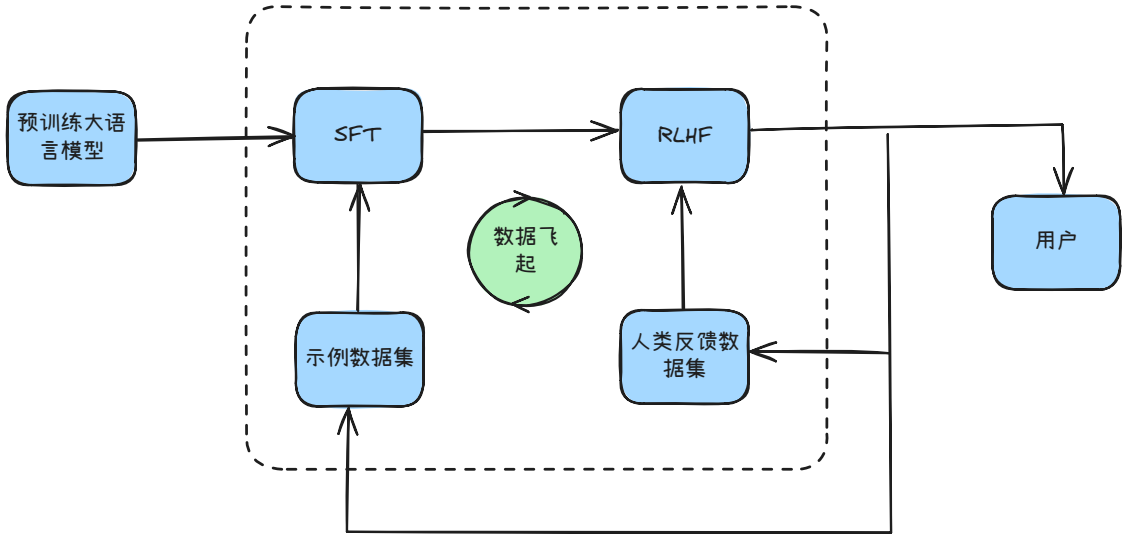


图 9.1 大语言模型训练流程

9.1.1 预训练

假设训练数据是“abcd”，那么预测下一个 token 的原理如下

a	b	c
↓	↓	↓
b	c	d

也就是我们希望如下的目标函数越大越好

$$J(\theta) = \pi_\theta(b|a)\pi_\theta(c|a,b)\pi_\theta(d|a,b,c) \tag{9.1}$$

从这里我们可以看出，在 LLM 这个环境中

- 策略π_θ是 LLM
- 状态是上下文，例如$\pi_\theta(d|a,b,c)$的状态S_t = **字符串** abc
- 动作：$\pi_\theta(d|a,b,c)$表示在状态S_t = **字符串** abc的情况下，A_t为输出字符 “d”。
- 动作概率：$\pi_\theta(d|a,b,c)$为采取动作的概率。

9.1.2 监督微调

假设训练数据是 “How are you? I am fine.<eos_token>”。
其中指令部分为 “How are you?”。
回答部分为 “I an fine.<eos_token>”。
那么训练方式如下：

How	are	you	?	I	am	fine	.
↓	↓	↓	↓	↓	↓	↓	↓
are	you	?	I	am	fine	.	<eos_token>

只有绿色部分计算损失。也就是我们要让下面的式子越大越好

$$\begin{aligned} J(\theta) = &\pi_\theta(\text{I}|\text{How are you?}) \times \\ &\pi_\theta(\text{am}|\text{How are you? I}) \times \\ &\pi_\theta(\text{fine}|\text{How are you? I am}) \times \\ &\pi_\theta(.|\text{How are you? I am fine}) \times \\ &\pi_\theta(<\text{eos_token}>|\text{How are you? I am fine.}) \end{aligned} \tag{9.2}$$

通过观察上面的目标函数，我们发现，如果 SFT 训练的轮数很多，那么会过拟合监督微调数据。模型会发生灾难性遗忘，因为在目标函数中没有约束新旧模型的偏差（KL 散度）。

> ⚡ 危险
>
> SFT 一般不会训练很多轮！

监督微调（Supervised Fine-Tuning，SFT）通常也是采用 “预测下一个词”（predict next token）的训练方式。

- 基础模型训练：大多数语言模型（如 GPT 系列）本身就是通过自回归方式训练的，即在给定上下文的情况下预测下一个词（token）。
- 监督微调（SFT）：在微调阶段，模型通常会使用带有标签的输入-输出对进行训练，比如对话数据、问答对等。训练目标依然是让模型在给定输入（上下文）的条件下，预测正确的下一个词。

换句话说，SFT 通过**有监督的数据**指导模型生成更符合特定任务或风格的输出，但训练目标仍然是最大化正确预测下一个词的概率。

训练阶段	训练方式	目标
预训练	自回归预测下一个词	学习语言的通用统计规律
监督微调	给定输入，预测下一个词	让模型生成更符合特定任务的输出

表 9.1 预训练和监督微调

所以通过 SFT 微调基础模型，可以让大模型输出我们喜欢的回答。

> ⚡ 危险
>
> 但是 SFT 无法让大模型**不输出我们不喜欢的回答**。

因为 SFT 的训练目标是“预测下一个 token”，它本质上是拟合训练数据中的分布。如果训练数据中存在不理想的回答，模型可能仍然学到这些模式。

所以我们要使用“强化学习”来对大语言模型进行微调，这就是“基于人类反馈的强化学习”。

> 🔥 RLHF
>
> RLHF: Reinforcement Learning From Human Feedback

当我们谈论大语言模型（LLM）的强化学习时，我们进入了一个完全不同的世界。我们不再训练智能体在倒立摆环境中的表现，而是对预训练好的大语言模型进行微调，使其符合人类的偏好。该模型不会与外部环境交互——它本质上是在探索自身的输出空间。

正如 OpenAI 和其他科研人员所发现的，这种方法对于将原始语言模型转化为辅助系统至关重要。正如 IBM 研究人员指出的那样，“RLHF 特别适合于目标复杂、定义不明确或难以指定的任务。”毕竟，如何用数学来定义“乐于助人”或“诚实”这样的概念呢？

这里的根本转变是：

1. 我们正在优化大语言模型以适应人类的偏好，而不是对环境的掌控
2. 我们的数据来自人类的判断，而不是环境互动
3. 我们需要在奖励最大化与保持接近原始预训练行为之间取得平衡

这种平衡行为使得 LLM 强化学习特别棘手，但也特别令人着迷！

9.2 RLHF 的关键技术

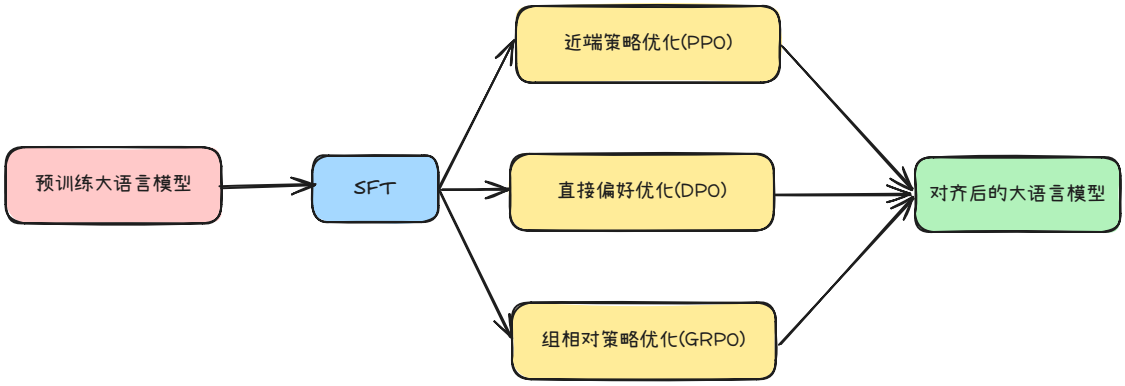


图 9.2 RLHF 常用算法

9.2.1 近端策略优化（PPO）

PPO 是 LLM 强化学习对齐技术的**重量级冠军**，因 OpenAI 开发的 InstructGPT 和 ChatGPT 而闻名。PPO 于 2017 年开发，解决了强化学习中的一个关键挑战：如何在不破坏训练稳定性的情况下进行有意义的更新。

PPO 成功的秘诀在于其“近端”特性——它对策略进行保守更新，防止模型在单次迭代中发生过大变化。这是通过其目标函数中巧妙的裁剪机制实现的：

$$J_{\text{PPO}}(\theta) = \mathbb{E}\left[\min\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)}A, \text{clip}\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)}, 1-\varepsilon, 1+\varepsilon\right)A\right)\right] \tag{9.3}$$

通过限制新旧策略之间的比例（通常在 1 ± 0.2 以内），PPO 可以确保模型在训练过程中不会偏离目标。

PPO 一直是实现人类反馈强化学习（RLHF）的首选算法，该算法遵循以下三个步骤：

1. 从预训练模型开始
2. 监督微调
3. 根据人类偏好训练奖励模型
4. 使用 PPO 微调 LLM，来最大化奖励，同时保持接近原来的 LLM。

9.2.2 直接偏好优化（DPO）

如果说 PPO 是一位谨慎的外科医生，能够精准地进行手术，那么 DPO 则是一位效率专家，他找到了通往同一目标的捷径。DPO 于 2023 年在一篇题为《直接偏好优化：你的语言模型其实是一个奖励模型》的论文中首次提出，它彻底消除了对单独奖励模型的需求。

DPO 的精妙之处在于其数学洞察力：奖励函数与最优策略之间存在直接映射。通过利用这种关系，DPO 将强化学习问题转化为基于人类偏好数据的更简单的分类问题。

与传统的三步 RLHF 流程不同，DPO 只需一个训练阶段即可实现相同的目标。这就像跳过中间环节，直接到达源头。

DPO 对从业者特别有吸引力的原因是：

1. 简单：无需训练单独的奖励模型
2. 效率：无需在训练期间进行昂贵的采样
3. 稳定性：组件越少，出错的可能性就越小
4. 性能：在控制输出属性方面，通常与 PPO-RLHF 相当或超过 PPO-RLHF

DPO 的目标函数如下：

$$J^{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = \mathbb{E}_{(y_w, y_l, x)\sim\mathcal{D}}\left[\log\left(\sigma\left(\beta\left(\log\frac{\pi_\theta(y_w|x)}{\pi_\theta(y_l|x)} - \log\frac{\pi_{\text{ref}}(y_w|x)}{\pi_{\text{ref}}(y_l|x)}\right)\right)\right)\right] \tag{9.4}$$

冻结的参考 LLM　偏好数据集　超参数　人类偏好的回答　正在训练的 LLM　sigmoid 函数　提示词　人类厌恶的回答

DPO 的目标是让$\frac{\pi_\theta(y_w|x)}{\pi_\theta(y_l|x)}$越大越好！

9.2.3 组相对策略优化（GRPO）

$$J^{\text{GRPO}}(\theta) = \mathbb{E}_{q\sim P(Q),\{o_i\}_{i=1}^{G}\sim\pi_{\theta_{\text{old}}}(Q|q)}\left[\mathcal{L}_{\text{GRPO}}(\theta)\right] \tag{9.5}$$

针对每个采样出来的问题q。使用旧策略生成G个不同的回答$\{o_1, o_2, ..., o_G\}$

从分布$P(Q)$中采样一些问题q。$P(Q)$表示用来训练模型的问题数据集

替代损失函数

图 9.3 GRPO 目标函数

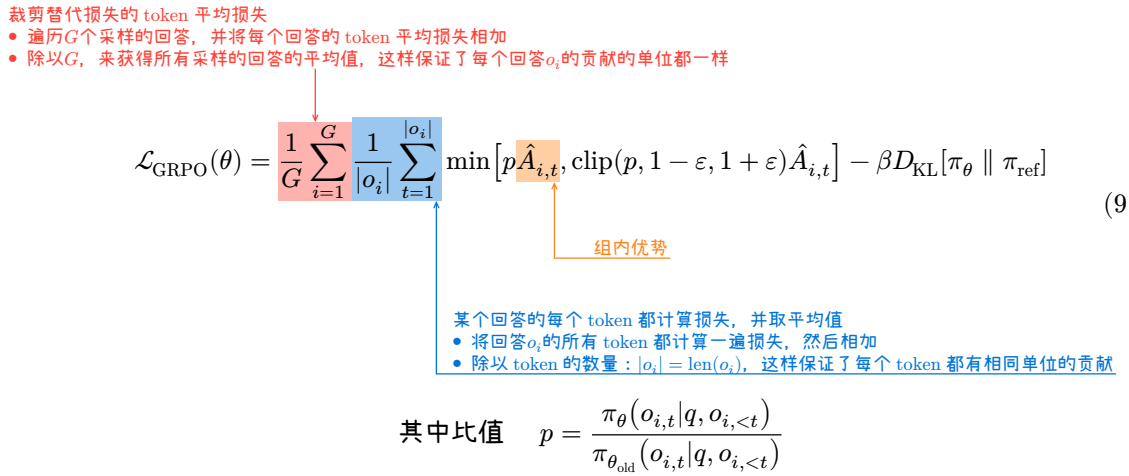


图 9.4 GRPO 替代损失函数

现在，如果我们能将 PPO 的可靠性与更高的效率以及对推理能力提升的专注结合起来，会怎么样呢？GRPO 应运而生，它是强化学习领域的最新成果之一，由 DeepSeek 开发，并用于训练其令人印象深刻的 DeepSeek-Math 和 DeepSeek-R1 模型。

GRPO 建立在 PPO 的基础上，但引入了几项巧妙的修改：

- GRPO 去掉了价值函数模型，减少了内存开销。
- GRPO 评估输出的一组回答而不是单个 token。
- GRPO 直接将 KL 散度纳入损失函数。

这种基于组（Group）的方法尤其巧妙。GRPO 不是单独评估每个 token，而是将完整的答案作为一个整体来看待 $\Longrightarrow$ 这是一种评估推理能力更自然的方式，其中整个解答过程都很重要，而不仅仅是单个步骤。

用 AWS 社区文章的话来说，“GRPO 用于计算优势的组相对方式与奖励模型的比较性质非常吻合，因为奖励模型通常是在同一问题的输出比较数据集上进行训练的。”

10. 使用 DPO 微调大语言模型

10.1 DPO 理论介绍以及 DPO 存在的问题

> 🔥 提示
>
> - DPO：Direct Preference Optimization
> - 直接偏好优化：你的大语言模型实际上是一个奖励模型

10.1.1 偏好数据集

DPO 需要偏好数据集来微调 LLM。偏好数据集的格式如下：

```JSON
{
  "prompt": "这部电影怎么样？",
  "chosen": "这部电影很好看。",
  "rejected": "这部电影不好看。"
}
```

数据集构建的方法：

- 针对同一个 prompt，通过调整温度，让 LLM 输出不同的回答。然后让数据标注工程师来标注对不同回答的偏好。温度示例网站。
- 在使用 ChatGPT 时，你可能注意到，偶尔会被要求在两个相似的答案中选择一个来继续对话。这个偏好会被记录下来，并用于在未来的偏好调整中改进模型。同样。
- 手工标注
- 合成数据：通过写提示词来让 LLM 生成偏好数据集（需要人工审核）。
- 使用网上的开源数据集
- ...

数据集容易存在的问题：

1. 正负例区分不明显

```JSON
{
  "prompt": "这部电影怎么样？",
```

```
  "chosen": "这部电影很好看。",
  "rejected": "这部电影挺好看。"
}
```

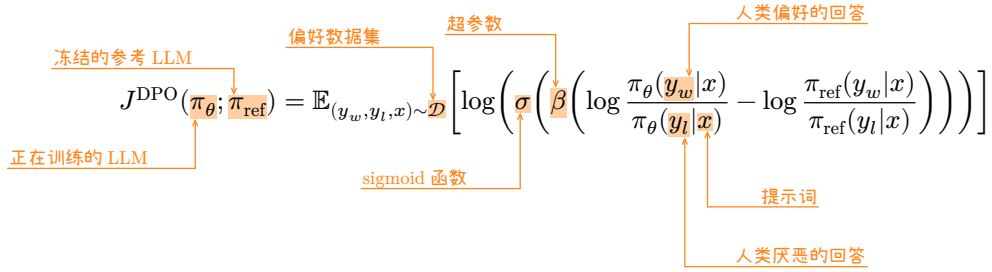
人类都无法识别哪个应该是正例，哪个应该是负例。

1. 数据集中存在偏好循环

```JSON
{
  "prompt": "这部电影怎么样？",
  "chosen": "这部电影很好看。",
  "rejected": "这部电影很差。"
},
{
  "prompt": "这部电影怎么样？",
  "chosen": "这部电影很差。",
  "rejected": "这部电影很好看。"
}
```

当模型看到以上数据时，就不知道人类的偏好是什么了。无法学到任何东西。因为第一个人标注的数据的偏好是$A \succ B$，第二个人标注的数据的偏好是$B \succ A$，那么模型看到的偏好是$A \succ B \succ A$。这就是偏好循环。

10.1.2 DPO 目标函数



$$J^{\text{DPO}}(\pi_\theta;\pi_{\text{ref}}) = \mathbb{E}_{(y_w,y_l,x)\sim\mathcal{D}}\left[\log\left(\sigma\left(\beta\left(\log\frac{\pi_\theta(y_w|x)}{\pi_\theta(y_l|x)} - \log\frac{\pi_{\text{ref}}(y_w|x)}{\pi_{\text{ref}}(y_l|x)}\right)\right)\right)\right] \tag{10.1}$$

这里我们思考一下，给定提示词x的情况下，模型输出补全y的概率是多少呢？如下所示：

$$\begin{aligned}\pi_\theta(y|x) &= P_{\text{LLM}_\theta}(y_0|x)P_{\text{LLM}_\theta}(y_1|x,y_0)\cdots P_{\text{LLM}_\theta}(y_t|x,y_{<t}) \\ &= \prod_{t=0}^{|y|} P_{\text{LLM}_\theta}(y_t|x,y_{<t})\end{aligned} \tag{10.2}$$

然后根据 `log` 的性质也就是$\log\frac{A}{B} = \log A - \log B$。可以推导出目标函数最内部的表达式的另一种形式。

$$\begin{aligned}&\log\frac{\pi_\theta(y_w|x)}{\pi_\theta(y_l|x)} - \log\frac{\pi_{\text{ref}}(y_w|x)}{\pi_{\text{ref}}(y_l|x)} \\ &= \log\pi_\theta(y_w|x) - \log\pi_\theta(y_l|x) - \log\pi_{\text{ref}}(y_w|x) + \log\log(\pi_{\text{ref}}(y_l|x)) \\ &= \log\pi_\theta(y_w|x) - \log\pi_{\text{ref}}(y_w|x) - (\log\pi_\theta(y_l|x) - \log\log(\pi_{\text{ref}}(y_l|x))) \\ &= \log\frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log\frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\end{aligned} \tag{10.3}$$

通过观察目标函数，我们知道在最开始训练时：$\pi_\theta = \pi_{\text{ref}}$，所以$\log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} = 0$。但由于外层有$\sigma$函数，所以不影响反向传播更新网络（$\sigma(0) = 0.5$）。

通过观察目标函数，我们可以知道随着训练的进行，$\frac{\pi_\theta(y_w|x)}{\pi_\theta(y_l|x)}$会越来越大。因为$\frac{\pi_{\text{ref}}(y_w|x)}{\pi_{\text{ref}}(y_l|x)}$是一个常数（作为正则化项存在）。

DPO 存在的问题

1. $\frac{\pi_\theta(y_w|x)}{\pi_\theta(y_l|x)}$的分子和分母可能同时增大或者减小。例如分母增大了 1.5 倍，分子增大了 3 倍。那么就导致了 LLM 输出人类偏好的回答的概率和输出人类讨厌的回答的概率都增加了。
2. $\frac{\pi_\theta(y_w|x)}{\pi_\theta(y_l|x)}$随着训练，分子越来越大，分母越来越小，结果导致了$\pi_\theta(y_w|x)$趋近于 1，$\pi_\theta(y_l|x)$趋近于 0，正则化项$\frac{\pi_{\text{ref}}(y_w|x)}{\pi_{\text{ref}}(y_l|x)}$没有起到作用，最终 LLM 彻底失去了探索能力。也就是出现了“过拟合”的问题。DPO 容易过拟合，泛化能力不足。所以 DPO 的训练轮数不能太多。
3. 数据集中存在的偏好循环会导致 LLM 学不到东西。例如数据集中存在$A \succ B$和$B \succ C$，那么模型可以学习到人类的偏好是$A \succ B \succ C$。如果这是还有一条数据中$C \succ A$，那么模型就懵了，不知道人类的偏好是什么了。因为模型看到的是$A \succ B \succ C \succ A$。也就是偏好循环。

尽管存在这些缺点，DPO 仍然是一种非常有效的工具；许多最成功和性能最好的开源 LLM 都是使用 DPO 进行指令微调的。

危险

个人认为 DPO 不算是强化学习，因为 DPO 需要训练数据集。而真正的强化学习的训练数据是由策略模型自己采样得来的。

10.2 DPO 实战

我们遵循 RLHF 的标准流程：预训练 LLM $\Longrightarrow$ SFT $\Longrightarrow$ DPO

10.2.1 第一步：对预训练模型进行基于指令的监督微调（SFT）

提示

- 基于指令的监督微调：Instruct SFT
- 目标：在给定问题（Question）的前提下，使得模型产生该回答（Answer）的概率最大化。

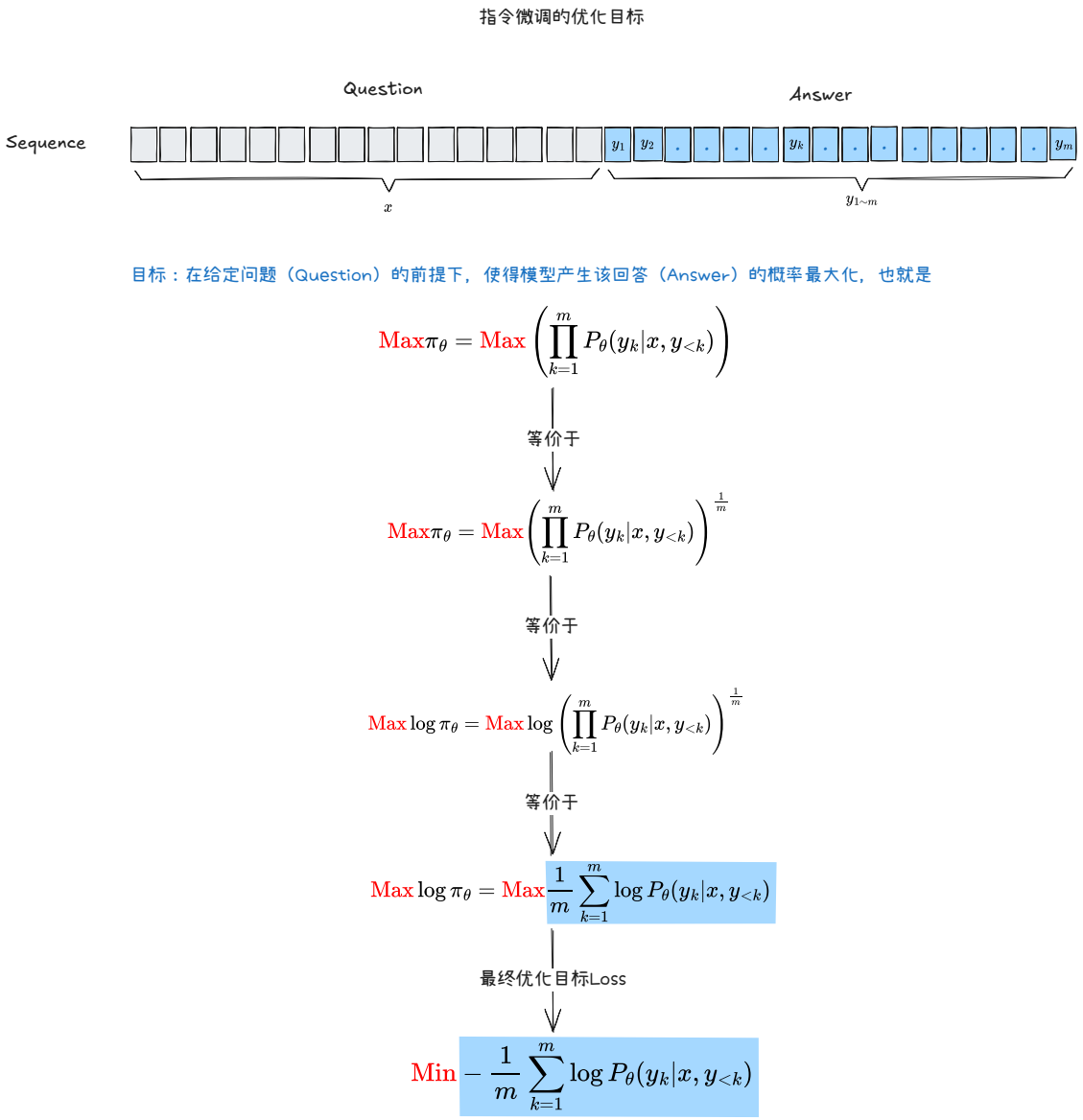


图 10.1 SFT 的损失函数

我们要注意的是在**基于指令的监督微调**中，我们只针对**答案**部分计算损失。

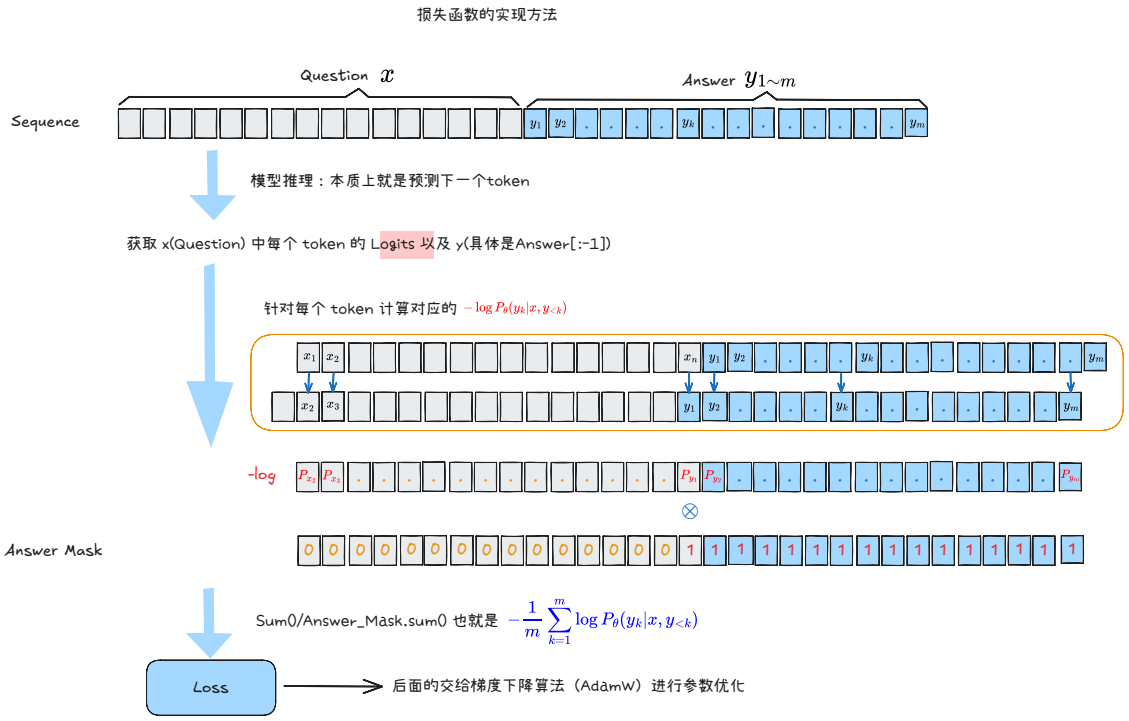


图 10.2 基于指令的监督微调只针对答案部分计算损失

我们将损失的计算推广到批次（batch）。

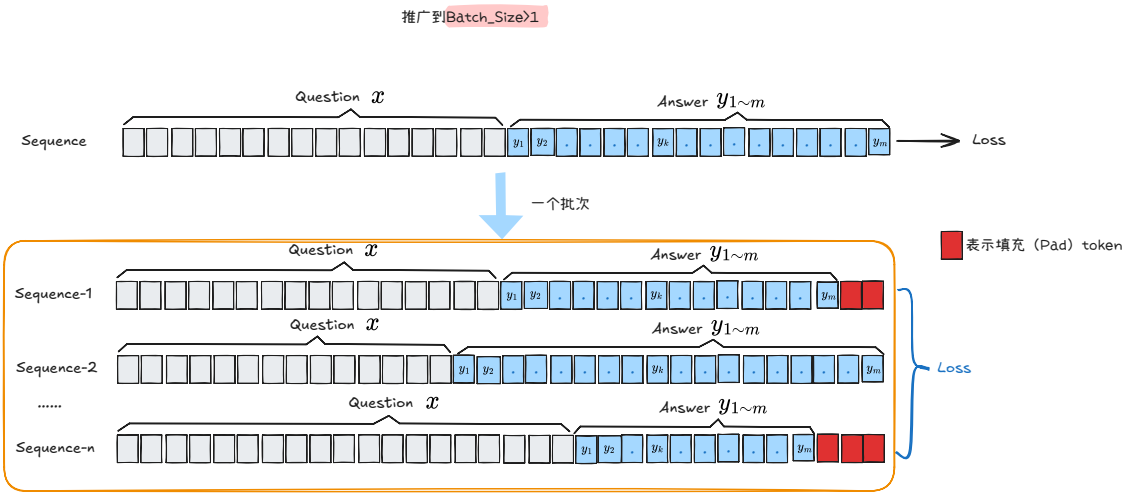


图 10.3 将只针对答案计算损失推广到批次

首先导入需要的依赖。

```python
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch
import numpy as np
import os
from dataclasses import dataclass
import datasets
```

```python
import time
```

接下来我们导入模型和分词器。

```python
device = "cuda"
model_path = "./Qwen3-0.6B-Base"

model = AutoModelForCausalLM.from_pretrained(model_path, dtype="auto", device_map="auto")
tokenizer = AutoTokenizer.from_pretrained(model_path)
```

然后我们设置一下生成文本的参数。保证测试的一致性。
其中 151645 为 <|im_end|>，151643 为 <|endoftext|>。

```python
model.generation_config.do_sample = True
model.generation_config.eos_token_id = [151645, 151643]
model.generation_config.pad_token_id = 151643
model.generation_config.temperature = 0.7
model.generation_config.top_p = 0.8
model.generation_config.top_k = 20
model.generation_config.repetition_penalty = 1.05
```

代码 10.4 模型生成的配置

然后我们定义一下 SFT 阶段的训练超参数。

```python
@dataclass
class SFTConfig:
    max_length = 2500
    batch_size = 2
    gradient_accumulation_steps = 8
    log_iter = 400
    max_lr = 2e-5
    min_lr = 2e-6
    warmup_steps = 1000
```

接下来我们导入训练数据并将训练数据转换成 input_ids。

```python
ultrachat_200k_data = datasets.load_dataset("./ultrachat_200k")

def tokenize_and_format(data):
    """使用模型自带的聊天模板格式化训练数据"""
    input_ids = tokenizer.apply_chat_template(
        data,
        tokenize = True,
        add_generation_prompt = False,
        truncation = True,
        max_length = 2500,
    )

    return input_ids

## 生成训练数据的input_ids
```

```python
train_data = []
i = 0
while True:
    data = ultrachat_200k_data["train_sft"][i]["messages"]
    # 添加系统提示词
    data.insert(
        0,
        {"content": "You are a helpful assistant", "role": "system"}
    )
    input_ids = tokenize_and_format(data)
    train_data.append(input_ids)
    i += 1
    if i % 1000 == 0:
        print(f"已处理{i}条数据")
    if i == 50000:
        break
```

接下来我们编写一下学习率的线性预热和余弦衰减的函数。

```python
def linear_warmup(current_step, warmup_steps, max_lr):
    if current_step < warmup_steps:
        return max_lr * current_step / warmup_steps
    else:
        return max_lr

def cosine_decay(current_step, warmup_steps, total_steps, max_lr, min_lr):
    if current_step < warmup_steps:
        return linear_warmup(current_step, warmup_steps, max_lr)
    else:
        progress = (current_step - warmup_steps) / (total_steps - warmup_steps)
        decay = 0.5 * (1 + np.cos(np.pi * progress))
        return (max_lr - min_lr) * decay + min_lr
```

代码 10.5 学习率调度：线性预热和余弦衰减

基于指令 SFT 的掩码设置

- SFT 和预训练的区别核心就是掩码掉“问题”部分的损失，而只看“回答”部分的损失，并仅基于回答部分的损失进行优化
- 实现方式：构造损失掩码，仅针对每轮对话（含多轮）的模型“输出”部分（也就是回答部分）进行损失计算

假设我们的多轮对话数据是

```json
data = [
    {
        "content": "你是谁？",
        "role": "user"
    },
    {
```

```
        "content": "我是左元。",
        "role": "assistant"
    },
    {
        "content": "你会强化学习吗？",
        "role": "user"
    },
    {
        "content": "略知一二。",
        "role": "assistant"
    }
]
```

经过模型的对话模板格式化之后是

```
<|im_start|>system
You are a helpful assistant<|im_end|>
<|im_start|>user
你是谁？<|im_end|>
<|im_start|>assistant
我是左元。<|im_end|>
<|im_start|>user
你会强化学习吗？<|im_end|>
<|im_start|>assistant
<think></think>略知一二。<|im_end|>
```

代码如下：

```python
def create_answer_mask(input_ids, tokenizer):
    """
    创建仅对助手回答部分计算损失的掩码

    Args:
        input_ids: 输入token序列 [batch_size, seq_len]
        tokenizer: 分词器

    Returns:
        answer_mask: 助手回答部分为1，其他部分为0的掩码
    """
    batch_size, seq_len = input_ids.shape
    answer_mask = torch.zeros_like(input_ids)

    # 获取结束标记的token id
    eos_token_id = tokenizer.encode("<|im_end|>")[0]

    for batch_idx in range(batch_size):
        # 找到所有 <|im_end|> 的位置
        eos_positions = torch.where(
            input_ids[batch_idx] == eos_token_id
        )[0].tolist()

```

```
        if len(eos_positions) < 2:  # 至少需要user和assistant各一个结束标记
            continue

        # 解析对话轮次
        user_ends, assistant_ends = \
            _parse_conversation_turns(eos_positions)

        # 为每个助手回答设置掩码
        _set_answer_masks(
            answer_mask[batch_idx],
            user_ends,
            assistant_ends,
            seq_len
        )

    return answer_mask


def _parse_conversation_turns(eos_positions):
    """
    解析对话轮次，分离用户和助手的结束位置

    对话格式：
    ◁im_start▷user\n{user_msg}◁im_end▷\n◁im_start▷assistant\n{assistant_msg}◁im_end▷\n

    eos_positions[0]: system结束（如果有）
    eos_positions[1]: 第1轮user结束
    eos_positions[2]: 第1轮assistant结束
    eos_positions[3]: 第2轮user结束
    eos_positions[4]: 第2轮assistant结束
    ...
    """
    # 跳过system系统提示词部分，从第一个user开始
    conversation_eos = eos_positions[1:]  # 去掉system的<im_end>

    # 偶数索引：user结束位置，奇数索引：assistant结束位置
    # pos + 1 为跳过<im_end>，此时pos+1指向`\n`
    user_ends = [pos + 1 for pos in conversation_eos[::2]] # 每隔2个取一个，从0开始，顺便跳过`<im_end>`
    assistant_ends = [pos + 1 for pos in conversation_eos[1::2]] # 每隔2个取一个，从1开始，顺便跳过`<im_end>`

    return user_ends, assistant_ends


def _set_answer_masks(mask, user_ends, assistant_ends, seq_len):
    """
    为助手回答部分设置掩码

```

```
    Args:
        mask: 当前样本的掩码 [seq_len]
        user_ends: 用户消息结束位置列表
        assistant_ends: 助手消息结束位置列表
        seq_len: 序列长度
    """
    num_user_turns = len(user_ends)
    num_assistant_turns = len(assistant_ends)

    if num_user_turns == num_assistant_turns:
        # 完整对话：每轮都有用户问题和助手回答
        for user_end, assistant_end in zip(user_ends, assistant_ends):
            answer_start = user_end + 3  # 跳过 ◁im_start▷assistant 这2个token，answer_start指向assistant后面的`\n`
            answer_end = assistant_end - 1  # 不包含<im_end>后面的`\n`
            mask[answer_start:answer_end] = 1 # 左闭右开区间，所以掩码不包含<im_end>

    elif num_user_turns == num_assistant_turns + 1:
        # 未完成对话：最后一轮助手回答被截断

        # 处理完整的对话轮次
        for user_end, assistant_end in zip(user_ends[:-1], assistant_ends):
            answer_start = user_end + 3
            answer_end = assistant_end - 1
            mask[answer_start:answer_end] = 1

        # 处理最后一轮被截断的助手回答
        last_user_end = user_ends[-1]
        last_answer_start = last_user_end + 3
        mask[last_answer_start:] = 1  # 到序列结尾
```

代码 10.6 答案掩码

设置模型的训练超参数：

```python
batch_size = SFTConfig.batch_size
gradient_accumulation_steps = SFTConfig.gradient_accumulation_steps
log_iter = SFTConfig.log_iter
max_lr = SFTConfig.max_lr
min_lr = SFTConfig.min_lr
warmup_steps = SFTConfig.warmup_steps
total_steps = len(train_data) // batch_size
optimizer = torch.optim.AdamW(model.parameters(), lr=max_lr)
```

编写日志记录函数

```python
with open(f"log.txt", "a") as my_file:
    my_file.write(f" \
        time:{time.strftime("%Y-%m-%d, %H:%M:%S")}, \
        batch_size:{batch_size}, \
        warmup_steps:{warmup_steps}, \
        max_lr:{max_lr}, \
        min_lr:{min_lr}\n")

#定义一个日志记录函数
def log_call(iters, iters_average_loss):
    with open(f"log.txt", "a") as my_file:
        my_file.write(f" \
            time:{time.strftime("%Y-%m-%d, %H:%M:%S")}, \
            iters:{iters+1}, \
            iters_average_Loss:{iters_average_loss:.4f}\n")
```

代码 10.7 日志相关配置

主训练循环如下

SFT训练循环

```python
model.train()
training_losses = []
model.zero_grad()  # 训练开始时清空梯度
skipped_batches_count = 0

total_batches = len(train_data) // batch_size

for batch_idx in range(total_batches):
    ## ==================== 数据准备阶段 ====================

    # 获取当前批次的原始数据
    current_batch_sequences = train_data[
        batch_idx * batch_size : (batch_idx + 1) * batch_size
    ]

    # 计算当前批次的最大序列长度，用于padding对齐
    max_sequence_length = max([len(sequence) for sequence in
current_batch_sequences])

    ### 对批次数据进行右填充，使所有序列长度一致以便并行计算
    padded_sequences_list = []
    ### 将◁endoftext▷设置为填充符
    pad_token_id = model.generation_config.eos_token_id[-1]

    for seq_idx in range(batch_size):
        # 原始的一条训练数据
        original_sequence = current_batch_sequences[seq_idx]
        # 要填充的长度
        padding_length = max_sequence_length - len(original_sequence)
```

```
29
30          # 使用EOS token进行右填充
31          padded_sequence = torch.nn.functional.pad(
32              torch.tensor(original_sequence),
33              (0, padding_length),
34              mode="constant",
35              value=pad_token_id
36          ).tolist()
37
38          padded_sequences_list.append(padded_sequence)
39
40      # 转换为张量
41      batch_input_tensor = torch.tensor(padded_sequences_list)
42
43      ## ==================== 构建输入输出对 ====================
44
45      # 构建因果语言模型的输入输出对：x→y（下一个词预测）
46      model_inputs = batch_input_tensor[:, :-1].to(device)    # 输入：前n-1个token
47      target_labels = batch_input_tensor[:, 1:].to(device)    # 标签：后n-1个token
48
49      ## ==================== 构建训练掩码 ====================
50
51      # 构建掩码矩阵来控制损失计算范围
52      # 1. padding_mask：标识哪些位置是填充token（不计算损失）
53      # 2. answer_mask：标识哪些位置是助手回答部分（只对回答计算损失）
54
55      ### 【填充掩码】：非填充token为1，填充token为0
56      ### padding_mask中的问题部分的掩码也是1
57      padding_mask = torch.where(target_labels == pad_token_id, 0, 1)
58
59      ### 【回答掩码】：只有助手回答部分为1，其他部分为0
60      assistant_answer_mask = create_answer_mask(model_inputs, tokenizer)
61
62      ### 【组合掩码】：同时满足"非填充"且"是回答部分"的token才计算损失
63      ### 取出交集，就是真正要计算的回答部分
64      final_loss_mask = (assistant_answer_mask & padding_mask)
65
66      ## ==================== 批次有效性检查 ====================
67
68      # 检查当前批次是否有效：如果某个样本的回答部分完全为空，则跳过该批次
69      # 这种情况通常发生在问题过长导致回答部分被截断时
70      tokens_per_sample = final_loss_mask.sum(dim=-1)  # 每个样本的有效回答token数
71      min_answer_tokens = tokens_per_sample.min().item()  # 最少的有效token数
72
73      if min_answer_tokens == 0:
74          print(f"跳过第{batch_idx + 1}批次：回答部分数据不足")
75          skipped_batches_count += 1
76          continue  # 跳过当前批次
77
```

```
    ## ==================== 模型前向传播 ====================

    # 执行前向传播，获取模型预测的logits
    # [batch_size, seq_length, vocab_size]
    model_logits = model(model_inputs).logits

    ## ==================== 损失计算 ====================

    # 计算带掩码的交叉熵损失
    # 步骤：logits → softmax → log → gather → 负对数似然 → 掩码过滤 → 平均

    # 1. 计算每个token的负对数似然损失，
    # 形状：[batch_size, seq_len, vocab_size]
    log_probabilities = torch.log(torch.softmax(model_logits, dim=-1))
    # 使用真正的目标token取出vocab_size长度的数组中token对应的对数概率
    # 形状：[batch_size, seq_len]
    gathered_log_probs = torch.gather(
        log_probabilities,
        dim=-1,
        index=target_labels.unsqueeze(2)
    )
    negative_log_likelihood = gathered_log_probs * (-1)  # 负对数似然
    token_losses = negative_log_likelihood.squeeze(2)

    # 2. 应用掩码并计算每个样本的平均损失
    masked_token_losses = torch.mul(token_losses, final_loss_mask)
    sample_losses = masked_token_losses.sum(dim=-1) \
                    / final_loss_mask.sum(dim=-1)

    # 3. 计算批次平均损失并应用梯度累积
    batch_average_loss = torch.nanmean(sample_losses) \
                         / gradient_accumulation_steps

    ## ==================== 反向传播和优化 ====================

    # 反向传播计算梯度
    batch_average_loss.backward()

    # 动态调整学习率（余弦衰减 + 预热）
    current_learning_rate = cosine_decay(
        batch_idx,
        warmup_steps,
        total_steps,
        max_lr,
        min_lr
    )

    # 更新优化器的学习率
    for param_group in optimizer.param_groups:
```

```
        param_group["lr"] = current_learning_rate

    # 梯度累积：只在累积步数达到或最后一个批次时更新权重
    is_accumulation_step = (batch_idx + 1) \
                           % gradient_accumulation_steps == 0
    is_final_batch = (batch_idx + 1) == total_batches

    if is_accumulation_step or is_final_batch:
        optimizer.step()        # 更新模型权重
        optimizer.zero_grad()   # 清空梯度缓存

    ## ==================== 训练日志记录 ====================

    # 记录当前批次的损失（还原梯度累积的缩放）
    actual_batch_loss =                     \
        batch_average_loss.item()           \
        *                                   \
        gradient_accumulation_steps
    training_losses.append(actual_batch_loss)

    # 定期输出训练进度
    should_log = (batch_idx + 1) % log_iter == 0 or is_final_batch

    if should_log:
        # 计算最近几个批次的平均损失
        recent_losses = training_losses[-log_iter:]
        recent_average_loss = np.nanmean(recent_losses)

        # 输出训练状态
        current_time = time.strftime("%Y-%m-%d %H:%M:%S")
        print(f"时间: {current_time} | "
              f"批次: {batch_idx + 1}/{total_batches} | "
              f"最近{len(recent_losses)}批次平均损失: {recent_average_loss:.4f} | "
              f"学习率: {current_learning_rate:.2e}")

        # 调用外部日志记录函数
        log_call(batch_idx, recent_average_loss)

## ==================== 训练完成总结 ====================

print("训练完成!")
print(f"训练统计:")
print(f"    - 总批次数: {total_batches}")
print(f"    - 跳过批次数: {skipped_batches_count}")
print(f"    - 有效批次数: {total_batches - skipped_batches_count}")
print(f"    - 最终平均损失: {np.nanmean(training_losses[-100:]):.4f}")

if skipped_batches_count > 0:
    skip_ratio = skipped_batches_count / total_batches * 100
```

```
    print(f"跳过批次占比: {skip_ratio:.2f}%")
    if skip_ratio > 10:
        print("建议: 跳过批次过多，考虑增加最大序列长度或优化数据预处理")

model.save_pretrained("./Qwen3-0.6B-SFT/")
tokenizer.save_pretrained("./Qwen3-0.6B-SFT/")
```

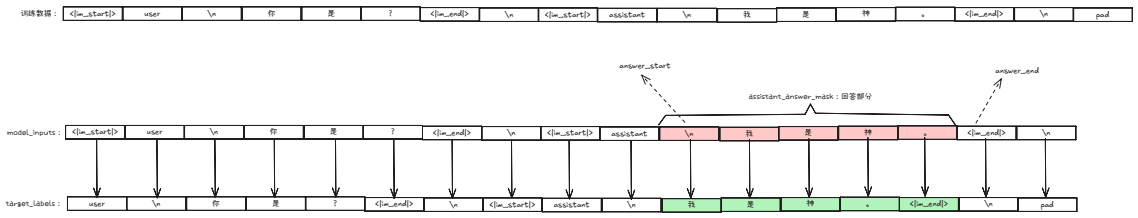


图 10.4 我们是从 model_inputs 中寻找回答部分，然后计算预测的损失

为什么要取交集？也就是代码 final_loss_mask = (assistant_answer_mask & padding_mask)为什么要这么写？

注意到 padding_mask = torch.where(target_labels == pad_token_id, 0, 1)是将 target_labels 中的填充符 pad_token 置为 0。因为我们是右填充逻辑。

在绝大多数标准场景下，这个“取交集”的操作确实是冗余的，但保留它也无伤大雅，甚至是一种防御性编程的好习惯。

下面为你详细拆解为什么它是“冗余”的，以及为什么在某些边缘情况下它又是“必要”的。

1. 为什么理论上是“没必要”的？（冗余性分析）

SFT（有监督微调）的核心逻辑是：只计算 Assistant（助手）回答部分的 Loss。

我们来看看这两个掩码的定义：

- padding_mask：标记哪些是真实内容（1），哪些是填充内容（0）。
- assistant_answer_mask：标记哪些是助手的回答内容（1），其他都是 0。

在标准的右填充（Right-padding）数据处理流程中：

- 所有的 Padding 都在句子的最末尾。
- 所有的 Assistant 回答都在 Padding 之前。
- 结论：assistant_answer_mask 为 1 的区域，必然是真实内容，因此该区域的 padding_mask 必然也是 1。

数学上来说：Answer_Set ⊂ Content_Set。

既然回答集合是内容集合的子集，那么 Answer & Content 自然就等于 Answer。所以取交集不会改变 assistant_answer_mask 本身的值。

2. 为什么保留它也是好的？（防御性编程）

虽然逻辑上冗余，但在工程实践中，保留 & padding_mask 有两个好处：

A. 防止掩码生成函数的 Bug

create_answer_mask 是一个复杂的逻辑函数（涉及查找◁im_start▷，◁im_end▷索引）。如果这个函数写得有瑕疵，比如：

- 索引计算错误，导致掩码划多了，划到了 Padding 区域。
- 数据集本身有脏数据，导致解析错位。

此时，& padding_mask 就像一道安全闸，强制把 Loss 计算限制在非 Padding 区域，防止模型去学习“预测 Padding Token”，避免模型变傻。

B. 处理截断（Truncation）的边缘情况

在代码中，我们使用了 truncation=True。如果一条数据非常长，恰好在 Assistant 回答的一半被截断了，且后面紧接着就是 Padding（这种情况在我们的代码中不会出现！虽然在truncation后通常没有 padding，但也取决于具体实现），双重掩码能确保万无一失。

10.2.2 第二步：使用 DPO 算法对 SFT 后的模型进行微调

> ⚡ 危险
>
> 我们使用 DPO 算法要微调的模型是上一节中保存的“Qwen3-0.6B-SFT”。

加载我们要微调的模型。

```python
from transformers import AutoModelForCausalLM, AutoTokenizer
from dataclasses import dataclass
import datasets
import torch
import numpy as np
import os

device = "cuda"
model_path = "./Qwen3-0.6B-SFT"

# 加载模型
model = AutoModelForCausalLM.from_pretrained(
    model_path,
    dtype="auto",
    device_map="auto"
)
# 冻结的参考模型
ref_model = AutoModelForCausalLM.from_pretrained(
    model_path,
    dtype="auto",
    device_map="auto"
)
# 加载分词器
tokenizer = AutoTokenizer.from_pretrained(model_path)
```

生成回答的配置和 SFT 时相同，见代码 10.4。

定义模型训练的超参数。

```python
@dataclass
class DPOConfig:
    max_length = 1700
    batch_size = 2
    gradient_accumulation_steps = 8
    beta = 0.5 # β是DPO目标函数中的超参数
    log_iter = 100
    max_lr = 1e-6
    min_lr = 1e-7
    warmup_steps:int = 300
```

接下来导入**正负例数据集**。

```python
binarized_data = datasets.load_dataset("./ultrafeedback_binarized")
```

处理数据集

```python
def tokenize_and_format(data):
    input_ids = tokenizer.apply_chat_template(
        data,
        tokenize = True,
        add_generation_prompt = False,
        truncation = True,
        max_length = DPOConfig.max_length,
    )

    return input_ids

## 生成偏好数据的input_ids
chosen_input_ids_list = []
i = 0
while True:
    data = binarized_data["train_sft"][i]["chosen"]
    data.insert(
        0,
        {"content": "You are a helpful assistant", "role": "system"}
    )
    input_ids = tokenize_and_format(data)
    chosen_input_ids_list.append(input_ids)
    i += 1
    if i % 10000 == 0 or i == len(binarized_data["train_sft"]):
        print(f"偏好数据已处理{i}条数据")
    if i == 30000:
        break
print("-" * 70)

################################################################################
## 生成不偏好数据的input_ids
rejected_input_ids_list = []
i = 0
while True:
    data = binarized_data["train_sft"][i]["rejected"]
    data.insert(
        0,
        {"content": "You are a helpful assistant", "role": "system"}
    )
    input_ids = tokenize_and_format(data)
    rejected_input_ids_list.append(input_ids)
    i += 1
    if i % 10000 == 0 or i == len(binarized_data["train_sft"]):
        print(f"非偏好数据已处理{i}条数据")
    if i == 30000:
```

```
        break

## 确保数据条数一致
assert len(chosen_input_ids_list) == len(rejected_input_ids_list)
```

设置一下训练的超参数

```python
beta = DPOConfig.beta # β超参数
batch_size = DPOConfig.batch_size
gradient_accumulation_steps = DPOConfig.gradient_accumulation_steps
log_iter = DPOConfig.log_iter
max_lr = DPOConfig.max_lr
min_lr = DPOConfig.min_lr
warmup_steps = DPOConfig.warmup_steps
total_steps = len(chosen_input_ids_list) // batch_size
optimizer = torch.optim.AdamW(model.parameters(), lr=max_lr)
```

- 配置 `logging` 日志记录模型训练过程参见代码 10.7。
- 复用 SFT 阶段设置的余弦衰减学习率曲线，参见代码 10.5。
- 将回答部分进行掩码的代码，参见代码 10.6。

计算平均对数概率

```python
def _compute_average_log_probability(logits, target_labels, mask):
    """
    计算带掩码的平均对数概率

    Args:
        logits: 模型输出 [batch_size, seq_len, vocab_size]
        target_labels: 目标标签 [batch_size, seq_len]
        mask: 计算掩码 [batch_size, seq_len]

    Returns:
        average_log_prob: 每个样本的平均对数概率 [batch_size]
    """
    # 计算softmax概率分布
    probabilities = torch.softmax(logits, dim=-1)

    # 计算对数概率
    log_probabilities = torch.log(probabilities)

    # 获取目标token的对数概率：
    # $[\log \pi(y_0|x), \log \pi(y_1|x, y_0), \log \pi(y_2|x, y_{<2}), ...]$
    gathered_log_probs = torch.gather(
        log_probabilities,
        dim=-1,
        index=target_labels.unsqueeze(2)
    ).squeeze(2)

    # 应用掩码并计算平均值
    masked_log_probs = torch.mul(gathered_log_probs, mask)

```

```
30      # $\frac{1}{T}(\log\pi(y_0|x)+\log\pi(y_1|x,y_0)+\log\pi(y_2|x,y_{<2})+\cdots)$
31      # $=\frac{1}{T}\left(\sum_{t=0}^{T}\log\pi(y_t|x,y_{<t})\right)=\log\left(\prod_{t=0}^{T}\pi(y_t|x,y_{<t})\right)^{\frac{1}{T}}$
32      # 这就是在提示词为x的条件下，生成一条回答y的概率。用强化学习的术语来说就是一条轨迹的概率。
33      average_log_prob = masked_log_probs.sum(dim=-1) / mask.sum(dim=-1)
34
35      return average_log_prob
```

DPO 训练循环如下：

DPO训练循环 Python

```
1   model.train()
2
3   # ==================== 训练指标记录列表 ====================
4   training_losses = []
5   # 偏好的回答的概率
6   preferred_log_probabilities = []
7   # 讨厌的回答的概率
8   rejected_log_probabilities = []
9   # 偏好的回答的奖励
10  preferred_rewards = []
11  # 讨厌的回答的奖励
12  rejected_rewards = []
13  reward_margins = []
14
15  model.zero_grad()  # 训练开始时清空梯度
16  skipped_batches_count = 0
17  total_batches = len(chosen_input_ids_list) // batch_size
18
19  for batch_idx in range(total_batches):
20      ## ==================== 获取批次数据 ====================
21
22      # 获取当前批次的偏好对数据
23      preferred_batch_sequences = chosen_input_ids_list[
24          batch_idx * batch_size:(batch_idx + 1) * batch_size
25      ]
26      rejected_batch_sequences = rejected_input_ids_list[
27          batch_idx * batch_size:(batch_idx + 1) * batch_size
28      ]
29
30      ## ==================== 数据填充对齐 ====================
31
32      # 计算各自批次的最大序列长度
33      preferred_max_length = max([len(sequence) for sequence in
        preferred_batch_sequences])
34      rejected_max_length = max([len(sequence) for sequence in
        rejected_batch_sequences])
35      # 使用eos token作为pad token
36      pad_token_id = model.generation_config.eos_token_id[-1]
37
38      ### 偏好数据填充处理
```

```
    preferred_padded_sequences = []
    for seq_idx in range(batch_size):
        original_sequence = preferred_batch_sequences[seq_idx]
        # 计算要填充多少个pad
        padding_length = preferred_max_length - len(original_sequence)
        # 在训练数据的末尾填充pad
        padded_sequence = torch.nn.functional.pad(
            torch.tensor(original_sequence),
            (0, padding_length),
            mode="constant",
            value=pad_token_id
        ).tolist()
        # 将填充过的数据放入列表
        preferred_padded_sequences.append(padded_sequence)

    preferred_batch_tensor = torch.tensor(preferred_padded_sequences)

    ### 拒绝数据填充处理
    rejected_padded_sequences = []
    for seq_idx in range(batch_size):
        original_sequence = rejected_batch_sequences[seq_idx]
        padding_length = rejected_max_length - len(original_sequence)

        padded_sequence = torch.nn.functional.pad(
            torch.tensor(original_sequence),
            (0, padding_length),
            mode="constant",
            value=pad_token_id
        ).tolist()

        rejected_padded_sequences.append(padded_sequence)

    rejected_batch_tensor = torch.tensor(rejected_padded_sequences)

    ## ==================== 构建输入输出对 ====================

    # 构建因果语言模型的输入输出对：x→y（下一个词预测）
    # 模型的输入：偏好的回答
    preferred_model_inputs = preferred_batch_tensor[:, :-1].to(device)
    # 真实的标签y_w
    preferred_target_labels = preferred_batch_tensor[:, 1:].to(device)

    rejected_model_inputs = rejected_batch_tensor[:, :-1].to(device)
    # 真实的标签y_l
    rejected_target_labels = rejected_batch_tensor[:, 1:].to(device)

    ## ==================== 构建训练掩码 ====================

    # 构建掩码矩阵：padding_mask（忽略填充token）+ answer_mask（只关注回答部分）
```

```
88
89      # pad_token_id 对应的置为 0 ，其它置为 1 。
90      preferred_padding_mask = torch.where(
91          preferred_target_labels == pad_token_id,
92          0,
93          1
94      )
95      rejected_padding_mask = torch.where(
96          rejected_target_labels == pad_token_id,
97          0,
98          1
99      )
100
101     # 助手回答的掩码：将助手回答的部分掩码为 1 。其它都是 0 。
102     preferred_answer_mask = create_answer_mask(
103         preferred_model_inputs,
104         tokenizer
105     )
106     rejected_answer_mask = create_answer_mask(
107         rejected_model_inputs,
108         tokenizer
109     )
110
111     # 最终掩码：取交集
112     preferred_final_mask = (preferred_answer_mask & preferred_padding_mask)
113     rejected_final_mask = (rejected_answer_mask & rejected_padding_mask)
114
115     ## ==================== 批次有效性检查 ====================
116
117     # 检查偏好对数据是否都有有效的回答部分
118     preferred_min_tokens = preferred_final_mask.sum(dim=-1).min().item()
119     rejected_min_tokens = rejected_final_mask.sum(dim=-1).min().item()
120
121     if preferred_min_tokens == 0 or rejected_min_tokens == 0:
122         print(f"跳过第{batch_idx + 1}批次：偏好对数据回答部分不足")
123         skipped_batches_count += 1
124         continue  # 跳过当前批次
125
126     ## ==================== 模型前向传播 ====================
127
128     # 训练模型对偏好数据的前向传播
129     preferred_logits = model(preferred_model_inputs).logits
130     torch.cuda.empty_cache()  # 清理GPU显存
131     torch.cuda.ipc_collect()
132
133     # 训练模型对拒绝数据的前向传播
134     rejected_logits = model(rejected_model_inputs).logits
135     torch.cuda.empty_cache()  # 清理GPU显存
136     torch.cuda.ipc_collect()
```

```
137
138     # 参考模型的前向传播（不计算梯度）
139     with torch.no_grad():
140         reference_preferred_logits = ref_model(preferred_model_inputs) \
141             .logits                                                    \
142             .detach()
143         reference_rejected_logits = ref_model(rejected_model_inputs)   \
144             .logits                                                    \
145             .detach()
146
147     ## ==================== DPO损失计算 ====================
148     """
149     DPO (Direct Preference Optimization) 论文: https://arxiv.org/pdf/2305.18290.pdf
150     核心思想：通过偏好对比学习，无需显式奖励模型
151     """
152
153     # 计算平均对数概率 (average_log_prob = True)
154     # 参考: https://github.com/huggingface/trl/blob/main/trl/trainer/dpo_trainer.
        py#L924
155
156     ### 训练模型的对数概率
157     ### 正在微调的模型，接收到正例的logits，计算对数概率
158     ### log π_θ(y_w|x)
159     preferred_log_prob = _compute_average_log_probability(
160         preferred_logits,
161         preferred_target_labels,
162         preferred_final_mask
163     )
164     ### log π_θ(y_l|x)
165     rejected_log_prob = _compute_average_log_probability(
166         rejected_logits,
167         rejected_target_labels,
168         rejected_final_mask
169     )
170
171     ### 参考模型的对数概率
172     ### log π_ref(y_w|x)
173     reference_preferred_log_prob = _compute_average_log_probability(
174         reference_preferred_logits,
175         preferred_target_labels,
176         preferred_final_mask
177     )
178     ### log π_ref(y_l|x)
179     reference_rejected_log_prob = _compute_average_log_probability(
180         reference_rejected_logits,
181         rejected_target_labels,
182         rejected_final_mask
183     )
184
185     # β log π_θ(y_w|x) / π_ref(y_w|x)
```

```
    preferred_implicit_reward =                                    \
        beta *                                                     \
        (preferred_log_prob - reference_preferred_log_prob)
    # β log πθ(yl|x) / πref(yl|x)
    rejected_implicit_reward =                                     \
        beta *                                                     \
        (rejected_log_prob - reference_rejected_log_prob)

    # β( log πθ(yw|x) / πref(yw|x) − log πθ(yl|x) / πref(yl|x) )
    reward_margin = preferred_implicit_reward - rejected_implicit_reward

    # DPO损失：
    # −log( σ( β( log πθ(yw|x) / πref(yw|x) − log πθ(yl|x) / πref(yl|x) ) ) )
    preference_probability = torch.nn.functional.sigmoid(reward_margin)
    sample_losses = -torch.log(preference_probability)

    # 批次平均损失 + 梯度累积
    batch_average_loss =                                \
        torch.nanmean(sample_losses) /                  \
        gradient_accumulation_steps

    ## ==================== 反向传播和优化 ====================

    batch_average_loss.backward()

    # 动态学习率调整
    current_learning_rate = cosine_decay(
        batch_idx,
        warmup_steps,
        total_steps,
        max_lr,
        min_lr
    )

    for param_group in optimizer.param_groups:
        param_group["lr"] = current_learning_rate

    # 梯度累积和权重更新
    is_accumulation_step = (batch_idx + 1) % gradient_accumulation_steps == 0
    is_final_batch = (batch_idx + 1) == total_batches

    if is_accumulation_step or is_final_batch:
        optimizer.step()        # 更新权重
        optimizer.zero_grad()   # 清空梯度

    ## ==================== 训练指标记录 ====================

    # 记录各项训练指标（detach避免梯度追踪）
    training_losses.append(
```

```
235             batch_average_loss.detach().item() * gradient_accumulation_steps)
236         preferred_log_probabilities.append(
237             torch.nanmean(preferred_log_prob.detach()).item())
238         rejected_log_probabilities.append(
239             torch.nanmean(rejected_log_prob.detach()).item())
240         preferred_rewards.append(
241             torch.nanmean(preferred_implicit_reward.detach()).item())
242         rejected_rewards.append(torch.nanmean(
243             rejected_implicit_reward.detach()).item())
244         reward_margins.append(
245             torch.nanmean(reward_margin.detach()).item())
246 
247         ## ==================== 训练日志输出 ====================
248 
249         should_log = (batch_idx + 1) % log_iter == 0 or is_final_batch
250 
251         if should_log:
252             # 计算最近批次的平均指标
253             recent_loss = np.nanmean(training_losses[-log_iter:])
254             recent_preferred_logprob = np.nanmean(
255                 preferred_log_probabilities[-log_iter:])
256             recent_rejected_logprob = np.nanmean(
257                 rejected_log_probabilities[-log_iter:])
258             recent_preferred_reward = np.nanmean(preferred_rewards[-log_iter:])
259             recent_rejected_reward = np.nanmean(rejected_rewards[-log_iter:])
260             recent_margin = np.nanmean(reward_margins[-log_iter:])
261 
262             # 格式化输出训练状态
263             current_time = time.strftime("%Y-%m-%d %H:%M:%S")
264             print(f"时间: {current_time}")
265             print(f"批次: {batch_idx + 1}/{total_batches}")
266             print(f"最近{log_iter}批次指标：")
267             print(f"   - 平均损失: {recent_loss:.4f}")
268             print(f"   - 偏好对数概率: {recent_preferred_logprob:.4f}")
269             print(f"   - 拒绝对数概率: {recent_rejected_logprob:.4f}")
270             print(f"   - 偏好奖励: {recent_preferred_reward:.4f}")
271             print(f"   - 拒绝奖励: {recent_rejected_reward:.4f}")
272             print(f"   - 奖励边际: {recent_margin:.4f}")
273             print(f"学习率: {current_learning_rate:.2e}")
274             print("-" * 80)
275 
276             # 调用外部日志记录
277             log_call(batch_idx, recent_loss)
278 
279 ## ==================== 训练完成总结 ====================
280 
281 print("DPO训练完成！")
282 print(f"训练统计：")
283 print(f"   - 总批次数: {total_batches}")
```

```
print(f"   - 跳过批次数: {skipped_batches_count}")
print(f"   - 有效批次数: {total_batches - skipped_batches_count}")

# 输出最终训练指标
if training_losses:
    final_metrics = {
        "loss": np.nanmean(training_losses[-100:]),
        "preferred_logprob": np.nanmean(preferred_log_probabilities[-100:]),
        "rejected_logprob": np.nanmean(rejected_log_probabilities[-100:]),
        "preferred_reward": np.nanmean(preferred_rewards[-100:]),
        "rejected_reward": np.nanmean(rejected_rewards[-100:]),
        "margin": np.nanmean(reward_margins[-100:])
    }

    print(f"最终指标 (最近100批次平均): ")
    for metric_name, metric_value in final_metrics.items():
        print(f"   - {metric_name}: {metric_value:.4f}")

if skipped_batches_count > 0:
    skip_ratio = skipped_batches_count / total_batches * 100
    print(f"跳过批次占比: {skip_ratio:.2f}%")
    if skip_ratio > 10:
        print("建议: 跳过批次过多，考虑增加最大序列长度或优化数据预处理")
```

11. 使用 PPO 微调大语言模型--复刻 InstructGPT

11.1 InstructGPT 训练流程

- Step-1：SFT，Supervised Fine-Tuning，有监督微调。顾名思义，它是在有监督（有标注）数据上微调训练得到的。这里的监督数据其实就是输入 Prompt，输出相应的回复，只不过这里的回复是人工编写的。这个工作要求比一般标注要高，其实算是一种创作了。
- Step-2：RM，Reward Model，奖励模型。具体来说，一个 Prompt 丢给前一步的 SFT，输出若干个（4-9 个）回复，由标注人员对这些回复进行排序。然后从 4-9 个中每次取 2 个，因为是有序的，就可以用来训练这个奖励模型，让模型学习到这个好坏评价。这一步非常关键，它就是所谓的 Human Feedback，引导下一步模型的进化方向。
- Step-3：RL，Reinforcement Learning，强化学习，使用 PPO 策略进行训练。PPO，Proximal Policy Optimization，近端策略优化，是一种强化学习优化方法，它背后的主要思想是避免每次太大的更新，提高训练的稳定性。具体过程如下：首先需要初始化一个语言模型，然后丢给它一个 Prompt，它生成一个回复，上一步的奖励模型给这个回复一个打分，这个打分回传给模型更新参数。这里的这个模型在强化学习视角下就是一个策略。这一步有个很重要的动作，就是更新模型时会考虑模型每一个 token 的输出和第一步 SFT 输出之间的差异性，要让它俩尽量相似。这是为了缓解强化学习可能的过度优化。

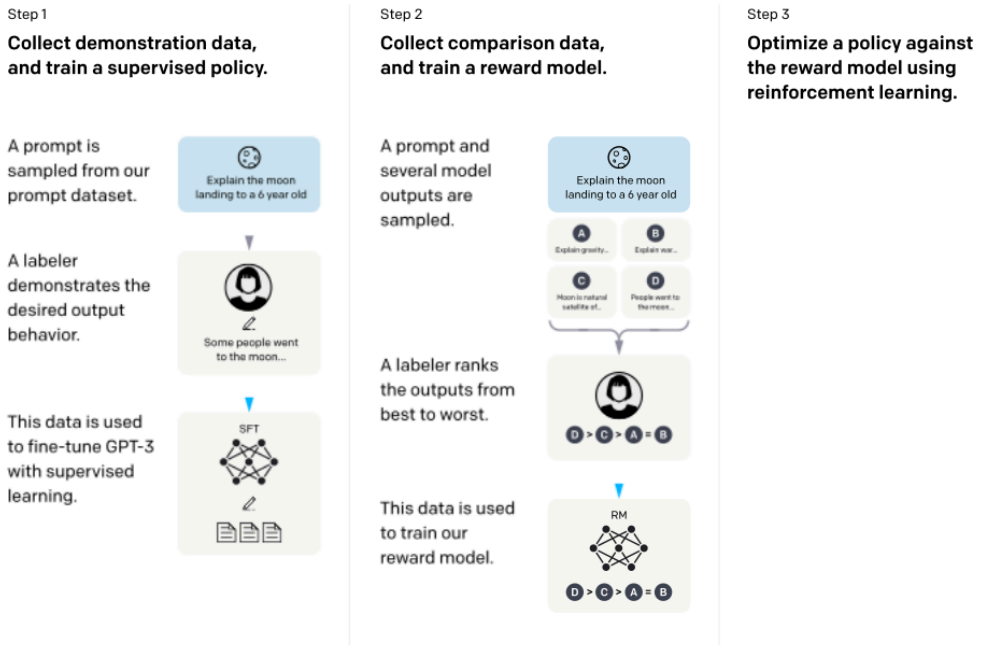


图 11.1 InstructGPT 训练流程

11.2 即时奖励R_t的计算

$$J_{\text{PPO}}(\theta) = \mathbb{E}\left[\min\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)}A, \text{clip}\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)}, 1-\varepsilon, 1+\varepsilon\right)A\right)\right] \tag{11.1}$$

PPO 的目标函数我们已经很熟悉了，优势A如果使用 1 步 TD 误差的话是：$\delta = R_t + \gamma V(S_{t+1}) - V(S_t)$。

在倒立摆环境中，只要木杆不倒下，那么$R_t = 1$。但是在大语言模型这个环境中，情况就要复杂多了。在大语言模型中，策略模型 LLM 采取的动作是输出一个 token，那么输出一个 token，我们应该给什么奖励R_t呢？

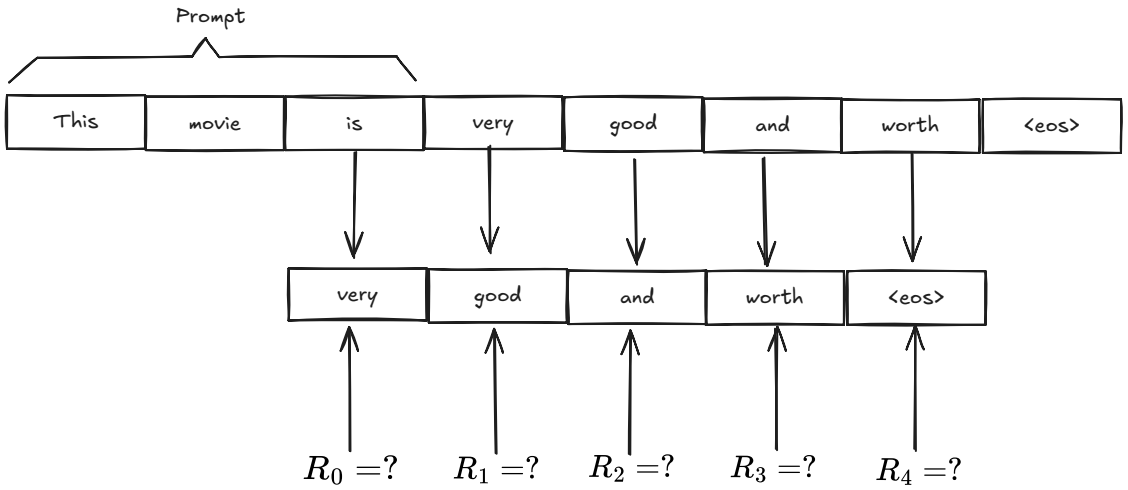


图 11.2 输出一个 token（采取动作），针对输出的这个 token，怎么给即时奖励R_t？

在 RLHF 中，奖励模型（Reward Model）的作用是针对一条完整的补全给出分数的。

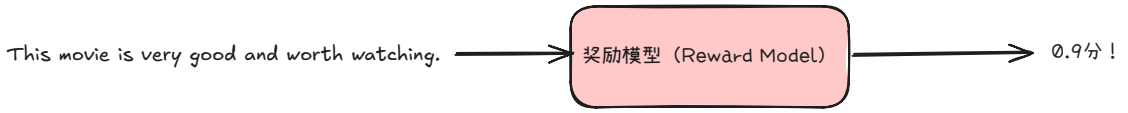


图 11.3 奖励模型是针对一条完整的补全给出得分的

在 InstructGPT (Ouyang et al., 2022) 的 PPO 阶段，奖励函数的设计非常精妙。它不仅仅是奖励模型（Reward Model, RM）给出的分数，还包含了一个至关重要的惩罚项。

即时奖励的计算公式分两种情况：

- <eos_token>之前的 token 的奖励如下计算：

$$R_t = -\beta \log \frac{\pi_\theta(y_t|x, y_{<t})}{\pi_{\text{ref}}(y_t|x, y_{<t})} \tag{11.2}$$

- 最后一个 token 也就是<eos_token>的奖励如下：

$$R_{<\text{eos_token}>} = R_T = \textbf{奖励模型给的得分} - \beta \log \frac{\pi_\theta(y_T|x, y_{<T})}{\pi_{\text{ref}}(y_T|x, y_{<T})} \tag{11.3}$$

InstructGPT 成功的关键秘诀是式子中的 KL 散度惩罚项$\beta \log \frac{\pi_\theta(y_t|x, y_{<t})}{\pi_{\text{ref}}(y_t|x, y_{<t})}$。

- 直观理解：我们不希望强化学习后的模型π_θ偏离原始的 SFT 模型π_{ref}太远。
- 为什么需要 KL 散度惩罚项？
  - 防止 Reward Hacking（奖励黑客）：如果没有约束，RL 模型会利用奖励模型的漏洞，生成一些人类读起来不通顺、但奖励模型误判为高分的奇怪句子（例如重复乱码）。
  - 保持语言流畅性：SFT 模型通常能生成通顺的自然语言。通过锚定 SFT 模型，保证了 RL 模型生成的句子依然是“人话”。
- 计算细节：这个惩罚是**逐 token 计算**的，也就是如果π_θ对某个 token 的生成概率远高于π_{ref}，说明模型在“冒险”偏离原来的轨道，惩罚就会变大。
- 奖励模型的打分鼓励模型往“人类喜欢”的方向走。
- KL 惩罚拽住模型，不让它跑得太偏，避免它为了高分而变成“怪兽”。

在标准的 InstructGPT 实现中，**奖励模型（RM）的分数通常只加在最后一个 token 上，而 KL 惩罚是加在每一个 token 上的**。

让我们把这个过程像切蛋糕一样切开来看：

1. 奖励的时间步分配

假设模型生成了一个长度为 T 的句子：$y = [y_1, y_2, ..., y_T]$。

在 PPO 的每一个时间步 t，智能体获得的即时奖励 R_t 是这样计算的：

时间步 (token)	即时奖励 R_t 的构成	解释
中间 token($t < T$)	只有 KL 惩罚 $R_t = -\beta \log \frac{\pi_\theta(y_t\|x, y_{<t})}{\pi_{\text{ref}}(y_t\|y_{<t})}$	此时句子还没写完，奖励模型无法打分。我们只关心这一步有没有“偏离初心”（KL 散度）。
最后一个 token($t = T$)	KL 惩罚+RM 总分 $R_T = -\beta \log \frac{\pi_\theta(y_t\|x, y_{<t})}{\pi_{\text{ref}}(y_t\|y_{<t})}$ + **奖励模型给的分数**	句子结束（遇到<eos_token>），奖励模型 RM 终于看完了整句话，给出一个得分，叠加在最后一步上。

2. 为什么要这样做？

你可能会问：“如果只奖励最后一个词，前面的词怎么知道自己做得好不好？”

虽然物理上的奖励（奖励模型给的得分）只发生在最后一步，但 PPO 使用了广义优势估计（GAE）来回传这个信号：

1. **价值函数的预测**：Critic 网络（价值网络）会预测每一个 token 的 $V(S_t)$。它会学到：“虽然我现在只拿到了微小的 KL 惩罚，但我知道只要我顺着这个方向走，最后能拿到巨大的**奖励模型给的得分**。”
2. **信号回传**：通过折扣因子 γ 和 GAE 参数 λ，最后一步的巨大奖励会沿着时间轴**向前传播**。
   - 如果奖励模型给的得分很高，那么 $y_{T-1}, y_{T-2}...$ 的优势函数（Advantage）都会变高。
   - 这就告诉了前面的 token：“你们铺垫得很好，导致最后结局很棒。”
3. 举个具体的例子

- Prompt: “你好”
- Response: “你好呀<eos_token>” (4 个 token)

假设 $\beta = 0.1$，奖励模型 RM 给整句打分 1.0。

- token 1 (“你”):
  - $R_1 = -0.1 \times \text{KL}(\textbf{你})$
  - （没有 RM 分数）
- token 2 (“好”):
  - $R_2 = -0.1 \times \text{KL}(\textbf{好})$
  - （没有 RM 分数）
- token 3 (“呀”):
  - $R_3 = -0.1 \times \text{KL}(\textbf{呀})$
  - （没有 RM 分数）
- token 4 (“<eos_token>”):
  - $R_4 = -0.1 \times \text{KL}(<\text{eos_token}>) + 1.0$
  - （这里加上了奖励模型 RM 的最终大奖）

💡 总结

- 奖励模型 RM 给的分数是“稀疏奖励”（Sparse Reward）：只在结局出现一次。
- KL 惩罚是“稠密奖励”（Dense Reward）：每一步都有，时刻约束模型不要乱说话。

- 依靠 GAE：把最后的稀疏奖励“抹匀”分摊给前面的每一个动作，让前面的词也能获得梯度更新。

11.3 PPO 微调 LLM 实战

应用场景：在电商场景中，很多商品由于没有人买而变成了“长尾商品”。而我们需要为这些长尾商品“自动”生成一些**正向**评论，来吸引买家。所以需要训练一个能够编写正向评论的 LLM。

1. 首先对一个预训练 LLM（gpt2, Qwen2.5-0.5B 等）进行 SFT，让 LLM 可以编写商品评论。这里的问题在于这个 LLM 虽然能够写电商评论，但写出来的可能是正向评论也可能是负向评论。
2. 训练一个奖励模型（Reward Model），可以针对正向评论打高分，针对负向评论打低分。
3. 使用奖励模型给 SFT 后的 LLM 输出的电商评论打分，如果分数高（说明是正向评论），则鼓励 SFT 后的 LLM 输出这条评论（提升输出这条评论的概率）。如果分数低（说明是负向评论），则抑制 SFT 模型输出这条评论的概率。

11.3.1 监督微调（SFT）

我们的目标是训练一个可以写电商评论的大语言模型。所以我们对 gpt2 进行微调，让它可以写电商评论。

由于我们使用的文本，既有正向情感的文本，也有负向情感的文本，所以训练出来的模型，在给定提示词（例如：“这本书”）时，可能写出正向情感的评论也可能写出负向情感的评论。

> 🔥 预训练模型
>
> 我们使用的底座模型是中文版 gpt2：“gpt2-chinese-cluecorpussmall”。

对gpt2进行SFT — Python

```python
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM, DataCollatorForLanguageModeling, pipeline, set_seed
from torch.utils.data import DataLoader
from datasets import load_dataset
from pprint import pprint

dataset = load_dataset("csv", data_files="online_shopping_10_cats.csv")
model_path = "./gpt2-chinese-cluecorpussmall"

ds_train = dataset["train"]
# 将评论长度为大于1024的过滤掉，因为gpt2的最大上下文长度为1024。
ds_train = ds_train.filter(lambda x : x["review"] != None and len(x["review"]) > 20 and len(x["review"]) < 1024)

tokenizer = AutoTokenizer.from_pretrained(model_path)
model = AutoModelForCausalLM.from_pretrained(model_path)

def tokenize(batch):
    return tokenizer(batch["review"])

map_kwargs = {
    "batched": True,
    "batch_size": 512,
    "remove_columns": ["cat", "label", "review"]
```

```
}

tokenized_dataset_train = ds_train.map(tokenize, **map_kwargs)

tokenized_dataset_train.set_format(type="torch")
# 将eos_token设置为pad_token
tokenizer.eos_token = tokenizer.pad_token
# 将mlm设置为False，那么数据会被整理为因果注意力模型使用的格式，
# 也就是预测下一个token任务需要的数据格式
data_collator = DataCollatorForLanguageModeling(
    tokenizer,
    mlm=False
)

dataloader_params = {
    "batch_size": 2,
    "collate_fn": data_collator
}

train_dataloader = DataLoader(
    tokenized_dataset_train,
    **dataloader_params
)

optimizer = torch.optim.AdamW(model.parameters(), lr=5e-5)
# 一般sft会训练1个epoch，也就是把训练数据看一遍就可以了
# 否则容易过拟合，造成灾难性遗忘
num_epochs = 1

device = torch.device("cuda")
model.to(device)
for epoch in range(num_epochs):
    model.train()
    for i, batch in enumerate(train_dataloader):
        batch = batch.to(device)
        outputs = model(**batch)
        loss = outputs.loss
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        if i % 100 == 0:
            print(f"Step: {i}, Loss: {loss.item()}")

model.save_pretrained("./gpt2-sft")
tokenizer.save_pretrained("./gpt2-sft")

# 测试微调后的模型
g = pipeline("text-generation", model="./gpt2-sft")
set_seed(42)
pprint(g("这本书真是", max_length=30, num_return_sequences=10))
```

11.3.2 训练奖励模型（Reward Model）

我们会将 gpt2 预训练模型微调成一个分类模型（奖励模型）。

我们要训练一个模型作为奖励模型，也就是给模型输入文本，模型可以给一个评分出来。

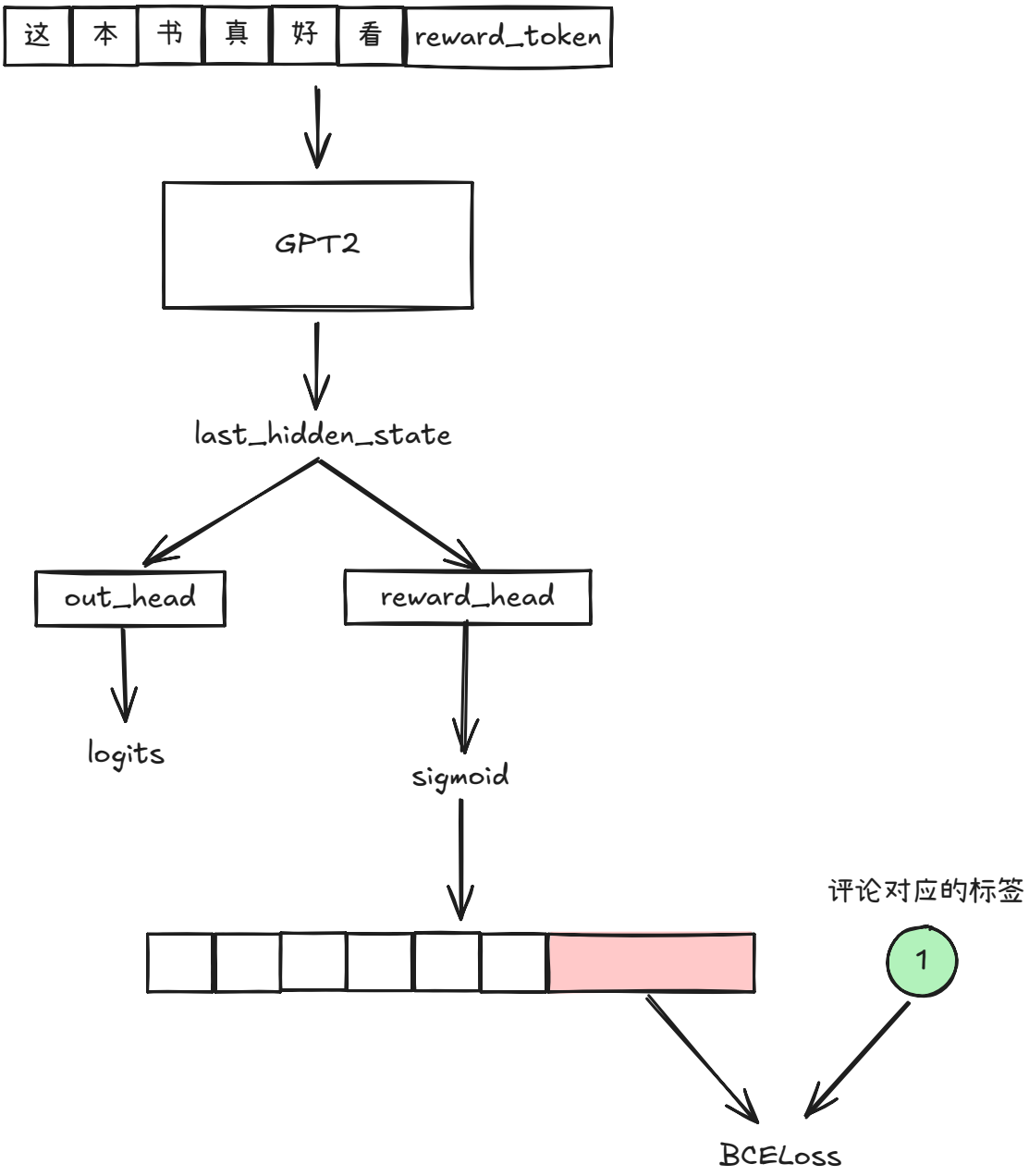


图 11.4 奖励模型训练流程示意图

我们的评论在输入奖励模型之前会先在末尾添加一个 reward_token，作为标记。作用和 Bert 用来训练分类模型时添加的 CLS_TOKEN 是一样的。

我们将评论输入 gpt2 模型，然后提取 gpt2 输出的最后一层隐藏层（last_hidden_state），并送入一个我们自己定义的线性层（reward_head），然后取输出的最后一个元素作为评分。

```python
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForCausalLM,
DataCollatorWithPadding
```

```
import torch
from torch import nn
import numpy as np
from torch.utils.data import DataLoader
from sklearn.metrics import confusion_matrix

model_path = "./gpt2-chinese-cluecorpussmall"
tokenizer = AutoTokenizer.from_pretrained(model_path)

tokenizer.eos_token = tokenizer.pad_token
# reward_token设置为eos_token
REWARD_TOKEN_ID = tokenizer.eos_token_id

ds = load_dataset("csv", data_files="online_shopping_10_cats.csv")
ds_train = ds["train"]

ds_train = ds_train.filter(lambda x : x["review"] != None and len(x["review"]) > 20 and len(x["review"]) < 1024)

def tokenize(batch):
    # 提取出文本内容
    outputs = tokenizer(batch["review"])
    # 每条数据一个评分，初始化为 0 。
    outputs["score"] = [0] * len(outputs["input_ids"])
    # 对每条数据的最后的reward_token进行评分
    outputs["score_index"] = [0] * len(outputs["input_ids"])
    for i in range(len(outputs["input_ids"])):
        # 第 i 条数据的末尾添加一个eos_token，作为reward_token
        outputs["input_ids"][i].append(REWARD_TOKEN_ID)
        # reward_token的掩码设置为1。
        outputs["attention_mask"][i].append(1)
        # 正向情感的文本评分为1。负向情感的评分为0。
        # 也就是使用数据集中的标签作为评分。
        outputs["score"][i] = float(batch["label"][i])
        # 对 reward_token 进行评分，也就是评分的索引为 reward_token 的索引。
        outputs["score_index"][i] = len(outputs["input_ids"][i]) - 1
    return outputs

map_kwargs = {
    "batched": True,
    "batch_size": 512,
    "remove_columns": ["cat", "label", "review"]
}

tokenized_dataset_train = ds_train.map(tokenize, **map_kwargs)

tokenized_dataset_train.set_format(type="torch")

class RewardModel(nn.Module):
    """奖励模型的结构"""
```

```
52      def __init__(self, model_name):
53          super().__init__()
54          self.llm = AutoModelForCausalLM.from_pretrained(model_name)
55          self.reward_head = nn.Linear(self.llm.config.hidden_size, 1)
56
57      def forward(self, input_ids, attention_mask):
58          # gpt2的前向传播，但是还要输出隐藏层
59          transformer_outputs = self.llm.forward(
60              input_ids=input_ids,
61              attention_mask=attention_mask,
62              output_hidden_states=True
63          )
64          # 提取最后一层隐藏层
65          last_hidden_state = transformer_outputs.hidden_states[-1]
66          # 使用reward_head线性层对last_hidden_state给出奖励
67          reward = self.reward_head(last_hidden_state).squeeze(-1)
68          # sigmoid用来将奖励搞到(0,1)范围内
69          return torch.sigmoid(reward)
70
71  model = RewardModel(model_path)
72
73  data_collator = DataCollatorWithPadding(tokenizer)
74
75  dataloader_params = {
76      "batch_size": 16,
77      "shuffle": True,
78      "collate_fn": data_collator
79  }
80
81  train_dataloader = DataLoader(
82      tokenized_dataset_train,
83      **dataloader_params
84  )
85
86  device = torch.device("cuda")
87
88  optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4)
89  # 二分类交叉熵损失
90  criterion = nn.BCELoss()
91  # 这里训练1轮，遵循InstructGPT论文中的训练设置。
92  num_epochs = 1 # N+ Implementation Detail paper
93
94  model.to(device)
95
96  for epoch in range(num_epochs):
97      model.train()
98      for i, batch in enumerate(train_dataloader):
99          inputs = batch.to(device)
100         model_inputs = {
101             "input_ids": inputs["input_ids"],
```

```
            "attention_mask": inputs["attention_mask"]
        }
        # 模型针对训练数据的打分
        scores = model(**model_inputs)
        batch_indices = torch.arange(scores.shape[0])
        # 模型对reward_token的打分
        score = scores[batch_indices, inputs["score_index"]]
        # 真实分数：0或者1
        target = inputs["score"]
        loss = criterion(score, target)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        print("Loss: ", loss.item())

torch.save(model.state_dict(), "reward_model.pt")

##########################################
############ 对模型进行评估 ##############
##########################################

model.eval()

all_predictions = []
all_labels = []

for i, batch in enumerate(train_dataloader):
    inputs = batch.to(device)
    model_inputs = {
        "input_ids": inputs["input_ids"],
        "attention_mask": inputs["attention_mask"]
    }
    with torch.no_grad():
        scores = model(**model_inputs)
        batch_indices = torch.arange(scores.shape[0])
        score = scores[batch_indices, inputs["score_index"]]
        target = inputs["score"]
    # 对评论的打分大于0.5分，视为预测为1。
    predictions = (score > 0.5).int()

    all_predictions.extend(predictions.cpu().numpy())
    all_labels.extend(target.cpu().numpy())

# 计算混淆矩阵
print(confusion_matrix(all_labels, all_predictions))
```

结果的格式如下：

```
混淆矩阵:
[[364,  60],   # 第一行：真实标签为0的情况：364个预测正确，60个预测错误
```

```
3   [ 31, 412]]   # 第二行：真实标签为1的情况：31个预测错误，412个预测正确
```

11.3.3 对 gpt2-sft 进行 PPO 微调

11.3.3.1 导入依赖

```python
from transformers import DataCollatorWithPadding, AutoModelForCausalLM, AutoTokenizer
from copy import deepcopy
from torch.utils.data import DataLoader
import random
from datasets import load_dataset
import torch
import torch.nn as nn
import torch.nn.functional as F
import numpy as np

model_path = "./gpt2-sft"
tokenizer = AutoTokenizer.from_pretrained(model_path)
tokenizer.pad_token = tokenizer.eos_token
```

11.3.3.2 准备提示词

我们需要搞一些提示词，给到 gpt2-sft，让它补全。补全的内容交给奖励模型评分。然后决定惩罚 gpt2-sft 还是奖励 gpt2-sft。

> ⚡ 危险
>
> 自己编写 prompt 太麻烦了，所以我们准备提示词的方式，是把数据集中文本的前几个字截取出来作为提示词。**后面的文本不再使用！**
>
> > 🔥 截取前几个字作为提示词
> >
> > 东西太小了，光有数量没有质量！很失望！

为PPO准备提示词

```python
ds = load_dataset("csv", data_files="online_shopping_10_cats.csv")
ds_train = ds["train"]

ds_train = ds_train.filter(lambda x: x["review"] != None and len(
    x["review"]) > 20 and len(x["review"]) < 1024)

# 截取评论数据的前2～8个token作为提示词
input_min_token_length = 2
input_max_token_length = 8
input_token_length_range = list(range(
    input_min_token_length,
    input_max_token_length))

def tokenize(sample):
    # 提示词token的数量随机选择一个
```

```python
    input_size = random.choice(input_token_length_range)
    # 如果input_size=3，截取review字段文本的前3个token出来
    sample["input_ids"] = tokenizer.encode(sample["review"])[:input_size]
    # 提示词掩码为1
    sample["attention_mask"] = [1] * len(sample["input_ids"])
    # 提示词对应的文本
    sample["query"] = tokenizer.decode(sample["input_ids"])
    return sample


map_kwargs = {
    "batched": False,
    "remove_columns": ["cat", "review", "label"]
}

tokenized_dataset_train = ds_train.map(tokenize, **map_kwargs)

tokenized_dataset_train.set_format(type="torch")

batch_size = 32

def collator(batch):
    return dict((key, [d[key] for d in batch]) for key in batch[0])

# 提示词组成的数据集
train_dataloader = DataLoader(
    tokenized_dataset_train,
    batch_size=batch_size,
    collate_fn=collator,
    shuffle=True
)
```

11.3.3.3 ActorCritic 模型结构

在倒立摆环境中，我们的 Actor 是 PolicyNet，Critic 是 ValueNet。

在 InstructGPT 中，演员模型和价值模型合并成了一个模型 ActorCriticModel。这样 actor 和 critic 可以共享 gpt2-sft 模型的权重。

演员评论家模型 Python

```python
class ActorCriticModel(nn.Module):
    """GPT2模型+价值头"""
    def __init__(self, model_path):
        super().__init__()
        # 这个要初始化为我们微调出来的gpt2-sft模型
        # actor演员模型：策略模型$\pi_\theta$
        self.llm = AutoModelForCausalLM.from_pretrained(model_path)
        # 添加价值头$V_\omega$
        # critic评论家模型：价值函数模型，价值头，线性层
        self.v_head = nn.Linear(self.llm.config.hidden_size, 1)

```

```python
    def forward(self, input_ids, attention_mask):
        # gpt2-sft模型的输出
        transformer_outputs = self.llm.forward(
            input_ids,
            attention_mask=attention_mask,
            output_hidden_states = True,
        )
        # 输出的token的logits
        logits = transformer_outputs.logits
        # 获取最后一层隐藏层
        last_hidden_state = transformer_outputs.hidden_states[-1]

        # 评估token的价值，评估的是最后一个隐藏层的价值
        value = self.v_head(last_hidden_state).squeeze(-1)
        # 返回输出的token的logits和token的价值
        return logits, value

    def generate(self, *args, **kwargs):
        """生成文本"""
        return self.llm.generate(*args, **kwargs)
```

11.3.3.4 加载模型

将需要的模型都加载到显存 Python

```python
device = torch.device("cuda")

# 将奖励模型加载
reward_model = RewardModel(model_path)
reward_model.load_state_dict(torch.load(
    "reward_model.pt",
    map_location="cpu"))
# 加载演员评论家模型
model = ActorCriticModel(model_path)
model = model.to(device)
reward_model = reward_model.to(device)
# 加载冻结的参考模型$\pi_{\text{ref}}$，只用来计算即时奖励$R_t$
ref_model = deepcopy(model)

# 设置reward_token
REWARD_TOKEN_ID = tokenizer.eos_token_id
```

11.3.3.5 奖励R_t的计算

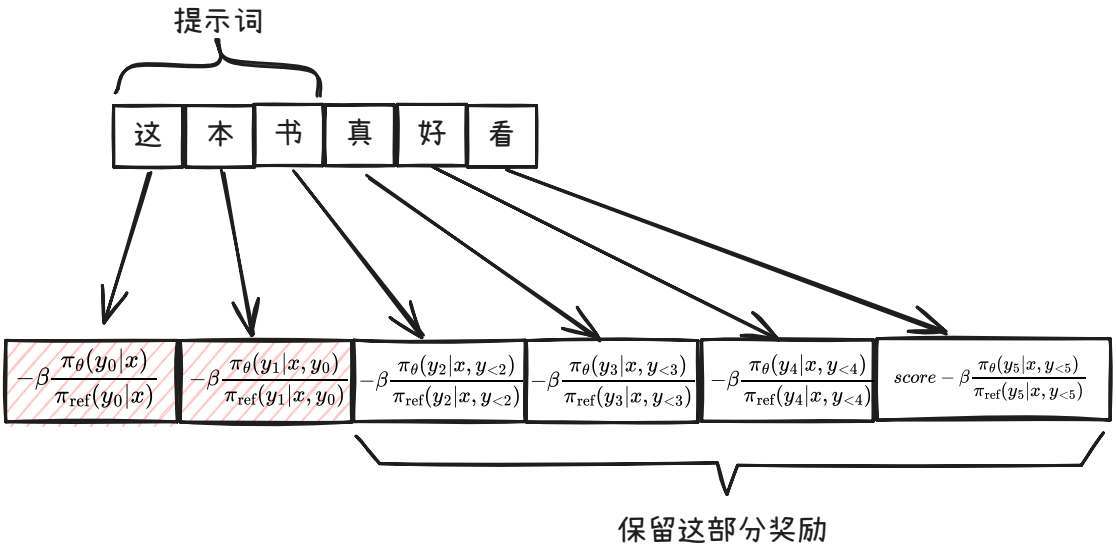


图 11.5 提示词为“这本书”，补全为“真好看”的R_t的计算

下面的函数 compute_rewards 在计算输出的**每个 token** 的R_t的同时，**顺便**计算了每个 token 的价值，也就是$V(S_t)$。

Python

```python
def compute_rewards(
    input_data, # 输入数据（提示词+补全）
    query_tensors, # 提示词张量
    response_tensors, # 补全的张量
    score_tensors # 奖励模型给出的分数的张量
):
    with torch.no_grad():
        # 正在微调的模型所输出的token的logits和token的价值（$[V(S_0), V(S_1), ..., V(S_T)]$）
        logits, values = model(**input_data) # (B, T, vocab_size)
        # 冻结的参考模型的输出
        ref_logits, _ = ref_model(**input_data)
        # 正在微调的模型的输出的对数概率log_softmax
        # 去掉最后一个token，因为是预测下一个token的任务
        logp = F.log_softmax(logits[:, :-1, :], dim=-1)
        # 冻结的模型的输出的对数概率
        ref_logp = F.log_softmax(ref_logits[:, :-1, :], dim=-1)
        # 实际生成的token序列
        # 自回归模型是预测下一个token，所以去掉第一个token
        labels = input_data["input_ids"][:, 1:] # (B, T-1)
        # 使用gather提取实际token的概率
        # logp 是 vocab_size 大小的张量
        logp = torch.gather(logp, 2, labels.unsqueeze(-1)).squeeze(-1) # (B, T-1)
        ref_logp = torch.gather(ref_logp, 2, labels.unsqueeze(-1)).squeeze(-1) # (B, T-1)
        # KL散度：$\left[\log\frac{\pi_\theta(y_0|x)}{\pi_{\text{ref}}(y_0|x)}, \log\frac{\pi_\theta(y_1|x,y_0)}{\pi_{\text{ref}}(y_1|x,y_0)}, ...\right]$
        kl = logp - ref_logp
```

```
        # kl散度的权重β
        beta = 0.2
        # 最终奖励的计算的右边式子：[−β log πθ(y0|x)/πref(y0|x), −β log πθ(y1|x,y0)/πref(y1|x,y0), ...]
        rewards = - beta * kl
        attention_mask = input_data["attention_mask"]
        # 预测下一个token，所以去掉第一个mask
        masks = torch.zeros_like(attention_mask[:, 1:])
        masks[:,:] = attention_mask[:, 1:]
        # 遍历批次中的每一个提示词张量
        for j in range(len(query_tensors)):
            # 补全开始的索引
            start = len(query_tensors[j]) - 1
            # 补全结束的索引
            end = start + len(response_tensors[j])
            # 提示词部分掩码为0
            masks[j, :start] = 0
            # 补全后面的填充token掩码为0
            masks[j, end:] = 0
            # 将奖励模型给出的分数加到补全的最后一个token的奖励上面，得到
            # [−β log πθ(y0|x)/πref(y0|x), −β log πθ(y1|x,y0)/πref(y1|x,y0), ..., score − β log πθ(yT|x,y<T)/πref(yT|x,y<T)]
            rewards[j, end - 1] += score_tensors[j]
            # 只留下掩码为1的部分的奖励
            rewards[j, :] *= masks[j, :]
            # 只留下掩码为1的部分的价值
            # 注意最后一个token的价值V(Send)去掉!
            values[j, :-1] *= masks[j, :]

    return logp, rewards, values[:, :-1], masks
```

values[j, :-1] *= masks[j, :]是为什么呢?

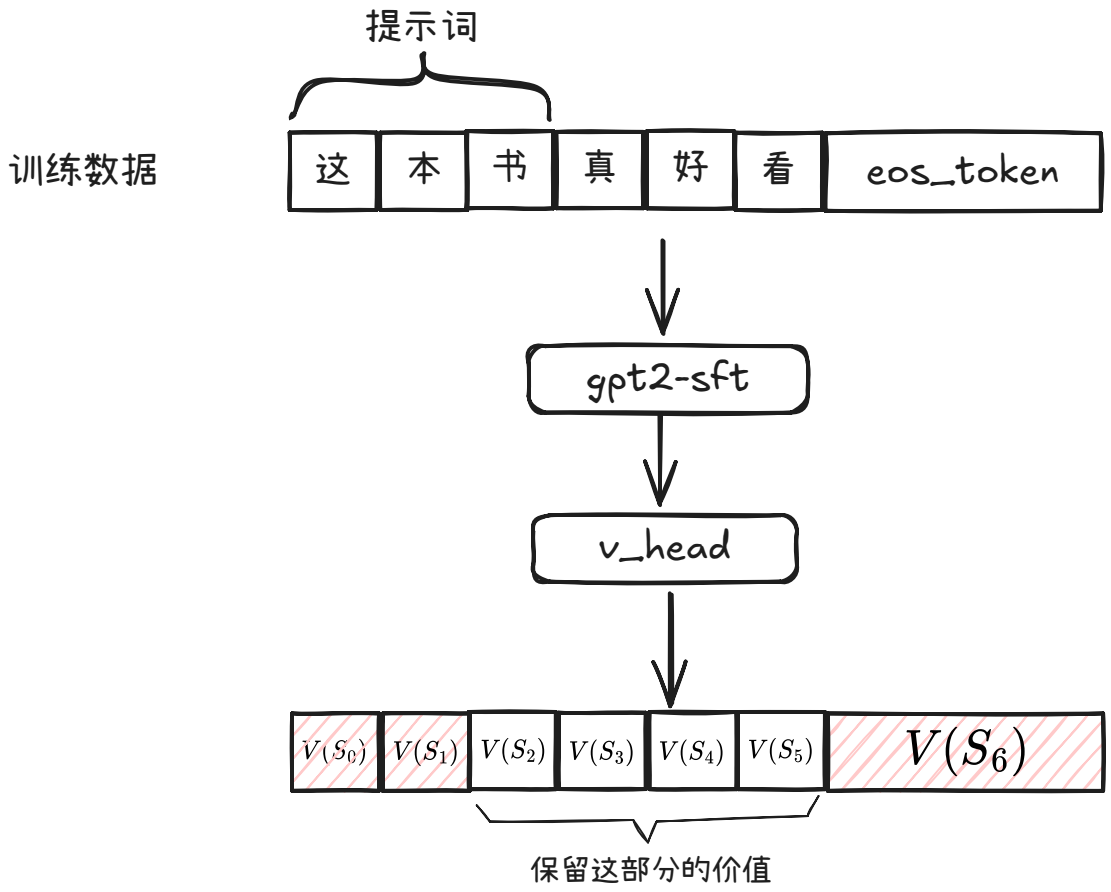


图 11.6 去掉最后一个的价值

11.3.3.6 广义优势估计（GAE）的计算

```python
def masked_mean(values, mask):
    # 计算带掩码的平均值
    return (values * mask).sum() / mask.sum()

def masked_var(values, mask):
    # 计算带掩码的方差
    mean = masked_mean(values, mask)
    centred_values = values - mean
    return masked_mean(centred_values ** 2, mask)

def masked_whiten(values, mask):
    """对数据进行带掩码的白化处理，让有效数据的方差变为1，但均值保持不变"""
    mean, var = masked_mean(values, mask), masked_var(values, mask)
    whitened = (values - mean) * torch.rsqrt(var + 1e-8)
    whitened += mean
    return whitened

def compute_advantage(rewards, values, masks):
    """广义优势估计（GAE）"""
```

```
    lastgae = 0.0
    advantage_reversed = []
    # 补全的token数量
    seq_length = rewards.shape[-1]
    # 折扣因子γ = 1，指数加权超参数λ = 0.95
    gamma, lam = 1.0, 0.95

    for t in reversed(range(seq_length)): # 逆序计算
        # V(S_{t+1})
        nextvalues = values[:, t + 1] if t < seq_length - 1 else 0.0
        # δ_t = R_t + γV(S_{t+1}) − V(S_t)
        delta = rewards[:, t] + gamma * nextvalues - values[:, t]
        # A_t^GAE = δ_t + γλA_{t+1}^GAE
        lastgae = delta + gamma * lam * lastgae
        advantage_reversed.append(lastgae)
    # [A_0^GAE, A_1^GAE, A_2^GAE, ...]
    advantages = torch.stack(advantage_reversed[::-1], dim=1)
    # 对广义优势估计进行了白化处理
    advantages = masked_whiten(advantages, masks)
    # GAETarget_t = A_t^GAE + V(S_t)
    gae_targets = advantages + values
    return advantages, gae_targets
```

11.3.3.7 损失的计算

```
def compute_loss(                                                    Python
    old_logprobs, # 旧策略输出的概率log π_{θ_old}(a_t|s_t)
    logprobs, # 正在微调的模型输出的对数概率log π_θ(a_t|s_t)
    vpreds, # 价值 V(S_t)
    masks, # 掩码
    advantages, # 广义优势估计A_t^GAE
    gae_targets # gae目标A_t^GAE + V(S_t)
):
    ratio = torch.exp(logprobs - old_logprobs) # π_θ(a_t|s_t) / π_{θ_old}(a_t|s_t)
    pg_loss1 = - ratio * advantages # −π_θ(a_t|s_t) / π_{θ_old}(a_t|s_t) A_t^GAE
    # −clip(π_θ(a_t|s_t) / π_{θ_old}(a_t|s_t), 1 − ε, 1 + ε) A_t^GAE
    pg_loss2 = - torch.clamp(ratio, 1 - 0.2, 1 + 0.2) * advantages
    # 求掩码部分的平均值
    pg_loss = masked_mean(torch.max(pg_loss1, pg_loss2), masks)
    # MSELoss(V(S_t), GAETarget) → 0
    v_loss = masked_mean((vpreds - gae_targets) ** 2, masks)
    # total_loss = loss_{π_θ} + 0.1 × loss_{V_ω}
    loss = pg_loss + 0.1 * v_loss

    return loss
```

11.3.3.8 PPO 训练循环

训练流程：

- 一条轨迹：提示词 + 策略模型输出的补全。
- 批次大小：32。所以我们针对 32 个提示词生成 32 条轨迹。那么还是一个提示词**一条轨迹**。
- 每条轨迹通过 ppo 使用 4 次更新策略模型。
- 更新后的策略模型作为下一轮训练的$\pi_{\theta_{\text{old}}}$。
- 由于 32 条轨迹同时计算新旧策略的概率比值，比较消耗 GPU。所以我们 4 条 4 条的计算。也就是 mini_batch_size=4。
- 使用 32 条轨迹更新 32 次策略模型π_θ。更新后的策略模型作为下一轮 ppo 微调的$\pi_{\theta_{\text{old}}}$。

使用旧策略采集的轨迹更新4次策略 Python

```python
data_collator = DataCollatorWithPadding(tokenizer=tokenizer)

learning_rate = 1e-5
optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate)
# 使用旧策略产生的轨迹（补全）更新4次模型
ppo_epochs = 4
num_epochs = 1
mini_batch_size = 4

def ppo_update(input_data, old_logprobs, masks, advantages, gae_targets):
    for ep in range(ppo_epochs): # π_θ被更新了32次
        # range(0, 32)
        batch_inds = list(range(batch_size)) # batch_size=32
        for start in range(0, batch_size, mini_batch_size):
            # 4条4条的使用
            mini_batch_inds = batch_inds[start:start+mini_batch_size]

            mb_model_inputs = {
                "input_ids": input_data["input_ids"][mini_batch_inds],
                "attention_mask": input_data["attention_mask"][mini_batch_inds]
            }
            # 模型的输出是token的logits和value
            # mb_vpreds: [V(S_0), V(S_1), ..., V(S_T)]
            mb_logits, mb_vpreds = model(**mb_model_inputs)
            # 去掉最后一个token
            mb_logits = F.log_softmax(mb_logits[:, :-1, :], dim=-1)
            # 取出真实标签对应的概率：[log π_θ(y_0|x), log π_θ(y_1|x, y_0), ...]
            mb_logprobs = torch.gather(
                mb_logits,
                2,
                mb_model_inputs["input_ids"][:, 1:].unsqueeze(-1)
            ).squeeze(-1)

            loss = compute_loss(
                old_logprobs[mini_batch_inds], # log π_θ_old(y_t|x, y_<t)
                mb_logprobs, # log π_θ(y_t|x, y_<t)
                mb_vpreds[:, :-1], # V(S_t)
                masks[mini_batch_inds],
                advantages[mini_batch_inds], # A_t^GAE
                gae_targets[mini_batch_inds] # A_t^GAE + V(S_t)
            )
```

```
42
43              optimizer.zero_grad()
44              # 使用4条轨迹更新了一次π_θ
45              loss.backward()
46              optimizer.step()
47              print("loss/total", loss.item())
48      print("ppo update finished")
```

主训练循环代码如下

```python
count = 0
for epoch in range(num_epochs):
    for batch in train_dataloader:
        if count == 1:
            break
        count += 1
        # 生成补全内容（回复）
        query_tensors = batch["input_ids"]  # 提示词的张量
        query_attention_masks = batch["attention_mask"]

        response_tensors = []  # 补全的张量
        query_response_tensors = []  # 提示词+补全的张量
        score_tensors = []  # 分数的张量

        for i, query in enumerate(query_tensors):
            query = query.to(device)
            query_attention_mask = query_attention_masks[i].to(device)
            # 随机挑一个补全的长度
            new_tokens = random.choice(list(range(
                output_min_length,
                output_max_length)))
            # 设置补全长度属性
            generation_kwargs["max_new_tokens"] = new_tokens
            # 使用π_θold采样一条轨迹：提示词 + 补全
            query_response = model.generate(
                input_ids=query.unsqueeze(0),
                attention_mask=query_attention_mask.unsqueeze(0),
                **generation_kwargs
            ).squeeze(0)
            # 补全的长度
            response_len = len(query_response) - len(query)
            # 补全的张量
            response_tensors.append(query_response[-response_len:])
            query_response_tensors.append(query_response)
            # 从奖励模型拿分数
            with torch.no_grad():
                # 提示词 + 补全 + reward_token
                query_response_score = torch.cat([
                    query_response,
                    torch.tensor([REWARD_TOKEN_ID]).to(device)])
```

```
                attention_mask = torch.ones_like(
                    query_response_score,
                    dtype=torch.long)
                # 奖励模型的评分
                score = reward_model(
                    query_response_score.unsqueeze(0),
                    attention_mask.unsqueeze(0)
                ).squeeze(0)[-1]
                # 将奖励模型的评分从(0,1)缩放到(-1,1)
                score = 2 * (score - 0.5)
            score_tensors.append(score)

        input_data = data_collator([
            {
                "input_ids": ids,
                "attention_mask": torch.ones_like(ids)
            }
            for ids in query_response_tensors
        ]).to(device)

        # 奖励和优势
        old_logprobs, rewards, values, masks = compute_rewards(
            input_data,
            query_tensors,
            response_tensors,
            score_tensors
        )
        # 计算GAE和GAE Target
        advantages, gae_targets = compute_advantage(rewards, values, masks)

        # 小批次训练
        if input_data["input_ids"].shape[0] != 32:
            break
        # 使用采集到的32条轨迹更新策略
        ppo_update(input_data, old_logprobs, masks, advantages, gae_targets)
```

11.3.3.9 评估模型

主要对比 PPO 微调后的模型和微调前的模型。对比方法：

- 选择一些提示词
- 使用 PPO 微调后的模型补全
- 送入奖励模型，打平均分
- 使用 PPO 微调前的模型补全
- 送入奖励模型，打平均分
- 对比打分情况

对比微调前后的模型的补全的打分情况 Python

```
train_gen_lengths = [0] * len(tokenized_dataset_train)
for i in range(len(tokenized_dataset_train)):
    train_gen_lengths[i] = random.choice(list(range(
```

```
        output_min_length,
        output_max_length)))


def validate():
    scores = []
    count = 0
    for b, batch in enumerate(train_dataloader):
        if count == 1:
            break
        count += 1
        # 生成补全内容
        query_tensors = batch["input_ids"]
        query_attention_masks = batch["attention_mask"]
        for i, query in enumerate(query_tensors):
            query = query.to(device)
            query_attention_mask = query_attention_masks[i].to(device)
            new_tokens = train_gen_lengths[b * len(query_tensors) + i]
            generation_kwargs["max_new_tokens"] = new_tokens
            query_response = model.generate(
                input_ids=query.unsqueeze(0),
                attention_mask=query_attention_mask.unsqueeze(0),
                **generation_kwargs
            ).squeeze(0)
            query_response_score = torch.cat([
                query_response,
                torch.tensor([REWARD_TOKEN_ID]).to(device)])
            attention_mask = torch.ones_like(
                query_response_score, dtype=torch.long)
            score = reward_model(
                query_response_score.unsqueeze(0),
                attention_mask.unsqueeze(0)
            ).squeeze(0)[-1]
            score = 2 * (score - 0.5)
            scores.append(score.item())
    print("平均分数：", sum(scores) / len(scores))


validate()

model_path = "./gpt2-sft"
model = ActorCriticModel(model_path).to(device)
validate()
```

12. 使用 GRPO 微调大语言模型——复刻 DeepSeek-R1

强化学习已成为增强大语言模型初始训练效果的强大工具，尤其是在推理密集型任务中。DeepSeek 最近在 DeepSeek-Math 和 DeepSeek-R1 模型上取得的突破，展现了 RL 在提升 LLM 数学推理和问题解决能力方面的巨大潜力。

这些成就得益于一种名为“组相对策略优化”（GRPO）的创新强化学习方法，该方法解决了将强化学习应用于语言模型的独特挑战。我们将深入探讨 GRPO 的工作原理，以及它为何代表了 LLM 训练的重大进步。

12.1 GRPO 简介

GRPO 目标函数如下：

$$J(\theta)^{\text{GRPO}} = \frac{1}{G}\sum_{i=1}^{G}\frac{1}{|\tau_i|}\sum_{t=1}^{|\tau_i|}\min\left[pA_{\tau_i,t}, \text{clip}(p, 1-\varepsilon, 1+\varepsilon)A_{\tau_i,t}\right] - \beta D_{\text{KL}}[\pi_\theta \parallel \pi_{\text{ref}}]$$
$$\text{其中比值} \quad p = \frac{\pi_\theta\left(a_{\tau_i,t}|s_{\tau_i,t}\right)}{\pi_{\theta_{\text{old}}}\left(a_{\tau_i,t}|s_{\tau_i,t}\right)} \tag{12.1}$$

先来说明一下 GRPO 目标函数中每个数学符号的含义：

- π_θ表示正在更新的策略。
- $\pi_{\theta_{\text{old}}}$表示上一轮训练好的旧策略。
- π_{ref}表示冻结的参考模型。
- G表示使用旧策略$\pi_{\theta_{\text{old}}}$采样的一组轨迹的数量，也就是如果我们使用旧策略采样了 10 条轨迹，那么$G = 10$。
- τ_i表示第i条轨迹。
- $|\tau_i|$表示第i条轨迹的动作数量。
- $\pi_\theta\left(a_{\tau_i,t}|s_{\tau_i,t}\right)$表示第$i$条轨迹的第$t$个时刻的状态为$s_{\tau_i,t}$，以及在这个状态下正在更新的策略$\pi_\theta$采取动作$a_{\tau_i,t}$的概率。
- $\pi_{\theta_{\text{old}}}\left(a_{\tau_i,t}|s_{\tau_i,t}\right)$表示第$i$条轨迹的第$t$个时刻的状态为$s_{\tau_i,t}$，以及在这个状态下正在更新的策略$\pi_{\theta_{\text{old}}}$采取动作$a_{\tau_i,t}$的概率。
- $A_{\tau_i,t}$表示第i条轨迹的第t个时刻的动作的优势。
- β是超参数。$D_{\text{KL}}[\pi_\theta|\pi_{\text{ref}}]$表示$\pi_\theta$和$\pi_{\text{ref}}$而偏离程度，也就是 KL 散度。

12.2 使用 DAPO（GRPO 的变种）微调 Qwen2.5-3B-Instruct

12.2.1 实现思路及任务要求

⚡ 我们要从零复刻一个类似 DeepSeek-R1 的模型，产生思维链！

我们的实现不采用原始的 GRPO 算法，而是采用字节提出的 DAPO 算法，DAPO 对原始的 GRPO 有如下几项改进：

1. **token 级的策略梯度损失**：每个 token 在策略梯度损失中具有同等权重。也就是每个 token 的优势等于 token 所在轨迹的组相对优势！
2. **移除 KL 散度**：策略梯度损失中不再使用 KL 散度。由于我们不再需要参考策略网络π_{ref}，这可以减少 GPU 内存的使用。

我们的算法伪代码如下：

🔥 DAPO 算法伪代码

1. 对于每个训练步骤，随机选取N个问题：$q_1, q_2, ..., q_N$。
2. 对于每个问题q_i，采样G条回答（轨迹）：$a_{i,1}, a_{i,2}, ..., a_{i,G}$ 。 ▷ G为一组轨迹中轨迹的数量
3. 计算每个回答$a_{i,j}$的奖励$r_{i,j}$。
   - $a_{i,j}$为第 i 个问题的第 j 条回答。
   - $r_{i,j}$为第 i 个问题的第 j 条回答的回报。
4. 计算每个问题 q_i 的奖励的平均值和标准差。

$$\begin{aligned} \mu_i &\leftarrow \text{mean}\left(r_{i,1}, r_{i,2}, \cdots, r_{i,G}\right) \\ \sigma_i &\leftarrow \text{std}\left(r_{i,1}, r_{i,2}, \cdots, r_{i,G}\right) \end{aligned} \tag{12.2}$$

5. 对于回答$a_{i,j}$中的每个 token，也就是t计算优势：
   - $A_{i,j}[t]$为第 i 个问题的第 j 条回答的第 t 个 token 的优势
   - 重要！输出的 token（动作）的优势等于 token 所在回答的组相对优势！

$$A_{i,j}[t] \leftarrow \frac{r_{i,j} - \mu_i}{\sigma_i} \tag{12.3}$$

6. 使用 GRPO 的目标函数计算策略梯度。为了简单起见，我们每次迭代只进行一次策略更新，这样 GRPO 的目标函数的梯度等价于原始策略梯度法中的梯度估计方法，针对每个 token 都进行梯度估计。

$$\nabla_\theta \log \pi_\theta\left(a_{i,j}[t]\right) \cdot A_{i,j}[t] \tag{12.4}$$

7. 使用梯度更新策略网络π_θ。
8. goto 1

⚡ 伪代码中第 6 步的解释

GRPO 目标函数最内层为

$$\min\left[\frac{\pi_\theta\left(a_{\tau_i,t}|s_{\tau_i,t}\right)}{\pi_{\theta_{\text{old}}}\left(a_{\tau_i,t}|s_{\tau_i,t}\right)}A_{\tau_i,t},\text{clip}\left(\frac{\pi_\theta\left(a_{\tau_i,t}|s_{\tau_i,t}\right)}{\pi_{\theta_{\text{old}}}\left(a_{\tau_i,t}|s_{\tau_i,t}\right)},1-\varepsilon,1+\varepsilon\right)A_{\tau_i,t}\right] \tag{12.5}$$

如果 GRPO 使用旧策略采集的轨迹只更新一次策略的话，相当于在$\theta=\theta_{\text{old}}$处进行求导（梯度）。由于此时$\theta=\theta_{\text{old}}$，所以裁剪不会发生。也就是如下：

$$\begin{aligned}
&\nabla_\theta \min\left[\frac{\pi_\theta\left(a_{\tau_i,t}|s_{\tau_i,t}\right)}{\pi_{\theta_{\text{old}}}\left(a_{\tau_i,t}|s_{\tau_i,t}\right)}A_{\tau_i,t},\text{clip}\left(\frac{\pi_\theta\left(a_{\tau_i,t}|s_{\tau_i,t}\right)}{\pi_{\theta_{\text{old}}}\left(a_{\tau_i,t}|s_{\tau_i,t}\right)},1-\varepsilon,1+\varepsilon\right)A_{\tau_i,t}\right]|_{\theta=\theta_{\text{old}}}\\
&=\nabla_\theta\left\{\frac{\pi_\theta\left(a_{\tau_i,t}|s_{\tau_i,t}\right)}{\pi_{\theta_{\text{old}}}\left(a_{\tau_i,t}|s_{\tau_i,t}\right)}A_{\tau_i,t}\right\}|_{\theta=\theta_{\text{old}}}\\
&=\frac{\nabla_\theta\pi_\theta\left(a_{\tau_i,t}|s_{\tau_i,t}\right)|_{\theta=\theta_{\text{old}}}}{\pi_{\theta_{\text{old}}}\left(a_{\tau_i,t}|s_{\tau_i,t}\right)}A_{\tau_i,t}\qquad\text{（log 梯度技巧）}\\
&=\nabla_\theta\left\{A_{\tau_i,t}\cdot\log\pi_\theta\left(a_{\tau_i,t}|s_{\tau_i,t}\right)\right\}|_{\theta=\theta_{\text{old}}}
\end{aligned} \tag{12.6}$$

我们的任务是什么？我们想要训练一个能够玩类似 24 点游戏的 LLM。叫做 CountDown Task。

我们将在 CountDown Task 中微调 Qwen2.5-3B-Instruct 模型。给定一个包含 3 个或 4 个数字的列表和一个目标数字，模型需要使用简单的算术运算（+、-、*、/）生成一个数学表达式，该表达式的求值结果等于目标数字。例如：

我们用的数据格式如下：

```
1 nums: [37, 81, 10], target: 34
```

我们会先将上面的数据格式化成如下数据：

```
◁im_start▷system
你是一个有用的助手。你首先在脑海中思考推理过程，然后为用户提供答案。◁im_end▷
◁im_start▷user
使用这些数字 [37 81 10]，创建一个等于 34 的等式。你可以使用基本算术运算（+、-、*、/），每个数字只能使用一次。在 <think> </think> 标签中展示你的解题过程。并在 <answer> </answer> 标签中返回最终答案，例如 <answer> (1 + 2) / 3 </answer>。◁im_end▷
◁im_start▷assistant
让我一步步来解决这个问题。
<think>
```

上面的数据输入给我们微调后的模型，模型应该输出的补全如下：

```
可以通过组合给定的数字 37 81 10 使它们满足运算为 34 。首先通过 81 减去 37 得到 44 ，再进行减法运算 44 减去 10 等于 34 ，最后将这两步结果利用小括号组合在一起形成数学表达式为 (81 - 37) - 10 等于 34 。
</think>
<answer> (81 - 37) - 10 </answer>◁im_end▷
```

完整的提示词+补全的数据如下：

```
◁im_start▷system
你是一个有用的助手。你首先在脑海中思考推理过程，然后为用户提供答案。◁im_end▷
◁im_start▷user
```

使用这些数字 [37 81 10]，创建一个等于 34 的等式。你可以使用基本算术运算（+、-、*、/），每个数字只能使用一次。在 <think> </think> 标签中展示你的解题过程。并在 <answer> </answer> 标签中返回最终答案，例如 <answer> (1 + 2) / 3 </answer>。◁im_end▷

◁im_start▷assistant

让我一步步来解决这个问题。

<think>可以通过组合给定的数字 37 81 10 使它们满足运算为 34 。首先通过 81 减去 37 得到 44 ，再进行减法运算 44 减去 10 等于 34 ，最后将这两步结果利用小括号组合在一起形成数学表达式为 (81 - 37) - 10 等于 34 。

</think>

<answer> (81 - 37) - 10 </answer>◁im_end▷

12.2.2 奖励函数的设计

在使用 GRPO 玩倒立摆游戏时，我们的奖励很简单，例如一条轨迹执行了 5 步动作，轨迹就结束了，那么这条轨迹的回报是 5。

那么在 LLM 这个环境中，我们应该如何给一条轨迹（模型输出的补全）奖励呢？因为 GRPO 是需要给一整条轨迹计算回报的。

在 GRPO 的实现中，奖励是两部分的总和：

1. **格式奖励**：当模型正确遵循指定的格式并带有思考和答案标签时，模型获得 0.1 的奖励，否则 0 的奖励。
2. **答案奖励**：如果模型的最终答案恰好使用了提供的数字，并且每个数字只使用了一次，并正确求值为目标值，则模型将获得 1 的奖励，否则将获得 0 的奖励。

例如下面的补全只能拿到 0.1 分的回报。因为格式是没问题的，但是<answer></answer>标签内的答案有问题，包含了不该有的= 34。

可以通过组合给定的数字 37 81 10 使它们满足运算为 34 。首先通过 81 减去 37 得到 44 ，再进行减法运算 44 减去 10 等于 34 ，最后将这两步结果利用小括号组合在一起形成数学表达式为 (81 - 37) - 10 等于 34 。

</think>

<answer> (81 - 37) - 10 = 34</answer>◁im_end▷

只有下面这种补全才能拿到 1.1 的满分。既带有<think></think>标签和<answer></answer>标签，<answer></answer>标签内的答案(81 - 37) - 10 也是完全正确的。

可以通过组合给定的数字 37 81 10 使它们满足运算为 34 。首先通过 81 减去 37 得到 44 ，再进行减法运算 44 减去 10 等于 34 ，最后将这两步结果利用小括号组合在一起形成数学表达式为 (81 - 37) - 10 等于 34 。

</think>

<answer> (81 - 37) - 10</answer>◁im_end▷

所以可以看到，我们的奖励函数是很严格的！

12.2.3 代码实现

创建文件夹

```shell
$ mkdir GRPO-Zero
```

12.2.3.1 模型结构

Qwen2 的模型结构，保存在文件 qwen2_model.py 中。

Qwen2.5模型结构

```python
import json
from dataclasses import dataclass
```

```
from pathlib import Path
from typing import Optional, Tuple, Union

import torch
import torch.nn.functional as F
from torch import nn


@dataclass
class Qwen2Config:
    attention_dropout: float = 0.0
    bos_token_id: int = 151643
    eos_token_id: int = 151645
    hidden_act: str = "silu"
    hidden_size: int = 2048
    initializer_range: float = 0.02
    intermediate_size: int = 11008
    max_position_embeddings: int = 32768
    max_window_layers: int = 70
    model_type: str = "qwen2"
    num_attention_heads: int = 16
    num_hidden_layers: int = 36
    num_key_value_heads: int = 2
    rms_norm_eps: float = 1e-06
    rope_theta: float = 1000000.0
    sliding_window: int = 32768
    tie_word_embeddings: bool = True
    torch_dtype: str = "bfloat16"
    use_cache: bool = True
    use_sliding_window: bool = False
    vocab_size: int = 151936


class RMSNorm(torch.nn.Module):
    def __init__(self, dim: int, eps: float = 1e-6):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(dim))

    def _norm(self, x):
        return x * torch.rsqrt(x.pow(2).mean(-1, keepdim=True) + self.eps)

    def forward(self, x):
        input_dtype = x.dtype
        x = x.to(torch.float32)
        x = self._norm(x).type_as(x)
        x = self.weight * x.to(input_dtype)
        return x


```

```python
def rotate_half(x):
    """Rotates half the hidden dims of the input."""
    x1 = x[..., : x.shape[-1] // 2]
    x2 = x[..., x.shape[-1] // 2 :]
    return torch.cat((-x2, x1), dim=-1)


def apply_rotary_pos_emb(q, k, cos, sin, unsqueeze_dim=2):
    cos = cos.unsqueeze(unsqueeze_dim)
    sin = sin.unsqueeze(unsqueeze_dim)
    q_embed = (q * cos) + (rotate_half(q) * sin)
    k_embed = (k * cos) + (rotate_half(k) * sin)
    return q_embed, k_embed


class Attention(nn.Module):
    def __init__(self, args: Qwen2Config):
        super().__init__()
        self.n_kv_heads = (
            args.num_attention_heads
            if args.num_key_value_heads is None
            else args.num_key_value_heads
        )
        self.n_heads = args.num_attention_heads
        self.n_kv_heads = self.n_kv_heads
        self.n_rep = self.n_heads // self.n_kv_heads
        self.head_dim = args.hidden_size // args.num_attention_heads

        self.q_proj = nn.Linear(
            args.hidden_size,
            args.num_attention_heads * self.head_dim,
            bias=True,
        )
        self.k_proj = nn.Linear(
            args.hidden_size,
            args.num_key_value_heads * self.head_dim,
            bias=True,
        )
        self.v_proj = nn.Linear(
            args.hidden_size,
            args.num_key_value_heads * self.head_dim,
            bias=True,
        )
        self.o_proj = nn.Linear(
            args.num_attention_heads * self.head_dim,
            args.hidden_size,
            bias=False,
        )
        self.args = args

```

```
    def init_kv_cache(
        self,
        max_batch_size: int,
        max_seq_len: int,
        dtype: torch.dtype,
        device: torch.device,
    ):
        cache_shape = (max_batch_size, max_seq_len, self.n_kv_heads,
        self.head_dim)
        cache_k = torch.zeros(cache_shape, dtype=dtype, device=device)
        cache_v = torch.zeros(cache_shape, dtype=dtype, device=device)
        self.register_buffer("cache_k", cache_k, persistent=False)
        self.register_buffer("cache_v", cache_v, persistent=False)

    def del_kv_cache(self):
        self.cache_k = None
        self.cache_v = None

    def forward(
        self,
        x: torch.Tensor,
        pos_embed: Tuple[torch.Tensor, torch.Tensor],
        start_pos: Optional[Union[int, torch.Tensor]] = None,
    ):
        bsz, seqlen, _ = x.shape
        xq, xk, xv = self.q_proj(x), self.k_proj(x), self.v_proj(x)
        xq = xq.view(bsz, seqlen, self.n_heads, self.head_dim)
        xk = xk.view(bsz, seqlen, self.n_kv_heads, self.head_dim)
        xv = xv.view(bsz, seqlen, self.n_kv_heads, self.head_dim)

        cos, sin = pos_embed
        xq, xk = apply_rotary_pos_emb(xq, xk, cos, sin, unsqueeze_dim=2)
        if start_pos is not None:
            # inference mode
            end_pos = start_pos + seqlen
            self.cache_k[:bsz, start_pos:end_pos, :, :] = xk
            self.cache_v[:bsz, start_pos:end_pos, :, :] = xv
            output = torch.nn.functional.scaled_dot_product_attention(
                query=xq.transpose(1, 2),
                key=self.cache_k[:bsz, :end_pos].transpose(1, 2),
                value=self.cache_v[:bsz, :end_pos].transpose(1, 2),
                is_causal=True if seqlen > 1 else False,
                enable_gqa=True,
            ).transpose(1, 2)
        else:
            # training mode
            output = torch.nn.functional.scaled_dot_product_attention(
                query=xq.transpose(1, 2),
                key=xk.transpose(1, 2),
                value=xv.transpose(1, 2),
                is_causal=True,
```

```
                enable_gqa=True,
            ).transpose(1, 2)
        output = output.reshape(bsz, seqlen, -1)
        return self.o_proj(output)


class FeedForward(nn.Module):
    def __init__(
        self,
        dim: int,
        intermediate_size: int,
    ):
        super().__init__()
        self.up_proj = nn.Linear(dim, intermediate_size, bias=False)
        self.down_proj = nn.Linear(intermediate_size, dim, bias=False)
        self.gate_proj = nn.Linear(dim, intermediate_size, bias=False)

    def forward(self, x):
        x = self.down_proj(F.silu(self.gate_proj(x)) * self.up_proj(x))
        return x


class TransformerBlock(nn.Module):
    def __init__(self, layer_id: int, args: Qwen2Config):
        super().__init__()
        self.n_heads = args.num_attention_heads
        self.dim = args.hidden_size
        self.head_dim = args.hidden_size // args.num_attention_heads
        self.self_attn = Attention(args)
        self.mlp = FeedForward(
            dim=args.hidden_size,
            intermediate_size=args.intermediate_size,
        )
        self.layer_id = layer_id
        self.input_layernorm = RMSNorm(args.hidden_size, eps=args.rms_norm_eps)
        self.post_attention_layernorm = RMSNorm(args.hidden_size,
eps=args.rms_norm_eps)

    def forward(
        self,
        x: torch.Tensor,
        pos_embed: Tuple[torch.Tensor, torch.Tensor],
        start_pos: Optional[Union[int, torch.Tensor]] = None,
    ):
        h = x + self.self_attn(self.input_layernorm(x), pos_embed,
start_pos=start_pos)
        out = h + self.mlp(self.post_attention_layernorm(h))
        return out


class Qwen2RotaryEmbedding(nn.Module):
```

```
    def __init__(self, config: Qwen2Config, device: torch.device):
        super().__init__()
        self.config = config
        base = config.rope_theta
        dim = config.hidden_size // config.num_attention_heads
        with torch.autocast(device_type=device.type, dtype=torch.float32):
            inv_freq = 1.0 / (
                base
                ** (torch.arange(0, dim, 2,
                dtype=torch.int64).float().to(device) / dim)
            )
        self.register_buffer("inv_freq", inv_freq, persistent=False)

    @torch.no_grad()
    def forward(self, x, pos):
        inv_freq = self.inv_freq[None, :, None].float().expand(pos.shape[0], -1,
        1)
        pos = pos[:, None, :].float()
        device_type = x.device.type
        with torch.autocast(device_type=device_type, enabled=False):
            freqs = (inv_freq.float().to(x.device) @ pos.float()).transpose(1, 2)
            emb = torch.cat((freqs, freqs), dim=-1)
            cos = emb.cos()
            sin = emb.sin()
        return cos.to(dtype=x.dtype), sin.to(dtype=x.dtype)


class Transformer(nn.Module):
    def __init__(self, params: Qwen2Config, device: torch.device):
        super().__init__()
        self.params = params
        self.vocab_size = params.vocab_size
        self.n_layers = params.num_hidden_layers

        self.embed_tokens = torch.nn.Embedding(params.vocab_size,
        params.hidden_size)
        with torch.device(device):
            self.rotary_emb = Qwen2RotaryEmbedding(config=params, device=device)

        self.layers = torch.nn.ModuleList()
        for layer_id in range(params.num_hidden_layers):
            self.layers.append(TransformerBlock(layer_id, params))

        self.norm = RMSNorm(params.hidden_size, eps=params.rms_norm_eps)
        if not params.tie_word_embeddings:
            self.lm_head = nn.Linear(params.hidden_size, params.vocab_size,
            bias=False)

    def output_proj(self, x):
        if self.params.tie_word_embeddings:
            return x @ self.embed_tokens.weight.T
```

```
249         else:
250             return self.lm_head(x)
251 
252     def forward(self, tokens: torch.Tensor):
253         _bsz, seqlen = tokens.shape
254         h = self.embed_tokens(tokens)
255         pos = torch.arange(0, seqlen, device=tokens.device, dtype=torch.int32)
256         pos_emb = self.rotary_emb(h, pos[None, :])
257 
258         pipe = []
259         for layer in self.layers:
260             pipe.append(lambda x, layer=layer: layer(x, pos_emb))
261         pipe.append(self.norm.forward)
262         pipe.append(self.output_proj)
263         return torch.utils.checkpoint.checkpoint_sequential(
264             pipe, len(pipe), h, use_reentrant=False
265         )
266 
267     def inference(self, tokens: torch.Tensor, start_pos: Union[int,
    torch.Tensor]):
268         _bsz, seqlen = tokens.shape
269         del _bsz
270         h = self.embed_tokens(tokens)
271 
272         pos = torch.arange(0, seqlen, device=tokens.device, dtype=torch.int32)
    [None, :]
273         if isinstance(start_pos, torch.Tensor):
274             pos = pos + start_pos[:, None]
275         else:  # int
276             pos.add_(start_pos)
277         pos_emb = self.rotary_emb(h, pos)
278 
279         for layer in self.layers:
280             h = layer(h, pos_emb, start_pos=start_pos)
281 
282         # only need the hidden state of the last token
283         # to predict the next token
284         h = h[:, -1:, :]
285         h = self.norm(h)
286 
287         output = self.output_proj(h)
288         return output
289 
290     def init_kv_cache(
291         self,
292         max_batch_size: int,
293         max_seq_len: int,
294         device: torch.device,
295         dtype: torch.dtype,
296     ):
297         for layer in self.layers:
```

```
298             layer.self_attn.init_kv_cache(
299                 max_batch_size, max_seq_len, dtype=dtype, device=device
300             )
301 
302     def del_kv_cache(self):
303         for layer in self.layers:
304             layer.self_attn.del_kv_cache()
305 
306     @classmethod
307     def from_pretrained(cls, ckpt_path, device: torch.device):
308         config_file = Path(ckpt_path) / "config.json"
309         with open(config_file, "r") as f:
310             config = json.load(f)
311         args = Qwen2Config(
312             attention_dropout=config["attention_dropout"],
313             bos_token_id=config["bos_token_id"],
314             eos_token_id=config["eos_token_id"],
315             hidden_act=config["hidden_act"],
316             hidden_size=config["hidden_size"],
317             initializer_range=config["initializer_range"],
318             intermediate_size=config["intermediate_size"],
319             max_position_embeddings=config["max_position_embeddings"],
320             max_window_layers=config["max_window_layers"],
321             model_type=config["model_type"],
322             num_hidden_layers=config["num_hidden_layers"],
323             num_attention_heads=config["num_attention_heads"],
324             num_key_value_heads=config["num_key_value_heads"],
325             vocab_size=config["vocab_size"],
326             rms_norm_eps=config["rms_norm_eps"],
327             rope_theta=config["rope_theta"],
328             sliding_window=config["sliding_window"],
329             use_sliding_window=config["use_sliding_window"],
330             use_cache=config["use_cache"],
331             tie_word_embeddings=config["tie_word_embeddings"],
332             torch_dtype=config["torch_dtype"],
333         )
334         with torch.device("meta"):
335             model = cls(params=args, device=device)
336 
337         import safetensors.torch
338 
339         model_weight_files = sorted(Path(ckpt_path).glob("model*.safetensors"))
340         weights = {}
341         for file in model_weight_files:
342             weights.update(safetensors.torch.load_file(file, device="cpu"))
343         # remove "model." prefix from keys
344         weights = {k.replace("model.", ""): v for k, v in weights.items()}
345         model.load_state_dict(weights, strict=True, assign=True)
346         return model.to(device)
```

12.2.3.2 分词器

分词器代码保存在文件 `tokenizer.py` 中。

分词器实现 Python

```python
import json
from pathlib import Path
from typing import Dict, List

from jinja2 import Environment
from tokenizers import Encoding
from tokenizers import Tokenizer as TokenizerBase


class Tokenizer:
    """Tokenizer with chat template supported using jinja2 engine"""

    def __init__(self, tokenizer_path: str):
        super().__init__()
        tokenizer_config_path = Path(tokenizer_path).parent / "tokenizer_config.json"
        self.tokenizer_config = json.load(open(tokenizer_config_path))
        self.tokenizer = TokenizerBase.from_file(tokenizer_path)
        self.chat_template = Environment().from_string(
            self.tokenizer_config["chat_template"]
        )
        self.eos_token = self.tokenizer_config["eos_token"]
        self.eos_token_id = self.tokenizer.token_to_id(self.eos_token)
        self.pad_token = self.tokenizer_config["pad_token"]
        self.pad_token_id = self.tokenizer.token_to_id(self.pad_token)

    def encode_chat(self, messages: List[Dict[str, str]]) -> str:
        return self.chat_template.render(messages=messages, add_generation_prompt=True)

    def encode_chat_with_response_prompt(
        self, messages: List[Dict[str, str]], prompt: str
    ) -> str:
        return self.encode_chat(messages) + prompt

    def tokenize(self, text: str) -> Encoding:
        return self.tokenizer.encode(text)

    def detokenize(self, token_ids: List[int]) -> str:
        return self.tokenizer.decode(token_ids, skip_special_tokens=False)
```

12.2.3.3 任务定义

首先创建 `data_types.py` ，内容如下：

Python

```python
from dataclasses import dataclass
from typing import Dict, List

```

```
4
5   @dataclass
6   class Episode:
7       """存储一个回合（Episode）或者说一条轨迹的所有相关信息"""
8       """一个回合 = 问题 + 一条回答"""
9       prefix: str # 问题
10      text: str # "问题+回答"整个文本
11      prefix_token_ids: List[int] # 问题的input_ids
12      prefix_tokens: List[str] # 问题的token组成的列表
13      generated_token_ids: List[int] # 生成的回答的token列表
14      is_finished: bool # 回答是否结束标志位
15      reward: float # 奖励
16      reward_info: Dict[str, float] # 详细的奖励信息
17
18
19  @dataclass
20  class MiniBatch:
21      """每个Step训练所需的微批次数据"""
22      prefix: List[str] # 问题列表
23      prefix_tokens: List[List[str]]
24      prefix_token_ids: List[List[int]]
25      numbers: List[List[int]] # 问题的数字列表
26      target: List[int] # 问题对应的答案数字
```

创建文件 `countdown_task.py` ，内容如下

```python
1   import re
2   from pathlib import Path
3   from typing import Any, Dict, List, Optional
4
5   import pandas as pd
6   from torch.utils.data import Dataset
7
8   from data_types import MiniBatch
9   from tokenizer import Tokenizer
10
11  SYSTEM_MESSAGE = (
12      "你是一个有用的助手。你首先在脑海中思考推理过程，"
13      "然后为用户提供答案。"
14  )
15  # `{numbers}` 和 `{target}` 是占位符，构建训练数据时会被替换
16  USER_TEMPLATE = (
17      "使用这些数字 {numbers}，创建一个等于 {target} 的等式。"
18      "你可以使用基本算术运算（+、-、*、/），每个数字只能使用一次。"
19      "在 <think> </think> 标签中展示你的解题过程。"
20      "并在 <answer> </answer> 标签中返回最终答案，例如 <answer> (1 + 2) / 3 </answer>。"
21  )
22
23  RESPONSE_PROMPT = "让我一步步来解决这个问题。\n<think>"
```

```python


class CountdownTasksDataset(Dataset):
    """准备训练数据集"""

    def __init__(
        self,
        tokenizer: Tokenizer, # 分词器
        data_path: str, # 数据集的路径
        split: str = "train",
        test_size: int = 100,
    ):
        data = pd.read_parquet(Path(data_path) / "data")
        # 索引 `test_size` 后面的数据用作测试数据
        self.data = (
            data.iloc[:-test_size]                \
            if split == "train"                   \
            else data.iloc[-test_size:]
        )
        self.tokenizer = tokenizer

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        item = self.data.iloc[idx].to_dict()
        item.update(
            self.encode_prefix(
                item["nums"], # 数字列表
                item["target"] # 目标数字
            )
        )
        return item

    def encode_prefix(self, numbers: List[int], target: int):
        """Prefix 是模型 *真正的* 输入，也就是问题"""
        # 格式化对话模板
        user_message = USER_TEMPLATE.format(
            numbers=numbers,
            target=target
        )
        prefix = self.tokenizer.encode_chat_with_response_prompt(
            [
                {"role": "system", "content": SYSTEM_MESSAGE},
                {"role": "user", "content": user_message},
            ],
            RESPONSE_PROMPT,
        )
        # 将问题切分
        tokens = self.tokenizer.tokenize(prefix)
```

```
74          return {
75              "prefix": prefix, # 问题字符串
76              "prefix_tokens": tokens.tokens, # 问题切分后的字符串列表
77              "prefix_token_ids": tokens.ids, # input_ids
78          }
79
80      @staticmethod
81      def collate_fn(batch: List[Dict[str, Any]]) -> MiniBatch:
82          """将数据整理到一个批次中"""
83          numbers = [item["nums"] for item in batch]
84          target = [item["target"] for item in batch]
85          prefix = [item["prefix"] for item in batch]
86          prefix_tokens = [
87              item["prefix_tokens"] for item in batch
88          ]
89          prefix_token_ids = [
90              item["prefix_token_ids"] for item in batch
91          ]
92          return MiniBatch(
93              numbers=numbers,
94              target=target,
95              prefix=prefix,
96              prefix_tokens=prefix_tokens,
97              prefix_token_ids=prefix_token_ids,
98          )
99
100
101 def format_reward_function(
102     response: str, # 模型的回答
103     end_token: Optional[str] = None # 结尾token
104 ) -> float:
105     """
106     检查模型的回复是否符合格式 <think>...</think><answer>...</answer>
107     """
108     # 如果存在end token，则去掉
109     if end_token and response.endswith(end_token):
110         response = response[: -len(end_token)]
111
112     think_regex = r"<think>.*?<\/think>"
113     answer_regex = r"<answer>.*?<\/answer>"
114     full_format_regex = \
115         r"^<think>.*?<\/think>\n<answer>.*?<\/answer>$"
116
117     think_match = re.search(think_regex, response, re.DOTALL)
118     answer_match = re.search(answer_regex, response, re.DOTALL)
119     full_format_match = re.match(
120         full_format_regex,
121         response,
122         re.DOTALL
123     )
```

```
    # 如果完全匹配，则给1分
    if full_format_match:
        return 1.0

    reward = 0.0
    # 如果有<think></think>标签对，则奖励加0.1分
    if think_match:
        reward += 0.1
    # 如果有<answer></answer>标签对，则奖励加0.5分
    if answer_match:
        reward += 0.5
    # 返回奖励
    return reward


def answer_reward_function(
    response: str, # 模型给出的回答
    numbers: List[int] = None, # 数字列表
    target: int = None # 目标数字
) -> float:
    """
    检查答案中：
    1. 是否使用了所有给的数字
    2. 每个数字是否使用了一次
    3. 答案中包含的表达式的求值结果是否等于目标数字
    """
    # 答案的正则表达式
    answer_regex = r"<answer>(.*?)<\/answer>"
    # 回答中是否有答案标签对
    answer_match = re.search(answer_regex, response, re.DOTALL)
    # 如果在回答中没有搜索到答案，那么给0分
    if not answer_match:
        return 0.0
    # 提取出答案的文本
    answer_content = answer_match.group(1)
    # 如果答案标签内没有东西，给0分
    if not answer_content:
        return 0.0
    # 如果答案标签中，除了表达式以外，还有其它内容，给0分
    allowed_chars = r"^[0-9+\-*/() ]+$"
    if not re.match(allowed_chars, answer_content):
        return 0.0

    # 检查答案中，每个数字是否只使用了一次
    used_numbers = [
        int(n) for n in re.findall(r"\d+", answer_content)
    ]
    if sorted(used_numbers) != sorted(numbers):
        return 0.0
```

```

    # 检查答案中包含的表达式的求值结果是否为目标数字
    try:
        result = eval(answer_content, {"__builtins__": None}, {})
        if abs(float(result) - float(target)) < 1e-5:
            return 1.0
    except:
        pass

    return 0.0


def reward_function(
    response: str,
    numbers: List[int] = None,
    target: int = None,
    end_token: str = None,
) -> Dict[str, Any]:
    """Countdown Task 的奖励函数。

    总奖励 = 0.1 * 格式奖励 + 答案准确性奖励
    """
    format_reward = format_reward_function(
        "<think>" + response,
        end_token
    )
    answer_reward = answer_reward_function(
        response,
        numbers,
        target
    )
    return {
        "reward": format_reward * 0.1 + answer_reward,
        "reward_info": {
            "format_reward": format_reward,
            "answer_reward": answer_reward,
        },
    }
```

12.2.3.4 GRPO（DAPO）算法的实现

创建文件 `grpo.py` ，内容如下：

```python
import dataclasses
import gc
import math
from collections import defaultdict
from typing import Callable, List

import numpy as np
import torch
```

```python

from data_types import Episode, MiniBatch
from qwen2_model import Transformer
from tokenizer import Tokenizer

# 采集轨迹，也就是回答
# 根据一个问题，采样多条回答
# 假设5个问题，每个问题8个回答，那么要并行采样40条轨迹
# 可能碰到的实现的坑，都来自并行推理
# 1. 问题的长度不一样
# 2. 有的补全结束的早，有的结束的晚。
@torch.no_grad()
def rollout(
    model: Transformer, # 生成回答的llm模型
    batch: MiniBatch, # N个问题
    tokenizer: Tokenizer,
    max_gen_len: int, # 最大生成长度
    num_answer_per_question: int, # 每个问题产生多少个回答
    reward_function: Callable, # 奖励函数
    device: torch.device,
    dtype: torch.dtype,
) -> List[Episode]:
    end_token = tokenizer.eos_token
    end_token_id = tokenizer.eos_token_id
    pad_token_id = tokenizer.pad_token_id
    # 问题：List[input_ids]
    prefix_token_ids = batch.prefix_token_ids
    # 批次中的问题数量 x 每个问题生成的回答数量 = 批次中的数据量
    bsz = len(batch.prefix) * num_answer_per_question
    # 最短问题长度
    min_prompt_len = min(len(t) for t in prefix_token_ids)
    # 最长问题长度
    max_prompt_len = max(len(t) for t in prefix_token_ids)
    # 总长度 = 最大生成长度 + 最大问题长度
    total_len = max_gen_len + max_prompt_len
    # 开启KV Cache，加速生成回答的速度
    model.init_kv_cache(
        max_batch_size=bsz,
        max_seq_len=total_len,
        device=device,
        dtype=dtype,
    )
    # 将所有token先初始化为填充符pad_token_id：批次中数据量 x 每条数据的总长度
    tokens = torch.full(
        (bsz, total_len),
        pad_token_id,
        dtype=torch.long,
        device=device
    )
```

```
    # 将问题部分填入
    # 第 k 个问题假设生成 num_answer_per_question 条回答
    # 那么 num_answer_per_question 条训练数据的前缀都是第k个问题
    for k, t in enumerate(prefix_token_ids):
        # 第k个问题的数据在批次中的偏移量
        offset = k * num_answer_per_question
        for i in range(num_answer_per_question):
            # 第k个问题的第i条完整数据的问题部分
            tokens[offset + i, : len(t)] = torch.tensor(
                t, dtype=torch.long, device=device
            )

    prev_pos = 0
    # 文本的掩码，填充符置为False
    input_text_mask = tokens != pad_token_id
    # 确保最小的问题长度小于总长度
    assert min_prompt_len < total_len
    # 标志位，标志一条回答（一条轨迹）是否结束，初始化为0
    is_finished = torch.zeros(
        (bsz,), dtype=torch.bool, device=device)
    # 并行的预测下一个token
    for cur_pos in range(min_prompt_len, total_len):
        print(
            f"\r* 生成轨迹:{cur_pos-min_prompt_len:>4d}/{total_len-
            min_prompt_len:>4d}",
            flush=True,
            end="",
        )
        # 针对批次中的所有训练数据，并行采样下一个token
        # 根据文本的 prev_pos~cur_pos 部分生成下一个token
        with torch.autocast(device_type=device.type, dtype=dtype):
            logits = model.inference(
                tokens[:, prev_pos:cur_pos],
                prev_pos
            )
        # logits ---> probs
        probs = torch.softmax(logits[:, -1], dim=-1)
        # 采样下一个token，具体使用了多项分布来采样
        next_token = torch.multinomial(probs, num_samples=1)
        next_token = next_token.reshape(-1)
        # 如果cur_pos这个索引已经有token了，那么直接作为下一个token
        # cur_pos这个所有已经有token，说明是长问题，那么这个token不需要预测
        # 注意：这里cur_pos对应的token不能是pad
        next_token = torch.where(
            input_text_mask[:, cur_pos], # cur_pos是否已经存在token了
            tokens[:, cur_pos], # 对于长的问题，cur_pos对应的已经有token了
            next_token # 对于最小长度的问题，选择预测出来的next_token
        )
        # 如果生成回答已经结束，那么下一个token是pad，
```

```
        # 如果没有结束，那么是next_token
        next_token = torch.where(
            is_finished,
            pad_token_id, # 对于短回答，回答已经结束，需要继续填充pad
            next_token # 对于长回答，回答没有结束，需要使用预测出来的token
        )
        # 将cur_pos赋值为下一个token
        tokens[:, cur_pos] = next_token
        # 如果有结尾标记
        if end_token_id is not None:
            # 检查这个结尾标记是否为生成下一个token得到的
            is_end_token = next_token == end_token_id
            # 如果cur_pos对应的是False，说明cur_pos是填充符
            # 说明这个token是生成的next token
            is_generated_token = ~input_text_mask[:, cur_pos]
            # 如果eos token是生成的，那么结束。
            is_finished = is_finished \
                          | (is_end_token & is_generated_token)
        prev_pos = cur_pos
        # 如果全部结束，那么跳出循环
        if is_finished.all():
            break
    # 删除kv cache
    model.del_kv_cache()
    # 手动垃圾回收
    gc.collect()
    # 清空cuda显存
    torch.cuda.empty_cache()
    is_finished_list = is_finished.tolist()
    tokens_list = tokens.tolist()

    # 准备存放输出回合的数组
    episodes = []
    # 遍历批次中的问题数量
    for i in range(bsz // num_answer_per_question):
        # 遍历第i条问题的第j条回答
        for j in range(num_answer_per_question):
            idx = i * num_answer_per_question + j
            # 截取出回答部分
            generated_token_ids =                   \
                tokens_list                         \
                [idx]                               \
                [len(batch.prefix_token_ids[i]):]
            # 删除填充token
            if pad_token_id in generated_token_ids:
                generated_token_ids = generated_token_ids[
                    :generated_token_ids.index(pad_token_id)
                ]
            # 生成的文本
```

```
            generated_text = \
                tokenizer.detokenize(generated_token_ids)
            # 计算第i个问题的第j条回答的奖励
            rewards = reward_function(
                # 生成的文本
                response=generated_text,
                # 数字列表
                numbers=batch.numbers[i],
                # 正确答案数字
                target=batch.target[i],
                end_token=end_token,
            )
            episode = Episode(
                prefix=batch.prefix[i],
                text=batch.prefix[i] + generated_text,
                prefix_token_ids=batch.prefix_token_ids[i],
                prefix_tokens=batch.prefix_tokens[i],
                generated_token_ids=generated_token_ids,
                is_finished=is_finished_list[idx],
                reward=rewards["reward"],
                reward_info=rewards["reward_info"],
            )
            episodes.append(episode)
    # 清除输出内容
    print("\r", end=" " * 100, flush=True)
    return episodes


def normalize_rewards_per_group(
    episodes: List[Episode]
) -> List[Episode]:
    """归一化每个组的奖励. 使用 prefix（问题） 区分不同的组."""
    """每条轨迹的reward字段替换为轨迹的优势"""
    groups = defaultdict(list)
    for episode in episodes:
        groups[tuple(episode.prefix)].append(episode)
    output = []
    # 遍历每个组，一个问题对应一组回答
    for group in groups.values():
        # [r_{i,0}, r_{i,1}, ...]
        group_rewards = [item.reward for item in group]
        # 每个组的回答的奖励的平均值
        mean_reward = np.mean(group_rewards)
        # 每个组的回答的奖励的标准差
        std_reward = np.std(group_rewards)
        # 遍历组中的每一条回答，然后计算这条回答的优势
        # (r_i - mean(r)) / (std(r) + ε)
        for episode in group:
            normalized_reward =                 \
```

```
                (episode.reward - mean_reward) \
                /                              \
                (std_reward + 1e-4)
            # reward字段，使用回答的组内优势替换掉奖励
            episode = dataclasses.replace(
                episode,
                reward=normalized_reward
            )
            output.append(episode)
    return output


def compute_entropy(logits: torch.Tensor) -> torch.Tensor:
    """计算熵，熵越小不确定性越小，用来监控模型训练的稳定性，不参与反向传播"""
    probs = torch.nn.functional.softmax(logits, dim=-1)
    entropy =                          \
        torch.logsumexp(logits, dim=-1) \
        -                              \
        torch.sum(probs * logits, dim=-1)
    return entropy


def update_policy(
    model, # 微调的模型
    optimizer, # 优化器
    episodes: List[Episode], # 轨迹（问题+回答）的数组
    micro_batch_size: int,
    pad_token_id: int,
    max_grad_norm: float, # 梯度裁剪，1.0
    device: torch.device,
    dtype: torch.dtype,
):
    """使用GRPO算法更新策略."""
    # 计算出每一条回答的组内优势
    episodes = normalize_rewards_per_group(episodes)
    # 按照回合的token数量排序，更有效的微批次训练
    episodes.sort(
        key=lambda x:                  \
        len(x.prefix_token_ids)        \
        +                              \
        len(x.generated_token_ids))
    num_target_tokens = sum(
        len(episode.generated_token_ids)
        for episode in episodes
    )
    entropy = 0.0

    for i in range(0, len(episodes), micro_batch_size):
        print(
```

```
            f"\r* 计算策略梯度: {i:>2d}/{len(episodes):>2d}",
            flush=True,
            end="",
        )
        j = min(i + micro_batch_size, len(episodes))
        batch_episodes = episodes[i:j]
        batch_lengths = [
            len(episode.prefix_token_ids)     \
            +                                 \
            len(episode.generated_token_ids)
            for episode in batch_episodes
        ]
        # 微批次中最长的轨迹长度
        batch_max_length = max(batch_lengths)
        batch_token_ids = [
            episode.prefix_token_ids      # 问题的input_ids
            + episode.generated_token_ids # 生成的回答的input_ids
            + [pad_token_id] * ( # 添加填充符pad
                  batch_max_length - batch_lengths[i]
              )
            for i, episode in enumerate(batch_episodes)
        ]
        batch_masks = [
            # 问题部分掩码是0
            [0] * len(episode.prefix_token_ids)
            # 回答部分掩码为1
            + [1] * len(episode.generated_token_ids)
            # 填充符掩码为0
            + [0] * (batch_max_length - batch_lengths[i])
            for i, episode in enumerate(batch_episodes)
        ]
        # 取出每个回合的优势 $\frac{r_i-\text{mean}(r)}{\text{std}(r)+\varepsilon}$
        batch_advantages = [
            episode.reward for episode in batch_episodes
        ]
        batch_token_ids = torch.tensor(
            batch_token_ids,
            device=device,
            dtype=torch.long
        )
        batch_masks = torch.tensor(
            batch_masks,
            device=device,
            dtype=torch.bool
        )
        batch_advantages = torch.tensor(
            batch_advantages, device=device, dtype=torch.float32
        )

```

```
302         with torch.autocast(device_type=device.type, dtype=dtype):
303             # 去掉最后一个token，输入
304             input_token_ids = batch_token_ids[:, :-1]
305             # 去掉第一个token，目标token
306             # 真实的目标token是来自上一轮的模型输出的回答
307             target_token_ids = batch_token_ids[:, 1:]
308             target_masks = batch_masks[:, 1:]
309             # logits是预测的下一个token
310             logits = model.forward(input_token_ids).float()
311         # 在 one-hot 分类里，
312         # 交叉熵等于对正确类别概率取负对数，
313         # 所以"负对数概率"与"交叉熵"指的是同一个目标函数。
314         # log π_θ(a|s) = −cross_entropy
315         # −∑_j a_j · log â_j = − log â_t，a_t是真实标签，â_t是模型预测为a_t的概率。
316         log_probs = -torch.nn.functional.cross_entropy(
317             logits.reshape(-1, logits.size(-1)),
318             target_token_ids.reshape(-1),
319             ignore_index=pad_token_id,
320             reduction="none",
321         ).reshape(input_token_ids.shape[0], -1)
322 
323         with torch.no_grad():
324             token_entropy = compute_entropy(logits)
325             entropy = entropy                                \
326                     +                                        \
327                     (token_entropy * target_masks).sum() \
328                     /                                        \
329                     num_target_tokens
330         # 对数概率乘以优势log π_θ(a|s) · A
331         obj = log_probs * batch_advantages[:, None]
332         # 计算每个token的平均目标
333         obj = (obj * target_masks).sum() / num_target_tokens
334         loss = -obj
335         # 每一轮都要进行反向传播，计算模型参数的导数，但不更新模型的参数
336         loss.backward()
337 
338     # 梯度裁剪
339     grad_norm = torch.nn.utils.clip_grad_norm_(
340         model.parameters(), max_norm=max_grad_norm
341     )
342     optimizer.step()
343     # 清空梯度
344     optimizer.zero_grad(set_to_none=True)
345     return {
346         "loss": loss.item(),
347         "grad_norm": grad_norm.item(),
348         "entropy": entropy.item(),
349     }
```

12.2.3.5 训练循环

创建文件 `train.py` ，内容如下：

```python
import html
import time
from argparse import ArgumentParser
from datetime import datetime
from pathlib import Path

import numpy as np
import torch
import yaml
from torch.utils.data import DataLoader
from torch.utils.tensorboard.writer import SummaryWriter

from countdown_task import CountdownTasksDataset, reward_function
from grpo import rollout, update_policy
from qwen2_model import Transformer
from tokenizer import Tokenizer


def evaluate(model, tokenizer, device, dtype):
    """每隔10个step，使用测试数据集评估一下，看能做对多少题"""
    test_dataset = CountdownTasksDataset(
        data_path="Countdown-Tasks-3to4",
        tokenizer=tokenizer,
        split="test",
        test_size=128, # 128条测试数据
    )
    generator = torch.Generator(device=device)
    # 批次大小减半，我们就可以生成2倍长的轨迹了
    dataloader = DataLoader(
        test_dataset,
        shuffle=False,
        collate_fn=CountdownTasksDataset.collate_fn,
        generator=generator,
        # 批次大小为256，减半
        batch_size=256 // 2,
        drop_last=False,
    )
    success = []
    for batch in dataloader:
        episodes = rollout(
            model=model,
            tokenizer=tokenizer,
            batch=batch,
            # 最大生成长度为1024，乘以2
            max_gen_len=1024 * 2,
            # 评估时，针对每个问题只生成1个回答
            num_answer_per_question=1,
```

```
48                 reward_function=reward_function,
49                 device=device,
50                 dtype=dtype,
51             )
52             success.extend([
53                 episode.reward_info["answer_reward"] \
54                 for episode in episodes
55             ])
56         return np.mean(success)
57
58
59     def main():
60         pretrained_model_path = Path(
61             "./Qwen2.5-3B-Instruct/"
62         )
63         device = torch.device("cuda")
64         dtype = torch.bfloat16
65         torch.set_default_device(device)
66         torch.random.manual_seed(1337)
67         # 批次大小
68         BATCH_SIZE = 256
69         # 每个批次32个问题
70         NUM_QUESTIONS_PER_BATCH = 32
71         # 每个问题产生8条回答
72         NUM_ANSWERS_PER_QUESTION = \
73             BATCH_SIZE // NUM_QUESTIONS_PER_BATCH
74
75         current_time = datetime.now().strftime(r"%Y%m%d-%H%M%S")
76         tb_writer = SummaryWriter(log_dir=f"./logs/{current_time}")
77         tokenizer = Tokenizer("./Qwen2.5-3B-Instruct/tokenizer.json")
78
79         train_dataset = CountdownTasksDataset(
80             data_path="./Countdown-Tasks-3to4/",
81             tokenizer=tokenizer,
82             split="train",
83             test_size=128,
84         )
85         generator = torch.Generator(device=device)
86         train_dataloader = DataLoader(
87             train_dataset,
88             shuffle=True,
89             collate_fn=CountdownTasksDataset.collate_fn,
90             generator=generator,
91             batch_size=NUM_QUESTIONS_PER_BATCH,
92         )
93
94         model = Transformer.from_pretrained(
95             pretrained_model_path,
96             device=device
97         ).train()
```

```
98
99      optimizer = torch.optim.AdamW(
100         model.parameters(),
101         lr=1.0e-5,
102         weight_decay=0.0,
103         betas=[0.9, 0.999],
104     )
105
106     start_time = time.time()
107     ckpt_dir = Path("ckpt")
108     ckpt_dir.mkdir(parents=True, exist_ok=True)
109
110     for step, batch in enumerate(train_dataloader, start=1):
111         # 生成轨迹（问题+回答）数据
112         episodes = rollout(
113             model=model,
114             tokenizer=tokenizer,
115             batch=batch,
116             max_gen_len=1024,
117             num_answer_per_question=NUM_ANSWERS_PER_QUESTION,
118             reward_function=reward_function,
119             device=device,
120             dtype=dtype,
121         )
122         # 更新策略
123         results = update_policy(
124             model=model,
125             optimizer=optimizer,
126             episodes=episodes,
127             micro_batch_size=2, # 微批次大小为2
128             pad_token_id=tokenizer.pad_token_id,
129             max_grad_norm=1.0, # 梯度裁剪到1.0
130             device=device,
131             dtype=dtype,
132         )
133         torch.cuda.synchronize()
134         end_time = time.time()
135         duration = end_time - start_time
136         start_time = end_time
137
138         # 计算一些统计信息，然后保存到日志文件中
139         reward = [episode.reward for episode in episodes]
140         formatted_reward = [
141             episode.reward_info["format_reward"] \
142             for episode in episodes
143         ]
144         answer_reward = [
145             episode.reward_info["answer_reward"] \
146             for episode in episodes
147         ]
```

```
        num_finished_episodes = sum(
            episode.is_finished for episode in episodes
        )
        mean_reward = np.mean(reward)
        std_reward = np.std(reward)
        success_rate = np.mean(answer_reward)
        format_reward = np.mean(formatted_reward)
        grad_norm = results["grad_norm"]
        entropy = results["entropy"]
        lr = optimizer.param_groups[0]["lr"]
        loss = results["loss"]
        mean_response_len = np.mean(
            [len(episode.generated_token_ids) \
             for episode in episodes]
        )
        print(
            f"\r步骤 {step}, 平均奖励: {mean_reward:.2f}, "
            f"计算正确率: {success_rate:.2f}, "
            f"梯度裁剪: {grad_norm:.2f}, 时长: {duration:.2f}, "
            f"结束的回合的数量: {num_finished_episodes}, "
            f"平均回答长度: {mean_response_len:.2f}, "
            f"熵: {entropy:.2f}"
        )
        # 每隔10步评估一次
        if step % 10 == 0:
            eval_success_rate = evaluate(
                model,
                tokenizer,
                device,
                dtype
            )
            print(f"\r评估数据集回答正确率: \
                  {eval_success_rate:.2f}" \
                  + " " * 100)
            tb_writer.add_scalar(
                "回答正确率/评估",
                eval_success_rate,
                step
            )

        tb_writer.add_scalar("损失", loss, step)
        tb_writer.add_scalar("平均奖励", mean_reward, step)
        tb_writer.add_scalar("奖励的标准差", std_reward, step)
        tb_writer.add_scalar(
            "回答正确率/训练", success_rate, step)
        tb_writer.add_scalar("格式奖励", format_reward, step)
        tb_writer.add_scalar("梯度裁剪", grad_norm, step)
        tb_writer.add_scalar("时长", duration, step)
        tb_writer.add_scalar(
```

```
197                 "结束的回合数量",
198                 num_finished_episodes,
199                 step
200             )
201             tb_writer.add_scalar("学习率", lr, step)
202             tb_writer.add_scalar(
203                 "平均回答长度", mean_response_len, step)
204             tb_writer.add_scalar("熵", entropy, step)
205             for i, episode in enumerate(episodes):
206                 # TensorBoard 将文本处理为markdown格式
207                 text = html.escape(episode.text)
208                 tb_writer.add_text(
209                     f"text_{i}",
210                     f"<pre>{text}</pre>",
211                     step
212                 )
213 
214             # 每隔100步保存模型的检查点
215             if step % 100 == 0:
216                 output_file = ckpt_dir / f"ckpt_{step:06d}.pt"
217                 torch.save(model.state_dict(), output_file)
218                 print(f"将检查点保存到 {output_file}")
219 
220 
221 if __name__ == "__main__":
222     main()
```

执行如下命令开始训练！

```
1 $ uv run train.py
```
Shell

12.3 GRPO 应用场景

12.3.1 医疗思维链

我们使用 Hugging Face 的 interleave_datasets 混合三个关键数据集：
PubMedQA（占总数据的 70%）：

- 临床问答，答案为 yes/no/maybe

GSM8K：

- 数学应用题，以保持数学推理能力

Health Benchmarks：

- 50+道医学专业选择题
- 从心脏病学到疫苗接种的类别

权重应该反映数据集的复杂性——PubMedQA 的曝光度增加了 3 倍，以处理其细微差别。我们这里对数据集进行了打乱。由于 PubMedQA 样本量增加了 3 倍，因此将这些样本展示给模型的几率也增加了 3 倍。
模型在训练过程中的答案

```
Prompt（提示）：

在以下科学背景下：

我们此前已经展示了对志愿者进行 4 周、共 19 次、每日亚红斑量紫外线（DUVR，模拟非极端暴露条件）照射所产生的不利影响。该光源的 UVA（320-400 nm）与 UVB（290-320 nm）辐照度比为 25，而典型用于代表夏季晴朗近天顶太阳辐照的模拟太阳光（SSR）的该比值接近 10。
在此，我们报告对之前研究的拓展，评估了一款广谱日常护肤产品（低 SPF：SPF 8，UVA-PF 7，UVA 3* 级防护）所提供的光防护。我们检测了与皮肤癌与光老化相关的细胞与分子光损伤标志物。
本研究显示，重复 DUVR 暴露所产生的生物学效应可以被一款广谱日常护理产品所预防，且其所提供的防护水平会随具体检测终点而变化。

回答下列问题：

一款广谱防晒是否能预防代表温带地区的日常亚红斑量太阳紫外线重复暴露所导致的累积性损伤？请用 "yes" / "no" / "maybe" 作答。你需要仔细阅读背景并推理后回答。

Model Response（模型回答）：

<reasoning>
提供的背景讨论了一项研究：重复的亚红斑量 UV（DUVR）暴露在 4 周内会产生不利影响。它还提到使用了一款广谱日常护理产品（SPF 8，UVA 防护因子 7，UVA 3* 评级）进行评估。研究的目的是判断该广谱产品是否能预防这类重复暴露引起的累积性损伤。文本中说明，该研究显示重复 DUVR 的生物学效应可以被这款产品预防，说明其具有保护作用。虽然不同终点的防护程度不同，但整体表述支持其确实能预防累积损伤。因此答案应为"yes"。
</reasoning>

<answer>
yes
</answer>
```

TRL 代码如下：

```python
from transformers import AutoModelForCausalLM, AutoTokenizer
from trl import GRPOConfig, GRPOTrainer
from datasets import load_dataset, Dataset, interleave_datasets, concatenate_datasets
import re


# Load and prep dataset
SYSTEM_PROMPT = """
Respond in the following format:
<reasoning>
...
</reasoning>
<answer>
...
</answer>
"""

XML_COT_FORMAT = """\
<reasoning>
```

```
{reasoning}
</reasoning>
<answer>
{answer}
</answer>
"""


def extract_xml_answer(text: str) -> str:
    answer = text.split("<answer>")[-1]
    answer = answer.split("</answer>")[0]
    return answer.strip()


def extract_hash_answer(text: str) -> str | None:
    if "####" not in text:
        return None
    return text.split("####")[1].strip()

# uncomment middle messages for 1-shot prompting


def get_datasets(split="train") -> Dataset:
    data = load_dataset('gsm8k',
                        'main')[split]  # type: ignore
    data = data.map(lambda x: {  # type: ignore
        'prompt': [
            {'role': 'system', 'content': SYSTEM_PROMPT},
            {'role': 'user', 'content': x['question']}
        ],
        'answer': extract_hash_answer(x['answer']),
        'db_set': 'gsm8k'
    })  # type: ignore
    data = data.remove_columns(['question'])

    # two times more than other datasets
    data_qa = load_dataset(
        "PubMedQA", "pqa_artificial")[split]
    data_qa = data_qa.filter(lambda x: len(
        "\n".join(x['context']['contexts'])) < 1024)  # avoid long traces
    data_qa = data_qa.map(lambda x: {  # type: ignore
        'prompt': [
            {'role': 'system', 'content': SYSTEM_PROMPT},
            {
                "role": "user",
                "content": "Given the scientific context below:\n" +
                "\n".join(x['context']['contexts']) +
                "\n\nAnswer the following question:\n" +
                x['question'] +
                " with 'yes', 'no' or 'maybe'. You need to carefully review the
context and reason before answering."
```

```
            },
        ],
        'answer': x['final_decision'],
        'db_set': 'pubmedqa'
    })  # type: ignore
    data_qa = data_qa.remove_columns(
        ['pubid', 'question', 'context', 'long_answer', 'final_decision'])

    categories = ['Lab_Medicine', 'Wearables', 'Dermatology', 'Gastroenterology',
'Internal_Medicine', 'Oncology', 'Orthopedics', 'General_Surgery',
'Ophthalmology', 'Audiology', 'Head_Neck_Surgery', 'Elderly_Care',
'Pediatrics', 'Allergy_Immunology', 'Rheumatology', 'Pharmacy',
'Obstetrics_Gynecology', 'Microbiology', 'Dentistry',
'Physical_Medicine_and_Rehabilitation', 'Neurology', 'Psychiatry',
'Pathology', 'Genetics', 'Rare_Diseases', 'Hematology',
                  'Emergency', 'Endocrinology', 'Radiology', 'Cardiology',
                  'Pulmonology', 'Infectious_Diseases', 'Critical_Care',
                  'Pediatric_Surgery', 'Neuroscience', 'Epidemiology',
                  'Fitness_Sports', 'Health_Education', 'Health_Economics',
                  'Health_Entrepreneurship', 'Hospital_Management',
                  'Mental_Health', 'Nutrition', 'Palliative_Care',
                  'Preventive_Medicine', 'Public_Health',
                  'Social_Media_Addiction', 'Sleep', 'Supplements', 'Vaccination',
                  'Work_Health', 'Wellbeing']
    data_mc = concatenate_datasets(
        [load_dataset("Health_Benchmarks", i)[i] for i in categories])
    data_mc = data_mc.map(lambda x: {  # type: ignore
        'prompt': [
            {'role': 'system', 'content': SYSTEM_PROMPT},
            {
                "role": "user",
                "content": "\n\nAnswer the following question:\n" +
                x['Questions'] +
                "\n With 'A', 'B', 'C' or 'D'. You need to carefully review the
context and reason before answering."
            },
        ],
        'answer': x['Answers'],
        'db_set': 'med_mc'
    })  # type: ignore
    data_mc = data_mc.remove_columns(['Answers', 'Questions'])

    dataset = concatenate_datasets([data, data_qa, data_mc])
    return dataset


dataset = get_datasets()
dataset = dataset.shuffle(seed=42)
train_test_split = dataset.train_test_split(test_size=0.1)
train_dataset = train_test_split["train"]
test_dataset = train_test_split["test"]
print(f"train size: {len(train_dataset)}, test size: {len(test_dataset)}")


```

```python
# Reward functions
def correctness_reward_func(prompts, completions, answer, db_set, **kwargs) -> list[float]:
    responses = [completion[0]['content'] for completion in completions]
    q = prompts[0][-1]['content']
    extracted_responses = [extract_xml_answer(r) for r in responses]
    print('-'*20, f"Question:\n{q}", f"\nAnswer:\n{answer[0]}",
          f"\nResponse:\n{responses[0]}", f"\nExtracted:\n{extracted_responses[0]}")
    rewards = []
    for r, a, dt in zip(extracted_responses, answer, db_set):
        if dt == "gsm8k":
            if a in r:
                rewards.append(1.0)
            elif r == a:
                rewards.append(2.0)
            else:
                rewards.append(0.0)
        else:
            rewards.append(2.0 if r.lower() == a.strip().lower() else 0.0)
    return rewards


def int_reward_func(completions, db_set, **kwargs) -> list[float]:
    responses = [completion[0]['content'] for completion in completions]
    extracted_responses = [extract_xml_answer(r) for r in responses]
    rewards = []
    for r, dt in zip(extracted_responses, db_set):
        if dt == "gsm8k":
            rewards.append(0.5 if r.isdigit() else 0.0)
        elif dt == "pubmedqa":
            rewards.append(0.5 if (
                'yes' in r.lower() or 'no' in r.lower() or 'maybe' in r.lower())
                else 0.0)
        else:
            rewards.append(0.5 if ('a' in r.lower() or 'b' in r.lower(
            ) or 'c' in r.lower() or 'd' in r.lower()) else 0.0)
    return rewards


def strict_format_reward_func(completions, **kwargs) -> list[float]:
    """Reward function that checks if the completion has a specific format."""
    pattern = r"^<reasoning>\n.*?\n</reasoning>\n<answer>\n.*?\n</answer>\n$"
    responses = [completion[0]["content"] for completion in completions]
    matches = [re.match(pattern, r) for r in responses]
    return [0.5 if match else 0.0 for match in matches]


def soft_format_reward_func(completions, **kwargs) -> list[float]:
    """Reward function that checks if the completion has a specific format."""
    pattern = r"<reasoning>.*?</reasoning>\s*<answer>.*?</answer>"
```

```
    responses = [completion[0]["content"] for completion in completions]
    matches = [re.match(pattern, r) for r in responses]
    return [0.5 if match else 0.0 for match in matches]


def count_xml(text) -> float:
    count = 0.0
    if text.count("<reasoning>\n") == 1:
        count += 0.125
    if text.count("\n</reasoning>\n") == 1:
        count += 0.125
    if text.count("\n<answer>\n") == 1:
        count += 0.125
        count -= len(text.split("\n</answer>\n")[-1])*0.001
    if text.count("\n</answer>") == 1:
        count += 0.125
        count -= (len(text.split("\n</answer>")[-1]) - 1)*0.001
    return count


def xmlcount_reward_func(completions, **kwargs) -> list[float]:
    contents = [completion[0]["content"] for completion in completions]
    return [count_xml(c) for c in contents]


training_args = GRPOConfig(output_dir="outputs", num_generations=2,
per_device_train_batch_size=4)


# 加载模型
model = AutoModelForCausalLM.from_pretrained(
    "../GRPO/Qwen2.5-3B-Instruct/",
    torch_dtype="auto",
    device_map="auto"
)

tokenizer = AutoTokenizer.from_pretrained(
    "../GRPO/Qwen2.5-3B-Instruct/")


trainer = GRPOTrainer(
    model=model,
    processing_class=tokenizer,
    reward_funcs=[
        xmlcount_reward_func,
        soft_format_reward_func,
        strict_format_reward_func,
        int_reward_func,
        correctness_reward_func,
    ],
```

```
206     args=training_args,
207     train_dataset=train_dataset,
208     eval_dataset=test_dataset,
209 )
210 trainer.train()
```

12.3.2 Text-To-SQL

GITHUB 地址

底座模型：Qwen2.5-Coder-7B-Instruct

任务

- 输入：请帮我写一段 SQL，要求查询出部门 35 岁以上的程序员
- 输出：SELECT * FROM database WHERE ...

该模型展现出强大的整体性能，SQL 生成准确率高（44/5 分，得分 4 或 5 分），推理质量优异（48/50 分，得分 4 或 5 分），格式遵循近乎完美（49/50 分，得分 5 分），且具有明确的教育价值。总体而言，88% 的输出得分达到 4.0 分或更高，反映出模型结果的一致性、结构良好且易于解释。

> 🔥 奖励函数编写要点
>
> 1. 使用正则表达式来实现格式奖励
> 2. 创建一个 sqlite3 数据库，用来校验 SQL 语句的正确性
> 3. 调用 DeepSeek 或者 GPT 接口，让外部大语言模型评估一下 CoT 的质量。

IV

多模态

13 Vision Transformer 219
13.1 导入库和模块 220
13.2 补丁嵌入（Patch Embedding） . . . 220
13.3 类别对应的 token 和位置编码 222
13.4 注意力头 224
13.5 多头注意力 226
13.6 Transformer 编码器 227
13.7 Vision Transformer 228
13.8 训练参数 230
13.9 加载 MNIST 数据集 230
13.10 训练循环 231
13.11 评估模型 231

14 CLIP 233
14.1 导入库和模块 233
14.2 图像和文本编码器 234
14.3 CLIP 模型 240

15 ClipCap（图生文） 251
15.1 ClipCap 的工作原理 251
15.2 关于训练 252
15.3 推理 252
15.4 ClipCap 的代码实现 253
15.5 应用：图生文 263
15.6 拓展：Qwen3-VL 264

16 扩散模型 265
16.1 通俗讲解 265
16.2 扩散模型理论简介 270
16.3 代码实现 279
16.4 完整代码 289

17 扩散模型背后的数学理论 297
17.1 正向扩散过程 298
17.2 逆向扩散过程 300

18 条件扩散模型 311
18.1 向扩散模型添加条件 311
18.2 条件扩散模型的实现 313
18.3 得分函数 321
18.4 分类器指引 324
18.5 无分类器指引 327
18.6 Stable Diffusion 334

19 DALL-E2 （文生图） 337
19.1 CLIP 338
19.2 先验模型（Prior Model） 338
19.3 条件扩散模型 352
19.4 训练配置 371
19.5 训练 373
19.6 训练结果 376

20 总结 379

13. Vision Transformer

> 提示
>
> ViT: Vision Transformer

基于自注意力的 Transformer 模型由 Vaswani 等人在 2017 年的论文《Attention Is All You Need》中首次提出，并已广泛应用于自然语言处理中。Transformer 模型是 OpenAI 用来创建 ChatGPT 的模型。Transformer 不仅适用于文本，还适用于图像，基本上可以处理任何序列数据。2021 年，Dosovitsky 等人在他们的论文《An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale》中引入了将 Transformer 用于计算机视觉任务（例如图像分类）的想法。在他们的论文中，与卷积网络相比，他们的 Vision Transformer 模型能够取得出色的结果，并且需要更少的资源来训练。

在本教程中，我们将从头开始构建一个 Vision Transformer 模型，并在 MNIST 数据集上进行测试。

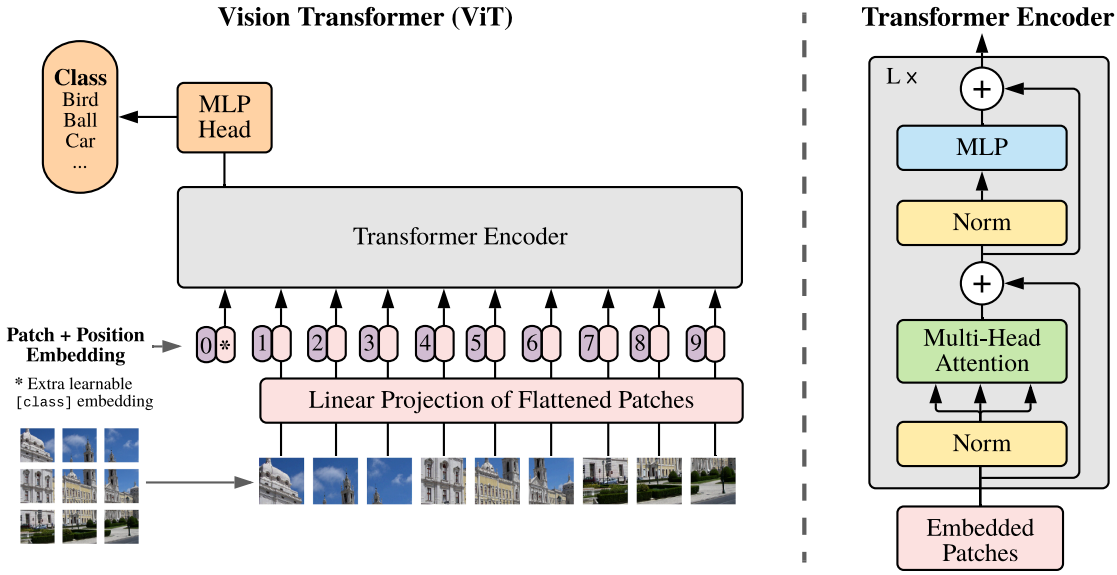


图 13.1 Vit 架构图

13.1 导入库和模块

本章所需依赖 Python

```python
import torch
import torch.nn as nn
import torchvision.transforms as T
from torch.optim import Adam
from torchvision.datasets.mnist import MNIST
from torch.utils.data import DataLoader
import numpy as np
```

13.2 补丁嵌入（Patch Embedding）

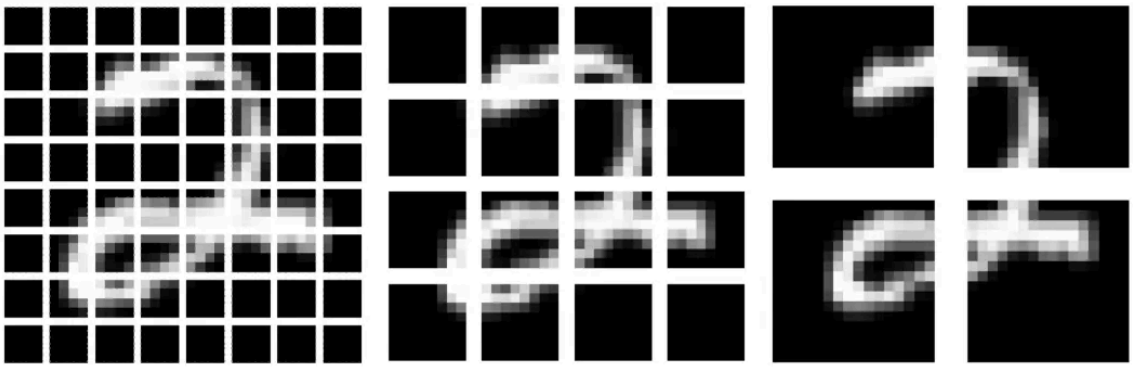


图 13.2 补丁的例子

补丁嵌入类 Python

```python
class PatchEmbedding(nn.Module):
    def __init__(
        self,
        d_model,    # 模型的维度
        img_size,   # 图片大小
        patch_size, # 补丁大小
        n_channels  # 通道数量
    ):
        super().__init__()

        self.d_model = d_model
        self.img_size = img_size
        self.patch_size = patch_size
        self.n_channels = n_channels

        self.linear_project = nn.Conv2d(
            self.n_channels, # in_channels
            self.d_model, # out_channels
            kernel_size=self.patch_size, # kernel_size
            stride=self.patch_size # stride
        )
```

```
22
23      # B: 批次大小
24      # C: 通道数量
25      # H: 图像高度
26      # W: 图像宽度
27      # P_col: 补丁的列
28      # P_row: 补丁的行
29      def forward(self, x):
30          x = self.linear_project(x) # (B, C, H, W) → (B, d_model, P_col, P_row)
31          x = x.flatten(2) # (B, d_model, P_col, P_row) → (B, d_model, P)
32          x = x.transpose(1, 2) # (B, d_model, P) → (B, P, d_model)
33          return x
```

创建 Vision Transformer 的第一步是将输入图像拆分为补丁，并创建这些补丁的线性嵌入序列。我们能够通过使用 PyTorch 的 Conv2d 方法来实现这一点。

Conv2d 方法获取输入图像，将它们拆分为补丁，并提供大小等于 d_model 的线性投影。通过将 kernel_size 和步幅设置为补丁大小，我们确保补丁大小正确且没有重叠。

```python
self.linear_project = nn.Conv2d(
    self.n_channels,
    self.d_model,
    kernel_size=self.patch_size,
    stride=self.patch_size
)
```

在 forward 方法中，我们通过 linear_project/Conv2d 方法传递具有形状 (B, C, H, W) 的输入，并输出形状 (B, d_model, P_col, P_row) 的张量。

```python
def forward(self, x):
    x = self.linear_project(x) # (B, C, H, W) → (B, d_model, P_col, P_row)
```

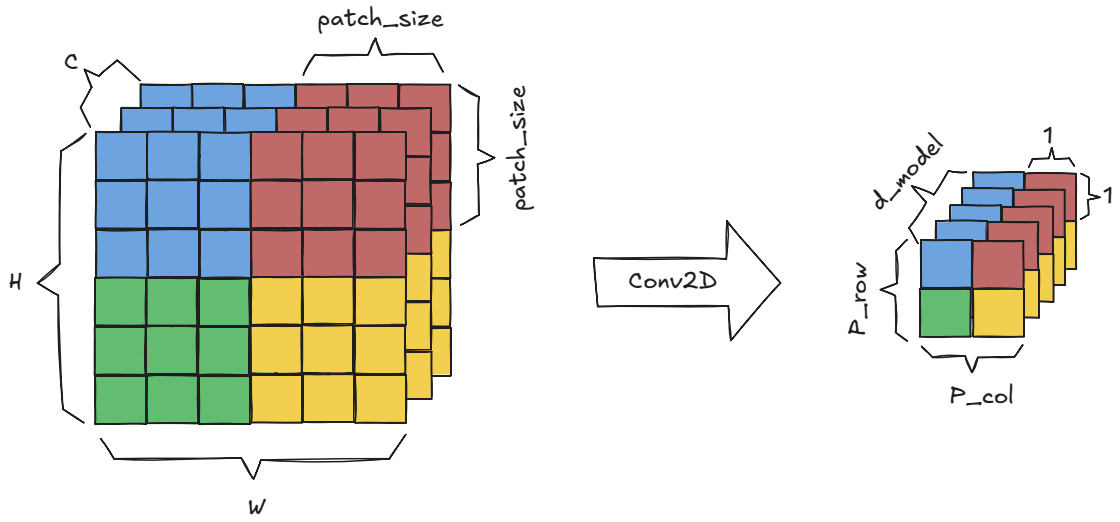


图 13.3 将图像转换为补丁：第一步

我们使用展平方法将补丁列和补丁行维度组合成一个补丁维度，从而得到 (B, d_model, P) 的形状

```python
x = x.flatten(2) # (B, d_model, P_col, P_row) → (B, d_model, P)
```

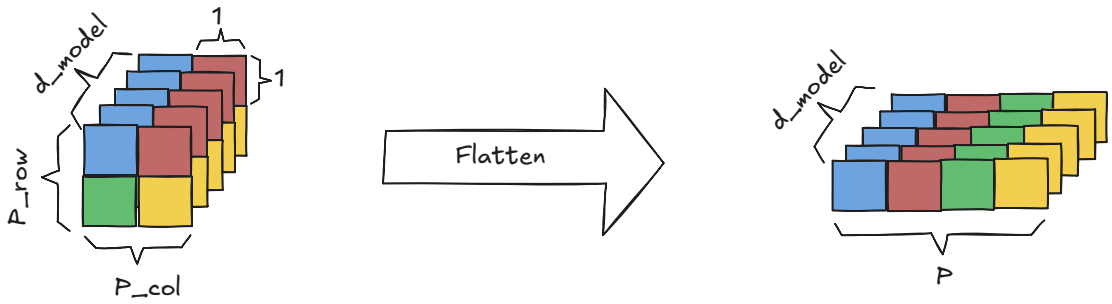


图 13.4 将图像转换为补丁：第二步

最后，我们使用转置方法切换 d_model 和补丁维度，得到 (B, P, d_model) 的形状。

```python
x = x.transpose(1, 2) # (B, d_model, P) → (B, P, d_model)
```

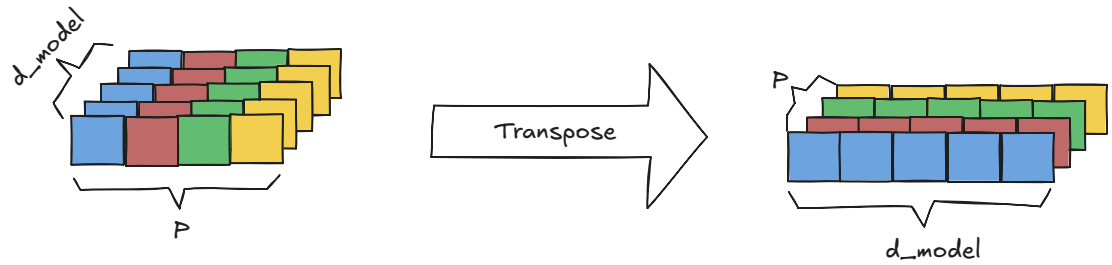


图 13.5 将图像转换为补丁：第三步

13.3 类别对应的 token 和位置编码

位置编码类

```python
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_seq_length):
        super().__init__()
        # 类别token
        self.cls_token = nn.Parameter(torch.randn(1, 1, d_model))
        # 创建位置编码
        pe = torch.zeros(max_seq_length, d_model)

        for pos in range(max_seq_length):
            for i in range(d_model):
                if i % 2 == 0:
                    pe[pos][i] = np.sin(pos/(10000 ** (i/d_model)))
                else:
                    pe[pos][i] = np.cos(pos/(10000 ** ((i-1)/d_model)))
        # 将位置编码固定
        self.register_buffer("pe", pe.unsqueeze(0))

    def forward(self, x):
        # 为批次中的每张图片分配一个类别token
```

```
        tokens_batch = self.cls_token.expand(x.size()[0], -1, -1)
        # 将类别token添加到每个图像的补丁嵌入数组的开头
        x = torch.cat((tokens_batch,x), dim=1)
        # 将位置编码添加到嵌入中
        x = x + self.pe
        return x
```

ViT 模型使用向补丁嵌入添加可学习的类别 token 的标准方法来执行分类。

```
# class token: 类别或者分类对应的token
self.cls_token = nn.Parameter(torch.randn(1, 1, d_model))
```

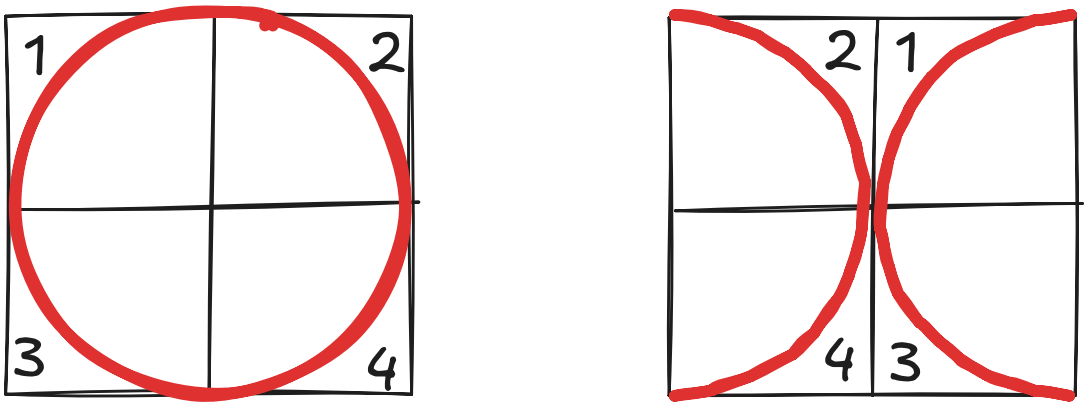


图 13.6 位置编码的作用

与 LSTM 等按顺序接受嵌入的模型不同，Transformer 并行的接受嵌入。虽然这提高了速度，但 Transformer 不知道序列的顺序是什么。这是一个很大的问题，因为改变序列的顺序很可能会改变其含义。上图就是一个例子，它显示更改图像的补丁顺序可以将图像从 O 更改为更接近 X 的图像。为了解决这个问题，需要将位置编码添加到嵌入中。每个位置编码对于它所表示的位置都是唯一的，这允许模型识别每个补丁嵌入应该在的位置。为了将位置编码添加到补丁嵌入中，它们必须具有相同的维度，`d_model` 。我们使用下面的公式获得位置编码。

定义 **13.1** — **位置编码.**

$$\begin{aligned} \mathrm{PE}_{(\mathrm{pos},2i)} &= \sin\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d_{\mathrm{model}}}}}\right) \\ \mathrm{PE}_{(\mathrm{pos},2i+1)} &= \cos\left(\frac{\mathrm{pos}}{10000^{\frac{2i}{d_{\mathrm{model}}}}}\right) \end{aligned} \tag{13.1}$$

```
# max_seq_length：补丁嵌入的数量，d_model：嵌入维度
pe = torch.zeros(max_seq_length, d_model)

for pos in range(max_seq_length):
    for i in range(d_model):
        if i % 2 == 0:
            pe[pos][i] = np.sin(pos/(10000 ** (i/d_model)))
        else:
            pe[pos][i] = np.cos(pos/(10000 ** ((i-1)/d_model)))
```

```
10  # 禁止pe更新，pe保存在了显存中，不会被反向传播更新
11  self.register_buffer('pe', pe.unsqueeze(0))
```

在 forward 方法中，输入是多个图像的一批补丁嵌入。例如，32x32 的图像可以分解为 16 个 8x8 大小的补丁。在此 max_seq_length 中，需要为 16+1=17 才能创建足够的位置嵌入，每个补丁一个，分类对应的 token 一个。

因此，我们需要使用 expand 函数才能使用 self.cls_token 为批处理中的每个图像创建分类对应的 token 。注意力机制会将有关整个序列的信息编码到序列中的每个 token 中。由于每个 token 都受到其自身信息的偏见，因此分类对应的 token 会创建序列中所有 token 的独立的摘要信息。

```python
def forward(self, x):
    tokens_batch = self.cls_token.expand(x.size()[0], -1, -1)
```

然后，使用 torch.cat 方法将这些分类 token 添加到每个补丁嵌入的开头。

```python
x = torch.cat((tokens_batch,x), dim=1)
```

位置编码在输出之前添加。

```python
x = x + self.pe
return x
```

这是添加分类 token 之前和之后的数据的样子：

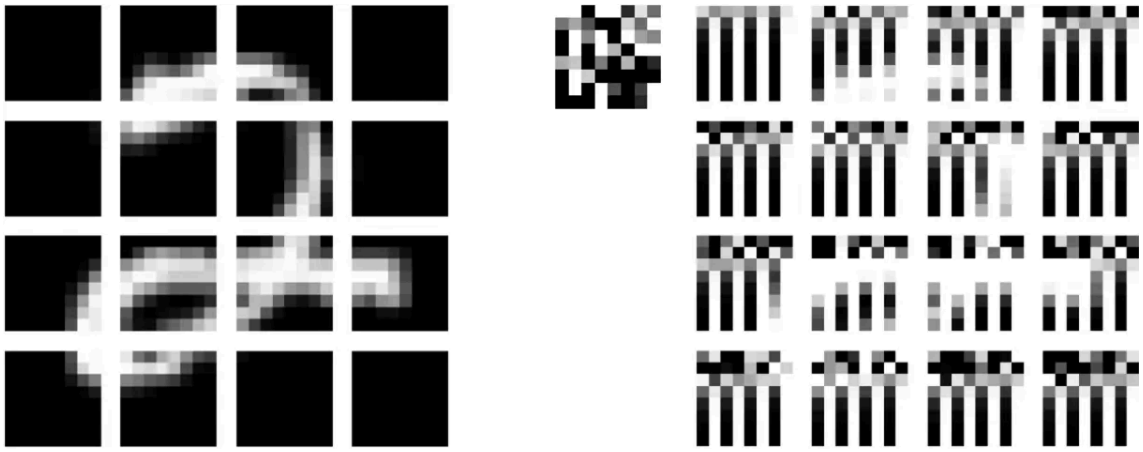


图 13.7 左图：使用 Conv2d 运算分成 16 个 8x8 块的 32x32 MNIST 图像。右图：添加位置编码和类别 token 后的 16 个图像补丁，使用随机数据初始化。

请注意，我们已经用 64 个卷积核初始化了 Conv2d 运算，每个卷积核中的每个补丁只占用一个像素，以免扭曲图像。

13.4 注意力头

注意力头

```python
class AttentionHead(nn.Module):
    def __init__(self, d_model, head_size):
        super().__init__()
        self.head_size = head_size

        self.query = nn.Linear(d_model, head_size)
```

```
        self.key = nn.Linear(d_model, head_size)
        self.value = nn.Linear(d_model, head_size)

    def forward(self, x):
        # 计算Q, K, V
        Q = self.query(x)
        K = self.key(x)
        V = self.value(x)

        # QK^T
        attention = Q @ K.transpose(-2,-1)

        # softmax(QK^T / sqrt(d_head))V
        attention = attention / (self.head_size ** 0.5)
        attention = torch.softmax(attention, dim=-1)
        attention = attention @ V

        return attention
```

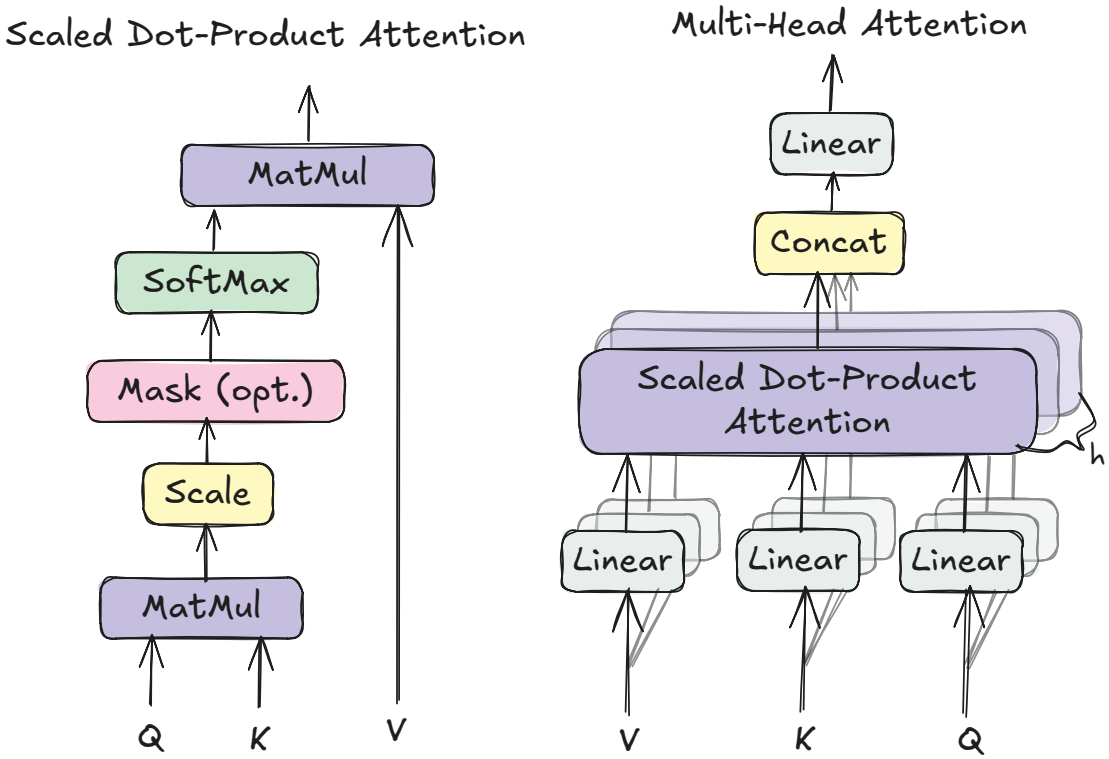


图 13.8 注意力机制

ViT 使用注意力机制，这是一种通信机制，允许模型专注于图像的重要部分。可以使用下面的公式计算注意力分数。

$$\text{Attention}\ (Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{13.2}$$

计算注意力的第一步是获取 token 的 Q、K 和 V。token 的 Q 是 token 要查找的内容，K 是 token 包含的内容，V 是 token 之间的互信息。Q、K 和 V 可以通过线性层传递 token 来计算。

```python
def forward(self, x):
    # 计算Q, K, V
    Q = self.query(x)
    K = self.key(x)
    V = self.value(x)
```

我们能够通过计算 Q 和 K 的点积来获取序列中 token 之间的关系。

```python
attention = Q @ K.transpose(-2,-1)
```

我们需要缩放这些值以控制初始化时的方差，以便 token 能够聚合来自多个其他 token 的信息。通过将点积除以注意力头大小的平方根来应用缩放。

```python
attention = attention / (self.head_size ** 0.5)
```

然后，我们需要对缩放的点积应用 `softmax` 。

```python
attention = torch.softmax(attention, dim=-1)
```

最后，我们需要得到 `softmax` 和 V 矩阵之间的点积。这本质上是在相应的 token 之间传递信息。

```python
attention = attention @ V
return attention
```

13.5 多头注意力

多头注意力

```python
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_heads):
        super().__init__()
        self.head_size = d_model // n_heads
        self.W_o = nn.Linear(d_model, d_model)
        self.heads = nn.ModuleList([
            AttentionHead(d_model, self.head_size) for _ in range(n_heads)
        ])

    def forward(self, x):
        # 拼接多个注意力头
        out = torch.cat([head(x) for head in self.heads], dim=-1)
        out = self.W_o(out)
        return out
```

多头注意力只是并行运行多个头的自注意力并将它们组合起来。我们可以通过将注意力头添加到模块列表中来做到这一点，

```python
self.heads = nn.ModuleList([
  AttentionHead(d_model, self.head_size)
  for _ in range(n_heads)
])
```

传递输入然后拼接。

```python
def forward(self, x):
```

```
    # 拼接多个注意力头
    out = torch.cat([head(x) for head in self.heads], dim=-1)
```

然后，我们需要将输出传递给另一个线性层。

```python
out = self.W_o(out)
return out
```

13.6 Transformer 编码器

Transformer编码器

```python
class TransformerEncoder(nn.Module):
    def __init__(self, d_model, n_heads, r_mlp=4):
        super().__init__()
        self.d_model = d_model
        self.n_heads = n_heads

        # 层归一化
        self.ln1 = nn.LayerNorm(d_model)

        # 多头注意力
        self.mha = MultiHeadAttention(d_model, n_heads)

        # 层归一化
        self.ln2 = nn.LayerNorm(d_model)

        # MLP
        self.mlp = nn.Sequential(
            nn.Linear(d_model, d_model*r_mlp),
            nn.GELU(),
            nn.Linear(d_model*r_mlp, d_model)
        )

    def forward(self, x):
        out = x + self.mha(self.ln1(x)) # 跳跃连接
        out = out + self.mlp(self.ln2(out)) # 跳跃连接
        return out
```

Transformer 编码器由两个子层组成：第一个子层执行多头注意力，第二个子层包含 MLP。多头注意力子层执行 token 之间的通信，而 MLP 子层允许 token 单独“思考”与它们通信的内容。

层归一化是一种优化技术，可跨其特征独立归一化批处理中的每个输入。对于我们的模型，我们将在每个子层的开头通过层归一化模块传递我们的输入。

```python
self.ln1 = nn.LayerNorm(d_model)
self.ln2 = nn.LayerNorm(d_model)
```

MLP 将由两个线性层组成，中间有一个 GELU 层。使用 GELU 代替 RELU，因为它没有 RELU 在零点不可导的限制。

```
    # 编码器的MLP
    self.mlp = nn.Sequential(
        nn.Linear(width, width*r_mlp),
```

```
        nn.GELU(),
        nn.Linear(width*r_mlp, width)
    )
```

在编码器的 forward 方法中，输入在执行多头注意力之前通过第一层归一化模块。通过执行多头注意力将原始输入添加到输出中，以创建残差连接。

然后，在输入 MLP 之前，它会通过另一个层归一化模块。通过将 MLP 的输出添加到第一个残差连接的输出，创建另一个残差连接。

残差连接用于通过创建一条路径来帮助防止梯度消失问题，以便梯度不受阻碍地反向传播回原始输入。

Python
```
def forward(self, x):
    out = x + self.mha(self.ln1(x))
    out = out + self.mlp(self.ln2(out))
    return out
```

13.7 Vision Transformer

ViT类 Python
```
class VisionTransformer(nn.Module):
    def __init__(
        self,
        d_model,
        n_classes,
        img_size,
        patch_size,
        n_channels,
        n_heads,
        n_layers
    ):
        super().__init__()

        assert img_size[0] % patch_size[0] == 0  \
           and img_size[1] % patch_size[1] == 0, \
           "img_size 必须能被 patch_size 整除"
        assert d_model % n_heads == 0, \
           "d_model 必须能被 n_heads 整除"

        self.d_model = d_model # 模型维度，嵌入的维度（宽度）
        self.n_classes = n_classes # 类别的数量
        self.img_size = img_size # 图片大小
        self.patch_size = patch_size # 补丁大小
        self.n_channels = n_channels # 通道数
        self.n_heads = n_heads # 注意力头的数量
        # 补丁的数量 = 32×32/4×4
        self.n_patches = (self.img_size[0] * self.img_size[1]) \
                         // (self.patch_size[0] * self.patch_size[1])
        # 序列的长度 = 1（分类token） + 补丁的数量
        self.max_seq_length = self.n_patches + 1
        # 补丁嵌入
```

```
        self.patch_embedding = PatchEmbedding(
            self.d_model,
            self.img_size,
            self.patch_size,
            self.n_channels
        )
        # 位置编码
        self.positional_encoding = PositionalEncoding(
            self.d_model,
            self.max_seq_length
        )
        self.transformer_encoder = nn.Sequential(*[
            TransformerEncoder(self.d_model, self.n_heads)
            for _ in range(n_layers)
        ])

        # 用于分类的MLP
        self.classifier = nn.Sequential(
            nn.Linear(self.d_model, self.n_classes),
            nn.Softmax(dim=-1)
        )

    def forward(self, images):
        # 将图片转换成补丁的嵌入（embedding）
        x = self.patch_embedding(images)
        # 添加位置编码
        x = self.positional_encoding(x)
        # 编码
        x = self.transformer_encoder(x)
        # 分类的线性层
        x = self.classifier(x[:,0])
        return x
```

在创建我们的 `VisionTransformer` 类时，我们首先需要确保输入图像可以均匀地分成大小为 `patch_size` 的补丁，并且模型的维数可以被注意力头的数量整除。

```python
assert img_size[0] % patch_size[0] == 0  \
   and img_size[1] % patch_size[1] == 0, \
   "img_size 必须能被 patch_size 整除"
assert d_model % n_heads == 0, \
   "d_model 必须能被 n_heads 整除"
```

我们还需要计算位置编码的最大序列长度，该长度等于补丁的数量加 1。补丁的数量可以通过将输入图像的高和宽的乘积除以补丁大小的高和宽的乘积来计算。

```python
self.n_patches = (self.img_size[0] * self.img_size[1]) \
               // (self.patch_size[0] * self.patch_size[1])
self.max_seq_length = self.n_patches + 1
```

`VisionTransformer` 还需要能够拥有多个编码器模块。这可以通过将编码器层列表放入顺序包装器中来实现。

```python
self.encoder = nn.Sequential(*[
    TransformerEncoder(self.d_model, self.n_heads) for _ in range(n_layers)
])
```

VisionTransformer 模型的最后一部分是 MLP 分类头。它由一个线性层和一个 softmax 层组成。

```python
self.classifier = nn.Sequential(
    nn.Linear(self.d_model, self.n_classes),
    nn.Softmax(dim=-1)
)
```

在 forward 方法中，输入图像首先经过补丁嵌入层，将图像分割成补丁，并获取这些补丁的线性嵌入序列。然后，它们经过位置编码层，添加分类 token 和位置编码，最后经过编码器模块。之后，分类 token 再经过分类 MLP，以确定图像的类别。

```python
def forward(self, images):
    x = self.patch_embedding(images)
    x = self.position_embedding(x)
    x = self.encoder(x)
    x = self.classifier(x[:,0])
    return x
```

我们已经完成了模型的构建。现在我们需要对其进行训练和测试。

13.8 训练参数

```python
d_model = 9 # 嵌入的维度9
n_classes = 10 # 类别数量为10
img_size = (32,32) # 图片大小为32x32
patch_size = (16,16) # 补丁的大小是16x16
n_channels = 1 # 灰度图片通道数量为1
n_heads = 3 # 3个注意力头
n_layers = 3 # 3层编码器
batch_size = 128 # 每个批次128张图片
epochs = 5 # 训练5个epoch
alpha = 0.005 # 学习率5e-3
```

13.9 加载 MNIST 数据集

```python
transform = T.Compose([
    T.Resize(img_size), # 28x28 ⟶ 32x32
    T.ToTensor() # 转换成torch.tensor
])

train_set = MNIST(
    root="datasets", train=True, download=True, transform=transform
)
test_set = MNIST(
    root="datasets", train=False, download=True, transform=transform
)

```

```
train_loader = DataLoader(train_set, shuffle=True, batch_size=batch_size)
test_loader = DataLoader(test_set, shuffle=False, batch_size=batch_size)
```

13.10 训练循环

下面的代码使用 MNIST 训练集训练我们的 VisionTransformer 类，并展示了整个 epoch 的平均损失。

```python
device = torch.device("cuda")

ViT = VisionTransformer(
    d_model,
    n_classes,
    img_size,
    patch_size,
    n_channels,
    n_heads,
    n_layers
).to(device)

optimizer = Adam(ViT.parameters(), lr=alpha)
criterion = nn.CrossEntropyLoss()

for epoch in range(epochs):
    training_loss = 0.0
    for i, data in enumerate(train_loader, 0):
        # 取出图像和对应的标签
        inputs, labels = data
        inputs, labels = inputs.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = ViT(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        training_loss += loss.item()

    print(f"Epoch {epoch + 1}/{epochs} loss: \
            {training_loss  / len(train_loader) :.3f}")
```

13.11 评估模型

```python
correct = 0
total = 0

with torch.no_grad():
    for data in test_loader:
        images, labels = data
        images, labels = images.to(device), labels.to(device)
```

```
        outputs = ViT(images)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
    print(f"\n预测准确率: {100 * correct // total} %")
```

使用此模型，我们仅用 5 个 epoch 就能够在 MNIST 数据集上实现约 92% 的准确率。此示例展示了自注意力机制可以替代深度卷积网络。

14. CLIP

计算机视觉系统历来仅限于一组固定的类别，CLIP 是一场革命，它允许通过“预测哪些图像和文本配对在一起”来识别开放世界中的对象。CLIP 能够通过学习批量训练数据的图像和文本特征之间的余弦相似度来预测这一点。这在下图的对比预训练部分显示，其中图像之间的点积特征 $\{I_1, I_2, ..., I_N\}$ 和文本特征 $\{T_1, T_2, ..., T_N\}$ 被占用。

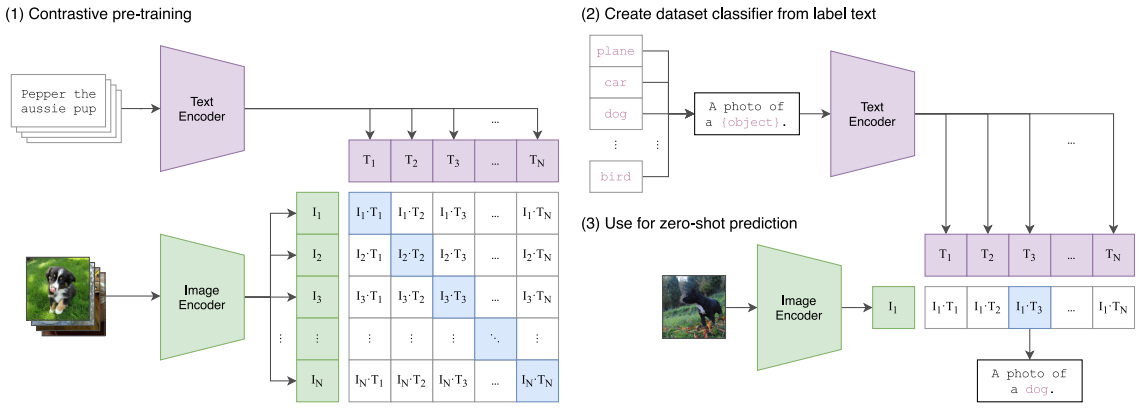


图 14.1 clip 原理，来自原始论文

在本章中，我们将从头开始构建 CLIP 并在 MNIST 数据集上对其进行测试。

14.1 导入库和模块

```python
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision.transforms as T
from torch.utils.data import Dataset, DataLoader
from datasets import load_dataset
import matplotlib.pyplot as plt
import numpy as np
```

14.2 图像和文本编码器

我们将首先构建图像和文本编码器。两者分别将图像和文本嵌入到单个token中，然后可用于对比损失计算。

14.2.1 位置嵌入

```python
class PositionalEmbedding(nn.Module):
    def __init__(self, width, max_seq_length):
        super().__init__()
        # 创建一个 (token的数量, 嵌入的维度) 形状的全0张量
        # width 就是 d_model
        pe = torch.zeros(max_seq_length, width)
        # 将位置编码信息填充到pe中
        for pos in range(max_seq_length):
            for i in range(width):
                if i % 2 == 0:
                    pe[pos][i] = np.sin(pos/(10000 ** (i/width)))
                else:
                    pe[pos][i] = np.cos(pos/(10000 ** ((i-1)/width)))
        # 位置编码信息进行冻结，不参与反向传播
        self.register_buffer('pe', pe.unsqueeze(0))

    def forward(self, x):
        # 直接将位置编码和token的嵌入进行相加
        x = x + self.pe
        return x
```

14.2.2 注意力头

```python
class AttentionHead(nn.Module):
    def __init__(self, width, head_size):
        super().__init__()
        self.head_size = head_size

        self.query = nn.Linear(width, head_size)
        self.key = nn.Linear(width, head_size)
        self.value = nn.Linear(width, head_size)

    def forward(self, x, mask=None):
        Q = self.query(x)
        K = self.key(x)
        V = self.value(x)

        attention = Q @ K.transpose(-2,-1)
        attention = attention / (self.head_size ** 0.5)
        # 掩码
        if mask is not None:
            attention = attention.masked_fill(mask == 0, float("-inf"))
        attention = torch.softmax(attention, dim=-1)
```

```
        attention = attention @ V
        return attention
```

Transformer 编码器和解码器之间的主要区别在于解码器使用注意力掩码，而编码器则不使用。虽然 CLIP 是仅编码器模型（Encoder-Only），但由于在分词时，对输入文本添加了 pad（填充符），所以仍然需要与文本编码器一起使用掩码。请注意，掩码是可选的，因此这个注意力头可用于文本和视觉编码器。

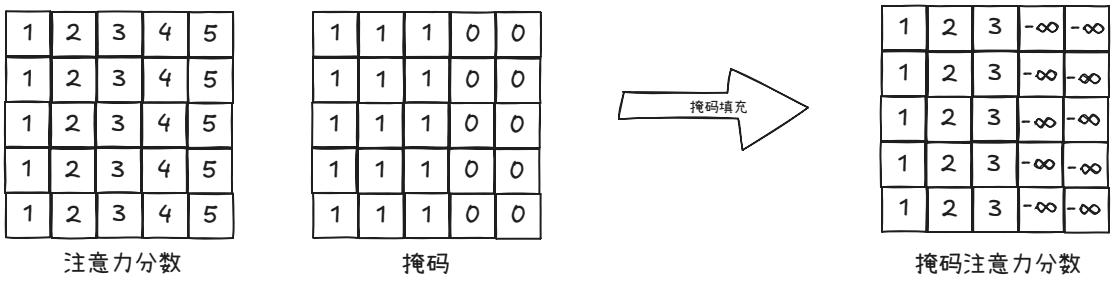


图 14.2 注意力分数的掩码

```python
# 使用注意力掩码
if mask is not None:
    attention = attention.masked_fill(mask == 0, float("-inf"))
```

14.2.3 多头注意力

```python
class MultiHeadAttention(nn.Module):
    def __init__(self, width, n_heads):
        super().__init__()
        self.head_size = width // n_heads
        self.W_o = nn.Linear(width, width)
        self.heads = nn.ModuleList([
            AttentionHead(width, self.head_size)
            for _ in range(n_heads)
        ])

    def forward(self, x, mask=None):
        # 拼接多个注意力头
        out = torch.cat([head(x, mask=mask) for head in self.heads], dim=-1)
        out = self.W_o(out)
        return out
```

14.2.4 Transformer 编码器

```python
class TransformerEncoder(nn.Module):
    def __init__(self, width, n_heads, r_mlp=4):
        super().__init__()
        self.width = width # 嵌入的维度d_model
        self.n_heads = n_heads

        # 层归一化
```

```
        self.ln1 = nn.LayerNorm(width)

        # 多头注意力
        self.mha = MultiHeadAttention(width, n_heads)

        # 层归一化
        self.ln2 = nn.LayerNorm(width)

        # MLP
        self.mlp = nn.Sequential(
            nn.Linear(self.width, self.width*r_mlp),
            nn.GELU(),
            nn.Linear(self.width*r_mlp, self.width)
        )

    def forward(self, x, mask=None):
        x = x + self.mha(self.ln1(x), mask=mask)
        x = x + self.mlp(self.ln2(x))
        return x
```

14.2.5 文本的分词器

我们的词汇表是 ascii 码，所以 `vocab_size = 256` 。

```python
def tokenizer(text, encode=True, mask=None, max_seq_length=32):
    if encode:
        out = chr(2) + text + chr(3) # 添加 SOT token 和 EOT token
        out = out + "".join([
            chr(0) for _ in range(max_seq_length-len(out))
        ]) # 添加Padding
        out = torch.IntTensor(list(out.encode("utf-8"))) # 对文本进行编码
        mask = torch.ones(len(out.nonzero()))
        mask = torch.cat((
            mask,
            torch.zeros(max_seq_length - len(mask))
        )).type(torch.IntTensor)
    else:
        # 将input_ids解码为text文本
        out = [chr(x) for x in text[1:len(mask.nonzero())-1]]
        out = "".join(out)
        mask = None

    return out, mask
```

Transformer 无法处理原始文本，因此我们需要做的第一件事是在将输入字符串通过文本编码器之前对其进行分词。

在本教程中，我们将进行一个简单版本的分词，其中我们只使用 UTF-8 编码。为什么使用 UTF-8 编码进行分词？因为我们只会在示例中使用简单的 ascii 码文本。对于更复杂的示例，可能需要使用 BPE 分词器。这是因为使用 UTF-8 编码时，词表大小（vocab_size）为 256，这意味着对于更复杂的示例，可能会有更长的输入序列，由于上下文长度有限，这在计算注意力时效率低下。

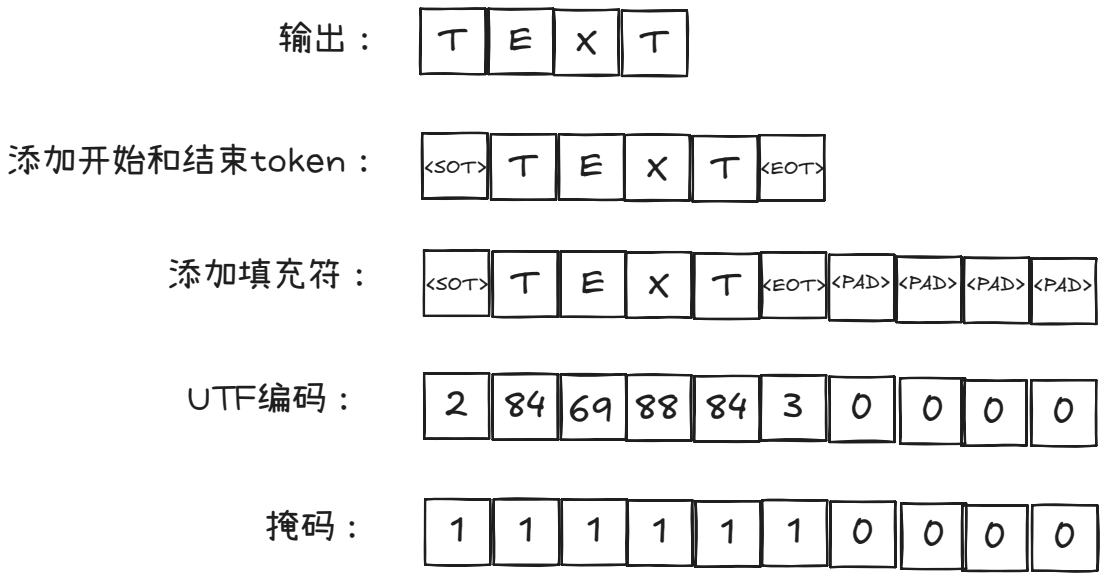


图 14.3 分词过程

分词器的第一步是将文本的开头和文本的结尾 token 添加到输入字符串中。

```python
text = chr(2) + text + chr(3)
```

添加文本的开头和文本的结尾 token 后，我们需要将序列的长度填充到最大序列长度。

```python
text = text + "".join([chr(0) for _ in range(10-len(text))])
```

我们通过将文本序列编码为 UTF-8 并将输出转换为 IntTensor 来完成分词。

```python
text = torch.IntTensor(list(text.encode("utf-8")))
```

对文本进行分词后，我们需要为文本创建掩码。虽然 Transformer 中通常使用的掩码用于确保 token 不与未来的 token 通信，但我们在这里应用的掩码只是使 pad token 被忽略。因此，掩码将只是一个大小等于最大序列长度的张量，其中元素在有填充的情况下为 0，否则为 1。

```python
mask = torch.ones(len(text.nonzero()))
mask = torch.cat((mask,torch.zeros(10-len(mask)))).type(torch.IntTensor)
```

14.2.6 文本编码器

```python
class TextEncoder(nn.Module):
    def __init__(
        self,
        vocab_size, # 词汇表大小=256
        width, # 宽度d_model
        max_seq_length, # 文本最大长度
        n_heads,
        n_layers,
        emb_dim # 嵌入维度
    ):
        super().__init__()
        self.max_seq_length = max_seq_length
```

```
13          self.encoder_embedding = nn.Embedding(vocab_size, width)
14          self.positional_embedding = PositionalEmbedding(
15              width,
16              max_seq_length
17          )
18          self.encoder = nn.ModuleList([
19              TransformerEncoder(width, n_heads)
20              for _ in range(n_layers)
21          ])
22          # 可学习投影（projection）
23          # W_{width × emb_dim}
24          self.projection = nn.Parameter(torch.randn(width, emb_dim))
25
26      def forward(self, text, mask=None):
27          # 文本嵌入
28          x = self.encoder_embedding(text)
29          # 位置嵌入
30          x = self.positional_embedding(x)
31          # Transformer编码器
32          for encoder_layer in self.encoder:
33              x = encoder_layer(x, mask=mask)
34          # 从EOT的嵌入抽取特征
35          x = x[
36              torch.arange(text.shape[0]), # 批次中数据的索引
37              # 取出掩码mask矩阵的第0行，加和再减1，就得到了EOT的索引
38              torch.sub(torch.sum(mask[:,0],dim=1),1)
39          ]
40          if self.projection is not None:                    } (1) 将文本嵌入映射到CLIP嵌入空间
41              x = x @ self.projection
42          # 向量x转换为模长为1的向量
43          x = x / torch.norm(x, dim=-1, keepdim=True)
44          return x
```

对于文本编码器，我们将使用常规 Transformer 模型。创建文本编码器的第一步是创建大小为 (vocab_size, width) 的嵌入表。此嵌入表包含一个向量表示，其大小等于词汇表中每个 token 的 Transformer 模型的 width 。

```
1  self.encoder_embedding = nn.Embedding(vocab_size, width)
```
Python

在输出 Transformer 的结果之前，我们需要将特征嵌入到联合嵌入空间中。我们将通过获取文本特征的点积以及使用 nn.Parameter 创建的可学习的投影来实现这一点。

```
1  self.projection = nn.Parameter(torch.randn(width, emb_dim))
```
Python

在 forward 方法中，我们要做的第一件事是通过嵌入表传递文本的 token。

```
1  x = self.encoder_embedding(text)
```
Python

然后，我们需要将位置编码添加到嵌入表的输出中。

```
1  x = self.positional_embedding(x)
```
Python

添加位置编码后，我们现在可以将其与掩码一起通过编码器层。

```python
for encoder_layer in self.encoder:
    x = encoder_layer(x, mask=mask)
```

编码器层的输出是文本的特征。我们将使用从 `EOT` 的嵌入中抽取的特征。

```python
# 从EOT的嵌入抽取特征
x = x[torch.arange(
  text.shape[0]),
  torch.sub(torch.sum(mask[:,0],dim=1),1)
]
```

最后，我们通过计算特征和投影之间的点积，将文本嵌入映射到 CLIP 嵌入空间中，并通过除以归一化的点积对其进行归一化。

> 为什么要做映射？
>
> 主要是为了在 CLIP 嵌入空间中，文本嵌入向量的维度和图像嵌入向量的维度一致。 向量除以向量的模长，就是模长为 1 的向量。这样文本嵌入向量和图像嵌入向量都变成了模长为 1 的向量。两个向量的点积，就是两个向量的余弦相似度！

```python
if self.projection is not None:
    x = x @ self.projection
x = x / torch.norm(x, dim=-1, keepdim=True)
return x
```

14.2.7 图像编码器

```python
class ImageEncoder(nn.Module):
    def __init__(
        self,
        width, # 补丁嵌入的维度d_model
        img_size,
        patch_size,
        n_channels,
        n_layers,
        n_heads,
        emb_dim
    ):
        super().__init__()

        assert img_size[0] % patch_size[0] == 0  \
           and img_size[1] % patch_size[1] == 0, \
           "img_size必须能被patch_size整除"
        assert width % n_heads == 0, \
           "width必须能被n_heads整除"

        self.n_patches = (img_size[0] * img_size[1]) \
                    // (patch_size[0] * patch_size[1])
        self.max_seq_length = self.n_patches + 1
        self.linear_project = nn.Conv2d(
            n_channels,
```

```
            width,
            kernel_size=patch_size,
            stride=patch_size
        )
        self.cls_token = nn.Parameter(torch.randn(1, 1, width))
        self.positional_embedding = PositionalEmbedding(
            width,
            self.max_seq_length
        )
        self.encoder = nn.ModuleList([
            TransformerEncoder(width,n_heads)
            for _ in range(n_layers)
        ])

        # W_{width × emb_dim}
        self.projection = nn.Parameter(torch.randn(width, emb_dim))

    def forward(self,x):
        # 补丁嵌入
        x = self.linear_project(x)
        x = x.flatten(2).transpose(1, 2)

        # 位置嵌入
        x = torch.cat((self.cls_token.expand(x.size()[0], -1, -1),x), dim=1)
        x = self.positional_embedding(x)

        # Transformer编码器
        for encoder_layer in self.encoder:
            x = encoder_layer(x)

        # 获取类别token
        x = x[:, 0, :]

        if self.projection is not None:                    } (1) 将图像嵌入映射到CLIP嵌入空间
            x = x @ self.projection
        # 向量x转换为模长为1的向量
        x = x / torch.norm(x, dim=-1, keepdim=True)
        return x
```

14.3 CLIP 模型

Python

```
class CLIP(nn.Module):
    def __init__(
        self,
        emb_dim, # 经过可学习投影后的嵌入维度
        vit_width, # 图像编码器的宽度(d_model)
        img_size,
        patch_size,
        n_channels,
```

```
        vit_layers,
        vit_heads,
        vocab_size,
        text_width, # 文本编码器的宽度（d_model）
        max_seq_length,
        text_heads,
        text_layers
    ):
        super().__init__()
        self.image_encoder = ImageEncoder(
            vit_width,
            img_size,
            patch_size,
            n_channels,
            vit_layers,
            vit_heads,
            emb_dim
        )
        self.text_encoder = TextEncoder(
            vocab_size,
            text_width,
            max_seq_length,
            text_heads,
            text_layers,
            emb_dim
        )
        # 可学习温度
        self.temperature = nn.Parameter(torch.ones([]) * np.log(1 / 0.07))
        self.device = torch.device("cuda")


    def forward(self, image, text, mask=None):
        # I_e是图像嵌入，形状 [B, D=emb_dim]
        I_e = self.image_encoder(image)
        # T_e是文本嵌入，形状 [B, D=emb_dim]
        T_e = self.text_encoder(text, mask=mask)

        # 缩放逐点余弦相似度[n, n]
        # 形状 I_e · T_e : [B, D] @ [D, B] ⟶ [B, B]
        logits = (I_e @ T_e.transpose(-2,-1)) * torch.exp(self.temperature)

        # 对称损失函数：labels形状为[B]，值为 [0, 1, 2, ..., B-1]
        labels = torch.arange(logits.shape[0]).to(self.device)
        # 从文本 ⟶ 图像方向，以文本嵌入 T_3 为例子，
        # 交叉熵损失的目标是让 T_3 · I_3 越大越好
        loss_i = nn.functional.cross_entropy(
            logits.transpose(-2,-1),
            labels
        )
```

```
        # 从图像 ⟶ 文本方向，以图像嵌入 I_3 为例，
        # 交叉熵损失的目标是让 I_3 · T_3 越大越好
        loss_t = nn.functional.cross_entropy(
            logits,
            labels
        )
        # 两个方向的损失求平均值
        loss = (loss_i + loss_t) / 2

        return loss
```

当给定一批图像和标题时，CLIP 应该告诉我们哪些标题与哪些图像搭配。它通过一起训练文本编码器和图像编码器来最大化应该在一起的对的成对余弦相似度分数，并最小化不应该在一起的对来做到这一点。

为此，我们首先需要从图像和文本编码器中获取特征的嵌入。

```python
def forward(self,image,text, mask=None):
    I_e = self.image_encoder(image)
    T_e = self.text_encoder(text, mask=mask)
```

使用嵌入的特征，我们可以通过使用嵌入的图像特征和嵌入文本特征的转置版本之间的点积来计算缩放的成对余弦相似度。余弦相似度应沿图中正确的图像和文本配对在一起的对角线最大化。

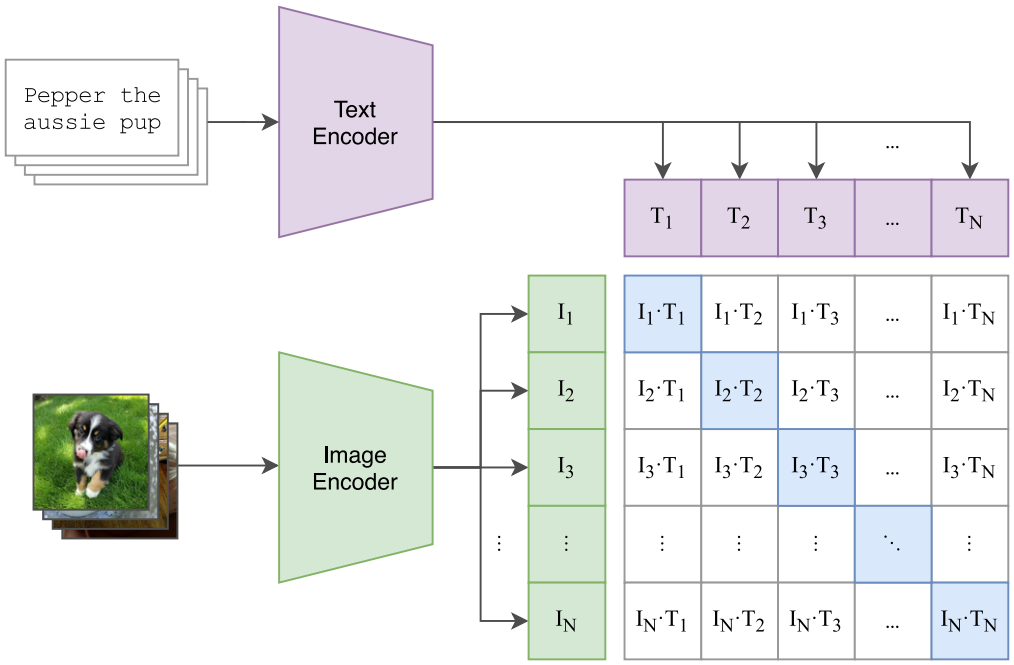


图 14.4 余弦相似度的计算

```python
logits = (I_e @ T_e.transpose(-2,-1)) * torch.exp(self.temperature)
```

这在 I_e 和 T_e 分别包含 N 个批次时工作，从而产生上图所示的矩阵。为了最大化相关图像之间的余弦相似度，CLIP 使用对称/对比损失。我们可以通过首先创建与批次中的每条数据相对应的标签来计算此损失。

```python
# 对称损失函数
labels = torch.arange(logits.shape[0]).to(self.device)
```

然后，我们计算沿 `logit` 行的交叉熵损失，以获得图像的损失。

```python
loss_i = nn.functional.cross_entropy(logits.transpose(-2,-1), labels)
```

文本的损失是通过计算沿列的交叉熵损失来计算的。

```python
loss_t = nn.functional.cross_entropy(logits, labels)
```

我们通过计算图像损失和文本损失之间的平均值来获得最终损失。

```python
loss = (loss_i + loss_t) / 2
return loss
```

14.3.1 数据

```python
class MNIST(Dataset):
    def __init__(self, train=True):
        self.dataset = load_dataset("./../datasets/clip-mnist/")
        self.transform = T.ToTensor()
        if train:
            self.split = "train"
        else:
            self.split = "test"

        self.captions = {
            0: "An image of Zero",
            1: "An image of One",
            2: "An image of Two",
            3: "An image of Three",
            4: "An image of Four",
            5: "An image of Five",
            6: "An image of Six",
            7: "An image of Seven",
            8: "An image of Eight",
            9: "An image of Nine"
        }

    def __len__(self):
        return self.dataset.num_rows[self.split]

    def __getitem__(self, i):
        # 取出第i张图片
        img = self.dataset[self.split][i]["image"]
        # 转换成张量
        img = self.transform(img)
        # 图片对应的文本，以及掩码
        cap, mask = tokenizer(
           self.captions[self.dataset[self.split][i]["label"]]
        )
        # 为什么要repeat?
        mask = mask.repeat(len(mask), 1)
        return {"image": img, "caption": cap, "mask": mask}
```

在本教程中，我们将使用 MNIST 数据集。我们选择这个数据集是因为它相当小并且保持训练时间合理。

```python
self.dataset = load_dataset("clip-mnist")
```

对于数据集中的每个条目，我们将需要 3 样东西：图像、文本和文本掩码。

对于图像，我们唯一需要进行的更改是将图像转换为张量。

```python
img = self.dataset[self.split][i]["image"]
img = self.transform(img)
```

对于标题，我们需要将其传递给我们创建的分词器，以获取 token 表示以及 token 的掩码。

```python
cap, mask = tokenizer(self.captions[self.dataset[self.split][i]["label"]])
```

我们从分词器获得的掩码是大小为 `max_seq_length` 的 1 维张量。在文本编码器中，掩码将应用于形状为 `(max_seq_length, max_seq_length)` 的注意力分数。因此，我们需要扩展掩码，以便将其应用于注意力分数的每一行。

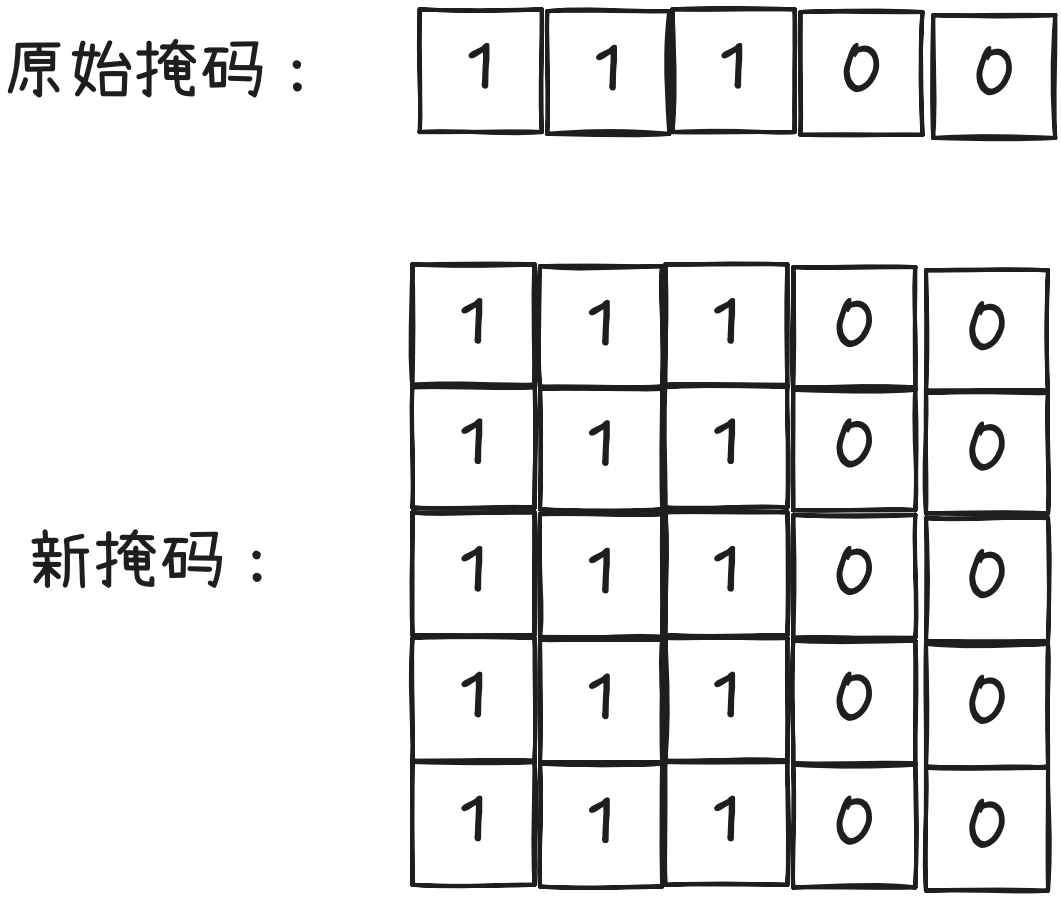


图 14.5 复制掩码

```python
mask = mask.repeat(len(mask), 1)
```

图像、标题和掩码作为字典保存在数据集中。

```python
return {"image": img, "caption": cap, "mask": mask}
```

不要和因果注意力混淆！

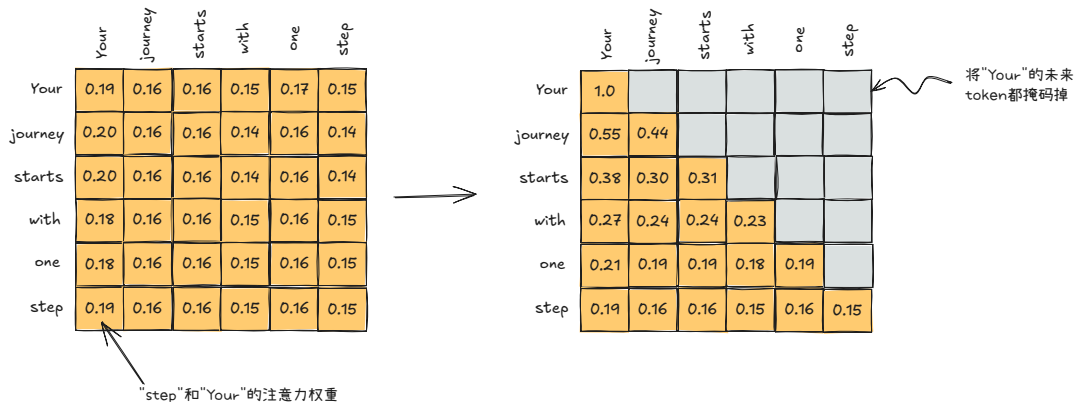


图 14.6 因果注意力分数的掩码

创建 [context_length , context_length] 的上三角矩阵。

```python
self.register_buffer(
    'mask',
    torch.triu(torch.ones(
        context_length,
        context_length
    ), diagonal = 1))
```

将注意力分数的上三角置为-torch.inf，下三角保留。得到注意力分数的下三角矩阵。

```python
attention_score.masked_fill_(
    self.mask[:seq_len, :seq_len].bool(),
    -torch.inf
)
```

14.3.2 训练参数

```python
emb_dim = 32 # 文本编码器和图像编码器输出的张量的维度
vit_width = 9 # 图像编码器的嵌入的宽度
img_size = (28,28)
patch_size = (14,14)
n_channels = 1
vit_layers = 3 # 图像编码器中编码器的层的数量
vit_heads = 3 # 图像编码器中注意力头的数量
vocab_size = 256 # 词汇表大小
text_width = 32 # 文本编码器的嵌入的宽度
max_seq_length = 32 # 最大序列长度
text_heads = 8 # 文本编码器中注意力头的数量
text_layers = 4 # 文本编码器中编码器的层数
lr = 1e-3 # 学习率
epochs = 10
batch_size = 128
```

14.3.3 加载数据集

```python
train_set = MNIST(train = True)
test_set = MNIST(train = False)

train_loader = DataLoader(train_set, shuffle=True, batch_size=batch_size)
test_loader = DataLoader(test_set, shuffle=False, batch_size=batch_size)
```

14.3.4 训练模型

```python
device = torch.device("cuda")

model = CLIP(
    emb_dim,
    vit_width,
    img_size,
    patch_size,
    n_channels,
    vit_layers,
    vit_heads,
    vocab_size,
    text_width,
    max_seq_length,
    text_heads,
    text_layers
).to(device)

optimizer = optim.Adam(model.parameters(), lr=lr)

best_loss = np.inf
for epoch in range(epochs):
    for i, data in enumerate(train_loader, 0):
        img = data["image"].to(device)
        cap = data["caption"].to(device)
        mask = data["mask"].to(device)
        loss = model(img, cap, mask)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    print(f"Epoch [{epoch+1}/{epochs}], Batch Loss: {loss.item():.3f}")

    # 保存模型
    if loss.item() ⩽ best_loss:
        best_loss = loss.item()
        torch.save(model.state_dict(), "./clip.pt")
        print("模型已经保存.")
```

14.3.5 评估模型

Python

```python
# 加载最好的模型
model = CLIP(
    emb_dim,
    vit_width,
    img_size,
    patch_size,
    n_channels,
    vit_layers,
    vit_heads,
    vocab_size,
    text_width,
    max_seq_length,
    text_heads,
    text_layers
).to(device)
model.load_state_dict(torch.load("./clip.pt", map_location=device))

# 获取数据集的标签和图片进行对比
text = torch.stack(
    [tokenizer(x)[0] for x in test_set.captions.values()]
).to(device)
mask = torch.stack(
    [tokenizer(x)[1] for x in test_set.captions.values()]
)
mask = mask.repeat(
    1,
    len(mask[0])
).reshape(
    len(mask),
    len(mask[0]
),len(mask[0])).to(device)

correct, total = 0, 0
with torch.no_grad():
    for data in test_loader:
        # 图像
        images = data["image"].to(device)
        # 文本
        labels = data["caption"].to(device)
        # 使用clip模型中的图像编码器对图像抽取特征
        image_features = model.image_encoder(images)
        # 使用clip模型中的文本编码器对文本抽取特征
        text_features = model.text_encoder(text, mask=mask)
        # 转换为模长为1的向量
        image_features /= image_features.norm(dim=-1, keepdim=True)
        text_features /= text_features.norm(dim=-1, keepdim=True)
        # I_e @ T_e^T: $I_e \cdot T_e$
        similarity = (
```

```
            100.0 * image_features @ text_features.T
        ).softmax(dim=-1)
        _, indices = torch.max(similarity, 1)
        # 预测结果
        pred = torch.stack([
            tokenizer(test_set.captions[int(i)])[0]
            for i in indices
        ]).to(device)
        # 预测正确的样本数量
        correct += int(sum(torch.sum((pred==labels),dim=1)//len(pred[0])))
        total += len(labels)

print(f'\n预测准确率: {100 * correct // total} %')
```

我们通过获取模型训练的标题并将其与实际标题进行比较来测试模型。在训练时，我们使用了相同的标题模板（`"A image of a（n） {class}"`），因此这个测试阶段与任何其他图像分类器几乎相同。我们实现了大约 85% 的模型准确率。

14.3.6 零样本分类

Python

```
# 加载模型
model = CLIP(
    emb_dim,
    vit_width,
    img_size,
    patch_size,
    n_channels,
    vit_layers,
    vit_heads,
    vocab_size,
    text_width,
    max_seq_length,
    text_heads,
    text_layers
).to(device)
model.load_state_dict(torch.load("./clip.pt", map_location=device))

# 标题
class_names = [
    "a photo of 0",
    "an image of one",
    "2",
    "3",
    "4",
    "5",
    "6",
    "7",
    "8",
    "9",
    "Trump",
```

```
    "Musk"
]

text = torch.stack(
    [tokenizer(x)[0] for x in class_names]
).to(device)
mask = torch.stack(
    [tokenizer(x)[1] for x in class_names]
)
mask = mask.repeat(
    1,
    len(mask[0])
).reshape(len(mask),len(mask[0]),len(mask[0])).to(device)

idx = 1000
# 去测试数据集中的第1000张图片
img = test_set[idx]["image"][None,:]
plt.imshow(img[0].permute(1, 2, 0), cmap="gray")
# 将图片的标题文本展示，例如"An Image Of Nine"
plt.title(tokenizer(
    test_set[idx]["caption"],
    encode=False,
    mask=test_set[idx]["mask"][0]
)[0])
plt.show()
img = img.to(device)
with torch.no_grad():
    # 抽取图片的特征
    image_features = model.image_encoder(img)
    # 抽取文本的特征
    text_features = model.text_encoder(text, mask=mask)

image_features /= image_features.norm(dim=-1, keepdim=True)
text_features /= text_features.norm(dim=-1, keepdim=True)
# 计算第1000张图片和所有文本的相似度
similarity = (
    100.0 * image_features @ text_features.T
).softmax(dim=-1)
# 返回所有文本中和图片特征最相似的5个文本
values, indices = similarity[0].topk(5)

# 打印结果
print("\n预测结果:\n")
for value, index in zip(values, indices):
    print(f"{class_names[int(index)]:>16s}: {100 * value.item():.2f}%")
```

对于 zero-shot 分类，我们将图像与类别的名称进行比较。我们输入标签以与图像进行比较，它将返回前 5 个预测以及预测的可能性。这不是 CLIP 执行 zero-shot 分类的最佳示例。使用 MNIST 数据集使模型易于训练，但标题不是很丰富。要真正理解 CLIP 的 zero-shot 功能，包含多个名称的训练集会更合适。真正的 zero-shot 检测将允许检测以前未见过的排列。

🔥 应用举例：文搜图

具体步骤如下：
1. 将数据库中的所有图片使用 `clip` 的图像编码器进行抽取特征
2. 将抽取的图片特征存入向量数据库
3. 将输入的搜索文本通过 `clip` 的文本编码器抽取文本特征
4. 使用余弦相似度找出向量数据库中和文本特征最接近的几张图片

15. ClipCap（图生文）

> **Image Captioning Model**
>
> 图片标题模型(image captioning model)是一种将图片作为输入并生成图片描述的模型。

下面是一个简单的示例，展示了一个图片标题模型：

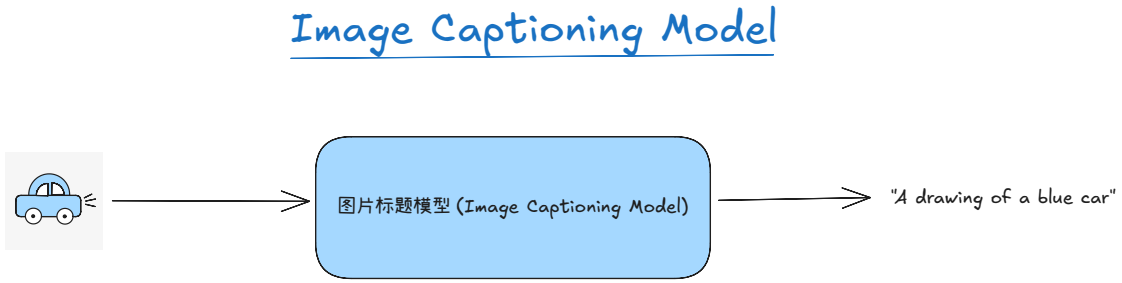


图 15.1 图片标题模型的简单示例

15.1 ClipCap 的工作原理

ClipCap 是一种结合了 CLIP 和 GPT-2 的图片标题架构。

CLIP 是我们将用来创建输入图像嵌入的模型。

GPT-2 是一种基于解码器的模型，用于生成文本。

ClipCap 的基本工作原理如下：

输入图像首先通过 CLIP 模型转换为嵌入，目的是利用这种嵌入(捕捉图像意义)来引导 GPT-2 生成文本。

但有一个问题：CLIP 和 GPT-2 的嵌入空间不同。所以我们不能直接把这个嵌入输入到 GPT-2 中。

为了解决这个问题，我们使用一个映射网络将 CLIP 嵌入映射到 GPT-2 的嵌入空间。

这些映射的图像嵌入称为前缀 (prefixes)，因为它们是 GPT-2 生成图片说明所需的上下文。

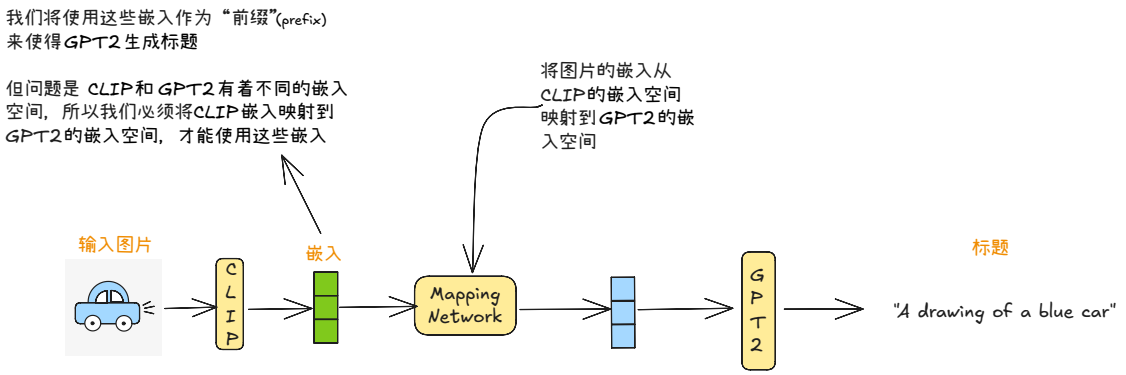


图 15.2 将图片的 CLIP 嵌入映射到 GPT2 的嵌入空间

15.2 关于训练

CLIP 生成的图像嵌入开箱即用已经足够好——所以我们不训练 CLIP 模型。

根据 GPT-2 是否经过微调，ClipCap 有两种变体 ：

- 如果我们对 GPT-2 进行微调 ，那么我们就用 MLP 作为映射网络。GPT-2 和 MLP 都经过训练。
- 如果我们不对 GPT-2 进行微调，那么我们就用 Transformer 架构（例如 Bert）作为映射网络，只有 Transformer 本身被训练。

就我而言，我选择对 GPT-2 模型进行微调 ，并使用 MLP 作为映射网络。

15.3 推理

在我们这里，这意味着为没见过的图片生成说明文字。

让我们用一辆蓝色汽车的图片作为推理过程的例子：

1. 输入图像被转换为 CLIP 图像嵌入。
2. CLIP 图像嵌入通过映射网络传递，生成前缀嵌入 。
3. 在时间步 t=1 时，我们将这些前缀嵌入传递到 GPT2。模型预测下一个 token——假设是“a”。我们将此附加到序列中。
4. 在 t=2 时，我们将更新后的输入序列（前缀+“a”）传递给 GPT2。它预测下一个 token——这次可能是“蓝色”。
5. 这一过程持续进行，模型一次预测一个 token，直到它：
   - 生成 <EOS>（序列结束）token，或
   - 生成的标题长度达到最大。

如下图所示

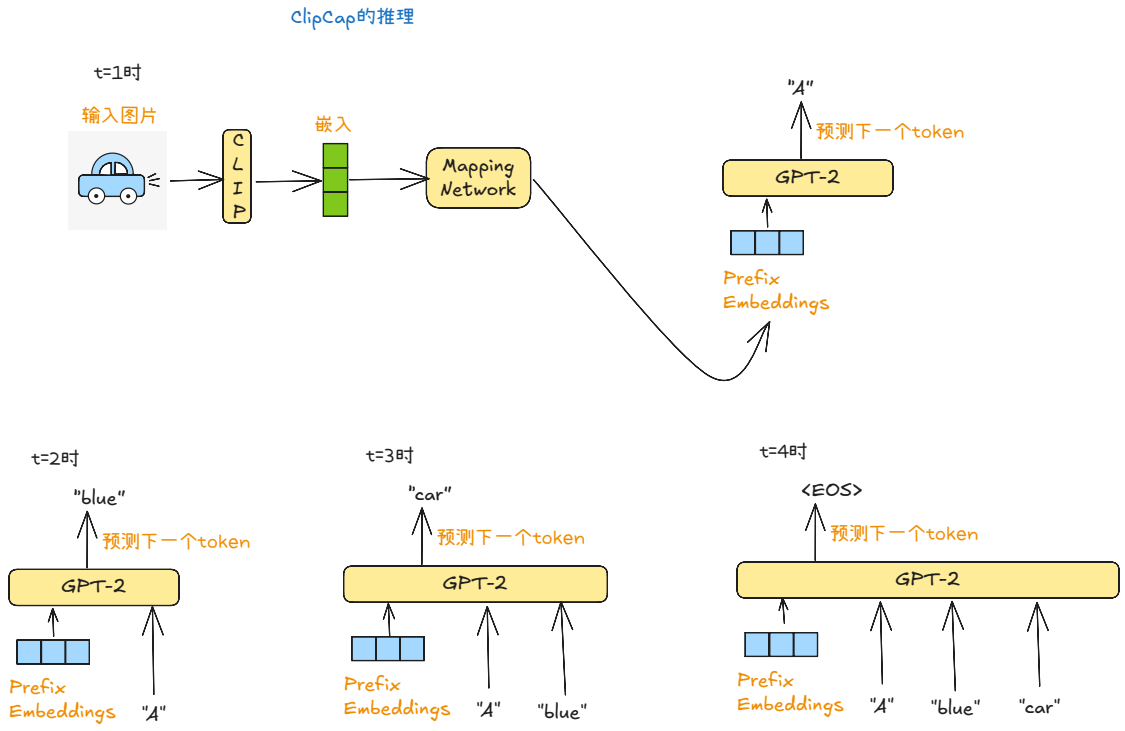


图 15.3 ClipCap 的推理过程

15.4 ClipCap 的代码实现

15.4.1 项目配置

config.py Python

```python
import torch

CLIP_MODEL_PATH = "./chinese-clip-vit-base-patch16"
# 一张图片的嵌入经过投影转换成10个token的embedding，每个embedding的dim是768
IMAGE_TOKEN_LENGTH = 10 # 图片的token的数量
MAX_LENGTH = 100 # 最大token数量
# clip对接的大语言模型
LLM_PATH = "./gpt2-chinese-cluecorpussmall"
LLM_WORD_EMBD_DIM = 768 # gpt2的词嵌入维度
IMAGE_EMBD_DIM = 512 # clip输出的图像嵌入的维度
device = torch.device("cuda")
```

15.4.2 处理数据

process_data.py Python

```python
from PIL import Image
import pickle
from transformers import ChineseCLIPProcessor, ChineseCLIPModel
from config import CLIP_MODEL_PATH

```

```python
def main():
    # 加载clip模型
    # clip模型只用来生成图片的嵌入，不进行微调。
    clip_model = ChineseCLIPModel.from_pretrained(CLIP_MODEL_PATH)
    # 加载clip处理器
    processor = ChineseCLIPProcessor.from_pretrained(CLIP_MODEL_PATH)
    # 将2张图片进行处理，处理完之后交给clip抽取特征
    inputs_1 = processor(images=Image.open("1.jpg"), return_tensors="pt")
    inputs_2 = processor(images=Image.open("2.jpg"), return_tensors="pt")
    # 获取第一张图片的嵌入（dim: 512）
    image_1_features = clip_model.get_image_features(**inputs_1)
    image_2_features = clip_model.get_image_features(**inputs_2)
    # 除以模长，归一化
    image_1_features = image_1_features / \
        image_1_features.norm(p=2, dim=-1, keepdim=True)  # normalize
    image_2_features = image_2_features / \
        image_2_features.norm(p=2, dim=-1, keepdim=True)  # normalize
    # key：图片id
    # value：图片的嵌入
    # 下面的字典也可以放在chroma这样的向量数据库
    image_id2embed = {
        1: image_1_features,
        2: image_2_features,
    }
    # 图片的id和图片标题对
    caption_list = [
        (1, "两只狗在雪地里嬉闹"),
        (2, "一件好看的立领的很特别的紫色风衣"),
    ]

    with open("caption_image.pkl", 'wb') as f:
        pickle.dump([caption_list, image_id2embed], f)

    print(f'图像嵌入的数量:{len(image_id2embed)}')
    print(f'图像文本的数量:{len(caption_list)}')


if __name__ == '__main__':
    main()
```

15.4.3 模型结构

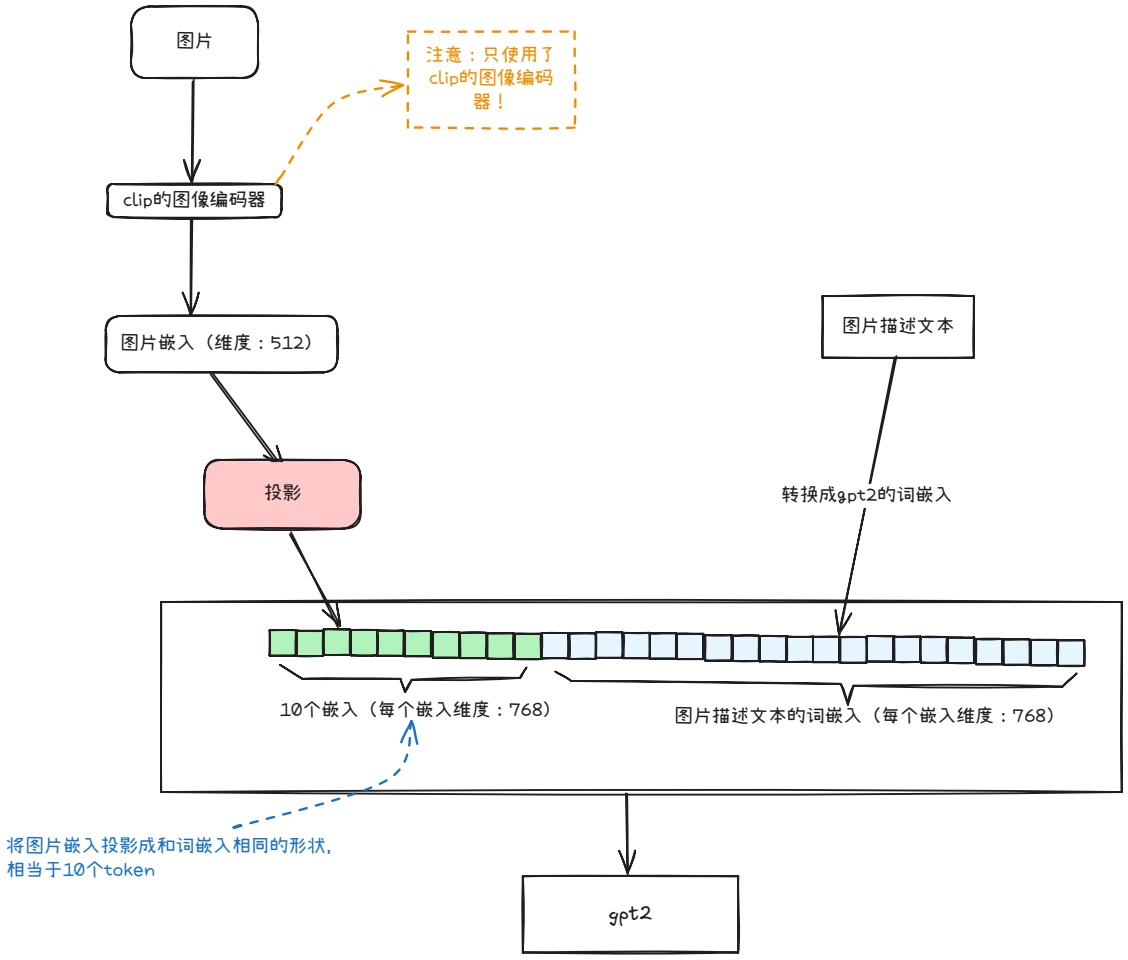


图 15.4 clipcap 的模型设计要点

model.py Python

```python
import torch
import torch.nn as nn
from transformers import GPT2LMHeadModel

from typing import Sequence
from config import LLM_PATH, IMAGE_TOKEN_LENGTH, IMAGE_EMBD_DIM


class MLP(nn.Module):
    """投影层"""
    def __init__(self, sizes: Sequence[int]):
        super().__init__()

        in_dim, h1, out_dim = sizes
        self.l1 = nn.Linear(in_dim, h1)
        self.act1 = nn.Tanh()
        self.l2 = nn.Linear(h1, out_dim)
```

```
18
19      def forward(self, x: torch.Tensor) -> torch.Tensor:
20          x = x.float()
21          x = self.l1(x)
22          x = self.act1(x)
23          x = self.l2(x)
24          return x
25
26
27  class ClipCaptionModel(nn.Module):
28      def __init__(self):
29          super(ClipCaptionModel, self).__init__()
30          # 大语言模型：用来生成图片的文本描述
31          self.gpt2 = GPT2LMHeadModel.from_pretrained(LLM_PATH)
32          # gpt2的词嵌入维度是768
33          self.word_embd_dim = self.gpt2.config.n_embd
34          # 投影层定义
35          self.projection = MLP((
36              # 输入维度是512,也就是clip的图像编码器输出的嵌入的维度
37              IMAGE_EMBD_DIM,
38              # (768 * 10) // 2
39              # 10是图片嵌入转换成的token数量
40              (self.word_embd_dim * IMAGE_TOKEN_LENGTH) // 2,
41              # 768 x 10，图片占10个token，每个token的嵌入和词嵌入相同是768维度
42              self.word_embd_dim * IMAGE_TOKEN_LENGTH
43          ))
44
45      def forward(self, image_embeds, caption_ids, mask):
46          # 张量形状：[B, 文本长度, gpt2的词嵌入的维度]
47          # 标题caption的每个token的词嵌入
48          caption_embeds = self.gpt2.transformer.wte(caption_ids)
49          # 将图片的嵌入转换为像词嵌入那样的维度：
50          # [B, 图片token的长度为10, gpt2的词嵌入的维度]
51          image_as_word_embeds = self.projection(
52              image_embeds
53          ).view(-1, IMAGE_TOKEN_LENGTH, self.word_embd_dim)
54          # 10个图片的token + 文本的token
55          # 张量形状：[B, 10+文本长度, 词嵌入维度]
56          embedding_cat = torch.cat((
57              image_as_word_embeds, # 图像的token
58              caption_embeds # 文本的token
59          ), dim=1)
60          out = self.gpt2(inputs_embeds=embedding_cat, attention_mask=mask)
61          # 张量形状：[B, 10+文本长度, 词嵌入维度]
62          logits = out.logits
63          return logits
```

15.4.4 准备训练数据集

clipcap_dataset.py Python

```python
import torch
from torch.utils.data import Dataset
import pickle
from typing import Tuple
from config import IMAGE_TOKEN_LENGTH, MAX_LENGTH


class ClipCapDataset(Dataset):
    def __init__(self, tokenizer):
        # 填充符
        pad_id = tokenizer.pad_token_id
        # 取出图片的文本和图片的嵌入
        with open("caption_image.pkl", 'rb') as f:
            caption_list, image_id2embed = pickle.load(f)
        print('图片嵌入的总数:{}'.format(len(image_id2embed)))
        print('图片描述的总数:{}'.format(len(caption_list)))

        image_embed_list = []
        caption_ids_list = []
        mask_list = []
        for image_id, caption in caption_list:
            # 使用图像id获取图像的特征（clip.image_encoder输出的）
            image_embed = image_id2embed[image_id]
            # 只对文本进行分词，不添加任何特殊token
            caption_ids = tokenizer.encode(
                caption,
                add_special_tokens=False
            )
            # 在文本的token列表后面添加一个分隔符token
            caption_ids.append(tokenizer.sep_token_id)

            # 截断
            # 只能留下前90个token，因为图像对应的token需要占用10个token的位置
            # 最终的数据是：图像的token列表 + 文本的token列表
            caption_ids = caption_ids[:MAX_LENGTH - IMAGE_TOKEN_LENGTH]
            # 图像部分和文本部分的token都要掩码
            mask = [1] * (IMAGE_TOKEN_LENGTH + len(caption_ids))

            # 填充pad
            padding_len = MAX_LENGTH          \
                        - IMAGE_TOKEN_LENGTH \
                        - len(caption_ids)
            caption_ids += [pad_id] * padding_len
            # 将填充符掩码为0
            mask += [0] * padding_len

            caption_ids = torch.tensor(caption_ids).long()
```

```
48              mask = torch.tensor(mask).long()
49
50              image_embed_list.append(image_embed)
51              caption_ids_list.append(caption_ids)
52              mask_list.append(mask)
53          # 保存训练数据
54          with open("train_data.pkl", 'wb') as f:
55              pickle.dump([
56                  image_embed_list, # clip输出的图片特征的列表
57                  caption_ids_list, # 图片文本的input_ids的列表
58                  mask_list # 掩码的列表
59              ], f)
60          self.image_embed_list = image_embed_list
61          self.caption_ids_list = caption_ids_list
62          self.mask_list = mask_list
63          print(f'训练数据总数：{len(self.image_embed_list)}')
64
65      def __len__(self) -> int:
66          return len(self.caption_ids_list)
67
68      def __getitem__(self, index: int) -> Tuple[torch.Tensor, ...]:
69          image_embed = self.image_embed_list[index]
70          caption_ids = self.caption_ids_list[index]
71          mask = self.mask_list[index]
72          return image_embed, caption_ids, mask
```

15.4.5 训练

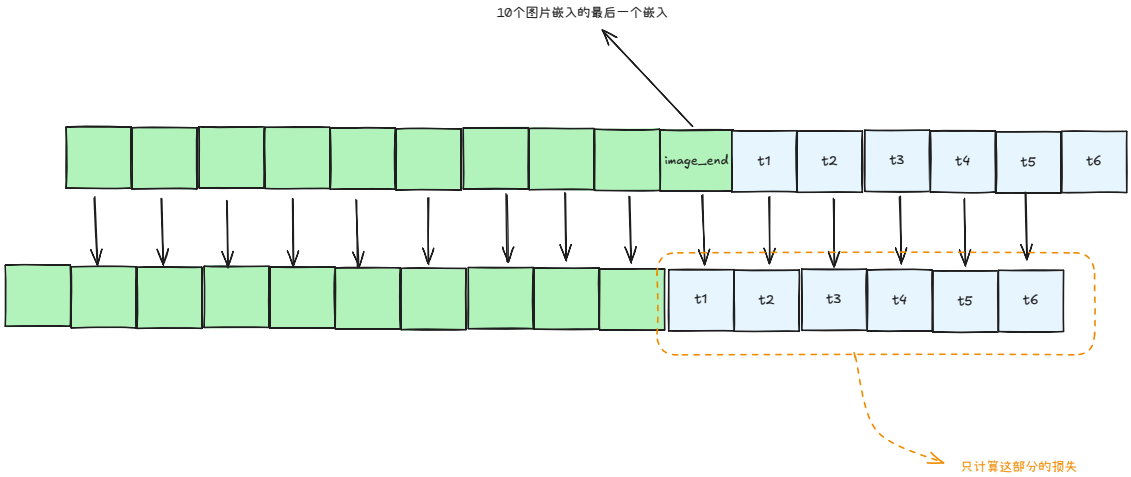


图 15.5 clipcap 的训练目标

train.py Python

```
1  import torch
2  from torch.utils.data import DataLoader
3  from transformers import BertTokenizer
4  from tqdm import tqdm
```

```
from clipcap_dataset import ClipCapDataset
from model import ClipCaptionModel
import torch.nn.functional as F
from config import LLM_PATH, IMAGE_TOKEN_LENGTH, device


def train(model, train_loader, optimizer):
    model.train()
    for _ in range(20):
        for _, data in enumerate(tqdm(train_loader)):
            image_embed, caption_ids, mask = data
            image_embed = image_embed.to(device)
            caption_ids = caption_ids.to(device)
            mask = mask.to(device)
            # 输出的logits
            logits = model(image_embed, caption_ids, mask)

            # 计算loss
            # [图片的最后一个token]，[两]，[只]，[狗]
            #          ↓            ↓    ↓
            #         [两]         [只]  [狗]
            shift_logits = logits[
                :,
                # 截取范围[图片的最后一个token～倒数第二个token]
                IMAGE_TOKEN_LENGTH - 1: -1, # 去掉最后一个token
                :
            ].contiguous().view(-1, logits.size(-1))
            # 预测目标
            shift_labels = caption_ids.view(-1)
            loss = F.cross_entropy(shift_logits, shift_labels)
            # logits.size(-1): 取 logits 的最后一维大小。一般最后一维是词表大小 vocab_size。
            # 原 logits 形状通常是 [B, L, V]（批大小、序列长度、词表大小）。
            # 经过切片后，shift_logits 的形状是 [B, L-1, V]。
            # 再 `.contiguous().view(-1, logits.size(-1))`
            # 就变成 [B*(L-1), V]，把前两维展平，便于和标签做交叉熵。
            # caption_ids 形状通常是 [B, L-1]（与 shift_logits 的时间步对齐）。
            # caption_ids.view(-1) 把它展平成 [B*(L-1)]，与 shift_logits 的第一维对齐，
            # 用于计算 CrossEntropyLoss(shift_logits, shift_labels)。

            loss.backward()
            optimizer.step()
            optimizer.zero_grad()

    torch.save(model.state_dict(), f'model.pt')


def main():
    # 分词器
```

```
    tokenizer = BertTokenizer.from_pretrained(LLM_PATH)
    # 加载模型
    model = ClipCaptionModel().to(device)

    dataset = ClipCapDataset(tokenizer)
    train_dataloader = DataLoader(dataset, batch_size=4, shuffle=True)
    optimizer = torch.optim.AdamW(model.parameters(), lr=2e-5)

    train(model, train_dataloader, optimizer)


if __name__ == '__main__':
    main()
```

15.4.6 推理

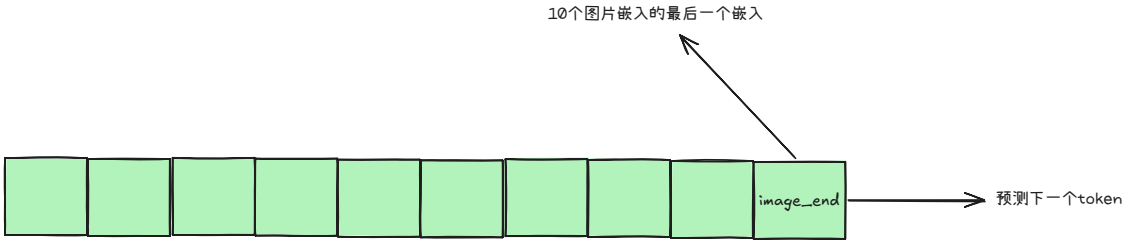


图 15.6 clipcap 只根据图片的嵌入预测文本的 token，也就是看图说话，不再需要文本了！

infer.py Python

```
from PIL import Image
import torch
from transformers import BertTokenizer, ChineseCLIPModel, ChineseCLIPProcessor
from model import ClipCaptionModel
import torch.nn.functional as F
from config import LLM_PATH, CLIP_MODEL_PATH, IMAGE_TOKEN_LENGTH,
LLM_WORD_EMBD_DIM, device, MAX_LENGTH


def generate(model, image_embeds, tokenizer):
    """
    :param image_embeds: [B, IMAGE_EMBD_DIM=512]
    """

    b_size = image_embeds.size(0)
    pad_id = tokenizer.pad_token_id
    sep_id = tokenizer.sep_token_id
    unk_id = tokenizer.unk_token_id
    temperature = 0.7

    cur_len = 0
    caption_ids = []    # 存储生成的caption

```

```
    # gpt2模型的输入: inputs_embeds:[B, 图片token的数量为10, gpt2的词嵌入维度768]
    # 先将图片特征投影为10个图片token
    inputs_embeds = model.projection(
        image_embeds
    ).view(-1, IMAGE_TOKEN_LENGTH, LLM_WORD_EMBD_DIM)
    finish_flag = [False] * b_size  # 第i个输入是否完成生成的标志

    while True:
        out = model.gpt2(inputs_embeds=inputs_embeds)
        logits = out.logits  # [B, len, vocab_size]
        # 采样下一个token
        next_token_logits = logits[:, -1, :]     # 取最后一个单词的预测分布
        next_token_logits = next_token_logits / temperature
        next_token_logits[:, unk_id] = -float('Inf')   # 将unk设为无穷小

        # 采样下一个token，多项分布
        next_token_ids = torch.multinomial(
            F.softmax(next_token_logits, dim=-1),
            num_samples=1
        ).squeeze(1).tolist()

        # 分别判断生成图片是否已生成完毕
        # index表示第index张正在生成文本的图片
        for index in range(len(next_token_ids)):
            token_id = next_token_ids[index]
            # 如果第i个句子已经生成结束
            if finish_flag[index]:
                next_token_ids[index] = pad_id
            # 如果第i个句子生成结束，预测到了分隔符
            elif token_id == sep_id:
                finish_flag[index] = True
            # 生成刚开始
            elif cur_len == 0:
                caption_ids.append([token_id])
            else:
                caption_ids[index].append(token_id)
        next_token_ids = torch.tensor(next_token_ids).to(device)
        next_token_embeds = model.gpt2.transformer.wte(
            next_token_ids).to(device).unsqueeze(1)
        # 将生成的next token拼接到上文的后面，继续生成
        inputs_embeds = torch.cat((inputs_embeds, next_token_embeds), dim=1)

        cur_len += 1 # 生成长度+1
        # 如果生成长度大于最大长度，或者所有图片的生成文本都结束了，退出生成过程。
        if cur_len > MAX_LENGTH or False not in finish_flag:
            break

    # 对token_id进行解码
    captions = []
```

```
    for caption_id in caption_ids:
        caption = tokenizer.convert_ids_to_tokens(caption_id)
        caption = ''.join(caption)
        captions.append(caption)

    return captions


def main():
    # 分词器
    tokenizer = BertTokenizer.from_pretrained(LLM_PATH)
    # 初始化模型
    model = ClipCaptionModel().to(device)
    # 加载权重
    model.load_state_dict(torch.load(
        "model.pt",
        map_location=device
    ), False)
    model.eval()

    # 加载clip模型
    clip_model = ChineseCLIPModel.from_pretrained(CLIP_MODEL_PATH)
    processor = ChineseCLIPProcessor.from_pretrained(CLIP_MODEL_PATH)
    inputs_1 = processor(images=Image.open("1.jpg"), return_tensors="pt")
    inputs_2 = processor(images=Image.open("2.jpg"), return_tensors="pt")
    image_1_features = clip_model.get_image_features(**inputs_1)
    image_2_features = clip_model.get_image_features(**inputs_2)
    image_1_features = image_1_features / \
        image_1_features.norm(p=2, dim=-1, keepdim=True)  # normalize
    image_2_features = image_2_features / \
        image_2_features.norm(p=2, dim=-1, keepdim=True)  # normalize
    # 将两张图片的特征打包成一个批次数据
    data = torch.stack([
        image_1_features,
        image_2_features
    ], dim=0).to(device)
    captions = generate(model, data, tokenizer)
    print(captions)
    captions = generate(model, data, tokenizer)
    print(captions)
    captions = generate(model, data, tokenizer)
    print(captions)
    captions = generate(model, data, tokenizer)
    print(captions)
    captions = generate(model, data, tokenizer)
    print(captions)


if __name__ == '__main__':
    main()
```

15.5 应用：图生文

当前电商平台上的商品描述主要依赖商家手工编写，质量参差不齐，存在描述不准确、信息不完整、语言不标准等问题。这不仅影响用户的购买决策，还增加了平台内容审核与优化的成本。如何利用智能化手段快速生成高质量的商品描述，成为亟待解决的核心问题。

图 15.7 A800 训练 10 个小时的效果

15.6 拓展：Qwen3-VL

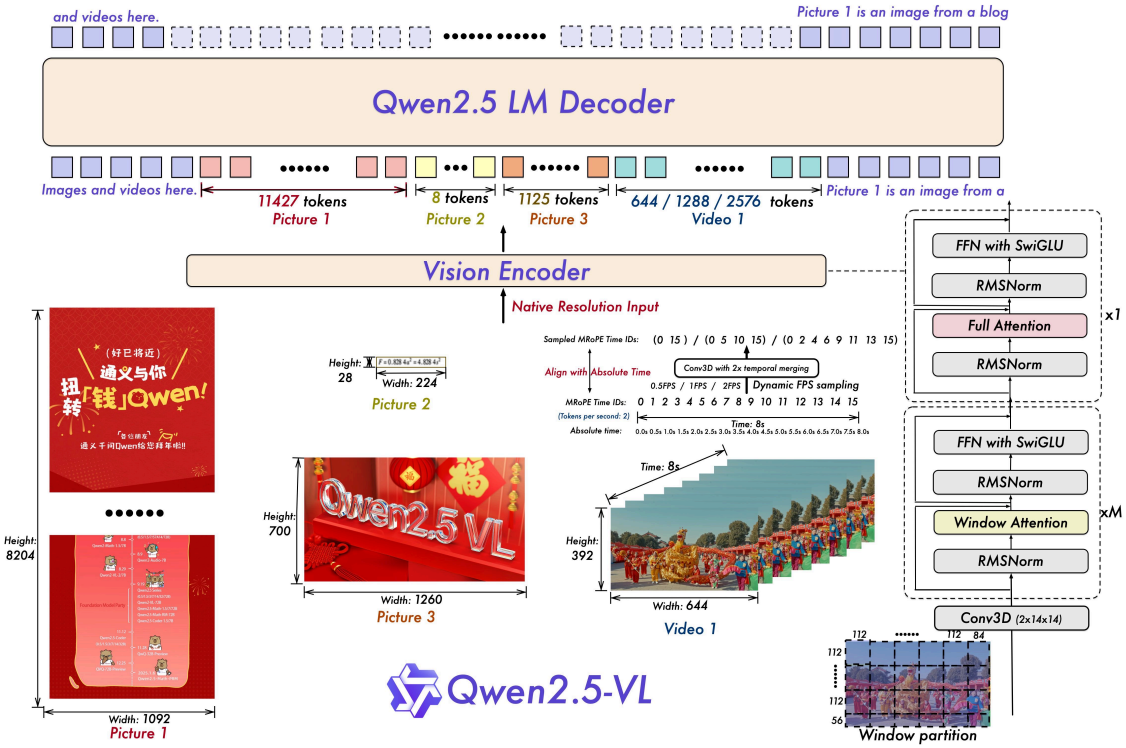


图 15.8 Qwen2.5-VL 框架展示了视觉编码器与语言模型解码器的集成，用以处理包括图像和视频在内的多模态输入。视觉编码器被设计为处理原生分辨率的输入并支持动态帧率采样。不同尺寸的图像和具有不同时帧率的视频帧被动态映射为长度各异的 token 序列。值得注意的是，MRoPE 在时间维度上将时间 ID 与绝对时间对齐，使模型能够更好地理解时间动态，例如事件的节奏和精确的时刻定位。处理后的视觉数据随后被输入到 Qwen2.5 LM 解码器。我们对 ViT 架构进行了重构，加入了诸如带 SwiGLU 激活的前馈网络（FFN）、用于归一化的 RMSNorm 以及基于窗口的注意力机制等先进组件，以提升性能和效率。

Qwen2.5-VL 的整体模型架构由三部分组成：

1. 大语言模型：Qwen2.5-VL 系列以大语言模型（LLM）作为其基础组件。该模型以 Qwen2.5 LLM 的预训练权重进行初始化。为了更好地满足多模态理解的需求，我们将一维 RoPE（旋转位置嵌入）修改为与绝对时间对齐的多模态旋转位置嵌入（Multimodal Rotary Position Embedding Aligned to Absolute Time）。
2. 视觉编码器：Qwen2.5-VL 的视觉编码器采用了重新设计的 ViT 架构。在结构上，我们引入了二维 RoPE 和窗口注意力，以支持原生输入分辨率并加速整个视觉编码器的计算。在训练和推理过程中，输入图像的高度和宽度在送入 ViT 之前会被调整为 28 的倍数。视觉编码器通过以 14 的步幅将图像切分为补丁来处理图像，生成一组图像特征。
3. 基于 MLP 的视觉-语言合并器：为了解决图像特征长序列带来的效率问题，我们采用一种简单但有效的方法：在将特征序列输入大型语言模型（LLM）之前对其进行压缩。具体来说，我们不是直接使用由 ViT 提取的原始的补丁嵌入，我们首先将空间上相邻的四个补丁嵌入分为一组。然后将这些分组嵌入串联起来，并通过一个两层的多层感知器（MLP）投影到与在 LLM 中使用的文本嵌入对齐的维度。该方法不仅降低了计算成本，还为动态压缩不同长度的图像特征序列提供了一种灵活的途径。

16. 扩散模型

扩散模型的兴起可以看作是近年来 AI 生成艺术作品领域取得突破的主要因素。

在图像创作方面，扩散模型已成为内容生成领域的前沿技术。尽管该模型于 2015 年首次推出，但已取得显著进展，并已成为 DALLE 和 Midjourney 等知名模型的核心机制。

> **DDPM 算法**
>
> Denoising Diffusion Probabilistic Models，去噪扩散概率模型

16.1 通俗讲解

16.1.1 扩散

16.1.1.1 物理学中的类比

想象一下一杯透明的水。如果我们加入少量其他颜色的液体，比如黄色的液体，会发生什么？黄色液体会逐渐均匀地扩散到整个玻璃杯中，最终的混合物会呈现出略带透明的黄色。

图 16.1 水滴的扩散过程

上面的过程被称为**正向扩散**：我们通过添加少量其他液体来改变环境状态。然而，进行**反向扩散**——将混合物恢复到其原始状态——是否同样容易？事实证明并非如此。即使在最好的情况下，实现这一点也需要高度复杂的机制。

16.1.1.2 将类比应用于机器学习

扩散也可以应用于图像。想象一下一张高质量的狗狗照片。我们可以通过逐渐添加随机噪声来轻松地变换这幅图像。结果，像素值会发生变化，使图像中的狗狗变得不那么明显，甚至无法辨认。这个变换过程称为**正向扩散**。

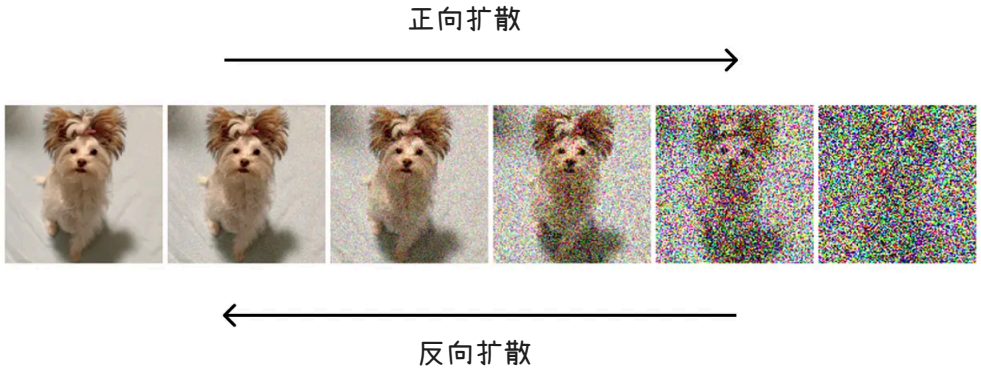


图 16.2 高清图像的扩散过程

我们也可以考虑反向操作：给定一张噪声图像，目标是重建原始图像。这项任务更具挑战性，因为与大量可能的噪声变化相比，可高度识别的图像状态要少得多。用前面提到的物理类比，这个过程称为**反向扩散**。

在本文中，我将通过示意图来解释它的工作原理。

16.1.2 扩散模型的架构

为了更好地理解扩散模型的结构，让我们分别检查两个扩散过程。

16.1.2.1 正向扩散

如前所述，前向扩散涉及逐步向图像添加噪声。然而，在实践中，这个过程要更加微妙一些。

最常见的方法是从均值为 0 的**高斯分布**中为图片中的每个像素采样一个随机值。然后将这个采样值（可以是正值也可以是负值）添加到像素的原始值中。对所有像素重复此操作会得到原始图像的噪声版本。

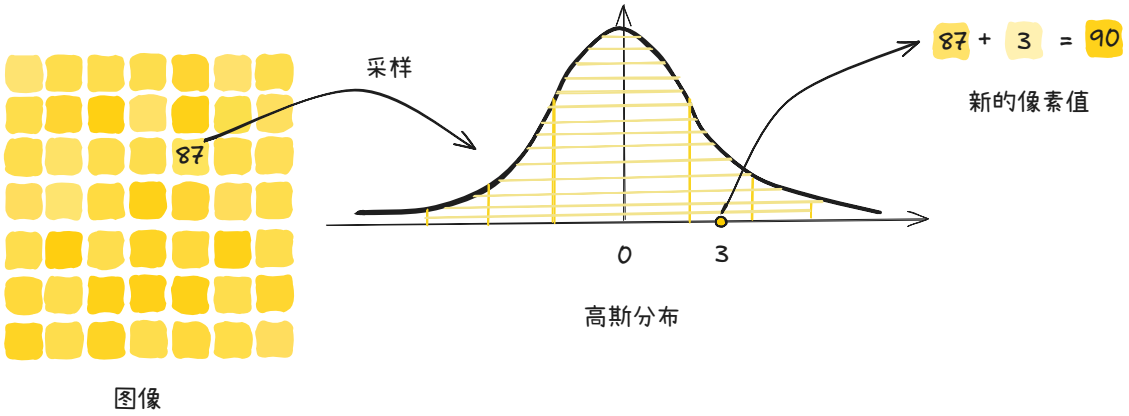


图 16.3 从高斯分布中采样一个随机值

> **通知**
>
> 所选的高斯分布通常方差较小，这意味着采样值通常较小。因此，每一步只会对图像产生微小的变化。
>
> 图中有 49 个像素点，那么需要从一个相同的高斯分布中采样 49 次噪声，然后将 49 个噪声分别加到 49 个像素点上。

正向扩散是一个迭代过程，其中噪声被多次应用于图像。随着每次迭代，生成的图像与原始图像的差异越来越大。经过数百次迭代（这在实际扩散模型中很常见）后，图像最终变得无法从纯噪声中识别出来。

16.1.2.2 反向扩散

现在你可能会问：执行所有这些正向扩散变换的目的是什么？答案是，每次迭代生成的图像都用于训练神经网络。

具体来说，假设我们在正向扩散过程中应用了 100 次连续噪声变换。然后，我们可以在每一步获取图像，并训练神经网络重建上一步的图像。预测图像与实际图像之间的差异使用损失函数计算——例如均方误差（MSE），它衡量两幅图像之间的平均像素差异。

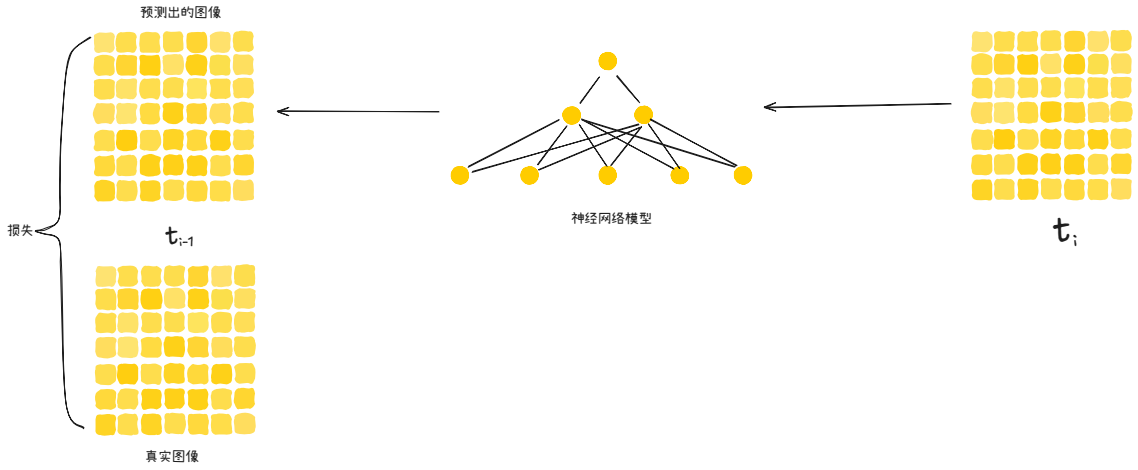


图 16.4 预测图像与实际图像之间的差异使用损失函数计算

通知

该模型的目标是检测添加的噪声并重建先前的图像。然后将预测图像与实际图像进行比较以计算损失。

这个例子展示了扩散模型重构原始图像的过程。同时，扩散模型可以被训练来预测添加到图像中的噪声。在这种情况下，要重构原始图像，只需要从前一次迭代的图像中减去预测的噪声就足够了。

虽然这两个任务看起来可能相似，但预测添加的噪声比图像重构要简单。

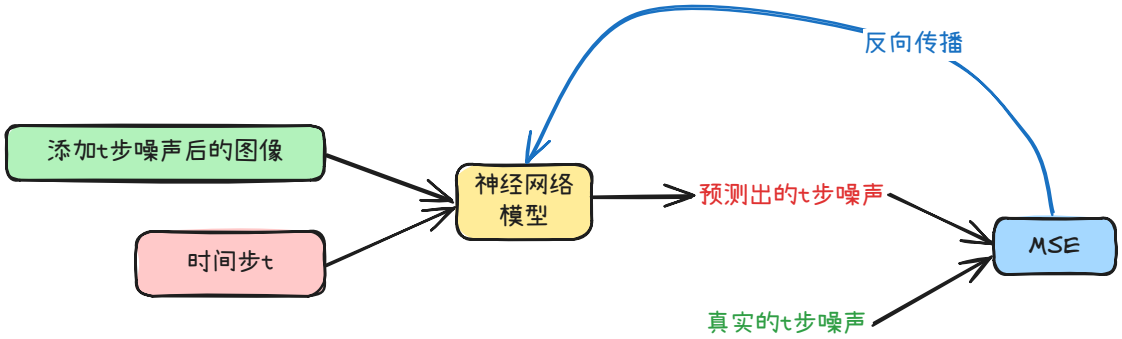


图 16.5 将噪声作为预测目标

16.1.3 模型设计

在对扩散技术有了基本的了解之后，有必要探索一些更高级的概念，以更好地理解扩散模型设计。

16.1.3.1 迭代次数

迭代次数是扩散模型中的关键参数之一：

通知

一方面，使用更多迭代次数意味着相邻步骤中的图像对差异会更小，从而使模型的学习任务更容易。另一方面，更高的迭代次数会增加计算成本。

虽然较少的迭代次数可以加快训练速度，但模型可能无法学习步骤之间的平滑过渡，从而导致性能不佳。

通常，迭代次数选择在 50 到 1000 之间。

16.1.3.2 神经网络架构

最常见的是，U-Net 架构被用作扩散模型的主干。以下是一些原因：

- U-Net 保留了输入和输出图像的尺寸，确保在整个逆扩散过程中图像大小保持一致。
- 其瓶颈架构能够在将整幅图像压缩到潜在空间后将其重建。同时，通过残差连接保留关键图像特征。
- U-Net 最初设计用于生物医学图像分割，其中像素级精度至关重要，它的优势可以很好地转化为需要精确预测单个像素值的扩散任务。

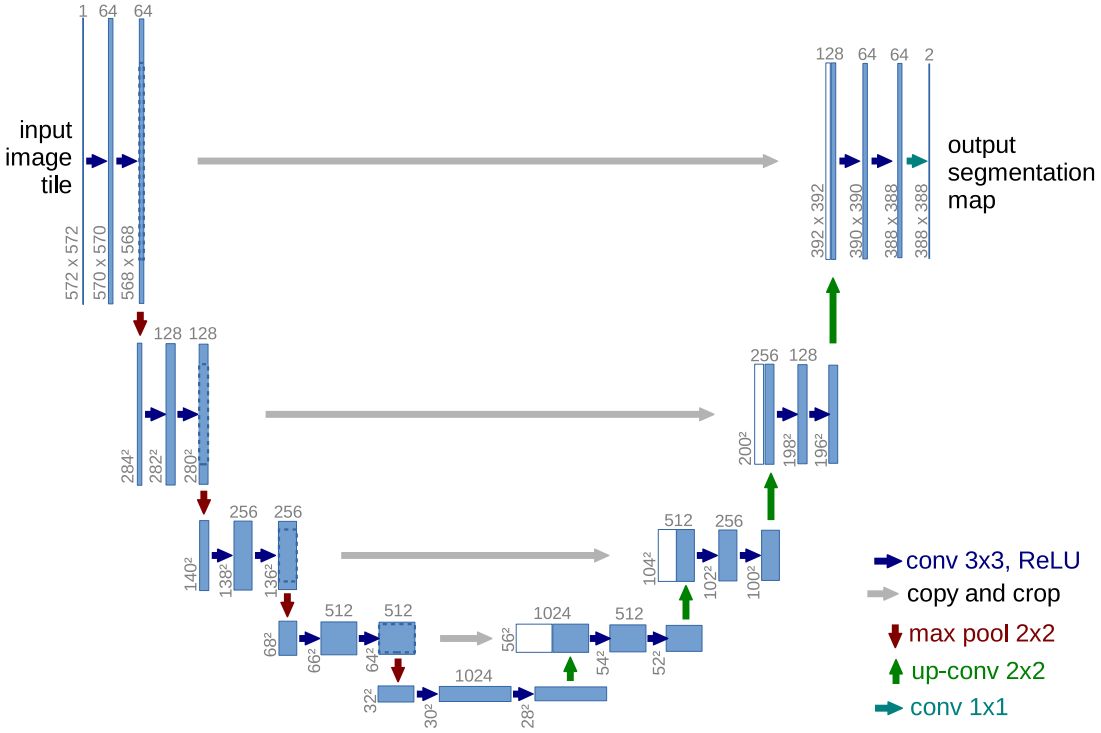


图 16.6 U-net 架构（以最低分辨率为 32×32 像素为例）。每个蓝色方框对应一个多通道特征图。方框顶部标注了通道数量。方框左下角标注了 x-y 尺寸。白色方框表示复制的特征图。箭头表示不同的操作。

16.1.3.3 共享网络

乍一看，似乎有必要为扩散过程的每次迭代训练一个单独的神经网络。虽然这种方法可行，并且可以产生高质量的推理结果，但从计算角度来看，它效率极低。例如，如果扩散过程包含 1000 个时间步，我们就需要训练 1000 个 U-Net 模型——这是一项极其耗时且资源密集的任务。

然而，我们可以观察到，**不同迭代中的任务配置本质上是相同的**：在每种情况下，我们都需要重建一个尺寸相同、且经过相似幅度噪声改变的图像。这一重要洞见促成了**在所有迭代中使用单个共享神经网络**的想法。

实际上，这意味着我们使用一个具有共享权重的 U-Net 模型，该模型基于来自不同扩散步骤的图像对进行训练。在推理过程中，含噪图像会多次通过同一个经过训练的 U-Net 模型，逐步进行优化，直到生成高质量的图像。

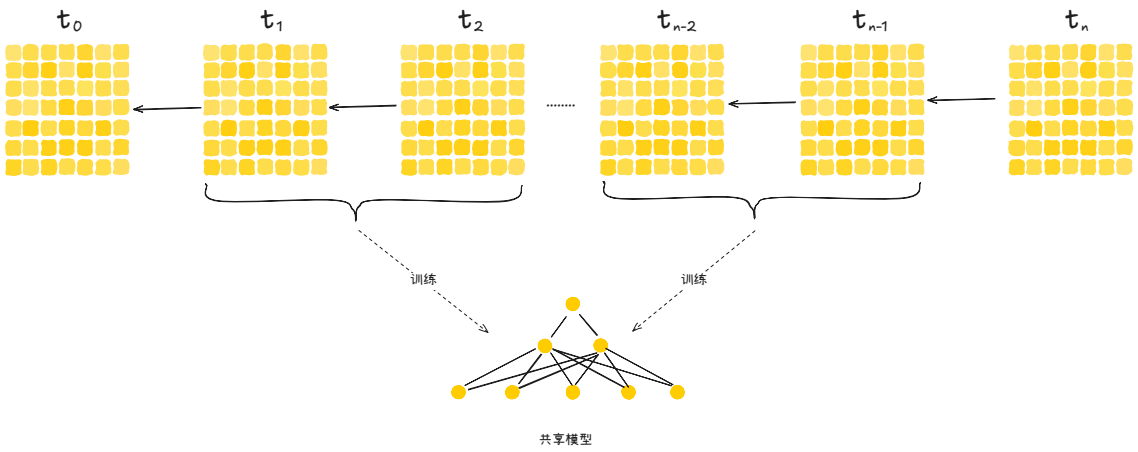


图 16.7 共享模型

虽然由于仅使用单一模型，生成质量可能会略有下降，但训练速度的提升却非常显著。

16.2 扩散模型理论简介

16.2.1 概述

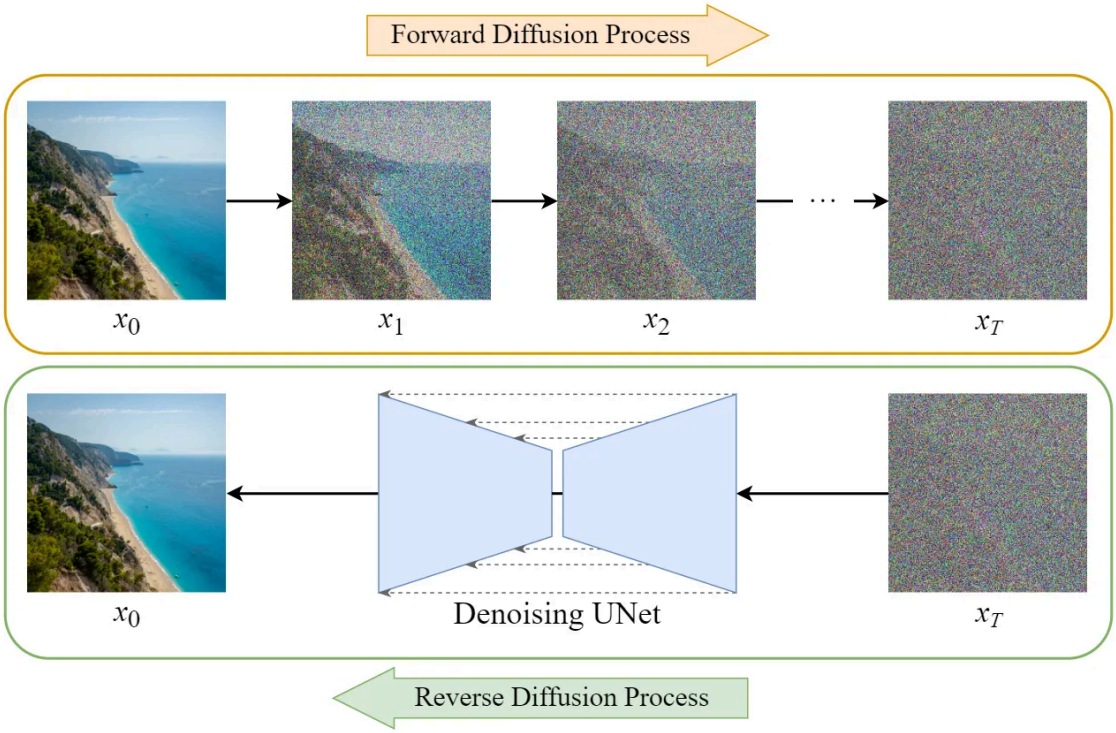


图 16.8 正向扩散和反向扩散

扩散模型的训练可以分为两部分：

1. 正向扩散过程 → 给图像添加噪声。
2. 反向扩散过程 → 从图像中去除噪声。

16.2.2 正向扩散过程

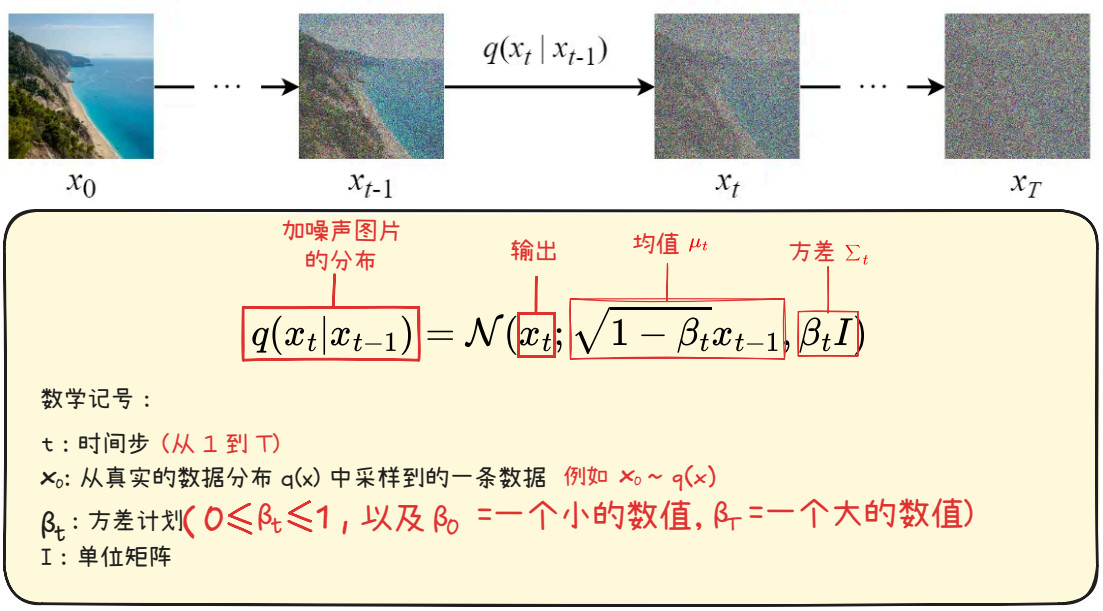


图 16.9 正向扩散公式图解

正态分布

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}} \tag{16.1}$$

方差计划

假设$\beta_{\text{start}} = 0.0002, \beta_{\text{end}} = 0.04$，则第 1 步添加的高斯噪声的方差是 0.0002，第 2 步添加的高斯噪声的方差是$0.0002 + \frac{0.04-0.0002}{1000}$。

前向扩散过程逐步将高斯噪声添加到输入图像x_0中，总共会有T步。该过程将产生一系列带噪声的图像样本$x_1, x_2, ..., x_T$。

当$T \to \infty$时，最终结果将变成完全噪声图像，就像从**各向同性**的高斯分布中采样出来的噪声一样。

首先，如果$z \sim \mathcal{N}(\mu, \sigma^2)$的话，那么正态分布可以写成如下公式：

$$z = \mu + \sigma\varepsilon \quad \text{其中 } \varepsilon \sim \mathcal{N}(0, 1) \tag{16.2}$$

利用这个技巧，我们可以将采样图像x_t表示如下：

$$x_t = \sqrt{1-\beta_t}x_{t-1} + \sqrt{\beta_t}\varepsilon_{t-1} \tag{16.3}$$

式（16.3）的代码实现，逐步向图片中添加噪声 Python

```
import torch
import torchvision.transforms as transforms
import matplotlib.pyplot as plt

image = plt.imread("./flower.png")
```

```
print(image.shape)

preprocess = transforms.ToTensor()
x = preprocess(image)
print(image.shape)

def reverse_to_img(x):
    x = x * 255
    x = x.clamp(0, 255)
    x = x.to(torch.uint8)
    to_pil = transforms.ToPILImage()
    return to_pil(x)

# 最大时间步
T = 1000
# 方差计划的起始值
beta_start = 0.0001
# 方差计划的结束值
beta_end = 0.02
betas = torch.linspace(beta_start, beta_end, T)
print(betas)

imgs = []

for t in range(T):
    if t % 100 == 0:
        img = reverse_to_img(x)
        imgs.append(img)

    beta = betas[t]
    eps = torch.randn_like(x) # 生成和x形状相同的噪声
    x = torch.sqrt(1 - beta) * x + torch.sqrt(beta) * eps

# 2行5列的方式显示10张图片
plt.figure(figsize=(15, 6))
for i, img in enumerate(imgs[:10]):
    plt.subplot(2, 5, i + 1)
    plt.imshow(img)
    plt.title(f"Noise: {i * 100}")
    plt.axis("off")

plt.show()
```

> ⚡ 一步一步的添加噪声太麻烦了！
>
> 根据上面的公式，如果想要从原始图片x_0得到添加了500步噪声的图片x_{500}需要迭代500次！

但我们不需要设计一种算法来迭代地向图像中添加噪声，而是可以使用闭式公式（解析解）在特定的时间步t直接对噪声图像进行采样。

给定原始图片x_0和时间步t可以直接得到添加了t步噪声的图像x_t。公式如下：

给定原始图片x_0和时间步t直接采样出x_t的公式

$$x_t = \sqrt{\overline{\alpha}_t} x_0 + \sqrt{1-\overline{\alpha}_t}\varepsilon \tag{16.4}$$

其中：
- $\overline{\alpha}_t = \alpha_t \alpha_{t-1} ... \alpha_1$
- $\alpha_t = 1 - \beta_t$
- $\varepsilon \sim \mathcal{N}(0,1)$

现在我们可以使用此公式在任何时间步直接对x_t进行采样，这使得前向扩散过程更快。

式（16.4）的实现，使用闭式解添加噪声 Python

```python
import torch
import torchvision.transforms as transforms
import matplotlib.pyplot as plt

image = plt.imread("./flower.png")
print(image.shape)

preprocess = transforms.ToTensor()
x = preprocess(image)
print(image.shape)


def reverse_to_img(x):
    x = x * 255
    x = x.clamp(0, 255)
    x = x.to(torch.uint8)
    to_pil = transforms.ToPILImage()
    return to_pil(x)


# 最大时间步
T = 1000
# 方差计划的起始值
beta_start = 0.0001
# 方差计划的结束值
beta_end = 0.02
betas = torch.linspace(beta_start, beta_end, T)
print(betas)

# 一步得到x_t，使用闭式解（closed form）
def add_noise(x_0, t, betas):
    T = len(betas)

    alphas = 1 - betas  # [α_1, α_2, ..., α_T]
    # cumprod功能：[1,2,3,4] ⟶ [1,2,6,24]
    alpha_bars = torch.cumprod(alphas, dim=0)
    t_idx = t - 1
    alpha_bar = alpha_bars[t_idx]  # ᾱ_t
```

```
39
40      eps = torch.randn_like(x_0)
41      # 闭式解
42      x_t = torch.sqrt(alpha_bar) * x_0 + torch.sqrt(1 - alpha_bar)*eps
43
44      return x_t
45
46  t = 100
47  x_t = add_noise(x, t, betas)
48
49  img = reverse_to_img(x_t)
50  plt.imshow(img)
51  plt.title(f"Noise: {t}")
52  plt.axis("off")
53  plt.show()
```

16.2.3 反向扩散过程

反向扩散过程是从一张完全的高斯噪声图片中，逐步去除噪声，来生成一张图片。这个逆向过程很难，相当于一滴墨水在水里面扩散开来（正向扩散），想要将变黑的水逆向为原来的墨水刚滴入水中时的状态。这几乎不可能做到，所以我们用神经网络来解决这个问题。下面是训练神经网络的原理。

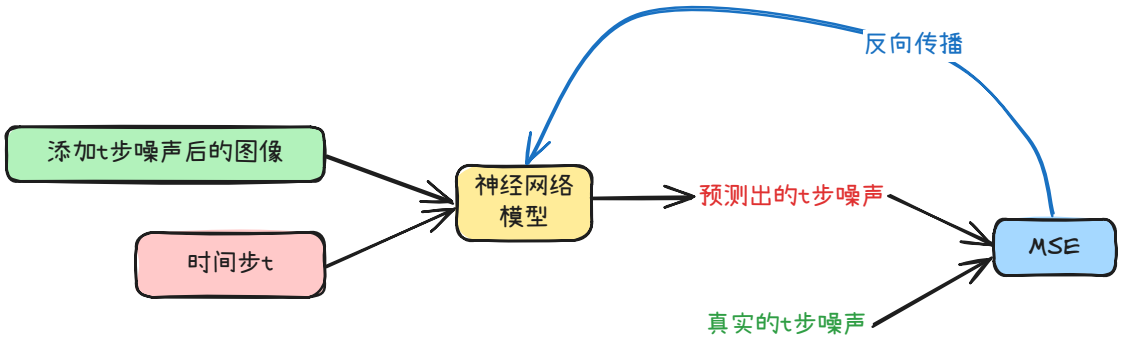


图 16.10 以噪声为预测目标

因此最终的训练目标如下：

$$\begin{gathered}\text{时间步 } t \text{ 的图片} \\ x_t = \sqrt{\overline{\alpha}_t} x_0 + \sqrt{1-\overline{\alpha}_t}\varepsilon \\ \mathcal{L}_{\text{simple}} = \mathbb{E}_{t,x_0,\varepsilon}\left[\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2\right]\end{gathered} \tag{16.5}$$

16.2.4 U-Net 模型

16.2.4.1 数据集

在每个 Epoch：

1. 将为每个训练样本（图像）选择一个随机时间步t。
2. 对每幅图像添加高斯噪声（对应于t）。
3. 将时间步转换为嵌入（向量）。

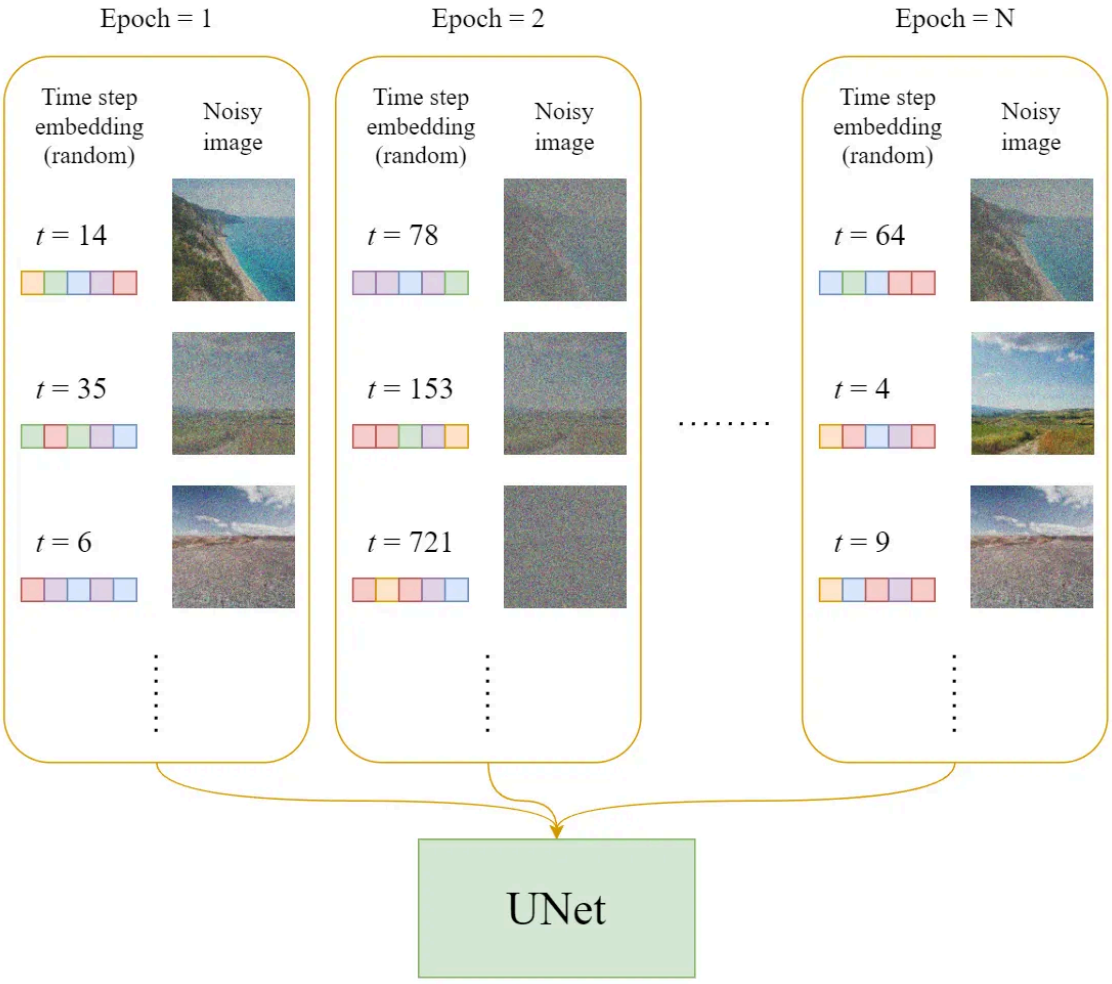


图 16.11 训练 unet

16.2.4.2 训练

1 Repeat
2 $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ ▷ 从数据集中抽取一张图片
3 $t \sim \text{Uniform}(\{1, ..., T\})$ ▷ 从均匀分布中采样一个时间步
4 $\varepsilon \sim \mathcal{N}(0, \mathbf{I})$ ▷ 从正态分布中采样一个噪声
5 使用梯度下降法，梯度为
6 $$\nabla_\theta \left\| \varepsilon - \varepsilon_\theta\left(\sqrt{\overline{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\overline{\alpha}_t}\varepsilon, t\right)\right\|^2 \quad (16.6)$$
7 Until 收敛

图 16.12 U-net 训练算法

官方的训练算法如上，下图是一个训练步骤的示意图：

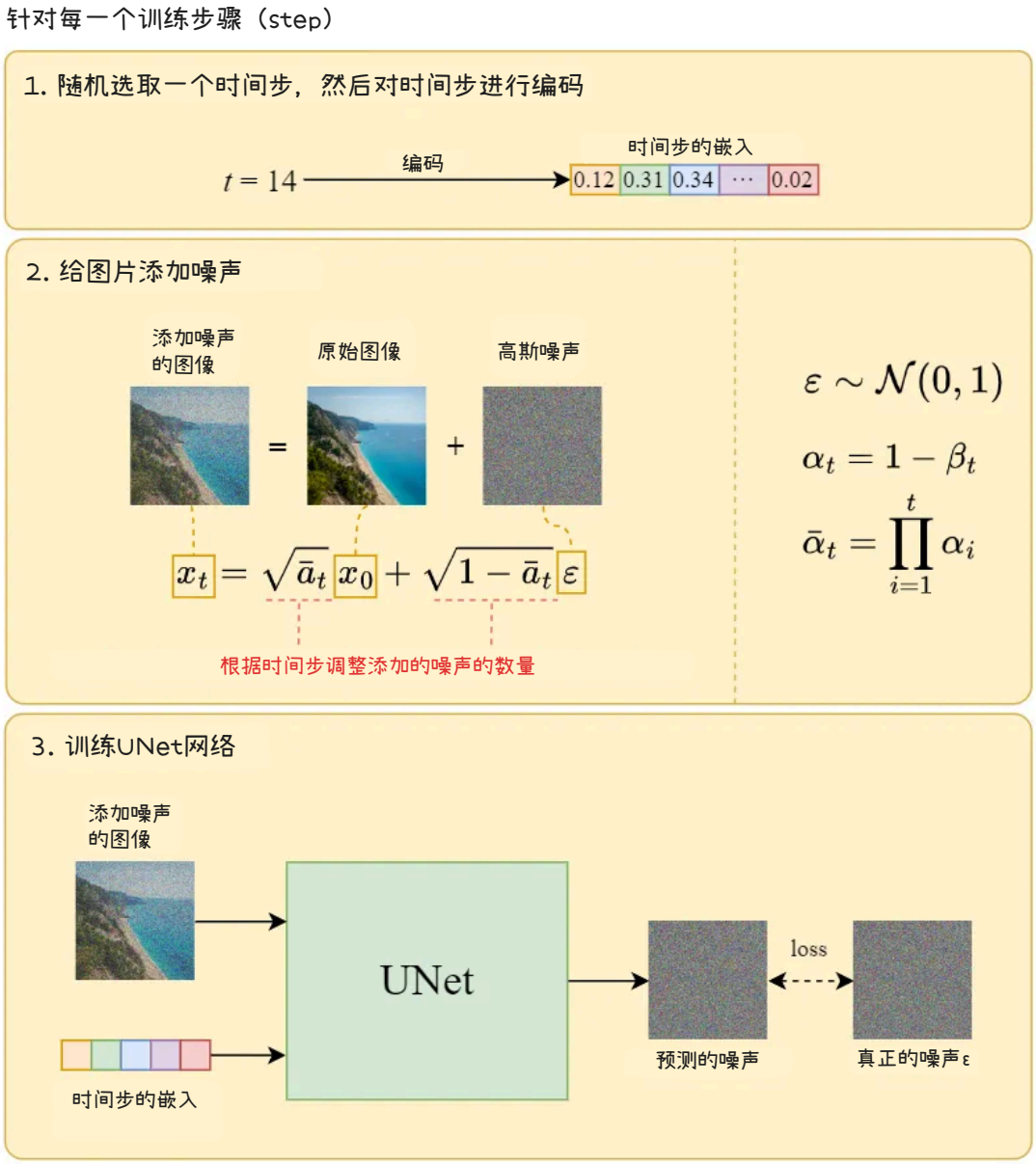


图 16.13 扩散模型训练步骤

16.2.4.3 反向扩散

$\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$ ▷ 从高斯分布中采样一张纯噪声图片

for $t = T, ..., 1$ do

  $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = 0$

  $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon_\theta(\boldsymbol{x}, t)\right) + \sigma_t\ \mathbf{z}$ ▷ σ_t一般取$\sqrt{\beta_t}$

end for

return $\mathbf{x}_0$

图 16.14 采样算法（去噪算法，反向扩散算法）

我们可以使用上述算法从噪声中生成图像。下图是它的说明：

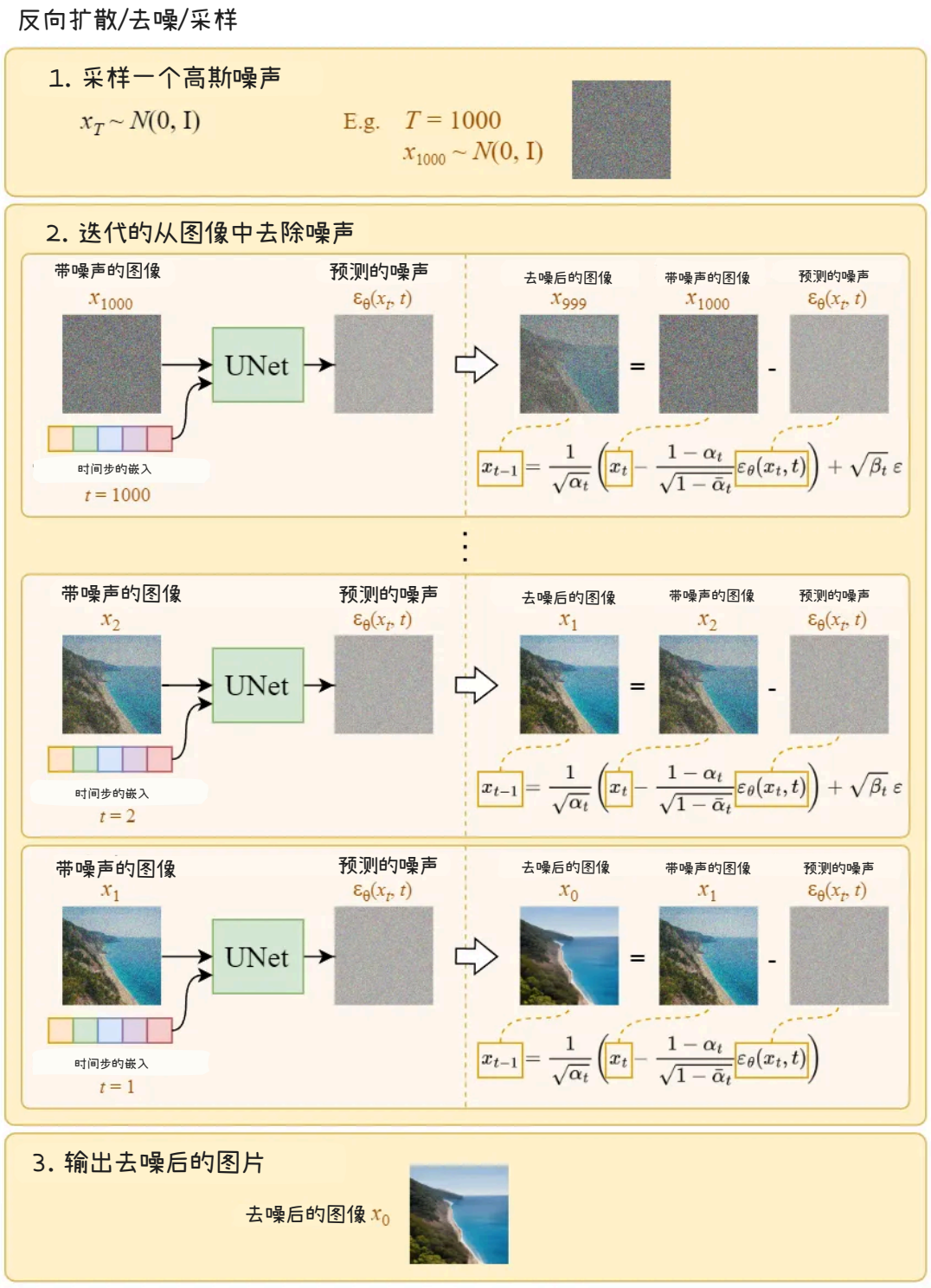


图 16.15 扩散模型的采样过程

请注意，在最后一步，我们只是输出学习到的平均值$\mu_\theta(x_1, 1)$，而不向其中添加噪声。

16.3 代码实现

16.3.1 扩散过程相关代码

我们需要针对扩散的时间步来制定一个方差的调度计划。每个时间步，都在前面一个时间步的图像中添加一个高斯噪声。在开始向图像中添加噪声的时候，需要高斯噪声的方差小一些，不至于一开始就把图像变得很模糊。而到了后面，添加噪声就可以大胆一些了，反正已经模糊了。所以后面添加的高斯噪声需要方差大一些。

> 🔥 高斯噪声的方差
>
> 公式中的β_t就是第t个时间步要添加的噪声的方差。

在代码中如下

```python
# self.betas: 方差计划调度表
self.betas = torch.linspace(
    beta_start, # beta_start: 起始β值，论文中等于0.0001
    beta_end, # beta_end: 结束β值，论文中等于0.02
    num_timesteps, # num_timesteps: 时间步的数量：1000
    device=device
)
```

而由于$\alpha_t = 1 - \beta_t$，所以有如下代码

```python
self.alphas = 1 - self.betas
```

这样就计算出了每个时间步的α_t。

而由于$\overline{\alpha}_t = \prod_{s=1}^{t} \alpha_s$，所以使用 `torch.cumprod` 来计算

```python
self.alpha_bars = torch.cumprod(self.alphas, dim=0)
```

由于我们已经使用重参数技巧来给图像添加噪声，也就是通过解析解可以直接得到添加了t个时间步的噪声的图像。所以前向扩散过程就用了这个公式。

> 🔥 给定原始图片$\boldsymbol{x_0}$和时间步$\boldsymbol{t}$直接采样出$\boldsymbol{x_t}$的公式
>
> $$x_t = \sqrt{\overline{\alpha}_t} x_0 + \sqrt{1 - \overline{\alpha}_t} \varepsilon \tag{16.7}$$
>
> 其中：
> - $\overline{\alpha}_t = \alpha_t \alpha_{t-1} \ldots \alpha_1 = \prod_{s=1}^{t} \alpha_s$
> - $\alpha_t = 1 - \beta_t$
> - $\varepsilon \sim \mathcal{N}(0, \mathbf{I})$

公式和代码基本是对应的。

```python
def add_noise(self, x_0, t):
    # x_0是原始图片，t是时间步
    # T：时间步的数量
    T = self.num_timesteps
    # 确保 1 ≤ t ≤ T
    assert (t >= 1).all() and (t <= T).all()

    t_idx = t - 1  # alpha_bars[0] is for t=1
```

```
        # ᾱ_t
        alpha_bar = self.alpha_bars[t_idx]  # (N,)
        N = alpha_bar.size(0)
        alpha_bar = alpha_bar.view(N, 1, 1, 1)  # (N, 1, 1, 1)
        # ε ~ 𝒩(0, 𝐈)
        # 噪声的形状或者点数等于图片的像素点
        noise = torch.randn_like(x_0, device=self.device)
        # x_t = √ᾱ_t x_0 + √(1 − ᾱ_t) ε
        x_t = torch.sqrt(alpha_bar) * x_0 + torch.sqrt(1 - alpha_bar) * noise
        # 返回第t步的图像x_t和添加的噪声noise，后面作为训练数据
        return x_t, noise
```

接下来编写反向去噪过程的代码，也就是从x_t预测x_{t-1}。

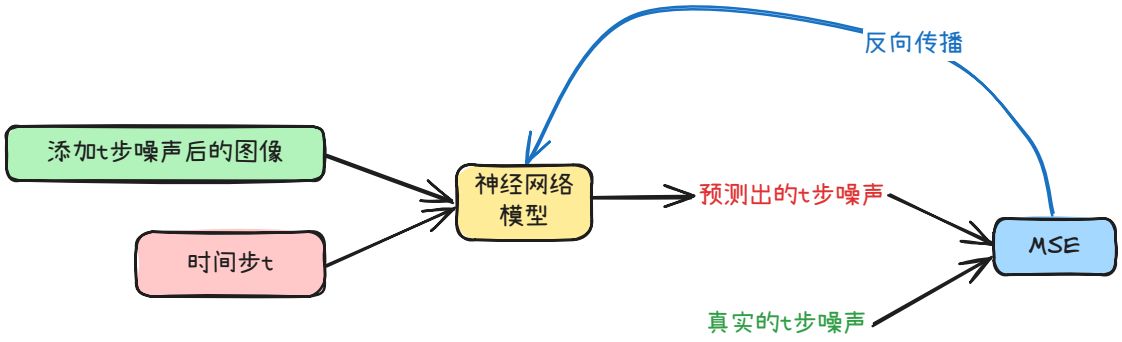


图 16.16 以噪声为预测目标

给定了时间步和添加噪声的图像，模型可以预测出这幅图片中的噪声有多少，那么我们从x_t中将预测出的噪声减掉，就可以去噪了！

```python
def denoise(self, model, x, t):
    # x是图片，模型会认为x是添加了t步噪声的图片
    # t是时间步
    T = self.num_timesteps
    assert (t >= 1).all() and (t <= T).all()

    t_idx = t - 1  # alphas[0] is for t=1
    alpha = self.alphas[t_idx] # α_t
    alpha_bar = self.alpha_bars[t_idx] # ᾱ_t
    alpha_bar_prev = self.alpha_bars[t_idx-1] # ᾱ_{t-1}

    N = alpha.size(0)
    alpha = alpha.view(N, 1, 1, 1)
    alpha_bar = alpha_bar.view(N, 1, 1, 1)
    alpha_bar_prev = alpha_bar_prev.view(N, 1, 1, 1)

    model.eval()
    with torch.no_grad():
      eps = model(x, t) # 根据时间步和图像x_t预测噪声
    model.train()
    # 公式中的z
```

```
    noise = torch.randn_like(x, device=self.device)
    # 如果时间步为1的话，我们就不再添加σ_z噪声
    noise[t == 1] = 0  # no noise at t=1
    # 均值μ_θ(x_t,t) = 1/√α_t (x_t - (1-α_t)/√(1-ᾱ_t) ε_θ(x_t,t))
    mu = (x - ((1-alpha) / torch.sqrt(1-alpha_bar)) * eps)
       / torch.sqrt(alpha)
    # 标准差σ_t = √((1-α_t)(1-ᾱ_{t-1})/(1-ᾱ_t))
    std = torch.sqrt((1-alpha) * (1-alpha_bar_prev) \
        / (1-alpha_bar))
    # 返回x_{t-1}
    return mu + noise * std
```

1 $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$ ▷ 从高斯分布中采样一张纯噪声图片

2 for $t = T, ..., 1$ do

3 $\quad \mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = 0$

4 $\quad \mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon_\theta(\boldsymbol{x}, t)\right) + \sigma_t\ \mathbf{z}$ ▷ 在 DDPM 论文中$\sigma_t = \sqrt{\frac{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}}$

5 end for

6 return $\mathbf{x}_0$

图 16.17 采样算法（去噪算法，反向扩散算法）

上面的代码实现了伪代码中的第 4 步。

16.3.2 U-Net 神经网络

U-Net 最初是为医学图像的语义分割而开发的模型。语义分割是为图像中的每个像素分配特定的类别标签的任务，如图所示。

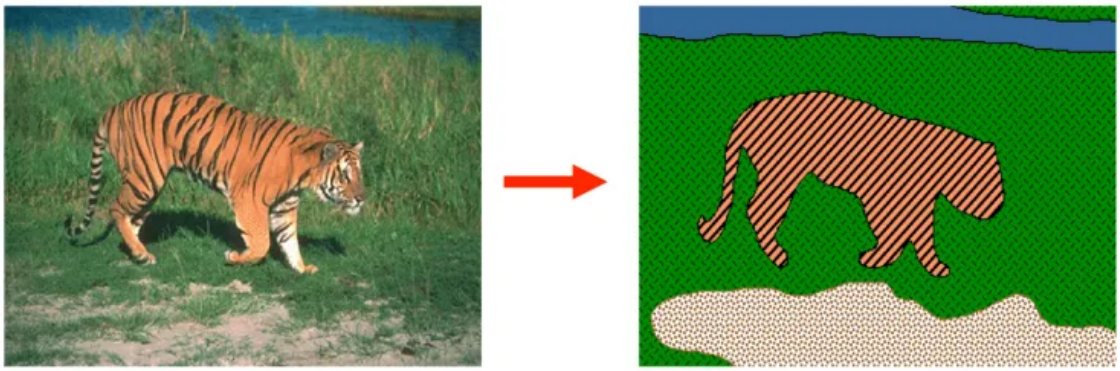


图 16.18 U-Net 执行语义分割

图中 U-Net 的输入是形状为(C, H, W)的图像数据，其中，C 是输入图像的通道数（RGB 图像为 3），H 是图像的高度，W 是图像的宽度。输出是形状为(D, H, W)的张量，其中 D 是要分类的类别数。模型对每个像素输出 D 个类别的概率分布。而在扩散模型中，输出被设置为(C, H, W)（D=C），即输入和输出的通道数都被设置为 C。

U-Net 的名称源于网络结构的形状，因为它与字母“U”的形状相似。

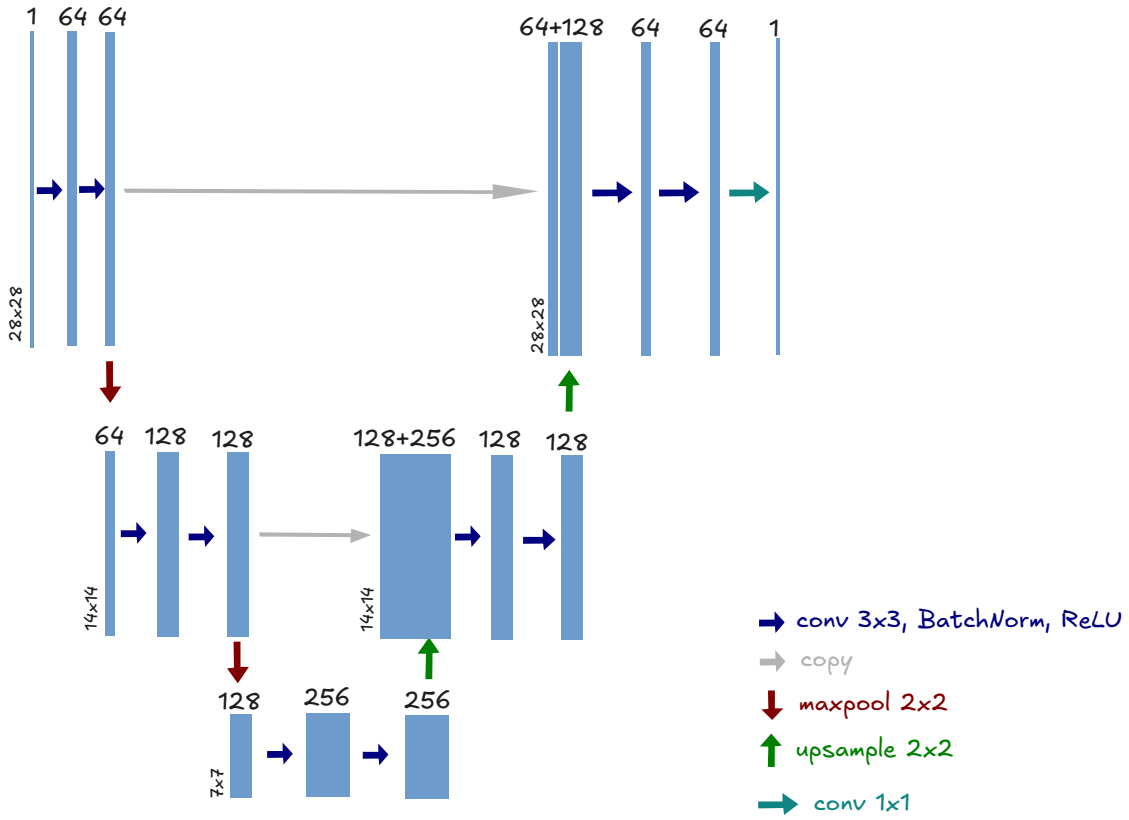


图 16.19 本章要实现的 U-Net

U-Net 的处理过程分为前半部分的“缩小阶段”和后半部分的“扩大阶段”。在前半部分的缩小阶段中，在卷积层进行处理的同时，特征图会逐渐缩小。缩小特征图的层称为“下采样层”。在后半部分的扩大阶段中，在卷积层进行特征抽取的同时，特征图会逐渐扩大，这与前半部分正好相反。扩大特征图的层称为“上采样层”。

U-Net 的重要特征是跳跃连接（skip connection）。这是一种在网络的缩小阶段和扩大阶段之间直接传递特征图的机制。这种跳跃连接使得 U-Net 能够捕捉对象整体的特征，同时使用更精细的空间位置信息进行处理。

如上图所示，U-Net 包含两个缩小阶段和两个扩大阶段。每个阶段由两个卷积层进行处理。为了实现这个 U-Net，首先要实现一个名为 ConvBlock 的类。如下图所示，该类分别对卷积层，批量归一化层和 ReLU 函数的处理执行了两次。

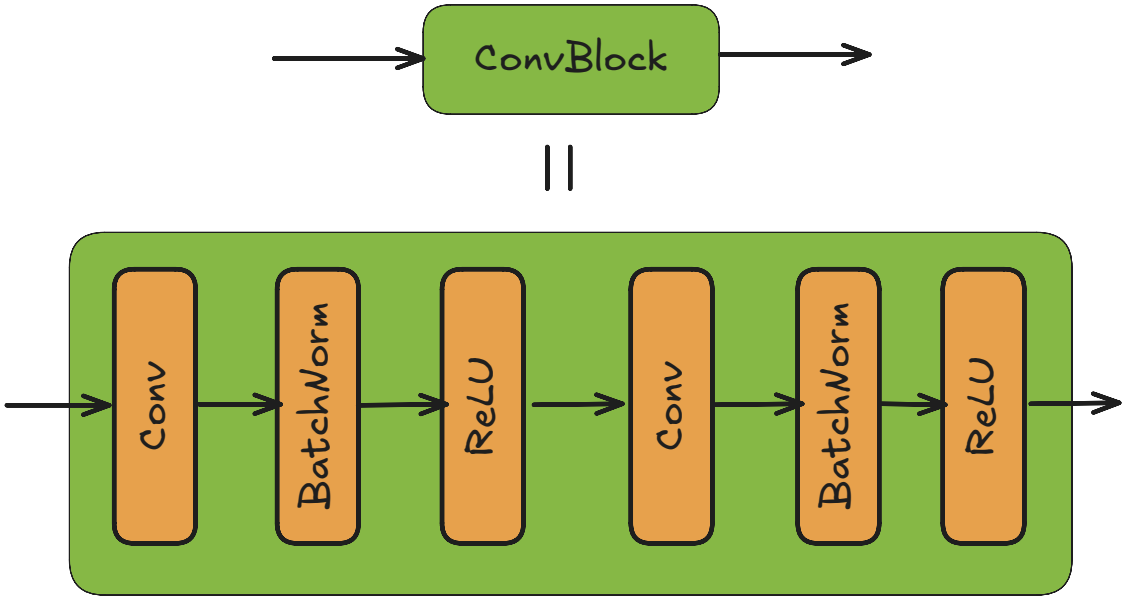


图 16.20 ConvBlock 类执行的处理

然后是 ConvBlock 类的代码，如下所示。

```python
import torch
from torch import nn

class ConvBlock(nn.Module):
    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.convs = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, 3, padding=1)
            nn.BatchNorm2d(out_ch)
            nn.ReLU()
            nn.Conv2d(out_ch, out_ch, 3, padding=1)
            nn.BatchNorm2d(out_ch)
            nn.ReLU()
        )

    def forward(self, x):
        return self.convs(x)
```

这段代码使用 nn.Sequential()串联了多个层。这样做的目的是使数据按顺序逐层通过。使用 ConvBlock 类，我们可以实现上图所示的 UNet 类。代码如下所示。

```python
class UNet(nn.Module):
    def __init__(self, in_ch=1):
        super().__init__()

        self.down1 = ConvBlock(in_ch, 64)
        self.down2 = ConvBlock(64, 128)
        self.bot1 = ConvBlock(128, 256)
        self.up2 = ConvBlock(128 + 256, 128)
```

```
        self.up1 = ConvBlock(128 + 64, 64)
        self.out = nn.Conv2d(64, in_ch, 1)

        self.maxpool = nn.MaxPool2d(2)
        self.upsample = nn.Upsample(scale_factor=2, mode="bilinear")

    def forward(self, x):
        x1 = self.down1(x)
        x = self.maxpool(x1)
        x2 = self.down2(x)
        x = self.maxpool(x2)
        x = self.bot1(x)

        x = self.upsample(x)
        x = torch.cat([x, x2], dim=1)
        x = self.up2(x)
        x = self.upsample(x)
        x = torch.cat([x, x1], dim=1)
        x = self.up1(x)
        x = self.out(x)
        return x
```

这段代码使用最大池化（`nn.MaxPool2d`）来缩小数据。这会使张量的大小缩小$\frac{1}{2}$。而在扩大数据的处理中，代码使用了双线性插值的上采样（`nn.Upsample`）。这会使张量的大小扩大2倍。

> 双线性插值
>
> 双线性插值是一种用于扩大图像大小的技术。双线性插值可以创建新的像素，其值基于原始图像中像素的值计算得出。这样就能平滑地扩大图像。

进行`U-Net`的跳跃连接的代码是`torch.cat([x, x1], dim=1)`。`torch.cat()`函数是连接张量的函数，其中`dim=1`指定了连接的维度。在本例中， 如果`x`的形状是`(N, C, H, W)`，`x1`的形状是`(N, I, H, W)`，那么连接后的张量的形状就是`(N, C + I, H, W)`。

上面我们实现了处理x_t的`U-Net`。剩下的任务是将时刻t引入`U-Net`中。需要注意的是这里的t是整数。在神经网络中，将整数变换为向量可以提高训练和预测的效率。扩散模型在对时刻t进行编码时，常常使用正弦位置编码。

正弦位置编码因在论文“`Attention Is All You Need`”的`Transformer`模型中使用而闻名。正弦位置编码的意思是“使用正弦波（`sin`波）对位置信息进行编码”。位置信息是指序列数据中每个元素出现的位置。例如，在自然语言处理任务中，单词出现的位置就相当于位置信息。扩散模型的时刻t也是位置信息。

正弦位置编码可以将整数t变换为向量$\boldsymbol{v}$。如果变换后的向量$\boldsymbol{v}$的维度为D，则其第i个元素的数学式如下所示。

$$\mathbf{v}_i = \begin{cases} \sin\left(\frac{t}{10000^{\frac{i}{D}}}\right), & i\text{为偶数时} \\ \cos\left(\frac{t}{10000^{\frac{i}{D}}}\right), & i\text{为奇数时} \end{cases} \tag{16.8}$$

上面的正弦位置编码不以绝对值来编码位置信息，而是通过具有循环特性的`sin`函数和`cos`函数来编码位置信息。这样一来，位置信息中的相对差异和循环模式就能清晰地显示出来，使模型能够更有效地学习序列数据中的相对位置关系。

下面我们来实现正弦位置编码。首先，我们将实现对单个时刻数据（整数）进行编码的函数_pos_encoding()。然后，我们将使用它来实现一个名为pos_encoding()的处理批量数据的的数。以下是_pos_encoding()的代码。

```python
import torch

def _pos_encoding(t, output_dim, device="cpu"):
    D = output_dim
    v = torch.zero(D, device=device)
    i = torch.arange(0, D, device=device) ①
    div_term = 10000 ** (i / D)

    v[0::2] = torch.sin(t / div_term[0::2]) ②
    v[1::2] = torch.cos(t / div_term[1::2])
    return v
```

这段代码基于输入时刻(t)和输出维度(output_dim)进行位置编码。代码①处通过torch.arange(D)创建张量[0, 1, ..., D]。代码②处使用切片语法 v[0::2]来指定“从 v 的第 0 个元素开始，每次跳过一个元素后的所有元素”。也就是说，指定向量 v 的偶数索引(0, 2, 4, ...)对应的元素，并对这些元素进行正弦编码。

接下来，我们将实现用于处理批量数据的正弦位置编码。为了易于理解，这里使用 for 语句来实现，代码如下所示。

```python
def pos_encoding(ts, output_dim, device="cpu"):
    batch_size = len(ts)
    v = torch.zeros(batch_size, output_dim, device=device)
    for i in range(batch_size):
        v[i] = _pos_encoding(ts[i], output_dim, device)
    return v
```

参数 ts 是张量。对于该张量的每个元素，调用刚刚实现的_pos_encoding 函数。这样我们就完成了正弦位置编码的实现。

最后将正弦位置编码嵌入到U-Net中。这里我们将正弦位置编码嵌入到之前实现的ConvBlock类中。ConvBlock是拥有两个卷积层的处理单元。我们使用ConvBlock实现了U-Net，如图所示。

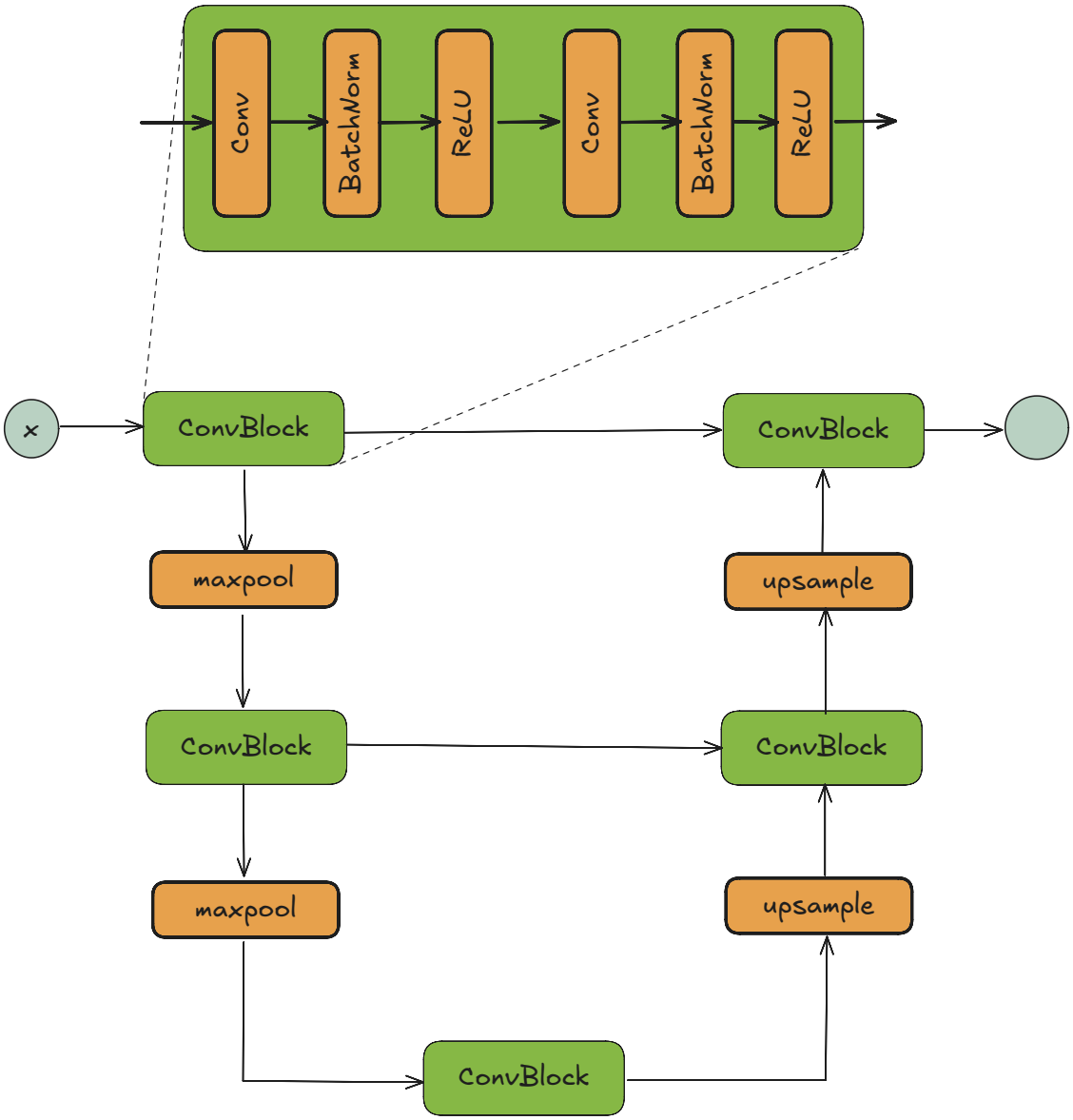


图 16.21 我们实现的 U-Net

这里要向 5 个 ConvBlock 添加正弦位置编码信息。

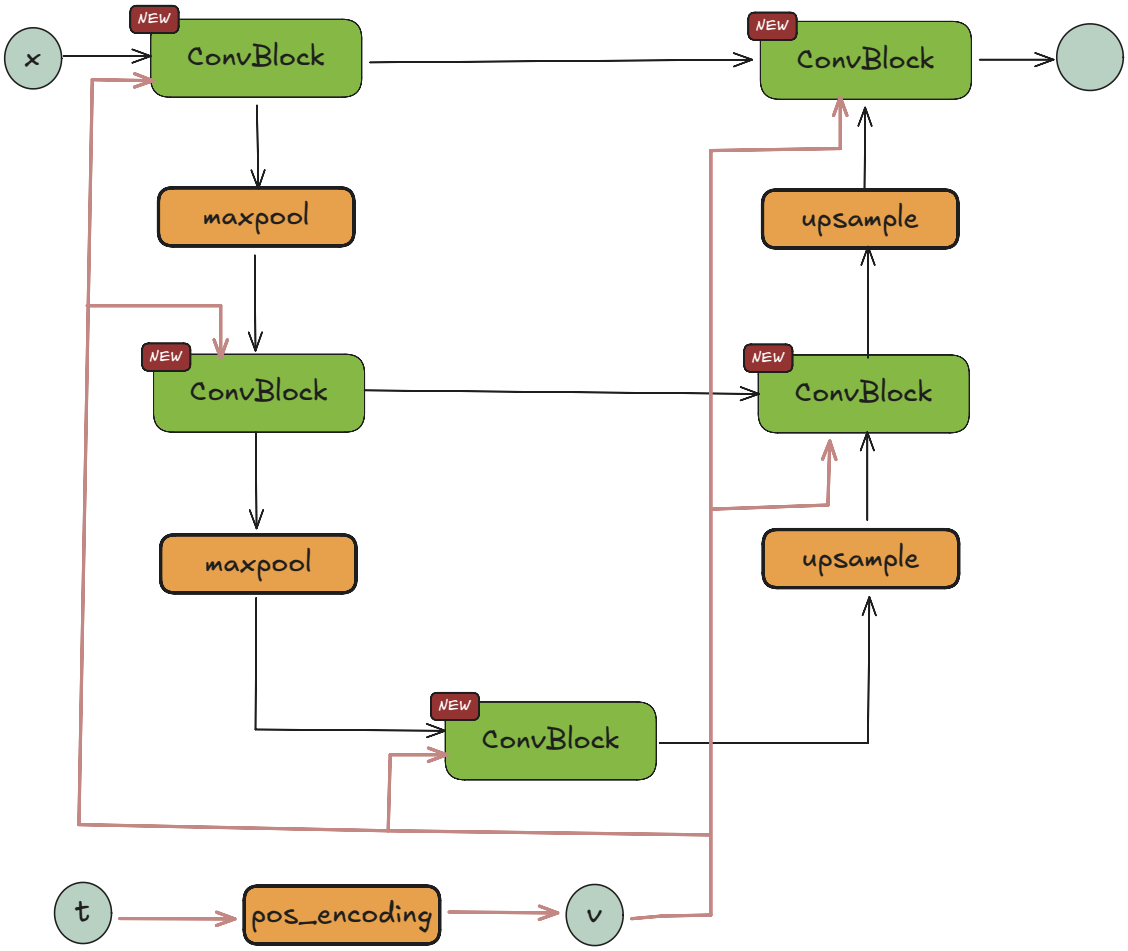


图 16.22 加入正弦位置编码的 U-Net

如图所示，新的 ConvBlock 中也将接收正弦位置编码信息 v 作为输入。然后在这个新的 ConvBlock 内部执行下图所示的处理。

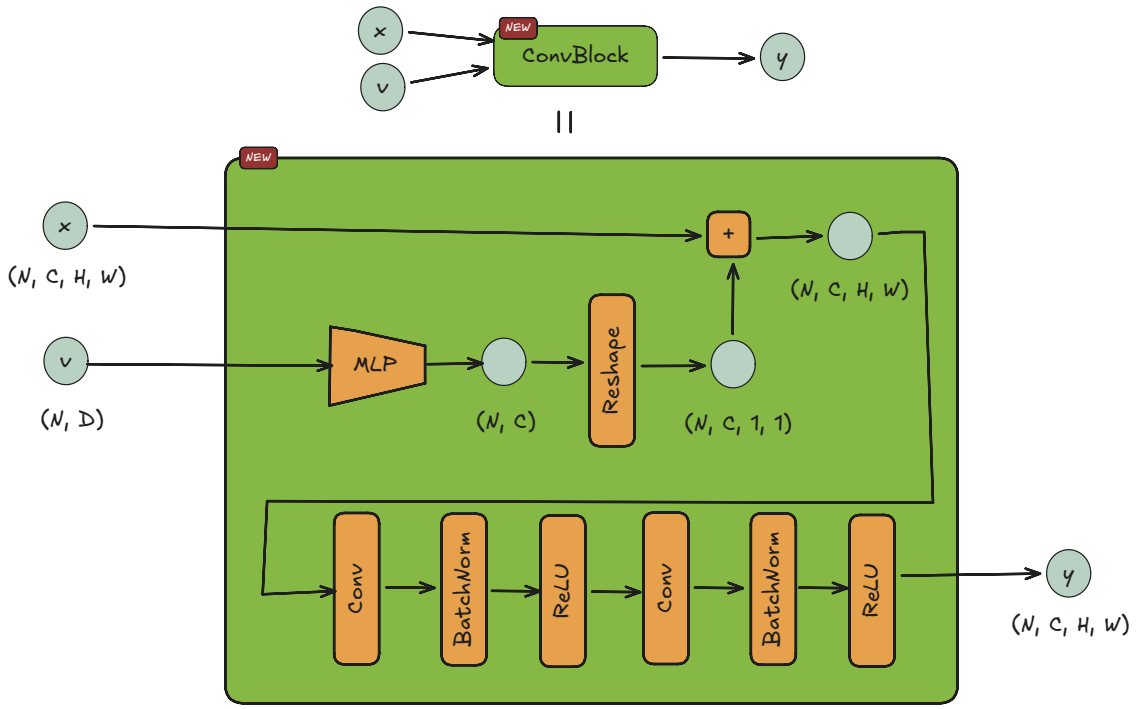


图 16.23 新的 ConvBlock 类

图中的 x 的形状为(N, C, H, W)，v 的形状为(N, D)。v 被 MLP（多层感知机，这里指由全连接层组成的神经网络）变换为(N, C)的形状，然后再被变换为(N, C, 1, 1)的形状。通过这个变形，广播函数得以应用于随后的加法运算中。新的 ConvBlock 的实现如下所示。

```python
class ConvBlock(nn.Module):
    def __init__(self, in_ch, out_ch, time_embed_dim):
        super().__init__()
        self.convs = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, 3, padding=1),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(),
            nn.Conv2d(out_ch, out_ch, 3, padding=1),
            nn.BatchNorm2d(out_ch),
            nn.ReLU()
        )
        self.mlp = nn.Sequential(
            nn.Linear(time_embed_dim, in_ch),
            nn.ReLU(),
            nn.Linear(in_ch, in_ch)
        )

    def forward(self, x, v):
        N, C, _, _ = x.shape
        v = self.mlp(v)
        v = v.view(N, C, 1, 1)
        y = self.convs(x + v)
        return y
```

参数 time_embed_dim 是通过正弦位置编码变换后的向量的维度。名为 self.mlp 的全连接层可以将维度为 time_embed_dim 的向量变换为维度为 in_ch 的向量。

最后使用这个新的 ConvBlock 类来实现 UNet 类。代码如下所示。

```python
class UNet(nn.Module):
    def __init__(self, in_ch=1, time_embed_dim=100):
        super().__init__()
        self.time_embed_dim = time_embed_dim

        self.down1 = ConvBlock(in_ch, 64, time_embed_dim)
        self.down2 = ConvBlock(64, 128, time_embed_dim)
        self.bot1 = ConvBlock(128, 256, time_embed_dim)
        self.up2 = ConvBlock(128 + 256, 128, time_embed_dim)
        self.up1 = ConvBlock(128 + 64, 64, time_embed_dim)
        self.out = nn.Conv2d(64, in_ch, 1)

        self.maxpool = nn.MaxPool2d(2)
        self.upsample = nn.Upsample(scale_factor=2, mode='bilinear')

    def forward(self, x, timesteps):
        v = pos_encoding(timesteps, self.time_embed_dim, x.device)

        x1 = self.down1(x, v)
        x = self.maxpool(x1)
        x2 = self.down2(x, v)
        x = self.maxpool(x2)

        x = self.bot1(x, v)

        x = self.upsample(x)
        x = torch.cat([x, x2], dim=1)
        x = self.up2(x, v)
        x = self.upsample(x)
        x = torch.cat([x, x1], dim=1)
        x = self.up1(x, v)
        x = self.out(x)
        return x
```

这样我们就完成了扩散模型使用的神经网络的实现。

16.4 完整代码

```python
import math
import torch
import torchvision
import matplotlib.pyplot as plt
from torchvision import transforms
from torch.utils.data import DataLoader
from torch.optim import Adam
import torch.nn.functional as F
from torch import nn
```

```
from tqdm import tqdm


img_size = 28
batch_size = 128
num_timesteps = 1000
epochs = 10
lr = 1e-3
device = "cuda"


def show_images(images, rows=2, cols=10):
    fig = plt.figure(figsize=(cols, rows))
    i = 0
    for r in range(rows):
        for c in range(cols):
            fig.add_subplot(rows, cols, i + 1)
            plt.imshow(images[i], cmap='gray')
            plt.axis('off')
            i += 1
    plt.show()

# 位置编码部分
def _pos_encoding(time_idx, output_dim, device='cpu'):
    t, D = time_idx, output_dim
    v = torch.zeros(D, device=device)

    i = torch.arange(0, D, device=device)
    div_term = torch.exp(i / D * math.log(10000))

    v[0::2] = torch.sin(t / div_term[0::2])
    v[1::2] = torch.cos(t / div_term[1::2])
    return v

def pos_encoding(timesteps, output_dim, device='cpu'):
    batch_size = len(timesteps)
    device = timesteps.device
    v = torch.zeros(batch_size, output_dim, device=device)
    for i in range(batch_size):
        v[i] = _pos_encoding(timesteps[i], output_dim, device)
    return v

class ConvBlock(nn.Module):
    def __init__(self, in_ch, out_ch, time_embed_dim):
        super().__init__()
        self.convs = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, 3, padding=1),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(),
            nn.Conv2d(out_ch, out_ch, 3, padding=1),
```

```
            nn.BatchNorm2d(out_ch),
            nn.ReLU()
        )
        self.mlp = nn.Sequential(
            nn.Linear(time_embed_dim, in_ch),
            nn.ReLU(),
            nn.Linear(in_ch, in_ch)
        )

    def forward(self, x, v):
        N, C, _, _ = x.shape
        v = self.mlp(v)
        v = v.view(N, C, 1, 1)
        y = self.convs(x + v)
        return y

class UNet(nn.Module):
    def __init__(self, in_ch=1, time_embed_dim=100):
        super().__init__()
        self.time_embed_dim = time_embed_dim

        self.down1 = ConvBlock(in_ch, 64, time_embed_dim)
        self.down2 = ConvBlock(64, 128, time_embed_dim)
        self.bot1 = ConvBlock(128, 256, time_embed_dim)
        self.up2 = ConvBlock(128 + 256, 128, time_embed_dim)
        self.up1 = ConvBlock(128 + 64, 64, time_embed_dim)
        self.out = nn.Conv2d(64, in_ch, 1)

        self.maxpool = nn.MaxPool2d(2)
        self.upsample = nn.Upsample(scale_factor=2, mode='bilinear')

    def forward(self, x, timesteps):
        v = pos_encoding(timesteps, self.time_embed_dim, x.device)

        x1 = self.down1(x, v)
        x = self.maxpool(x1)
        x2 = self.down2(x, v)
        x = self.maxpool(x2)

        x = self.bot1(x, v)

        x = self.upsample(x)
        x = torch.cat([x, x2], dim=1)
        x = self.up2(x, v)
        x = self.upsample(x)
        x = torch.cat([x, x1], dim=1)
        x = self.up1(x, v)
        x = self.out(x)
        return x

```

```
110
111 class Diffuser:
112     def __init__(self,
113         num_timesteps=1000, # 时间步T=1000
114         beta_start=0.0001, # 方差的起始值β_0 = 0.0001
115         beta_end=0.02, # 方差的最终值β_T = 0.02
116         device="cpu"
117     ):
118         self.num_timesteps = num_timesteps
119         self.device = device
120         # 方差调度计划
121         self.betas = torch.linspace(
122             beta_start,
123             beta_end,
124             num_timesteps,
125             device=device
126         )
127         # α_t = 1 − β_t
128         self.alphas = 1 - self.betas
129         # ᾱ_t = α_t α_{t−1} ··· α_1
130         self.alpha_bars = torch.cumprod(self.alphas, dim=0)
131
132     def add_noise(self, x_0, t):
133         T = self.num_timesteps
134         assert (t ⩾ 1).all() and (t ⩽ T).all()
135
136         t_idx = t - 1  # alpha_bars[0] is for t=1
137         alpha_bar = self.alpha_bars[t_idx]  # (N,)
138         N = alpha_bar.size(0)
139         alpha_bar = alpha_bar.view(N, 1, 1, 1)  # (N, 1, 1, 1)
140
141         noise = torch.randn_like(x_0, device=self.device)
142         x_t = torch.sqrt(alpha_bar) * x_0 \
143             + torch.sqrt(1 - alpha_bar) * noise
144         return x_t, noise
145
146     def denoise(self, model, x, t):
147         """去除一步噪声，x为时间步t的带噪声图片x_t"""
148         T = self.num_timesteps
149         assert (t ⩾ 1).all() and (t ⩽ T).all()
150
151         t_idx = t - 1  # alphas[0] is for t=1
152         alpha = self.alphas[t_idx]
153         alpha_bar = self.alpha_bars[t_idx]
154         alpha_bar_prev = self.alpha_bars[t_idx-1]
155
156         N = alpha.size(0)
157         alpha = alpha.view(N, 1, 1, 1)
158         alpha_bar = alpha_bar.view(N, 1, 1, 1)
159         alpha_bar_prev = alpha_bar_prev.view(N, 1, 1, 1)
```

```
160
161         model.eval()
162         with torch.no_grad():
163             eps = model(x, t)
164         model.train()
165
166         noise = torch.randn_like(x, device=self.device)
167         noise[t == 1] = 0  # no noise at t=1
168
169         mu = (x - ((1-alpha) / torch.sqrt(1-alpha_bar)) * eps) \
170            / torch.sqrt(alpha)
171         std = torch.sqrt(
172             (1-alpha) * (1-alpha_bar_prev) / (1-alpha_bar)
173         )
174         # x_{t-1}
175         return mu + noise * std
176
177     def reverse_to_img(self, x):
178         x = x * 255
179         x = x.clamp(0, 255)
180         x = x.to(torch.uint8)
181         x = x.cpu()
182         to_pil = transforms.ToPILImage()
183         return to_pil(x)
184
185     def sample(self, model, x_shape=(20, 1, 28, 28)):
186         """从纯噪声图片x_1000反向扩散出x_0"""
187         batch_size = x_shape[0]
188         # 采样一张白噪声图片x_1000出来
189         x = torch.randn(x_shape, device=self.device)
190         # for t = T, T-1, ..., 0
191         for i in tqdm(range(self.num_timesteps, 0, -1)):
192             t = torch.tensor(
193                 [i] * batch_size,
194                 device=self.device,
195                 dtype=torch.long
196             )
197             # 一步去噪，x_t → x_{t-1}
198             x = self.denoise(model, x, t)
199
200         images = [
201             self.reverse_to_img(x[i])
202             for i in range(batch_size)
203         ]
204         return images
205
206
207 preprocess = transforms.ToTensor()
208 dataset = torchvision.datasets.MNIST(
209     root="./../datasets",
```

```
    download=True,
    transform=preprocess
)
dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=True)

diffuser = Diffuser(num_timesteps, device=device)
model = UNet()
model.to(device)
optimizer = Adam(model.parameters(), lr=lr)

losses = []
for epoch in range(epochs):
    loss_sum = 0.0
    cnt = 0

    # 每个 epoch 都生成采样的图像 ======================
    images = diffuser.sample(model)
    show_images(images)
    # ================================================

    for images, labels in tqdm(dataloader):
        optimizer.zero_grad()
        x = images.to(device)
        # 随机取一个时间步
        t = torch.randint(1, num_timesteps+1, (len(x),), device=device)
        # x_noisy是$x_t$，noise是添加的真正的噪声
        x_noisy, noise = diffuser.add_noise(x, t)
        # 模型根据$x_t$和时间步$t$，预测给$x_t$添加的噪声
        noise_pred = model(x_noisy, t)
        # 添加的真实噪声和预测噪声之间进行均方误差计算
        loss = F.mse_loss(noise, noise_pred)

        loss.backward()
        optimizer.step()

        loss_sum += loss.item()
        cnt += 1

    loss_avg = loss_sum / cnt
    losses.append(loss_avg)
    print(f"Epoch {epoch} | Loss: {loss_avg}")

# 画出损失
plt.plot(losses)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.show()

# 从完全噪声的图片反向扩散
images = diffuser.sample(model)
```

```
260  show_images(images)
```

17. 扩散模型背后的数学理论

由于我们正在创建一个生成模型，我们希望找到真实图像的分布。

给定一个目标和未知的概率分布，可以构造一条马尔可夫链，用来近似该目标概率分布。执行近似时包含的步骤越多，样本的分布就越接近真实分布。

当当前决策仅取决于你所在的位置而非起点时，过程就是马尔可夫过程。存在概率转移矩阵，我们可以构建一个链结构，称为马尔可夫链。

$$
\begin{aligned}
&p(x) \propto e^{-f(x)} \Rightarrow \log p(x) = -f(x) + \text{常数} && (1)\\
&\therefore x_t = x_{t-1} - \eta_t \nabla f(x_{t-1}) && (2)\\
&x_{t+1} = x_t - \frac{\varepsilon}{2}\nabla f(x_t) + \sqrt{\varepsilon}\mathcal{N}(0, I) && (3)
\end{aligned}
\tag{17.1}
$$

图 17.1　郎之万动力学

郎之万动力学是目前最流行的 MCMC（马尔可夫蒙特卡洛采样）方法之一。如果我们有一个概率分布 $p(x)$ 且是马尔可夫分布，则它属于指数分布类。这在图 17.1(1)中给出，其中$f(x)$是某个使得该方程成立的函数。假设我们没有直接从该分布中采样的方法。假设我们只能知道函数在每一点的梯度$\nabla f(x)$。如果是这样，我们可以应用梯度下降来求出分布的模态。这由图 17.1(2)给出，其中η是时间步t的学习率。如果我们能找到分布的模态，那么我们就能找到密度中具有峰值的区域。由于在我们的训练数据中，会有很多“好”图像，这意味着去到峰值时，会得到这些好图像。郎之万采样是对该梯度下降算法的有趣修改，由图 17.1(3)给出。这里我们在梯度下降公式的每个步骤中添加一个噪声项。

一个问题是，如果想加快过程，可能会选择ε的值过高，这可能导致上述方程(3)不稳定并发散。所以，如果我们能提供某种形式的保证，保证更新规则会收敛，那就更好了。

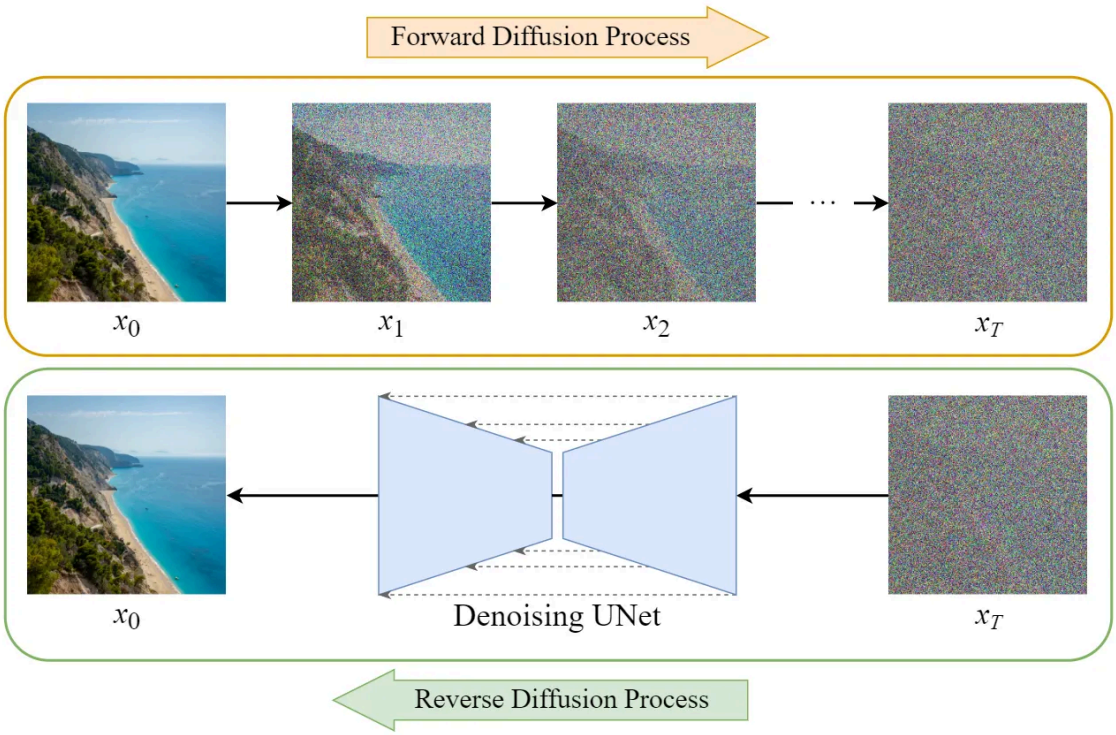


图 17.2 正向扩散和反向扩散

扩散模型的训练可以分为两部分：

1. 正向扩散过程 → 给图像添加噪声。
2. 反向扩散过程 → 从图像中去除噪声。

17.1 正向扩散过程

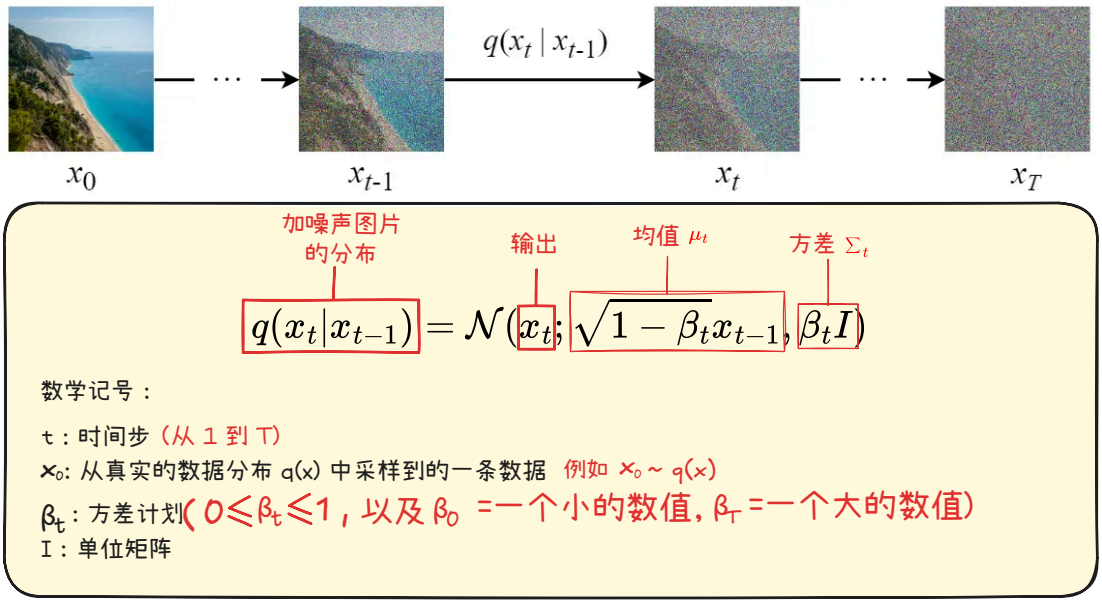


图 17.3 正向扩散公式图解

正向扩散过程逐步将高斯噪声添加到输入图像x_0中，总共会有T步。该过程将产生一系列带噪声的图像样本$x_1, ..., x_T$。

当$T \to \infty$时，最终结果将变成完全噪声图像，就像从**各向同性**的高斯分布中采样出来的噪声一样。

但我们不需要设计一种算法来迭代地向图像中添加噪声，而是可以使用闭式公式（解析解）在特定的时间步t直接对噪声图像进行采样。

前向扩散的闭式公式（解析解）

可以使用**重参数化技巧**推导出解析形式的采样公式。

> 🔥 重参数技巧
>
> 首先，如果 $z \sim \mathcal{N}(\mu, \sigma^2)$ 的话，那么有下面的结论
>
> $$z = \mu + \sigma\varepsilon \quad \text{其中 } \varepsilon \sim \mathcal{N}(0,1) \tag{17.2}$$
>
> 两个正态分布（独立）随机变量的和也是正态分布，也就是如果$X \sim \mathcal{N}(\mu_X, \sigma_X^2)$和$Y \sim \mathcal{N}(\mu_Y, \sigma_Y^2)$，那么对于$Z = X + Y$有$Z \sim \mathcal{N}(\mu_X + \mu_Y, \sigma_X^2 + \sigma_Y^2)$，也就是
>
> $$z = \mu_X + \mu_Y + \sqrt{\sigma_X^2 + \sigma_Y^2}\varepsilon \quad \text{其中 } \varepsilon \sim \mathcal{N}(0,1) \tag{17.3}$$

利用这个技巧，我们可以将采样图像x_t表示如下：

$$x_t = \sqrt{1-\beta_t}x_{t-1} + \sqrt{\beta_t}\varepsilon_{t-1} \tag{17.4}$$

然后我们可以递归地展开它来得到闭合形式的公式：

我们先来写一些前置结论

$$\begin{gathered}
\varepsilon_0, ..., \varepsilon_{t-2}, \varepsilon_{t-1} \sim \mathcal{N}(0, I) \\
\overline{\varepsilon}_0, ..., \overline{\varepsilon}_{t-2}, \overline{\varepsilon}_{t-1} \sim \mathcal{N}(0, I) \\
\varepsilon \sim \mathcal{N}(0, I) \\
\alpha_t = 1 - \beta_t \\
\overline{\alpha}_t = \prod_{i=1}^{t} \alpha_i
\end{gathered} \tag{17.5}$$

然后我们开始推导

$$
\begin{aligned}
x_t &= \sqrt{1-\beta_t}x_{t-1} + \sqrt{\beta_t}\varepsilon_{t-1} \\
&= \sqrt{\alpha_t}x_{t-1} + \sqrt{1-\alpha_t}\varepsilon_{t-1} \\
&\quad \text{一步加噪声公式} \\
&= \sqrt{\alpha_t}\left(\sqrt{\alpha_{t-1}}x_{t-2} + \sqrt{1-\alpha_{t-1}}\varepsilon_{t-2}\right) + \sqrt{1-\alpha_t}\varepsilon_{t-1} \\
&= \sqrt{\alpha_t\alpha_{t-1}}x_{t-2} + \sqrt{\alpha_t(1-\alpha_{t-1})}\varepsilon_{t-2} + \sqrt{1-\alpha_t}\varepsilon_{t-1} \\
&= \sqrt{\alpha_t\alpha_{t-1}}x_{t-2} + 0 + \sqrt{\alpha_t(1-\alpha_{t-1})}\varepsilon_{t-2} + 0 + \sqrt{1-\alpha_t}\varepsilon_{t-1} \\
&= \sqrt{\alpha_t\alpha_{t-1}}x_{t-2} + \mathcal{N}(0, \alpha_t(1-\alpha_{t-1})\boldsymbol{I}) + \mathcal{N}(0, (1-\alpha_t)\boldsymbol{I}) \\
&= \sqrt{\alpha_t\alpha_{t-1}}x_{t-2} + \mathcal{N}(0, (1-\alpha_t\alpha_{t-1})\boldsymbol{I}) \\
&= \sqrt{\alpha_t\alpha_{t-1}}x_{t-2} + \sqrt{1-\alpha_t\alpha_{t-1}}\bar{\varepsilon}_{t-2} \\
&\vdots \\
&= \sqrt{\alpha_t\alpha_{t-1}\cdots\alpha_1}x_0 + \sqrt{1-\alpha_t\alpha_{t-1}\cdots\alpha_1}\varepsilon \\
&= \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\varepsilon
\end{aligned} \tag{17.6}
$$

> 提示
>
> 所有ε都是 `i.i.d.`（独立同分布）标准正态随机变量。
>
> 使用不同的符号和下标来区分它们非常重要，因为它们是独立的，并且在采样后它们的值可能会有所不同。

重复这些步骤将为我们提供以下仅取决于输入图像x_0的公式：

$$
x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\varepsilon \tag{17.7}
$$

现在我们可以使用此公式在任何时间步骤直接对x_t进行采样，这使得前向过程更快。

17.2 逆向扩散过程

前向过程的联合概率分布是

$$
q(x_0x_1\cdots x_T) = q(x_0)q(x_1|x_0)\cdots q(x_T|x_{T-1}) \tag{17.8}
$$

反向过程的联合概率分布是

$$
q(x_0x_1\cdots x_T) = q(x_T)q(x_{T-1}|x_T)\cdots q(x_0|x_1) \tag{17.9}
$$

联合概率的乘法公式是

$$
q(x_t, x_{t-1}) = q(x_t|x_{t-1})q(x_{t-1}) = q(x_{t-1}|x_t)q(x_t) \tag{17.10}
$$

贝叶斯公式

$$
q(x_{t-1}|x_t) = \frac{q(x_t|x_{t-1})q(x_{t-1})}{q(x_t)} \tag{17.11}
$$

贝叶斯推断

$$
q(x_{t-1}|x_t) \propto q(x_t|x_{t-1})q(x_{t-1}) \tag{17.12}
$$

贝叶斯推断为什么要忽略掉分母$q(x_t)$呢？因为我们要计算的是，在**固定**x_t的情况下，求x_{t-1}的概率，所以$q(x_t)$是个常数。可以忽略掉。

现在的问题是，$q(x_t|x_{t-1})$我们已经知道公式了，但$q(x_{t-1})$我们不知道怎么计算。

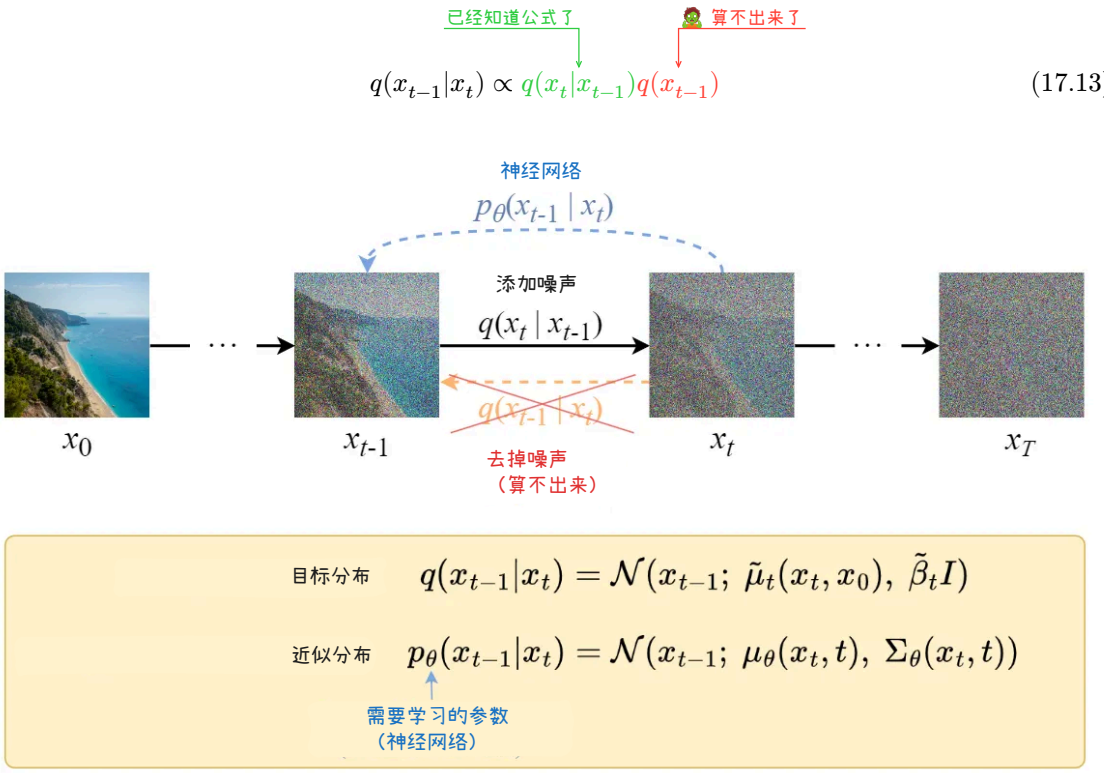


图 17.4 去噪的解析解无法计算

定理 17.1 — Feller. 如果前向过程的马尔可夫链的转移概率分布$q(x_t|x_{t-1})$是高斯分布，且β_t充分小，那么反向过程的马尔可夫链的转移概率分布$q(x_{t-1}|x_t)$也是高斯分布。

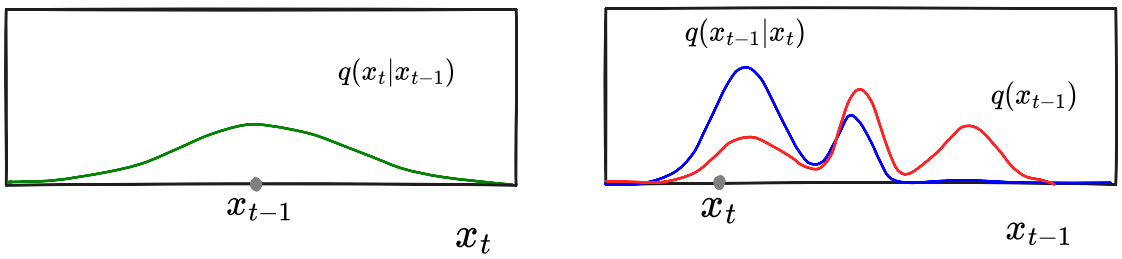


图 17.5 通过贝叶斯定理可以对反向分布$q(x_{t-1}|x_t)$进行推断。右图上的红色曲线显示了用 3 个高斯分布混合表示的边缘分布$q(x_{t-1})$，而左图显示了以x_{t-1}为中心的高斯前向噪声过程$q(x_t|x_{t-1})$作为x_t的分布。通过将它们相乘并进行归一化，我们得到了蓝色曲线所示的对于特定选择的x_t的分布$q(x_{t-1}|x_t)$。因为左图的分布相对较宽，对应着较大的方差β_t，所以分布$q(x_{t-1}|x_t)$具有复杂的多峰结构。

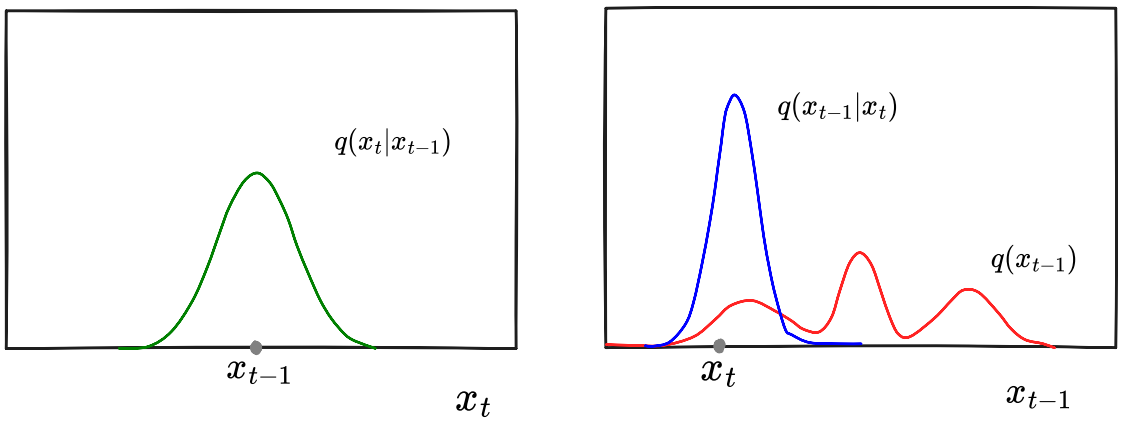


图 17.6 左图中的高斯分布$q(x_t|x_{t-1})$的方差β_t要小的多。我们可以看到右图中相应的分布$q(x_t|x_{t-1})$（蓝色）接近高斯分布，具有与$q(x_t|x_{t-1})$类似的方差

我们可以将边缘分布$q(x_{t-1})$写成以下形式

$$q(x_{t-1}) = \int q(x_{t-1}|\mathbf{x}_0)q(\mathbf{x}_0)\mathrm{d}\mathbf{x}_0 \tag{17.14}$$

其中 $q(x_{t-1}|\mathbf{x}_0)$ 可以由前向扩散的闭式解搞定。然而，上式的分布是难以处理的。因为我们必须对未知的数据密度$q(\mathbf{x}_0)$进行积分。

如果我们使用训练数据集的样本来近似积分，我们得到一个复杂的分布，表示为高斯混合分布。

> 提示
>
> 无论多么复杂的概率分布，都可以由多个高斯分布加权相加来拟合。概率分布的万能逼近定理。

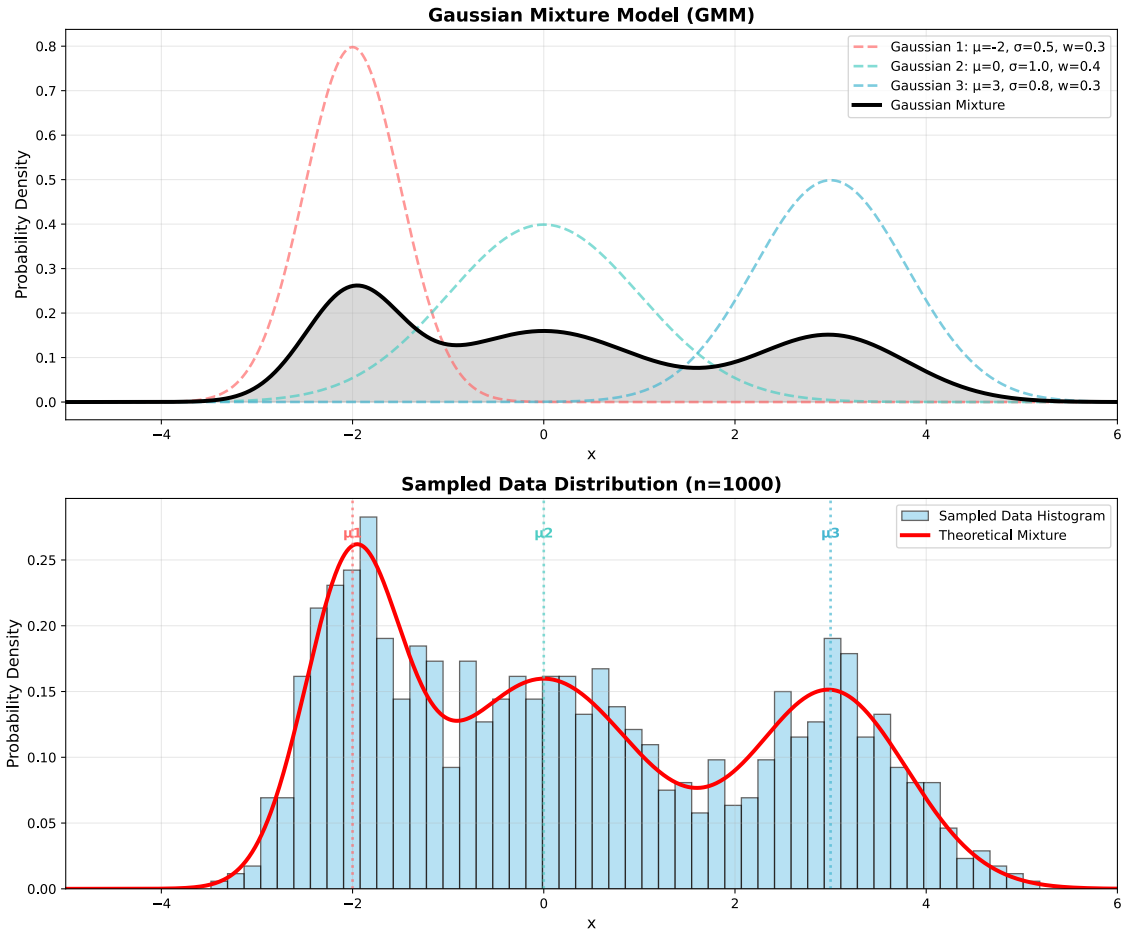


图 17.7 高斯混合模型（GMM）

相反，我们考虑条件分布的反向分布，条件是数据向量$\mathbf{x}_0$，定义为$q(x_{t-1}|x_t, \mathbf{x}_0)$，我们很快将看到它实际上是是一个简单的高斯分布。 直观上讲，这是合理的，因为对于给定的噪声图像，很难猜测是哪个更低噪声的图像产生了它，而如果我们还知道起始图像，那么问题就变得容易得多。

我们可以使用贝叶斯定理计算这个条件分布：

$$q(x_{t-1}|x_t, \mathbf{x}_0) = \frac{q(x_t|x_{t-1}, \mathbf{x}_0)q(x_{t-1}|\mathbf{x}_0)}{q(x_t|\mathbf{x}_0)} \tag{17.15}$$

现在我们利用前向过程的马尔可夫性质来写

$$q(x_t|x_{t-1}, \mathbf{x}_0) = q(x_t|x_{t-1}) \tag{17.16}$$

引理 **17.1** 第t步到第$t-1$步在第0步的条件下的转移概率分布$q(x_{t-1}|x_t,\mathbf{x}_0)$是以下高斯分布：

$$q(x_{t-1}|x_t,\mathbf{x}_0)=\mathcal{N}\left(x_{t-1};\tilde{\boldsymbol{\mu}}_t(x_t,\mathbf{x}_0),\tilde{\beta}_t\boldsymbol{I}\right) \tag{17.17}$$

其均值是

$$\tilde{\boldsymbol{\mu}}_t(x_t,\mathbf{x}_0)=\frac{\sqrt{\alpha_t}(1-\overline{\alpha}_{t-1})}{1-\overline{\alpha}_t}x_t+\frac{\sqrt{\overline{\alpha}_{t-1}}(1-\alpha_t)}{1-\overline{\alpha}_t}\mathbf{x}_0 \tag{17.18}$$

方差的系数是

$$\tilde{\beta}_t=\frac{1-\overline{\alpha}_{t-1}}{1-\overline{\alpha}_t}\beta_t=\frac{(1-\overline{\alpha}_{t-1})(1-\alpha_t)}{1-\overline{\alpha}_t} \tag{17.19}$$

证明 使用贝叶斯定理，利用前向过程的转移概率是高斯分布的性质，可以推导出反向过程的条件转移概率分布也是高斯分布，并得到其概率密度函数。

$$\begin{aligned}
&q(x_{t-1}|x_t,\mathbf{x}_0)\\
&=\frac{q(x_{t-1}|\mathbf{x}_0)q(x_t|x_{t-1},\mathbf{x}_0)}{q(x_t|\mathbf{x}_0)}\\
&=\frac{q(x_{t-1}|\mathbf{x}_0)q(x_t|x_{t-1})}{q(x_t|\mathbf{x}_0)}\\
&=\frac{\mathcal{N}\left(x_{t-1};\sqrt{\overline{\alpha}_{t-1}}\mathbf{x}_0,(1-\overline{\alpha}_{t-1})\boldsymbol{I}\right)\mathcal{N}\left(x_t;\sqrt{\alpha}_t x_{t-1},(1-\alpha_t)\boldsymbol{I}\right)}{\mathcal{N}\left(x_t;\sqrt{\overline{\alpha}_t}\mathbf{x}_0,(1-\overline{\alpha}_t)\boldsymbol{I}\right)}\\
&\propto\exp\left\{-\frac{1}{2}\left[\frac{\left(x_t-\sqrt{\alpha_t}x_{t-1}\right)^2}{\beta_t}+\frac{\left(x_{t-1}-\sqrt{1-\overline{\alpha}_{t-1}}\mathbf{x}_0\right)^2}{1-\overline{\alpha}_{t-1}}-\frac{\left(x_t-\sqrt{\overline{\alpha}_t}\mathbf{x}_0\right)^2}{1-\overline{\alpha}_t}\right]\right\}\\
&=\exp\left\{-\frac{1}{2}\left[\frac{x_t^2-2\sqrt{\alpha_t}x_t x_{t-1}+\alpha_t x_{t-1}^2}{\beta_t}+\frac{x_{t-1}^2-2\sqrt{\overline{\alpha}_{t-1}}x_{t-1}\mathbf{x}_0+\overline{\alpha}_{t-1}\mathbf{x}_0^2}{1-\overline{\alpha}_{t-1}}-\frac{\left(x_t-\sqrt{\overline{\alpha}_t}\mathbf{x}_0\right)^2}{1-\overline{\alpha}_t}\right]\right\}\\
&=\exp\left\{-\frac{1}{2}\left[\left(\frac{\alpha_t}{\beta_t}+\frac{1}{1-\overline{\alpha}_{t-1}}\right)x_{t-1}^2-2\left(\frac{\sqrt{\alpha_t}}{\beta_t}x_t+\frac{\sqrt{\overline{\alpha}_{t-1}}}{1-\overline{\alpha}_{t-1}}\mathbf{x}_0\right)x_{t-1}+C(x_t,\mathbf{x}_0)\right]\right\}
\end{aligned} \tag{17.20}$$

$C(x_t,\mathbf{x}_0)$是对于x_{t-1}的常数。高斯分布的方差系数是

$$\begin{aligned}
\tilde{\beta}_t&=\frac{1}{\frac{\alpha_t}{\beta_t}+\frac{1}{1-\overline{\alpha}_{t-1}}}\\
&=\frac{1-\overline{\alpha}_{t-1}}{1-\overline{\alpha}_t}\beta_t \qquad (\beta_t=1-\alpha_t)\\
&=\frac{(1-\overline{\alpha}_{t-1})(1-\alpha_t)}{1-\overline{\alpha}_t}
\end{aligned} \tag{17.21}$$

均值是

$$\begin{aligned}
\tilde{\mu}_t(x_t,\mathbf{x}_0)&=\frac{\frac{\sqrt{\alpha}_t}{\beta_t}x_t+\frac{\sqrt{\overline{\alpha}_{t-1}}}{1-\overline{\alpha}_{t-1}}\mathbf{x}_0}{\frac{\alpha_t}{\beta_t}+\frac{1}{1-\overline{\alpha}_{t-1}}}\\
&=\left(\frac{\sqrt{\alpha}_t}{\beta_t}x_t+\frac{\sqrt{\overline{\alpha}_{t-1}}}{1-\overline{\alpha}_{t-1}}\mathbf{x}_0\right)\frac{1-\overline{\alpha}_{t-1}}{1-\overline{\alpha}_t}(1-\alpha_t)\\
&=\frac{\sqrt{\alpha_t}(1-\overline{\alpha}_{t-1})}{1-\overline{\alpha}_t}x_t+\frac{\sqrt{\overline{\alpha}_{t-1}}(1-\alpha_t)}{1-\overline{\alpha}_t}\mathbf{x}_0
\end{aligned} \tag{17.22}$$

证明完毕

根据前向扩散的闭式解

$$x_t = \sqrt{\overline{\alpha}_t} x_0 + \sqrt{1-\overline{\alpha}_t}\varepsilon \tag{17.23}$$

可以得到

$$x_0 = \frac{1}{\sqrt{\overline{\alpha}_t}}\left(x_t - \sqrt{1-\overline{\alpha}_t}\varepsilon\right) \tag{17.24}$$

代入 $\tilde{\mu}_t(x_t, \mathbf{x}_0)$ 可以得到其只依赖 x_t 的形式：

$$\begin{aligned}\tilde{\mu}_t(x_t, \mathbf{x}_0) &= \frac{\sqrt{\alpha_t}(1-\overline{\alpha}_{t-1})}{1-\overline{\alpha}_t} x_t + \frac{\sqrt{\overline{\alpha}_{t-1}}(1-\alpha_t)}{1-\overline{\alpha}_t}\frac{1}{\sqrt{\overline{\alpha}_t}}\left(x_t - \sqrt{1-\overline{\alpha}_t}\varepsilon\right) \\ &= \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{1-\alpha_t}{\sqrt{1-\overline{\alpha}_t}}\varepsilon\right)\end{aligned} \tag{17.25}$$

假设前向过程和反向过程都是马尔可夫链，前向过程的转移概率分布是高斯分布。在此基础上可以得出，反向过程的转移概率分布也是高斯分布，并且能够求出第t步前向的转移概率分布$q(x_t|x_0)$和反向的条件转移概率分布$q(x_{t-1}|x_t, x_0)$。

前向过程表示一步步加噪的随机过程，由超参数$\beta_t(1, 2, ..., T)$控制，是事先确定的。反向过程表示一步步去噪的随机过程，由学习得到的神经网络控制。

DDPM用神经网络表示反向的转移概率分布$p_\theta(x_{t-1}|x_t), t = 1, 2, ..., T$。由定理可知，这个转移概率分布是高斯分布

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}, \boldsymbol{\mu}_\theta(x_t, t), \boldsymbol{\Sigma}_\theta(x_t, t)) \tag{17.26}$$

其均值和方差由神经网络决定。神经网络的输入是样本x_t和步数t，输出是均值$\boldsymbol{\mu}_\theta$和方差$\boldsymbol{\Sigma}_\theta$，参数是$\theta$。

反向过程的联合概率分布表示为

$$p(x_0 x_1 \cdots x_T) = p(x_T)\mathcal{N}(x_{T-1}; \boldsymbol{\mu}_\theta(x_T, T), \boldsymbol{\Sigma}_\theta(x_T, T)) \cdots \mathcal{N}(x_1; \boldsymbol{\mu}_\theta(x_1, 1), \boldsymbol{\Sigma}_\theta(x_1, 1)) \tag{17.27}$$

假设每一步的协方差矩阵是对角阵。

$$\boldsymbol{\Sigma}_\theta(x_t, t) = \sigma_t^2 \boldsymbol{I} \tag{17.28}$$

前向过程从第$t-1$步到第t步根据$q(x_t|x_{t-1})$采样，由x_{t-1}得到x_t。反向过程从第t步到第$t-1$步根据$p_\theta(x_{t-1}|x_t)$采样，由x_t得到x_{t-1}。原理上$p_\theta(x_{t-1}|x_t)$应该近似转移概率分布$q(x_{t-1}|x_t)$。但$q(x_{t-1}|x_t)$难以计算，DDPM 实际上使用$p_\theta(x_{t-1}|x_t)$近似条件转移概率分布$q(x_{t-1}|x_t, x_0)$。

我们的目的是希望$p_\theta(\mathbf{x}_0)$能够逼近理想中的那个$p(\mathbf{x}_0)$。

定理 **17.2** — **詹生不等式.** $\mathbb{E}_{q(x)}[\log f(x)] \leq \log \mathbb{E}_{q(x)}[f(x)]$

推导**证据下界**：ELBO。

ELBO

ELBO：Evidence Lower Bound，证据下界，变分下界

$$
\begin{aligned}
&\log p_\theta(x_0) \\
&= \log \int p_\theta(x_0 x_1 \cdots x_T) \mathrm{d}x_1 \mathrm{d}x_2 ... \mathrm{d}x_T \\
&= \log \int p_\theta(x_0 x_1 \cdots x_T) \frac{q(x_1 x_2 \cdots x_T | x_0)}{q(x_1 x_2 \cdots x_T | x_0)} \mathrm{d}x_1 \mathrm{d}x_2 ... \mathrm{d}x_T \\
&= \log \mathbb{E}_{q(x_1 x_2 \cdots x_T | x_0)} \left[\frac{p_\theta(x_0 x_1 \cdots x_T)}{q(x_1 x_2 \cdots x_T | x_0)} \right] \qquad \text{（显示为期望值）} \\
&\geq \mathbb{E}_{q(x_1 x_2 \cdots x_T | x_0)} \left[\log \frac{p_\theta(x_0 x_1 \cdots x_T)}{q(x_1 x_2 \cdots x_T | x_0)} \right] \qquad \text{（詹生不等式）}
\end{aligned}
\tag{17.29}
$$

那么就有

$$
\underbrace{\mathbb{E}_{q(x_0)} \log p_\theta(x_0)}_{\text{证据}} \geq \underbrace{\mathbb{E}_{q(x_0 x_1 x_2 \cdots x_T)} \left[\log \frac{p_\theta(x_0 x_1 \cdots x_T)}{q(x_1 x_2 \cdots x_T | x_0)} \right]}_{\text{证据下界}} \tag{17.30}
$$

直接最大化左边是不可行的，也就是不可求导，无法优化。所以 DDPM 通过最大化变分下界来优化目标。所以损失函数得到以下结果，其中用$p_\theta(x_{t-1}|x_t)$近似$q(x_{t-1}|x_t, x_0)$。

$$
\begin{aligned}
&L(\theta) \\
&= \mathbb{E}_{q(x_0 x_1 x_2 \cdots x_T)} \left[\log \frac{q(x_1 x_2 \cdots x_T | x_0)}{p_\theta(x_0 x_1 \cdots x_T)} \right] \\
&= \mathbb{E}_{q(x_0 x_1 x_2 \cdots x_T)} \left[\log \frac{\prod_{t=1}^T q(x_t | x_{t-1})}{p_\theta(x_T) \prod_{t=1}^T p_\theta(x_{t-1} | x_t)} \right] \\
&= \mathbb{E}_{q(x_0 x_1 x_2 \cdots x_T)} \left[-\log p_\theta(x_T) + \sum_{t=1}^T \log \frac{q(x_t | x_{t-1})}{p_\theta(x_{t-1} | x_t)} \right] \\
&= \mathbb{E}_{q(x_0 x_1 x_2 \cdots x_T)} \left[-\log p_\theta(x_T) + \sum_{t=2}^T \log \frac{q(x_t | x_{t-1})}{p_\theta(x_{t-1} | x_t)} + \log \frac{q(x_1 | x_0)}{p_\theta(x_0 | x_1)} \right] \\
&= \mathbb{E}_{q(x_0 x_1 x_2 \cdots x_T)} \left[-\log p_\theta(x_T) + \sum_{t=2}^T \log \frac{q(x_{t-1} | x_t, x_0)}{p_\theta(x_{t-1} | x_t)} + \sum_{t=2}^T \log \frac{q(x_t | x_0)}{q(x_{t-1} | x_0)} + \log \frac{q(x_1 | x_0)}{p_\theta(x_0 | x_1)} \right] \\
&= \mathbb{E}_{q(x_0 x_1 x_2 \cdots x_T)} \left[-\log p_\theta(x_T) + \sum_{t=2}^T \log \frac{q(x_{t-1} | x_t, x_0)}{p_\theta(x_{t-1} | x_t)} + \log \frac{q(x_T | x_0)}{q(x_1 | x_0)} + \log \frac{q(x_1 | x_0)}{p_\theta(x_0 | x_1)} \right] \\
&= \mathbb{E}_{q(x_0 x_1 x_2 \cdots x_T)} \left[\log \frac{q(x_T | x_0)}{p_\theta(x_T)} + \sum_{t=2}^T \log \frac{q(x_{t-1} | x_t, x_0)}{p_\theta(x_{t-1} | x_t)} - \log p_\theta(x_0 | x_1) \right] \\
&= \underbrace{\mathbb{E}_{q(x_0, x_T)} \left[\log \frac{q(x_T | x_0)}{p_\theta(x_T)} \right]}_{L_T} + \sum_{t=2}^T \underbrace{\mathbb{E}_{q(x_0, x_{t-1}, x_t)} \left[\log \frac{q(x_{t-1} | x_t, x_0)}{p_\theta(x_{t-1} | x_t)} \right]}_{L_{t-1}} - \underbrace{\mathbb{E}_{q(x_0, x_1)} [\log p_\theta(x_0 | x_1)]}_{L_0}
\end{aligned}
\tag{17.31}
$$

这里的分布 $q(x_{t-1}|x_t, x_0)$ 的定义如下：

$$
\begin{aligned}
q(x_{t-1} | x_t, x_0) &= \mathcal{N}\left(x_{t-1}; \tilde{\boldsymbol{\mu}}_t(x_t, x_0), \tilde{\beta}_t \boldsymbol{I}\right) \\
\tilde{\boldsymbol{\mu}}_t(x_t, x_0) &= \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \overline{\alpha}_t}} \varepsilon \right)
\end{aligned}
\tag{17.32}
$$

与之对应，分布$p_\theta(x_{t-1}|x_t)$的定义如下：

$$
\begin{aligned}
p_\theta(x_{t-1}|x_t) &= \mathcal{N}(x_{t-1};\tilde{\boldsymbol{\mu}}_\theta(x_t,t),\sigma_t^2\boldsymbol{I}) \\
\tilde{\boldsymbol{\mu}}_\theta(x_t,t) &= \frac{1}{\sqrt{\alpha_t}}\left(x_t-\frac{1-\alpha_t}{\sqrt{1-\overline{\alpha}_t}}\boldsymbol{\varepsilon}_\theta(x_t,t)\right)
\end{aligned}
\tag{17.33}
$$

假设两个分布$q(x_{t-1}|x_t,x_0)$和$p_\theta(x_{t-1}|x_t)$均值具有相同的形式：方差相同，即设$\tilde{\beta}_t=\sigma_t^2$。因此，神经网络简化为$\varepsilon_\theta(x_t,t)$，其输入是样本$x_t$和步数$t$，输出是噪声$\varepsilon$，$\theta$是参数。

DDPM 的学习实际通过简化的损失函数的最小化进行，训练一个预测噪声的神经网络。

🔥 KL 散度

$$
\begin{aligned}
D_{\mathrm{KL}}(P\|Q) &= \sum_{x\in\mathcal{X}}P(x)\log\frac{P(x)}{Q(x)} \\
&= \mathbb{E}_{x\sim P}\left[\log\frac{P(x)}{Q(x)}\right]
\end{aligned}
\tag{17.34}
$$

将损失函数展开，第一项损失 L_T 如下：

$$
L_T=\mathbb{E}_{q(x_0)}[\mathrm{KL}\ (q(x_T|x_0)\|p_\theta(x_T))] \tag{17.35}
$$

由于 q 没有可学习的参数θ，而 $p_\theta(x_T)$ 是完全的高斯噪声，因此该项在训练期间将是一个常数，因此可以忽略不计。

这个损失是常数，所以对最小化不起作用。

中间各项的损失 $L_{t-1}(t=T,T-1,\cdots,2)$ 如下。通过计算分布 $q(x_{t-1}|x_t,x_0)$ 和 $p_\theta(x_{t-1}|x_t)$ 的 KL 散度的期望得到，期望是针对分布 $q(x_0)$ 和 $p(\varepsilon)$ 的。

$$
\begin{aligned}
L_{t-1} &= \mathbb{E}_{q(x_0,x_{t-1},x_t)}\left[\log\frac{q(x_{t-1}|x_t,x_0)}{p_\theta(x_{t-1}|x_t)}\right] \\
&= \int q(x_{t-1},x_t|x_0)\log\frac{q(x_{t-1}|x_t,x_0)}{p_\theta(x_{t-1}|x_t)}\mathrm{d}x_{t-1}\mathrm{d}x_t \\
&= \int q(x_t|x_0)q(x_{t-1}|x_t,x_0)\log\frac{q(x_{t-1}|x_t,x_0)}{p_\theta(x_{t-1}|x_t)}\mathrm{d}x_{t-1}\mathrm{d}x_t \\
&= \int q(x_t|x_0)\underbrace{\int q(x_{t-1}|x_t,x_0)\log\frac{q(x_{t-1}|x_t,x_0)}{p_\theta(x_{t-1}|x_t)}\mathrm{d}x_{t-1}}_{\text{KL 散度}}\mathrm{d}x_t \\
&= \mathbb{E}_{q(x_t|x_0)}[D_{\mathrm{KL}}(q(x_{t-1}|x_t,x_0)\|p_\theta(x_{t-1}|x_t))] \\
&= \mathbb{E}_{q(x_t|x_0)}\left[\frac{1}{2\sigma^2(t)}\parallel\boldsymbol{\mu}(x_t,x_0)-\boldsymbol{\mu}_\theta(x_t,t)\parallel^2\right] \\
&= \mathbb{E}_{q(x_t|x_0)}\left[\frac{1}{2\sigma^2(t)}\left\|\frac{1}{\sqrt{\alpha_t}}\left[x_t-\frac{1-\alpha_t}{\sqrt{1-\overline{\alpha}_t}}\varepsilon\right]-\frac{1}{\sqrt{\alpha_t}}\left[x_t-\frac{1-\alpha_t}{\sqrt{1-\overline{\alpha}_t}}\varepsilon_\theta(x_t,t)\right]\right\|^2\right] \\
&= \mathbb{E}_{q(x_t|x_0)}\left[\frac{1}{2\sigma^2(t)}\frac{(1-\alpha_t)^2}{\alpha_t(1-\overline{\alpha}_t)}\|\varepsilon-\varepsilon_\theta(x_t,t)\|^2\right] \\
&= \mathbb{E}_{q(x_t|x_0)}\left[\frac{1}{2\sigma^2(t)}\frac{(1-\alpha_t)^2}{\alpha_t(1-\overline{\alpha}_t)}\left\|\varepsilon-\varepsilon_\theta\left(\sqrt{\overline{\alpha}_t}x_0+\sqrt{1-\overline{\alpha}_t}\varepsilon,t\right)\right\|^2\right]
\end{aligned}
\tag{17.36}
$$

最后一项损失 L_0 如下：

$$
L_0=\mathbb{E}_{q(x_1|x_0)}[-\log p_\theta(x_0|x_1)] \tag{17.37}
$$

我们想计算的是条件概率$p_\theta(x_0|x_1)$，其中：

- x_0是离散的图像数据（像素值离散，通常是$0 \sim 255$的整数值）。
- x_1是扩散过程中的一个中间变量（通常是连续变量，浮点数）。

$p_\theta(x_0|x_1)$本身是个连续的高斯分布，我们却想要求解某个离散的整数像素值x_0^i的概率（x_0^i是原始图片x_0的第i个像素点），直接用像素点取值，只能取到概率密度值，想要得到某个像素点的概率，需要积分一个特别小的区间。由于像素值$0 \sim 255$被归一化到了$[-1, 1]$区间。所以 DDPM 作者选择的积分区间是$(x_0^i - \frac{1}{255}, x_0^i + \frac{1}{255})$。

那么此时有

$$\begin{aligned} p_\theta(x_0|x_1) &= \prod_{i=1}^{D} \int_{x_0^i - \frac{1}{255}}^{x_0^i + \frac{1}{255}} \mathcal{N}\left(x; \mu_\theta^i(x_1, 1), \sigma_1^2\right) \mathrm{d}x \\ &\approx \prod_{i=1}^{D} \mathcal{N}\left(x; \mu_\theta^i(x_1, 1), \sigma_1^2\right) \times \frac{2}{255} \\ &= \prod_{i=1}^{D} \frac{1}{\sqrt{2\pi\sigma_1^2}} \exp\left(-\frac{\left(x_0^i - \mu_\theta^i(x_1, 1)\right)^2}{2\sigma_1^2}\right) \times \frac{2}{255} \end{aligned} \tag{17.38}$$

其中D是图片像素点的数量。

两边取对数可以得到

$$\log p(x_0|x_1) = -\frac{1}{2\sigma^2}\|x_0 - \mu_\theta(x_1, 1)\|^2 + C \tag{17.39}$$

其中C是常数，对优化（求导）没有影响。

针对损失 $L_{t-1}, t = T, T-1, \cdots, 2$ ，忽略系数，只对平方损失部分进行优化。针对损失 L_0 ，假设进行同样的平方损失优化。忽略损失 L_T ，这样得到以下简化的整体损失函数：

$$\begin{aligned} L'(\theta) &= \sum_{t=1}^{T} \mathbb{E}_{q(x_t|x_0)}\left[\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2\right] \\ &= \sum_{t=1}^{T} \mathbb{E}_{q(x_t|x_0)}\left[\left\|\varepsilon - \varepsilon_\theta\left(\sqrt{\overline{\alpha}_t}x_0 + \sqrt{1-\overline{\alpha}_t}\varepsilon, t\right)\right\|^2\right] \end{aligned} \tag{17.40}$$

神经网络$\varepsilon_\theta(x_t, t)$预测的是前向过程第$t$步的高斯噪声，其直观的解释是，这样的神经网络也能对反向过程的第t步$(t = 1, 2, \cdots, T)$进行有效的去噪。

反向过程的第t步到第$t-1$步的转移概率分布，可以利用学习得到的神经网络$\varepsilon_\theta(x_t, t)$计算。使用随机变量表示形式：

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{1-\alpha_t}{\sqrt{1-\overline{\alpha}_t}}\varepsilon_\theta(x_t, t)\right) + \sigma_t\varepsilon \tag{17.41}$$

上面的式子称为 DDPM 反向过程的迭代公式，用于数据生成。设每一步的方差系数与对应的前向过程的方差系数相同，$\sigma_t = \sqrt{\beta_t}$。

> 🔥 两个正态分布之间的 KL 散度
>
> 两个正态分布之间的 KL 散度可以通过解析的方式求得。如果$q(\boldsymbol{z}) = \mathcal{N}(\boldsymbol{z}; \boldsymbol{\mu}_1, \boldsymbol{\sigma}_1^2\boldsymbol{I}), p(\boldsymbol{z}) = \mathcal{N}(\boldsymbol{z}; \boldsymbol{\mu}_2, \boldsymbol{\sigma}_2^2\boldsymbol{I})$，那么 KL 散度可以表示为以下式子（式子中的$H$是$\boldsymbol{z}$的维度）。
>
> $$D_{\mathrm{KL}}(q \parallel p) = -\frac{1}{2}\sum_{h=1}^{H}\left(1 + \log\frac{\sigma_{1,h}^2}{\sigma_{2,h}^2} - \frac{\left(\mu_{1,h} - \mu_{2,h}\right)^2}{\sigma_{2,h}^2} - \frac{\sigma_{1,h}^2}{\sigma_{2,h}^2}\right) \tag{17.42}$$

期望值的线性性质和相关变量的期望值

① 期望的线性性质

期望值的线性性质是指有下面的等式成立。

$$\mathbb{E}_{p(x,y)}[x+y]=\mathbb{E}_{p(x)}[x]+\mathbb{E}_{p(y)}[y] \tag{17.43}$$

上式中的x和y是随机变量，假定它们相互关联（$p(x,y)\neq p(x)p(y)$）。上式成立的证明如下所示。

$$\begin{aligned}\mathbb{E}_{p(x,y)}[x+y]&=\iint(x+y)p(x,y)\mathrm{d}x\mathrm{d}y\\&=\iint xp(x,y)\mathrm{d}x\mathrm{d}y+\iint yp(x,y)\mathrm{d}x\mathrm{d}y\\&=\int x\underbrace{\left(\int p(x,y)\mathrm{d}y\right)}_{p(x)}\mathrm{d}x+\int y\underbrace{\left(\int p(x,y)\mathrm{d}x\right)}_{p(y)}\mathrm{d}y\\&=\int xp(x)\mathrm{d}x+\int yp(y)\mathrm{d}y\\&=\mathbb{E}_{p(x)}[x]+\mathbb{E}_{p(y)}[y]\end{aligned} \tag{17.44}$$

这里展示的等式的意思是“和的期望值”等于“期望值的和”。这一关系也可以扩展到T个随机变量，因此有下面的式子成立。

$$\mathbb{E}_{p(x_1x_2\cdots x_T)}\left[\sum_{t=1}^{T}x_t\right]=\sum_{t=1}^{T}\mathbb{E}_{p(x_1x_2\cdots x_T)}[x_t] \tag{17.45}$$

② 相关变量的期望值

这里假设$f(x)$是以x为参数的任意函数。因此，有下式成立。

$$\mathbb{E}_{p(x,y)}[f(x)]=\mathbb{E}_{p(x)}[f(x)] \tag{17.46}$$

证明过程如下所示。

$$\begin{aligned}\mathbb{E}_{p(x,y)}[f(x)]&=\iint f(x)p(x,y)\mathrm{d}x\mathrm{d}y\\&=\iint f(x)p(x)p(y|x)\mathrm{d}x\mathrm{d}y\\&=\int f(x)p(x)\underbrace{\int p(y|x)\mathrm{d}y}_{=1}\mathrm{d}x\\&=\int f(x)p(x)\mathrm{d}x\\&=\mathbb{E}_{p(x)}[f(x)]\end{aligned} \tag{17.47}$$

“在$p(x,y)$分布下$f(x)$的期望值”等同于“在$p(x)$分布下$f(x)$的期望值”。重点在于概率分布中与期望值内部的对象（$f(x)$）内容无关的随机变量可以被“消除”。因此，有下面的等式成立。

$$\mathbb{E}_{p(x_1x_2\cdots x_T)}[f(x_t)]=\mathbb{E}_{p(x_t)}[f(x_t)] \tag{17.48}$$

另外，$f(x_{t-1},x_t)$的期望值如下所示。

$$\mathbb{E}_{p(x_1x_2\cdots x_T)}[f(x_{t-1},x_t)]=\mathbb{E}_{p(x_{t-1},x_t)}[f(x_{t-1},x_t)] \tag{17.49}$$

18. 条件扩散模型

我们之前对数据x的概率$p(x)$进行了建模。但在实用层面，我们更希望对条件概率$p(x|y)$进行建模（其中y表示条件）。如果能成功建立$p(x|y)$的模型，那么就可以通过条件y控制想生成的x。

条件y可以是文本、图像或标签等。如果y是一张低分辨率的图像，那么可以考虑将其变换为高分辨率的图像。这就是被称为超分辨率成像（`super-resolution imaging`）的技术。使用扩散模型进行超分辨率处理的模型包括级联扩散模型（`cascaded diffusion model`）等。

本节将以y作为标签的例子进行说明。具体来说，我们将创建一个模型，在给定`MNIST`数据的数字标签后，该模型能够生成与该标签相对应的图像。

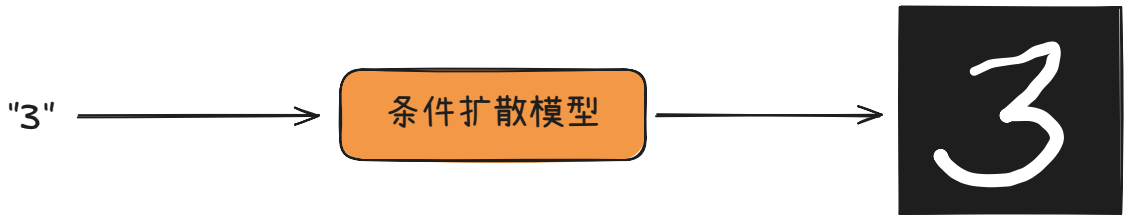


图 18.1　本章将要实现的条件扩散模型

18.1 向扩散模型添加条件

首先，我们从复习开始。在扩散模型中，我们可以选择在何处使用神经网络。例如，可以考虑使用神经网络对$\boldsymbol{\mu}_\theta(x_t, t)$和$\boldsymbol{\epsilon}_\theta(x_t, t)$进行建模。

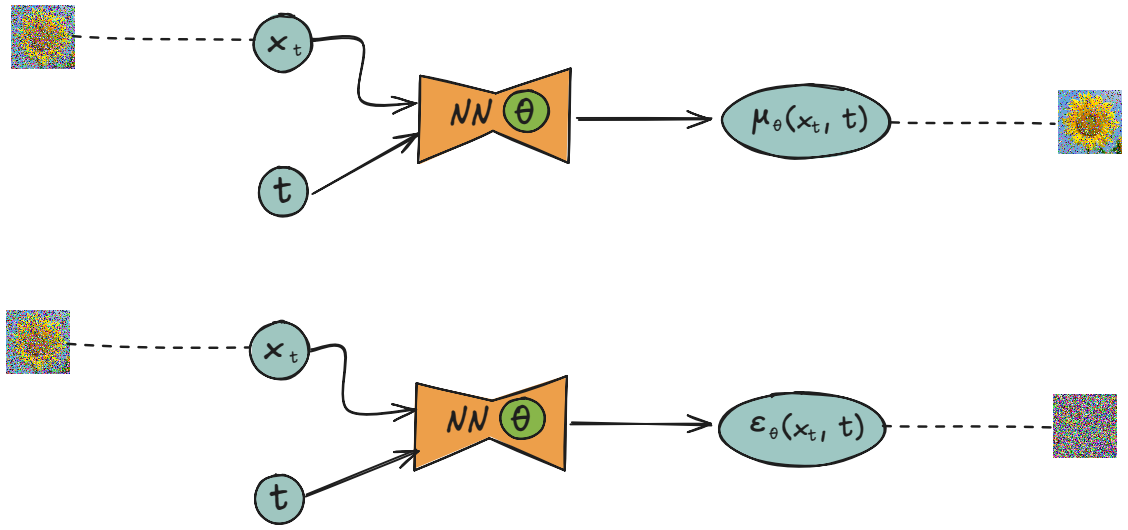


图 18.2 预测时刻$t-1$的图像（正态分布的均值向量）的横型（上）和预测添加到x_t中的噪声的模型（下）

下面思考使用神经网络对$\boldsymbol{\mu}_\theta(x_t,t)$建模的情况。在这种情祝下，$p_\theta(x_{t-1}|x_t)$的数学式如下所示。

$$p_\theta(x_{t-1}|x_t)=\mathcal{N}\left(x_{t-1};\boldsymbol{\mu}(x_t,t),\sigma_q^2(t)\boldsymbol{I}\right) \tag{18.1}$$

数据x_0的概率$p_\theta(x_0)$的数学式如下所示：

$$\begin{aligned} p_\theta(x_0) &= \int p_\theta(x_0,x_1,\cdots,x_T)\mathrm{d}\ x_1\cdots\ \mathrm{d}\ x_T && \text{（概率的边际化）} \\ &= \int p_\theta(x_0|x_1)\cdots p_\theta(x_{T-1}|x_T)p(x_T)\ \mathrm{d}\ x_1\cdots\ \mathrm{d}\ x_T && \text{（马尔可夫性质）} \end{aligned} \tag{18.2}$$

这里有 $p(x_T)=\mathcal{N}(x_T;\mathbf{0},\boldsymbol{I})$。

为了得到条件扩散模型，我们要建模的对象是$p_\theta(x_0|y)$，而不是$p_\theta(x_0)$。实现这个目标的最简单的方法是在每个时刻的$p_\theta(x_{t-1}|x_t)$中添加条件y。具体的数学式如下所示：

$$p_\theta(x_0|y)=\int p_\theta(x_0|x_1,y)\cdots p_\theta(x_{T-1}|x_T,y)p(x_T)\mathrm{d}x_1\mathrm{d}x_T \tag{18.3}$$

此时的 $p_\theta(x_{t-1}|x_t,y)$ 的数学式如下所示：

$$p_\theta(x_{t-1}|x_t,y)=\mathcal{N}\left(x_{t-1};\boldsymbol{\mu}_\theta(x_t,t,y),\sigma_q^2(t)\boldsymbol{I}\right) \tag{18.4}$$

在常规的扩散模型中，$\boldsymbol{\mu}_\theta(x_t,t)$通常由神经网络进行建模。而在条件扩散模型中，如$\boldsymbol{\mu}_\theta(x_t,t,y)$所示，会额外添加参数$y$。换言之，通过向神经网络中添加$y$，可以使模型“进化”为条件扩散模型。同样，对于预测噪声的神经网络，也可以通过将参数y添加到$\varepsilon_\theta(x_t,t)$中，使其变为$\varepsilon_\theta(x_t,t,y)$来实现相应的功能（参见 图 18.3 ）。

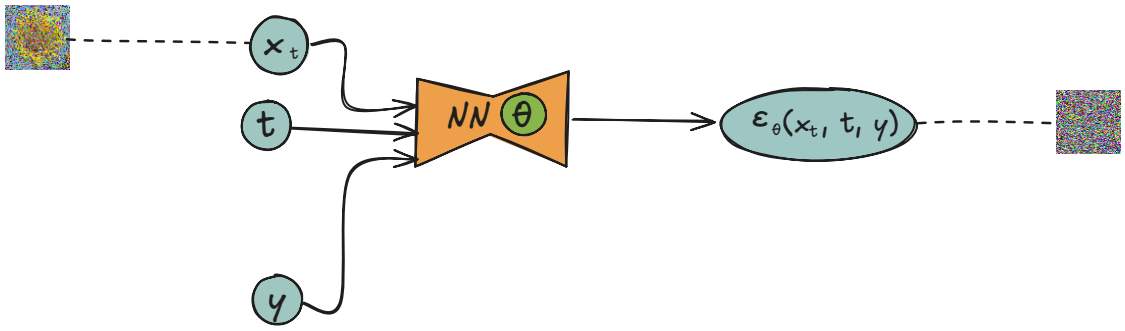


图 18.3 条件扩散模型的神经网络

18.2 条件扩散模型的实现

我们已经使用神经网络实现了$\varepsilon_\theta(x_t, t)$。$\varepsilon_\theta(x_t, t)$的参数$t$是整数，可以通过正弦位置编码变换为向量。而新添加的条件y是标签，也是整数。这个整数y可以通过嵌入层（`embedding layer`）变换为向量。具体来说，y被变换成向量，然后与变换后的t向量相加。

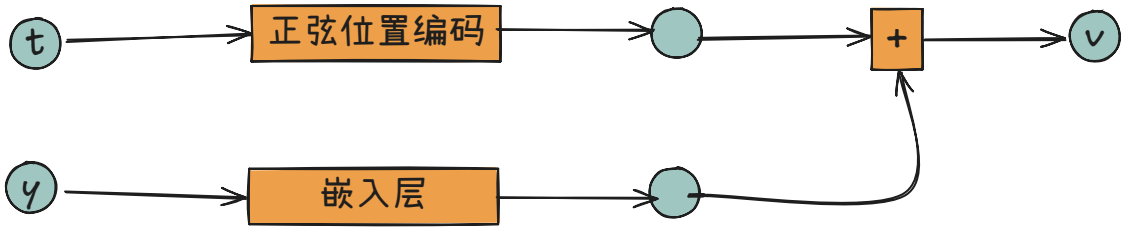


图 18.4 在扩散模型中添加嵌入层

> 提示
>
> 嵌入层的初始值被设置为随机值，然后通过训练进行优化。这样就可以训练得到与每个标签相对应的匹配任务的向量。而由于与时刻 t 相关的特定任务的训练要素很少，因此我们使用了一种称为正弦位置编码的固定向量变换的方法。

以下是代码示例。这里我们在上一章中实现的`UNet`类中添加了名为`UNetCond`类的代码，类名中的`Cond`是`Conditional`（条件）的缩写。

```python
class UNetCond(nn.Module):
    def __init__(self, in_ch=1, time_embed_dim=100, num_labels=None):
        super().__init__()
        self.time_embed_dim = time_embed_dim

        self.down1 = ConvBlock(in_ch, 64, time_embed_dim)
        self.down2 = ConvBlock(64, 128, time_embed_dim)
        self.bot1 = ConvBlock(128, 256, time_embed_dim)
        self.up2 = ConvBlock(128 + 256, 128, time_embed_dim)
        self.up1 = ConvBlock(128 + 64, 64, time_embed_dim)
        self.out = nn.Conv2d(64, in_ch, 1)

        self.maxpool = nn.MaxPool2d(2)
```

```
        self.upsample = nn.Upsample(scale_factor=2, mode="bilinear")

        # ❶ 处理标签的嵌入层
        if num_labels is not None:
            self.label_emb = nn.Embedding(num_labels, time_embed_dim)

    def forward(self, x, timesteps, labels=None):
        t = pos_encoding(timesteps, self.time_embed_dim)

        # ❷ 标签的处理
        if labels is not None:
            t += self.label_emb(labels)

        x1 = self.down1(x, t)
        x = self.maxpool(x1)
        x2 = self.down2(x, t)
        x = self.maxpool(x2)

        x = self.bot1(x, t)

        x = self.upsample(x)
        x = torch.cat([x, x2], dim=1)
        x = self.up2(x, t)
        x = self.upsample(x)
        x = torch.cat([x, x1], dim=1)
        x = self.up1(x, t)
        x = self.out(x)
        return x
```

代码的主要修改有两处。在代码❶处，我们准备了处理标签的嵌入层（`nn.Embedding`）。具体来说，就是通过`nn.Embedding(num_labels, time_embed_dim)`将共计`num_labels`个不同整数变换为`time_embed_dim`维的向量。在代码❷处，通过`nn.Embedding`层处理标签，然后再加到变换后的时间数据的向量中。

接下来是`Diffuser`类。对于`Diffuser`类，修改生成数据的方法。下面只展示了修改部分的代码。

Python

```
class Diffuser:
    def denoise(self, model, x, t, labels):
        # ...（省略）
        with torch.no_grad():
            eps = model(x, t, labels) # 同时提供labels
            # ...

    def sample(self, model, x_shape=(20, 1, 28, 28), labels=None):
        #...
        if labels is None:
            labels = torch.randint(0, 10, (len(x),), device=self.device)
        #...
        for i in tqdm(range(self.num_timesteps, 0, -1)):
            t = torch.tensor([i] * batch_size, device=self.device,
                             dtype=torch.long)
            x = self.denoise(model, x, t, labels) # 还提供labels
```

```
        #...
        return images, labels
```

修改的部分还向模型提供了标签。具体来说，标签是在生成数据和去噪过程中提供的。最后是训练的代码。

```python
diffuser = Diffuser(num_timesteps, device=device)
model = UNetCond(num_labels=10) ❶
model.to(device)
optimizer = Adam(model.parameters(), lr=lr)

losses = []
for epoch in range(epochs):
    loss_sum = 0.0
    cnt = 0

    for images, labels in tqdm(dataloader):
        optimizer.zero_grad()
        x = images.to(device) ❷
        t = torch.randint(1, num_timesteps+1, (len(x),), device=device)

        x_noisy, noise = diffuser.add_noise(x, t)
        noise_pred = model(x_noisy, t, labels) ❸
        loss = F.mse_loss(noise, noise_pred)

        loss.backward()
        optimizer.step()

        loss_sum += loss.item()
        cnt += 1
```

训练代码的修改内容如下所示。

❶ `UNetCond(num_labels=10)`

创建一个具有 10 个分类的条件扩散模型。

❷ `labels.to(device)`

在 `device` 上准备标签数据。

❸ `model(x_noisy, t, labels)`

向模型提供 `labels` 进行训练。

以上就是条件扩散模型的实现。现在运行代码。经过 10 轮训练后，最终生成了下图所示的图像。

虽然仍有改进的余地，但生成的图像是符合条件的。在下一节中，我们将探讨进一步改进这个条件扩散模型的方法。

完整代码如下：

```python
import math
import torch
import torchvision
import matplotlib.pyplot as plt
from torchvision import transforms
from torch.utils.data import DataLoader
from torch.optim import Adam
import torch.nn.functional as F
from torch import nn
```

```
from tqdm import tqdm


img_size = 28
batch_size = 128
num_timesteps = 1000
epochs = 10
lr = 1e-3
device = "cuda" if torch.cuda.is_available() else "cpu"


def show_images(images, labels=None, rows=2, cols=10):
    fig = plt.figure(figsize=(cols, rows))
    i = 0
    for r in range(rows):
        for c in range(cols):
            ax = fig.add_subplot(rows, cols, i + 1)
            plt.imshow(images[i], cmap='gray')
            if labels is not None:
                ax.set_xlabel(labels[i].item())
            ax.get_xaxis().set_ticks([])
            ax.get_yaxis().set_ticks([])
            i += 1
    plt.tight_layout()
    plt.show()

def _pos_encoding(time_idx, output_dim, device="cpu"):
    t, D = time_idx, output_dim
    v = torch.zeros(D, device=device)

    i = torch.arange(0, D, device=device)
    div_term = torch.exp(i / D * math.log(10000))

    v[0::2] = torch.sin(t / div_term[0::2])
    v[1::2] = torch.cos(t / div_term[1::2])
    return v

def pos_encoding(timesteps, output_dim, device='cpu'):
    batch_size = len(timesteps)
    device = timesteps.device
    v = torch.zeros(batch_size, output_dim, device=device)
    for i in range(batch_size):
        v[i] = _pos_encoding(timesteps[i], output_dim, device)
    return v

class ConvBlock(nn.Module):
    def __init__(self, in_ch, out_ch, time_embed_dim):
        super().__init__()
        self.convs = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, 3, padding=1),
```

```
            nn.BatchNorm2d(out_ch),
            nn.ReLU(),
            nn.Conv2d(out_ch, out_ch, 3, padding=1),
            nn.BatchNorm2d(out_ch),
            nn.ReLU()
        )
        self.mlp = nn.Sequential(
            nn.Linear(time_embed_dim, in_ch),
            nn.ReLU(),
            nn.Linear(in_ch, in_ch)
        )

    def forward(self, x, v):
        N, C, _, _ = x.shape
        v = self.mlp(v)
        v = v.view(N, C, 1, 1)
        y = self.convs(x + v)
        return y

class UNetCond(nn.Module):
    def __init__(self, in_ch=1, time_embed_dim=100, num_labels=None):
        super().__init__()
        self.time_embed_dim = time_embed_dim

        self.down1 = ConvBlock(in_ch, 64, time_embed_dim)
        self.down2 = ConvBlock(64, 128, time_embed_dim)
        self.bot1 = ConvBlock(128, 256, time_embed_dim)
        self.up2 = ConvBlock(128 + 256, 128, time_embed_dim)
        self.up1 = ConvBlock(128 + 64, 64, time_embed_dim)
        self.out = nn.Conv2d(64, in_ch, 1)

        self.maxpool = nn.MaxPool2d(2)
        self.upsample = nn.Upsample(scale_factor=2, mode='bilinear')
        # 如果条件标签不为None
        if num_labels is not None:
            # 将标签转换为嵌入，嵌入和时间步的嵌入形状相同
            self.label_emb = nn.Embedding(num_labels, time_embed_dim)

    def forward(self, x, timesteps, labels=None):
        t = pos_encoding(timesteps, self.time_embed_dim)
        # 如果标签不为空，将标签转换为嵌入，然后加到时间步的嵌入上面。
        if labels is not None:
            t += self.label_emb(labels)

        x1 = self.down1(x, t)
        x = self.maxpool(x1)
        x2 = self.down2(x, t)
        x = self.maxpool(x2)

        x = self.bot1(x, t)
```

```
        x = self.upsample(x)
        x = torch.cat([x, x2], dim=1)
        x = self.up2(x, t)
        x = self.upsample(x)
        x = torch.cat([x, x1], dim=1)
        x = self.up1(x, t)
        x = self.out(x)
        return x


class Diffuser:
    def __init__(
        self,
        num_timesteps=1000,
        beta_start=0.0001,
        beta_end=0.02,
        device='cpu'
    ):
        self.num_timesteps = num_timesteps
        self.device = device
        self.betas = torch.linspace(
            beta_start,
            beta_end,
            num_timesteps,
            device=device
        )
        self.alphas = 1 - self.betas
        self.alpha_bars = torch.cumprod(self.alphas, dim=0)

    def add_noise(self, x_0, t):
        T = self.num_timesteps
        assert (t ≥ 1).all() and (t ⩽ T).all()

        t_idx = t - 1  # alpha_bars[0] is for t=1
        alpha_bar = self.alpha_bars[t_idx]  # (N,)
        alpha_bar = alpha_bar.view(
            alpha_bar.size(0), 1, 1, 1)  # (N, 1, 1, 1)

        noise = torch.randn_like(x_0, device=self.device)
        x_t = torch.sqrt(alpha_bar) * x_0 \
            + torch.sqrt(1 - alpha_bar) * noise
        return x_t, noise

    def denoise(self, model, x, t, labels):
        T = self.num_timesteps
        assert (t ≥ 1).all() and (t ⩽ T).all()

        t_idx = t - 1  # alphas[0] is for t=1
        alpha = self.alphas[t_idx]
```

```
        alpha_bar = self.alpha_bars[t_idx]
        alpha_bar_prev = self.alpha_bars[t_idx-1]

        N = alpha.size(0)
        alpha = alpha.view(N, 1, 1, 1)
        alpha_bar = alpha_bar.view(N, 1, 1, 1)
        alpha_bar_prev = alpha_bar_prev.view(N, 1, 1, 1)

        model.eval()
        with torch.no_grad():
            eps = model(x, t, labels)  # 添加了标签的嵌入
        model.train()

        noise = torch.randn_like(x, device=self.device)
        noise[t == 1] = 0  # no noise at t=1

        mu = (x - ((1-alpha) / torch.sqrt(1-alpha_bar)) * eps) \
           / torch.sqrt(alpha)
        std = torch.sqrt((1-alpha) * (1-alpha_bar_prev) / (1-alpha_bar))
        return mu + noise * std

    def reverse_to_img(self, x):
        x = x * 255
        x = x.clamp(0, 255)
        x = x.to(torch.uint8)
        x = x.cpu()
        to_pil = transforms.ToPILImage()
        return to_pil(x)

    def sample(self, model, x_shape=(20, 1, 28, 28), labels=None):
        batch_size = x_shape[0]
        x = torch.randn(x_shape, device=self.device)
        if labels is None:
            labels = torch.randint(0, 10, (len(x),), device=self.device)

        for i in tqdm(range(self.num_timesteps, 0, -1)):
            t = torch.tensor(
                [i] * batch_size,
                device=self.device,
                dtype=torch.long
            )
            # 带着标签去噪
            x = self.denoise(model, x, t, labels)

        images = [self.reverse_to_img(x[i]) for i in range(batch_size)]
        return images, labels


preprocess = transforms.ToTensor()
dataset = torchvision.datasets.MNIST(
```

```
    root='./../datasets', download=True, transform=preprocess)
dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=True)

diffuser = Diffuser(num_timesteps, device=device)
model = UNetCond(num_labels=10)
model.to(device)
optimizer = Adam(model.parameters(), lr=lr)

losses = []
for epoch in range(epochs):
    loss_sum = 0.0
    cnt = 0

    # generate samples every epoch ====================
    images, labels = diffuser.sample(model)
    show_images(images, labels)
    # ================================================

    for images, labels in tqdm(dataloader):
        optimizer.zero_grad()
        x = images.to(device)
        # 使用标签数据
        labels = labels.to(device)
        t = torch.randint(1, num_timesteps+1, (len(x),), device=device)

        x_noisy, noise = diffuser.add_noise(x, t)
        # 带着标签预测噪声
        noise_pred = model(x_noisy, t, labels)
        loss = F.mse_loss(noise, noise_pred)

        loss.backward()
        optimizer.step()

        loss_sum += loss.item()
        cnt += 1

    loss_avg = loss_sum / cnt
    losses.append(loss_avg)
    print(f'Epoch {epoch} | Loss: {loss_avg}')

# plot losses
plt.plot(losses)
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.show()

# generate samples
images, labels = diffuser.sample(model)
show_images(images, labels)
```

18.3 得分函数

在上面的内容里面，我们实现了一个简单的条件扩散模型。这个“简单”的意思是我们只是向模型提供了条件而已。因此，模型有可能会轻视我们提供的条件，甚至在最坏的情况下有可能会忽略我们提供的条件。接下来，我们将介绍一种称为“指引”的方法。指引是一种将给定条件纳入模型并给予更多重视的机制。

要理解指引机制，首先需要了解得分函数。

18.3.1 什么是得分函数

截至目前，本章讨论的去噪扩散模型与另一类相对独立发展的深度生成式模型密切相关，它们都基于得分匹配（`score matching`）。这些模型利用的得分函数（或斯坦因得分）则定义为对数似然函数关于数据向量$\boldsymbol{x}$的梯度。

$$s(x) = \nabla_x \log p(x) \tag{18.5}$$

这里需要强调的是，梯度是针对**数据向量**计算的，而不是针对任何参数向量。注意$s(x)$是一个与x维度相同的向量值函数，并且其中的每个元素$s_i(x) = \frac{\partial \log p(x)}{\partial x_i}$都与$x$的相应元素$x_i$相关联。例如，如果$x$是一个图像，那么$s(x)$也可以表示为一个同尺寸的图像，其中对应的像素是图像的得分。下图显示了二维中的概率密度示例以及相应的得分函数。

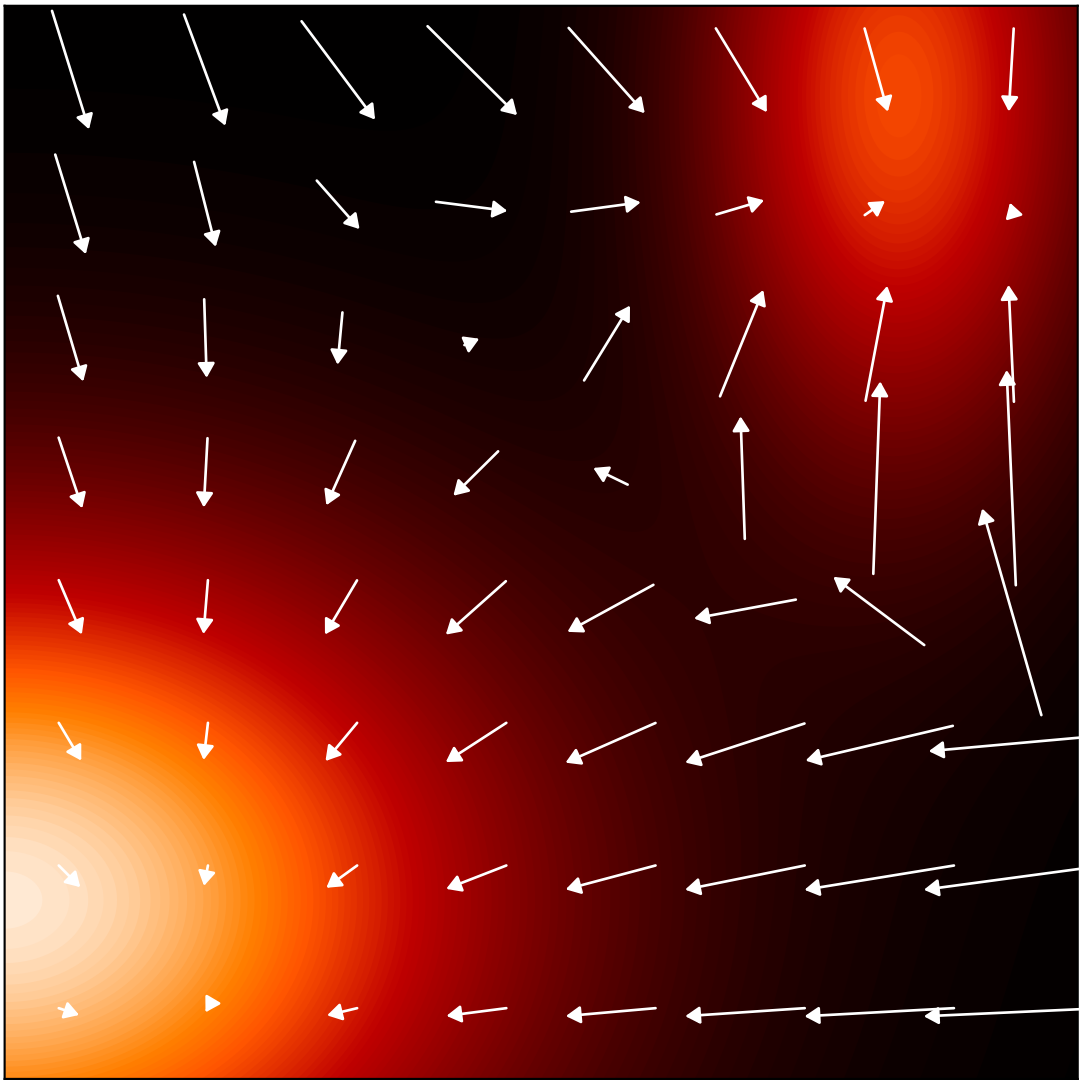


图 18.5 得分函数的示意图，图中显示了两种信息的融合：由高斯混合分布构成的二维分布，表示为热图；以及由 式 (18.5) 定义的相应得分函数，其作为向量绘制在x值的规则网络上

为了理解得分函数的用处，考虑两个函数$q(x)$和$p(x)$，它们具有得分相等的属性，所以对于x的所有值，$\nabla_x \log q(x) = \nabla_x \log p(x)$。如果我们对这个等式的两边关于$x$积分并取指数，则可以得到$q(x) = Kp(x)$，其中$K$是一个独立于$x$的常数。所以如果我们能学习到一个得分函数的模型$s(x, w)$，就可以重建原始数据分布，所得结果与之前的相比，最多相差一个常数倍数。

在实现扩散模型时，我们使用神经网络对$\varepsilon_\theta(x_t, t)$进行了建模。$\varepsilon_\theta(x_t, t)$基于$x_t$和$t$来推断噪声$\varepsilon$。下图是这个过程的示意图。

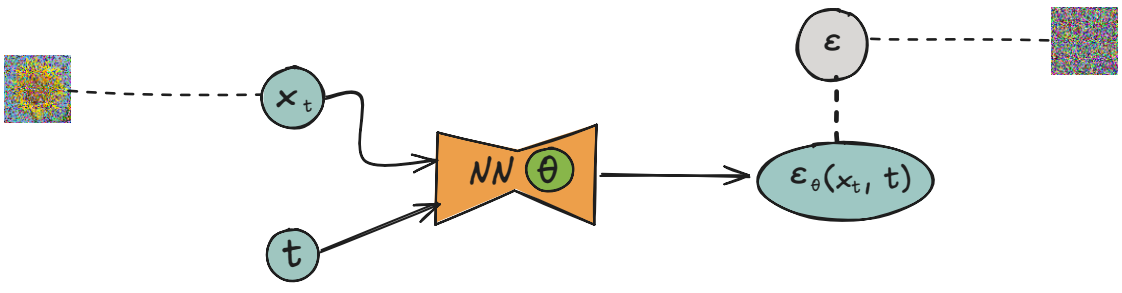


图 18.6 扩散模型中使用的推断噪声ε的神经网络

此时有以下与ε相关的数学式成立（稍后将给出证明）。

$$\varepsilon \approx -\sqrt{1-\overline{\alpha}_t}\nabla_{x_t}\log p(x_t) \tag{18.6}$$

∇是表示梯度的符号，读作“那勃乐”（Nabla）。$\nabla_{x_t}\log p(x_t)$是对数似然$\log p(x_t)$对输入数据x_t的梯度，称为“得分函数”或“得分”。

> 提示
>
> $p(x_t)$是表示数据x_t为“真”的随机密度函数。而$p_\theta(x_t)$表示使用参数对真的概率密度函数进行近似的概率密度函数。由于$\nabla_{x_t}\log p(x_t)$和$\nabla_{x_t}\log p_\theta(x_t)$表示关于输入$x_t$的梯度，因此称为“得分”。另外，在某些领域，关于参数的梯度（$\nabla_\theta \log p_\theta(x_t)$）也称为“得分”。在本书中，我们将对输入的梯度称为“得分”。

根据式式（18.6），ε可以近似表示为 $\nabla_{x_t}\log p(x_t)$。值得注意的是，ε 与 $\nabla_{x_t}\log p(x_t)$之间只相差负常数倍（$-\sqrt{1-\overline{\alpha}_t}$）。这说明以$-\sqrt{1-\overline{\alpha}_t}\varepsilon$来代替$\varepsilon$作为训练数据的神经网络也是可行的。

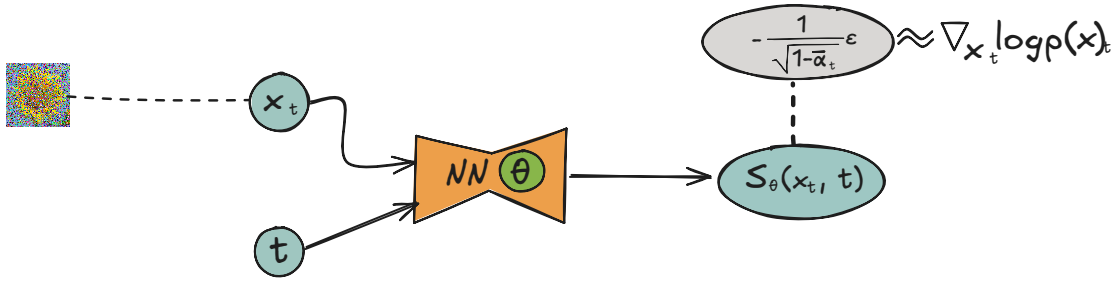


图 18.7 用于推断得分的神经网络

图中的神经网络将得分的近似值 $-\sqrt{1-\overline{\alpha}_t}\varepsilon$ 推断为 $s_\theta(x_t, t)$ 。因此，我们也可以通过推断得分的神经网络来实现扩散模型。

> 提示
>
> 生成模型中包括对得分进行建模的方法。这些模型统称为基于得分的模型。以对数似然最大化的方式可以推导出各种模型（GMM、VAE、扩散模型等）。这些模型可以被称为基于似然的模型。重要的是，扩散模型也可以作为基于得分的模型推导得出。

18.3.2 式（18.6）的证明

现在让我们来证明 式（18.6）成立。

$$\varepsilon \approx -\sqrt{1-\overline{\alpha}_t}\nabla_{x_t}\log p(x_t) \tag{18.7}$$

我们先来复习一下。时刻t的噪声数据x_t可以基于以下正态分布从原始数据x_0生成。

$$q(x_t|x_0) = \mathcal{N}\left(x_t; \sqrt{\overline{\alpha}_t}x_0, (1-\overline{\alpha})\boldsymbol{I}\right) \tag{18.8}$$

利用重参数化技巧，可以通过以下式子得到x_t的样本。

$$\begin{aligned} \varepsilon &\sim \mathcal{N}(0, \boldsymbol{I}) \\ x_t &= \sqrt{\overline{\alpha}_t}x_0 + \sqrt{1-\overline{\alpha}_t}\varepsilon \end{aligned} \tag{18.9}$$

接下来要用到的是 Tweedie 公式，这个公式如下所示。

Tweedie 公式

当基于 $x \sim \mathcal{N}(x; \boldsymbol{\mu}, \boldsymbol{\Sigma})$ 得到样本 x 时，有以下式子成立。

$$\mathbb{E}[\boldsymbol{\mu}|x] = x + \Sigma \nabla_x \log p(x) \tag{18.10}$$

式子中的$\boldsymbol{\mu}$被视为随机变量。左侧的$\mathbb{E}[\boldsymbol{\mu}|x]$是在$x$作为条件下的$\boldsymbol{\mu}$的期望值。右侧的$\nabla_x \log p(x)$表示得分。我们将这个 Tweedie 公式应用于式 (18.8)。

Tweedie 公式的应用

当基于 $x_t \sim \mathcal{N}\left(x_t; \sqrt{\overline{\alpha}_t} x_0, (1-\overline{\alpha}_t)\boldsymbol{I}\right)$得到样本$x_t$时，有以下式子成立。

$$\begin{aligned} \mathbb{E}\left[\sqrt{\overline{\alpha}_t} x_0 | x_t\right] &= x_t + (1-\overline{\alpha}_t)\boldsymbol{I}\nabla_{x_t} \log p(x_t) \\ &= x_t + (1-\overline{\alpha}_t)\nabla_{x_t} \log p(x_t) \end{aligned} \tag{18.11}$$

然后，利用 式 (18.9) 中的 $x_t = \sqrt{\overline{\alpha}_t} x_0 + \sqrt{1-\overline{\alpha}_t}\varepsilon$ 成立这一事实将式子展开，如下所示。

$$\begin{aligned} & \mathbb{E}\left[\sqrt{\overline{\alpha}_t} x_0 | x_t\right] = x_t + (1-\overline{\alpha}_t)\nabla_{x_t} \log p(x_t) \\ \Leftrightarrow\ & \mathbb{E}\left[x_t - \sqrt{1-\overline{\alpha}_t}\varepsilon | x_t\right] = x_t + (1-\overline{\alpha}_t)\nabla_{x_t} \log p(x_t) \\ \Leftrightarrow\ & \mathbb{E}[x_t|x_t] - \mathbb{E}\left[\sqrt{1-\overline{\alpha}_t}\varepsilon | x_t\right] = x_t + (1-\overline{\alpha}_t)\nabla_{x_t} \log p(x_t) \\ \Leftrightarrow\ & x_t - \sqrt{1-\overline{\alpha}_t}\mathbb{E}[\varepsilon|x_t] = x_t + (1-\overline{\alpha}_t)\nabla_{x_t} \log p(x_t) \\ \therefore\ & \mathbb{E}[\varepsilon|x_t] = (1-\overline{\alpha}_t)\nabla_{x_t} \log p(x_t) \end{aligned} \tag{18.12}$$

期望值 $\mathbb{E}[\varepsilon|x_t]$ 可以用蒙特卡洛方法近似。这里我们用一个采样数据来近似x_t。这样就得到了以下数学式。

$$\varepsilon \approx -\sqrt{1-\overline{\alpha}_t}\nabla_{x_t} \log p(x_t) \tag{18.13}$$

这样我们就完成了推导。

18.4 分类器指引

在上面的内容中，我们了解到可以通过预测得分的神经网络来实现扩散模型。在本节中，我们将从得分的角度来研究扩散模型，并推导出一种称为指引的方法。指引有两种主要类型：分类器指引（classifier guidance）和无分类器指引（classifier-free guidance）。我们先来介绍分类器指引。

18.4.1 什么是分类器

分类器指引是一种使用分类器指导数据生成的方法。分类器是一个能够对数据进行分类的预训练神经网络，如图所示。

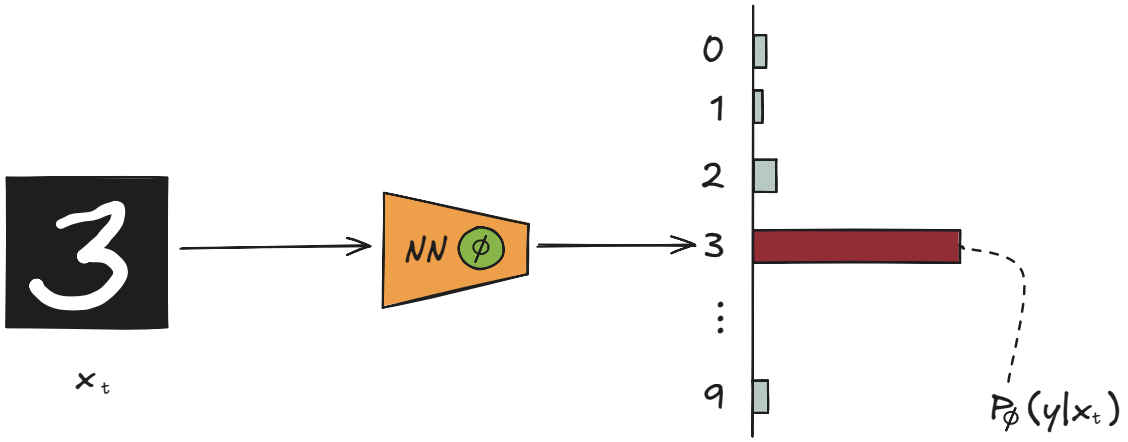


图 18.8 使用预训练神经网络进行分类的示例（参数为ϕ）

图中的神经网络将时刻t的噪声图像x_t作为输入，并输出类别y的概率$p_\phi(y|x_t)$。利用这个分类器的神经网络，可以将普通的扩散模型变换为条件扩散模型。在这个过程中，我们需要掌握以下两点。

- 扩散模型可以通过预测得分 $\nabla_{x_t}\log p(x_t)$ 的神经网络来实现。
- （基于同样的原理）条件扩散模型可以通过预测条件概率得分 $\nabla_{x_t}\log p(x_t|y)$ 的神经网络来实现。

在记住以上两点之后，我们进行式子的变形，推导出分类器指引。

18.4.2 分类器指引的推导

首先，根据贝叶斯定理，将条件概率$p(x_t|y)$表示为下面的数学式。

$$p(x_t|y)=\frac{p(x_t)p(y|x_t)}{p(y)} \tag{18.14}$$

然后，计算关于 x_t 的梯度（∇_{x_t}）。

$$\begin{aligned}\nabla_{x_t}\log p(x_t|y)&=\nabla_{x_t}\log\left(\frac{p(x_t)p(y|x_t)}{p(y)}\right)\\&=\nabla_{x_t}\log p(x_t)+\nabla_{x_t}\log p(y|x_t)-\underbrace{\nabla_{x_t}\log p(y)}_{0}\\&=\nabla_{x_t}\log p(x_t)+\nabla_{x_t}\log p(y|x_t)\end{aligned} \tag{18.15}$$

中间的式子中出现 $\nabla_{x_t}\log p(y)$ ，但由于 $p(y)$ 中不包含 x_t ，因此 $\nabla_{x_t}\log p(y)=0$ 。由此得到的数学式如下所示。

$$\underbrace{\nabla_{x_t}\log p(x_t|y)}_{\text{条件得分}}=\underbrace{\nabla_{x_t}\log p(x_t)}_{\text{❶得分}}+\underbrace{\nabla_{x_t}\log p(y|x_t)}_{\text{❷分类器的对数似然的梯度}} \tag{18.16}$$

根据这个式子可以推导出分类器指引。下面是对式子中的要点的说明。

❶ 此项可以使用预测得分的神经网络$s_\theta(x_t,t)$来计算。

❷ 此项可以使用作为分类器的神经网络来计算。

由此可见，我们可以利用得分和分类器这两个神经网络来表示条件得分（而一旦有了条件得分，便可以实现条件扩散模型）。另外，❷的$\nabla_{x_t}\log p(y|x_t)$可以很容易地通过反向传播求出。

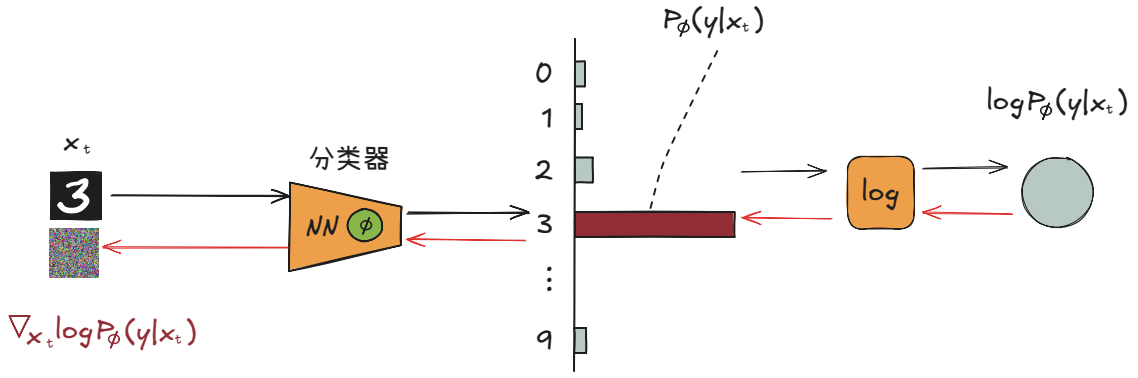


图 18.9 通过反向传播可以求出$\nabla_{x_t} \log p(y|x_t)$

上图中的 $\nabla_{x_t} \log p(y|x_t)$ 显示了在当前 x_t 下，类别 y 的对数似然增加最快的方向。因此，如果沿着 $\nabla_{x_t} \log p(y|x_t)$ 的方向更新 x_t ，那么更新后的图像被分类为类别 y 的概率会更高。

分类器指引的理念是使用得分和分类器来表示条件得分。通常，我们会向分类器指引中引入权重γ，这样就能调整分类器的贡献度。引入后的数学式如下所示。

$$\nabla_{x_t} \log p(x_t|y) = \nabla_{x_t} \log p(x_t) + \gamma \nabla_{x_t} \log p(y|x_t) \tag{18.17}$$

权重γ是人为设定的值（超参数），用于调整分类器向类别y方向引导的程度。γ值越大，条件y的作用就越显著。使用表示推断得分的神经网络$s_\theta(x_t, t)$和表示分类器的$p_\phi(y|x_t)$，可以将式子改写为如下形式。

$$\nabla_{x_t} \log p(x_t|y) \approx s_\theta(x_t, t) + \gamma \nabla_{x_t} \log p(y|x_t) \tag{18.18}$$

作为参考，上式所执行的处理如下图所示。

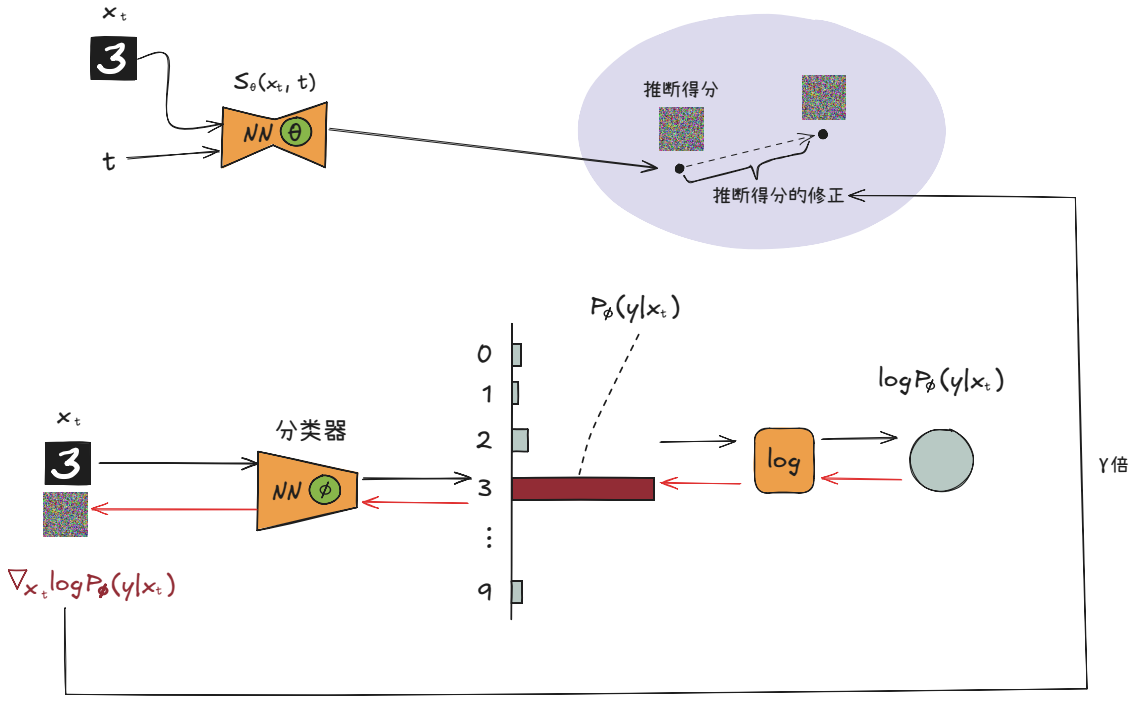


图 18.10 使用了推断得分的神经网络的分类器指引

如图所示，分类器可以与常规的扩散模型（推断得分的模型）相结合，作为条件扩散模型来生成数据。以上就是分类器指引的核心思想。

18.5 无分类器指引

分类器指引可以生成强调条件的数据。然而，分类器指引在实际应用中存在一个问题，即需要单独准备一个分类器。针对分类器指引的这一缺点进行改进的技术是无分类器指引。

18.5.1 无分类器指引的理论知识

顾名思义，无分类器指引就是不需要分类器的指引。它的机制可以通过以下过程得出。

$$
\begin{aligned}
&\nabla_{x_t} \log p(x_t|y) \\
&= \nabla_{x_t} \log p(x_t) + \gamma \nabla_{x_t} \log p(y|x_t) \\
&= \nabla_{x_t} \log p(x_t) + \gamma \nabla_{x_t} \log \frac{p(x_t|y)p(y)}{p(x_t)} \\
&= \nabla_{x_t} \log p(x_t) + \gamma \left(\nabla_{x_t} \log p(x_t|y) + \underbrace{\nabla_{x_t} \log p(y)}_{0} - \nabla_{x_t} \log p(x_t) \right) \\
&= \nabla_{x_t} \log p(x_t) + \gamma \big(\nabla_{x_t} \log p(x_t|y) - \nabla_{x_t} \log p(x_t) \big)
\end{aligned}
\tag{18.19}
$$

上面我们使用贝叶斯定理进行了式子的展开，最后得到了上面的式子。根据上面的式子，可以推导出无分类器指引。上式的意思是从点 $\nabla_{x_t} \log p(x_t)$ 出发，沿着 $\nabla_{x_t} \log p(x_t|y)$ 的方向前进了 γ 倍的距离。下图是这个式子的示意图。

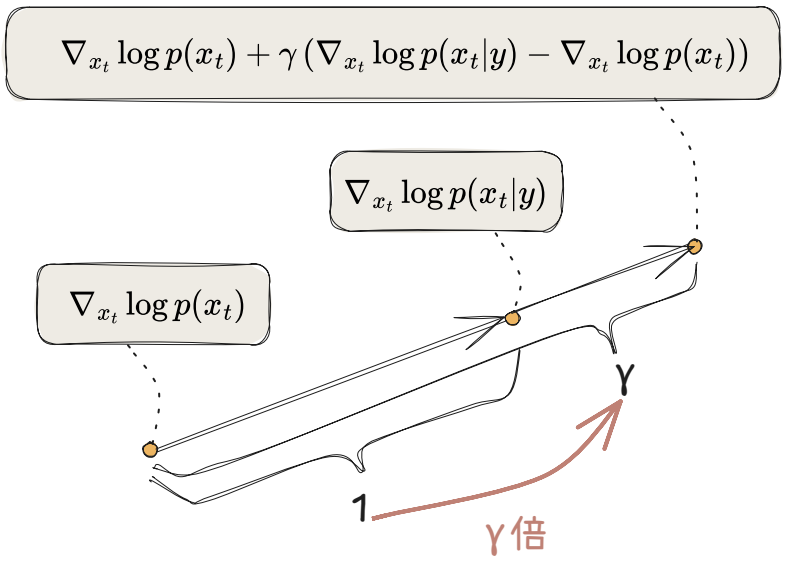


图 18.11 上面式子的可视化

式子中的 $\nabla_{x_t} \log p(x_t)$ 和 $\nabla_{x_t} \log p(x_t|y)$ 分别表示无条件得分和条件得分。它们可以由以下两个神经网络进行推断。

- 无条件得分推断器：$s_{\theta_1}(x_t, t)$
- 条件得分的推断器：$s_{\theta_2}(x_t, t, y)$

不过，准备两个模型很麻烦。更简单的方法是使用一个条件得分推断器 $s_{\theta_2}(x_t, t, y)$ ，并按如下方式建模。

- 无条件得分的推断器：$s_\theta(x_t, t, \emptyset)$
- 条件得分的推断器：$s_\theta(x_t, t, y)$

这里我们用$\emptyset$表示“无条件”对应的类别。条件y可以被嵌入层变换为向量，但当类别为$\emptyset$时，它会被变换为“零向量”，使其不包含任何信息。根据以上信息，无分类器指引的数学式可以表示成如下形式。

$$\nabla_{x_t} \log p(x_t|y) \approx s_\theta(x_t, t, \emptyset) + \gamma(s_\theta(x_t, t, y) - s_\theta(x_t, t, \emptyset)) \tag{18.20}$$

下图展示了这一计算过程。

上面介绍的是推断得分函数的神经网络，但由于得分和噪声之间只有常数倍的差别，因此同样的机制也适用于推断噪声的神经网络。

> 提示
>
> 在提供文本的图像生成服务中，我们经常会用到一种名为反向提示词（negative prompt）的技术。反向提示词是一种指定不希望生成的文本的技术。在上面的公式的$\emptyset$中插入反向提示词即可实现这一技术。

18.5.2 无分类器指引的实现

下面我们通过修改节中的部分代码来实现无分类器指引。先来回忆一下前面已经实现的UNetCond类，然后在这个类中实现 forward()方法，代码如下所示。

```python
class UNetCond(nn.Module):
    #...（省略）
    def forward(self, x, timesteps, labels=None):
        t = pos_encoding(timesteps, self.time_embed_dim)

        if labels is not None:
            t += self.label_emb(labels)
        #...
```

从上面的代码来看，只有指定了参数labels，才会处理类标签。如果没有指定参数（当labels=None时），则不做任何处理。这相当于“无条件”的处理。

无分类器指引的训练是在“无条件”和“有条件”两种情况下进行的条件扩散模型的训练。例如，我们可以以一定的比例进行“无条件”的训练，除此之外进行“有条件”的训练。基于这一点，我们可以按如下方式实现。

```python
for epoch in range(epochs):
    loss_sum = 0.0
    cnt = 0

    for images, labels in tqdm(dataloader):
        optimizer.zero_grad()
        x = images.to(device)
        labels = labels.to(device)
        t = torch.randint(1, num_timesteps+1, (len(x),), device=device)

        # 以10%的概率进行"无条件"的训练
        if np.random.random() < 0.1:
```

```
            labels = None

        x_noisy, noise = diffuser.add_noise(x, t)
        noise_pred = model(x_noisy, t, labels)
        loss = F.mse_loss(noise, noise_pred)
        loss.backward()
        optimizer.step()

        loss_sum += loss.item()
        cnt += 1
```

上面的代码以 10% 的概率进行了“无条件”的训练。对于“无条件”的情况，设置 `labels=None`。

最后是 `Diffuser` 类进行去噪处理的代码。

```python
class Diffuser:
    #...（省略）

    def denoise(self, model, x, t, labels, gamma):
        #...
        with torch.no_grad():
            eps_cond = model(x, t, labels)
            eps_uncond = model(x, t)
            eps = eps_uncond + gamma * (eps_cond - eps_uncond)
        #...
```

参数 `gamma` 的值越大，就越应该重视条件部分。另外，这里实现的是如下数学式的代码。

$$\varepsilon \approx \varepsilon_\theta(x_t, t, \emptyset) + \gamma(\varepsilon_\theta(x_t, t, y) - \varepsilon_\theta(x_t, t, \emptyset)) \tag{18.21}$$

经过以上修改之后，我们就完成了无分类器指引的实现。这里以$\gamma = 3$（`gamma=3.0`）生成了数据。

无分类器指引可以通过γ来调整条件的重视程度。而且由于无需另外训练分类器，因此在资源和时间方面更为高效。

完整代码如下：

```python
import math
import numpy as np
import torch
import torchvision
import matplotlib.pyplot as plt
from torchvision import transforms
from torch.utils.data import DataLoader
from torch.optim import Adam
import torch.nn.functional as F
from torch import nn
from tqdm import tqdm


img_size = 28
batch_size = 128
num_timesteps = 1000
epochs = 10
lr = 1e-3
device = "cuda" if torch.cuda.is_available() else "cpu"
```

```
20
21
22  def show_images(images, labels=None, rows=2, cols=10):
23      fig = plt.figure(figsize=(cols, rows))
24      i = 0
25      for r in range(rows):
26          for c in range(cols):
27              ax = fig.add_subplot(rows, cols, i + 1)
28              plt.imshow(images[i], cmap="gray")
29              if labels is not None:
30                  ax.set_xlabel(labels[i].item())
31              ax.get_xaxis().set_ticks([])
32              ax.get_yaxis().set_ticks([])
33              i += 1
34      plt.tight_layout()
35      plt.show()
36
37  def _pos_encoding(time_idx, output_dim, device="cpu"):
38      t, D = time_idx, output_dim
39      v = torch.zeros(D, device=device)
40
41      i = torch.arange(0, D, device=device)
42      div_term = torch.exp(i / D * math.log(10000))
43
44      v[0::2] = torch.sin(t / div_term[0::2])
45      v[1::2] = torch.cos(t / div_term[1::2])
46      return v
47
48  def pos_encoding(timesteps, output_dim, device="cpu"):
49      batch_size = len(timesteps)
50      device = timesteps.device
51      v = torch.zeros(batch_size, output_dim, device=device)
52      for i in range(batch_size):
53          v[i] = _pos_encoding(timesteps[i], output_dim, device)
54      return v
55
56  class ConvBlock(nn.Module):
57      def __init__(self, in_ch, out_ch, time_embed_dim):
58          super().__init__()
59          self.convs = nn.Sequential(
60              nn.Conv2d(in_ch, out_ch, 3, padding=1),
61              nn.BatchNorm2d(out_ch),
62              nn.ReLU(),
63              nn.Conv2d(out_ch, out_ch, 3, padding=1),
64              nn.BatchNorm2d(out_ch),
65              nn.ReLU()
66          )
67          self.mlp = nn.Sequential(
68              nn.Linear(time_embed_dim, in_ch),
69              nn.ReLU(),
```

```
            nn.Linear(in_ch, in_ch)
        )

    def forward(self, x, v):
        N, C, _, _ = x.shape
        v = self.mlp(v)
        v = v.view(N, C, 1, 1)
        y = self.convs(x + v)
        return y

class UNetCond(nn.Module):
    def __init__(self, in_ch=1, time_embed_dim=100, num_labels=None):
        super().__init__()
        self.time_embed_dim = time_embed_dim

        self.down1 = ConvBlock(in_ch, 64, time_embed_dim)
        self.down2 = ConvBlock(64, 128, time_embed_dim)
        self.bot1 = ConvBlock(128, 256, time_embed_dim)
        self.up2 = ConvBlock(128 + 256, 128, time_embed_dim)
        self.up1 = ConvBlock(128 + 64, 64, time_embed_dim)
        self.out = nn.Conv2d(64, in_ch, 1)

        self.maxpool = nn.MaxPool2d(2)
        self.upsample = nn.Upsample(scale_factor=2, mode="bilinear")

        if num_labels is not None:
            self.label_emb = nn.Embedding(num_labels, time_embed_dim)

    def forward(self, x, timesteps, labels=None):
        t = pos_encoding(timesteps, self.time_embed_dim)

        if labels is not None:
            t += self.label_emb(labels)

        x1 = self.down1(x, t)
        x = self.maxpool(x1)
        x2 = self.down2(x, t)
        x = self.maxpool(x2)

        x = self.bot1(x, t)

        x = self.upsample(x)
        x = torch.cat([x, x2], dim=1)
        x = self.up2(x, t)
        x = self.upsample(x)
        x = torch.cat([x, x1], dim=1)
        x = self.up1(x, t)
        x = self.out(x)
        return x

```

```python

class Diffuser:
    def __init__(self, num_timesteps=1000, beta_start=0.0001, beta_end=0.02, device="cpu"):
        self.num_timesteps = num_timesteps
        self.device = device
        self.betas = torch.linspace(beta_start, beta_end, num_timesteps, device=device)
        self.alphas = 1 - self.betas
        self.alpha_bars = torch.cumprod(self.alphas, dim=0)

    def add_noise(self, x_0, t):
        T = self.num_timesteps
        assert (t >= 1).all() and (t <= T).all()

        t_idx = t - 1  # alpha_bars[0] is for t=1
        alpha_bar = self.alpha_bars[t_idx]  # (N,)
        alpha_bar = alpha_bar.view(alpha_bar.size(0), 1, 1, 1)  # (N, 1, 1, 1)

        noise = torch.randn_like(x_0, device=self.device)
        x_t = torch.sqrt(alpha_bar) * x_0 + torch.sqrt(1 - alpha_bar) * noise
        return x_t, noise

    def denoise(self, model, x, t, labels, gamma):
        T = self.num_timesteps
        assert (t >= 1).all() and (t <= T).all()

        t_idx = t - 1  # alphas[0] is for t=1
        alpha = self.alphas[t_idx]
        alpha_bar = self.alpha_bars[t_idx]
        alpha_bar_prev = self.alpha_bars[t_idx-1]

        N = alpha.size(0)
        alpha = alpha.view(N, 1, 1, 1)
        alpha_bar = alpha_bar.view(N, 1, 1, 1)
        alpha_bar_prev = alpha_bar_prev.view(N, 1, 1, 1)

        model.eval()
        with torch.no_grad():
            eps = model(x, t, labels)
            eps_uncond = model(x, t)
            eps = eps_uncond + gamma * (eps - eps_uncond)
        model.train()

        noise = torch.randn_like(x, device=self.device)
        noise[t == 1] = 0  # no noise at t=1

        mu = (x - ((1-alpha) / torch.sqrt(1-alpha_bar)) * eps) / torch.sqrt(alpha)
        std = torch.sqrt((1-alpha) * (1-alpha_bar_prev) / (1-alpha_bar))
        return mu + noise * std

```

```
    def reverse_to_img(self, x):
        x = x * 255
        x = x.clamp(0, 255)
        x = x.to(torch.uint8)
        x = x.cpu()
        to_pil = transforms.ToPILImage()
        return to_pil(x)

    def sample(self, model, x_shape=(20, 1, 28, 28), labels=None, gamma=3.0):
        batch_size = x_shape[0]
        x = torch.randn(x_shape, device=self.device)
        if labels is None:
            labels = torch.randint(0, 10, (len(x),), device=self.device)

        for i in tqdm(range(self.num_timesteps, 0, -1)):
            t = torch.tensor([i] * batch_size, device=self.device,
dtype=torch.long)
            x = self.denoise(model, x, t, labels, gamma)

        images = [self.reverse_to_img(x[i]) for i in range(batch_size)]
        return images, labels


preprocess = transforms.ToTensor()
dataset = torchvision.datasets.MNIST(root="./data", download=True,
transform=preprocess)
dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=True)

diffuser = Diffuser(num_timesteps, device=device)
model = UNetCond(num_labels=10)
model.to(device)
optimizer = Adam(model.parameters(), lr=lr)

losses = []
for epoch in range(epochs):
    loss_sum = 0.0
    cnt = 0

    # generate samples every epoch ====================
    #images, labels = diffuser.sample(model)
    #show_images(images, labels)
    # ================================================

    for images, labels in tqdm(dataloader):
        optimizer.zero_grad()
        x = images.to(device)
        labels = labels.to(device)
        t = torch.randint(1, num_timesteps+1, (len(x),), device=device)

        if np.random.random() < 0.1:
            labels = None
```

```
218
219         x_noisy, noise = diffuser.add_noise(x, t)
220         noise_pred = model(x_noisy, t, labels)
221         loss = F.mse_loss(noise, noise_pred)
222
223         loss.backward()
224         optimizer.step()
225
226         loss_sum += loss.item()
227         cnt += 1
228
229     loss_avg = loss_sum / cnt
230     losses.append(loss_avg)
231     print(f"Epoch {epoch} | Loss: {loss_avg}")
232
233 # plot losses
234 plt.plot(losses)
235 plt.xlabel("Epoch")
236 plt.ylabel("Loss")
237 plt.show()
238
239 # generate samples
240 images, labels = diffuser.sample(model)
241 show_images(images, labels)
```

18.6 Stable Diffusion

到目前为止，我们已经实现了处理 MNIST 的小型扩散模型以及小型条件扩散模型。当然，现代的扩散模型规模相当庞大，而且经过了多项改进。下面是一些进一步改进扩散模型的指引。这里我们介绍一个著名的模型——Stable Diffusion，它能够生成下图所示的高清图像。另外，它的代码和预训练的权重数据是公开的，任何人都可以运行这些代码。

图 18.12 Stable Diffusion 生成的高清图像

19. DALL-E2 （文生图）

> 🔥 DALL-E2
>
> - 输入：文本提示词
> - 输出：图像

> 🔥 训练 DALL-E2 的数据集
>
> 图文对！

去噪扩散概率模型（DDPM）是一种流行的生成式人工智能模型，由 Ho 等人于 2020 年提出，并由 Nichol 等人于 2021 年对其进行了改进。这些模型背后的基本思想是，在正向扩散过程中将噪声添加到图像中，以便训练模型预测在反向扩散过程中应在特定时间步去除的噪声。在对图像进行采样（去噪）时，需要从纯噪声的图像开始，并在每个时间步迭代地去除模型预测的噪声，直到获得最终图像。

为了让 DDPM 生成多种类型的图像，同时仍允许用户选择所需的图像类型，模型需要根据某些输入进行条件调节（条件扩散模型）。Ramesh 等人提出了一种名为 unCLIP 的条件调节方法，该方法已用于 OpenAI 的 DALL-E 2 模型。在 Ramesh 等人描述的方法中，输入的图片标题或者描述首先被传递到一个**先验网络**（prior model），该网络将使用经过训练的 CLIP 模型来获取 CLIP 文本嵌入。然后，仅解码器架构的 Transformer 使用这些文本嵌入来生成可能的 CLIP 图像嵌入。先验网络生成的 CLIP 图像嵌入将被解码器网络（由 UNet 模型组成）用于条件调节所创建的图像。在本章中，我们将使用此过程构建一个简单的扩散模型。

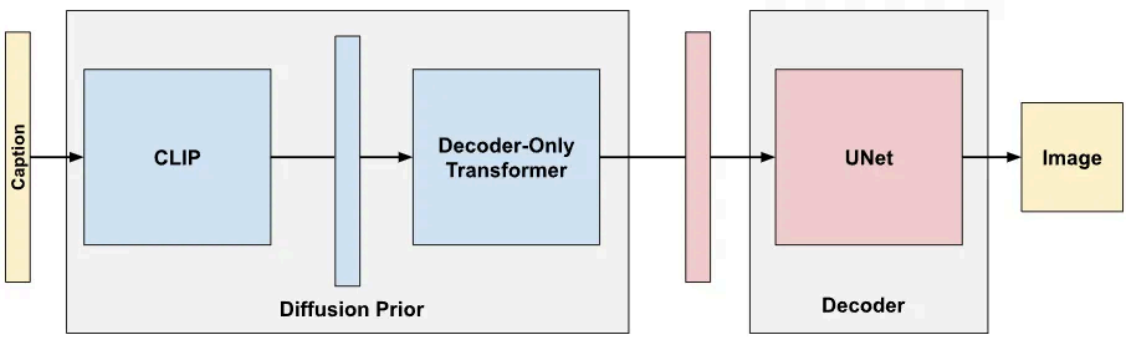


图 19.1 DALL-E2 模型整体架构

> **DALL-E2 需要训练 3 个模型**
>
> 1. CLIP 模型
> 2. 先验模型：Decoder-Only Transformer，先验模型的目标是生成高质量的**条件**。
> 3. 条件扩散模型：UNet

19.1 CLIP

为了从文本创建扩散图像（文生图），我们将使用 CLIP 模型生成的嵌入（文本特征向量，图像特征向量）。从 CLIP 获得的文本嵌入用于调节先验模型，使其扩散相应的图像嵌入。然后，这些图像嵌入用于调节解码器模型，用来指导解码器生成对应的图像。

> **提示**
>
> CLIP 模型训练完毕后，冻结起来供后续使用。

19.2 先验模型（Prior Model）

> **危险**
>
> 训练先验模型时，需要使用上一节训练的 CLIP 模型，CLIP 模型必须**冻结**！

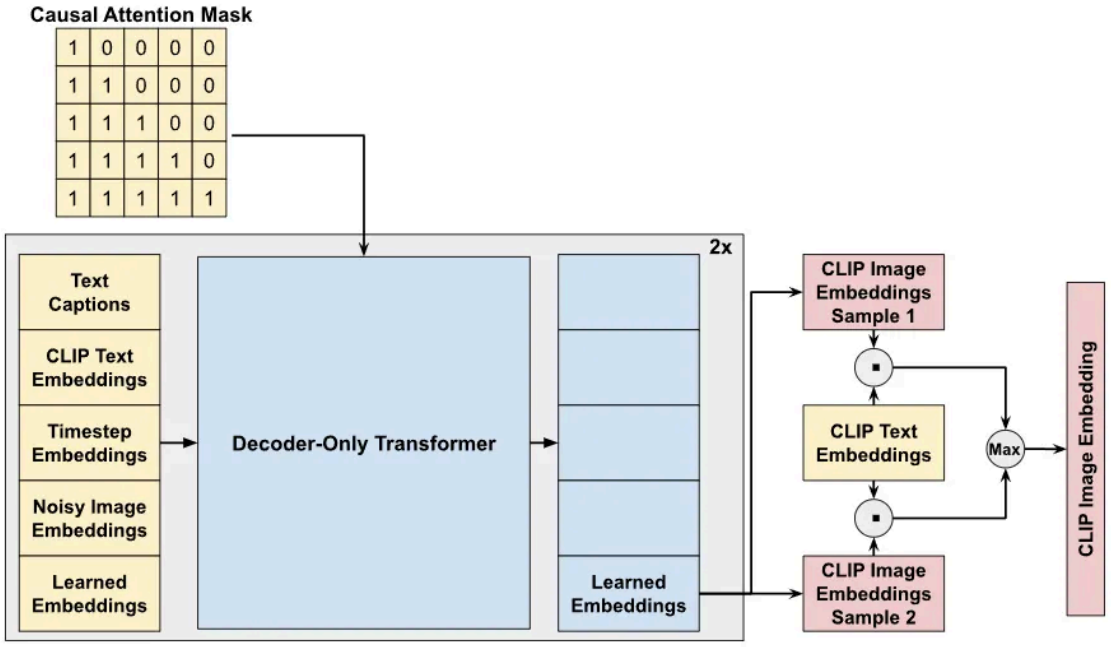


图 19.2 先验模型架构图

先验模型的作用：**根据文本标题预测 CLIP 图像嵌入** 。也可以放弃先验模型，而以 CLIP 文本嵌入为条件，而不是用先验模型生成的 CLIP 图像嵌入作为条件，但使用先验模型的效果最佳。

先验模型是一个仅解码器架构的 Transformer。

19.2.1 先验模型的训练

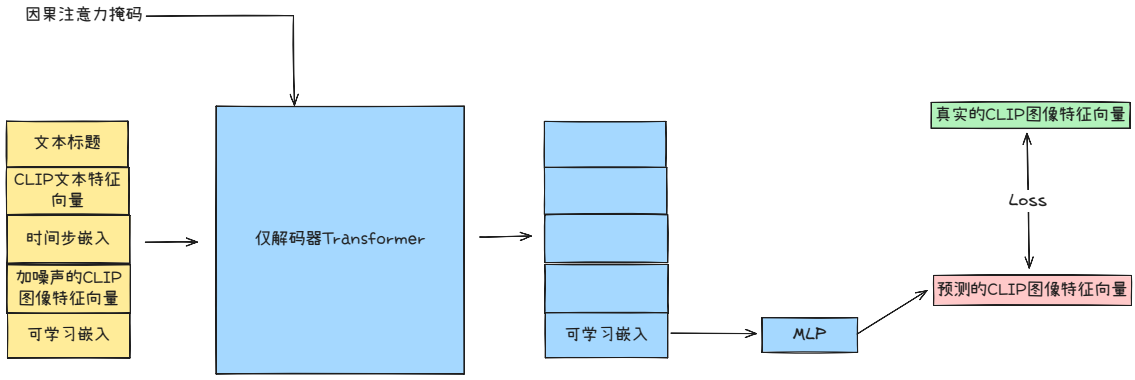


图 19.3 先验模型的训练原理

要训练一个模型，我们必须搞清楚模型的输入和输出（预测目标）。

训练先验模型时，我们手上有的是：图文对和一个训练好的 CLIP 模型。

先验模型的输入有 6 个：

1. Text Captions：图文对中的“文本”。
2. CLIP Text Embeddings：图文对中的“文本”经过上一节训练好的**冻结的 CLIP 模型**生成的 CLIP 文本嵌入。
3. Timestep Embeddings：随机选取的时间步t的嵌入。

4. `Noisy Image Embeddings`：添加了t步噪声的 CLIP 图像嵌入。这个 CLIP 图像嵌入是图文对中的“图像”经过上一节训练好的**冻结的 CLIP 模型**生成的 CLIP 图像嵌入。
5. `Learned Embeddings`：一个随机初始化的可学习的嵌入。
6. `Causal Attention Mask`：因果注意力掩码。因为我们要训练的是一个 `Decoder-Only Transformer` 架构的模型，所以需要使用因果注意力掩码。

> ⚡ 危险
>
> `Noisy Image Embeddings` 是针对 CLIP **图片嵌入**添加噪声，而不是针对图片添加噪声！

先验模型的输出（预测目标）：

- 图文对中的“图像”经过上一节训练好的**冻结的 CLIP 模型**生成的 CLIP 图像嵌入。

训练细节：

1. 6 个输入经过 `Decoder-Only Transformer` 之后会输出 5 个张量，我们取**最后一个**，也就是图中蓝色的 `Learned Embeddings`。
2. `Learned Embeddings` 会送入一个 MLP，输出的就是预测的 CLIP 图片嵌入。
3. 预测的 CLIP 图片嵌入与真正的 CLIP 图片嵌入计算损失，并反向传播更新网络。

19.2.2 先验模型的推理

> 🔥 提示
>
> 在推理时，我们手上只有：
> 1. 自己编写的提示词
> 2. 训练好的 CLIP 模型
> 3. 训练好的先验模型

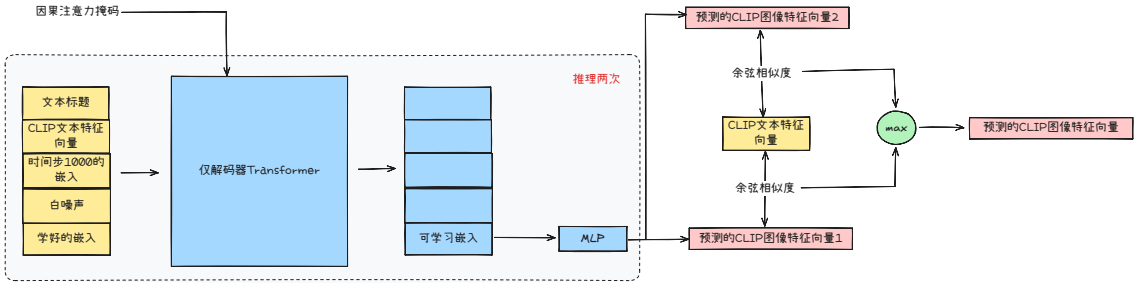


图 19.4 先验模型的推理流程

先验模型的输入有 6 个：

1. `Text Captions`：自己编写的文本提示词。
2. `CLIP Text Embeddings`：“自己编写的文本提示词”经过上一节训练好的**冻结的 CLIP 模型**生成的 CLIP 文本嵌入。
3. `Timestep Embeddings`：最大时间步$T = 1000$的时间步嵌入。
4. `Noisy Image Embeddings`：纯噪声构成的嵌入（和图片嵌入的形状相同，从高斯分布中采样得到）。
5. `Learned Embeddings`：训练好的参数（先验模型训练完之后就在模型中了）。
6. `Causal Attention Mask`：因果注意力掩码。

> 提示
>
> 所以只有 `Text Captions` 是我们自己编写的文本提示词，其它输入都是模型或者程序自动生成的。

先验模型的输出：

1. 6 个输入经过 Decoder-Only Transformer 之后会输出 5 个张量，我们取**最后一个**，也就是图中蓝色的 `Learned Embeddings`。
2. `Learned Embeddings` 会送入两次 MLP，生成两个预测的 CLIP 图片嵌入。
3. 两个预测的CLIP图片嵌入分别和CLIP文本嵌入计算余弦相似度，然后选取最相似的一个作为预测。

19.2.3 关键代码解释

19.2.3.1 余弦调度的方差计划

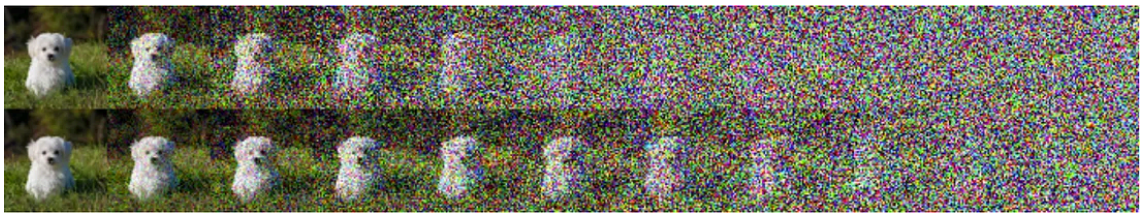


图 19.5 线性调度方差和余弦调度方差的对比

对于方差调度，在 Ho 等人的原始 DDPM 论文中，采用了线性调度。虽然该调度在高分辨率图像上效果良好，但当图像较小（64x64 或更小）时，前向过程最终会产生过多的噪声。为了解决这个问题，Nichol 等人建议使用余弦调度，其公式如下所示。

$$
\begin{aligned}
\beta_t &= 1 - \frac{\overline{\alpha}_t}{\overline{\alpha}_{t-1}} \\
\overline{\alpha}_t &= \frac{f(t)}{f(0)} \\
f(t) &= \cos\left(\frac{\frac{t}{T}+s}{1+s}\cdot\frac{\pi}{2}\right)^2
\end{aligned}
\tag{19.1}
$$

这些值被剪裁为小于或等于 0.999，以防止在 `t = T` 附近出现奇点。因为我们使用包含低分辨率图像的 FashionMNIST 数据集，所以我们将使用余弦方差计划。

综合起来，前向扩散过程的代码看起来是这样的：

```python
# Gets elements from indicies and makes sure output has a certain dimension
def extract_and_expand(x, idx, shape):
    return x[idx].reshape(idx.shape[0], *((1, ) * (len(shape) - 1)))

# 返回方差调度计划
def get_beta_schedule(schedule, max_time, s=0.008):
    if schedule == "linear":
        scale = 1000 / max_time
        betas = torch.linspace(1e-4  * scale, 0.02  * scale, max_time)
    elif schedule == "cosine":
```

```
        t = torch.linspace(0, max_time, max_time + 1)
        a_bars = torch.cos(
            (((t / max_time) + s) / (1 + s)) * (np.pi / 2)
        ) ** 2
        a_bars = a_bars / a_bars[0]
        betas = 1 - (a_bars[1:] / a_bars[:-1])
        betas = torch.clamp(betas, min=0, max=0.999)
    else:
        Exception("Beta schedule not implemented.")

    return betas

def get_schedule_values(config):
        schedule_values = {}
        schedule_values["betas"] = get_beta_schedule(
            config.prior.schedule,
            config.prior.max_time
        ).to(config.device)
        schedule_values["alphas"] = 1.0 - schedule_values["betas"]
        schedule_values["alpha_bars"] = torch.cumprod(
            schedule_values["alphas"],
            axis = 0
        )
        schedule_values["sqrt_recip_alphas"] = torch.sqrt(
            1.0 / schedule_values["alphas"])
        schedule_values["sqrt_alpha_bars"] = torch.sqrt(
            schedule_values["alpha_bars"]
        )
        schedule_values["sqrt_one_minus_alpha_bars"] = torch.sqrt(
            1.0 - schedule_values["alpha_bars"]
        )
        schedule_values["alpha_bars_prev"] = torch.cat((
            torch.ones(1, device=config.device),
            schedule_values["alpha_bars"][:-1])
        )
        schedule_values["sigma"] = schedule_values["betas"] \
            * (1.0 - schedule_values["alpha_bars_prev"])    \
            / (1.0 - schedule_values["alpha_bars"])
        return schedule_values

# 获取x_t，前向扩散过程
def forward_diffusion(x_0, schedule_values, t):
    # 噪声和图片的尺寸一样
    noise = torch.randn_like(x_0)
    sqrt_alpha_bars = extract_and_expand(
        schedule_values["sqrt_alpha_bars"], t, x_0.shape)
    sqrt_one_minus_alpha_bars = extract_and_expand(
        schedule_values["sqrt_one_minus_alpha_bars"], t, x_0.shape)
    x_noisy = (sqrt_alpha_bars * x_0) \
            + (sqrt_one_minus_alpha_bars * noise)
```

```
    # 返回x_t, 和添加的噪声
    return x_noisy, noise
```

19.2.3.2 时间步嵌入

时间步嵌入是扩散的重要组成部分。这是因为不同时间步的图像具有不同的噪声量。为了在我们的模型中利用这些信息，我们将使用正弦位置编码。这些位置编码与 Transformer 中常用的位置编码相同。主要区别在于，我们的输入时间步很可能不会按顺序排列，并且包含所有可能的时间步，因此我们只需要获取与输入时间步对应的位置编码。

```python
class SinusoidalPositionalEncodings(nn.Module):
    def __init__(
        self,
        max_seq_length,    # 最大序列长度
        width              # 模型的宽度（嵌入的维度d_model）
    ):
        super().__init__()
        # Create positional encodings
        pe = torch.zeros(max_seq_length, width)
        for pos in range(max_seq_length):
            for i in range(width):
                if i % 2 == 0:
                    pe[pos][i] = np.sin(pos/(10000**(i/width)))
                else:
                    pe[pos][i] = np.cos(pos/(10000**((i-1)/width)))

        self.register_buffer("pe", pe)

    def forward(self, x):
        # Get positional encodings corresponding to inputted timesteps
        x = self.pe[x]
        return x
```

然后，这些时间编码通过 MLP 来进一步捕获时间信息。

```python
self.time_mlp = nn.Sequential(
    SinusoidalPositionalEmbedding(
        config.decoder.max_time,
        config.decoder.model_channels
    ),
    nn.Linear(
        config.decoder.model_channels,
        config.decoder.cond_channels
    ),
    nn.SiLU(),
    nn.Linear(
        config.decoder.cond_channels,
        config.decoder.cond_channels
    )
)
```

有多种方法可以调节时间步嵌入的输入，这些方法在本章的 UNet：残差块部分中进行了描述。

19.2.3.3 冻结模型

先验模型首先将图像的文本标题作为输入，然后获取 CLIP 文本嵌入和图像嵌入。加载 CLIP 模型时，所有层都应冻结，并将模式设置为 eval。

```python
def freeze_model(model, set_eval=True):
    if set_eval:
        model.eval()

    for param in model.parameters():
        param.requires_grad = False

# Constructor
self.clip = CLIP(config).to(config.device)
self.clip.load_state_dict(torch.load(
    config.clip.model_location,
    map_location=config.device
))
# 冻结clip模型
freeze_model(self.clip)

# Forward
image_embeddings = self.clip.image_encoder(
    images
) # (B, C, H, W) → (B, latent_dim)
text_embeddings = self.clip.text_encoder(
    captions,
    mask=masks
) # (B, text_seq_length) → (B, latent_dim)
```

19.2.3.4 加噪声的 CLIP 图片嵌入

然后，它会获取批次中每个图像的随机时间步，并使用它们从前向扩散中获取噪声 CLIP 图像嵌入。

> ⚡ 危险
>
> 不是对图片加噪声，而是对 CLIP 图片嵌入加噪声！

```python
# Forward
# 随机采样一个时间步
timesteps = torch.randint(
    0, self.config.prior.max_time, (images.shape[0],)) # (B, )

noisy_image_embedding, _ = forward_diffusion(
    image_embeddings, # 对图片的嵌入加噪声
    self.schedule_values, # 方差调度计划
    timesteps # 时间步
)
```

然后将时间步通过 MLP 以获得时间步嵌入。

```python
# Constructor
```

```
    self.time_mlp = nn.Sequential(
        SinusoidalPositionalEmbedding(
            config.prior.max_time,
            config.latent_dim
        ),
        nn.Linear(
            config.latent_dim,
            config.latent_dim * config.prior.r_mlp,
            bias=config.prior.bias
        ),
        nn.SiLU(),
        nn.Linear(
            config.latent_dim * config.prior.r_mlp,
            config.latent_dim,
            bias=config.prior.bias
        )
    )

    # Forward
    timestep_embeddings = self.time_mlp(timesteps) # (B, ) → (B, latent_dim)
```

19.2.3.5 可学习的嵌入

先验模型的另一个重要部分是可以训练的嵌入层。这些嵌入是 Torch 的一个参数，将用于预测最终输出。从构造函数中学习到的嵌入需要在 Forward 方法中进行扩展，以便批次中的每个项目都有一个嵌入。

```python
# 初始化一个随机的需要学习的嵌入参数
self.learned_embedding = nn.Parameter(torch.randn(config.latent_dim))

# Forward
# 为批次中的每一项复制一份可学习参数
learned_embeddings = self.learned_embedding.repeat(
    images.shape[0],
    1
) # (latent_dim) → (B, latent_dim)
```

19.2.3.6 输入的拼接

文本标题、CLIP 文本嵌入、时间步嵌入、加噪声的 CLIP 图像嵌入以及需要学习的嵌入将被连接成一个序列。所有这些项都将具有形状 (B, latent_dim)，但我们需要在中间添加一个额外的维度，使它们具有形状 (B, 1, latent_dim)。我们将在这个新维度上进行连接，使序列具有形状 (B, 5, latent_dim)。如果使用包含更高质量图像的数据集，一种提高模型质量的可能方法是将文本嵌入、图像嵌入和/或时间步嵌入传递到卷积层，以增加序列长度。

```python
tokens = torch.cat((
    captions,               # 图像标题
    text_embeddings,        # CLIP文本嵌入
    timestep_embeddings,    # 时间步嵌入
    noisy_image_embedding,  # 加噪声的CLIP图像嵌入
    learned_embeddings      # 需要学习的嵌入
), dim=1) # (B, 5, latent_dim)
```

然后，该序列会输入到一个带有因果注意力掩码的仅解码器架构的 Transformer。因果注意力掩码是一个由 1 组成的下三角矩阵，它使得某个 token 只能关注在它之前出现的 token。

```python
# 初始化
self.decoder = nn.ModuleList(
    [TransformerBlock(
        config.latent_dim,
        cond_width=config.latent_dim,
        n_heads=config.prior.n_heads,
        dropout=config.prior.dropout,
        r_mlp=config.prior.r_mlp,
        bias=config.prior.bias
    ) for _ in range(config.prior.n_layers)]
)

self.register_buffer(
    "causal_attention_mask",
    torch.tril(torch.ones(5, 5))[None, :]
)

# Forward
for block in self.decoder:
    tokens = block(tokens, mask=self.causal_attention_mask)
```

19.2.3.7 预测目标

最后，我们从 Transformer 的输出中获取学习到的嵌入，并将其传递给 LayerNorm 和 Linear 层以获得预测的图像嵌入。

```python
# 初始化
self.output = nn.Sequential(
    nn.LayerNorm(config.latent_dim),
    nn.Linear(
        config.latent_dim,
        config.latent_dim,
        bias=config.decoder.bias
    )
)

# Forward: 取最后一个训练出来的可学习嵌入
pred_image_embeddings = self.output(tokens[:, -1, :])
```

虽然通常情况下，扩散模型会预测噪声，并通过迭代去除噪声来生成样本，但 unCLIP 论文中提到，最好直接预测 CLIP 图像嵌入。该模型的损失函数应该是预测值与实际 CLIP 图像嵌入之间的均方误差损失。

```python
loss = nn.functional.mse_loss(pred_image_embeddings, image_embeddings)
```

在采样过程中，为了提高质量，模型应该生成两个 CLIP 图像嵌入的样本，并选择与 CLIP 文本嵌入点积较高的样本。

```python
def get_one_sample(self, text_embeddings, captions):
    # 获取一个和CLIP文本嵌入形状相同的白噪声
```

```
    # 而clip的图像嵌入和文本嵌入的形状相同
    noisy_image_embeddings = torch.randn(
        text_embeddings.shape,
        device=self.config.device
    )

    # 所有项的时间步都是最大时间步T，因为图像嵌入目前都是纯噪声
    timesteps = torch.full(
        (captions.shape[0],),
        self.config.prior.max_time - 1
    )

    # 获取时间步嵌入
    timestep_embeddings = self.time_mlp(
        timesteps
    ) # (B, ) → (B, latent_dim)

    # (B, latent_dim) → (B, 1, latent_dim)
    timestep_embeddings = timestep_embeddings[:, None, :]

    # 为批次中每一项都创建一个需要学习的嵌入
    learned_embeddings = self.learned_embedding.repeat(
        captions.shape[0],
        1
    ) # (latent_dim) → (B, latent_dim)

    # (B, latent_dim) → (B, 1, latent_dim)
    learned_embeddings = learned_embeddings[:, None, :]

    tokens = torch.cat((
        captions,               # 图像标题
        text_embeddings,        # CLIP文本嵌入
        timestep_embeddings,    # 最大时间步的嵌入（比如T=1000）
        noisy_image_embeddings,  # 白噪声
        learned_embeddings      # 需要学习的嵌入
    ), dim=1) # (B, 5, latent_dim)

    # 和因果注意力掩码一起输入给transformer块
    for block in self.decoder:
        tokens = block(tokens, mask=self.causal_attention_mask)

    # 预测的图像嵌入（取出学习到的嵌入，输入给投影层）
    pred_image_embeddings = self.output(tokens[:, -1, :])

    return pred_image_embeddings

def sample(self, captions, masks=None):
    # 获取CLIP文本嵌入
    t_emb = self.clip.text_encoder(
```

```
        captions,
        mask=masks
    ) # (B, text_seq_length) -> (B, latent_dim)

    # (B, latent_dim) -> (B, 1, latent_dim)
    text_embeddings = t_emb[:, None, :]

    # 使图片文本的长度等于 latent dimension
    if self.config.text_seq_length >= self.config.latent_dim:
        # (B, max_seq_len) -> (B, latent_dim)
        captions = captions[:, :self.config.latent_dim]
    else:
        captions = nn.functional.pad(
            captions,
            (0, self.config.latent_dim - self.config.text_seq_length)
        ) # (B, max_seq_len) -> (B, latent_dim)

    # (B, latent_dim) -> (B, 1, latent_dim)
    captions = captions[:, None, :]

    # 获取两个采样
    sample_1 = self.get_one_sample(text_embeddings, captions)
    sample_2 = self.get_one_sample(text_embeddings, captions)

    gen_image_embeddings = torch.zeros(sample_1.shape)

    # 选择和文本嵌入点积更大的样本
    for i in range(gen_image_embeddings.shape[0]):
        if sample_1[i] @ t_emb[i] >= sample_2[i] @ t_emb[i]:
            gen_image_embeddings[i] = sample_1[i]
        else:
            gen_image_embeddings[i] = sample_2[i]
    # 预测的图片嵌入
    return gen_image_embeddings
```

19.2.3.8 先验模型完整代码

综合以上所有，扩散先验模型的代码看起来应该是这样的：

```python
class DiffusionPrior(nn.Module):
    def __init__(self, config):
        super().__init__()
        # 加载clip模型
        self.clip = CLIP(config).to(config.device)
        self.clip.load_state_dict(torch.load(
            config.clip.model_location,
            map_location=config.device
        ))
        freeze_model(self.clip)

        self.config = config
```

```
13
14          self.time_mlp = nn.Sequential(
15              SinusoidalPositionalEmbedding(
16                  config.prior.max_time,
17                  config.latent_dim
18              ),
19              nn.Linear(
20                  config.latent_dim,
21                  config.latent_dim * config.prior.r_mlp,
22                  bias=config.prior.bias
23              ),
24              nn.SiLU(),
25              nn.Linear(
26                  config.latent_dim * config.prior.r_mlp,
27                  config.latent_dim,
28                  bias=config.prior.bias
29              )
30          )
31
32          self.learned_embedding = nn.Parameter(
33              torch.randn(config.latent_dim)
34          )
35
36          self.schedule_values = get_schedule_values(config)
37
38          # Transformer blocks
39          self.decoder = nn.ModuleList(
40              [TransformerBlock(
41                  config.latent_dim,
42                  cond_width=config.latent_dim,
43                  n_heads=config.prior.n_heads,
44                  dropout=config.prior.dropout,
45                  r_mlp=config.prior.r_mlp,
46                  bias=config.prior.bias
47              ) for _ in range(config.prior.n_layers)]
48          )
49
50          # Output Projection，输出为预测的图片嵌入
51          self.output = nn.Sequential(
52              nn.LayerNorm(config.latent_dim),
53              nn.Linear(
54                  config.latent_dim,
55                  config.latent_dim,
56                  bias=config.decoder.bias
57              )
58          )
59
60          self.register_buffer(
61              "causal_attention_mask",
62              torch.tril(torch.ones(5, 5))[None, :]
```

```
        )

    def get_one_sample(self, text_embeddings, captions):
        # Get image embeddings that are pure noise
        noisy_image_embeddings = torch.randn(text_embeddings.shape,
device=self.config.device)

        # timestep is max for all items because image embeddings are pure noise
        timesteps = torch.full((captions.shape[0],), self.config.prior.max_time -
1)

        # Get timestep embeddings
        timestep_embeddings = self.time_mlp(timesteps) # (B, ) → (B, latent_dim)
        timestep_embeddings = timestep_embeddings[:, None, :] # (B, latent_dim) →
(B, 1, latent_dim)

        # Expand learned embedding so that there is one for each item in batch
        learned_embeddings = self.learned_embedding.repeat(captions.shape[0], 1) #
(latent_dim) → (B, latent_dim)
        learned_embeddings = learned_embeddings[:, None, :] # (B, latent_dim) →
(B, 1, latent_dim)

        tokens = torch.cat((
            captions,               # Image Caption
            text_embeddings,        # CLIP Text Embedding
            timestep_embeddings,    # Timestep Embedding
            noisy_image_embeddings,  # Noisy CLIP Image Embedding
            learned_embeddings      # Learned Embedding
        ), dim=1) # (B, 5, latent_dim)

        # Pass through transformer blocks with causal attention mask
        for block in self.decoder:
            tokens = block(tokens, mask=self.causal_attention_mask)

        # Get learned embeddings and pass through output projection to get CLIP
image embeddings
        pred_image_embeddings = self.output(tokens[:, -1, :])

        return pred_image_embeddings

    def sample(self, captions, masks=None):
        # Get CLIP text embeddings
        t_emb = self.clip.text_encoder(captions, mask=masks) # (B,
text_seq_length) → (B, latent_dim)
        text_embeddings = t_emb[:, None, :] # (B, latent_dim) → (B, 1,
latent_dim)

        # Make caption length equal to latent dimension
        if self.config.text_seq_length ⩾ self.config.latent_dim:
            captions = captions[:, :self.config.latent_dim]  # (B, max_seq_len) →
(B, latent_dim)
        else:
```

```
            captions = nn.functional.pad(captions, (0, self.config.latent_dim - self.config.text_seq_length)) # (B, max_seq_len) → (B, latent_dim)

        captions = captions[:, None, :] # (B, latent_dim) → (B, 1, latent_dim)

        # Getting two samples
        sample_1 = self.get_one_sample(text_embeddings, captions)
        sample_2 = self.get_one_sample(text_embeddings, captions)

        gen_image_embeddings = torch.zeros(sample_1.shape)

        # Choosing the samples with the higher dot product with text embeddings
        for i in range(gen_image_embeddings.shape[0]):
            if sample_1[i] @ t_emb[i] ⩾ sample_2[i] @ t_emb[i]:
                gen_image_embeddings[i] = sample_1[i]
            else:
                gen_image_embeddings[i] = sample_2[i]

        return gen_image_embeddings

    def forward(self, images, captions, masks=None):
        # Get CLIP image embeddings
        image_embeddings = self.clip.image_encoder(images) # (B, C, H, W) → (B, latent_dim)

        # Get CLIP text embeddings
        text_embeddings = self.clip.text_encoder(captions, mask=masks) # (B, text_seq_length) → (B, latent_dim)
        text_embeddings = text_embeddings[:, None, :] # (B, latent_dim) → (B, 1, latent_dim)

        # Make caption length equal to latent dimension
        if self.config.text_seq_length ⩾ self.config.latent_dim:
            captions = captions[:, :self.config.latent_dim]  # (B, max_seq_len) → (B, latent_dim)
        else:
            captions = nn.functional.pad(captions, (0, self.config.latent_dim - self.config.text_seq_length)) # (B, max_seq_len) → (B, latent_dim)

        captions = captions[:, None, :] # (B, latent_dim) → (B, 1, latent_dim)

        # Get random timesteps for forward diffusion
        timesteps = torch.randint(0, self.config.prior.max_time, (images.shape[0],)) # (B, )

        # Get timestep embeddings
        timestep_embeddings = self.time_mlp(timesteps) # (B, ) → (B, latent_dim)
        timestep_embeddings = timestep_embeddings[:, None, :] # (B, latent_dim) → (B, 1, latent_dim)

        # Perform forward diffusion to get noisy CLIP image embeddings
        noisy_image_embedding, _ = forward_diffusion(image_embeddings, self.schedule_values, timesteps)
```

```
150         noisy_image_embedding = noisy_image_embedding[:, None, :] # (B,
            latent_dim) → (B, 1, latent_dim)
151
152         # Expand learned embedding so that there is one for each item in batch
153         learned_embeddings = self.learned_embedding.repeat(images.shape[0], 1) #
            (latent_dim) → (B, latent_dim)
154         learned_embeddings = learned_embeddings[:, None, :] # (B, latent_dim) →
            (B, 1, latent_dim)
155
156         tokens = torch.cat((
157             captions,               # Image Caption
158             text_embeddings,        # CLIP Text Embedding
159             timestep_embeddings,    # Timestep Embedding
160             noisy_image_embedding,  # Noisy CLIP Image Embedding
161             learned_embeddings      # Learned Embedding
162         ), dim=1) # (B, 5, latent_dim)
163
164         # Pass through transformer blocks with causal attention mask
165         for block in self.decoder:
166             tokens = block(tokens, mask=self.causal_attention_mask)
167
168         # Get learned embeddings and pass through output projection to get CLIP
            image embeddings
169         pred_image_embeddings = self.output(tokens[:, -1, :])
170
171         loss = nn.functional.mse_loss(
172            pred_image_embeddings,
173            image_embeddings
174         )
175
176         return loss
```

19.3 条件扩散模型

> ⚡ 危险
>
> 训练用于生成图片的条件扩散模型时，CLIP 模型和先验模型都必须**冻结**！

19.3.1 概述

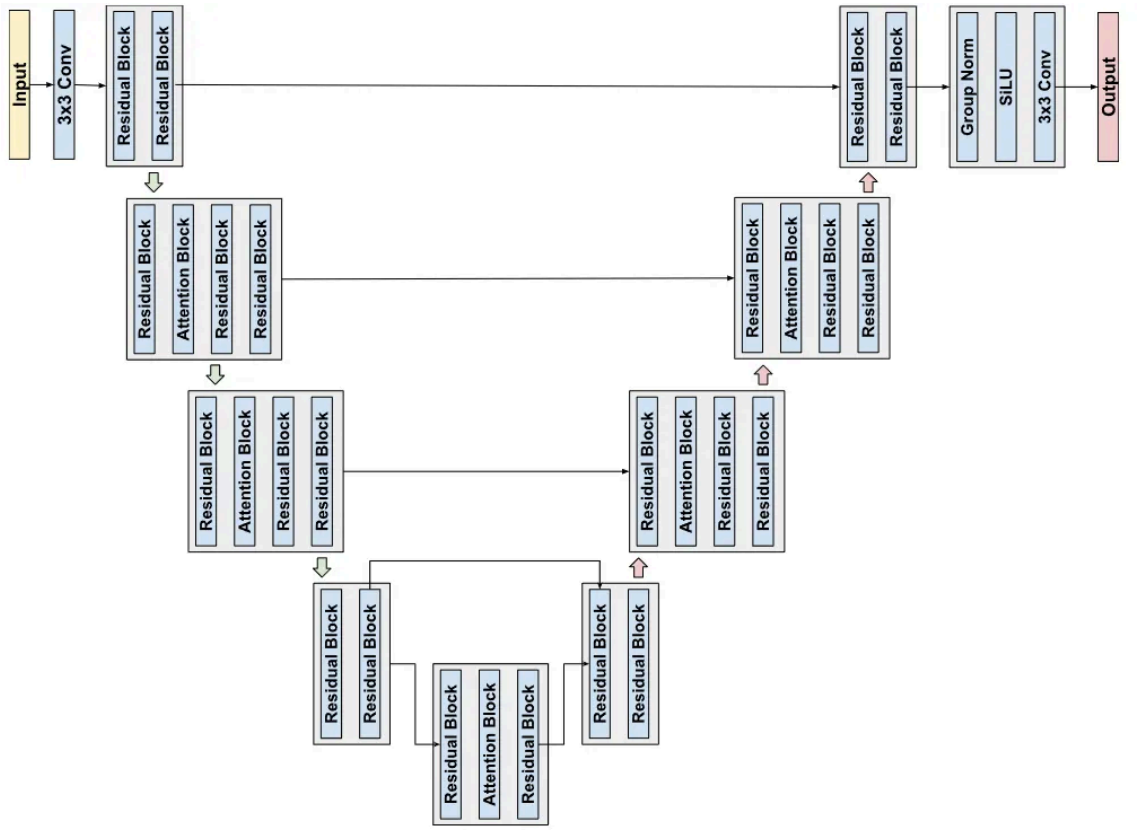


图 19.6 解码器架构

条件扩散模型是模型中生成图像的部分。它通过预测每个时间步应该去除的噪声，并迭代地从噪声图像中去除预测的噪声来实现这一点。

最常用作其主干的两个模型架构是 `UNet` 和 `Transformer`。

`Transformer` 架构的主要优势之一是其可扩展性，它可以很好地扩展至大型数据集和更复杂的模型。另一个优势是，虽然 `UNet` 主要用于图像，但 `Transformer` 更加灵活，只需进行修改即可用于其他数据类型。

`UNet` 是许多图像相关任务的主要选择。这是因为它擅长通过卷积层获取局部信息，同时通过跳跃连接保持高分辨率特征。`UNet` 的另一个优点是输入和输出的形状应该相同，这在图像扩散中非常有用。

`UNet` 的工作原理是，接收输入并将其传入编码器层，编码器层会识别/捕获相关特征，同时降低分辨率。然后，输入会传入解码器层，解码器层会尝试定位特征，同时将分辨率提升至原始形状。由于编码器层包含空间信息，因此会从编码器到解码器添加跳跃连接，以帮助保留这些信息。

19.3.2 残差块

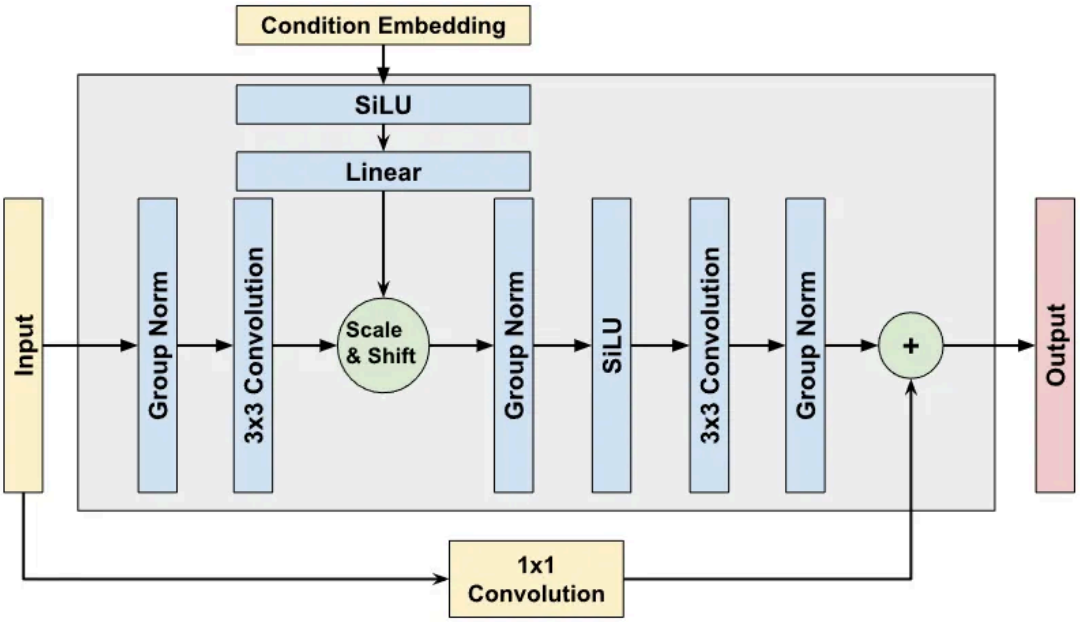


图 19.7 残差块的架构

编码器和解码器层均由残差块构成，这些残差块包含用于识别和定位特征的卷积。这些残差块通常由归一化、激活、卷积、归一化、激活、卷积组成；然而，我们的残差块将是归一化、卷积、归一化、激活、卷积、归一化（如图所示）。Han 等人的论文证明了这种架构可以在保持非线性的同时提升性能。

```python
# 初始化
self.layers1 = nn.Sequential(
    nn.GroupNorm(n_groups, d_in),
    nn.Conv2d(d_in, d_out, kernel_size, padding=1)
)

self.layers2 = nn.Sequential(
    nn.GroupNorm(n_groups, d_out),
    nn.SiLU(),
    nn.Dropout(p=dropout),
    nn.Conv2d(d_out, d_out, kernel_size, padding=1),
    nn.GroupNorm(n_groups, d_out)
)

# Forward
x = self.layers1(x_0)
x = self.layers2(x)
```

对于残差块的归一化部分，我们将使用 GroupNorm（组归一化）。之所以使用 GroupNorm 而不是 BatchNorm，是因为它的性能与批次大小无关，这使得它在小批次或可变批次大小上表现更佳。使用 GroupNorm 也有助于提高训练期间的稳定性。

残差块的激活部分用于实现非线性特性。虽然 ReLU 通常用于残差网络，但我们将使用 SiLU 作为激活函数。在 Ramachandran 等人的论文中，尽管模型和超参数是专门为 ReLU 设置的，但 SiLU

的表现优于 ReLU 和其他激活函数。由于 SiLU 的简单性以及与 ReLU 的相似性，因此只需用它来代替 ReLU 即可轻松实现。

对于卷积，我们将使用 3x3 核大小和 SAME 填充的 Conv2d 来保留空间维度。SAME 填充的工作原理是在输入的边界添加 0，以确保输出形状与未填充时的输入形状相同。第一个卷积会将输入从 d_in 通道投影到 d_out 通道，而第二个卷积则仅保留 d_out 通道。

残差块也将根据输入的嵌入信息进行调节。执行调节的主要方法之一是将嵌入添加到输入中。对于我们的模型，我们将执行线性投影以获得比例和偏差值。然后将输入乘以比例，然后添加偏差。Nichol 和 Dhariwal 表明，与加法相比，使用这种调节方法可以提高 FID 分数。编写此部分代码时需要注意的一点是，调节嵌入的维度可能比输入的维度少。因此，我们需要在嵌入的末尾添加维度。例如，如果输入的形状为(B, C, L)，而嵌入的形状为(B, C)，则需要添加一个维度以使其形状为(B, C, 1)。

```python
# 初始化
self.use_scale_shift = use_scale_shift

self.cond_layers = nn.Sequential(
    nn.SiLU(),
    nn.Linear(model_channels, d_out * 2 if use_scale_shift else d_out)
)

# Forward
emb = self.cond_layers(emb)

while len(emb.shape) < len(x.shape):
    emb = emb[..., None]

if self.use_scale_shift:
    y_s, y_b = emb.chunk(2, dim=1)
    x = y_s * x + y_b
else:
    x += emb
```

调节是在第一次卷积之后进行的，而不是在残差块的开始处进行的，因为它允许模型在使用时间步信息对其进行细化之前获取基本特征，从而获得更好的表示。

最后，通过将 ResNet 模块的原始输入添加到输出来创建跳跃连接。如果需要更改通道数以匹配输出，则对输入执行 1x1 卷积。这些跳跃连接用于增强特征传播。

```python
# 初始化
self.residual = nn.Conv2d(d_in, d_out, 1) if d_in != d_out else nn.Identity()

# Forward
x += self.residual(x_0)
```

将所有这些放在一起，残差块的最终代码应该是这样的：

```python
class ResidualBlock(nn.Module):
    def __init__(
        self,
        d_in,
        d_out,
        cond_channels=128,
        n_groups=8, # 组归一化GroupNorm每一组的数据量
```

```
        kernel_size=(3,3),
        dropout=0.0,
        use_scale_shift=True
    ):
        super().__init__()

        self.use_scale_shift = use_scale_shift

        self.layers1 = nn.Sequential(
            nn.GroupNorm(n_groups, d_in),
            nn.SiLU(),
            nn.Conv2d(d_in, d_out, kernel_size, padding=1)
        )

        # Activation & Linear Projection for Embedding
        self.cond_layers = nn.Sequential(
            nn.SiLU(),
            # d_out multiplied by 2 in order to split into scale & shift if
            necessary
            nn.Linear(cond_channels, d_out * 2 if use_scale_shift else d_out)
        )

        self.layers2 = nn.Sequential(
            nn.GroupNorm(n_groups, d_out),
            nn.SiLU(),
            nn.Dropout(p=dropout),
            nn.Conv2d(d_out, d_out, kernel_size, padding=1),
            nn.GroupNorm(n_groups, d_out)
        )

        self.residual = nn.Conv2d(d_in, d_out, 1) \
                        if d_in != d_out          \
                        else nn.Identity()

    def forward(self, x_0, emb):
        # x_0是原始图片，emb是图片的clip嵌入
        x = self.layers1(x_0)

        emb = self.cond_layers(emb)

        # Adding dimensions to embedding
        while len(emb.shape) < len(x.shape):
            emb = emb[..., None]

        # Conditioning input with embedding
        if self.use_scale_shift:
            # Getting scale and shift
            y_s, y_b = emb.chunk(2, dim=1)
            # Performing scale and shift
            x = y_s * x + y_b
```

```
        else:
            # Adding embedding to input
            x += emb

        x = self.layers2(x)

        # Skip Connection
        x += self.residual(x_0)

        return x
```

19.3.3 注意力块

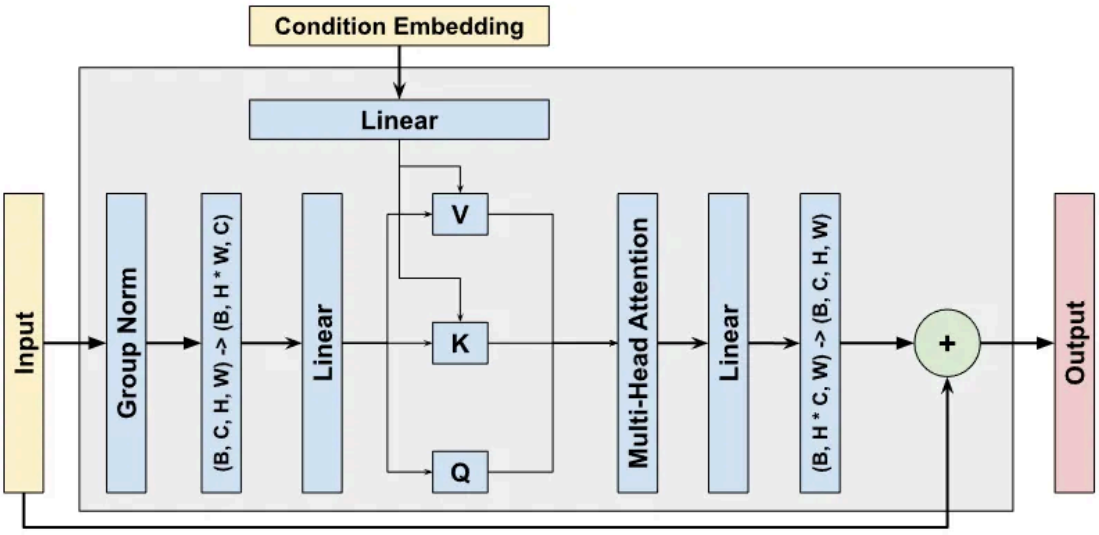


图 19.8 注意力块

在我们的模型中，我们将把注意力模块放置在编码器和解码器内层以及瓶颈层（bottle 层，连接编码器和解码器的底层）的残差模块之后。例如，如果编码器和解码器有四层，那么第二层和第三层就会有一个注意力模块。

使用注意力模块对我们的模型有诸多好处。其一是它有助于捕捉图像不同部分之间的空间关系信息。其二是它有助于衡量模型特征的重要性。最后，注意力模块还可以用来调节图像生成过程。

有多种方法可以为注意力模块实现注意力机制。其中一些选项包括自注意力和交叉注意力。我们将使用的方法是 GLIDE 模型中用到的两种方法的混合。对于这种方法，我们将从输入和条件信息中获取 K 和 V，并将它们连接在一起以获得最终的 K 和 V。

```
Q, K, V = self.qkv(x).chunk(3, dim=-1)

Q = Q.view(B, L, self.n_heads, self.head_size).transpose(1, 2)

K = K.view(B, L, self.n_heads, self.head_size).transpose(1, 2)

V = V.view(B, L, self.n_heads, self.head_size).transpose(1, 2)

# Concatenating keys and values of input and condition
if cond is not None:
```

```
    k_c, v_c = self.cond_kv(cond).chunk(2, dim=-1)
    k_c = k_c.view(B, cond.shape[1], self.n_heads, self.head_size).transpose(1, 2)
    v_c = v_c.view(B, cond.shape[1], self.n_heads, self.head_size).transpose(1, 2)
    K = torch.cat((K, k_c), dim=-2)
    V = torch.cat((V, v_c), dim=-2)
```

需要注意的是，输入图像的形状为`(B, C, H, W)`，但对于我们的注意力机制，我们希望其形状为`(B, L, C)`。因此，我们需要将 `H` 和 `W` 维度合并，并将其与 `C` 维度转置。执行注意力机制后，我们需要将输出转换回其原始形状。

```python
b, c, h, w = x_0.shape

# Changing shape to perform attention
x = x.permute(0, 2, 3, 1).view(b, h * w, c) # (B, C, H, W) -> (B, H * W, C)

# Attention
x = self.attention(x, cond)

# Changing back to original shape
x = x.view(b, h, w, c).permute(0, 3, 1, 2)
```

最终的代码看起来应该是这样的：

```python
class AttentionBlock(nn.Module):
    def __init__(self, n_channels, cond_channels, n_groups=8, n_heads=1, dropout=0.0):
        super().__init__()

        assert n_channels % n_heads == 0, "n_channels must be divisible by n_heads"

        self.n_heads = n_heads
        self.head_size = n_channels // n_heads
        self.scale = self.head_size ** -0.5

        self.group_norm = nn.GroupNorm(n_groups, n_channels)

        self.qkv = nn.Linear(n_channels, n_channels * 3)

        self.cond_kv = nn.Linear(cond_channels, n_channels * 2)

        self.out_proj = nn.Linear(n_channels, n_channels)

        self.dropout = nn.Dropout(dropout)

    def attention(self, x, cond=None):
        B, L, _ = x.shape

        # Getting queries, keys, and values for input
        Q, K, V = self.qkv(x).chunk(3, dim=-1)

        Q = Q.view(B, L, self.n_heads, self.head_size).transpose(1, 2)

```

```
        K = K.view(B, L, self.n_heads, self.head_size).transpose(1, 2)

        V = V.view(B, L, self.n_heads, self.head_size).transpose(1, 2)

        # Concatenating keys and values of condition to keys and values of input
        if cond is not None:
            k_c, v_c = self.cond_kv(cond).chunk(2, dim=-1)
            k_c = k_c.view(B, cond.shape[1], self.n_heads,
            self.head_size).transpose(1, 2)
            v_c = v_c.view(B, cond.shape[1], self.n_heads,
            self.head_size).transpose(1, 2)
            K = torch.cat((K, k_c), dim=-2)
            V = torch.cat((V, v_c), dim=-2)

        # Get dot product between queries and keys
        attention = torch.matmul(Q, K.transpose(-2, -1))

        # Scale
        attention = attention * self.scale

        # Applying softmax
        attention = torch.softmax(attention, dim=-1)

        # Get dot product with values
        attention = torch.matmul(attention, V)

        # Combine heads
        attention = attention.transpose(1, 2)
        attention = attention.contiguous().view(x.shape)

        # Output projection
        attention = self.out_proj(attention)

        # Dropout
        attention = self.dropout(attention)

        return attention

    def forward(self, x_0, cond=None):
        b, c, h, w = x_0.shape

        # Group normalization
        x = self.group_norm(x_0)

        # Changing shape to perform attention
        x = x.permute(0, 2, 3, 1).view(b, h * w, c) # (B, C, H, W) → (B, H * W, C)

        # Attention
        x = self.attention(x, cond)

        # Changing back to original shape
```

```
        x = x.view(b, h, w, c).permute(0, 3, 1, 2)

        # Residual connection
        x = x + x_0

        return x
```

19.3.4 下采样

在每个编码器层之间，我们需要降低输入的分辨率。主要的下采样方法有两种：池化和使用卷积层。池化无需参数，这使得它的计算效率更高，并且有助于防止过拟合。卷积层的优势在于它本身带有参数，这使得它能够学习并保留重要的特征。我们将在模型中使用步长 2，这将使输入的分辨率降低 2 倍。

我们将对模型进行编码，以便能够使用任一方法，但是在训练模型时，我们将使用卷积方法。

```python
class Downsample(nn.Module):
    def __init__(self, n_channels, kernel_size=(3,3), stride=2, down_pool=False):
        super().__init__()

        if down_pool:
            self.down = nn.AvgPool2d(stride)
        else:
            self.down = nn.Conv2d(n_channels, n_channels, kernel_size,
            stride=stride, padding=1)

    def forward(self, x):
        x = self.down(x)
        return x
```

19.3.5 上采样

在每个解码器层之间，我们需要提高输入的分辨率。为此，我们首先进行插值，将输入的高度和宽度乘以 2。之后，我们将它传入一个卷积层，该层将学习并保留重要特征，同时确保通道数正确。

```python
class Upsample(nn.Module):
    def __init__(self, d_in, d_out, kernel_size=(3,3)):
        super().__init__()

        self.conv = nn.Conv2d(d_in, d_out, kernel_size, padding=1)

    def forward(self, x):
        # interpolate：插值
        x = nn.functional.interpolate(x, scale_factor=2)
        x = self.conv(x)
        return x
```

19.3.6 模型的训练

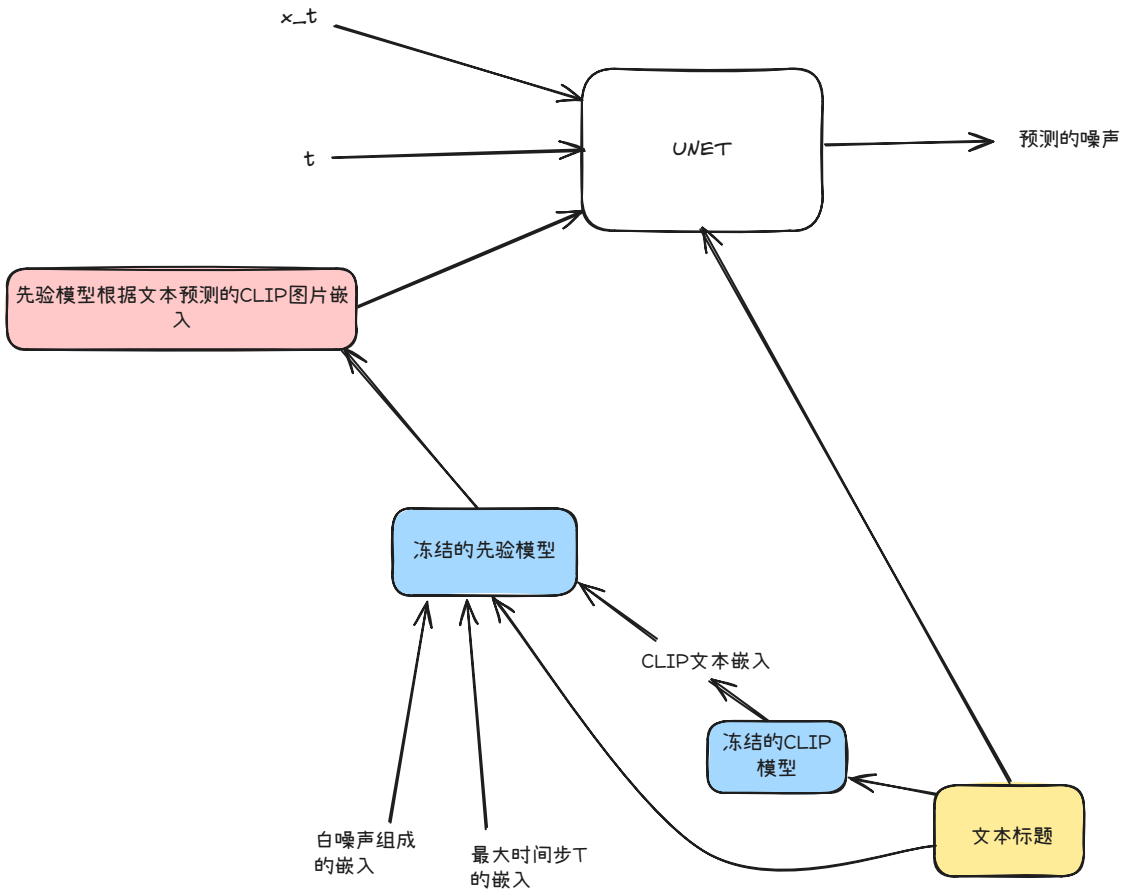


图 19.9 解码器训练流程

在训练解码器时，模型需要做的第一件事就是设置条件信息。为此，我们首先需要从先验模型中采样以获取 CLIP 图像嵌入。与 CLIP 模型一样，在加载先验模型时，应冻结层并将模式设置为 eval。

```python
# 初始化先验网络
self.prior = DiffusionPrior(config).to(config.device)
self.prior.load_state_dict(torch.load(
    config.prior.model_location,
    map_location=config.device
))
# 冻结模型
freeze_model(self.prior)

# Forward
# 得到预测的图片clip嵌入
img_embeddings = self.prior.sample(caption, mask).to(x.device)
```

之后，需要使用输入的时间步来获取时间步嵌入。然后将先验模型生成的 CLIP 图像嵌入投影并添加到时间步嵌入中。这些嵌入将用于调节模型的残差块。

```python
# 投影层，将clip图片嵌入转换成和时间步嵌入相同的形状，方便相加
self.img_projection = nn.Sequential(
```

```
    nn.Linear(config.latent_dim, config.decoder.cond_channels),
    nn.SiLU(),
    nn.Linear(config.decoder.cond_channels, config.decoder.cond_channels)
)

#Forward
# 条件嵌入 = 时间步嵌入 + clip图片嵌入
c_emb = self.time_mlp(time) + self.img_projection(img_embeddings)

for module in self.encoder:
    if isinstance(module, ResidualBlock):
        x = module(x, c_emb)
```

对于注意力条件信息，我们首先需要将文本标题传入文本 `Transformer` 编码器。在 `unCLIP` 论文中，之所以使用这些文本编码，是因为 `Ramesh` 等人认为这有助于学习 `CLIP` 无法学习的自然语言知识。但在测试过程中，他们发现这种方法效果不佳，因此这部分是可选的。

Python

```
# Constructor
self.text_embedding = nn.Embedding(config.vocab_size, config.latent_dim)
self.positional_encodings =
nn.Parameter(torch.randn(config.text_seq_length,config.latent_dim) *
(config.latent_dim ** -0.5))

self.text_encoder = nn.ModuleList(
    [TransformerBlock(
        config.latent_dim,
        cond_width=config.latent_dim,
        n_heads=config.decoder.n_heads,
        dropout=config.decoder.dropout,
        r_mlp=config.decoder.r_mlp,
        bias=config.decoder.bias
     ) for _ in range(config.decoder.text_layers)]
)

self.final_ln = nn.LayerNorm(config.latent_dim)

# Function
def encode_text(self, text, mask=None):
    x = self.text_embedding(text)

    x = x + self.positional_encodings

    for block in self.text_encoder:
        x = block(x, mask=mask)

    x = self.final_ln(x)

    return x

# Forward
text_encodings = self.encode_text(text, mask)
```

对文本进行编码后，CLIP 图像嵌入被投影到四个额外的 token 中，并连接到编码文本的末尾。

```python
# Constructor
self.get_img_tokens = nn.Linear(1, config.decoder.n_img_tokens)

# Forward
img_tokens = self.get_img_tokens(img_embeddings[..., None]).permute(0, 2, 1)

c_attn = torch.cat([text_encodings, img_tokens], dim=1)
```

设置条件信息后，噪声图像将通过初始卷积层，以获得初始模型通道的通道数。

```python
# Constructor
ch = config.decoder.model_channels

self.in_conv = nn.Conv2d(
    config.img_channels,
    ch,
    config.decoder.kernel_size,
    padding=1
)

# Forward
x = self.in_conv(x)
```

现在，含噪图像已达到所需的通道数，它们可以与条件信息一起通过 UNet 层。UNet 将包含四个编码器层和解码器层，中间有一个瓶颈层。对于每层的通道数，我们将使用[1, 2, 4, 8]作为层通道数与模型通道数的比率。UNet 的编码器、解码器和瓶颈层都将包含两个残差块。在瓶颈残差块之间以及编码器和解码器内层的每个残差块之后，放置了注意力块。跳跃连接连接位于编码器和解码器的残差块之间。

```python
# Config
model_channels:int = 32
channel_ratios:list[int] = field(default_factory=lambda: [1, 2, 4, 8])
n_layer_blocks:int = 2

# Constructor
# UNet Encoder Layers
self.encoder = nn.ModuleList([])
for r in config.decoder.channel_ratios:
    for _ in range(config.decoder.n_layer_blocks):
        self.encoder.append(ResidualBlock(ch, config.decoder.model_channels * r,
        config.decoder.cond_channels, config.decoder.n_groups,
        config.decoder.kernel_size, config.decoder.dropout,
        config.decoder.use_scale_shift))

        ch = config.decoder.model_channels * r
        if r != config.decoder.channel_ratios[0] and r !=
        config.decoder.channel_ratios[-1]:
            self.encoder.append(AttentionBlock(ch, config.latent_dim,
            config.decoder.n_groups, config.decoder.n_heads,
            config.decoder.dropout))

    if r != config.decoder.channel_ratios[-1]:
```

```
        self.encoder.append(Downsample(ch, config.decoder.kernel_size, config.decoder.stride, config.decoder.down_pool))

# UNet Bottleneck Layers
self.bottleneck = nn.ModuleList([])
for block in range(config.decoder.n_layer_blocks):
    self.bottleneck.append(ResidualBlock(ch, ch, config.decoder.cond_channels, config.decoder.n_groups, config.decoder.kernel_size, config.decoder.dropout, config.decoder.use_scale_shift))

    if block != config.decoder.n_layer_blocks - 1:
        self.bottleneck.append(AttentionBlock(ch, config.latent_dim, config.decoder.n_groups, config.decoder.n_heads, config.decoder.dropout))

# UNet Decoder Layers
self.decoder = nn.ModuleList([])
for r in range(len(config.decoder.channel_ratios))[::-1]:
    for _ in range(config.decoder.n_layer_blocks):
        self.decoder.append(ResidualBlock(ch * 2, ch, config.decoder.cond_channels, config.decoder.n_groups, config.decoder.kernel_size, config.decoder.dropout, config.decoder.use_scale_shift))

        if r != 0 and r!= len(config.decoder.channel_ratios) - 1:
            self.decoder.append(AttentionBlock(ch, config.latent_dim, config.decoder.n_groups, config.decoder.n_heads, config.decoder.dropout))

    if r != 0:
        ch = config.decoder.model_channels * config.decoder.channel_ratios[r-1]
        self.decoder.append(Upsample(config.decoder.model_channels * config.decoder.channel_ratios[r], ch, config.decoder.kernel_size))

# Forward
for module in self.encoder:
    if isinstance(module, ResidualBlock):
        x = module(x, c_emb)
        self.connections.append(x)
    elif isinstance(module, AttentionBlock):
        x = module(x, cond=c_attn)
    else:
        x = module(x)

for module in self.bottleneck:
    if isinstance(module, ResidualBlock):
        x = module(x, c_emb)
    elif isinstance(module, AttentionBlock):
        x = module(x, cond=c_attn)
    else:
        x = module(x)

for module in self.decoder:
    if isinstance(module, ResidualBlock):
        x = torch.cat([x, self.connections.pop()], dim=1)
```

```
            x = module(x, c_emb)
        elif isinstance(module, AttentionBlock):
            x = module(x, cond=c_attn)
        else:
            x = module(x)
```

然后，UNet 解码器层的输出会经过 GroupNorm 和 SiLU 激活层，之后使用 Conv2d 让输出恢复到原始通道数。

```python
# Constructor
self.output = nn.Sequential(
    nn.GroupNorm(config.decoder.n_groups, config.decoder.model_channels),
    nn.SiLU(),
    nn.Conv2d(config.decoder.model_channels, config.img_channels,
    config.decoder.kernel_size, padding=1)
)

# Forward
x = self.output(x)
```

将所有内容放在一起，解码器模型的最终代码将如下所示：

```python
class Decoder(nn.Module):
    def __init__(self, config):
        super().__init__()
        # Loading Prior Model
        self.prior = DiffusionPrior(config).to(config.device)
        self.prior.load_state_dict(torch.load(
            config.prior.model_location,
            map_location=config.device
        ))
        freeze_model(self.prior)

        # MLP to get time embeddings
        self.time_mlp = nn.Sequential(
            SinusoidalPositionalEmbedding(
                config.decoder.max_time,
                config.decoder.model_channels
            ),
            nn.Linear(
                config.decoder.model_channels,
                config.decoder.cond_channels
            ),
            nn.SiLU(),
            nn.Linear(
                config.decoder.cond_channels,
                config.decoder.cond_channels
            )
        )

        # MLP to project CLIP image embeddings
        self.img_projection = nn.Sequential(
```

```
            nn.Linear(config.latent_dim, config.decoder.cond_channels),
            nn.SiLU(),
            nn.Linear(config.decoder.cond_channels, config.decoder.cond_channels)
        )

        # Projection to get image tokens
        self.get_img_tokens = nn.Linear(1, config.decoder.n_img_tokens)

        # Embedding layer for text captions
        self.text_embedding = nn.Embedding(config.vocab_size, config.latent_dim)

        # Learned positional encodings for text captions
        self.positional_encodings = nn.Parameter(torch.randn(
            config.text_seq_length,
            config.latent_dim
        ) * (config.latent_dim ** -0.5))

        # Transformer encoder blocks to encoder text captions
        self.text_encoder = nn.ModuleList(
            [TransformerBlock(
                config.latent_dim,
                cond_width=config.latent_dim,
                n_heads=config.decoder.n_heads,
                dropout=config.decoder.dropout,
                r_mlp=config.decoder.r_mlp,
                bias=config.decoder.bias
            ) for _ in range(config.decoder.text_layers)]
        )

        self.final_ln = nn.LayerNorm(config.latent_dim)

        ch = config.decoder.model_channels

        # Initial convolution
        self.in_conv = nn.Conv2d(
            config.img_channels,
            ch,
            config.decoder.kernel_size,
            padding=1
        )

        # UNet Encoder Layers
        self.encoder = nn.ModuleList([])
        for r in config.decoder.channel_ratios:
            for _ in range(config.decoder.n_layer_blocks):
                self.encoder.append(ResidualBlock(
                    ch,
                    config.decoder.model_channels * r,
                    config.decoder.cond_channels,
                    config.decoder.n_groups,
```

```
                    config.decoder.kernel_size,
                    config.decoder.dropout,
                    config.decoder.use_scale_shift
                ))

                ch = config.decoder.model_channels * r
                if r != config.decoder.channel_ratios[0] \
                    and r != config.decoder.channel_ratios[-1]:
                    self.encoder.append(AttentionBlock(
                        ch,
                        config.latent_dim,
                        config.decoder.n_groups,
                        config.decoder.n_heads,
                        config.decoder.dropout
                    ))

            if r != config.decoder.channel_ratios[-1]:
                self.encoder.append(Downsample(
                    ch,
                    config.decoder.kernel_size,
                    config.decoder.stride,
                    config.decoder.down_pool
                ))

        # UNet Bottleneck Layers
        self.bottleneck = nn.ModuleList([])
        for block in range(config.decoder.n_layer_blocks):
            self.bottleneck.append(ResidualBlock(
                ch,
                ch,
                config.decoder.cond_channels,
                config.decoder.n_groups,
                config.decoder.kernel_size,
                config.decoder.dropout,
                config.decoder.use_scale_shift
            ))

            if block != config.decoder.n_layer_blocks - 1:
                self.bottleneck.append(AttentionBlock(
                    ch,
                    config.latent_dim,
                    config.decoder.n_groups,
                    config.decoder.n_heads,
                    config.decoder.dropout
                ))

        # UNet Decoder Layers
        self.decoder = nn.ModuleList([])
        for r in range(len(config.decoder.channel_ratios))[::-1]:
            for _ in range(config.decoder.n_layer_blocks):
```

```
                self.decoder.append(ResidualBlock(
                    ch * 2,
                    ch,
                    config.decoder.cond_channels,
                    config.decoder.n_groups,
                    config.decoder.kernel_size,
                    config.decoder.dropout,
                    config.decoder.use_scale_shift
                ))

                if r != 0 and r!= len(config.decoder.channel_ratios) - 1:
                    self.decoder.append(AttentionBlock(
                        ch,
                        config.latent_dim,
                        config.decoder.n_groups,
                        config.decoder.n_heads,
                        config.decoder.dropout
                    ))

            if r != 0:
                ch = config.decoder.model_channels \
                   * config.decoder.channel_ratios[r-1]
                self.decoder.append(Upsample(
                    config.decoder.model_channels \
                        * config.decoder.channel_ratios[r],
                    ch,
                    config.decoder.kernel_size
                ))

        # Output projection
        self.output = nn.Sequential(
            nn.GroupNorm(
                config.decoder.n_groups,
                config.decoder.model_channels
            ),
            nn.SiLU(),
            nn.Conv2d(
                config.decoder.model_channels,
                config.img_channels,
                config.decoder.kernel_size,
                padding=1
            )
        )

        # Skip connections
        self.connections = []

    def encode_text(self, text, mask=None):
        x = self.text_embedding(text)

```

```
        x = x + self.positional_encodings

        for block in self.text_encoder:
            x = block(x, mask=mask)

        x = self.final_ln(x)

        return x

    def forward(self, x, time, caption=None, mask=None):
        # Sample prior model to get CLIP image embeddings
        # 使用文本标题输入给先验模型prior model，获得预测的clip图片嵌入
        img_embeddings = self.prior.sample(caption, mask).to(x.device)

        # 条件嵌入 = 时间步嵌入 + clip图片嵌入
        c_emb = self.time_mlp(time) + self.img_projection(img_embeddings)

        # Get conditioning information for attention blocks
        c_attn = self.get_img_tokens(img_embeddings[..., None]).permute(0, 2, 1)
        # 图片标题为空，那么unet是一个无条件的扩散模型
        if caption is not None:
            c_attn = torch.cat([self.encode_text(caption, mask), c_attn], dim=1)

        # Initial convolution
        x = self.in_conv(x)

        # UNet encoder layers
        for module in self.encoder:
            if isinstance(module, ResidualBlock):
                x = module(x, c_emb)
                self.connections.append(x)
            elif isinstance(module, AttentionBlock):
                x = module(x, cond=c_attn)
            else:
                x = module(x)

        # UNet bottleneck layers
        for module in self.bottleneck:
            if isinstance(module, ResidualBlock):
                x = module(x, c_emb)
            elif isinstance(module, AttentionBlock):
                x = module(x, cond=c_attn)
            else:
                x = module(x)

        # UNet decoder layers
        for module in self.decoder:
            if isinstance(module, ResidualBlock):
                x = torch.cat([x, self.connections.pop()], dim=1)
                x = module(x, c_emb)
```

```
231             elif isinstance(module, AttentionBlock):
232                 x = module(x, cond=c_attn)
233             else:
234                 x = module(x)
235 
236         # Output projection
237         x = self.output(x)
238 
239         return x
```

19.3.7 损失的计算

训练解码器模型的一个重要部分是损失函数。我们的损失函数首先要做的就是对批次中的每个图像进行随机时间步的采样。

```python
timesteps = torch.randint(
    0,
    config.decoder.max_time,
    (image.shape[0],),
    device=config.device,
    dtype=torch.long
)
```

然后使用这些时间步和方差计划值来获取通过前向扩散添加的噪声图像和噪声。

```python
# 方差计划
schedule_values = get_schedule_values(config)
# x_t, 真实的噪声noise
noisy_image, noise = forward_diffusion(
    image,
    schedule_values,
    timesteps
)
```

然后将噪声图像、时间步、文本标题和文本掩码传递到模型中以获得预测噪声。

```python
# x_t, 时间步, 图片标题（最终被处理为条件信息，clip图片嵌入）
pred_noise = decoder(noisy_image, timesteps, caption, mask)
```

最后，我们可以使用模型的预测噪声和前向扩散函数的实际噪声来计算损失。我们将用于解码器的损失是预测噪声和实际噪声之间的均方误差损失。

```python
# 预测的噪声和真实噪声之间计算均方误差
loss = nn.functional.mse_loss(pred_noise, noise)
```

综合起来，我们得到：

```python
# Calculating Loss
get_schedule_values(config)
timesteps = torch.randint(0, config.decoder.max_time, (image.shape[0],), device=config.device, dtype=torch.long)
noisy_image, noise = forward_diffusion(image, schedule_values, timesteps)
pred_noise = decoder(noisy_image, timesteps, caption, mask)
loss = nn.functional.mse_loss(pred_noise, noise)
```

19.4 训练配置

```python
@dataclass
class CLIPConfig:
    # Vision Transformer
    patch_size:tuple[int,int] = (4,4)
    vit_width:int = 256
    vit_layers:int = 6
    vit_heads:int = 8
    # Text Transformer
    text_width:int = 256
    text_layers:int = 6
    text_heads:int = 8
    # Attention
    dropout:float = 0.2
    r_mlp:int = 4
    bias:bool = False
    # Training
    augment_data:bool = True
    num_workers:int = 0
    batch_size:int = 128
    lr:float = 5e-4
    lr_min:float = 1e-5
    weight_decay:float = 1e-4
    epochs:int = 200
    warmup_epochs:int = 5
    grad_max_norm:float = 1.0
    get_val_accuracy:bool = False
    model_location:str = "../clip_fmnist.pt"

@dataclass
class PriorConfig:
    # Diffusion
    max_time:int = 1000
    schedule:str = "cosine"
    schedule_offset:float = 0.008
    # Transformer Decoder
    width:int = 256
    n_layers:int = 6
    n_heads:int = 8
    # Attention
    dropout:float = 0.2
    r_mlp:int = 4
    bias:bool = False
    # Training
    augment_data:bool = False
    num_workers:int = 0
    batch_size:int = 128
    lr:float = 5e-4
    lr_min:float = 1e-5
```

```
    weight_decay:float = 1e-4
    epochs:int = 150
    warmup_epochs:int = 5
    grad_max_norm:float = 1.0
    model_location:str = "../prior_fmnist.pt"

@dataclass
class DecoderConfig:
    # Diffusion
    max_time:int = 1000
    schedule:str = "cosine"
    # UNet
    n_groups:int = 8
    kernel_size:tuple[int, int] = (3,3)
    model_channels:int = 32
    cond_channels:int = 128
    channel_ratios:list[int] = field(default_factory=lambda: [1, 2, 4, 8])
    n_layer_blocks:int = 2
    dropout:float = 0.1
    use_scale_shift:bool = True
    n_heads:int = 8
    stride:int = 2
    down_pool:bool = False
    r_mlp:int = 4
    bias:bool = False
    text_layers:int = 4
    n_img_tokens:int = 4
    # Training
    augment_data:bool = False
    num_workers:int = 0
    batch_size:int = 32
    lr:float = 5e-4
    lr_min:float = 1e-5
    weight_decay:float = 1e-4
    epochs:int = 100
    warmup_epochs:int = 5
    grad_max_norm:float = 1.0
    sample_after_epoch:bool = True
    model_location:str = "../decoder_fmnist.pt"

@dataclass
class FMNISTConfig:
    latent_dim:int = 256
    # Dataset Info
    dataset:str = "fashion_mnist"
    data_location:str = "./datasets"
    img_size:tuple[int,int] = (32,32)
    img_channels:int = 1
    vocab_size:int = 256
    text_seq_length:int = 64
```

```
    # Data Augmentation / Normalization
    prob_hflip:float = 0.5
    crop_padding:int = 4
    train_mean:list[float] = field(default_factory=lambda: [0.2855552])
    train_std:list[float] = field(default_factory=lambda: [0.33848408])
    # Training
    train_val_split:tuple[int,int] = (50000, 10000)
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    # Model Configs
    clip = CLIPConfig()
    prior = PriorConfig()
    decoder = DecoderConfig()
```

19.5 训练

对于这个文本到图像的扩散模型，我们实际上需要训练三个独立的模型：CLIP、扩散先验模型和扩散解码器 UNET。包含这三个训练脚本的所有代码会非常多。

对于数据集，所有图像的尺寸都从(28,28)调整为(32,32)。图像也进行了归一化，平均值和标准差设置为训练样本的平均值和标准差。

Python
```
transform = T.Compose([
    T.Resize(config.img_size),
    T.ToTensor(),
    T.Normalize(config.train_mean, config.train_std)
])
```

对于 CLIP 模型的训练集，我还通过随机水平翻转和随机裁剪实现了训练分割的数据增强。

Python
```
transform = T.Compose([
    T.Resize(config.img_size),
    T.RandomHorizontalFlip(p=config.prob_hflip)
    T.RandomCrop(config.img_size[0], padding=config.crop_padding)
    T.ToTensor(),
    T.Normalize(config.train_mean, config.train_std)
])
```

在训练这三个模型时，当使用权重衰减时，我使用 AdamW 作为优化器，否则使用 Adam。

Python
```
if config.clip.weight_decay == 0:
    optimizer = Adam(clip.parameters(), lr=config.clip.lr)
else:
    optimizer = AdamW(clip.parameters(), lr=config.clip.lr,
    weight_decay=config.clip.weight_decay)
```

对于所有模型，我还使用了带有线性预热的余弦退火学习率调度器。该调度器在每个 epoch 结束时更新。

Python
```
if config.clip.warmup_epochs > 0:
    warmup = lr_scheduler.LinearLR(optimizer=optimizer, start_factor=(1 /
    config.clip.warmup_epochs), end_factor=1.0,
    total_iters=(config.clip.warmup_epochs - 1), last_epoch=-1)

scheduler = lr_scheduler.CosineAnnealingLR(optimizer, T_max=(config.clip.epochs -
config.clip.warmup_epochs), eta_min=config.clip.lr_min)
```

所有模型还使用了最大范数为 1.0 的梯度剪裁，以帮助防止梯度爆炸。

```python
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=config.[model].grad_max_norm)
```

在训练 CLIP 模型时，训练数据被随机分成训练集和验证集。当某个 epoch 的验证损失小于或等于前一个最低验证损失时，模型权重会被保存。

采样过程的第一步是随机生成仅包含噪声的图像。

```python
B = prompts.shape[0]
# Get completely noisy image
img = torch.randn((B, config.img_channels, config.img_size[0], config.img_size[1]), device=config.device)
```

得到噪声图像后，我们将计算一些需要计算 x_{t-1} 的时间表值。

```python
def get_schedule_values(config):
        schedule_values = {}
        schedule_values["betas"] = get_beta_schedule(config.decoder.schedule, config.decoder.max_time).to(config.device)
        schedule_values["alphas"] = 1.0 - schedule_values["betas"]
        schedule_values["alpha_bars"] = torch.cumprod(schedule_values["alphas"], axis = 0)
        schedule_values["sqrt_recip_alphas"] = torch.sqrt(1.0 / schedule_values["alphas"])
        schedule_values["sqrt_alpha_bars"] = torch.sqrt(schedule_values["alpha_bars"])
        schedule_values["sqrt_one_minus_alpha_bars"] = torch.sqrt(1.0 - schedule_values["alpha_bars"])
        schedule_values["alpha_bars_prev"] = torch.cat((torch.ones(1, device=config.device), schedule_values["alpha_bars"][:-1]))
        schedule_values["sigma"] = schedule_values["betas"] * (1.0 - schedule_values["alpha_bars_prev"]) / (1.0 - schedule_values["alpha_bars"])
        return schedule_values

schedule_values = get_schedule_values(config)
```

然后，我们需要以相反的顺序迭代遍历所有时间步，从 0 到 max_time-1。我们还需要扩展时间步，以便批次中的每条数据都有一个时间步。

```python
for t in range(0, config.max_time)[::-1]:
    timesteps = torch.full((B,), t, device=config.device, dtype=torch.long)
```

对于每个时间步的采样，第一步是获取该时间步的调度值。获取该时间步的调度值后，需要对其进行扩展，使其维度数等于图像的维度数。

```python
# Getting schedule values for timestep
sqrt_recip_alphas_t = extract_and_expand(schedule_values["sqrt_recip_alphas"], timesteps, img.shape)
betas_t = extract_and_expand(schedule_values["betas"], timesteps, img.shape)
sqrt_one_minus_alpha_bars_t = extract_and_expand(schedule_values["sqrt_one_minus_alpha_bars"], timesteps, img.shape)
sigma_t = extract_and_expand(schedule_values["sigma"], timesteps, img.shape)
```

之后，我们使用解码器模型来预测时间步t的噪声。

```python
# Predicting noise at timestep t with decoder
pred_noise = decoder(img, timesteps, caption=prompt, mask=mask)
```

如果不是最终时间步，我们还需要生成随机噪声z。我们需要这个值和σ_t，因为我们的模型直接预测从x_t到x_0的噪声，而不是从x_t到x_{t-1}的噪声。因此，模型会预测从x_t到x_0的噪声，并将从x_{t-1}到x_0的噪声加回去。这样一来，只有从x_t到x_{t-1}的噪声被移除。最后一步不需要σ_t和z值，因为从x_t到x_0的预测噪声与从x_t到x_{t-1}的预测噪声相同。

```python
# Generating random noise
z = torch.randn_like(img) if t > 0 else 0
```

利用生成的噪声、方差计划值和预测噪声，我们可以计算时间步 `t-1` 的图像。

```python
# Calculating image at timestep t-1
img = sqrt_recip_alphas_t * (img - (betas_t / sqrt_one_minus_alpha_bars_t) * pred_noise) + (sigma_t * z)
img = torch.clamp(img, -1.0, 1.0)
```

综合起来，采样和图像的代码将如下所示：

```python
@torch.no_grad()
def sample_image(config, prompt, mask, schedule_values=None):
    # Load decoder model
    decoder = Decoder(config).to(config.device)
    decoder.load_state_dict(torch.load(config.decoder.model_location, map_location=config.device))
    decoder.eval()

    B = prompt.shape[0]
    # Get completely noisy image
    img = torch.randn((B, config.img_channels, config.img_size[0], config.img_size[1]), device=config.device)

    # Calculate schedule values
    if schedule_values is None:
        schedule_values = get_schedule_values(config)

    for t in range(0, config.decoder.max_time)[::-1]:
        # Setting the timesteps for all the items in the batch
        timesteps = torch.full((B,), t, device=config.device, dtype=torch.long)

        # Getting schedule values for timestep
        sqrt_recip_alphas_t = extract_and_expand(schedule_values["sqrt_recip_alphas"], timesteps, img.shape)
        betas_t = extract_and_expand(schedule_values["betas"], timesteps, img.shape)
        sqrt_one_minus_alpha_bars_t = extract_and_expand(schedule_values["sqrt_one_minus_alpha_bars"], timesteps, img.shape)
        sigma_t = extract_and_expand(schedule_values["sigma"], timesteps, img.shape)

        # Predicting noise at timestep t with decoder
        pred_noise = decoder(img, timesteps, caption=prompt, mask=mask)

        # Generating random noise
        z = torch.randn_like(img) if t > 0 else 0
```

```
31
32          # Calculating image at timestep t-1
33          img = sqrt_recip_alphas_t * (img - (betas_t / sqrt_one_minus_alpha_bars_t)
            * pred_noise) + (sigma_t * z)
34
35          img = torch.clamp(img, -1.0, 1.0)
36
37      return img
```

19.6 训练结果

为了查看模型的结果，我将展示每个标题的反向扩散过程。为了查看这一点，我们创建了一个修改版的 sample_image 函数，用于绘制反向扩散过程中 10 个时间步的图像。

```python
# Displaying Results
config = FMNISTConfig()
captions = {
    0: "An image of a t-shirt/top",
    1: "An image of trousers",
    2: "An image of a pullover",
    3: "An image of a dress",
    4: "An image of a coat",
    5: "An image of a sandal",
    6: "An image of a shirt",
    7: "An image of a sneaker",
    8: "An image of a bag",
    9: "An image of an ankle boot"
}
sample_captions = torch.stack([tokenizer(x, text_seq_length=config.text_seq_length)
[0] for x in captions.values()]).to(config.device)
sample_masks = torch.stack([tokenizer(x, text_seq_length=config.text_seq_length)[1]
for x in captions.values()]).to(config.device)
for i in range(len(sample_captions)):
    caption = sample_captions[None, (i % len(sample_captions))]
    mask = sample_masks[None, (i % len(sample_masks))]
    test = sample_plot_image(config, caption, mask)
```

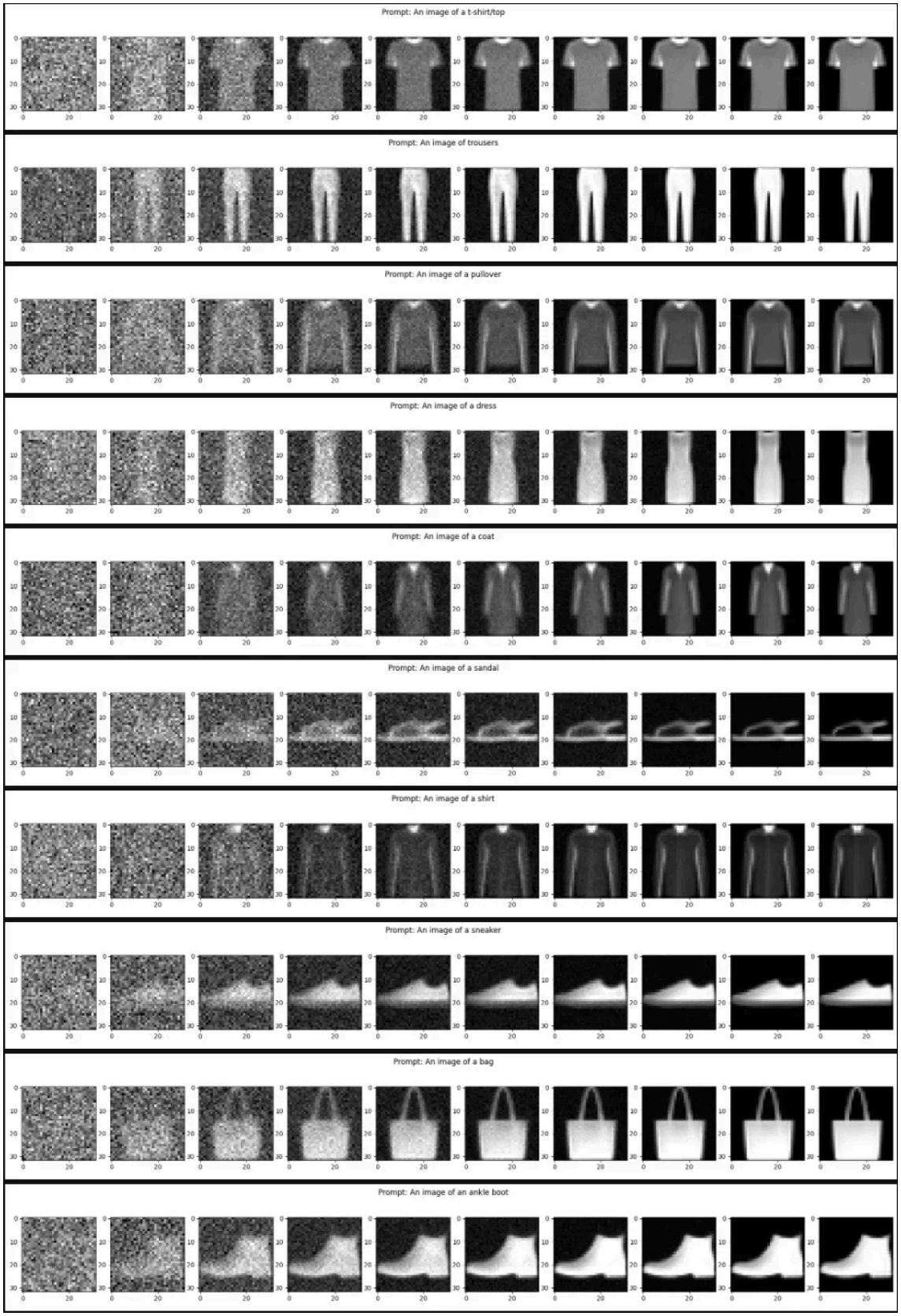

Prompt: An image of a t-shirt/top
Prompt: An image of trousers
Prompt: An image of a pullover
Prompt: An image of a dress
Prompt: An image of a coat
Prompt: An image of a sandal
Prompt: An image of a shirt
Prompt: An image of a sneaker
Prompt: An image of a bag
Prompt: An image of an ankle boot

图 19.10 训练结果

20. 总结

1. ViT 可以对解决手写数字识别问题。ViT 可以解决图像分类的问题。
   a. 将图片切成 patch（补丁），添加一个分类 token。
   b. 每个 patch 都会加上位置编码信息。
   c. NLP Transformer, Audio Transformer, Vision Transformer
2. Clip 的目标是将图片和文本对齐。
   a. 训练一个图像编码器(ViT)，训练一个文本编码器(NLP Transformer)
   b. 要求图文对输出的两个嵌入的余弦相似度很高，说明图文对很对齐。
   c. 可以解决的问题是**文搜图**。
3. 你的文搜图是怎么实现的呢?
   a. 我使用了 clip-chinese-vit-patch16 预训练模型，结合本公司标注的图文对（1 万对～10 万对）进一步微调。
   b. 将（图片 id，图片的路径等信息，图片经过模型编码后的嵌入张量）存入 chroma 之类的向量数据库。
   c. 输入文本，文本经过 clip 模型输出文本嵌入。然后使用文本嵌入在向量数据库中检索相似的向量，并返回图片文件的各类信息。
4. ClipCap：Clip + Caption，目标是给模型输入图片，能够生成图片对应的文字描述。也就是**图生文**。
   a. Clip（预训练模型 clip-chinese-vit-patch16，冻结） + 生成模型（GPT2，Qwen0.5B）。
   b. Clip 输出的图像嵌入的维度和 gpt2 要求的词嵌入维度不同，需要一个 MLP 转换形状。
   c. 将图像嵌入通过一个 mlp 转换成 10 个 token 嵌入。然后和文本的 token 嵌入列表进行拼接。然后对 gpt2 和 mlp 进行微调。
   d. 微调的方法：预测下一个 token，但**只计算文本部分的损失**。
   e. 构建数据集：图文对。一张图片对应多条文本。例如：1 张图片有 10 条文本，那么是 10 对训练数据。50 万对图文对，使用 A800，训练 20 个小时。如果换生成模型，那么需要的 GPU 资源会多很多。
   f. 标准的 clipcap 是冻结 clip 模型和冻结生成模型，只微调 mlp。但是如果生成模型不够好，那么效果堪忧。
5. 扩散模型的目标是：生成没见过的图片。
   a. 正向扩散：逐步给图片添加噪声（解析解，一步采样到$x_{\{500\}}$）。需要图片，随机选择的时间步，以及**方差计划**。
   b. 反向扩散：
      i. 需要训练一个 UNET（输入和输出的分辨率一致。先下采样，再上采样，残差连接）。

ii. 输入：$(x_{500}, t=500)$。输出是预测的噪声。训练方式是预测的噪声和真实添加的噪声进行 mse loss。

iii. 逐步去噪：(**纯噪声图片** $x_{1000}, t=1000$) 发送给 UNET，unet 输出预测的噪声，$(x_{1000} - \text{noise}) \to x_{999}$ 。接下来，$(x_{999}, t=999) \to \text{noise}_{999}$，$x_{999} - \text{noise}_{999} \to x_{998}$ 。。。。

6. 带条件的扩散模型的目标是给定提示词，生成提示词对应的没见过的图片，**文生图**。

a. 使用图文对训练 clip 模型，后续使用需要冻结。

b. 训练一个 prior model（先验模型）。先验模型要解决的问题是：能够给定提示词，先验模型可以输出预测的 clip 图像嵌入。先验模型的目的是想要生成一个高质量的条件。

i. （训练阶段）模型的输入：（图片的文本 id 列表，图片的文本 clip 嵌入，随机选择的时间步$t=500$，针对图片的图像 clip 嵌入添加 500 步噪声后的图片嵌入，需要学习的嵌入）。预测目标是**图像 clip 嵌入**。

ii. （推理阶段）模型的输入：（图片的文本 id 列表，图片的文本 clip 嵌入，时间步是 1000，和图像嵌入相同形状的白噪声，嵌入）。输出：预测的 clip 图片嵌入。

iii. 训练好先验模型以后，后续使用需要冻结。

c. 训练一个带条件的 unet，用来扩散出图像。

i. （训练阶段）模型的输入：($x_{t=500}$，$t=500$，条件=先验模型根据提示词预测的图片嵌入)。输出是预测的噪声。

ii. unet 里面加了**注意力模块**。使用注意力模块处理**提示词信息**，捕获图像和提示词的全局信息。

iii. 反向扩散：输入是(白噪声$x_{1000}, t=1000$， 先验模型根据提示词预测的图片嵌入，提示词(用来给注意力模块))。然后逐步去噪。

A. 反向传播算法

A.1 损失函数

$$\sigma(x) = \frac{1}{1+e^{-x}} \tag{1.1}$$

$$\sigma'(x) = \sigma(x)(1-\sigma(x)) \tag{1.2}$$

$$y = \sigma(wx+b) \tag{1.3}$$

$$\mathcal{L} = (y-\hat{y})^2 \tag{1.4}$$

求导

$$\frac{\partial \mathcal{L}}{\partial w} = ? \tag{1.5}$$

$$\frac{\partial \mathcal{L}}{\partial b} = ? \tag{1.6}$$

训练数据：$(1,2)$和$(2,3)$
初始化了一个神经网络参数$w=2.0, b=3.0$

A.2 前向传播

- 中间结果：$m_0 = wx = 2.0 \times 1.0 = 2.0$
- 中间结果：$m_1 = m_0 + b = 2.0 + 3.0 = 5.0$
- 中间结果：$m_2 = \sigma(m_1) = 0.9933071490757268$
- 中间结果：$m_3 = m_2 - 2.0 = 0.9933071490757268 - 2.0 = -1.0066928509242732$
- 最终结果：$\mathcal{L} = m_3^2$

A.3 反向传播

$$
\begin{aligned}
\frac{\partial \mathcal{L}}{\partial w} &= \frac{\partial \mathcal{L}}{\partial m_3} \cdot \frac{\partial m_3}{\partial m_2} \cdot \frac{\partial m_2}{\partial m_1} \cdot \frac{\partial m_1}{\partial m_0} \\
&= 2 \cdot m_3 \cdot 1 \cdot m_2 \cdot (1 - m_2) \cdot 1 \cdot x \\
&= 2 \times (-1.0066928509242732) \times 0.9933071490757268 \times (1 - 0.9933071490757268) \times 1.0 \\
&= -0.013385102246024565
\end{aligned} \tag{1.7}
$$

$$
\alpha = 50 \tag{1.8}
$$

$$
w = 2.0 - 50 * -0.013385102246024565 \tag{1.9}
$$

A.4 代码实现

```python
import torch
import torch.nn as nn

x = torch.tensor([1.0])             # 形状 [N, 1]，每行一个标量输入
# w = torch.randn(1, 1, requires_grad=True)  # 标量权重（等价于单个参数）
w = torch.tensor([2.0], requires_grad=True)
# b = torch.randn(1, requires_grad=True)     # 标量偏置
b = torch.tensor([3.0], requires_grad=True)

# 前向：y = sigmoid(w*x + b)
y = torch.sigmoid(x @ w + b)     # 结果形状 [N, 1]
# 构造标量损失以便反向传播
target = torch.tensor([2.0])
loss = nn.MSELoss()(y, target)

# 反向传播
loss.backward()

print("loss:", loss.item())
print("w:", w.item(), "b:", b.item())
print("w.grad:", w.grad.item(), "b.grad:", b.grad.item())

# 可选：一步手动更新
lr = 0.1
with torch.no_grad():
    w -= lr * w.grad
    b -= lr * b.grad
    w.grad.zero_()
    b.grad.zero_()
```

B. uv 教程

```shell
$ pip install uv
$ uv venv rlhf-env # 创建虚拟环境
$ source rlhf-env/bin/activate # 进入虚拟环境
$ uv pip install jinja2
$ uv pip install pandas
$ uv pip install pyarrow
$ uv pip install pyyaml
$ uv pip install safetensors
$ uv pip install tensorboard
$ uv pip install tokenizers
$ uv pip install torch
$ uv pip install transformers
```

配置国内源

```shell
# 推荐使用清华源
$ echo 'export UV_DEFAULT_INDEX="https://pypi.tuna.tsinghua.edu.cn/simple"' >> ~/.bashrc

# 让配置立即生效
$ source ~/.bashrc

# 检查环境变量
$ echo $UV_DEFAULT_INDEX
```

```shell
$ uv pip install modelscope
$ uv pip install addict
$ uv pip install datasets==2.21.0
$ uv pip install transformers[serving]

$ modelscope download --model Qwen/Qwen3-0.6B-Base --local_dir ./
```

C. KV Cache

```python
import time
import tiktoken
import torch
import torch.nn as nn


class MultiHeadAttention(nn.Module):
    def __init__(self, d_in, d_out, context_length, dropout, num_heads, qkv_bias=False):
        super().__init__()
        assert d_out % num_heads == 0, "d_out must be divisible by num_heads"

        self.d_out = d_out
        self.num_heads = num_heads
        self.head_dim = d_out // num_heads  # Reduce the projection dim to match desired output dim

        self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_key = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.out_proj = nn.Linear(d_out, d_out)  # Linear layer to combine head outputs
        self.dropout = nn.Dropout(dropout)
        self.register_buffer(
            "mask",
            torch.triu(torch.ones(context_length, context_length), diagonal=1),
            persistent=False
        )

        ############################################################
        # NEW
        self.register_buffer("cache_k", None, persistent=False)
        self.register_buffer("cache_v", None, persistent=False)
```

```
        self.ptr_current_pos = 0
        ############################################################
    def forward(self, x, use_cache=False):
        b, num_tokens, d_in = x.shape

        keys_new = self.W_key(x)  # Shape: (b, num_tokens, d_out)
        values_new = self.W_value(x)
        queries = self.W_query(x)

        # We implicitly split the matrix by adding a `num_heads` dimension
        # Unroll last dim: (b, num_tokens, d_out) -> (b, num_tokens, num_heads, head_dim)
        keys_new = keys_new.view(b, num_tokens, self.num_heads, self.head_dim)
        values_new = values_new.view(b, num_tokens, self.num_heads, self.head_dim)
        queries = queries.view(b, num_tokens, self.num_heads, self.head_dim)

        ############################################################
        # NEW
        if use_cache:
            if self.cache_k is None:
                self.cache_k, self.cache_v = keys_new, values_new
            else:
                self.cache_k = torch.cat([self.cache_k, keys_new], dim=1)
                self.cache_v = torch.cat([self.cache_v, values_new], dim=1)
            keys, values = self.cache_k, self.cache_v
        else:
            keys, values = keys_new, values_new
        ############################################################

        # Transpose: (b, num_tokens, num_heads, head_dim) -> (b, num_heads, num_tokens, head_dim)
        keys = keys.transpose(1, 2)
        queries = queries.transpose(1, 2)
        values = values.transpose(1, 2)

        # Compute scaled dot-product attention (aka self-attention) with a causal mask
        attn_scores = queries @ keys.transpose(2, 3)  # Dot product for each head

        ############################################################
        # NEW
        num_tokens_Q = queries.shape[-2]
        num_tokens_K = keys.shape[-2]
        if use_cache:
            mask_bool = self.mask.bool()[
                self.ptr_current_pos:self.ptr_current_pos + num_tokens_Q, :num_tokens_K
            ]
            self.ptr_current_pos += num_tokens_Q
        ############################################################
```

```
78            # Original mask truncated to the number of tokens and converted to boolean
79            else:
80                mask_bool = self.mask.bool()[:num_tokens_Q, :num_tokens_K]
81
82            # Use the mask to fill attention scores
83            attn_scores.masked_fill_(mask_bool, -torch.inf)
84
85            attn_weights = torch.softmax(attn_scores / keys.shape[-1]**0.5, dim=-1)
86            attn_weights = self.dropout(attn_weights)
87
88            # Shape: (b, num_tokens, num_heads, head_dim)
89            context_vec = (attn_weights @ values).transpose(1, 2)
90
91            # Combine heads, where self.d_out = self.num_heads * self.head_dim
92            context_vec = context_vec.contiguous().view(b, num_tokens, self.d_out)
93            context_vec = self.out_proj(context_vec)  # optional projection
94
95            return context_vec
96
97        ##################################################
98        # NEW
99        def reset_cache(self):
100           self.cache_k, self.cache_v = None, None
101           self.ptr_current_pos = 0
102       ##################################################
103
104
105   class LayerNorm(nn.Module):
106       def __init__(self, emb_dim):
107           super().__init__()
108           self.eps = 1e-5
109           self.scale = nn.Parameter(torch.ones(emb_dim))
110           self.shift = nn.Parameter(torch.zeros(emb_dim))
111
112       def forward(self, x):
113           mean = x.mean(dim=-1, keepdim=True)
114           var = x.var(dim=-1, keepdim=True, unbiased=False)
115           norm_x = (x - mean) / torch.sqrt(var + self.eps)
116           return self.scale * norm_x + self.shift
117
118
119   class GELU(nn.Module):
120       def __init__(self):
121           super().__init__()
122
123       def forward(self, x):
124           return 0.5 * x * (1 + torch.tanh(
125               torch.sqrt(torch.tensor(2.0 / torch.pi)) *
126               (x + 0.044715 * torch.pow(x, 3))
127           ))
```

```
128
129
130 class FeedForward(nn.Module):
131     def __init__(self, cfg):
132         super().__init__()
133         self.layers = nn.Sequential(
134             nn.Linear(cfg["emb_dim"], 4 * cfg["emb_dim"]),
135             GELU(),
136             nn.Linear(4 * cfg["emb_dim"], cfg["emb_dim"]),
137         )
138
139     def forward(self, x):
140         return self.layers(x)
141
142
143 class TransformerBlock(nn.Module):
144     def __init__(self, cfg):
145         super().__init__()
146         self.att = MultiHeadAttention(
147             d_in=cfg["emb_dim"],
148             d_out=cfg["emb_dim"],
149             context_length=cfg["context_length"],
150             num_heads=cfg["n_heads"],
151             dropout=cfg["drop_rate"],
152             qkv_bias=cfg["qkv_bias"])
153         self.ff = FeedForward(cfg)
154         self.norm1 = LayerNorm(cfg["emb_dim"])
155         self.norm2 = LayerNorm(cfg["emb_dim"])
156         self.drop_shortcut = nn.Dropout(cfg["drop_rate"])
157
158     def forward(self, x, use_cache=False):
159         # Shortcut connection for attention block
160         shortcut = x
161         x = self.norm1(x)
162
163         # x = self.att(x)   # Shape [batch_size, num_tokens, emb_size]
164         ##################################################
165         # NEW
166         x = self.att(x, use_cache=use_cache)
167         ##################################################
168
169         x = self.drop_shortcut(x)
170         x = x + shortcut  # Add the original input back
171
172         # Shortcut connection for feed-forward block
173         shortcut = x
174         x = self.norm2(x)
175         x = self.ff(x)
176         x = self.drop_shortcut(x)
177         x = x + shortcut  # Add the original input back
```

```
178
179         return x
180
181
182 class GPTModel(nn.Module):
183     def __init__(self, cfg):
184         super().__init__()
185         self.tok_emb = nn.Embedding(cfg["vocab_size"], cfg["emb_dim"])
186         self.pos_emb = nn.Embedding(cfg["context_length"], cfg["emb_dim"])
187         self.drop_emb = nn.Dropout(cfg["drop_rate"])
188
189         # self.trf_blocks = nn.Sequential(
190         #     *[TransformerBlock(cfg) for _ in range(cfg["n_layers"])])
191         ########################################################
192         # NEW
193         self.trf_blocks = nn.ModuleList(
194             [TransformerBlock(cfg) for _ in range(cfg["n_layers"])])
195
196         self.current_pos = 0
197         ########################################################
198
199         self.final_norm = LayerNorm(cfg["emb_dim"])
200         self.out_head = nn.Linear(cfg["emb_dim"], cfg["vocab_size"], bias=False)
201
202     def forward(self, in_idx, use_cache=False):
203         batch_size, seq_len = in_idx.shape
204         tok_embeds = self.tok_emb(in_idx)
205
206         # pos_embeds = self.pos_emb(torch.arange(seq_len, device=in_idx.device))
207
208         ########################################################
209         # NEW
210
211         if use_cache:
212             pos_ids = torch.arange(self.current_pos, self.current_pos + seq_len,
                device=in_idx.device, dtype=torch.long)
213             self.current_pos += seq_len
214         else:
215             pos_ids = torch.arange(0, seq_len, device=in_idx.device,
                dtype=torch.long)
216         pos_embeds = self.pos_emb(pos_ids).unsqueeze(0)
217         ########################################################
218
219         x = tok_embeds + pos_embeds  # Shape [batch_size, num_tokens, emb_size]
220         x = self.drop_emb(x)
221
222         # x = self.trf_blocks(x)
223         ########################################################
224         # NEW
225         for blk in self.trf_blocks:
226             x = blk(x, use_cache=use_cache)
```

```
        ##################################################

        x = self.final_norm(x)
        logits = self.out_head(x)
        return logits

    ##################################################
    # NEW
    def reset_kv_cache(self):
        for blk in self.trf_blocks:
            blk.att.reset_cache()
        self.current_pos = 0
    ##################################################


def generate_text_simple(model, idx, max_new_tokens, context_size):
    # idx is (B, T) array of indices in the current context
    for _ in range(max_new_tokens):

        # Crop current context if it exceeds the supported context size
        # E.g., if LLM supports only 5 tokens, and the context size is 10
        # then only the last 5 tokens are used as context
        idx_cond = idx[:, -context_size:]

        # Get the predictions
        with torch.no_grad():
            logits = model(idx_cond)

        # Focus only on the last time step
        # (batch, n_token, vocab_size) becomes (batch, vocab_size)
        logits = logits[:, -1, :]

        # Get the idx of the vocab entry with the highest logits value
        idx_next = torch.argmax(logits, dim=-1, keepdim=True)  # (batch, 1)

        # Append sampled index to the running sequence
        idx = torch.cat((idx, idx_next), dim=1)  # (batch, n_tokens+1)

    return idx


##################################################
# NEW
def generate_text_simple_cached(model, idx, max_new_tokens,
                                context_size=None, use_cache=True):
    model.eval()
    ctx_len = context_size or model.pos_emb.num_embeddings

    with torch.no_grad():
        if use_cache:
```

```
277             # Init cache with full prompt
278             model.reset_kv_cache()
279             logits = model(idx[:, -ctx_len:], use_cache=True)
280
281             for _ in range(max_new_tokens):
282                 # a) pick the token with the highest log-probability (greedy sampling)
283                 next_idx = logits[:, -1].argmax(dim=-1, keepdim=True)
284                 # b) append it to the running sequence
285                 idx = torch.cat([idx, next_idx], dim=1)
286                 # c) feed model only the new token
287                 logits = model(next_idx, use_cache=True)
288         else:
289             for _ in range(max_new_tokens):
290                 logits = model(idx[:, -ctx_len:], use_cache=False)
291                 next_idx = logits[:, -1].argmax(dim=-1, keepdim=True)
292                 idx = torch.cat([idx, next_idx], dim=1)
293
294     return idx
295 #####################################################
296
297
298 def main():
299     GPT_CONFIG_124M = {
300         "vocab_size": 50257,     # Vocabulary size
301         "context_length": 1024,  # Context length
302         "emb_dim": 768,          # Embedding dimension
303         "n_heads": 12,           # Number of attention heads
304         "n_layers": 12,          # Number of layers
305         "drop_rate": 0.1,        # Dropout rate
306         "qkv_bias": False        # Query-Key-Value bias
307     }
308
309     torch.manual_seed(123)
310     model = GPTModel(GPT_CONFIG_124M)
311     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
312     model.to(device)
313     model.eval()  # disable dropout
314
315     start_context = "Hello, I am"
316
317     tokenizer = tiktoken.get_encoding("gpt2")
318     encoded = tokenizer.encode(start_context)
319     encoded_tensor = torch.tensor(encoded, device=device).unsqueeze(0)
320
321     print(f"\n{50*'='}\n{22*' '}IN\n{50*'='}")
322     print("\nInput text:", start_context)
323     print("Encoded input text:", encoded)
324     print("encoded_tensor.shape:", encoded_tensor.shape)
325
326     if torch.cuda.is_available():
```

```
        torch.cuda.synchronize()
    start = time.time()

    # token_ids = generate_text_simple(
    #     model=model,
    #     idx=encoded_tensor,
    #     max_new_tokens=200,
    #     context_size=GPT_CONFIG_124M["context_length"]
    # )

    ##################################################
    # NEW
    token_ids = generate_text_simple_cached(
        model=model,
        idx=encoded_tensor,
        max_new_tokens=200,
    )
    ##################################################

    if torch.cuda.is_available():
        torch.cuda.synchronize()
    total_time = time.time() - start

    decoded_text = tokenizer.decode(token_ids.squeeze(0).tolist())

    print(f"\n\n{50*'='}\n{22*' '}OUT\n{50*'='}")
    print("\nOutput:", token_ids)
    print("Output length:", len(token_ids[0]))
    print("Output text:", decoded_text)

    print(f"\nTime: {total_time:.2f} sec")
    print(f"{int(len(token_ids[0])/total_time)} tokens/sec")
    if torch.cuda.is_available():
        max_mem_bytes = torch.cuda.max_memory_allocated()
        max_mem_gb = max_mem_bytes / (1024 ** 3)
        print(f"Max memory allocated: {max_mem_gb:.2f} GB")


if __name__ == "__main__":
    main()
```